CÓDIGO FUENTE - PROYECTO ERP

Autora: Sandy Iturra Mena

Archivo: index.html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description"
        content="ERP Educativo para estudiantes de Contabilidad y Finanzas - Aprende la
lógica de sistemas empresariales">
    <title>EDU-TRACE ERP - Sistema de Gestión Empresarial (ERP) Educativo con Trazabilidad
Pedagógica Integrada</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <script src="https://cdn.sheetjs.com/xlsx-0.20.0/package/dist/xlsx.full.min.js">
</script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js">
</script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf-
autotable/3.5.31/jspdf.plugin.autotable.min.js"></script>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/FileSaver.js/2.0.5/FileSaver.min.js"></script>
    <link rel="stylesheet" href="css/main.css">
    <link rel="stylesheet" href="css/components.css">
    <link rel="stylesheet" href="css/dashboard.css">
    <link rel="stylesheet" href="css/trazabilidad.css">
    <link rel="stylesheet" href="css/system-flow-fix.css">
</head>
<body>
    <div id="loading-screen" class="loading-screen">
        <div class="loading-content">
            <div class="loading-logo">
                <i class="fas fa-graduation-cap"></i>
            </div>
            <h1>EDU-TRACE ERP</h1>
            <p>Cargando sistema...</p>
            <div class="loading-bar">
                <div class="loading-progress"></div>
            </div>
        </div>
    </div>
    <div id="app" class="app-container hidden">
        <aside id="sidebar" class="sidebar">
            <div class="sidebar-header">
                <div class="logo">
                    <i class="fas fa-graduation-cap"></i>
                    <span>EDU-TRACE ERP</span>
                </div>
                <button id="sidebar-toggle" class="sidebar-toggle">
                    <i class="fas fa-bars"></i>
                </button>
            </div>
            <div class="company-selector" id="company-selector">
                <div class="current-company" id="current-company">
                    <i class="fas fa-briefcase"></i>
                    <span id="company-name">Sin empresa</span>
                    <i class="fas fa-chevron-down"></i>
                </div>
            </div>
            <nav class="sidebar-nav">
                <ul class="nav-list">
                    <li class="nav-item" data-module="dashboard">
                        <a href="#" class="nav-link active">
                            <i class="fas fa-th-large"></i>
                            <span>Dashboard</span>
```

```
                        </a>
                    </li>
                    <li class="nav-item has-submenu" data-module="contabilidad">
                        <a href="#" class="nav-link">
                            <i class="fas fa-book"></i>
                            <span>Contabilidad (GL)</span>
                            <i class="fas fa-chevron-right submenu-arrow"></i>
                        </a>
                        <ul class="submenu">
                            <li class="submenu-item" data-view="plan-cuentas"><a
href="#">Plan de Cuentas</a></li>
                            <li class="submenu-item" data-view="asientos"><a
href="#">Asientos Contables</a></li>
                            <li class="submenu-item" data-view="libro-mayor"><a
href="#">Libro Mayor</a></li>
                            <li class="submenu-item" data-view="libro-diario"><a
href="#">Libro Diario</a></li>
                            <li class="submenu-item" data-view="libros-auxiliares"><a
href="#">Libros Auxiliares</a>
                            </li>
                        </ul>
                    </li>
                    <li class="nav-item has-submenu" data-module="compras">
                        <a href="#" class="nav-link">
                            <i class="fas fa-shopping-cart"></i>
                            <span>Compras (MM)</span>
                            <i class="fas fa-chevron-right submenu-arrow"></i>
                        </a>
                        <ul class="submenu">
                            <li class="submenu-item" data-view="proveedores"><a
href="#">Proveedores</a></li>
                            <li class="submenu-item" data-view="ordenes-compra"><a
href="#">Órdenes de Compra</a></li>
                            <li class="submenu-item" data-view="facturas-proveedor"><a
href="#">Facturas Proveedor</a>
                            </li>
                        </ul>
                    </li>
                    <li class="nav-item has-submenu" data-module="ventas">
                        <a href="#" class="nav-link">
                            <i class="fas fa-cash-register"></i>
                            <span>Ventas (SD)</span>
                            <i class="fas fa-chevron-right submenu-arrow"></i>
                        </a>
                        <ul class="submenu">
                            <li class="submenu-item" data-view="clientes"><a
href="#">Clientes</a></li>
                            <li class="submenu-item" data-view="pedidos-venta"><a
href="#">Pedidos de Venta</a></li>
                            <li class="submenu-item" data-view="facturas-cliente"><a
href="#">Facturas Cliente</a></li>
                        </ul>
                    </li>
                    <li class="nav-item has-submenu" data-module="inventario">
                        <a href="#" class="nav-link">
                            <i class="fas fa-boxes"></i>
                            <span>Inventario (WM)</span>
                            <i class="fas fa-chevron-right submenu-arrow"></i>
                        </a>
                        <ul class="submenu">
                            <li class="submenu-item" data-view="productos"><a
href="#">Productos</a></li>
                            <li class="submenu-item" data-view="movimientos-stock"><a
href="#">Movimientos de Stock</a>
                            </li>
                            <li class="submenu-item" data-view="valoracion"><a
href="#">Valorización</a></li>
                        </ul>
                    </li>
                    <li class="nav-item has-submenu" data-module="tesoreria">
                        <a href="#" class="nav-link">
                            <i class="fas fa-university"></i>
                            <span>Tesorería (FI)</span>
                            <i class="fas fa-chevron-right submenu-arrow"></i>
                        </a>
                        <ul class="submenu">
                            <li class="submenu-item" data-view="cuentas-pagar"><a
href="#">Control de Pagos</a></li>
                            <li class="submenu-item" data-view="cuentas-cobrar"><a
href="#">Control de Cobros</a></li>
                            <li class="submenu-item" data-view="bancos"><a
```

```
href="#">Bancos</a></li>
                            <li class="submenu-item" data-view="conciliacion"><a
href="#">Conciliación Bancaria</a></li>
                        </ul>
                    </li>
                    <li class="nav-item has-submenu disabled-module" data-module="costos"
data-disabled="true">
                        <a href="#" class="nav-link disabled">
                            <i class="fas fa-calculator"></i>
                            <span>Costos (CO)</span>
                            <span class="module-badge etapa2">Etapa 2</span>
                        </a>
                    </li>
                    <li class="nav-item has-submenu disabled-module" data-module="rrhh"
data-disabled="true">
                        <a href="#" class="nav-link disabled">
                            <i class="fas fa-users"></i>
                            <span>RRHH (HR)</span>
                            <span class="module-badge etapa2">Etapa 2</span>
                        </a>
                    </li>
                    <li class="nav-item has-submenu disabled-module" data-
module="trazabilidad" data-disabled="true">
                        <a href="#" class="nav-link disabled">
                            <i class="fas fa-route"></i>
                            <span>Trazabilidad</span>
                            <span class="module-badge etapa2">Etapa 2</span>
                        </a>
                    </li>
                    <li class="nav-item has-submenu" data-module="reportes">
                        <a href="#" class="nav-link">
                            <i class="fas fa-chart-bar"></i>
                            <span>Reportes</span>
                            <i class="fas fa-chevron-right submenu-arrow"></i>
                        </a>
                        <ul class="submenu">
                            <li class="submenu-item" data-view="balance-general"><a
href="#">Balance General</a></li>
                            <li class="submenu-item" data-view="balance-comprobacion"><a
href="#">Balance de
                                    Comprobación</a></li>
                            <li class="submenu-item" data-view="estado-resultados"><a
href="#">Estado de Resultados</a>
                            </li>
                            <li class="submenu-item" data-view="ratios-financieros"><a
href="#">Ratios Financieros</a>
                            </li>
                            <li class="submenu-item" data-view="flujo-caja"><a
href="#">Flujo de Caja</a></li>
                            <li class="submenu-item" data-view="analisis-cuentas"><a
href="#">Análisis de Cuentas</a>
                            </li>
                        </ul>
                    </li>
                    <li class="nav-item has-submenu" data-module="admin">
                        <a href="#" class="nav-link">
                            <i class="fas fa-cog"></i>
                            <span>Administración</span>
                            <i class="fas fa-chevron-right submenu-arrow"></i>
                        </a>
                        <ul class="submenu">
                            <li class="submenu-item" data-view="empresas"><a
href="#">Gestión de Empresas</a></li>
                            <li class="submenu-item" data-view="import-export"><a
href="#">Importar/Exportar</a></li>
                            <li class="submenu-item" data-view="configuracion"><a
href="#">Configuración</a></li>
                        </ul>
                    </li>
                </ul>
            </nav>
            <div class="sidebar-footer">
                <div class="user-info">
                    <div class="user-avatar">
                        <i class="fas fa-user-graduate"></i>
                    </div>
                    <div class="user-details">
                        <span class="user-name" id="user-display-name">Estudiante</span>
                        <span class="user-role" id="user-role">Operador</span>
                    </div>
                </div>
```

```
            </div>
        </aside>
        <main class="main-content">
            <header class="top-header">
                <div class="header-left">
                    <nav class="breadcrumbs" id="breadcrumbs">
                        <a href="#dashboard" class="breadcrumb-item">Inicio</a>
                    </nav>
                </div>
                <div class="header-right">
                    <div class="header-actions">
                        <button class="header-btn" id="btn-quick-entry" title="Entrada
Rápida">
                            <i class="fas fa-bolt"></i>
                        </button>
                        <button class="header-btn" id="notifications-btn"
title="Notificaciones">
                            <i class="fas fa-bell"></i>
                            <span class="notification-badge">3</span>
                        </button>
                        <button class="header-btn" id="help-btn" title="Ayuda">
                            <i class="fas fa-question-circle"></i>
                        </button>
                        <button class="header-btn" id="theme-toggle" title="Cambiar Tema">
                            <i class="fas fa-moon"></i>
                        </button>
                        <div class="dropdown">
                            <button class="header-btn" id="export-dropdown-toggle"
title="Exportar">
                                <i class="fas fa-download"></i>
                            </button>
                            <div class="dropdown-menu" id="export-dropdown">
                                <a href="#" data-export="excel"><i class="fas fa-file-
excel"></i> Excel</a>
                                <a href="#" data-export="csv"><i class="fas fa-file-csv">
</i> CSV</a>
                                <a href="#" data-export="pdf"><i class="fas fa-file-pdf">
</i> PDF</a>
                                <a href="#" data-export="json"><i class="fas fa-file-
code"></i> JSON (Backup)</a>
                            </div>
                        </div>
                    </div>
                </div>
            </header>
            <div class="content-wrapper" id="content-wrapper">
            </div>
        </main>
    </div>
    <div id="modal-container" class="modal-container hidden">
        <div class="modal-overlay"></div>
        <div class="modal" id="modal">
            <div class="modal-header">
                <h2 class="modal-title" id="modal-title">Modal Title</h2>
                <button class="modal-close" id="modal-close">
                    <i class="fas fa-times"></i>
                </button>
            </div>
            <div class="modal-body" id="modal-body">
            </div>
            <div class="modal-footer" id="modal-footer">
            </div>
        </div>
    </div>
    <div id="toast-container" class="toast-container"></div>
    <script src="js/utils/helpers.js"></script>
    <script src="js/utils/formatters.js"></script>
    <script src="js/database/db.js"></script>
    <script src="js/database/migrations.js"></script>
    <script src="js/services/company-service.js"></script>
    <script src="js/services/accounting-service.js"></script>
    <script src="js/services/export-service.js"></script>
    <script src="js/services/validation-service.js"></script>
    <script src="js/services/traceability-service.js"></script>
    <script src="js/components/modal.js"></script>
    <script src="js/components/toast.js"></script>
    <script src="js/components/table.js"></script>
    <script src="js/components/charts.js"></script>
    <script src="js/components/tutorial.js"></script>
    <script src="js/components/help.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/jspdf/2.5.1/jspdf.umd.min.js">
```

```
</script>
    <script src="js/components/system-flow.js"></script>
    <script src="js/modules/dashboard.js"></script>
    <script src="js/modules/contabilidad.js"></script>
    <script src="js/modules/compras.js"></script>
    <script src="js/modules/ventas.js"></script>
    <script src="js/modules/inventario.js"></script>
    <script src="js/modules/tesoreria.js"></script>
    <script src="js/modules/costos.js"></script>
    <script src="js/modules/rrhh.js"></script>
    <script src="js/modules/reportes.js"></script>
    <script src="js/modules/admin.js"></script>
    <script src="js/modules/trazabilidad.js"></script>
    <script src="js/app.js"></script>
</body>
</html>
```

Archivo: diagrama-contable.html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sistema Contable - Diagrama de Flujo Interactivo</title>
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.4.0/css/all.min.css">
    <link href="https://fonts.googleapis.com/css2?
family=Inter:wght@300;400;500;600;700&display=swap" rel="stylesheet">
    <style>
        :root {
            --primary: #f59e0b;
            --primary-light: #fbbf24;
            --primary-dark: #d97706;
            --secondary: #10b981;
            --accent: #3b82f6;
            --purple: #8b5cf6;
            --dark: #1f2937;
            --gray-100: #f3f4f6;
            --gray-200: #e5e7eb;
            --gray-300: #d1d5db;
            --gray-500: #6b7280;
            --gray-700: #374151;
            --white: #ffffff;
            --shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1), 0 2px 4px -1px rgba(0, 0, 0,
0.06);
            --shadow-lg: 0 10px 15px -3px rgba(0, 0, 0, 0.1), 0 4px 6px -2px rgba(0, 0, 0,
0.05);
        }
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
        body {
            font-family: 'Inter', sans-serif;
            background: linear-gradient(135deg, #f8fafc 0%, #e2e8f0 100%);
            min-height: 100vh;
            color: var(--dark);
        }
        .container {
            max-width: 1400px;
            margin: 0 auto;
            padding: 2rem;
        }
        /* Header */
        .header {
            text-align: center;
            margin-bottom: 2rem;
        }
        .header h1 {
            font-size: 2.5rem;
            font-weight: 700;
            background: linear-gradient(135deg, var(--primary) 0%, var(--primary-dark)
100%);
```

```
            -webkit-background-clip: text;
            -webkit-text-fill-color: transparent;
            background-clip: text;
            margin-bottom: 0.5rem;
        }
        .header p {
            font-size: 1rem;
            color: var(--gray-500);
            max-width: 800px;
            margin: 0 auto;
            line-height: 1.6;
        }
        .back-btn {
            position: fixed;
            top: 1rem;
            left: 1rem;
            background: var(--white);
            border: none;
            padding: 0.75rem 1.25rem;
            border-radius: 0.5rem;
            font-size: 0.9rem;
            font-weight: 600;
            cursor: pointer;
            box-shadow: var(--shadow);
            display: flex;
            align-items: center;
            gap: 0.5rem;
            color: var(--dark);
            text-decoration: none;
            transition: all 0.2s;
        }
        .back-btn:hover {
            transform: translateY(-2px);
            box-shadow: var(--shadow-lg);
        }
        /* Tabs */
        .tabs {
            display: flex;
            justify-content: center;
            gap: 0.5rem;
            margin-bottom: 2rem;
            flex-wrap: wrap;
        }
        .tab-btn {
            background: var(--white);
            border: 2px solid var(--gray-200);
            padding: 0.75rem 1.5rem;
            border-radius: 2rem;
            font-size: 0.9rem;
            font-weight: 600;
            cursor: pointer;
            transition: all 0.2s;
            display: flex;
            align-items: center;
            gap: 0.5rem;
        }
        .tab-btn:hover {
            border-color: var(--primary);
            color: var(--primary);
        }
        .tab-btn.active {
            background: var(--primary);
            color: var(--white);
            border-color: var(--primary);
        }
        /* Tab Content */
        .tab-content {
            display: none;
        }
        .tab-content.active {
            display: block;
        }
        /* Flow Diagram */
        .flow-diagram {
            display: flex;
            align-items: center;
            justify-content: center;
            gap: 2rem;
            margin: 2rem 0;
            flex-wrap: wrap;
        }
```

```
.flow-arrow {
    font-size: 2rem;
    color: var(--gray-300);
}
/* Stage Cards */
.stage {
    position: relative;
}
.stage-main {
    background: var(--white);
    border-radius: 1rem;
    padding: 1.5rem;
    text-align: center;
    box-shadow: var(--shadow-lg);
    cursor: pointer;
    transition: all 0.3s ease;
    min-width: 180px;
}
.stage-main:hover {
    transform: translateY(-5px);
}
.stage-main.entrada {
    border-top: 4px solid var(--secondary);
}
.stage-main.proceso {
    background: var(--primary);
    color: var(--white);
    border-radius: 50%;
    width: 200px;
    height: 200px;
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
}
.stage-main.salida {
    border-top: 4px solid var(--accent);
}
.stage-icon {
    font-size: 2rem;
    margin-bottom: 0.75rem;
}
.stage-main.entrada .stage-icon {
    color: var(--secondary);
}
.stage-main.proceso .stage-icon {
    color: var(--white);
}
.stage-main.salida .stage-icon {
    color: var(--accent);
}
.stage-title {
    font-size: 1.25rem;
    font-weight: 600;
    margin-bottom: 0.25rem;
}
.stage-subtitle {
    font-size: 0.85rem;
    opacity: 0.8;
}
/* Sub Items */
.sub-items {
    position: absolute;
    display: flex;
    flex-direction: column;
    gap: 0.5rem;
}
.entrada .sub-items {
    left: -180px;
    top: 50%;
    transform: translateY(-50%);
}
.salida .sub-items {
    right: -200px;
    top: 50%;
    transform: translateY(-50%);
}
.sub-item {
    background: var(--white);
    border-radius: 2rem;
    padding: 0.6rem 1rem;
```

```
            font-size: 0.8rem;
            font-weight: 500;
            box-shadow: var(--shadow);
            display: flex;
            align-items: center;
            gap: 0.5rem;
            cursor: pointer;
            transition: all 0.3s ease;
            white-space: nowrap;
        }
        .sub-item:hover {
            background: var(--primary);
            color: var(--white);
            transform: scale(1.05);
        }
        .sub-item .plus {
            width: 20px;
            height: 20px;
            background: var(--primary);
            color: var(--white);
            border-radius: 50%;
            display: flex;
            align-items: center;
            justify-content: center;
            font-size: 0.75rem;
            font-weight: 700;
        }
        .sub-item:hover .plus {
            background: var(--white);
            color: var(--primary);
        }
        /* ERP Modules Section */
        .erp-modules {
            background: var(--white);
            border-radius: 1rem;
            padding: 2rem;
            margin-top: 2rem;
            box-shadow: var(--shadow);
        }
        .erp-modules h3 {
            font-size: 1.25rem;
            font-weight: 600;
            margin-bottom: 1.5rem;
            display: flex;
            align-items: center;
            gap: 0.5rem;
            color: var(--purple);
        }
        .modules-grid {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
            gap: 1rem;
        }
        .module-card {
            background: linear-gradient(135deg, var(--gray-100) 0%, var(--white) 100%);
            border: 2px solid var(--gray-200);
            border-radius: 0.75rem;
            padding: 1.25rem;
            text-align: center;
            cursor: pointer;
            transition: all 0.3s;
        }
        .module-card:hover {
            border-color: var(--primary);
            transform: translateY(-3px);
        }
        .module-card i {
            font-size: 1.75rem;
            margin-bottom: 0.5rem;
        }
        .module-card.compras i {
            color: #ef4444;
        }
        .module-card.ventas i {
            color: #22c55e;
        }
        .module-card.inventario i {
            color: #f97316;
        }
        .module-card.tesoreria i {
            color: #3b82f6;
```

```
        }
        .module-card.contabilidad i {
            color: #8b5cf6;
        }
        .module-card.reportes i {
            color: #06b6d4;
        }
        .module-card h4 {
            font-size: 0.95rem;
            font-weight: 600;
            margin-bottom: 0.25rem;
        }
        .module-card p {
            font-size: 0.75rem;
            color: var(--gray-500);
        }
        .module-card .badge {
            display: inline-block;
            background: var(--primary);
            color: var(--white);
            font-size: 0.65rem;
            padding: 0.2rem 0.5rem;
            border-radius: 1rem;
            margin-top: 0.5rem;
        }
        /* Description Section */
        .description-section {
            background: linear-gradient(135deg, var(--primary) 0%, var(--primary-dark)
100%);
            border-radius: 1rem;
            padding: 1.5rem;
            margin-top: 2rem;
            color: var(--white);
        }
        .description-header {
            text-align: center;
            margin-bottom: 1rem;
        }
        .description-header h2 {
            font-size: 1.25rem;
            font-weight: 600;
        }
        .description-content {
            background: var(--white);
            border-radius: 0.75rem;
            padding: 1.5rem;
            color: var(--dark);
        }
        .description-content p {
            line-height: 1.7;
            margin-bottom: 0.75rem;
            font-size: 0.95rem;
        }
        /* Modal Styles */
        .modal-overlay {
            position: fixed;
            top: 0;
            left: 0;
            right: 0;
            bottom: 0;
            background: rgba(0, 0, 0, 0.6);
            display: none;
            align-items: center;
            justify-content: center;
            z-index: 1000;
            padding: 1rem;
        }
        .modal-overlay.active {
            display: flex;
        }
        .modal {
            background: var(--white);
            border-radius: 1rem;
            max-width: 900px;
            width: 100%;
            max-height: 90vh;
            overflow-y: auto;
            box-shadow: 0 25px 50px -12px rgba(0, 0, 0, 0.25);
            animation: modalIn 0.3s ease;
        }
        @keyframes modalIn {
```

```
            from {
                opacity: 0;
                transform: scale(0.95) translateY(-20px);
            }
            to {
                opacity: 1;
                transform: scale(1) translateY(0);
            }
        }
        .modal-header {
            padding: 1.25rem 1.5rem;
            border-bottom: 1px solid var(--gray-200);
            display: flex;
            align-items: center;
            justify-content: space-between;
            position: sticky;
            top: 0;
            background: var(--white);
            z-index: 10;
        }
        .modal-header h3 {
            font-size: 1.25rem;
            font-weight: 600;
            display: flex;
            align-items: center;
            gap: 0.5rem;
        }
        .modal-header h3 i {
            color: var(--primary);
        }
        .modal-close {
            background: none;
            border: none;
            font-size: 1.25rem;
            color: var(--gray-500);
            cursor: pointer;
            padding: 0.5rem;
            border-radius: 0.5rem;
            transition: all 0.2s;
        }
        .modal-close:hover {
            background: var(--gray-100);
            color: var(--dark);
        }
        .modal-body {
            padding: 1.5rem;
        }
        /* Content Sections in Modal */
        .content-section {
            margin-bottom: 1.5rem;
        }
        .content-section:last-child {
            margin-bottom: 0;
        }
        .content-section h4 {
            font-size: 1rem;
            font-weight: 600;
            color: var(--primary-dark);
            margin-bottom: 0.75rem;
            padding-bottom: 0.5rem;
            border-bottom: 2px solid var(--primary-light);
        }
        .content-section p {
            line-height: 1.7;
            color: var(--gray-700);
            margin-bottom: 0.75rem;
            font-size: 0.9rem;
        }
        .content-section ul {
            list-style: none;
            padding-left: 0;
        }
        .content-section li {
            padding: 0.6rem 0.75rem;
            background: var(--gray-100);
            border-radius: 0.5rem;
            margin-bottom: 0.4rem;
            display: flex;
            align-items: flex-start;
            gap: 0.5rem;
            font-size: 0.9rem;
```

```
}
.content-section li i {
    color: var(--primary);
    margin-top: 0.15rem;
    font-size: 0.85rem;
}
/* Account Box */
.account-box {
    background: linear-gradient(135deg, #fef3c7 0%, #fde68a 100%);
    border-radius: 0.75rem;
    padding: 1.25rem;
    margin: 0.75rem 0;
}
.account-box h5 {
    font-weight: 600;
    margin-bottom: 0.5rem;
    color: var(--primary-dark);
    font-size: 0.95rem;
}
.account-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
    gap: 0.75rem;
}
.account-item {
    background: var(--white);
    padding: 0.75rem;
    border-radius: 0.5rem;
    text-align: center;
}
.account-item .label {
    font-size: 0.7rem;
    color: var(--gray-500);
    margin-bottom: 0.15rem;
}
.account-item .value {
    font-weight: 600;
    color: var(--dark);
    font-size: 0.85rem;
}
/* T Account */
.t-account {
    background: var(--white);
    border: 2px solid var(--gray-300);
    border-radius: 0.5rem;
    margin: 0.75rem 0;
    overflow: hidden;
}
.t-account-header {
    background: var(--primary);
    color: var(--white);
    padding: 0.6rem;
    text-align: center;
    font-weight: 600;
    font-size: 0.9rem;
}
.t-account-body {
    display: grid;
    grid-template-columns: 1fr 1fr;
}
.t-account-side {
    padding: 0.75rem;
    text-align: center;
}
.t-account-side:first-child {
    border-right: 2px solid var(--gray-300);
}
.t-account-side h6 {
    font-size: 0.7rem;
    color: var(--gray-500);
    margin-bottom: 0.35rem;
}
.t-account-side .amount {
    font-size: 1.1rem;
    font-weight: 700;
}
.t-account-side.debit .amount {
    color: #dc2626;
}
.t-account-side.credit .amount {
    color: #16a34a;
```

```
        }
        /* Flow Example */
        .flow-example {
            background: var(--gray-100);
            border-radius: 0.75rem;
            padding: 1.25rem;
            margin: 0.75rem 0;
        }
        .flow-example h5 {
            font-weight: 600;
            margin-bottom: 0.75rem;
            display: flex;
            align-items: center;
            gap: 0.5rem;
            font-size: 0.95rem;
        }
        .flow-example h5 i {
            color: var(--primary);
        }
        .flow-steps {
            display: flex;
            flex-direction: column;
            gap: 0.5rem;
        }
        .flow-step {
            display: flex;
            align-items: flex-start;
            gap: 0.75rem;
        }
        .step-number {
            width: 28px;
            height: 28px;
            background: var(--primary);
            color: var(--white);
            border-radius: 50%;
            display: flex;
            align-items: center;
            justify-content: center;
            font-weight: 600;
            flex-shrink: 0;
            font-size: 0.85rem;
        }
        .step-content {
            flex: 1;
            font-size: 0.9rem;
        }
        .step-content strong {
            color: var(--dark);
        }
        /* Link Button */
        .link-btn {
            display: inline-flex;
            align-items: center;
            gap: 0.5rem;
            background: var(--accent);
            color: var(--white);
            padding: 0.6rem 1rem;
            border-radius: 0.5rem;
            font-size: 0.85rem;
            font-weight: 600;
            text-decoration: none;
            cursor: pointer;
            border: none;
            transition: all 0.2s;
            margin-top: 0.75rem;
        }
        .link-btn:hover {
            background: #2563eb;
            transform: translateY(-2px);
        }
        .link-btn.success {
            background: var(--secondary);
        }
        .link-btn.success:hover {
            background: #059669;
        }
        /* Responsive */
        @media (max-width: 1200px) {
            .sub-items {
                position: relative;
                left: auto !important;
```

```
            right: auto !important;
            top: auto !important;
            transform: none !important;
            flex-direction: row;
            flex-wrap: wrap;
            justify-content: center;
            margin-top: 1rem;
        }
    }
    @media (max-width: 768px) {
        .flow-diagram {
            flex-direction: column;
        }
        .flow-arrow {
            transform: rotate(90deg);
        }
        .header h1 {
            font-size: 1.75rem;
        }
        .back-btn {
            position: static;
            margin-bottom: 1rem;
        }
    }
    /* Print Button */
    .print-btn {
        position: fixed;
        bottom: 1.5rem;
        right: 1.5rem;
        background: var(--primary);
        color: var(--white);
        border: none;
        padding: 0.75rem 1.25rem;
        border-radius: 2rem;
        font-size: 0.9rem;
        font-weight: 600;
        cursor: pointer;
        box-shadow: var(--shadow-lg);
        display: flex;
        align-items: center;
        gap: 0.5rem;
        transition: all 0.3s ease;
    }
    .print-btn:hover {
        background: var(--primary-dark);
        transform: translateY(-2px);
    }
    /* Simulation Styles */
    .simulation-container {
        background: var(--white);
        border-radius: 1rem;
        box-shadow: var(--shadow-lg);
        overflow: hidden;
    }
    .simulation-header {
        background: linear-gradient(135deg, var(--purple) 0%, #7c3aed 100%);
        color: var(--white);
        padding: 1.5rem;
        text-align: center;
    }
    .simulation-header h3 {
        font-size: 1.5rem;
        margin-bottom: 0.5rem;
    }
    .simulation-header p {
        opacity: 0.9;
        font-size: 0.95rem;
    }
    .simulation-progress {
        display: flex;
        justify-content: center;
        gap: 0.5rem;
        padding: 1rem;
        background: var(--gray-100);
        border-bottom: 1px solid var(--gray-200);
        flex-wrap: wrap;
    }
    .progress-step {
        display: flex;
        align-items: center;
        gap: 0.5rem;
```

```
            padding: 0.5rem 1rem;
            border-radius: 2rem;
            font-size: 0.8rem;
            font-weight: 600;
            background: var(--white);
            border: 2px solid var(--gray-300);
            color: var(--gray-500);
            cursor: pointer;
            transition: all 0.3s;
        }
        .progress-step:hover {
            border-color: var(--primary);
        }
        .progress-step.active {
            background: var(--primary);
            border-color: var(--primary);
            color: var(--white);
        }
        .progress-step.completed {
            background: var(--secondary);
            border-color: var(--secondary);
            color: var(--white);
        }
        .progress-step i {
            font-size: 0.9rem;
        }
        .simulation-body {
            padding: 2rem;
        }
        .sim-step {
            display: none;
        }
        .sim-step.active {
            display: block;
            animation: fadeIn 0.5s ease;
        }
        @keyframes fadeIn {
            from {
                opacity: 0;
                transform: translateY(10px);
            }
            to {
                opacity: 1;
                transform: translateY(0);
            }
        }
        /* Document Preview */
        .document-preview {
            background: var(--white);
            border: 2px solid var(--gray-300);
            border-radius: 0.5rem;
            max-width: 600px;
            margin: 1rem auto;
            box-shadow: var(--shadow);
            overflow: hidden;
        }
        .doc-header {
            background: linear-gradient(135deg, #1e40af 0%, #3b82f6 100%);
            color: var(--white);
            padding: 1rem;
            display: flex;
            justify-content: space-between;
            align-items: flex-start;
        }
        .doc-header.venta {
            background: linear-gradient(135deg, #166534 0%, #22c55e 100%);
        }
        .doc-company h4 {
            font-size: 1.1rem;
            margin-bottom: 0.25rem;
        }
        .doc-company p {
            font-size: 0.75rem;
            opacity: 0.9;
        }
        .doc-number {
            text-align: right;
        }
        .doc-number .type {
            font-size: 0.7rem;
            opacity: 0.9;
```

```
        }
        .doc-number .number {
            font-size: 1.25rem;
            font-weight: 700;
        }
        .doc-info {
            padding: 1rem;
            background: var(--gray-100);
            font-size: 0.85rem;
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 0.5rem;
        }
        .doc-info-item {
            display: flex;
            flex-direction: column;
        }
        .doc-info-item .label {
            font-size: 0.7rem;
            color: var(--gray-500);
            text-transform: uppercase;
        }
        .doc-info-item .value {
            font-weight: 600;
        }
        .doc-table {
            width: 100%;
            border-collapse: collapse;
            font-size: 0.85rem;
        }
        .doc-table th {
            background: var(--gray-100);
            padding: 0.75rem;
            text-align: left;
            font-weight: 600;
            border-bottom: 2px solid var(--gray-300);
        }
        .doc-table td {
            padding: 0.75rem;
            border-bottom: 1px solid var(--gray-200);
        }
        .doc-table .right {
            text-align: right;
        }
        .doc-totals {
            background: var(--gray-50);
            padding: 1rem;
        }
        .doc-totals-row {
            display: flex;
            justify-content: flex-end;
            gap: 2rem;
            padding: 0.25rem 0;
            font-size: 0.9rem;
        }
        .doc-totals-row.total {
            font-weight: 700;
            font-size: 1.1rem;
            color: var(--primary-dark);
            border-top: 2px solid var(--gray-300);
            padding-top: 0.5rem;
            margin-top: 0.5rem;
        }
        .doc-totals-row .label {
            color: var(--gray-500);
        }
        /* Analysis Box */
        .analysis-box {
            background: linear-gradient(135deg, #fef3c7 0%, #fde68a 100%);
            border: 2px solid var(--primary);
            border-radius: 0.75rem;
            padding: 1.5rem;
            margin: 1.5rem 0;
        }
        .analysis-box h4 {
            color: var(--primary-dark);
            margin-bottom: 1rem;
            display: flex;
            align-items: center;
            gap: 0.5rem;
        }
```

```
.analysis-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 1rem;
}
.analysis-item {
    background: var(--white);
    padding: 1rem;
    border-radius: 0.5rem;
    text-align: center;
}
.analysis-item .icon {
    font-size: 1.5rem;
    margin-bottom: 0.5rem;
}
.analysis-item.activo .icon {
    color: #3b82f6;
}
.analysis-item.pasivo .icon {
    color: #ef4444;
}
.analysis-item.ingreso .icon {
    color: #22c55e;
}
.analysis-item.gasto .icon {
    color: #f97316;
}
.analysis-item .name {
    font-weight: 600;
    font-size: 0.9rem;
}
.analysis-item .type {
    font-size: 0.75rem;
    color: var(--gray-500);
}
.analysis-item .movement {
    font-size: 0.8rem;
    margin-top: 0.5rem;
    padding: 0.25rem 0.5rem;
    border-radius: 1rem;
    display: inline-block;
}
.analysis-item .movement.debit {
    background: #fecaca;
    color: #991b1b;
}
.analysis-item .movement.credit {
    background: #bbf7d0;
    color: #166534;
}
/* Journal Entry Animation */
.journal-entry-animated {
    background: var(--white);
    border: 2px solid var(--gray-300);
    border-radius: 0.75rem;
    overflow: hidden;
    max-width: 700px;
    margin: 1rem auto;
}
.journal-header {
    background: var(--purple);
    color: var(--white);
    padding: 1rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
}
.journal-header h4 {
    display: flex;
    align-items: center;
    gap: 0.5rem;
}
.journal-row {
    display: grid;
    grid-template-columns: 2fr 1fr 1fr;
    border-bottom: 1px solid var(--gray-200);
}
.journal-row.header {
    background: var(--gray-100);
    font-weight: 600;
}
```

```
.journal-row>div {
    padding: 0.75rem;
}
.journal-row .account {
    display: flex;
    align-items: center;
    gap: 0.5rem;
}
.journal-row .debit {
    color: #dc2626;
    font-weight: 600;
    text-align: right;
}
.journal-row .credit {
    color: #16a34a;
    font-weight: 600;
    text-align: right;
}
.journal-row.animate {
    animation: slideIn 0.5s ease;
}
@keyframes slideIn {
    from {
        opacity: 0;
        transform: translateX(-20px);
    }
    to {
        opacity: 1;
        transform: translateX(0);
    }
}
.journal-totals {
    background: var(--gray-100);
    display: grid;
    grid-template-columns: 2fr 1fr 1fr;
    font-weight: 700;
}
.journal-totals>div {
    padding: 0.75rem;
}
/* Ledger T Account */
.ledger-accounts {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 1rem;
    margin: 1rem 0;
}
.ledger-t {
    background: var(--white);
    border: 2px solid var(--gray-300);
    border-radius: 0.5rem;
    overflow: hidden;
}
.ledger-t-header {
    background: var(--accent);
    color: var(--white);
    padding: 0.75rem;
    text-align: center;
    font-weight: 600;
    font-size: 0.9rem;
}
.ledger-t-header.asset {
    background: #3b82f6;
}
.ledger-t-header.liability {
    background: #ef4444;
}
.ledger-t-header.revenue {
    background: #22c55e;
}
.ledger-t-body {
    display: grid;
    grid-template-columns: 1fr 1fr;
    min-height: 80px;
}
.ledger-t-side {
    padding: 0.75rem;
    text-align: center;
    font-size: 0.85rem;
}
.ledger-t-side:first-child {
```

```
            border-right: 2px solid var(--gray-300);
        }
        .ledger-t-side h6 {
            font-size: 0.7rem;
            color: var(--gray-500);
            margin-bottom: 0.5rem;
        }
        .ledger-t-side .amount {
            font-weight: 700;
            font-size: 1rem;
        }
        .ledger-t-side.debit .amount {
            color: #dc2626;
        }
        .ledger-t-side.credit .amount {
            color: #16a34a;
        }
        .ledger-t-balance {
            background: var(--gray-100);
            padding: 0.5rem;
            text-align: center;
            font-size: 0.8rem;
            font-weight: 600;
        }
        /* Financial Statements Mini */
        .fs-container {
            display: grid;
            grid-template-columns: 1fr 1fr;
            gap: 1.5rem;
            margin: 1rem 0;
        }
        .fs-statement {
            background: var(--white);
            border: 2px solid var(--gray-300);
            border-radius: 0.75rem;
            overflow: hidden;
        }
        .fs-header {
            padding: 1rem;
            text-align: center;
            color: var(--white);
        }
        .fs-header.balance {
            background: linear-gradient(135deg, #1e40af 0%, #3b82f6 100%);
        }
        .fs-header.results {
            background: linear-gradient(135deg, #166534 0%, #22c55e 100%);
        }
        .fs-header h4 {
            font-size: 1rem;
        }
        .fs-header p {
            font-size: 0.75rem;
            opacity: 0.9;
        }
        .fs-body {
            padding: 1rem;
            font-size: 0.85rem;
        }
        .fs-section {
            margin-bottom: 1rem;
        }
        .fs-section h5 {
            font-size: 0.8rem;
            color: var(--gray-500);
            text-transform: uppercase;
            margin-bottom: 0.5rem;
            padding-bottom: 0.25rem;
            border-bottom: 1px solid var(--gray-200);
        }
        .fs-row {
            display: flex;
            justify-content: space-between;
            padding: 0.35rem 0;
        }
        .fs-row.highlight {
            background: var(--primary-light);
            padding: 0.5rem;
            margin: 0.5rem -1rem;
            font-weight: 700;
        }
```

```
        .fs-row.total {
            border-top: 2px solid var(--gray-300);
            margin-top: 0.5rem;
            padding-top: 0.5rem;
            font-weight: 700;
        }
        .fs-row .positive {
            color: #16a34a;
        }
        .fs-row .negative {
            color: #dc2626;
        }
        /* Navigation Buttons */
        .sim-nav {
            display: flex;
            justify-content: space-between;
            margin-top: 2rem;
            padding-top: 1.5rem;
            border-top: 1px solid var(--gray-200);
        }
        .sim-btn {
            padding: 0.75rem 1.5rem;
            border-radius: 0.5rem;
            font-size: 0.9rem;
            font-weight: 600;
            cursor: pointer;
            border: 2px solid var(--gray-300);
            background: var(--white);
            color: var(--dark);
            display: flex;
            align-items: center;
            gap: 0.5rem;
            transition: all 0.2s;
        }
        .sim-btn:hover {
            border-color: var(--primary);
            background: var(--primary-light);
        }
        .sim-btn.primary {
            background: var(--primary);
            border-color: var(--primary);
            color: var(--white);
        }
        .sim-btn.primary:hover {
            background: var(--primary-dark);
            border-color: var(--primary-dark);
        }
        .sim-btn:disabled {
            opacity: 0.5;
            cursor: not-allowed;
        }
        @media (max-width: 768px) {
            .fs-container {
                grid-template-columns: 1fr;
            }
            .simulation-progress {
                gap: 0.25rem;
            }
            .progress-step {
                padding: 0.4rem 0.75rem;
                font-size: 0.7rem;
            }
        }
    </style>
</head>
<body>
    <a href="index.html" class="back-btn">
        <i class="fas fa-arrow-left"></i> Volver al Sistema
    </a>
    <div class="container">
        <header class="header">
            <h1><i class="fas fa-graduation-cap"></i> Curso: Sistema Contable</h1>
            <p>
                Aprende el proceso contable completo: desde los documentos mercantiles
hasta los estados financieros.
                Diseñado para estudiantes de primer año.
            </p>
        </header>
        <div class="tabs">
            <button class="tab-btn active" onclick="showTab('flujo')">
                <i class="fas fa-sitemap"></i> Flujo Contable
```

```
            </button>
            <button class="tab-btn" onclick="showTab('modulos')">
                <i class="fas fa-cubes"></i> Módulos del ERP
            </button>
            <button class="tab-btn" onclick="showTab('cuentas')">
                <i class="fas fa-book"></i> Plan de Cuentas
            </button>
            <button class="tab-btn" onclick="showTab('asientos')">
                <i class="fas fa-edit"></i> Asientos Contables
            </button>
            <button class="tab-btn" onclick="showTab('ejemplo')">
                <i class="fas fa-play-circle"></i> Ejemplo Completo
            </button>
            <button class="tab-btn" onclick="showTab('ratios')">
                <i class="fas fa-chart-bar"></i> Ratios Financieros
            </button>
            <button class="tab-btn" onclick="showTab('sistemas')">
                <i class="fas fa-book"></i> Sistemas Contables
            </button>
            <button class="tab-btn" onclick="showTab('flujos')">
                <i class="fas fa-exchange-alt"></i> Flujos de Trabajo
            </button>
        </div>
        <div id="tab-flujo" class="tab-content active">
            <div class="flow-diagram">
                <div class="stage entrada">
                    <div class="sub-items">
                        <div class="sub-item" onclick="openModal('documento')">
                            <span class="plus">+</span> Documento Mercantil
                        </div>
                        <div class="sub-item" onclick="openModal('elementos')">
                            <span class="plus">+</span> Elementos Contables
                        </div>
                        <div class="sub-item" onclick="openModal('registro')">
                            <span class="plus">+</span> Reglas de Registro
                        </div>
                    </div>
                    <div class="stage-main entrada" onclick="openModal('entrada')">
                        <div class="stage-icon"><i class="fas fa-sign-in-alt"></i></div>
                        <div class="stage-title">Entrada</div>
                        <div class="stage-subtitle">Documentación</div>
                    </div>
                </div>
                <div class="flow-arrow"><i class="fas fa-arrow-right"></i></div>
                <div class="stage proceso">
                    <div class="stage-main proceso" onclick="openModal('proceso')">
                        <div class="stage-icon"><i class="fas fa-cogs"></i></div>
                        <div class="stage-title">Proceso</div>
                        <div class="stage-subtitle">Registro Contable</div>
                    </div>
                </div>
                <div class="flow-nexus"
                    style="display: flex; flex-direction: column; align-items: center;
justify-content: center; gap: 0.5rem; cursor: pointer;"
                    onclick="openModal('balance')">
                    <i class="fas fa-arrow-right" style="color: #94a3b8; font-size:
1.25rem;"></i>
                    <div style="background: #f3e8ff; color: #7c3aed; border: 1px dashed
#c084fc; padding: 0.5rem 1rem; border-radius: 0.5rem; font-size: 0.75rem; font-weight:
600; text-align: center; box-shadow: 0 1px 3px rgba(0,0,0,0.1); transition: all 0.2s
ease;"
                        onmouseover="this.style.transform='scale(1.05)';"
onmouseout="this.style.transform='scale(1)';">
                        <i class="fas fa-list-check"
                            style="margin-bottom: 0.2rem; display: block;">
</i>Balance<br>Comprobación
                    </div>
                    <i class="fas fa-arrow-right" style="color: #94a3b8; font-size:
1.25rem;"></i>
                </div>
                <div class="stage salida">
                    <div class="stage-main salida" onclick="openModal('salida')">
                        <div class="stage-icon"><i class="fas fa-file-export"></i></div>
                        <div class="stage-title">Salida</div>
                        <div class="stage-subtitle">Reportes</div>
                    </div>
                    <div class="sub-items">
                        <div class="sub-item" onclick="openModal('situacion')">
                            <span class="plus">+</span> Estado Situación
                        </div>
                        <div class="sub-item" onclick="openModal('resultados')">
```

```
                            <span class="plus">+</span> Estado Resultados
                        </div>
                    </div>
                </div>
            </div>
            <div class="description-section">
                <div class="description-header">
                    <h2><i class="fas fa-book-open"></i> Descripción del Proceso
Contable</h2>
                </div>
                <div class="description-content">
                    <p>
                        El <strong>proceso contable</strong> es el conjunto de pasos
ordenados para registrar,
                        clasificar
                        y resumir las transacciones económicas de una empresa, produciendo
información financiera útil.
                    </p>
                    <p>
                        <strong>Principio fundamental:</strong> Toda transacción afecta al
menos a dos cuentas
                        (partida doble), donde los débitos siempre deben igualar a los
créditos.
                    </p>
                </div>
            </div>
        </div>
        <div id="tab-modulos" class="tab-content">
            <div class="erp-modules">
                <h3><i class="fas fa-project-diagram"></i> Relación con los Módulos del
ERP</h3>
                <p style="margin-bottom: 1.5rem; color: var(--gray-500);">
                    Cada módulo del sistema genera documentos y transacciones que se
registran contablemente.
                    Haz clic para ver ejemplos en el sistema.
                </p>
                <div class="modules-grid">
                    <div class="module-card compras" onclick="goToModule('compras',
'facturas-proveedor')">
                        <i class="fas fa-shopping-cart"></i>
                        <h4>MM - Compras</h4>
                        <p>Órdenes de compra y facturas de proveedor</p>
                        <span class="badge">Ver en el sistema →</span>
                    </div>
                    <div class="module-card ventas" onclick="goToModule('ventas',
'facturas-cliente')">
                        <i class="fas fa-chart-line"></i>
                        <h4>SD - Ventas</h4>
                        <p>Cotizaciones y facturas a clientes</p>
                        <span class="badge">Ver en el sistema →</span>
                    </div>
                    <div class="module-card inventario" onclick="goToModule('inventario',
'movimientos-stock')">
                        <i class="fas fa-boxes"></i>
                        <h4>WM - Inventario</h4>
                        <p>Stock y movimientos de mercadería</p>
                        <span class="badge">Ver en el sistema →</span>
                    </div>
                    <div class="module-card tesoreria" onclick="goToModule('tesoreria',
'bancos')">
                        <i class="fas fa-landmark"></i>
                        <h4>FI - Tesorería</h4>
                        <p>Bancos, pagos y cobros</p>
                        <span class="badge">Ver en el sistema →</span>
                    </div>
                    <div class="module-card contabilidad"
onclick="goToModule('contabilidad', 'asientos')">
                        <i class="fas fa-book"></i>
                        <h4>GL - Contabilidad</h4>
                        <p>Asientos, libro mayor y diario</p>
                        <span class="badge">Ver en el sistema →</span>
                    </div>
                    <div class="module-card reportes" onclick="goToModule('reportes',
'balance-general')">
                        <i class="fas fa-chart-pie"></i>
                        <h4>Reportes</h4>
                        <p>Estados financieros</p>
                        <span class="badge">Ver en el sistema →</span>
                    </div>
                </div>
            </div>
```

```
            <div class="description-section" style="margin-top: 1.5rem;">
                <div class="description-header">
                    <h2><i class="fas fa-link"></i> Flujo de Información entre
Módulos</h2>
                </div>
                <div class="description-content">
                    <p><strong>1. Compras → Inventario → Contabilidad</strong></p>
                    <p style="margin-left: 1rem; font-size: 0.9rem;">
                        Al registrar una factura de compra, automáticamente se actualiza
el inventario
                        y se genera un asiento contable (Mercaderías + IVA Crédito =
Proveedores).
                    </p>
                    <p><strong>2. Ventas → Inventario → Contabilidad</strong></p>
                    <p style="margin-left: 1rem; font-size: 0.9rem;">
                        Al emitir una factura de venta, se rebaja el inventario y se
genera un asiento
                        (Clientes = Ingresos por Ventas + IVA Débito).
                    </p>
                    <p><strong>3. Tesorería → Contabilidad</strong></p>
                    <p style="margin-left: 1rem; font-size: 0.9rem;">
                        Los pagos y cobros actualizan las cuentas bancarias y generan
asientos que
                        cancelan las cuentas por pagar (Proveedores) o por cobrar
(Clientes).
                    </p>
                </div>
            </div>
        </div>
        <div id="tab-cuentas" class="tab-content">
            <div class="erp-modules">
                <h3><i class="fas fa-sitemap"></i> Clasificación de Cuentas Contables</h3>
                <div class="account-box">
                    <h5>Los 5 Elementos Contables</h5>
                    <div class="account-grid">
                        <div class="account-item">
                            <div class="label">ACTIVO</div>
                            <div class="value">Lo que TIENE</div>
                        </div>
                        <div class="account-item">
                            <div class="label">PASIVO</div>
                            <div class="value">Lo que DEBE</div>
                        </div>
                        <div class="account-item">
                            <div class="label">PATRIMONIO</div>
                            <div class="value">De los dueños</div>
                        </div>
                        <div class="account-item">
                            <div class="label">INGRESO</div>
                            <div class="value">Lo que GANA</div>
                        </div>
                        <div class="account-item">
                            <div class="label">GASTO</div>
                            <div class="value">Lo que GASTA</div>
                        </div>
                    </div>
                </div>
                <div style="margin-top: 1.5rem;">
                    <h4 style="margin-bottom: 1rem;">Ecuación Contable Fundamental</h4>
                    <p
                        style="font-size: 1.5rem; text-align: center; padding: 1rem;
background: var(--primary-light); border-radius: 0.5rem; color: var(--dark);">
                        <strong>ACTIVO = PASIVO + PATRIMONIO</strong>
                    </p>
                </div>
                <div style="margin-top: 1.5rem;">
                    <h4 style="margin-bottom: 1rem;">Naturaleza de las Cuentas</h4>
                    <table style="width: 100%; border-collapse: collapse; font-size:
0.9rem;">
                        <thead>
                            <tr style="background: var(--gray-100);">
                                <th
                                    style="padding: 0.75rem; text-align: left; border-
bottom: 2px solid var(--gray-300);">
                                    Tipo</th>
                                <th
                                    style="padding: 0.75rem; text-align: center; border-
bottom: 2px solid var(--gray-300);">
                                    Aumenta con</th>
                                <th
                                    style="padding: 0.75rem; text-align: center; border-
```

```
bottom: 2px solid var(--gray-300);">
                                    Disminuye con</th>
                                <th
                                    style="padding: 0.75rem; text-align: center; border-
bottom: 2px solid var(--gray-300);">
                                    Saldo Normal</th>
                            </tr>
                        </thead>
                        <tbody>
                            <tr>
                                <td style="padding: 0.75rem; border-bottom: 1px solid
var(--gray-200);">Activo</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#dc2626; font-weight: 600;">
                                    Débito</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#16a34a;">Crédito</td>
                                <td style="padding: 0.75rem; text-align:
center;">Deudor</td>
                            </tr>
                            <tr>
                                <td style="padding: 0.75rem; border-bottom: 1px solid
var(--gray-200);">Pasivo</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#16a34a; font-weight: 600;">
                                    Crédito</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#dc2626;">Débito</td>
                                <td style="padding: 0.75rem; text-align:
center;">Acreedor</td>
                            </tr>
                            <tr>
                                <td style="padding: 0.75rem; border-bottom: 1px solid
var(--gray-200);">Patrimonio</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#16a34a; font-weight: 600;">
                                    Crédito</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#dc2626;">Débito</td>
                                <td style="padding: 0.75rem; text-align:
center;">Acreedor</td>
                            </tr>
                            <tr>
                                <td style="padding: 0.75rem; border-bottom: 1px solid
var(--gray-200);">Ingreso</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#16a34a; font-weight: 600;">
                                    Crédito</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#dc2626;">Débito</td>
                                <td style="padding: 0.75rem; text-align:
center;">Acreedor</td>
                            </tr>
                            <tr>
                                <td style="padding: 0.75rem;">Gasto</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#dc2626; font-weight: 600;">
                                    Débito</td>
                                <td style="padding: 0.75rem; text-align: center; color:
#16a34a;">Crédito</td>
                                <td style="padding: 0.75rem; text-align:
center;">Deudor</td>
                            </tr>
                        </tbody>
                    </table>
                </div>
                <button class="link-btn" onclick="goToModule('contabilidad', 'plan-
cuentas')">
                    <i class="fas fa-external-link-alt"></i> Ver Plan de Cuentas en el
Sistema
                </button>
            </div>
        </div>
        <div id="tab-asientos" class="tab-content">
            <div class="erp-modules">
                <h3><i class="fas fa-edit"></i> Ejemplos de Asientos Contables</h3>
                <div class="flow-example">
                    <h5><i class="fas fa-shopping-cart"></i> Compra de Mercaderías a
Crédito</h5>
                    <p style="margin-bottom: 1rem;"><strong>Situación:</strong> Compra de
productos por $1.000.000 + IVA
```

```
                        (19%)</p>
                    <table
                        style="width: 100%; border-collapse: collapse; background: white;
border-radius: 0.5rem; overflow: hidden; font-size: 0.9rem;">
                        <thead style="background: var(--secondary); color: white;">
                            <tr>
                                <th style="padding: 0.75rem; text-align:
left;">Cuenta</th>
                                <th style="padding: 0.75rem; text-align: right;">Debe</th>
                                <th style="padding: 0.75rem; text-align:
right;">Haber</th>
                            </tr>
                        </thead>
                        <tbody>
                            <tr style="border-bottom: 1px solid var(--gray-200);">
                                <td style="padding: 0.75rem;">Mercaderías (Activo ↑)</td>
                                <td style="padding: 0.75rem; text-align: right; color:
#dc2626; font-weight: 600;">
                                    $1.000.000</td>
                                <td style="padding: 0.75rem;"></td>
                            </tr>
                            <tr style="border-bottom: 1px solid var(--gray-200);">
                                <td style="padding: 0.75rem;">IVA Crédito Fiscal (Activo
↑)</td>
                                <td style="padding: 0.75rem; text-align: right; color:
#dc2626; font-weight: 600;">
                                    $190.000</td>
                                <td style="padding: 0.75rem;"></td>
                            </tr>
                            <tr>
                                <td style="padding: 0.75rem;">Proveedores (Pasivo ↑)</td>
                                <td style="padding: 0.75rem;"></td>
                                <td style="padding: 0.75rem; text-align: right; color:
#16a34a; font-weight: 600;">
                                    $1.190.000</td>
                            </tr>
                        </tbody>
                    </table>
                </div>
                <div class="flow-example">
                    <h5><i class="fas fa-chart-line"></i> Venta de Productos a
Crédito</h5>
                    <p style="margin-bottom: 1rem;"><strong>Situación:</strong> Venta de
productos por $2.000.000 + IVA
                        (19%)</p>
                    <table
                        style="width: 100%; border-collapse: collapse; background: white;
border-radius: 0.5rem; overflow: hidden; font-size: 0.9rem;">
                        <thead style="background: var(--accent); color: white;">
                            <tr>
                                <th style="padding: 0.75rem; text-align:
left;">Cuenta</th>
                                <th style="padding: 0.75rem; text-align: right;">Debe</th>
                                <th style="padding: 0.75rem; text-align:
right;">Haber</th>
                            </tr>
                        </thead>
                        <tbody>
                            <tr style="border-bottom: 1px solid var(--gray-200);">
                                <td style="padding: 0.75rem;">Clientes (Activo ↑)</td>
                                <td style="padding: 0.75rem; text-align: right; color:
#dc2626; font-weight: 600;">
                                    $2.380.000</td>
                                <td style="padding: 0.75rem;"></td>
                            </tr>
                            <tr style="border-bottom: 1px solid var(--gray-200);">
                                <td style="padding: 0.75rem;">Ingresos por Ventas (Ingreso
↑)</td>
                                <td style="padding: 0.75rem;"></td>
                                <td style="padding: 0.75rem; text-align: right; color:
#16a34a; font-weight: 600;">
                                    $2.000.000</td>
                            </tr>
                            <tr>
                                <td style="padding: 0.75rem;">IVA Débito Fiscal (Pasivo ↑)
</td>
                                <td style="padding: 0.75rem;"></td>
                                <td style="padding: 0.75rem; text-align: right; color:
#16a34a; font-weight: 600;">
                                    $380.000</td>
                            </tr>
```

```
                        </tbody>
                    </table>
                </div>
                <div class="flow-example">
                    <h5><i class="fas fa-money-bill-wave"></i> Pago a Proveedor</h5>
                    <p style="margin-bottom: 1rem;"><strong>Situación:</strong> Pago de
factura por $1.190.000 con
                        transferencia bancaria</p>
                    <table
                        style="width: 100%; border-collapse: collapse; background: white;
border-radius: 0.5rem; overflow: hidden; font-size: 0.9rem;">
                        <thead style="background: var(--purple); color: white;">
                            <tr>
                                <th style="padding: 0.75rem; text-align:
left;">Cuenta</th>
                                <th style="padding: 0.75rem; text-align: right;">Debe</th>
                                <th style="padding: 0.75rem; text-align:
right;">Haber</th>
                            </tr>
                        </thead>
                        <tbody>
                            <tr style="border-bottom: 1px solid var(--gray-200);">
                                <td style="padding: 0.75rem;">Proveedores (Pasivo ↓)</td>
                                <td style="padding: 0.75rem; text-align: right; color:
#dc2626; font-weight: 600;">
                                    $1.190.000</td>
                                <td style="padding: 0.75rem;"></td>
                            </tr>
                            <tr>
                                <td style="padding: 0.75rem;">Banco (Activo ↓)</td>
                                <td style="padding: 0.75rem;"></td>
                                <td style="padding: 0.75rem; text-align: right; color:
#16a34a; font-weight: 600;">
                                    $1.190.000</td>
                            </tr>
                        </tbody>
                    </table>
                </div>
                <button class="link-btn success" onclick="goToModule('contabilidad',
'asientos')">
                    <i class="fas fa-external-link-alt"></i> Ver Libro Diario en el
Sistema
                </button>
            </div>
        </div>
        <div id="tab-ejemplo" class="tab-content">
            <div class="simulation-container">
                <div class="simulation-header">
                    <h3><i class="fas fa-play-circle"></i> Simulación: Ciclo Completo
Compra-Venta</h3>
                    <p>Sigue paso a paso cómo una compra y posterior venta afectan los
estados financieros, incluyendo
                        el Estado de Resultados</p>
                </div>
                <div class="simulation-progress">
                    <div class="progress-step active" onclick="goToSimStep(1)">
                        <i class="fas fa-shopping-cart"></i> 1. Compra
                    </div>
                    <div class="progress-step" onclick="goToSimStep(2)">
                        <i class="fas fa-search"></i> 2. Análisis
                    </div>
                    <div class="progress-step" onclick="goToSimStep(3)">
                        <i class="fas fa-edit"></i> 3. Asiento
                    </div>
                    <div class="progress-step" onclick="goToSimStep(4)">
                        <i class="fas fa-columns"></i> 4. Mayor
                    </div>
                    <div class="progress-step" onclick="goToSimStep(5)">
                        <i class="fas fa-receipt"></i> 5. Venta
                    </div>
                    <div class="progress-step" onclick="goToSimStep(6)">
                        <i class="fas fa-book"></i> 6. Asiento Venta
                    </div>
                    <div class="progress-step" onclick="goToSimStep(7)">
                        <i class="fas fa-columns"></i> 7. Mayor Venta
                    </div>
                    <div class="progress-step" onclick="goToSimStep(8)">
                        <i class="fas fa-chart-pie"></i> 8. Estados Finales
                    </div>
                </div>
                <div class="simulation-body">
```

```
                    <div id="sim-step-1" class="sim-step active">
                        <h4 style="margin-bottom: 1rem; display: flex; align-items:
center; gap: 0.5rem;">
                            <i class="fas fa-file-invoice" style="color: var(--accent);">
</i>
                            Paso 1: El Hecho Económico genera un Documento Mercantil
                        </h4>
                        <p style="margin-bottom: 1.5rem; color: var(--gray-700);">
                            La empresa <strong>"Comercial Los Andes Ltda."</strong>
realiza una compra de mercaderías a
                            crédito.
                            El proveedor emite una <strong>Factura Electrónica</strong>
que respalda esta transacción.
                        </p>
                        <div class="document-preview">
                            <div class="doc-header">
                                <div class="doc-company">
                                    <h4><i class="fas fa-building"></i> DISTRIBUIDORA TECH
SPA</h4>
                                    <p>RUT: 76.543.210-K • Av. Providencia 1234,
Santiago</p>
                                </div>
                                <div class="doc-number">
                                    <div class="type">FACTURA ELECTRÓNICA</div>
                                    <div class="number">N° 00542</div>
                                </div>
                            </div>
                            <div class="doc-info">
                                <div class="doc-info-item">
                                    <span class="label">Fecha Emisión</span>
                                    <span class="value">19/12/2025</span>
                                </div>
                                <div class="doc-info-item">
                                    <span class="label">Condición de Pago</span>
                                    <span class="value">Crédito 30 días</span>
                                </div>
                                <div class="doc-info-item">
                                    <span class="label">Señor(es)</span>
                                    <span class="value">Comercial Los Andes Ltda.</span>
                                </div>
                                <div class="doc-info-item">
                                    <span class="label">RUT</span>
                                    <span class="value">76.123.456-7</span>
                                </div>
                            </div>
                            <table class="doc-table">
                                <thead>
                                    <tr>
                                        <th>Cant.</th>
                                        <th>Descripción</th>
                                        <th class="right">P. Unit.</th>
                                        <th class="right">Total</th>
                                    </tr>
                                </thead>
                                <tbody>
                                    <tr>
                                        <td>10</td>
                                        <td>Notebook Lenovo ThinkPad L15</td>
                                        <td class="right">$500.000</td>
                                        <td class="right">$5.000.000</td>
                                    </tr>
                                </tbody>
                            </table>
                            <div class="doc-totals">
                                <div class="doc-totals-row">
                                    <span class="label">NETO:</span>
                                    <span>$5.000.000</span>
                                </div>
                                <div class="doc-totals-row">
                                    <span class="label">IVA 19%:</span>
                                    <span>$950.000</span>
                                </div>
                                <div class="doc-totals-row total">
                                    <span class="label">TOTAL:</span>
                                    <span>$5.950.000</span>
                                </div>
                            </div>
                        </div>
                        <div class="analysis-box"
                            style="background: linear-gradient(135deg, #dbeafe 0%, #bfdbfe
100%); border-color: var(--accent);">
```

```
                            <h4><i class="fas fa-info-circle"></i> ¿Qué información
extraemos del documento?</h4>
                            <ul style="list-style: none; padding: 0;">
                                <li
                                    style="padding: 0.5rem; background: white; border-
radius: 0.5rem; margin-bottom: 0.5rem;">
                                    <strong>Tipo de transacción:</strong> Compra de
mercaderías
                                </li>
                                <li
                                    style="padding: 0.5rem; background: white; border-
radius: 0.5rem; margin-bottom: 0.5rem;">
                                    <strong>Forma de pago:</strong> A crédito (30 días) →
Genera deuda con proveedor
                                </li>
                                <li
                                    style="padding: 0.5rem; background: white; border-
radius: 0.5rem; margin-bottom: 0.5rem;">
                                    <strong>Montos:</strong> Neto $5.000.000 + IVA
$950.000 = Total $5.950.000
                                </li>
                                <li style="padding: 0.5rem; background: white; border-
radius: 0.5rem;">
                                    <strong>IVA:</strong> Es recuperable (Crédito Fiscal)
→ Se registra como Activo
                                </li>
                            </ul>
                        </div>
                        <div class="sim-nav">
                            <button class="sim-btn" disabled><i class="fas fa-arrow-left">
</i> Anterior</button>
                            <button class="sim-btn primary"
onclick="nextSimStep()">Siguiente <i
                                    class="fas fa-arrow-right"></i></button>
                        </div>
                    </div>
                    <div id="sim-step-2" class="sim-step">
                        <h4 style="margin-bottom: 1rem; display: flex; align-items:
center; gap: 0.5rem;">
                            <i class="fas fa-search" style="color: var(--secondary);"></i>
                            Paso 2: Análisis e Identificación de Cuentas
                        </h4>
                        <p style="margin-bottom: 1.5rem; color: var(--gray-700);">
                            Analizamos el documento para identificar qué elementos
contables se afectan,
                            su naturaleza (Activo, Pasivo, etc.) y si aumentan o
disminuyen.
                        </p>
                        <div class="analysis-box">
                            <h4><i class="fas fa-magnifying-glass"></i> Identificación de
Elementos Contables</h4>
                            <div class="analysis-grid">
                                <div class="analysis-item activo">
                                    <div class="icon"><i class="fas fa-box"></i></div>
                                    <div class="name">Mercaderías</div>
                                    <div class="type">ACTIVO</div>
                                    <div class="movement debit">↑ AUMENTA = DÉBITO</div>
                                </div>
                                <div class="analysis-item activo">
                                    <div class="icon"><i class="fas fa-receipt"></i></div>
                                    <div class="name">IVA Crédito Fiscal</div>
                                    <div class="type">ACTIVO</div>
                                    <div class="movement debit">↑ AUMENTA = DÉBITO</div>
                                </div>
                                <div class="analysis-item pasivo">
                                    <div class="icon"><i class="fas fa-handshake"></i>
</div>
                                    <div class="name">Proveedores</div>
                                    <div class="type">PASIVO</div>
                                    <div class="movement credit">↑ AUMENTA = CRÉDITO</div>
                                </div>
                            </div>
                        </div>
                        <div class="flow-example" style="margin-top: 1.5rem;">
                            <h5><i class="fas fa-lightbulb"></i> Regla de Oro: Partida
Doble</h5>
                            <p style="margin-bottom: 1rem;">
                                Toda transacción afecta <strong>al menos 2
cuentas</strong>.
                                La suma de los DÉBITOS debe ser igual a la suma de los
CRÉDITOS.
```

```
                            </p>
                            <div
                                style="display: flex; justify-content: center; align-
items: center; gap: 1rem; flex-wrap: wrap;">
                                <div
                                    style="background: #fecaca; padding: 1rem 2rem;
border-radius: 0.5rem; text-align: center;">
                                    <div style="font-size: 0.8rem; color:
#991b1b;">DEBE</div>
                                    <div style="font-size: 1.5rem; font-weight: 700;
color: #dc2626;">$5.950.000</div>
                                </div>
                                <div style="font-size: 2rem; font-weight: 700; color:
var(--gray-300);">=</div>
                                <div
                                    style="background: #bbf7d0; padding: 1rem 2rem;
border-radius: 0.5rem; text-align: center;">
                                    <div style="font-size: 0.8rem; color:
#166534;">HABER</div>
                                    <div style="font-size: 1.5rem; font-weight: 700;
color: #16a34a;">$5.950.000</div>
                                </div>
                            </div>
                        </div>
                        <div class="sim-nav">
                            <button class="sim-btn" onclick="prevSimStep()"><i class="fas
fa-arrow-left"></i>
                                Anterior</button>
                            <button class="sim-btn primary"
onclick="nextSimStep()">Siguiente <i
                                    class="fas fa-arrow-right"></i></button>
                        </div>
                    </div>
                    <div id="sim-step-3" class="sim-step">
                        <h4 style="margin-bottom: 1rem; display: flex; align-items:
center; gap: 0.5rem;">
                            <i class="fas fa-edit" style="color: var(--purple);"></i>
                            Paso 3: Registro en el Libro Diario (Asiento Contable)
                        </h4>
                        <p style="margin-bottom: 1.5rem; color: var(--gray-700);">
                            Con el análisis completado, registramos la transacción en el
<strong>Libro Diario</strong>.
                            Este es el registro cronológico de todas las operaciones.
                        </p>
                        <div class="journal-entry-animated">
                            <div class="journal-header">
                                <h4><i class="fas fa-book"></i> LIBRO DIARIO</h4>
                                <span>Asiento N° 45 • 19/12/2025</span>
                            </div>
                            <div class="journal-row header">
                                <div>Cuenta</div>
                                <div style="text-align: right;">Debe</div>
                                <div style="text-align: right;">Haber</div>
                            </div>
                            <div class="journal-row animate" style="animation-delay:
0.2s;">
                                <div class="account"><i class="fas fa-box" style="color:
#3b82f6;"></i> Mercaderías
                                </div>
                                <div class="debit">$5.000.000</div>
                                <div></div>
                            </div>
                            <div class="journal-row animate" style="animation-delay:
0.4s;">
                                <div class="account"><i class="fas fa-receipt"
style="color: #3b82f6;"></i> IVA Crédito
                                    Fiscal</div>
                                <div class="debit">$950.000</div>
                                <div></div>
                            </div>
                            <div class="journal-row animate" style="animation-delay:
0.6s;">
                                <div class="account"><i class="fas fa-handshake"
style="color: #ef4444;"></i>
                                    Proveedores</div>
                                <div></div>
                                <div class="credit">$5.950.000</div>
                            </div>
                            <div class="journal-totals">
                                <div style="font-style: italic; color: var(--gray-
500);">TOTALES</div>
```

```
                                <div class="debit" style="text-align:
right;">$5.950.000</div>
                                <div class="credit" style="text-align:
right;">$5.950.000</div>
                            </div>
                        </div>
                        <div style="text-align: center; margin-top: 1rem; font-size:
0.9rem; color: var(--gray-500);">
                            <em>Glosa: Compra de mercaderías según Factura N° 00542 de
Distribuidora Tech SPA</em>
                        </div>
                        <div class="analysis-box"
                            style="background: linear-gradient(135deg, #f3e8ff 0%, #e9d5ff
100%); border-color: var(--purple);">
                            <h4 style="color: #7c3aed;"><i class="fas fa-check-circle">
</i> ¿Por qué el asiento está
                                cuadrado?</h4>
                            <ul style="list-style: none; padding: 0; margin: 0;">
                                <li style="padding: 0.4rem; font-size: 0.9rem;">✓
Mercaderías (Activo) aumenta → se
                                    DEBITA por $5.000.000</li>
                                <li style="padding: 0.4rem; font-size: 0.9rem;">✓ IVA
Crédito (Activo) aumenta → se
                                    DEBITA por $950.000</li>
                                <li style="padding: 0.4rem; font-size: 0.9rem;">✓
Proveedores (Pasivo) aumenta → se
                                    ACREDITA por $5.950.000</li>
                                <li style="padding: 0.4rem; font-size: 0.9rem; font-
weight: 600;">= DÉBITOS ($5.950.000)
                                    = CRÉDITOS ($5.950.000) ✓</li>
                            </ul>
                        </div>
                        <div class="sim-nav">
                            <button class="sim-btn" onclick="prevSimStep()"><i class="fas
fa-arrow-left"></i>
                                Anterior</button>
                            <button class="sim-btn primary"
onclick="nextSimStep()">Siguiente <i
                                    class="fas fa-arrow-right"></i></button>
                        </div>
                    </div>
                    <div id="sim-step-4" class="sim-step">
                        <h4 style="margin-bottom: 1rem; display: flex; align-items:
center; gap: 0.5rem;">
                            <i class="fas fa-columns" style="color: var(--secondary);">
</i>
                            Paso 4: Mayorización (Libro Mayor)
                        </h4>
                        <p style="margin-bottom: 1.5rem; color: var(--gray-700);">
                            El asiento se "traspasa" al <strong>Libro Mayor</strong>,
donde cada cuenta tiene su propia
                            página (Cuenta T).
                            Esto permite ver el saldo acumulado de cada cuenta.
                        </p>
                        <div class="ledger-accounts">
                            <div class="ledger-t">
                                <div class="ledger-t-header asset">
                                    <i class="fas fa-box"></i> MERCADERÍAS
                                </div>
                                <div class="ledger-t-body">
                                    <div class="ledger-t-side debit">
                                        <h6>DEBE</h6>
                                        <div class="amount">$5.000.000</div>
                                    </div>
                                    <div class="ledger-t-side credit">
                                        <h6>HABER</h6>
                                        <div class="amount">-</div>
                                    </div>
                                </div>
                                <div class="ledger-t-balance">Saldo Deudor:
$5.000.000</div>
                            </div>
                            <div class="ledger-t">
                                <div class="ledger-t-header asset">
                                    <i class="fas fa-receipt"></i> IVA CRÉDITO FISCAL
                                </div>
                                <div class="ledger-t-body">
                                    <div class="ledger-t-side debit">
                                        <h6>DEBE</h6>
                                        <div class="amount">$950.000</div>
```

```
                                    </div>
                                    <div class="ledger-t-side credit">
                                        <h6>HABER</h6>
                                        <div class="amount">-</div>
                                    </div>
                                </div>
                                <div class="ledger-t-balance">Saldo Deudor: $950.000</div>
                            </div>
                            <div class="ledger-t">
                                <div class="ledger-t-header liability">
                                    <i class="fas fa-handshake"></i> PROVEEDORES
                                </div>
                                <div class="ledger-t-body">
                                    <div class="ledger-t-side debit">
                                        <h6>DEBE</h6>
                                        <div class="amount">-</div>
                                    </div>
                                    <div class="ledger-t-side credit">
                                        <h6>HABER</h6>
                                        <div class="amount">$5.950.000</div>
                                    </div>
                                </div>
                                <div class="ledger-t-balance">Saldo Acreedor:
$5.950.000</div>
                            </div>
                        </div>
                        <div class="flow-example">
                            <h5><i class="fas fa-balance-scale"></i> Verificación:
Ecuación Contable</h5>
                            <p>Después de esta transacción, la ecuación contable sigue en
equilibrio:</p>
                            <div
                                style="display: flex; align-items: center; justify-
content: center; flex-wrap: wrap; gap: 0.5rem; margin: 1.5rem 0; padding: 1rem;
background: white; border-radius: 0.75rem;">
                                <div style="text-align: center; min-width: 140px;">
                                    <div
                                        style="background: #dbeafe; padding: 0.75rem
1.25rem; border-radius: 0.5rem; border: 2px solid #3b82f6;">
                                        <div
                                            style="font-size: 0.75rem; color: #1e40af;
font-weight: 600; text-transform: uppercase;">
                                            Activo</div>
                                        <div style="font-size: 1.1rem; font-weight: 700;
color: #16a34a;">+$5.950.000
                                        </div>
                                    </div>
                                </div>
                                <div style="font-size: 2rem; font-weight: 700; color:
#374151; padding: 0 0.5rem;">=
                                </div>
                                <div style="text-align: center; min-width: 120px;">
                                    <div
                                        style="background: #fecaca; padding: 0.75rem
1.25rem; border-radius: 0.5rem; border: 2px solid #ef4444;">
                                        <div
                                            style="font-size: 0.75rem; color: #991b1b;
font-weight: 600; text-transform: uppercase;">
                                            Pasivo</div>
                                        <div style="font-size: 1.1rem; font-weight: 700;
color: #dc2626;">+$5.950.000
                                        </div>
                                    </div>
                                </div>
                                <div style="font-size: 2rem; font-weight: 700; color:
#374151; padding: 0 0.5rem;">+
                                </div>
                                <div style="text-align: center; min-width: 130px;">
                                    <div
                                        style="background: #e9d5ff; padding: 0.75rem
1.25rem; border-radius: 0.5rem; border: 2px solid #8b5cf6;">
                                        <div
                                            style="font-size: 0.75rem; color: #5b21b6;
font-weight: 600; text-transform: uppercase;">
                                            Patrimonio</div>
                                        <div style="font-size: 1.1rem; font-weight: 700;
color: #7c3aed;">$0</div>
                                    </div>
                                </div>
                            </div>
                            <div
```

```
                                style="text-align: center; background: #fef3c7; border:
2px solid #f59e0b; border-radius: 0.5rem; padding: 0.75rem;">
                                <span style="font-size: 0.85rem; color: #92400e;">
                                    <i class="fas fa-info-circle"></i> La compra aumenta
<strong>Activo</strong>
                                    (Mercaderías + IVA) y <strong>Pasivo</strong>
(Proveedores) por igual
                                </span>
                            </div>
                        </div>
                        <div class="sim-nav">
                            <button class="sim-btn" onclick="prevSimStep()"><i class="fas
fa-arrow-left"></i>
                                Anterior</button>
                            <button class="sim-btn primary"
onclick="nextSimStep()">Siguiente <i
                                    class="fas fa-arrow-right"></i></button>
                        </div>
                    </div>
                    <div id="sim-step-5" class="sim-step">
                        <h4 style="margin-bottom: 1rem; display: flex; align-items:
center; gap: 0.5rem;">
                            <i class="fas fa-receipt" style="color: #22c55e;"></i>
                            Paso 5: Venta de Mercaderías - Factura de Venta
                        </h4>
                        <p style="margin-bottom: 1.5rem; color: var(--gray-700);">
                            Ahora la empresa vende <strong>5 de los 10 notebooks</strong>
comprados a un cliente.
                            Se emite una <strong>Factura de Venta</strong> con un margen
de ganancia del 50%.
                        </p>
                        <div class="document-preview">
                            <div class="doc-header venta">
                                <div class="doc-company">
                                    <h4><i class="fas fa-building"></i> COMERCIAL LOS
ANDES LTDA.</h4>
                                    <p>RUT: 76.123.456-7 • Av. Apoquindo 4500, Las
Condes</p>
                                </div>
                                <div class="doc-number">
                                    <div class="type">FACTURA ELECTRÓNICA</div>
                                    <div class="number">N° 00128</div>
                                </div>
                            </div>
                            <div class="doc-info">
                                <div class="doc-info-item">
                                    <span class="label">Fecha Emisión</span>
                                    <span class="value">22/12/2025</span>
                                </div>
                                <div class="doc-info-item">
                                    <span class="label">Condición de Pago</span>
                                    <span class="value">Crédito 30 días</span>
                                </div>
                                <div class="doc-info-item">
                                    <span class="label">Señor(es)</span>
                                    <span class="value">Constructora Los Andes S.A.</span>
                                </div>
                                <div class="doc-info-item">
                                    <span class="label">RUT</span>
                                    <span class="value">76.890.123-4</span>
                                </div>
                            </div>
                            <table class="doc-table">
                                <thead>
                                    <tr>
                                        <th>Cant.</th>
                                        <th>Descripción</th>
                                        <th class="right">P. Unit.</th>
                                        <th class="right">Total</th>
                                    </tr>
                                </thead>
                                <tbody>
                                    <tr>
                                        <td>5</td>
                                        <td>Notebook Lenovo ThinkPad L15</td>
                                        <td class="right">$750.000</td>
                                        <td class="right">$3.750.000</td>
                                    </tr>
                                </tbody>
                            </table>
                            <div class="doc-totals">
```

```
                                <div class="doc-totals-row">
                                    <span class="label">NETO:</span>
                                    <span>$3.750.000</span>
                                </div>
                                <div class="doc-totals-row">
                                    <span class="label">IVA 19%:</span>
                                    <span>$712.500</span>
                                </div>
                                <div class="doc-totals-row total">
                                    <span class="label">TOTAL:</span>
                                    <span>$4.462.500</span>
                                </div>
                            </div>
                        </div>
                        <div class="analysis-box"
                            style="background: linear-gradient(135deg, #dcfce7 0%, #bbf7d0
100%); border-color: #22c55e;">
                            <h4><i class="fas fa-calculator"></i> Análisis de la
Venta</h4>
                            <div class="analysis-grid">
                                <div class="analysis-item" style="text-align: left;">
                                    <div style="font-size: 0.8rem; color: var(--gray-
500);">Precio de Venta (5 unidades)
                                    </div>
                                    <div style="font-size: 1.2rem; font-weight: 700;
color: #22c55e;">$3.750.000</div>
                                </div>
                                <div class="analysis-item" style="text-align: left;">
                                    <div style="font-size: 0.8rem; color: var(--gray-
500);">Costo (5 × $500.000)</div>
                                    <div style="font-size: 1.2rem; font-weight: 700;
color: #dc2626;">$2.500.000</div>
                                </div>
                                <div class="analysis-item" style="text-align: left;">
                                    <div style="font-size: 0.8rem; color: var(--gray-
500);">Utilidad Bruta</div>
                                    <div style="font-size: 1.2rem; font-weight: 700;
color: #166534;">$1.250.000</div>
                                </div>
                                <div class="analysis-item" style="text-align: left;">
                                    <div style="font-size: 0.8rem; color: var(--gray-
500);">Margen de Ganancia</div>
                                    <div style="font-size: 1.2rem; font-weight: 700;
color: #f59e0b;">50%</div>
                                </div>
                            </div>
                        </div>
                        <div class="flow-example" style="margin-top: 1.5rem;">
                            <h5><i class="fas fa-lightbulb"></i> ¿Qué cuentas se afectan
en una venta?</h5>
                            <div class="analysis-grid" style="margin-top: 1rem;">
                                <div class="analysis-item activo">
                                    <div class="icon"><i class="fas fa-users"></i></div>
                                    <div class="name">Clientes por Cobrar</div>
                                    <div class="type">ACTIVO</div>
                                    <div class="movement debit">↑ AUMENTA = DÉBITO</div>
                                </div>
                                <div class="analysis-item ingreso">
                                    <div class="icon"><i class="fas fa-chart-line"></i>
</div>
                                    <div class="name">Ingresos por Ventas</div>
                                    <div class="type">INGRESO</div>
                                    <div class="movement credit">↑ AUMENTA = CRÉDITO</div>
                                </div>
                                <div class="analysis-item pasivo">
                                    <div class="icon"><i class="fas fa-receipt"></i></div>
                                    <div class="name">IVA Débito Fiscal</div>
                                    <div class="type">PASIVO</div>
                                    <div class="movement credit">↑ AUMENTA = CRÉDITO</div>
                                </div>
                                <div class="analysis-item gasto">
                                    <div class="icon"><i class="fas fa-box-open"></i>
</div>
                                    <div class="name">Costo de Ventas</div>
                                    <div class="type">GASTO</div>
                                    <div class="movement debit">↑ AUMENTA = DÉBITO</div>
                                </div>
                                <div class="analysis-item activo">
                                    <div class="icon"><i class="fas fa-box"></i></div>
                                    <div class="name">Mercaderías</div>
                                    <div class="type">ACTIVO</div>
```

```
                                    <div class="movement credit">↓ DISMINUYE =
CRÉDITO</div>
                                </div>
                            </div>
                        </div>
                        <div class="sim-nav">
                            <button class="sim-btn" onclick="prevSimStep()"><i class="fas
fa-arrow-left"></i>
                                Anterior</button>
                            <button class="sim-btn primary"
onclick="nextSimStep()">Siguiente <i
                                    class="fas fa-arrow-right"></i></button>
                        </div>
                    </div>
                    <div id="sim-step-6" class="sim-step">
                        <h4 style="margin-bottom: 1rem; display: flex; align-items:
center; gap: 0.5rem;">
                            <i class="fas fa-book" style="color: var(--purple);"></i>
                            Paso 6: Asientos Contables de la Venta
                        </h4>
                        <p style="margin-bottom: 1.5rem; color: var(--gray-700);">
                            La venta genera <strong>dos asientos contables</strong>: uno
para reconocer el ingreso
                            y otro para reconocer el costo de la mercadería vendida.
                        </p>
                        <div class="journal-entry-animated" style="margin-bottom:
1.5rem;">
                            <div class="journal-header"
                                style="background: linear-gradient(135deg, #166534 0%,
#22c55e 100%);">
                                <h4><i class="fas fa-book"></i> ASIENTO 1: Reconocimiento
del Ingreso</h4>
                                <span>Asiento N° 46 • 22/12/2025</span>
                            </div>
                            <div class="journal-row header">
                                <div>Cuenta</div>
                                <div style="text-align: right;">Debe</div>
                                <div style="text-align: right;">Haber</div>
                            </div>
                            <div class="journal-row animate" style="animation-delay:
0.2s;">
                                <div class="account"><i class="fas fa-users" style="color:
#3b82f6;"></i> Clientes por
                                    Cobrar</div>
                                <div class="debit">$4.462.500</div>
                                <div></div>
                            </div>
                            <div class="journal-row animate" style="animation-delay:
0.4s;">
                                <div class="account"><i class="fas fa-chart-line"
style="color: #22c55e;"></i> Ingresos
                                    por Ventas</div>
                                <div></div>
                                <div class="credit">$3.750.000</div>
                            </div>
                            <div class="journal-row animate" style="animation-delay:
0.6s;">
                                <div class="account"><i class="fas fa-receipt"
style="color: #ef4444;"></i> IVA Débito
                                    Fiscal</div>
                                <div></div>
                                <div class="credit">$712.500</div>
                            </div>
                            <div class="journal-totals">
                                <div style="font-style: italic; color: var(--gray-
500);">TOTALES</div>
                                <div class="debit" style="text-align:
right;">$4.462.500</div>
                                <div class="credit" style="text-align:
right;">$4.462.500</div>
                            </div>
                        </div>
                        <div class="journal-entry-animated">
                            <div class="journal-header"
                                style="background: linear-gradient(135deg, #dc2626 0%,
#f97316 100%);">
                                <h4><i class="fas fa-book"></i> ASIENTO 2: Reconocimiento
del Costo</h4>
                                <span>Asiento N° 47 • 22/12/2025</span>
                            </div>
                            <div class="journal-row header">
```

```
                                <div>Cuenta</div>
                                <div style="text-align: right;">Debe</div>
                                <div style="text-align: right;">Haber</div>
                            </div>
                            <div class="journal-row animate" style="animation-delay:
0.2s;">
                                <div class="account"><i class="fas fa-box-open"
style="color: #f97316;"></i> Costo de
                                        Ventas</div>
                                <div class="debit">$2.500.000</div>
                                <div></div>
                            </div>
                            <div class="journal-row animate" style="animation-delay:
0.4s;">
                                <div class="account"><i class="fas fa-box" style="color:
#3b82f6;"></i> Mercaderías
                                </div>
                                <div></div>
                                <div class="credit">$2.500.000</div>
                            </div>
                            <div class="journal-totals">
                                <div style="font-style: italic; color: var(--gray-
500);">TOTALES</div>
                                <div class="debit" style="text-align:
right;">$2.500.000</div>
                                <div class="credit" style="text-align:
right;">$2.500.000</div>
                            </div>
                        </div>
                        <div class="analysis-box"
                            style="background: linear-gradient(135deg, #f3e8ff 0%, #e9d5ff
100%); border-color: var(--purple); margin-top: 1.5rem;">
                            <h4 style="color: #7c3aed;"><i class="fas fa-info-circle"></i>
¿Por qué dos asientos?</h4>
                            <ul style="list-style: none; padding: 0; margin: 0;">
                                <li style="padding: 0.4rem; font-size: 0.9rem;">
                                    <strong>Asiento 1:</strong> Registra el INGRESO por la
venta y genera el derecho a
                                    cobrar al cliente
                                </li>
                                <li style="padding: 0.4rem; font-size: 0.9rem;">
                                    <strong>Asiento 2:</strong> Registra el COSTO de la
mercadería vendida y rebaja el
                                    inventario
                                </li>
                                <li style="padding: 0.4rem; font-size: 0.9rem; font-
weight: 600; color: #166534;">
                                    La diferencia entre el Ingreso ($3.750.000) y el Costo
($2.500.000) es la
                                    <strong>Utilidad Bruta: $1.250.000</strong>
                                </li>
                            </ul>
                        </div>
                        <div class="sim-nav">
                            <button class="sim-btn" onclick="prevSimStep()"><i class="fas
fa-arrow-left"></i>
                                Anterior</button>
                            <button class="sim-btn primary"
onclick="nextSimStep()">Siguiente <i
                                    class="fas fa-arrow-right"></i></button>
                        </div>
                    </div>
                    <div id="sim-step-7" class="sim-step">
                        <h4 style="margin-bottom: 1rem; display: flex; align-items:
center; gap: 0.5rem;">
                            <i class="fas fa-columns" style="color: var(--secondary);">
</i>
                            Paso 7: Libro Mayor Actualizado (Post-Venta)
                        </h4>
                        <p style="margin-bottom: 1.5rem; color: var(--gray-700);">
                            Después de la compra y la venta, el Libro Mayor muestra los
saldos actualizados de todas las
                            cuentas afectadas.
                        </p>
                        <div class="ledger-accounts">
                            <div class="ledger-t">
                                <div class="ledger-t-header asset">
                                    <i class="fas fa-box"></i> MERCADERÍAS
                                </div>
                                <div class="ledger-t-body">
                                    <div class="ledger-t-side debit">
```

```
                                        <h6>DEBE</h6>
                                        <div class="amount">$5.000.000</div>
                                        <div style="font-size: 0.7rem; color: var(--gray-
500);">(Compra)</div>
                                    </div>
                                    <div class="ledger-t-side credit">
                                        <h6>HABER</h6>
                                        <div class="amount">$2.500.000</div>
                                        <div style="font-size: 0.7rem; color: var(--gray-
500);">(Venta)</div>
                                    </div>
                                </div>
                                <div class="ledger-t-balance">Saldo Deudor:
$2.500.000</div>
                            </div>
                            <div class="ledger-t">
                                <div class="ledger-t-header asset">
                                    <i class="fas fa-users"></i> CLIENTES POR COBRAR
                                </div>
                                <div class="ledger-t-body">
                                    <div class="ledger-t-side debit">
                                        <h6>DEBE</h6>
                                        <div class="amount">$4.462.500</div>
                                        <div style="font-size: 0.7rem; color: var(--gray-
500);">(Venta)</div>
                                    </div>
                                    <div class="ledger-t-side credit">
                                        <h6>HABER</h6>
                                        <div class="amount">-</div>
                                    </div>
                                </div>
                                <div class="ledger-t-balance">Saldo Deudor:
$4.462.500</div>
                            </div>
                            <div class="ledger-t">
                                <div class="ledger-t-header revenue">
                                    <i class="fas fa-chart-line"></i> INGRESOS POR VENTAS
                                </div>
                                <div class="ledger-t-body">
                                    <div class="ledger-t-side debit">
                                        <h6>DEBE</h6>
                                        <div class="amount">-</div>
                                    </div>
                                    <div class="ledger-t-side credit">
                                        <h6>HABER</h6>
                                        <div class="amount">$3.750.000</div>
                                        <div style="font-size: 0.7rem; color: var(--gray-
500);">(Venta)</div>
                                    </div>
                                </div>
                                <div class="ledger-t-balance" style="background: #dcfce7;
color: #166534;">Saldo
                                    Acreedor: $3.750.000</div>
                            </div>
                            <div class="ledger-t">
                                <div class="ledger-t-header" style="background: #f97316;">
                                    <i class="fas fa-box-open"></i> COSTO DE VENTAS
                                </div>
                                <div class="ledger-t-body">
                                    <div class="ledger-t-side debit">
                                        <h6>DEBE</h6>
                                        <div class="amount">$2.500.000</div>
                                        <div style="font-size: 0.7rem; color: var(--gray-
500);">(Venta)</div>
                                    </div>
                                    <div class="ledger-t-side credit">
                                        <h6>HABER</h6>
                                        <div class="amount">-</div>
                                    </div>
                                </div>
                                <div class="ledger-t-balance" style="background: #fef3c7;
color: #b45309;">Saldo Deudor:
                                    $2.500.000</div>
                            </div>
                            <div class="ledger-t">
                                <div class="ledger-t-header liability">
                                    <i class="fas fa-receipt"></i> IVA DÉBITO FISCAL
                                </div>
                                <div class="ledger-t-body">
                                    <div class="ledger-t-side debit">
                                        <h6>DEBE</h6>
```

```
                                        <div class="amount">-</div>
                                    </div>
                                    <div class="ledger-t-side credit">
                                        <h6>HABER</h6>
                                        <div class="amount">$712.500</div>
                                        <div style="font-size: 0.7rem; color: var(--gray-
500);">(Venta)</div>
                                    </div>
                                </div>
                                <div class="ledger-t-balance">Saldo Acreedor:
$712.500</div>
                            </div>
                            <div class="ledger-t">
                                <div class="ledger-t-header liability">
                                    <i class="fas fa-handshake"></i> PROVEEDORES
                                </div>
                                <div class="ledger-t-body">
                                    <div class="ledger-t-side debit">
                                        <h6>DEBE</h6>
                                        <div class="amount">-</div>
                                    </div>
                                    <div class="ledger-t-side credit">
                                        <h6>HABER</h6>
                                        <div class="amount">$5.950.000</div>
                                        <div style="font-size: 0.7rem; color: var(--gray-
500);">(Compra)</div>
                                    </div>
                                </div>
                                <div class="ledger-t-balance">Saldo Acreedor:
$5.950.000</div>
                            </div>
                        </div>
                        <div class="flow-example">
                            <h5><i class="fas fa-balance-scale"></i> Verificación:
Ecuación Contable Actualizada</h5>
                            <p>Después de la compra y la venta, la ecuación sigue
equilibrada:</p>
                            <div
                                style="display: flex; align-items: center; justify-
content: center; flex-wrap: wrap; gap: 0.5rem; margin: 1.5rem 0; padding: 1rem;
background: white; border-radius: 0.75rem;">
                                <div style="text-align: center; min-width: 140px;">
                                    <div
                                        style="background: #dbeafe; padding: 0.75rem
1.25rem; border-radius: 0.5rem; border: 2px solid #3b82f6;">
                                        <div
                                            style="font-size: 0.75rem; color: #1e40af;
font-weight: 600; text-transform: uppercase;">
                                            Activo</div>
                                        <div style="font-size: 1.25rem; font-weight: 700;
color: #1e40af;">$17.912.500
                                        </div>
                                    </div>
                                </div>
                                <div style="font-size: 2rem; font-weight: 700; color:
#374151; padding: 0 0.5rem;">=
                                </div>
                                <div style="text-align: center; min-width: 120px;">
                                    <div
                                        style="background: #fecaca; padding: 0.75rem
1.25rem; border-radius: 0.5rem; border: 2px solid #ef4444;">
                                        <div
                                            style="font-size: 0.75rem; color: #991b1b;
font-weight: 600; text-transform: uppercase;">
                                            Pasivo</div>
                                        <div style="font-size: 1.25rem; font-weight: 700;
color: #dc2626;">$6.662.500
                                        </div>
                                    </div>
                                </div>
                                <div style="font-size: 2rem; font-weight: 700; color:
#374151; padding: 0 0.5rem;">+
                                </div>
                                <div style="text-align: center; min-width: 130px;">
                                    <div
                                        style="background: #e9d5ff; padding: 0.75rem
1.25rem; border-radius: 0.5rem; border: 2px solid #8b5cf6;">
                                        <div
                                            style="font-size: 0.75rem; color: #5b21b6;
font-weight: 600; text-transform: uppercase;">
                                            Patrimonio</div>
```

```
                                        <div style="font-size: 1.25rem; font-weight: 700;
color: #7c3aed;">$11.250.000
                                        </div>
                                    </div>
                                </div>
                            </div>
                            <div
                                style="text-align: center; background: #f0fdf4; border:
2px solid #22c55e; border-radius: 0.5rem; padding: 0.75rem; margin-top: 0.5rem;">
                                <div
                                    style="display: flex; align-items: center; justify-
content: center; gap: 1rem; flex-wrap: wrap;">
                                    <span style="font-size: 0.9rem; color: #166534;">
                                        <strong>$17.912.500</strong> =
<strong>$6.662.500</strong> +
                                        <strong>$11.250.000</strong>
                                    </span>
                                    <span
                                        style="background: #22c55e; color: white; padding:
0.25rem 0.75rem; border-radius: 1rem; font-size: 0.8rem; font-weight: 600;">
                                        <i class="fas fa-check"></i> ¡CUADRADO!
                                    </span>
                                </div>
                            </div>
                            <p
                                style="text-align: center; margin-top: 1rem; font-size:
0.85rem; color: var(--gray-500);">
                                <i class="fas fa-info-circle"></i> El Patrimonio aumentó
en <strong
                                    style="color: #16a34a;">$1.250.000</strong> (la
Utilidad Bruta de la venta)
                            </p>
                        </div>
                        <div class="sim-nav">
                            <button class="sim-btn" onclick="prevSimStep()"><i class="fas
fa-arrow-left"></i>
                                Anterior</button>
                            <button class="sim-btn primary"
onclick="nextSimStep()">Siguiente <i
                                    class="fas fa-arrow-right"></i></button>
                        </div>
                    </div>
                    <div id="sim-step-8" class="sim-step">
                        <h4 style="margin-bottom: 1rem; display: flex; align-items:
center; gap: 0.5rem;">
                            <i class="fas fa-chart-pie" style="color: var(--primary);">
</i>
                            Paso 8: Estados Financieros con Impacto de la Venta
                        </h4>
                        <p style="margin-bottom: 1.5rem; color: var(--gray-700);">
                            Ahora los Estados Financieros muestran el <strong>impacto
completo</strong>: el Balance
                            General
                            y el <strong>Estado de Resultados</strong> reflejan la
utilidad generada por la venta.
                        </p>
                        <div class="fs-container">
                            <div class="fs-statement">
                                <div class="fs-header balance">
                                    <h4><i class="fas fa-balance-scale"></i> Estado de
Situación Financiera</h4>
                                    <p>Balance General al 31/12/2025</p>
                                </div>
                                <div class="fs-body">
                                    <div class="fs-section">
                                        <h5>Activos</h5>
                                        <div class="fs-row">
                                            <span>Caja</span>
                                            <span>$2.000.000</span>
                                        </div>
                                        <div class="fs-row">
                                            <span>Banco</span>
                                            <span>$8.000.000</span>
                                        </div>
                                        <div class="fs-row highlight">
                                            <span>Clientes por Cobrar</span>
                                            <span class="positive">$4.462.500 ↑</span>
                                        </div>
                                        <div class="fs-row">
                                            <span>Mercaderías</span>
                                            <span>$2.500.000</span>
```

```
                                        </div>
                                        <div class="fs-row">
                                            <span>IVA Crédito Fiscal</span>
                                            <span>$950.000</span>
                                        </div>
                                        <div class="fs-row total">
                                            <span>TOTAL ACTIVOS</span>
                                            <span>$17.912.500</span>
                                        </div>
                                    </div>
                                    <div class="fs-section">
                                        <h5>Pasivos</h5>
                                        <div class="fs-row">
                                            <span>Proveedores</span>
                                            <span>$5.950.000</span>
                                        </div>
                                        <div class="fs-row highlight">
                                            <span>IVA Débito Fiscal</span>
                                            <span class="negative">$712.500 ↑</span>
                                        </div>
                                        <div class="fs-row total">
                                            <span>TOTAL PASIVOS</span>
                                            <span>$6.662.500</span>
                                        </div>
                                    </div>
                                    <div class="fs-section">
                                        <h5>Patrimonio</h5>
                                        <div class="fs-row">
                                            <span>Capital</span>
                                            <span>$10.000.000</span>
                                        </div>
                                        <div class="fs-row highlight">
                                            <span>Resultado del Ejercicio</span>
                                            <span class="positive">$1.250.000 ↑</span>
                                        </div>
                                        <div class="fs-row total">
                                            <span>TOTAL PATRIMONIO</span>
                                            <span>$11.250.000</span>
                                        </div>
                                    </div>
                                    <div class="fs-section"
                                        style="background: linear-gradient(135deg, #dbeafe
0%, #bfdbfe 100%); margin: 0 -1rem -1rem -1rem; padding: 1rem; border-radius: 0 0 0.5rem
0.5rem;">
                                        <div class="fs-row total"
                                            style="font-size: 1.1rem; border-top: none;
margin-top: 0; padding-top: 0;">
                                            <span style="color: #1e40af;"><strong>TOTAL
PASIVO +
                                                    PATRIMONIO</strong></span>
                                            <span style="color: #1e40af;">
<strong>$17.912.500</strong></span>
                                        </div>
                                        <div style="text-align: center; margin-top:
0.5rem;">
                                            <span
                                                style="background: #22c55e; color: white;
padding: 0.25rem 0.75rem; border-radius: 1rem; font-size: 0.75rem; font-weight: 600;">
                                                <i class="fas fa-check"></i> = TOTAL
ACTIVOS ✓
                                            </span>
                                        </div>
                                    </div>
                                </div>
                            </div>
                            <div class="fs-statement">
                                <div class="fs-header results">
                                    <h4><i class="fas fa-chart-line"></i> Estado de
Resultados</h4>
                                    <p>Del 01/01/2025 al 31/12/2025</p>
                                </div>
                                <div class="fs-body">
                                    <div class="fs-section">
                                        <h5>Ingresos Operacionales</h5>
                                        <div class="fs-row highlight">
                                            <span>Ventas</span>
                                            <span class="positive">$3.750.000 ↑</span>
                                        </div>
                                        <div class="fs-row total">
                                            <span>TOTAL INGRESOS</span>
                                            <span>$3.750.000</span>
```

```
                                        </div>
                                    </div>
                                    <div class="fs-section">
                                        <h5>Costos de Venta</h5>
                                        <div class="fs-row highlight">
                                            <span>Costo de Mercaderías Vendidas</span>
                                            <span class="negative">($2.500.000) ↑</span>
                                        </div>
                                        <div class="fs-row total">
                                            <span>COSTO DE VENTAS</span>
                                            <span>($2.500.000)</span>
                                        </div>
                                    </div>
                                    <div class="fs-section"
                                        style="background: linear-gradient(135deg, #dcfce7
0%, #bbf7d0 100%); padding: 1rem; border-radius: 0.5rem; margin: -1rem; margin-top:
0.5rem;">
                                        <h5 style="color: #166534; border-bottom-color:
#22c55e;">Resultado Operacional
                                        </h5>
                                        <div class="fs-row total" style="font-size:
1.2rem;">
                                            <span style="color: #166534;">
<strong>RESULTADO OPERACIONAL (Margen
                                                    Bruto)</strong></span>
                                            <span style="color: #166534;">
<strong>$1.250.000</strong></span>
                                        </div>
                                    </div>
                                    <div class="fs-section" style="margin-top: 0.5rem;">
                                        <h5>Otros Ingresos y Gastos</h5>
                                        <div class="fs-row">
                                            <span>Otros Ingresos</span>
                                            <span style="color: #059669;">$200.000</span>
                                        </div>
                                        <div class="fs-row">
                                            <span>Gastos de Administración</span>
                                            <span style="color: #dc2626;">($800.000)
</span>
                                        </div>
                                        <div class="fs-row">
                                            <span>Gastos de Ventas</span>
                                            <span style="color: #dc2626;">($300.000)
</span>
                                        </div>
                                        <div class="fs-row">
                                            <span>Gastos Financieros</span>
                                            <span style="color: #dc2626;">($100.000)
</span>
                                        </div>
                                    </div>
                                    <div class="fs-section"
                                        style="background: linear-gradient(135deg,
#d1fae5, #f0fdf4); padding: 1rem; border-radius: 0.5rem; margin-top: 0.5rem; border: 2px
solid #16a34a;">
                                        <h5 style="color: #16a34a; border-bottom-color:
#22c55e;">Resultado del
                                            Ejercicio</h5>
                                        <div class="fs-row total" style="font-size:
1.25rem; ">
                                            <span style="color: #16a34a;">
<strong>RESULTADO NETO DEL EJERCICIO
                                                    (Ganancia)</strong></span>
                                            <span style="color: #16a34a;">
<strong>$250.000</strong></span>
                                        </div>
                                        <p
                                            style="margin: 0.5rem 0 0; font-size: 0.85rem;
color: #15803d; text-align: center;">
                                            <i class="fas fa-info-circle"></i> Ingresos -
Costos - Gastos = Ganancia
                                        </p>
                                    </div>
                                </div>
                            </div>
                        </div>
                        <div class="analysis-box"
                            style="background: linear-gradient(135deg, #dcfce7 0%, #bbf7d0
100%); border-color: var(--secondary); margin-top: 1.5rem;">
                            <h4 style="color: #166534;"><i class="fas fa-graduation-cap">
</i> ¡Ciclo Contable Completo!
```

```
                            </h4>
                            <p>Has seguido el ciclo completo de Compra-Venta y su impacto
en los Estados Financieros:
                            </p>
                            <div style="display: flex; flex-wrap: wrap; gap: 0.5rem;
margin-top: 1rem;">
                                <span
                                    style="background: white; padding: 0.5rem 1rem;
border-radius: 2rem; font-size: 0.85rem;">
                                    <i class="fas fa-check" style="color: #16a34a;"></i>
Factura de Compra
                                </span>
                                <span
                                    style="background: white; padding: 0.5rem 1rem;
border-radius: 2rem; font-size: 0.85rem;">
                                    <i class="fas fa-check" style="color: #16a34a;"></i>
Asiento de Compra
                                </span>
                                <span
                                    style="background: white; padding: 0.5rem 1rem;
border-radius: 2rem; font-size: 0.85rem;">
                                    <i class="fas fa-check" style="color: #16a34a;"></i>
Libro Mayor (Compra)
                                </span>
                                <span
                                    style="background: white; padding: 0.5rem 1rem;
border-radius: 2rem; font-size: 0.85rem;">
                                    <i class="fas fa-check" style="color: #16a34a;"></i>
Factura de Venta
                                </span>
                                <span
                                    style="background: white; padding: 0.5rem 1rem;
border-radius: 2rem; font-size: 0.85rem;">
                                    <i class="fas fa-check" style="color: #16a34a;"></i>
Asientos de Venta
                                </span>
                                <span
                                    style="background: white; padding: 0.5rem 1rem;
border-radius: 2rem; font-size: 0.85rem;">
                                    <i class="fas fa-check" style="color: #16a34a;"></i>
Libro Mayor Final
                                </span>
                                <span
                                    style="background: #fef3c7; padding: 0.5rem 1rem;
border-radius: 2rem; font-size: 0.85rem; font-weight: 600;">
                                    <i class="fas fa-star" style="color: #f59e0b;"></i>
Estados Financieros con Utilidad
                                </span>
                            </div>
                            <div style="margin-top: 1.5rem; padding: 1rem; background:
white; border-radius: 0.5rem;">
                                <strong><i class="fas fa-key" style="color: #f59e0b;"></i>
Concepto Clave:</strong>
                                <p style="margin: 0.5rem 0 0 0; font-size: 0.9rem;">
                                    El <strong>Estado de Resultados</strong> muestra las
operaciones que generan
                                    utilidades o pérdidas
                                    (Ingresos - Costos - Gastos). La compra de mercaderías
NO afecta los resultados
                                    hasta que se VENDEN,
                                    momento en que se reconoce tanto el ingreso como el
costo correspondiente.
                                </p>
                            </div>
                        </div>
                        <div class="sim-nav">
                            <button class="sim-btn" onclick="prevSimStep()"><i class="fas
fa-arrow-left"></i>
                                Anterior</button>
                            <button class="sim-btn primary" onclick="goToSimStep(1)"><i
class="fas fa-redo"></i>
                                Reiniciar Simulación</button>
                        </div>
                    </div>
                </div>
            </div>
            <div style="display: flex; justify-content: center; gap: 1rem; flex-wrap:
wrap; margin-top: 2rem;">
                <button class="link-btn" onclick="goToModule('compras', 'facturas-
proveedor')">
                    <i class="fas fa-external-link-alt"></i> Facturas de Compra
```

```
                </button>
                <button class="link-btn success" onclick="goToModule('contabilidad',
'asientos')">
                    <i class="fas fa-external-link-alt"></i> Libro Diario
                </button>
                <button class="link-btn" style="background: var(--secondary);"
                    onclick="goToModule('reportes', 'balance-comprobacion')">
                    <i class="fas fa-external-link-alt"></i> Balance de Comprobación
                </button>
                <button class="link-btn" style="background: var(--purple);"
                    onclick="goToModule('reportes', 'balance-general')">
                    <i class="fas fa-external-link-alt"></i> Balance General
                </button>
            </div>
        </div>
    </div>
    <div id="tab-ratios" class="tab-content">
        <div class="description-section" style="background: white;">
            <div class="description-header" style="background: var(--primary);">
                <h2 style="color: white;"><i class="fas fa-chart-bar"></i> Ratios
Financieros</h2>
            </div>
            <div class="description-body" style="color: #333;">
                <p style="font-size: 1.1rem; margin-bottom: 2rem; color: #333;">
                    Los <strong>ratios financieros</strong> son indicadores numéricos
calculados a partir de los estados
                    financieros.
                </p>
                <div
                    style="margin-bottom: 2rem; padding: 1.5rem; background: linear-
gradient(135deg, #e0f2fe, #f0f9ff); border-radius: 1rem; border-left: 5px solid #06b6d4;">
                    <h3 style="margin: 0 0 1rem; color: #0891b2;"><i class="fas fa-tint">
</i> Ratios de Liquidez</h3>
                    <p style="margin-bottom: 1rem; color: #333;">Miden la capacidad de
pago a corto plazo.</p>
                    <div style="display: grid; gap: 0.75rem;">
                        <div style="background: white; padding: 1rem; border-radius:
0.5rem; color: #333;">
                            <strong>Razón Corriente</strong> = Activo Corriente / Pasivo
Corriente
                        </div>
                        <div style="background: white; padding: 1rem; border-radius:
0.5rem; color: #333;">
                            <strong>Prueba Ácida</strong> = (Activo Corriente -
Inventario) / Pasivo Corriente
                        </div>
                    </div>
                </div>
                <div
                    style="margin-bottom: 2rem; padding: 1.5rem; background: linear-
gradient(135deg, #dcfce7, #f0fdf4); border-radius: 1rem; border-left: 5px solid #10b981;">
                    <h3 style="margin: 0 0 1rem; color: #059669;"><i class="fas fa-chart-
line"></i> Ratios de
                        Rentabilidad</h3>
                    <p style="margin-bottom: 1rem; color: #333;">Evalúan la capacidad de
generar utilidades.</p>
                    <div style="display: grid; gap: 0.75rem;">
                        <div style="background: white; padding: 1rem; border-radius:
0.5rem; color: #333;">
                            <strong>ROA</strong> = Resultado del Ejercicio / Total Activos
× 100
                        </div>
                        <div style="background: white; padding: 1rem; border-radius:
0.5rem; color: #333;">
                            <strong>ROE</strong> = Resultado del Ejercicio / Patrimonio ×
100
                        </div>
                        <div style="background: white; padding: 1rem; border-radius:
0.5rem; color: #333;">
                            <strong>Margen Neto</strong> = Resultado del Ejercicio /
Ventas × 100
                        </div>
                    </div>
                </div>
                <div
                    style="margin-bottom: 2rem; padding: 1.5rem; background: linear-
gradient(135deg, #ede9fe, #f5f3ff); border-radius: 1rem; border-left: 5px solid #8b5cf6;">
                    <h3 style="margin: 0 0 1rem; color: #7c3aed;"><i class="fas fa-shield-
alt"></i> Ratios de Solvencia
                    </h3>
                    <p style="margin-bottom: 1rem; color: #333;">Miden la capacidad de
```

```
cumplir obligaciones de largo
                        plazo.</p>
                    <div style="display: grid; gap: 0.75rem;">
                        <div style="background: white; padding: 1rem; border-radius:
0.5rem; color: #333;">
                            <strong>Endeudamiento</strong> = Total Pasivos / Total Activos
× 100
                        </div>
                        <div style="background: white; padding: 1rem; border-radius:
0.5rem; color: #333;">
                            <strong>Autonomía</strong> = Patrimonio / Total Activos × 100
                        </div>
                    </div>
                </div>
                <button class="link-btn" style="background: #f59e0b;"
                    onclick="goToModule('reportes', 'ratios-financieros')">
                    <i class="fas fa-external-link-alt"></i> Ver Ratios en el Sistema
                </button>
            </div>
        </div>
    </div>
    <div id="tab-sistemas" class="tab-content">
        <div class="description-section" style="background: white;">
            <div class="description-header" style="background: var(--primary);">
                <h2 style="color: white;"><i class="fas fa-book"></i> Guía: Sistemas de
Contabilización</h2>
            </div>
            <div class="description-body" style="color: #333;">
                <p style="font-size: 1.1rem; margin-bottom: 2rem; color: #333;">
                    EDU-TRACE ERP soporta <strong>dos sistemas de contabilización</strong>
que reflejan métodos
                    utilizados en la práctica contable real:
                </p>
                <div
                    style="margin-bottom: 2rem; padding: 2rem; background: linear-
gradient(135deg, #dbeafe, #eff6ff); border-radius: 1rem; border: 3px solid #3b82f6;">
                    <h3 style="margin: 0 0 1rem; color: #1e40af;"><i class="fas fa-book">
</i> 📘 Sistema Jornalizador
                        (Diario Único)</h3>
                    <p style="margin-bottom: 1rem; color: #333;">El sistema jornalizador
registra <strong>todas las
                            transacciones directamente</strong> en el Libro Diario, sin
importar su origen o módulo.</p>
                    <h4 style="color: #1e40af; margin: 1rem 0 0.5rem;">Características:
</h4>
                    <ul style="margin: 0 0 1rem; padding-left: 1.5rem; color: #333;">
                        <li>✅ Registro inmediato de cada operación</li>
                        <li>✅ Visualización directa en el Libro Diario</li>
                        <li>✅ Ideal para empresas pequeñas</li>
                        <li>✅ Más simple de entender para principiantes</li>
                    </ul>
                    <div style="background: white; padding: 1rem; border-radius: 0.5rem;
margin-top: 1rem;">
                        <h5 style="margin: 0 0 0.5rem; color: #1e40af;"><i class="fas fa-
receipt"></i> Ejemplo Práctico
                            - Venta al contado:</h5>
                        <div
                            style="background: #f8fafc; padding: 1rem; border-radius:
0.5rem; font-family: monospace; font-size: 0.9rem; color: #333;">
                            <strong>Libro Diario - Asiento #001</strong><br>
                            Fecha: 15/03/2024<br>
                            ----------------<br>
                            Caja

             $119.0
00
                            (D)<br>
                                Ventas

             $100.0
00
                            (C)<br>
                                IVA Débito

          $19.000 (C)
                        </div>
                    </div>
                </div>
                <div
                    style="margin-bottom: 2rem; padding: 2rem; background: linear-
```

```
gradient(135deg, #dcfce7, #f0fdf4); border-radius: 1rem; border: 3px solid #10b981;">
                    <h3 style="margin: 0 0 1rem; color: #065f46;"><i class="fas fa-layer-
group"></i> 📗 Sistema
                        Centralizador (Libros Auxiliares)</h3>
                    <p style="margin-bottom: 1rem; color: #333;">El sistema centralizador
registra las transacciones en
                        <strong>libros auxiliares por módulo</strong> y luego las
consolida mensualmente en el Libro
                        Diario.
                    </p>
                    <h4 style="color: #065f46; margin: 1rem 0 0.5rem;">Características:
</h4>
                    <ul style="margin: 0 0 1rem; padding-left: 1.5rem; color: #333;">
                        <li>✅ Organización por módulos (Ventas, Compras, Tesorería)</li>
                        <li>✅ Centralización mensual</li>
                        <li>✅ Refleja práctica empresarial real</li>
                        <li>✅ Reduce volumen en Libro Diario</li>
                    </ul>
                    <div style="background: white; padding: 1rem; border-radius: 0.5rem;
margin-bottom: 1rem;">
                        <h5 style="margin: 0 0 0.5rem; color: #065f46;"><i class="fas fa-
sitemap"></i> Flujo de Trabajo:
                        </h5>
                        <div
                            style="background: #f8fafc; padding: 1rem; border-radius:
0.5rem; font-family: monospace; font-size: 0.85rem; color: #333;">
                            <strong>1. REGISTRO DIARIO</strong><br>
                              ├─ Módulo Ventas → Libro Auxiliar de Ventas<br>
                              ├─ Módulo Compras → Libro Auxiliar de Compras<br>
                              └─ Módulo Tesorería → Libro Auxiliar de
Tesorería<br><br>
                            <strong>2. FIN DE MES</strong><br>
                              └─ CENTRALIZACIÓN<br>
                                  ├─ Consolida totales por
cuenta<br>
                                  └─ Genera 1 asiento
resumen por módulo<br><br>
                            <strong>3. LIBRO DIARIO</strong><br>
                              └─ Solo contiene asientos centralizados
                        </div>
                    </div>
                    <div style="background: white; padding: 1rem; border-radius: 0.5rem;">
                        <h5 style="margin: 0 0 0.5rem; color: #065f46;"><i class="fas fa-
file-invoice"></i> Ejemplo:
                            Centralización Marzo 2024</h5>
                        <div
                            style="background: #f8fafc; padding: 1rem; border-radius:
0.5rem; font-family: monospace; font-size: 0.85rem; color: #333;">
                            <strong>Asiento Resumen - Ventas Marzo 2024</strong><br>
                            ————————————————————<br>
                            Caja
           $1.500.000
                            (D)<br>
                            Banco
           $2.300.000 (D)<br>
                                Ventas

            $3.200.000
                            (C)<br>
                                IVA Débito

          $608.000 (C)
                        </div>
                    </div>
                </div>
                <div
                    style="margin-bottom: 2rem; padding: 1.5rem; background: #fefce8;
border-radius: 1rem; border: 2px solid #eab308;">
                    <h4 style="margin: 0 0 1rem; color: #a16207;"><i class="fas fa-
lightbulb"></i> 💡 ¿Cuándo usar cada
                        sistema?</h4>
                    <div style="display: grid; grid-template-columns: repeat(auto-fit,
minmax(250px, 1fr)); gap: 1rem;">
                        <div
                            style="background: white; padding: 1rem; border-radius:
0.5rem; border-left: 4px solid #3b82f6;">
                            <h5 style="margin: 0 0 0.5rem; color: #1e40af;">Usa
Jornalizador si:</h5>
                            <ul style="margin: 0; padding-left: 1.25rem; font-size:
```

```
0.9rem; color: #333;">
                                <li>Estás comenzando con contabilidad</li>
                                <li>Menos de 50 transacciones/mes</li>
                                <li>Quieres simpleza y visión directa</li>
                            </ul>
                        </div>
                        <div
                            style="background: white; padding: 1rem; border-radius:
0.5rem; border-left: 4px solid #10b981;">
                            <h5 style="margin: 0 0 0.5rem; color: #065f46;">Usa
Centralizador si:</h5>
                            <ul style="margin: 0; padding-left: 1.25rem; font-size:
0.9rem; color: #333;">
                                <li>Ya dominas el Jornalizador</li>
                                <li>Más de 50 transacciones/mes</li>
                                <li>Quieres aprender práctica real</li>
                            </ul>
                        </div>
                    </div>
                </div>
                <div style="margin-bottom: 2rem; overflow-x: auto;">
                    <h4 style="color: #1e40af; margin-bottom: 1rem;"><i class="fas fa-
table"></i> 📊 Comparación</h4>
                    <table style="width: 100%; border-collapse: collapse; font-size:
0.9rem;">
                        <thead>
                            <tr style="background: var(--primary); color: white;">
                                <th style="padding: 0.75rem; text-align: left; border: 1px
solid #ddd;">Aspecto</th>
                                <th style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">Jornalizador
                                </th>
                                <th style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">Centralizador
                                </th>
                            </tr>
                        </thead>
                        <tbody style="color: #333;">
                            <tr style="background: white;">
                                <td style="padding: 0.75rem; border: 1px solid #ddd;">
<strong>Asientos/mes</strong></td>
                                <td style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">1 por
                                    transacción</td>
                                <td style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">3-5
                                    consolidados</td>
                            </tr>
                            <tr style="background: #f9fafb;">
                                <td style="padding: 0.75rem; border: 1px solid #ddd;">
<strong>Libro Diario</strong></td>
                                <td style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">Detallado</td>
                                <td style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">Resumido</td>
                            </tr>
                            <tr style="background: white;">
                                <td style="padding: 0.75rem; border: 1px solid #ddd;">
<strong>Complejidad</strong></td>
                                <td style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">⭐ Baja</td>
                                <td style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">⭐⭐⭐ Media</td>
                            </tr>
                            <tr style="background: #f9fafb;">
                                <td style="padding: 0.75rem; border: 1px solid #ddd;">
<strong>Nivel</strong></td>
                                <td style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">Principiante
                                </td>
                                <td style="padding: 0.75rem; text-align: center; border:
1px solid #ddd;">
                                    Intermedio/Avanzado</td>
                            </tr>
                        </tbody>
                    </table>
                </div>
                <div
                    style="padding: 1.5rem; background: #eff6ff; border-left: 4px solid
#3b82f6; border-radius: 0.5rem; margin-bottom: 1rem;">
```

```
                    <h4 style="margin: 0 0 0.5rem; color: #1e40af;"><i class="fas fa-cog">
</i> 🔧 ¿Cómo configurar en
                        EDU-TRACE ERP?</h4>
                    <ol style="margin: 0.5rem 0 0; padding-left: 1.5rem; color: #333;">
                        <li>Ve a <strong>Administración</strong> →
<strong>Configuración</strong></li>
                        <li>En "Parámetros Contables"</li>
                        <li>Selecciona "Sistema de Contabilización":
                            <ul style="margin: 0.25rem 0; padding-left: 1rem;">
                                <li>🔵 <strong>Jornalizador</strong> (Diario Único)</li>
                                <li>🟢 <strong>Centralizador</strong> (Libros Auxiliares)
</li>
                            </ul>
                        </li>
                        <li>Guarda los cambios</li>
                    </ol>
                </div>
                <button class="link-btn" style="background: #3b82f6;"
onclick="goToModule('admin', 'configuracion')">
                    <i class="fas fa-cog"></i> Ir a Configuración
                </button>
            </div>
        </div>
    </div>
    <div id="tab-flujos" class="tab-content">
        <div class="description-section" style="background: white;">
            <div class="description-header" style="background: var(--purple);">
                <h2 style="color: white;"><i class="fas fa-exchange-alt"></i> Flujos de
Trabajo y Ciclos Operativos
                </h2>
            </div>
            <div class="description-body" style="color: #333; padding: 2rem;">
                <p style="font-size: 1.1rem; margin-bottom: 2.5rem; color: #333; text-
align: center;">
                    Comprender la diferencia entre <strong>Gestión Operativa</strong> y
                    <strong>Contabilización</strong> es clave.
                    No todos los documentos generan asientos contables.
                </p>
                <div style="margin-bottom: 3.5rem;">
                    <h3
                        style="color: #1e40af; border-bottom: 2px solid #bfdbfe; padding-
bottom: 0.5rem; margin-bottom: 1.5rem;">
                        <i class="fas fa-shopping-cart"></i> 1. Ciclo de Compras
(Adquisiciones)
                    </h3>
                    <div
                        style="display: flex; flex-direction: column; gap: 1rem; align-
items: center; margin-bottom: 2rem;">
                        <div
                            style="display: flex; align-items: center; gap: 1rem; flex-
wrap: wrap; justify-content: center;">
                            <div
                                style="background: #f8fafc; border: 2px solid #cbd5e1;
padding: 1rem; border-radius: 0.75rem; text-align: center; width: 180px;">
                                <div style="font-size: 0.7rem; color: #64748b; font-
weight: 700;">GESTIÓN</div>
                                <i class="fas fa-file-invoice"
                                    style="font-size: 1.5rem; color: #64748b; margin:
0.5rem 0;"></i>
                                <div style="font-weight: 600;">Orden de Compra</div>
                                <div style="font-size: 0.75rem; color: #ef4444; margin-
top: 0.25rem;"><i
                                        class="fas fa-times-circle"></i> No
Contabiliza</div>
                            </div>
                            <i class="fas fa-arrow-right" style="color: #cbd5e1;"></i>
                            <div
                                style="background: #f8fafc; border: 2px solid #cbd5e1;
padding: 1rem; border-radius: 0.75rem; text-align: center; width: 180px;">
                                <div style="font-size: 0.7rem; color: #64748b; font-
weight: 700;">LOGÍSTICA</div>
                                <i class="fas fa-truck-loading"
                                    style="font-size: 1.5rem; color: #64748b; margin:
0.5rem 0;"></i>
                                <div style="font-weight: 600;">Recepción</div>
                                <div style="font-size: 0.75rem; color: #ef4444; margin-
top: 0.25rem;"><i
                                        class="fas fa-times-circle"></i> No
Contabiliza</div>
                            </div>
```

```
                            <i class="fas fa-arrow-right" style="color: #3b82f6;"></i>
                            <div
                                style="background: #eff6ff; border: 2px solid #3b82f6;
padding: 1rem; border-radius: 0.75rem; text-align: center; width: 200px; box-shadow: var(-
-shadow);">
                                <div style="font-size: 0.7rem; color: #1e40af; font-
weight: 700;">CONTABILIDAD</div>
                                <i class="fas fa-file-signature"
                                    style="font-size: 1.5rem; color: #3b82f6; margin:
0.5rem 0;"></i>
                                <div style="font-weight: 700;">Factura Proveedor</div>
                                <div style="font-size: 0.75rem; color: #22c55e; font-
weight: 600; margin-top: 0.25rem;">
                                    <i class="fas fa-check-circle"></i> SÍ CONTABILIZA
                                </div>
                            </div>
                            <i class="fas fa-arrow-right" style="color: #10b981;"></i>
                            <div
                                style="background: #f0fdf4; border: 2px solid #10b981;
padding: 1rem; border-radius: 0.75rem; text-align: center; width: 180px; box-shadow: var(-
-shadow);">
                                <div style="font-size: 0.7rem; color: #166534; font-
weight: 700;">TESORERÍA</div>
                                <i class="fas fa-money-bill-wave"
                                    style="font-size: 1.5rem; color: #10b981; margin:
0.5rem 0;"></i>
                                <div style="font-weight: 700;">Pago</div>
                                <div style="font-size: 0.75rem; color: #22c55e; font-
weight: 600; margin-top: 0.25rem;">
                                    <i class="fas fa-check-circle"></i> SÍ CONTABILIZA
                                </div>
                            </div>
                        </div>
                    </div>
                    <div
                        style="background: #eff6ff; padding: 1.5rem; border-radius: 1rem;
border-left: 5px solid #3b82f6;">
                        <h4 style="margin: 0 0 1rem; color: #1e40af;"><i class="fas fa-
lightbulb"></i> Concepto
                            Clave: El momento de la verdad</h4>
                        <p style="margin: 0; font-size: 0.95rem; line-height: 1.6;">
                            La <strong>Orden de Compra</strong> es una intención. Solo
cuando recibimos la
                            <strong>Factura</strong> y la <strong>contabilizamos</strong>,
nace la obligación legal
                            de pago (Pasivo) y el reconocimiento del gasto o activo
(Mercadería).
                        </p>
                        <div style="display: flex; gap: 1rem; margin-top: 1.5rem; flex-
wrap: wrap;">
                            <button class="link-btn" onclick="goToModule('compras',
'facturas-proveedor')">
                                <i class="fas fa-external-link-alt"></i> Módulo Compras
                            </button>
                            <button class="link-btn success"
onclick="goToModule('tesoreria', 'cuentas-pagar')">
                                <i class="fas fa-wallet"></i> Módulo Tesorería (Pagos)
                            </button>
                        </div>
                    </div>
                </div>
                <div style="margin-bottom: 2rem;">
                    <h3
                        style="color: #166534; border-bottom: 2px solid #bbf7d0; padding-
bottom: 0.5rem; margin-bottom: 1.5rem;">
                        <i class="fas fa-cash-register"></i> 2. Ciclo de Ventas (Ingresos)
                    </h3>
                    <div
                        style="display: flex; flex-direction: column; gap: 1rem; align-
items: center; margin-bottom: 2rem;">
                        <div
                            style="display: flex; align-items: center; gap: 1rem; flex-
wrap: wrap; justify-content: center;">
                            <div
                                style="background: #f8fafc; border: 2px solid #cbd5e1;
padding: 1rem; border-radius: 0.75rem; text-align: center; width: 180px;">
                                <div style="font-size: 0.7rem; color: #64748b; font-
weight: 700;">GESTIÓN</div>
                                <i class="fas fa-shopping-basket"
                                    style="font-size: 1.5rem; color: #64748b; margin:
0.5rem 0;"></i>
```

```
                                <div style="font-weight: 600;">Pedido de Venta</div>
                                <div style="font-size: 0.75rem; color: #ef4444; margin-
top: 0.25rem;"><i
                                        class="fas fa-times-circle"></i> No
Contabiliza</div>
                            </div>
                            <i class="fas fa-arrow-right" style="color: #cbd5e1;"></i>
                            <div
                                style="background: #f8fafc; border: 2px solid #cbd5e1;
padding: 1rem; border-radius: 0.75rem; text-align: center; width: 180px;">
                                <div style="font-size: 0.7rem; color: #64748b; font-
weight: 700;">LOGÍSTICA</div>
                                <i class="fas fa-box" style="font-size: 1.5rem; color:
#64748b; margin: 0.5rem 0;"></i>
                                <div style="font-weight: 600;">Despacho</div>
                                <div style="font-size: 0.75rem; color: #ef4444; margin-
top: 0.25rem;"><i
                                        class="fas fa-times-circle"></i> No
Contabiliza</div>
                            </div>
                            <i class="fas fa-arrow-right" style="color: #10b981;"></i>
                            <div
                                style="background: #f0fdf4; border: 2px solid #10b981;
padding: 1rem; border-radius: 0.75rem; text-align: center; width: 200px; box-shadow: var(-
-shadow);">
                                <div style="font-size: 0.7rem; color: #166534; font-
weight: 700;">CONTABILIDAD</div>
                                <i class="fas fa-file-invoice-dollar"
                                    style="font-size: 1.5rem; color: #22c55e; margin:
0.5rem 0;"></i>
                                <div style="font-weight: 700;">Factura Cliente</div>
                                <div style="font-size: 0.75rem; color: #22c55e; font-
weight: 600; margin-top: 0.25rem;">
                                    <i class="fas fa-check-circle"></i> SÍ CONTABILIZA
                                </div>
                            </div>
                            <i class="fas fa-arrow-right" style="color: #3b82f6;"></i>
                            <div
                                style="background: #eff6ff; border: 2px solid #3b82f6;
padding: 1rem; border-radius: 0.75rem; text-align: center; width: 180px; box-shadow: var(-
-shadow);">
                                <div style="font-size: 0.7rem; color: #1e40af; font-
weight: 700;">TESORERÍA</div>
                                <i class="fas fa-hand-holding-usd"
                                    style="font-size: 1.5rem; color: #3b82f6; margin:
0.5rem 0;"></i>
                                <div style="font-weight: 700;">Cobro</div>
                                <div style="font-size: 0.75rem; color: #22c55e; font-
weight: 600; margin-top: 0.25rem;">
                                    <i class="fas fa-check-circle"></i> SÍ CONTABILIZA
                                </div>
                            </div>
                        </div>
                    </div>
                    <div
                        style="background: #f0fdf4; padding: 1.5rem; border-radius: 1rem;
border-left: 5px solid #22c55e;">
                        <h4 style="margin: 0 0 1rem; color: #166534;"><i class="fas fa-
info-circle"></i> Importante:
                            Reconocimiento de Ingresos</h4>
                        <p style="margin: 0; font-size: 0.95rem; line-height: 1.6;">
                            En el ERP, el ingreso se reconoce al <strong>contabilizar la
factura</strong>. El cobro
                            posterior solo cambia la forma del activo (de una Cuenta por
Cobrar a Caja/Banco), pero
                            el ingreso ya ocurrió.
                        </p>
                        <div style="display: flex; gap: 1rem; margin-top: 1.5rem; flex-
wrap: wrap;">
                            <button class="link-btn success" onclick="goToModule('ventas',
'facturas-cliente')">
                                <i class="fas fa-external-link-alt"></i> Módulo Ventas
                            </button>
                            <button class="link-btn" style="background: #3b82f6;"
                                onclick="goToModule('tesoreria', 'cuentas-cobrar')">
                                <i class="fas fa-handshake"></i> Módulo Tesorería (Cobros)
                            </button>
                        </div>
                    </div>
                </div>
            </div>
```

```
        </div>
    </div>
    <div class="modal-overlay" id="modal-overlay" onclick="closeModal(event)">
        <div class="modal" onclick="event.stopPropagation()">
            <div class="modal-header">
                <h3 id="modal-title"><i class="fas fa-info-circle"></i> Título</h3>
                <button class="modal-close" onclick="closeModal()"><i class="fas fa-
times"></i></button>
            </div>
            <div class="modal-body" id="modal-body">
            </div>
        </div>
    </div>
    <button class="print-btn" onclick="window.print()">
        <i class="fas fa-print"></i> Imprimir
    </button>
    <script>
        // Tab switching
        function showTab(tabId) {
            document.querySelectorAll('.tab-content').forEach(t =>
t.classList.remove('active'));
            document.querySelectorAll('.tab-btn').forEach(b =>
b.classList.remove('active'));
            document.getElementById('tab-' + tabId).classList.add('active');
            event.target.closest('.tab-btn').classList.add('active');
        }
        // Navigate to module in main system
        function goToModule(module, view) {
            // Build the target URL with hash
            const targetHash = `#${module}/${view}`;
            // Try to use opener window if available
            if (window.opener && !window.opener.closed) {
                try {
                    // Check if App is available in opener
                    if (window.opener.App && typeof window.opener.App.navigate ===
'function') {
                        window.opener.App.navigate(module, view);
                        window.opener.focus();
                        return;
                    }
                } catch (e) {
                    console.log('Cannot access opener, redirecting...', e);
                }
            }
            // Get the base path (same directory as this file)
            const basePath = window.location.href.substring(0,
window.location.href.lastIndexOf('/') + 1);
            const targetUrl = basePath + 'index.html' + targetHash;
            // Open in new window or redirect
            const newWindow = window.open(targetUrl, 'EDU-TRACE ERP');
            if (newWindow) {
                newWindow.focus();
            } else {
                // Fallback: redirect current window
                window.location.href = targetUrl;
            }
        }
        // Simulation navigation
        let currentSimStep = 1;
        const totalSimSteps = 8;
        function goToSimStep(step) {
            currentSimStep = step;
            // Hide all steps
            document.querySelectorAll('.sim-step').forEach(s =>
s.classList.remove('active'));
            // Show current step
            document.getElementById('sim-step-' + step).classList.add('active');
            // Update progress
            document.querySelectorAll('.progress-step').forEach((p, i) => {
                p.classList.remove('active', 'completed');
                if (i + 1 < step) {
                    p.classList.add('completed');
                } else if (i + 1 === step) {
                    p.classList.add('active');
                }
            });
        }
        function nextSimStep() {
            if (currentSimStep < totalSimSteps) {
                goToSimStep(currentSimStep + 1);
            }
```

```
        }
        function prevSimStep() {
            if (currentSimStep > 1) {
                goToSimStep(currentSimStep - 1);
            }
        }
        // Modal content
        const modalContent = {
            entrada: {
                title: '<i class="fas fa-sign-in-alt"></i> Etapa de Entrada',
                content: `
                    <div class="content-section">
                        <h4>¿Qué es la Etapa de Entrada?</h4>
                        <p>Es la primera fase del proceso contable donde se recopilan y
analizan los <strong>documentos mercantiles</strong> que respaldan las transacciones
económicas.</p>
                        <ul>
                            <li><i class="fas fa-check"></i> <strong>Recopilar documentos:
</strong> Facturas, boletas, notas de crédito, recibos, contratos de arriendo y contratos
de servicios</li>
                            <li><i class="fas fa-check"></i> <strong>Verificar
autenticidad:</strong> Comprobar que sean legítimos y legales</li>
                            <li><i class="fas fa-check"></i> <strong>Identificar el hecho:
</strong> Determinar el tipo de transacción</li>
                            <li><i class="fas fa-check"></i> <strong>Clasificar cuentas:
</strong> Identificar qué cuentas se afectarán</li>
                        </ul>
                    </div>
                    <button class="link-btn" onclick="goToModule('compras', 'facturas-
proveedor')">
                        <i class="fas fa-external-link-alt"></i> Ver Facturas de Proveedor
                    </button>
                    <button class="link-btn success" onclick="goToModule('ventas',
'facturas-cliente')">
                        <i class="fas fa-external-link-alt"></i> Ver Facturas de Cliente
                    </button>
                `
            },
            documento: {
                title: '<i class="fas fa-file-invoice"></i> Documento Mercantil',
                content: `
                    <div class="content-section">
                        <h4>¿Qué es un Documento Mercantil?</h4>
                        <p>Es cualquier documento escrito que sirve como
<strong>comprobante o respaldo</strong> de una transacción comercial.</p>
                        <ul>
                            <li><i class="fas fa-file-invoice-dollar"></i>
<strong>Factura:</strong> Detalla venta o compra con IVA</li>
                            <li><i class="fas fa-receipt"></i> <strong>Boleta:</strong>
Comprobante para consumidor final</li>
                            <li><i class="fas fa-undo"></i> <strong>Nota de Crédito:
</strong> Anula o reduce una factura</li>
                            <li><i class="fas fa-plus-circle"></i> <strong>Nota de Débito:
</strong> Aumenta el monto de una factura</li>
                            <li><i class="fas fa-truck"></i> <strong>Guía de Despacho:
</strong> Acredita traslado de mercaderías</li>
                            <li><i class="fas fa-file-contract"></i> <strong>Contrato de
Arriendo:</strong> Acuerdo de uso de un bien físico</li>
                            <li><i class="fas fa-file-signature"></i> <strong>Contrato de
Servicios:</strong> Acuerdo para la prestación de un servicio</li>
                        </ul>
                    </div>
                `
            },
            elementos: {
                title: '<i class="fas fa-tags"></i> Elementos Contables',
                content: `
                    <div class="account-box">
                        <h5>Los 5 Elementos Contables</h5>
                        <div class="account-grid">
                            <div class="account-item"><div class="label">ACTIVO</div><div
class="value">Lo que TIENE</div></div>
                            <div class="account-item"><div class="label">PASIVO</div><div
class="value">Lo que DEBE</div></div>
                            <div class="account-item"><div class="label">PATRIMONIO</div>
<div class="value">De los dueños</div></div>
                            <div class="account-item"><div class="label">INGRESO</div><div
class="value">Lo que GANA</div></div>
                            <div class="account-item"><div class="label">GASTO</div><div
class="value">Lo que GASTA</div></div>
                        </div>
```

```
                    </div>
                    <div class="content-section">
                        <h4>Ecuación Contable</h4>
                        <p style="font-size: 1.25rem; text-align: center; padding: 1rem;
background: var(--primary-light); border-radius: 0.5rem;">
                            <strong>ACTIVO = PASIVO + PATRIMONIO</strong>
                        </p>
                    </div>
                    <button class="link-btn" onclick="goToModule('contabilidad', 'plan-
cuentas')">
                        <i class="fas fa-external-link-alt"></i> Ver Plan de Cuentas
                    </button>
                `
            },
            registro: {
                title: '<i class="fas fa-book"></i> Reglas de Registro',
                content: `
                    <div class="content-section">
                        <h4>Débito vs Crédito</h4>
                        <ul>
                            <li><i class="fas fa-arrow-up" style="color: #dc2626;"></i>
<strong>ACTIVO aumenta:</strong> Se DEBITA</li>
                            <li><i class="fas fa-arrow-down" style="color: #16a34a;"></i>
<strong>ACTIVO disminuye:</strong> Se ACREDITA</li>
                            <li><i class="fas fa-arrow-up" style="color: #16a34a;"></i>
<strong>PASIVO aumenta:</strong> Se ACREDITA</li>
                            <li><i class="fas fa-arrow-down" style="color: #dc2626;"></i>
<strong>PASIVO disminuye:</strong> Se DEBITA</li>
                            <li><i class="fas fa-arrow-up" style="color: #16a34a;"></i>
<strong>INGRESO aumenta:</strong> Se ACREDITA</li>
                            <li><i class="fas fa-arrow-up" style="color: #dc2626;"></i>
<strong>GASTO aumenta:</strong> Se DEBITA</li>
                        </ul>
                    </div>
                    <div class="t-account">
                        <div class="t-account-header">Cuenta T - Ejemplo: CAJA</div>
                        <div class="t-account-body">
                            <div class="t-account-side debit"><h6>DEBE</h6><div
class="amount">+ Aumenta</div></div>
                            <div class="t-account-side credit"><h6>HABER</h6><div
class="amount">- Disminuye</div></div>
                        </div>
                    </div>
                `
            },
            proceso: {
                title: '<i class="fas fa-cogs"></i> Etapa de Proceso',
                content: `
                    <div class="content-section">
                        <h4>¿Qué es el Registro Contable?</h4>
                        <p>Es la fase donde las transacciones se <strong>registran
formalmente</strong> en los libros contables, siguiendo la <strong>partida doble</strong>.
</p>
                    </div>
                    <div class="content-section">
                        <h4>Libros Contables</h4>
                        <ul>
                            <li><i class="fas fa-book"></i> <strong>Libro Diario:</strong>
Registro cronológico de todas las transacciones</li>
                            <li><i class="fas fa-columns"></i> <strong>Libro Mayor:
</strong> Agrupa movimientos por cuenta</li>
                            <li><i class="fas fa-list-check"></i> <strong>Balance de
Comprobación:</strong> Actúa como el nexo contable comprobatorio entre los módulos y la
posterior preparación de los Estados Financieros</li>
                        </ul>
                    </div>
                    <button class="link-btn" onclick="goToModule('contabilidad',
'asientos')">
                        <i class="fas fa-external-link-alt"></i> Ver Libro Diario
                    </button>
                    <button class="link-btn success" onclick="goToModule('contabilidad',
'libro-mayor')">
                        <i class="fas fa-external-link-alt"></i> Ver Libro Mayor
                    </button>
                    <button class="link-btn" style="background-color: var(--purple);"
onclick="goToModule('reportes', 'balance-comprobacion')">
                        <i class="fas fa-external-link-alt"></i> Ver Balance de
Comprobación
                    </button>
                `
            },
```

```
            salida: {
                title: '<i class="fas fa-file-export"></i> Etapa de Salida',
                content: `
                    <div class="content-section">
                        <h4>Estados Financieros</h4>
                        <ul>
                            <li><i class="fas fa-chart-pie"></i> <strong>Estado de
Situación (Balance General):</strong> Activos, Pasivos y Patrimonio</li>
                            <li><i class="fas fa-chart-line"></i> <strong>Estado de
Resultados:</strong> Ingresos y Gastos</li>
                        </ul>
                    </div>
                    <button class="link-btn" onclick="goToModule('reportes', 'balance-
general')">
                        <i class="fas fa-external-link-alt"></i> Ver Balance General
                    </button>
                    <button class="link-btn success" onclick="goToModule('reportes',
'estado-resultados')">
                        <i class="fas fa-external-link-alt"></i> Ver Estado de Resultados
                    </button>
                `
            },
            balance: {
                title: '<i class="fas fa-balance-scale"></i> Balance de Comprobación',
                content: `
                    <div class="content-section">
                        <h4>¿Qué es?</h4>
                        <p>Lista TODAS las cuentas con sus saldos deudores y acreedores.
Objetivo: actuar como nexo contable y revisar que la suma de débitos sea idéntica a
créditos previo a armar Estados Financieros.</p>
                    </div>
                    <div class="t-account">
                        <div class="t-account-header">Ejemplo: Cuenta CAJA</div>
                        <div class="t-account-body">
                            <div class="t-account-side debit"><h6>DEBE</h6><div
class="amount">$5.000.000</div></div>
                            <div class="t-account-side credit"><h6>HABER</h6><div
class="amount">$2.000.000</div></div>
                        </div>
                    </div>
                    <p style="text-align: center; font-size: 1.1rem;"><strong>Saldo
Deudor: $3.000.000</strong></p>
                    <button class="link-btn" onclick="goToModule('reportes', 'balance-
comprobacion')">
                        <i class="fas fa-external-link-alt"></i> Ver Balance de
Comprobación
                    </button>
                `
            },
            situacion: {
                title: '<i class="fas fa-chart-pie"></i> Estado de Situación Financiera',
                content: `
                    <div class="content-section">
                        <h4>¿Qué es?</h4>
                        <p>También llamado <strong>Balance General</strong>. Una
"fotografía" de la empresa: qué tiene (Activos), qué debe (Pasivos) y cuánto pertenece a
los dueños (Patrimonio).</p>
                    </div>
                    <div class="account-box">
                        <h5>Estructura</h5>
                        <div class="account-grid">
                            <div class="account-item"><div class="label">ACTIVOS</div><div
class="value">$18.000.000</div></div>
                            <div class="account-item"><div class="label">PASIVOS</div><div
class="value">$8.000.000</div></div>
                            <div class="account-item"><div class="label">PATRIMONIO</div>
<div class="value">$10.000.000</div></div>
                        </div>
                        <p style="text-align: center; margin-top: 0.75rem; font-size:
0.9rem;">
                            <strong>Activo ($18M) = Pasivo ($8M) + Patrimonio ($10M)
✓</strong>
                        </p>
                    </div>
                    <button class="link-btn" onclick="goToModule('reportes', 'balance-
general')">
                        <i class="fas fa-external-link-alt"></i> Ver Balance General
                    </button>
                `
            },
            resultados: {
```

```
                title: '<i class="fas fa-chart-line"></i> Estado de Resultados',
                content: `
                    <div class="content-section">
                        <h4>¿Qué es?</h4>
                        <p>Muestra el <strong>rendimiento económico</strong> de la empresa
durante un período. Indica si ganó o perdió dinero.</p>
                    </div>
                    <div class="flow-example">
                        <h5><i class="fas fa-calculator"></i> Estructura del Estado de
Resultados</h5>
                        <div class="flow-steps">
                            <div class="flow-step"><div class="step-number">+</div><div
class="step-content">Ingresos por Ventas: <strong>$15.000.000</strong></div></div>
                            <div class="flow-step"><div class="step-number">-</div><div
class="step-content">Costo de Ventas: <strong>($9.000.000)</strong></div></div>
                            <div class="flow-step" style="background: #f3f4f630; margin:
0.5rem 0; border-left: 4px solid #10b981;">
                                <div class="step-number">=</div>
                                <div class="step-content" style="color: #10b981; font-
weight: 700;">Resultado Operacional (Margen Bruto): $6.000.000</div>
                            </div>
                            <div class="flow-step"><div class="step-number">+</div><div
class="step-content">Otros Ingresos: <strong>$500.000</strong></div></div>
                            <div class="flow-step"><div class="step-number">-</div><div
class="step-content">Gastos de Administración: <strong>($2.000.000)</strong></div></div>
                            <div class="flow-step"><div class="step-number">-</div><div
class="step-content">Gastos de Ventas: <strong>($800.000)</strong></div></div>
                            <div class="flow-step"><div class="step-number">-</div><div
class="step-content">Gastos Financieros: <strong>($200.000)</strong></div></div>
                            <div class="flow-step" style="background: #10b98120; margin-
top: 0.5rem; border-left: 4px solid #16a34a; padding: 1rem;">
                                <div class="step-number" style="background: #16a34a;">=
</div>
                                <div class="step-content" style="color: #16a34a; font-
weight: 700; font-size: 1.1rem;">Resultado Neto del Ejercicio (Ganancia): $3.500.000</div>
                            </div>
                        </div>
                        <p style="margin-top: 1rem; padding: 0.75rem; background: #f0fdf4;
border-left: 4px solid #16a34a; border-radius: 0.5rem; font-size: 0.9rem;">
                            <i class="fas fa-info-circle" style="color: #16a34a;"></i>
<strong>Nota:</strong> Si el resultado es negativo, se denomina <strong>Pérdida del
Ejercicio</strong>.
                        </p>
                    </div>
                    <button class="link-btn success" onclick="goToModule('reportes',
'estado-resultados')">
                        <i class="fas fa-external-link-alt"></i> Ver Estado de Resultados
                    </button>
                `
            },
            ratios: {
                title: '<i class="fas fa-chart-bar"></i> Ratios Financieros',
                content: `
                    <div class="content-section">
                        <h4>¿Qué son los Ratios Financieros?</h4>
                        <p>Son <strong>indicadores numéricos</strong> calculados a partir
de los estados financieros que permiten evaluar el desempeño, la salud financiera y la
eficiencia de una empresa.</p>
                    </div>
                    <div class="flow-example">
                        <h5><i class="fas fa-tint"></i> Ratios de Liquidez</h5>
                        <div class="flow-steps">
                            <div class="flow-step"><div class="step-number">1</div><div
class="step-content"><strong>Razón Corriente</strong> = Activo Corriente / Pasivo
Corriente</div></div>
                            <div class="flow-step"><div class="step-number">2</div><div
class="step-content"><strong>Prueba Ácida</strong> = (Act. Corr. - Inventario) / Pasivo
Corriente</div></div>
                        </div>
                    </div>
                    <div class="flow-example">
                        <h5><i class="fas fa-chart-line"></i> Ratios de Rentabilidad</h5>
                        <div class="flow-steps">
                            <div class="flow-step"><div class="step-number">1</div><div
class="step-content"><strong>ROA</strong> = Resultado del Ejercicio / Total Activos ×
100</div></div>
                            <div class="flow-step"><div class="step-number">2</div><div
class="step-content"><strong>ROE</strong> = Resultado del Ejercicio / Patrimonio ×
100</div></div>
                            <div class="flow-step"><div class="step-number">3</div><div
class="step-content"><strong>Margen Neto</strong> = Resultado del Ejercicio / Ventas ×
```

```
100</div></div>
                        </div>
                    </div>
                    <div class="flow-example">
                        <h5><i class="fas fa-shield-alt"></i> Ratios de Solvencia</h5>
                        <div class="flow-steps">
                            <div class="flow-step"><div class="step-number">1</div><div
class="step-content"><strong>Endeudamiento</strong> = Total Pasivos / Total Activos ×
100</div></div>
                            <div class="flow-step"><div class="step-number">2</div><div
class="step-content"><strong>Autonomía</strong> = Patrimonio / Total Activos × 100</div>
</div>
                        </div>
                    </div>
                    <button class="link-btn" style="background: #f59e0b;"
onclick="goToModule('reportes', 'ratios-financieros')">
                        <i class="fas fa-external-link-alt"></i> Ver Ratios Financieros en
el Sistema
                    </button>
                `
            },
            sistemas: {
                title: '<i class="fas fa-book"></i> Guía: Sistemas de Contabilización',
                content: `
                    <div class="content-section">
                        <h4>¿Qué son?</h4>
                        <p>EDU-TRACE ERP soporta <strong>dos sistemas de
contabilización</strong> que determinan cómo se registran las transacciones contables:</p>
                    </div>
                    <div class="flow-example">
                        <h5><i class="fas fa-book"></i> 1. Jornalizador (Diario Único)
</h5>
                        <div class="flow-steps">
                            <div class="flow-step"><div class="step-number">✓</div><div
class="step-content">Todas las transacciones se registran <strong>directamente</strong> en
el Libro Diario</div></div>
                            <div class="flow-step"><div class="step-number">✓</div><div
class="step-content">Ideal para <strong>microempresas y estudiantes</strong></div></div>
                            <div class="flow-step"><div class="step-number">✓</div><div
class="step-content">Simple, directo, visibilidad inmediata</div></div>
                        </div>
                    </div>
                    <div class="flow-example">
                        <h5><i class="fas fa-books"></i> 2. Centralizador (Libros
Auxiliares)</h5>
                        <div class="flow-steps">
                            <div class="flow-step"><div class="step-number">1</div><div
class="step-content">Las transacciones se registran primero en <strong>libros
auxiliares</strong> por módulo (Ventas, Compras, etc.)</div></div>
                            <div class="flow-step"><div class="step-number">2</div><div
class="step-content">Al final del mes, se <strong>centralizan</strong> (consolidan) en el
Libro Diario</div></div>
                            <div class="flow-step"><div class="step-number">✓</div><div
class="step-content">Ideal para <strong>PYMEs y grandes empresas</strong></div></div>
                            <div class="flow-step"><div class="step-number">✓</div><div
class="step-content">Reduce volumen del Libro Diario, mejor organización</div></div>
                        </div>
                    </div>
                    <div style="padding: 1rem; background: #eff6ff; border-left: 4px solid
#3b82f6; border-radius: 0.5rem; margin-top: 1rem;">
                        <p style="margin: 0; font-size: 0.95rem;">
                            <i class="fas fa-cog" style="color: #3b82f6;"></i>
<strong>Configuración:</strong>
                            El sistema se elige en <strong>Administración → Configuración
→ Sistema de Contabilización</strong>.
                            Por defecto: <strong>Jornalizador</strong>.
                        </p>
                    </div>
                    <button class="link-btn" style="background: #3b82f6;"
onclick="goToModule('admin', 'configuracion')">
                        <i class="fas fa-cog"></i> Ir a Configuración
                    </button>
                `
            }
        };
        function openModal(topic) {
            const content = modalContent[topic];
            if (!content) return;
            document.getElementById('modal-title').innerHTML = content.title;
            document.getElementById('modal-body').innerHTML = content.content;
```

```
            document.getElementById('modal-overlay').classList.add('active');
            document.body.style.overflow = 'hidden';
        }
        function closeModal(event) {
            if (event && event.target !== event.currentTarget) return;
            document.getElementById('modal-overlay').classList.remove('active');
            document.body.style.overflow = '';
        }
        document.addEventListener('keydown', (e) => { if (e.key === 'Escape')
closeModal(); });
    </script>
</body>
</html>
```

Archivo: css\components.css

```
.card {
    background: var(--bg-secondary);
    border: 1px solid var(--border-light);
    border-radius: var(--radius-xl);
    box-shadow: var(--shadow-sm);
    overflow: hidden;
    transition: box-shadow var(--transition-fast), transform var(--transition-fast)
}
.card:hover {
    box-shadow: var(--shadow-md)
}
.card-header {
    padding: var(--space-5);
    border-bottom: 1px solid var(--border-light);
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: var(--space-4)
}
.card-title {
    font-size: var(--font-size-lg);
    font-weight: 600;
    color: var(--text-primary);
    display: flex;
    align-items: center;
    gap: var(--space-3)
}
.card-title i {
    color: var(--primary-500)
}
.card-body {
    padding: var(--space-5)
}
.card-footer {
    padding: var(--space-4) var(--space-5);
    border-top: 1px solid var(--border-light);
    background: var(--neutral-50)
}
.btn {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: var(--space-2);
    padding: var(--space-2) var(--space-4);
    font-family: inherit;
    font-size: var(--font-size-sm);
    font-weight: 500;
    line-height: 1.5;
    border: none;
    border-radius: var(--radius-md);
    cursor: pointer;
    transition: all var(--transition-fast);
    white-space: nowrap
}
.btn:disabled {
    opacity: .5;
    cursor: not-allowed
}
.btn-primary {
```

```
    background: var(--primary-500);
    color: #fff
}
.btn-primary:hover:not(:disabled) {
    background: var(--primary-600)
}
.btn-secondary {
    background: var(--neutral-200);
    color: var(--text-primary)
}
.btn-secondary:hover:not(:disabled) {
    background: var(--neutral-300)
}
.btn-success {
    background: var(--success-500);
    color: #fff
}
.btn-success:hover:not(:disabled) {
    background: var(--success-600)
}
.btn-danger {
    background: var(--error-500);
    color: #fff
}
.btn-danger:hover:not(:disabled) {
    background: var(--error-600)
}
.btn-outline {
    background: transparent;
    border: 1px solid var(--border-medium);
    color: var(--text-primary)
}
.btn-outline:hover:not(:disabled) {
    background: var(--neutral-100);
    border-color: var(--primary-500);
    color: var(--primary-500)
}
.btn-ghost {
    background: transparent;
    color: var(--text-secondary)
}
.btn-ghost:hover:not(:disabled) {
    background: var(--neutral-100);
    color: var(--text-primary)
}
.btn-sm {
    padding: var(--space-1) var(--space-3);
    font-size: var(--font-size-xs)
}
.btn-lg {
    padding: var(--space-3) var(--space-6);
    font-size: var(--font-size-base)
}
.btn-icon {
    width: 36px;
    height: 36px;
    padding: 0
}
.btn-group {
    display: flex;
    gap: var(--space-2)
}
.form-group {
    margin-bottom: var(--space-4)
}
.form-label {
    display: block;
    margin-bottom: var(--space-2);
    font-size: var(--font-size-sm);
    font-weight: 500;
    color: var(--text-primary)
}
.form-label.required::after {
    content: '*';
    margin-left: var(--space-1);
    color: var(--error-500)
}
.form-control {
    width: 100%;
    padding: var(--space-2) var(--space-3);
    font-family: inherit;
```

```
    font-size: var(--font-size-sm);
    border: 1px solid var(--border-medium);
    border-radius: var(--radius-md);
    background: var(--bg-secondary);
    color: var(--text-primary);
    transition: border-color var(--transition-fast), box-shadow var(--transition-fast)
}
.form-control:focus {
    outline: none;
    border-color: var(--primary-500);
    box-shadow: 0 0 0 3px rgba(10, 110, 209, .15)
}
.form-control:disabled {
    background: var(--neutral-100);
    cursor: not-allowed
}
.form-control.error {
    border-color: var(--error-500)
}
.form-control.success {
    border-color: var(--success-500)
}
textarea.form-control {
    min-height: 100px;
    resize: vertical
}
select.form-control {
    appearance: none;
    background-image: url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='12' height='12' fill='%236a6d70' viewBox='0 0 16 16'%3E%3Cpath d='M8 11L3 6h10l-5
5z'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: right var(--space-3) center;
    padding-right: var(--space-8)
}
.form-hint {
    margin-top: var(--space-1);
    font-size: var(--font-size-xs);
    color: var(--text-tertiary)
}
.form-error {
    margin-top: var(--space-1);
    font-size: var(--font-size-xs);
    color: var(--error-500)
}
.form-row {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: var(--space-4)
}
.input-group {
    display: flex
}
.input-group .form-control {
    flex: 1;
    border-radius: var(--radius-md) 0 0 var(--radius-md)
}
.input-group .btn {
    border-radius: 0 var(--radius-md) var(--radius-md) 0
}
.input-prefix,
.input-suffix {
    display: flex;
    align-items: center;
    padding: 0 var(--space-3);
    background: var(--neutral-100);
    border: 1px solid var(--border-medium);
    font-size: var(--font-size-sm);
    color: var(--text-secondary)
}
.input-prefix {
    border-right: 0;
    border-radius: var(--radius-md) 0 0 var(--radius-md)
}
.input-suffix {
    border-left: 0;
    border-radius: 0 var(--radius-md) var(--radius-md) 0
}
.checkbox-group,
.radio-group {
    display: flex;
```

```
    flex-direction: column;
    gap: var(--space-2)
}
.checkbox-item,
.radio-item {
    display: flex;
    align-items: center;
    gap: var(--space-2);
    cursor: pointer
}
.checkbox-item input,
.radio-item input {
    width: 18px;
    height: 18px;
    accent-color: var(--primary-500)
}
.table-container {
    overflow-x: auto;
    border: 1px solid var(--border-light);
    border-radius: var(--radius-lg)
}
.data-table {
    width: 100%;
    border-collapse: collapse;
    font-size: var(--font-size-sm)
}
.data-table th,
.data-table td {
    padding: var(--space-3) var(--space-4);
    text-align: left;
    border-bottom: 1px solid var(--border-light)
}
.data-table th {
    background: var(--neutral-50);
    font-weight: 600;
    color: var(--text-primary);
    white-space: nowrap;
    position: sticky;
    top: 0
}
.data-table tbody tr:hover {
    background: var(--primary-50)
}
.data-table tbody tr:last-child td {
    border-bottom: none
}
.data-table .text-right {
    text-align: right
}
.data-table .text-center {
    text-align: center
}
.data-table .actions {
    display: flex;
    gap: var(--space-1);
    justify-content: flex-end
}
.table-empty {
    padding: var(--space-8);
    text-align: center;
    color: var(--text-tertiary)
}
.table-empty i {
    font-size: 3rem;
    margin-bottom: var(--space-4);
    opacity: .5
}
.badge {
    display: inline-flex;
    align-items: center;
    gap: var(--space-1);
    padding: var(--space-1) var(--space-2);
    font-size: var(--font-size-xs);
    font-weight: 500;
    border-radius: var(--radius-sm)
}
.badge-primary {
    background: var(--primary-50);
    color: var(--primary-600)
}
.badge-success {
```

```
    background: var(--success-50);
    color: var(--success-600)
}
.badge-warning {
    background: var(--warning-50);
    color: var(--warning-600)
}
.badge-error {
    background: var(--error-50);
    color: var(--error-600)
}
.badge-info {
    background: var(--info-50);
    color: var(--info-500)
}
.badge-neutral {
    background: var(--neutral-200);
    color: var(--neutral-700)
}
.tabs {
    border-bottom: 1px solid var(--border-light);
    margin-bottom: var(--space-4)
}
.tab-list {
    display: flex;
    gap: var(--space-1);
    overflow-x: auto
}
.tab-btn {
    padding: var(--space-3) var(--space-4);
    background: transparent;
    border: none;
    border-bottom: 2px solid transparent;
    font-family: inherit;
    font-size: var(--font-size-sm);
    font-weight: 500;
    color: var(--text-secondary);
    cursor: pointer;
    transition: all var(--transition-fast);
    white-space: nowrap
}
.tab-btn:hover {
    color: var(--text-primary)
}
.tab-btn.active {
    color: var(--primary-500);
    border-bottom-color: var(--primary-500)
}
.tab-content {
    display: none
}
.tab-content.active {
    display: block
}
.alert {
    display: flex;
    align-items: flex-start;
    gap: var(--space-3);
    padding: var(--space-4);
    border-radius: var(--radius-lg);
    margin-bottom: var(--space-4)
}
.alert i {
    font-size: var(--font-size-lg);
    flex-shrink: 0;
    margin-top: 2px
}
.alert-content {
    flex: 1
}
.alert-title {
    font-weight: 600;
    margin-bottom: var(--space-1)
}
.alert-info {
    background: var(--info-50);
    color: var(--info-500);
    border-left: 4px solid var(--info-500)
}
.alert-success {
    background: var(--success-50);
```

```
    color: var(--success-500);
    border-left: 4px solid var(--success-500)
}
.alert-warning {
    background: var(--warning-50);
    color: var(--warning-500);
    border-left: 4px solid var(--warning-500)
}
.alert-error {
    background: var(--error-50);
    color: var(--error-500);
    border-left: 4px solid var(--error-500)
}
.stat-card {
    background: var(--bg-secondary);
    border: 1px solid var(--border-light);
    border-radius: var(--radius-xl);
    padding: var(--space-5);
    display: flex;
    align-items: flex-start;
    gap: var(--space-4)
}
.stat-icon {
    width: 48px;
    height: 48px;
    border-radius: var(--radius-lg);
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: var(--font-size-xl)
}
.stat-icon.primary {
    background: var(--primary-50);
    color: var(--primary-500)
}
.stat-icon.success {
    background: var(--success-50);
    color: var(--success-500)
}
.stat-icon.warning {
    background: var(--warning-50);
    color: var(--warning-500)
}
.stat-icon.error {
    background: var(--error-50);
    color: var(--error-500)
}
.stat-content {
    flex: 1
}
.stat-label {
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
    margin-bottom: var(--space-1)
}
.stat-value {
    font-size: var(--font-size-2xl);
    font-weight: 700;
    color: var(--text-primary)
}
.stat-change {
    display: inline-flex;
    align-items: center;
    gap: var(--space-1);
    font-size: var(--font-size-xs);
    font-weight: 500;
    margin-top: var(--space-2)
}
.stat-change.positive {
    color: var(--success-500)
}
.stat-change.negative {
    color: var(--error-500)
}
.empty-state {
    text-align: center;
    padding: var(--space-12);
    color: var(--text-secondary)
}
.empty-state i {
    font-size: 4rem;
```

```
    opacity: .3;
    margin-bottom: var(--space-4)
}
.empty-state h3 {
    font-size: var(--font-size-lg);
    color: var(--text-primary);
    margin-bottom: var(--space-2)
}
.empty-state p {
    max-width: 400px;
    margin: 0 auto var(--space-4)
}
.pagination {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: var(--space-1);
    margin-top: var(--space-4)
}
.page-btn {
    width: 36px;
    height: 36px;
    display: flex;
    align-items: center;
    justify-content: center;
    background: transparent;
    border: 1px solid var(--border-light);
    border-radius: var(--radius-md);
    font-size: var(--font-size-sm);
    color: var(--text-primary);
    cursor: pointer;
    transition: all var(--transition-fast)
}
.page-btn:hover:not(:disabled) {
    background: var(--neutral-100);
    border-color: var(--primary-500)
}
.page-btn.active {
    background: var(--primary-500);
    border-color: var(--primary-500);
    color: #fff
}
.page-btn:disabled {
    opacity: .5;
    cursor: not-allowed
}
.page-info {
    padding: 0 var(--space-3);
    font-size: var(--font-size-sm);
    color: var(--text-secondary)
}
.search-box {
    position: relative;
    max-width: 300px
}
.search-box input {
    width: 100%;
    padding-left: var(--space-8)
}
.search-box i {
    position: absolute;
    left: var(--space-3);
    top: 50%;
    transform: translateY(-50%);
    color: var(--text-tertiary)
}
.toolbar {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: var(--space-4);
    flex-wrap: wrap;
    margin-bottom: var(--space-4)
}
.toolbar-left,
.toolbar-right {
    display: flex;
    align-items: center;
    gap: var(--space-3)
}
.page-header {
```

```
    margin-bottom: var(--space-6)
}
.page-title {
    font-size: var(--font-size-2xl);
    font-weight: 700;
    color: var(--text-primary)
}
.page-subtitle {
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
    margin-top: var(--space-1)
}
.section {
    margin-bottom: var(--space-6)
}
.section-title {
    font-size: var(--font-size-lg);
    font-weight: 600;
    color: var(--text-primary);
    margin-bottom: var(--space-4);
    padding-bottom: var(--space-2);
    border-bottom: 1px solid var(--border-light)
}
.avatar {
    width: 40px;
    height: 40px;
    border-radius: var(--radius-full);
    background: var(--primary-100);
    color: var(--primary-600);
    display: flex;
    align-items: center;
    justify-content: center;
    font-weight: 600;
    font-size: var(--font-size-sm)
}
.avatar-sm {
    width: 32px;
    height: 32px;
    font-size: var(--font-size-xs)
}
.avatar-lg {
    width: 56px;
    height: 56px;
    font-size: var(--font-size-lg)
}
.divider {
    height: 1px;
    background: var(--border-light);
    margin: var(--space-4) 0
}
.divider-vertical {
    width: 1px;
    height: 100%;
    background: var(--border-light);
    margin: 0 var(--space-4)
}
.text-danger {
    color: var(--error-500) !important
}
.text-success {
    color: var(--success-500) !important
}
.text-warning {
    color: var(--warning-500) !important
}
.text-info {
    color: var(--info-500) !important
}
```

Archivo: css\dashboard.css

```
.dashboard-grid {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: var(--space-4)
```

```
}
.dashboard-grid .span-2 {
    grid-column: span 2
}
.dashboard-grid .span-3 {
    grid-column: span 3
}
.dashboard-grid .span-4 {
    grid-column: span 4
}
@media(max-width:1200px) {
    .dashboard-grid {
        grid-template-columns: repeat(2, 1fr)
    }
    .dashboard-grid .span-3,
    .dashboard-grid .span-4 {
        grid-column: span 2
    }
}
@media(max-width:768px) {
    .dashboard-grid {
        grid-template-columns: 1fr
    }
    .dashboard-grid .span-2,
    .dashboard-grid .span-3,
    .dashboard-grid .span-4 {
        grid-column: span 1
    }
}
.kpi-card {
    background: var(--bg-secondary);
    border: 1px solid var(--border-light);
    border-radius: var(--radius-xl);
    padding: var(--space-5);
    position: relative;
    overflow: hidden;
    transition: transform var(--transition-fast), box-shadow var(--transition-fast)
}
.kpi-card:hover {
    transform: translateY(-2px);
    box-shadow: var(--shadow-md)
}
.kpi-card::before {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    height: 4px;
    background: var(--primary-500)
}
.kpi-card.success::before {
    background: var(--success-500)
}
.kpi-card.warning::before {
    background: var(--warning-500)
}
.kpi-card.error::before {
    background: var(--error-500)
}
.kpi-header {
    display: flex;
    align-items: flex-start;
    justify-content: space-between;
    margin-bottom: var(--space-3)
}
.kpi-icon {
    width: 44px;
    height: 44px;
    border-radius: var(--radius-lg);
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: var(--font-size-xl)
}
.kpi-icon.primary {
    background: linear-gradient(135deg, var(--primary-400), var(--primary-600));
    color: #fff
}
.kpi-icon.success {
    background: linear-gradient(135deg, #4caf50, var(--success-500));
```

```
    color: #fff
}
.kpi-icon.warning {
    background: linear-gradient(135deg, #ff9800, var(--warning-500));
    color: #fff
}
.kpi-icon.error {
    background: linear-gradient(135deg, #f44336, var(--error-500));
    color: #fff
}
.kpi-menu {
    color: var(--text-tertiary);
    cursor: pointer
}
.kpi-value {
    font-size: var(--font-size-3xl);
    font-weight: 700;
    color: var(--text-primary);
    line-height: 1.2
}
.kpi-label {
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
    margin-top: var(--space-1)
}
.kpi-footer {
    display: flex;
    align-items: center;
    gap: var(--space-2);
    margin-top: var(--space-3);
    padding-top: var(--space-3);
    border-top: 1px solid var(--border-light);
    font-size: var(--font-size-xs);
    color: var(--text-secondary)
}
.kpi-trend {
    display: inline-flex;
    align-items: center;
    gap: var(--space-1);
    font-weight: 500
}
.kpi-trend.up {
    color: var(--success-500)
}
.kpi-trend.down {
    color: var(--error-500)
}
.chart-card {
    background: var(--bg-secondary);
    border: 1px solid var(--border-light);
    border-radius: var(--radius-xl);
    overflow: hidden
}
.chart-header {
    padding: var(--space-4) var(--space-5);
    border-bottom: 1px solid var(--border-light);
    display: flex;
    align-items: center;
    justify-content: space-between
}
.chart-title {
    font-size: var(--font-size-base);
    font-weight: 600;
    color: var(--text-primary)
}
.chart-subtitle {
    font-size: var(--font-size-xs);
    color: var(--text-tertiary);
    margin-top: var(--space-1)
}
.chart-actions {
    display: flex;
    gap: var(--space-2)
}
.chart-body {
    padding: var(--space-4);
    min-height: 300px
}
.chart-legend {
    display: flex;
    flex-wrap: wrap;
```

```
    gap: var(--space-4);
    padding: var(--space-3) var(--space-5);
    border-top: 1px solid var(--border-light);
    background: var(--neutral-50)
}
.legend-item {
    display: flex;
    align-items: center;
    gap: var(--space-2);
    font-size: var(--font-size-xs);
    color: var(--text-secondary)
}
.legend-color {
    width: 12px;
    height: 12px;
    border-radius: var(--radius-sm)
}
.activity-list {
    display: flex;
    flex-direction: column
}
.activity-item {
    display: flex;
    gap: var(--space-3);
    padding: var(--space-3) 0;
    border-bottom: 1px solid var(--border-light)
}
.activity-item:last-child {
    border-bottom: none
}
.activity-icon {
    width: 36px;
    height: 36px;
    border-radius: var(--radius-full);
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: var(--font-size-sm);
    flex-shrink: 0
}
.activity-icon.sale {
    background: var(--success-50);
    color: var(--success-500)
}
.activity-icon.purchase {
    background: var(--info-50);
    color: var(--info-500)
}
.activity-icon.payment {
    background: var(--warning-50);
    color: var(--warning-500)
}
.activity-icon.entry {
    background: var(--primary-50);
    color: var(--primary-500)
}
.activity-content {
    flex: 1;
    min-width: 0
}
.activity-title {
    font-size: var(--font-size-sm);
    font-weight: 500;
    color: var(--text-primary);
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis
}
.activity-desc {
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
    margin-top: var(--space-1)
}
.activity-time {
    font-size: var(--font-size-xs);
    color: var(--text-tertiary);
    white-space: nowrap
}
.quick-actions {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));
```

```
    gap: var(--space-3)
}
.quick-action-btn {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: var(--space-2);
    padding: var(--space-4);
    background: var(--bg-secondary);
    border: 1px solid var(--border-light);
    border-radius: var(--radius-lg);
    cursor: pointer;
    transition: all var(--transition-fast);
    text-decoration: none;
    color: var(--text-primary)
}
.quick-action-btn:hover {
    border-color: var(--primary-500);
    background: var(--primary-50);
    transform: translateY(-2px)
}
.quick-action-btn i {
    font-size: var(--font-size-2xl);
    color: var(--primary-500)
}
.quick-action-btn span {
    font-size: var(--font-size-xs);
    font-weight: 500;
    text-align: center
}
.financial-summary {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: var(--space-4)
}
.summary-item {
    text-align: center;
    padding: var(--space-4)
}
.summary-item:not(:last-child) {
    border-right: 1px solid var(--border-light)
}
.summary-value {
    font-size: var(--font-size-xl);
    font-weight: 700;
    color: var(--text-primary)
}
.summary-value.positive {
    color: var(--success-500)
}
.summary-value.negative {
    color: var(--error-500)
}
.summary-label {
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
    margin-top: var(--space-1)
}
.period-selector {
    display: flex;
    gap: var(--space-1);
    background: var(--neutral-100);
    border-radius: var(--radius-md);
    padding: var(--space-1)
}
.period-btn {
    padding: var(--space-1) var(--space-3);
    background: transparent;
    border: none;
    border-radius: var(--radius-sm);
    font-size: var(--font-size-xs);
    font-weight: 500;
    color: var(--text-secondary);
    cursor: pointer;
    transition: all var(--transition-fast)
}
.period-btn:hover {
    color: var(--text-primary)
}
.period-btn.active {
    background: var(--bg-secondary);
```

```
    color: var(--primary-500);
    box-shadow: var(--shadow-sm)
}
.progress-bar {
    height: 8px;
    background: var(--neutral-200);
    border-radius: var(--radius-full);
    overflow: hidden
}
.progress-fill {
    height: 100%;
    border-radius: var(--radius-full);
    transition: width var(--transition-normal)
}
.progress-fill.primary {
    background: linear-gradient(90deg, var(--primary-400), var(--primary-600))
}
.progress-fill.success {
    background: linear-gradient(90deg, #4caf50, var(--success-500))
}
.progress-fill.warning {
    background: linear-gradient(90deg, #ff9800, var(--warning-500))
}
.progress-fill.error {
    background: linear-gradient(90deg, #f44336, var(--error-500))
}
.welcome-banner {
    background: linear-gradient(135deg, var(--primary-600), var(--primary-800));
    color: #fff;
    border-radius: var(--radius-xl);
    padding: var(--space-6);
    margin-bottom: var(--space-4);
    position: relative;
    overflow: hidden
}
.welcome-banner::before {
    content: '';
    position: absolute;
    top: -50%;
    right: -10%;
    width: 300px;
    height: 300px;
    background: rgba(255, 255, 255, .1);
    border-radius: 50%
}
.welcome-banner::after {
    content: '';
    position: absolute;
    bottom: -30%;
    right: 10%;
    width: 200px;
    height: 200px;
    background: rgba(255, 255, 255, .05);
    border-radius: 50%
}
.welcome-content {
    position: relative;
    z-index: 1
}
.welcome-title {
    font-size: var(--font-size-2xl);
    font-weight: 700;
    margin-bottom: var(--space-2)
}
.welcome-subtitle {
    font-size: var(--font-size-base);
    opacity: .9
}
.welcome-actions {
    margin-top: var(--space-4);
    display: flex;
    gap: var(--space-3)
}
.welcome-btn {
    padding: var(--space-2) var(--space-4);
    background: rgba(255, 255, 255, .2);
    border: 1px solid rgba(255, 255, 255, .3);
    border-radius: var(--radius-md);
    color: #fff;
    font-weight: 500;
    cursor: pointer;
```

```
    transition: all var(--transition-fast);
    text-decoration: none;
    font-size: var(--font-size-sm);
    display: inline-flex;
    align-items: center;
    gap: var(--space-2)
}
.welcome-btn:hover {
    background: rgba(255, 255, 255, .3)
}
.welcome-btn.primary {
    background: #fff;
    color: var(--primary-600);
    border-color: #fff
}
.welcome-btn.primary:hover {
    background: rgba(255, 255, 255, .9)
}
.balance-overview {
    display: flex;
    align-items: center;
    gap: var(--space-6)
}
.balance-chart-container {
    width: 200px;
    height: 200px;
    position: relative
}
.balance-center {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    text-align: center
}
.balance-center-value {
    font-size: var(--font-size-xl);
    font-weight: 700;
    color: var(--text-primary)
}
.balance-center-label {
    font-size: var(--font-size-xs);
    color: var(--text-secondary)
}
.balance-details {
    flex: 1
}
.balance-item {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: var(--space-2) 0;
    border-bottom: 1px dashed var(--border-light)
}
.balance-item:last-child {
    border-bottom: none
}
.balance-item-label {
    display: flex;
    align-items: center;
    gap: var(--space-2);
    font-size: var(--font-size-sm);
    color: var(--text-secondary)
}
.balance-item-dot {
    width: 10px;
    height: 10px;
    border-radius: var(--radius-full)
}
.balance-item-value {
    font-size: var(--font-size-sm);
    font-weight: 600;
    color: var(--text-primary)
}
.document-status-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(120px, 1fr));
    gap: var(--space-3)
}
.doc-status-item {
    text-align: center;
```

```
    padding: var(--space-3);
    background: var(--neutral-50);
    border-radius: var(--radius-lg)
}
.doc-status-count {
    font-size: var(--font-size-xl);
    font-weight: 700;
    color: var(--text-primary)
}
.doc-status-label {
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
    margin-top: var(--space-1)
}
.doc-status-badge {
    display: inline-block;
    padding: var(--space-1) var(--space-2);
    border-radius: var(--radius-sm);
    font-size: 10px;
    font-weight: 600;
    margin-top: var(--space-2)
}
.authorship-banner {
    background: linear-gradient(135deg, #1a1a2e 0%, #16213e 50%, #0f3460 100%);
    border-radius: var(--radius-xl);
    overflow: hidden;
    position: relative;
    box-shadow: 0 10px 40px rgba(0, 0, 0, 0.15);
}
.authorship-banner::before {
    content: '';
    position: absolute;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background:
        radial-gradient(circle at 20% 80%, rgba(120, 119, 198, 0.15) 0%, transparent 50%),
        radial-gradient(circle at 80% 20%, rgba(255, 119, 115, 0.1) 0%, transparent 50%);
    pointer-events: none;
}
.authorship-content {
    display: flex;
    align-items: center;
    gap: var(--space-6);
    padding: var(--space-6);
    position: relative;
    z-index: 1;
}
.authorship-logo {
    width: 70px;
    height: 70px;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    border-radius: var(--radius-xl);
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
    box-shadow: 0 8px 25px rgba(102, 126, 234, 0.4);
}
.authorship-logo i {
    font-size: 2rem;
    color: white;
}
.authorship-info {
    flex: 1;
}
.authorship-title {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    margin-bottom: var(--space-1);
}
.authorship-title .system-name {
    font-size: var(--font-size-2xl);
    font-weight: 800;
    background: linear-gradient(90deg, #fff, #a0c4ff);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
}
```

```
.authorship-title .version-badge {
    background: rgba(255, 255, 255, 0.15);
    color: #a0c4ff;
    padding: 4px 12px;
    border-radius: var(--radius-full);
    font-size: 0.7rem;
    font-weight: 600;
    border: 1px solid rgba(255, 255, 255, 0.2);
}
.authorship-subtitle {
    color: rgba(255, 255, 255, 0.7);
    font-size: var(--font-size-sm);
    margin-bottom: var(--space-3);
}
.authorship-credits {
    display: flex;
    flex-wrap: wrap;
    gap: var(--space-4);
}
.credit-line {
    display: flex;
    align-items: center;
    gap: var(--space-2);
    color: rgba(255, 255, 255, 0.85);
    font-size: var(--font-size-sm);
}
.credit-line i {
    color: #667eea;
    font-size: 0.9rem;
}
.credit-line strong {
    color: #a0c4ff;
}
.authorship-badge {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-4);
    background: rgba(255, 255, 255, 0.08);
    border-radius: var(--radius-lg);
    border: 1px solid rgba(255, 255, 255, 0.1);
    flex-shrink: 0;
}
.institution-logo {
    width: 50px;
    height: 50px;
    background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);
    border-radius: var(--radius-lg);
    display: flex;
    align-items: center;
    justify-content: center;
}
.institution-logo i {
    font-size: 1.5rem;
    color: white;
}
.institution-text {
    color: white;
}
.institution-name {
    font-weight: 600;
    font-size: var(--font-size-sm);
}
.institution-dept {
    color: rgba(255, 255, 255, 0.6);
    font-size: var(--font-size-xs);
}
.authorship-footer {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: var(--space-3);
    padding: var(--space-3) var(--space-6);
    background: rgba(0, 0, 0, 0.2);
    border-top: 1px solid rgba(255, 255, 255, 0.08);
    font-size: var(--font-size-xs);
    color: rgba(255, 255, 255, 0.5);
}
.authorship-footer .separator {
    opacity: 0.3;
}
```

```
@media (max-width: 1024px) {
    .authorship-content {
        flex-wrap: wrap;
    }
    .authorship-badge {
        width: 100%;
        justify-content: center;
    }
}
@media (max-width: 768px) {
    .authorship-content {
        flex-direction: column;
        text-align: center;
    }
    .authorship-credits {
        justify-content: center;
    }
    .authorship-footer {
        flex-direction: column;
        gap: var(--space-1);
    }
    .authorship-footer .separator {
        display: none;
    }
}
```

Archivo: css\main.css

```
:root {
    --primary-50: #e3f2fd;
    --primary-100: #bbdefb;
    --primary-200: #90caf9;
    --primary-300: #64b5f6;
    --primary-400: #42a5f5;
    --primary-500: #0a6ed1;
    --primary-600: #0854a0;
    --primary-700: #064280;
    --primary-800: #043260;
    --secondary-500: #e97500;
    --secondary-600: #cc6600;
    --success-50: #e8f5e9;
    --success-500: #107e3e;
    --success-600: #0d6630;
    --warning-50: #fff8e1;
    --warning-500: #df6e0c;
    --warning-600: #c45f0a;
    --error-50: #ffebee;
    --error-500: #bb0000;
    --error-600: #990000;
    --info-50: #e1f5fe;
    --info-500: #0854a0;
    --neutral-0: #fff;
    --neutral-50: #fafafa;
    --neutral-100: #f5f5f5;
    --neutral-200: #eee;
    --neutral-300: #e0e0e0;
    --neutral-400: #bdbdbd;
    --neutral-500: #9e9e9e;
    --neutral-600: #757575;
    --neutral-700: #616161;
    --neutral-800: #424242;
    --neutral-900: #212121;
    --bg-primary: #f7f7f7;
    --bg-secondary: #fff;
    --bg-tertiary: #fafafa;
    --bg-sidebar: #1a2035;
    --bg-sidebar-hover: #252d44;
    --bg-sidebar-active: #0a6ed1;
    --text-primary: #32363a;
    --text-secondary: #6a6d70;
    --text-tertiary: #89919a;
    --text-inverse: #fff;
    --text-link: #0a6ed1;
    --border-light: #e5e5e5;
    --border-medium: #d9d9d9;
```

```
    --shadow-sm: 0 1px 3px rgba(0, 0, 0, .1);
    --shadow-md: 0 4px 6px rgba(0, 0, 0, .1);
    --shadow-lg: 0 10px 15px rgba(0, 0, 0, .1);
    --shadow-xl: 0 20px 25px rgba(0, 0, 0, .1);
    --space-1: .25rem;
    --space-2: .5rem;
    --space-3: .75rem;
    --space-4: 1rem;
    --space-5: 1.25rem;
    --space-6: 1.5rem;
    --space-8: 2rem;
    --font-family: 'Inter', sans-serif;
    --font-size-xs: .75rem;
    --font-size-sm: .875rem;
    --font-size-base: 1rem;
    --font-size-lg: 1.125rem;
    --font-size-xl: 1.25rem;
    --font-size-2xl: 1.5rem;
    --font-size-3xl: 1.875rem;
    --radius-sm: .25rem;
    --radius-md: .375rem;
    --radius-lg: .5rem;
    --radius-xl: .75rem;
    --radius-full: 9999px;
    --transition-fast: 150ms ease;
    --transition-normal: 250ms ease;
    --z-dropdown: 100;
    --z-sticky: 200;
    --z-fixed: 300;
    --z-modal: 500;
    --z-toast: 600;
    --sidebar-width: 260px;
    --sidebar-collapsed-width: 70px;
    --header-height: 60px;
}
[data-theme="dark"] {
    --bg-primary: #121212;
    --bg-secondary: #1e1e1e;
    --bg-tertiary: #2d2d2d;
    --bg-sidebar: #0d0d0d;
    --bg-sidebar-hover: #1a1a1a;
    --text-primary: #e0e0e0;
    --text-secondary: #a0a0a0;
    --border-light: #333;
    --border-medium: #444;
    --neutral-100: #2d2d2d;
    --neutral-200: #3d3d3d;
}
*,
*::before,
*::after {
    box-sizing: border-box;
    margin: 0;
    padding: 0
}
html {
    font-size: 16px;
    scroll-behavior: smooth;
    -webkit-font-smoothing: antialiased
}
body {
    font-family: var(--font-family);
    font-size: var(--font-size-sm);
    color: var(--text-primary);
    background: var(--bg-primary);
    min-height: 100vh;
    overflow-x: hidden
}
.loading-screen {
    position: fixed;
    inset: 0;
    background: linear-gradient(135deg, var(--primary-700), var(--primary-800));
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 9999;
    transition: opacity .35s, visibility .35s
}
.loading-screen.hidden {
    opacity: 0;
    visibility: hidden;
```

```
    pointer-events: none
}
.loading-content {
    text-align: center;
    color: var(--text-inverse)
}
.loading-logo {
    width: 80px;
    height: 80px;
    background: rgba(255, 255, 255, .15);
    border-radius: var(--radius-xl);
    display: flex;
    align-items: center;
    justify-content: center;
    margin: 0 auto var(--space-6);
    animation: pulse 2s ease-in-out infinite
}
.loading-logo i {
    font-size: 2.5rem
}
.loading-content h1 {
    font-size: var(--font-size-3xl);
    font-weight: 700;
    margin-bottom: var(--space-2)
}
.loading-content p {
    font-size: var(--font-size-base);
    opacity: .8;
    margin-bottom: var(--space-6)
}
.loading-bar {
    width: 200px;
    height: 4px;
    background: rgba(255, 255, 255, .2);
    border-radius: var(--radius-full);
    overflow: hidden;
    margin: 0 auto
}
.loading-progress {
    height: 100%;
    background: #fff;
    animation: loadProg 1.5s ease forwards
}
@keyframes loadProg {
    0% {
        width: 0
    }
    50% {
        width: 70%
    }
    100% {
        width: 100%
    }
}
@keyframes pulse {
    0%,
    100% {
        box-shadow: 0 0 0 0 rgba(255, 255, 255, .4);
        transform: scale(1)
    }
    50% {
        box-shadow: 0 0 20px 10px rgba(255, 255, 255, .1);
        transform: scale(1.05)
    }
}
.app-container {
    display: flex;
    min-height: 100vh;
    opacity: 1;
    transition: opacity var(--transition-normal)
}
.app-container.hidden {
    opacity: 0
}
.sidebar {
    width: var(--sidebar-width);
    height: 100vh;
    background: var(--bg-sidebar);
    color: var(--text-inverse);
    position: fixed;
    left: 0;
```

```
    top: 0;
    display: flex;
    flex-direction: column;
    z-index: var(--z-fixed);
    transition: width var(--transition-normal);
    overflow: hidden
}
.sidebar.collapsed {
    width: var(--sidebar-collapsed-width)
}
.sidebar-header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: var(--space-4) var(--space-5);
    border-bottom: 1px solid rgba(255, 255, 255, .1);
    min-height: var(--header-height)
}
.logo {
    display: flex;
    align-items: center;
    gap: var(--space-3)
}
.logo i {
    font-size: 1.5rem;
    color: var(--primary-400)
}
.logo span {
    font-size: var(--font-size-xl);
    font-weight: 700;
    white-space: nowrap
}
.sidebar.collapsed .logo span {
    display: none
}
.sidebar-toggle {
    background: transparent;
    border: none;
    color: var(--text-inverse);
    font-size: var(--font-size-lg);
    cursor: pointer;
    padding: var(--space-2);
    border-radius: var(--radius-md);
    transition: background var(--transition-fast)
}
.sidebar-toggle:hover {
    background: rgba(255, 255, 255, .1)
}
.company-selector {
    padding: var(--space-4) var(--space-5);
    border-bottom: 1px solid rgba(255, 255, 255, .1)
}
.current-company {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-3) var(--space-4);
    background: rgba(255, 255, 255, .08);
    border-radius: var(--radius-lg);
    cursor: pointer;
    transition: background var(--transition-fast)
}
.current-company:hover {
    background: rgba(255, 255, 255, .12)
}
.current-company i:first-child {
    color: var(--secondary-500)
}
.current-company span {
    flex: 1;
    font-size: var(--font-size-sm);
    font-weight: 500;
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis
}
.current-company i:last-child {
    font-size: var(--font-size-xs);
    opacity: .7
}
.sidebar.collapsed .company-selector {
```

```
    padding: var(--space-2)
}
.sidebar.collapsed .current-company span,
.sidebar.collapsed .current-company i:last-child {
    display: none
}
.sidebar-nav {
    flex: 1;
    overflow-y: auto;
    padding: var(--space-4) 0
}
.nav-list {
    list-style: none
}
.nav-item {
    margin-bottom: var(--space-1)
}
.nav-link {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-3) var(--space-5);
    color: rgba(255, 255, 255, .7);
    text-decoration: none;
    font-size: var(--font-size-sm);
    transition: all var(--transition-fast);
    position: relative
}
.nav-link:hover {
    color: var(--text-inverse);
    background: var(--bg-sidebar-hover)
}
.nav-link.active {
    color: var(--text-inverse);
    background: var(--bg-sidebar-active)
}
.nav-link.active::before {
    content: '';
    position: absolute;
    left: 0;
    top: 0;
    bottom: 0;
    width: 3px;
    background: #fff;
    border-radius: 0 var(--radius-sm) var(--radius-sm) 0
}
.nav-link i:first-child {
    width: 20px;
    text-align: center;
    font-size: var(--font-size-base)
}
.nav-link span {
    flex: 1;
    white-space: nowrap
}
.submenu-arrow {
    font-size: var(--font-size-xs);
    transition: transform var(--transition-fast)
}
.nav-item.has-submenu.open .submenu-arrow {
    transform: rotate(90deg)
}
.sidebar.collapsed .nav-link span,
.sidebar.collapsed .submenu-arrow {
    display: none
}
.submenu {
    list-style: none;
    max-height: 0;
    overflow: hidden;
    transition: max-height var(--transition-normal);
    background: rgba(0, 0, 0, .15)
}
.nav-item.has-submenu.open .submenu,
.nav-item.has-submenu.expanded .submenu {
    max-height: 500px
}
.submenu-item a,
.submenu li a {
    display: block;
    padding: var(--space-2) var(--space-5);
```

```
    padding-left: calc(var(--space-5) + 32px);
    color: rgba(255, 255, 255, .6);
    text-decoration: none;
    font-size: var(--font-size-sm);
    transition: all var(--transition-fast)
}
.submenu-item a:hover,
.submenu li a:hover {
    color: var(--text-inverse);
    background: rgba(255, 255, 255, .05)
}
.submenu-item.active a,
.submenu li.active a {
    color: var(--primary-300);
    background: rgba(255, 255, 255, .08)
}
.sidebar.collapsed .submenu {
    display: none
}
.nav-item.disabled-module .nav-link {
    opacity: 0.5;
    cursor: not-allowed;
    pointer-events: none;
}
.nav-item.disabled-module .nav-link.disabled {
    color: rgba(255, 255, 255, 0.4);
}
.nav-item.disabled-module:hover .nav-link {
    background: transparent;
}
.module-badge {
    font-size: 0.65rem;
    padding: 2px 6px;
    border-radius: var(--radius-full);
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.5px;
    margin-left: auto;
}
.module-badge.etapa2 {
    background: linear-gradient(135deg, #6366f1, #8b5cf6);
    color: white;
    box-shadow: 0 2px 4px rgba(99, 102, 241, 0.3);
}
.module-badge.nuevo {
    background: linear-gradient(135deg, #10b981, #059669);
    color: white;
}
.module-badge.beta {
    background: linear-gradient(135deg, #f59e0b, #d97706);
    color: white;
}
.nav-item.disabled-module {
    position: relative;
}
.nav-item.disabled-module::after {
    content: 'Disponible en la próxima versión';
    position: absolute;
    left: 100%;
    top: 50%;
    transform: translateY(-50%);
    background: var(--neutral-800);
    color: white;
    padding: 8px 12px;
    border-radius: var(--radius-md);
    font-size: 0.75rem;
    white-space: nowrap;
    opacity: 0;
    visibility: hidden;
    transition: all var(--transition-fast);
    z-index: 1000;
    pointer-events: none;
    margin-left: 10px;
}
.nav-item.disabled-module:hover::after {
    opacity: 1;
    visibility: visible;
}
.sidebar.collapsed .nav-item.disabled-module::after {
    left: var(--sidebar-collapsed-width);
}
```

```
.sidebar.collapsed .module-badge {
    display: none;
}
.sidebar-footer {
    padding: var(--space-4) var(--space-5);
    border-top: 1px solid rgba(255, 255, 255, .1)
}
.user-info {
    display: flex;
    align-items: center;
    gap: var(--space-3)
}
.user-avatar {
    width: 40px;
    height: 40px;
    background: linear-gradient(135deg, var(--primary-500), var(--primary-700));
    border-radius: var(--radius-full);
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0
}
.user-avatar i {
    font-size: var(--font-size-lg)
}
.user-details {
    display: flex;
    flex-direction: column;
    min-width: 0
}
.user-name {
    font-size: var(--font-size-sm);
    font-weight: 500;
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis
}
.user-role {
    font-size: var(--font-size-xs);
    color: rgba(255, 255, 255, .6)
}
.sidebar.collapsed .user-details {
    display: none
}
.main-content {
    flex: 1;
    margin-left: var(--sidebar-width);
    display: flex;
    flex-direction: column;
    min-height: 100vh;
    transition: margin-left var(--transition-normal)
}
.sidebar.collapsed~.main-content {
    margin-left: var(--sidebar-collapsed-width)
}
.top-header {
    height: var(--header-height);
    background: var(--bg-secondary);
    border-bottom: 1px solid var(--border-light);
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 0 var(--space-6);
    position: sticky;
    top: 0;
    z-index: var(--z-sticky);
    box-shadow: var(--shadow-sm)
}
.header-left,
.header-right {
    display: flex;
    align-items: center;
    gap: var(--space-4)
}
.breadcrumb,
.breadcrumbs {
    display: flex;
    align-items: center;
    gap: var(--space-2);
    font-size: var(--font-size-sm);
    color: var(--text-secondary)
```

```
}
.breadcrumb-item {
    color: var(--text-secondary);
    text-decoration: none
}
.breadcrumb-item:hover {
    color: var(--text-primary)
}
.breadcrumb-separator {
    color: var(--text-tertiary);
    margin: 0 var(--space-1)
}
.breadcrumb-item.active,
.breadcrumb-item:last-child {
    color: var(--text-primary);
    font-weight: 500
}
.header-actions {
    display: flex;
    align-items: center;
    gap: var(--space-2)
}
.header-btn {
    width: 36px;
    height: 36px;
    background: transparent;
    border: none;
    border-radius: var(--radius-md);
    color: var(--text-secondary);
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    position: relative;
    transition: all var(--transition-fast)
}
.header-btn:hover {
    background: var(--neutral-100);
    color: var(--text-primary)
}
.notification-badge {
    position: absolute;
    top: 4px;
    right: 4px;
    width: 16px;
    height: 16px;
    background: var(--error-500);
    color: #fff;
    font-size: 10px;
    font-weight: 600;
    border-radius: var(--radius-full);
    display: flex;
    align-items: center;
    justify-content: center
}
.dropdown {
    position: relative
}
.dropdown-menu {
    position: absolute;
    top: 100%;
    right: 0;
    margin-top: var(--space-2);
    background: var(--bg-secondary);
    border: 1px solid var(--border-light);
    border-radius: var(--radius-lg);
    box-shadow: var(--shadow-lg);
    min-width: 180px;
    opacity: 0;
    visibility: hidden;
    transform: translateY(-10px);
    transition: all var(--transition-fast);
    z-index: var(--z-dropdown)
}
.dropdown:hover .dropdown-menu,
.dropdown.open .dropdown-menu,
.dropdown-menu.show {
    opacity: 1;
    visibility: visible;
    transform: translateY(0)
}
```

```
.dropdown-menu a {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-3) var(--space-4);
    color: var(--text-primary);
    text-decoration: none;
    font-size: var(--font-size-sm);
    transition: background var(--transition-fast)
}
.dropdown-menu a:hover {
    background: var(--neutral-100)
}
.dropdown-menu a i {
    width: 16px;
    color: var(--text-secondary)
}
.content-wrapper {
    flex: 1;
    padding: var(--space-6);
    max-width: 1400px;
    width: 100%;
    margin: 0 auto
}
::-webkit-scrollbar {
    width: 8px;
    height: 8px
}
::-webkit-scrollbar-track {
    background: var(--neutral-100)
}
::-webkit-scrollbar-thumb {
    background: var(--neutral-400);
    border-radius: var(--radius-full)
}
::-webkit-scrollbar-thumb:hover {
    background: var(--neutral-500)
}
.hidden {
    display: none !important
}
@media(max-width:1024px) {
    .sidebar {
        transform: translateX(-100%)
    }
    .sidebar.open {
        transform: translateX(0)
    }
    .main-content,
    .sidebar.collapsed~.main-content {
        margin-left: 0
    }
}
@media(max-width:768px) {
    .content-wrapper {
        padding: var(--space-4)
    }
}
```

Archivo: css\system-flow-fix.css

```
.flow-diagram-container {
    display: grid;
    grid-template-columns: 1fr 300px;
    gap: var(--space-4);
    height: auto;
    max-height: none;
    overflow-y: auto;
}
.flow-legend {
    grid-column: 1 / -1;
    background: linear-gradient(135deg, var(--primary-50), var(--info-50));
    padding: var(--space-3) var(--space-4);
    border-radius: var(--radius-lg);
    margin-bottom: var(--space-2);
```

```
}
.flow-legend h4 {
    margin: 0 0 var(--space-1);
    font-size: var(--font-size-sm);
    color: var(--primary-700);
}
.flow-legend p {
    margin: 0 0 var(--space-2);
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
}
.legend-items {
    display: flex;
    gap: var(--space-4);
    font-size: var(--font-size-xs);
}
.legend-items span {
    display: flex;
    align-items: center;
    gap: var(--space-2);
}
.flow-diagram {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: var(--space-2);
    overflow-y: auto;
    padding: var(--space-2);
}
.flow-level {
    display: flex;
    justify-content: center;
    gap: var(--space-4);
    width: 100%;
}
.flow-node {
    background: var(--bg-secondary);
    border: 2px solid var(--border-light);
    border-radius: var(--radius-xl);
    padding: var(--space-3);
    display: flex;
    align-items: center;
    gap: var(--space-3);
    cursor: pointer;
    transition: all var(--transition-fast);
    position: relative;
    min-width: 180px;
}
.flow-node:hover {
    border-color: var(--primary-500);
    transform: translateY(-2px);
    box-shadow: var(--shadow-lg);
}
.flow-node.selected {
    border-color: var(--primary-500);
    background: var(--primary-50);
    box-shadow: 0 0 0 4px rgba(10, 110, 209, 0.2);
}
.node-icon {
    width: 40px;
    height: 40px;
    border-radius: var(--radius-lg);
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
    font-size: var(--font-size-lg);
    flex-shrink: 0;
}
.node-icon.mm {
    background: linear-gradient(135deg, #df6e0c, #c45f0a);
}
.node-icon.sd {
    background: linear-gradient(135deg, #107e3e, #0d6630);
}
.node-icon.wm {
    background: linear-gradient(135deg, #00b894, #00a187);
}
.node-icon.fi {
    background: linear-gradient(135deg, #6c5ce7, #5f3dc4);
}
```

```
.node-icon.hr {
    background: linear-gradient(135deg, #fd79a8, #e84393);
}
.node-icon.gl {
    background: linear-gradient(135deg, #0a6ed1, #064280);
}
.node-icon.reports {
    background: linear-gradient(135deg, #00cec9, #00b5ad);
}
.node-icon.co {
    background: linear-gradient(135deg, #fdcb6e, #f39c12);
}
.node-icon.trace {
    background: linear-gradient(135deg, #a29bfe, #6c5ce7);
}
.node-content h4 {
    margin: 0;
    font-size: var(--font-size-sm);
    font-weight: 600;
}
.node-content p {
    margin: var(--space-1) 0 0;
    font-size: var(--font-size-xs);
    color: var(--text-tertiary);
    line-height: 1.3;
}
.node-badge {
    position: absolute;
    top: -8px;
    right: -8px;
    background: var(--neutral-700);
    color: white;
    font-size: 10px;
    padding: 2px 8px;
    border-radius: var(--radius-full);
}
.core-node {
    background: linear-gradient(135deg, var(--primary-50), var(--primary-100));
    border-color: var(--primary-500);
    border-width: 3px;
    min-width: 350px;
}
.core-node .node-icon {
    width: 50px;
    height: 50px;
    font-size: var(--font-size-xl);
}
.core-badge {
    display: inline-block;
    background: var(--primary-500);
    color: white;
    font-size: 10px;
    padding: 2px 8px;
    border-radius: var(--radius-full);
    margin-top: var(--space-2);
}
.output-node {
    border-style: dashed;
}
.entry-point .node-icon {
    background: linear-gradient(135deg, var(--neutral-600), var(--neutral-800));
}
.flow-arrows-row {
    display: flex;
    justify-content: center;
    padding: var(--space-2) 0;
}
.arrow-down {
    color: var(--primary-500);
    font-size: var(--font-size-xl);
    animation: bounce 1s infinite;
}
@keyframes bounce {
    0%,
    100% {
        transform: translateY(0);
    }
    50% {
        transform: translateY(4px);
    }
}
```

```
.arrow-connector {
    width: 100%;
    max-width: 800px;
}
.connector-svg {
    width: 100%;
    height: 60px;
}
.arrow-connector.all-to-one .connector-svg {
    height: 80px;
}
.flow-details {
    background: var(--bg-tertiary);
    border-radius: var(--radius-xl);
    padding: var(--space-4);
    overflow-y: auto;
}
.flow-details h4 {
    margin: 0 0 var(--space-2);
    color: var(--text-tertiary);
    font-size: var(--font-size-sm);
}
.flow-details.active h4 {
    color: var(--primary-600);
}
.module-detail {
    animation: fadeIn 0.3s ease;
}
.detail-header {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    margin-bottom: var(--space-3);
}
.detail-icon {
    width: 48px;
    height: 48px;
    background: var(--primary-500);
    border-radius: var(--radius-lg);
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
    font-size: var(--font-size-xl);
}
.detail-header h4 {
    margin: 0;
    font-size: var(--font-size-base);
    color: var(--text-primary);
}
.detail-description {
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
    margin-bottom: var(--space-4);
}
.detail-section {
    margin-bottom: var(--space-4);
}
.detail-section h5 {
    margin: 0 0 var(--space-2);
    font-size: var(--font-size-xs);
    color: var(--text-tertiary);
    text-transform: uppercase;
    letter-spacing: 0.5px;
}
.detail-section ul {
    margin: 0;
    padding-left: var(--space-4);
    font-size: var(--font-size-sm);
}
.detail-section li {
    margin-bottom: var(--space-1);
}
.accounting-section {
    background: var(--bg-secondary);
    padding: var(--space-3);
    border-radius: var(--radius-lg);
    border-left: 4px solid var(--success-500);
}
.accounting-example {
    width: 100%;
```

```
    font-size: var(--font-size-xs);
    border-collapse: collapse;
}
.accounting-example th {
    text-align: left;
    padding: var(--space-2);
    background: var(--neutral-100);
    font-weight: 600;
}
.accounting-example td {
    padding: var(--space-1) var(--space-2);
    border-bottom: 1px solid var(--border-light);
}
.accounting-example .debe {
    color: var(--info-600);
    font-weight: 600;
}
.accounting-example .haber {
    color: var(--success-600);
    font-weight: 600;
}
.accounting-example .separator {
    height: 10px;
    border: none;
}
.central-module-info,
.reports-info {
    font-size: var(--font-size-sm);
}
.equation {
    text-align: center;
    font-size: var(--font-size-lg);
    font-weight: 700;
    color: var(--primary-600);
    padding: var(--space-3);
    background: var(--primary-50);
    border-radius: var(--radius-lg);
    margin: var(--space-3) 0;
}
.reports-info ul {
    padding-left: var(--space-4);
}
.traceability-info ul {
    padding-left: var(--space-4);
    font-size: var(--font-size-sm);
}
@keyframes fadeIn {
    from {
        opacity: 0;
        transform: translateY(10px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}
@media(max-width: 900px) {
    .flow-diagram-container {
        grid-template-columns: 1fr;
    }
    .flow-level.operations,
    .flow-level.support {
        flex-direction: column;
        align-items: center;
    }
    .connector-svg {
        display: none;
    }
    .flow-arrows-row.complex,
    .flow-arrows-row.final {
        display: none;
    }
}
```

Archivo: css\trazabilidad.css

```
.trazabilidad-module .module-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: var(--space-6);
    margin-bottom: var(--space-6);
    background: linear-gradient(135deg, var(--primary-600) 0%, var(--primary-800) 100%);
    border-radius: var(--radius-xl);
    color: white;
}
.trazabilidad-module .header-content {
    display: flex;
    align-items: center;
    gap: var(--space-4);
}
.trazabilidad-module .header-icon {
    width: 60px;
    height: 60px;
    border-radius: var(--radius-lg);
    background: rgba(255, 255, 255, 0.2);
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1.75rem;
}
.trazabilidad-module .header-text h1 {
    font-size: var(--font-size-2xl);
    font-weight: 700;
    margin: 0;
}
.trazabilidad-module .header-text p {
    margin: var(--space-1) 0 0 0;
    opacity: 0.9;
    font-size: var(--font-size-sm);
}
.trazabilidad-grid {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: var(--space-4);
}
.trazabilidad-grid .span-2 {
    grid-column: span 2;
}
.trazabilidad-grid .span-4 {
    grid-column: span 4;
}
@media (max-width: 1200px) {
    .trazabilidad-grid {
        grid-template-columns: repeat(2, 1fr);
    }
    .trazabilidad-grid .span-4 {
        grid-column: span 2;
    }
}
@media (max-width: 768px) {
    .trazabilidad-grid {
        grid-template-columns: 1fr;
    }
    .trazabilidad-grid .span-2,
    .trazabilidad-grid .span-4 {
        grid-column: span 1;
    }
}
.progress-overview {
    display: flex;
    gap: var(--space-6);
    align-items: flex-start;
}
.progress-ring {
    position: relative;
    width: 140px;
    height: 140px;
    border-radius: 50%;
    background: conic-gradient(var(--primary-500) calc(var(--progress) * 3.6deg),
            var(--border-color) calc(var(--progress) * 3.6deg));
    display: flex;
```

```
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
}
.progress-ring-inner {
    width: 110px;
    height: 110px;
    border-radius: 50%;
    background: var(--bg-primary);
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
}
.progress-ring-inner .progress-value {
    font-size: var(--font-size-xl);
    font-weight: 700;
    color: var(--text-primary);
}
.progress-ring-inner .progress-label {
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
}
.progress-steps {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: var(--space-2);
}
.progress-step {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-2) var(--space-3);
    border-radius: var(--radius-md);
    cursor: pointer;
    transition: all 0.2s;
    border: 1px solid transparent;
}
.progress-step:hover {
    background: var(--bg-secondary);
    border-color: var(--border-color);
}
.progress-step.completed {
    opacity: 0.8;
}
.progress-step .step-indicator {
    width: 28px;
    height: 28px;
    border-radius: 50%;
    background: var(--bg-tertiary);
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: var(--font-size-sm);
    font-weight: 600;
    color: var(--text-secondary);
    flex-shrink: 0;
}
.progress-step.completed .step-indicator {
    background: var(--success-100);
    color: var(--success-600);
}
.progress-step .step-content {
    flex: 1;
    min-width: 0;
}
.progress-step .step-name {
    font-weight: 500;
    font-size: var(--font-size-sm);
    color: var(--text-primary);
}
.progress-step .step-desc {
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
    white-space: nowrap;
    overflow: hidden;
    text-overflow: ellipsis;
}
.progress-step .step-arrow {
    color: var(--text-tertiary);
```

```
    font-size: var(--font-size-xs);
}
.integrity-healthy {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    padding: var(--space-8);
    text-align: center;
}
.integrity-healthy i {
    font-size: 3rem;
    color: var(--success-500);
    margin-bottom: var(--space-3);
}
.integrity-healthy p {
    color: var(--text-secondary);
    margin: 0;
}
.integrity-issues {
    display: flex;
    flex-direction: column;
    gap: var(--space-2);
}
.issue-item {
    display: flex;
    align-items: center;
    gap: var(--space-2);
    padding: var(--space-2) var(--space-3);
    border-radius: var(--radius-md);
    font-size: var(--font-size-sm);
}
.issue-item.error {
    background: var(--error-50);
    color: var(--error-700);
}
.issue-item.warning {
    background: var(--warning-50);
    color: var(--warning-700);
}
.issue-item i {
    flex-shrink: 0;
}
.empty-transactions {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    padding: var(--space-8);
    text-align: center;
    color: var(--text-secondary);
}
.empty-transactions i {
    font-size: 2.5rem;
    margin-bottom: var(--space-3);
    opacity: 0.5;
}
.transactions-list {
    display: flex;
    flex-direction: column;
    gap: var(--space-2);
}
.transaction-item {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-3);
    border-radius: var(--radius-md);
    cursor: pointer;
    transition: all 0.2s;
    border: 1px solid var(--border-color);
}
.transaction-item:hover {
    background: var(--bg-secondary);
    border-color: var(--primary-300);
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}
.transaction-item .tx-icon {
    width: 40px;
    height: 40px;
    border-radius: var(--radius-md);
```

```
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1rem;
    flex-shrink: 0;
}
.transaction-item .tx-content {
    flex: 1;
    min-width: 0;
}
.transaction-item .tx-title {
    font-weight: 500;
    font-size: var(--font-size-sm);
    color: var(--text-primary);
}
.transaction-item .tx-number {
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
    font-family: var(--font-mono);
}
.transaction-item .tx-meta {
    display: flex;
    align-items: center;
    gap: var(--space-2);
}
.transaction-item .tx-module {
    font-size: var(--font-size-xs);
    color: var(--text-tertiary);
    background: var(--bg-tertiary);
    padding: var(--space-1) var(--space-2);
    border-radius: var(--radius-sm);
}
.business-flows {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
    gap: var(--space-4);
}
.flow-card {
    border: 1px solid var(--border-color);
    border-radius: var(--radius-lg);
    padding: var(--space-4);
    cursor: pointer;
    transition: all 0.3s;
    background: var(--bg-primary);
}
.flow-card:hover {
    border-color: var(--primary-400);
    box-shadow: 0 4px 20px rgba(var(--primary-500-rgb), 0.15);
    transform: translateY(-2px);
}
.flow-card .flow-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: var(--space-2);
}
.flow-card .flow-header h4 {
    margin: 0;
    font-size: var(--font-size-md);
    color: var(--text-primary);
}
.flow-card .flow-description {
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
    margin-bottom: var(--space-3);
}
.flow-card .flow-preview {
    display: flex;
    align-items: center;
    gap: 0;
    margin-bottom: var(--space-3);
}
.flow-step-mini {
    display: flex;
    align-items: center;
}
.flow-step-mini .step-dot {
    width: 12px;
    height: 12px;
    border-radius: 50%;
}
```

```
.flow-step-mini .step-line {
    width: 24px;
    height: 2px;
    background: var(--border-color);
}
.flow-card .flow-footer {
    display: flex;
    justify-content: flex-end;
}
.trace-results-modal {
    display: flex;
    flex-direction: column;
    gap: var(--space-6);
}
.trace-summary {
    display: flex;
    flex-wrap: wrap;
    gap: var(--space-4);
    padding: var(--space-4);
    background: var(--bg-secondary);
    border-radius: var(--radius-lg);
}
.trace-summary .summary-stat {
    display: flex;
    flex-direction: column;
    gap: var(--space-1);
}
.trace-summary .stat-label {
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
    text-transform: uppercase;
    letter-spacing: 0.05em;
}
.trace-summary .stat-value {
    font-size: var(--font-size-md);
    font-weight: 600;
    color: var(--text-primary);
}
.trace-flow h4,
.financial-impact h4,
.pedagogical-notes h4,
.trace-timeline h4 {
    display: flex;
    align-items: center;
    gap: var(--space-2);
    font-size: var(--font-size-md);
    margin-bottom: var(--space-3);
    color: var(--text-primary);
}
.flow-visualization {
    display: flex;
    align-items: center;
    flex-wrap: wrap;
    gap: var(--space-2);
    padding: var(--space-4);
    background: var(--bg-secondary);
    border-radius: var(--radius-lg);
    overflow-x: auto;
}
.flow-node {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: var(--space-2);
    padding: var(--space-3);
    background: var(--bg-primary);
    border-radius: var(--radius-md);
    border: 2px solid var(--border-color);
    min-width: 100px;
    text-align: center;
}
.flow-node.origin {
    border-color: var(--primary-500);
}
.flow-node .node-icon {
    width: 36px;
    height: 36px;
    border-radius: var(--radius-md);
    display: flex;
    align-items: center;
    justify-content: center;
```

```
}
.flow-node .node-label {
    font-size: var(--font-size-xs);
    font-weight: 600;
    color: var(--text-primary);
}
.flow-node .node-number {
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
    font-family: var(--font-mono);
}
.flow-node .node-module {
    font-size: 10px;
    color: var(--text-tertiary);
}
.flow-arrow {
    color: var(--text-tertiary);
    font-size: var(--font-size-md);
}
.financial-impact .table {
    width: 100%;
    border-collapse: collapse;
}
.financial-impact .table th,
.financial-impact .table td {
    padding: var(--space-2) var(--space-3);
    border-bottom: 1px solid var(--border-color);
    text-align: left;
}
.financial-impact .table th {
    font-weight: 600;
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
    background: var(--bg-secondary);
}
.financial-impact .table .text-right {
    text-align: right;
}
.financial-impact .table tfoot th {
    background: var(--bg-tertiary);
    color: var(--text-primary);
}
.pedagogical-notes .notes-grid {
    display: grid;
    gap: var(--space-3);
}
.note-card {
    padding: var(--space-4);
    background: linear-gradient(135deg, var(--primary-50) 0%, var(--primary-100) 100%);
    border-radius: var(--radius-md);
    border-left: 4px solid var(--primary-500);
}
.note-card h5 {
    margin: 0 0 var(--space-2) 0;
    font-size: var(--font-size-sm);
    color: var(--primary-700);
}
.note-card p {
    margin: 0;
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
    line-height: 1.5;
}
.trace-timeline .timeline {
    display: flex;
    flex-direction: column;
    gap: var(--space-2);
    padding-left: var(--space-4);
    border-left: 2px solid var(--border-color);
}
.trace-timeline .timeline-item {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    position: relative;
}
.trace-timeline .timeline-dot {
    width: 12px;
    height: 12px;
    border-radius: 50%;
    position: absolute;
```

```
    left: calc(-1 * var(--space-4) - 7px);
}
.trace-timeline .timeline-content {
    display: flex;
    gap: var(--space-3);
    font-size: var(--font-size-sm);
}
.trace-timeline .timeline-date {
    color: var(--text-tertiary);
    min-width: 80px;
}
.trace-timeline .timeline-desc {
    color: var(--text-primary);
}
.validation-rules {
    margin-bottom: var(--space-6);
}
.validation-rules h4 {
    margin-bottom: var(--space-3);
    color: var(--text-primary);
}
.rules-grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
    gap: var(--space-3);
}
.rule-card {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-3);
    background: var(--bg-secondary);
    border-radius: var(--radius-md);
    border: 1px solid var(--border-color);
}
.rule-card .rule-icon {
    width: 40px;
    height: 40px;
    border-radius: var(--radius-md);
    display: flex;
    align-items: center;
    justify-content: center;
    flex-shrink: 0;
}
.rule-card .rule-icon.success {
    background: var(--success-100);
    color: var(--success-600);
}
.rule-card .rule-content {
    flex: 1;
    min-width: 0;
}
.rule-card .rule-content h5 {
    margin: 0;
    font-size: var(--font-size-sm);
    color: var(--text-primary);
}
.rule-card .rule-content p {
    margin: var(--space-1) 0 0 0;
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
}
.rule-card .rule-status {
    font-size: var(--font-size-xs);
    padding: var(--space-1) var(--space-2);
    border-radius: var(--radius-full);
    background: var(--success-100);
    color: var(--success-700);
}
.validation-results {
    margin-top: var(--space-4);
}
.results-placeholder {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    padding: var(--space-8);
    color: var(--text-secondary);
    text-align: center;
}
```

```
.results-placeholder i {
    font-size: 2.5rem;
    margin-bottom: var(--space-3);
    opacity: 0.5;
}
.validation-summary {
    display: flex;
    align-items: center;
    gap: var(--space-4);
    padding: var(--space-4);
    background: var(--bg-secondary);
    border-radius: var(--radius-lg);
    margin-bottom: var(--space-4);
}
.validation-summary .summary-icon {
    width: 60px;
    height: 60px;
    border-radius: 50%;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 1.5rem;
}
.validation-summary .summary-icon.success {
    background: var(--success-100);
    color: var(--success-600);
}
.validation-summary .summary-icon.warning {
    background: var(--warning-100);
    color: var(--warning-600);
}
.validation-summary .summary-text h4 {
    margin: 0;
    color: var(--text-primary);
}
.validation-summary .summary-text p {
    margin: var(--space-1) 0 0 0;
    color: var(--text-secondary);
}
.validation-details {
    display: flex;
    flex-direction: column;
    gap: var(--space-3);
}
.validation-item {
    padding: var(--space-3);
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
}
.validation-item .item-header {
    display: flex;
    gap: var(--space-2);
    margin-bottom: var(--space-2);
}
.validation-item .item-type {
    font-weight: 500;
    color: var(--text-primary);
}
.validation-item .item-number {
    font-family: var(--font-mono);
    color: var(--text-secondary);
}
.validation-item .item-issues {
    display: flex;
    flex-direction: column;
    gap: var(--space-2);
}
.validation-item .issue {
    display: flex;
    align-items: center;
    gap: var(--space-2);
    font-size: var(--font-size-sm);
    padding: var(--space-2);
    border-radius: var(--radius-sm);
}
.validation-item .issue.error {
    background: var(--error-50);
    color: var(--error-700);
}
.validation-item .issue.warning {
    background: var(--warning-50);
```

```
    color: var(--warning-700);
}
.progress-hero {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: var(--space-8);
    padding: var(--space-6);
    background: linear-gradient(135deg, var(--primary-50) 0%, var(--bg-secondary) 100%);
    border-radius: var(--radius-xl);
    margin-bottom: var(--space-6);
}
.progress-circle-large {
    position: relative;
    width: 180px;
    height: 180px;
    border-radius: 50%;
    background: conic-gradient(var(--primary-500) calc(var(--progress) * 3.6deg),
            var(--bg-tertiary) calc(var(--progress) * 3.6deg));
    display: flex;
    align-items: center;
    justify-content: center;
}
.progress-circle-large .circle-content {
    width: 150px;
    height: 150px;
    border-radius: 50%;
    background: var(--bg-primary);
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
}
.progress-circle-large .percentage {
    font-size: 2.5rem;
    font-weight: 700;
    color: var(--primary-600);
}
.progress-circle-large .label {
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
}
.progress-stats {
    display: flex;
    flex-direction: column;
    gap: var(--space-4);
}
.progress-stats .stat {
    text-align: center;
    padding: var(--space-3) var(--space-6);
    background: var(--bg-primary);
    border-radius: var(--radius-md);
    border: 1px solid var(--border-color);
}
.progress-stats .stat-value {
    font-size: var(--font-size-2xl);
    font-weight: 700;
    color: var(--text-primary);
    display: block;
}
.progress-stats .stat-label {
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
}
.progress-timeline {
    position: relative;
    padding-left: var(--space-12);
}
.progress-timeline::before {
    content: '';
    position: absolute;
    left: 20px;
    top: 0;
    bottom: 0;
    width: 2px;
    background: var(--border-color);
}
.progress-timeline .timeline-item {
    position: relative;
    padding-bottom: var(--space-6);
}
```

```
.progress-timeline .timeline-item:last-child {
    padding-bottom: 0;
}
.progress-timeline .timeline-marker {
    position: absolute;
    left: calc(-1 * var(--space-12) + 8px);
    width: 28px;
    height: 28px;
    border-radius: 50%;
    background: var(--bg-tertiary);
    border: 2px solid var(--border-color);
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: var(--font-size-sm);
    font-weight: 600;
    color: var(--text-secondary);
    z-index: 1;
}
.progress-timeline .timeline-item.completed .timeline-marker {
    background: var(--success-500);
    border-color: var(--success-500);
    color: white;
}
.progress-timeline .timeline-item.current .timeline-marker {
    background: var(--primary-500);
    border-color: var(--primary-500);
    color: white;
    animation: pulse 2s infinite;
}
@keyframes pulse {
    0%,
    100% {
        box-shadow: 0 0 0 0 rgba(var(--primary-500-rgb), 0.4);
    }
    50% {
        box-shadow: 0 0 0 8px rgba(var(--primary-500-rgb), 0);
    }
}
.progress-timeline .timeline-connector {
    position: absolute;
    left: calc(-1 * var(--space-12) + 8px + 13px);
    top: 28px;
    width: 2px;
    height: calc(100% - 28px);
    background: var(--border-color);
}
.progress-timeline .timeline-item.completed .timeline-connector {
    background: var(--success-500);
}
.progress-timeline .timeline-content {
    padding: var(--space-3) var(--space-4);
    background: var(--bg-secondary);
    border-radius: var(--radius-md);
    border: 1px solid var(--border-color);
}
.progress-timeline .timeline-item.current .timeline-content {
    border-color: var(--primary-300);
    background: linear-gradient(135deg, var(--primary-50) 0%, var(--bg-secondary) 100%);
}
.progress-timeline .timeline-content h4 {
    margin: 0 0 var(--space-1) 0;
    font-size: var(--font-size-md);
    color: var(--text-primary);
}
.progress-timeline .timeline-content p {
    margin: 0 0 var(--space-3) 0;
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
}
.progress-timeline .completed-badge {
    display: inline-flex;
    align-items: center;
    gap: var(--space-1);
    font-size: var(--font-size-sm);
    color: var(--success-600);
}
.completion-message {
    text-align: center;
    padding: var(--space-8);
    background: linear-gradient(135deg, var(--success-50) 0%, var(--success-100) 100%);
```

```
    border-radius: var(--radius-xl);
    border: 2px solid var(--success-200);
}
.completion-message .completion-icon {
    width: 80px;
    height: 80px;
    border-radius: 50%;
    background: var(--success-500);
    color: white;
    display: flex;
    align-items: center;
    justify-content: center;
    font-size: 2rem;
    margin: 0 auto var(--space-4) auto;
}
.completion-message h3 {
    margin: 0 0 var(--space-2) 0;
    color: var(--success-700);
}
.completion-message p {
    margin: 0 0 var(--space-4) 0;
    color: var(--text-secondary);
    max-width: 500px;
    margin-left: auto;
    margin-right: auto;
}
.flow-selector {
    display: flex;
    gap: var(--space-2);
    margin-bottom: var(--space-4);
}
.flow-tab {
    padding: var(--space-2) var(--space-4);
    background: var(--bg-secondary);
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    font-size: var(--font-size-sm);
    cursor: pointer;
    transition: all 0.2s;
}
.flow-tab:hover {
    background: var(--bg-tertiary);
}
.flow-tab.active {
    background: var(--primary-500);
    border-color: var(--primary-500);
    color: white;
}
.flow-diagram-container {
    min-height: 400px;
    background: var(--bg-secondary);
    border-radius: var(--radius-lg);
    padding: var(--space-4);
}
.flow-placeholder {
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
    height: 300px;
    color: var(--text-secondary);
}
.flow-placeholder i {
    font-size: 3rem;
    margin-bottom: var(--space-3);
    opacity: 0.5;
}
.flow-diagram .flow-header {
    text-align: center;
    margin-bottom: var(--space-6);
}
.flow-diagram .flow-header h3 {
    margin: 0;
    color: var(--text-primary);
}
.flow-diagram .flow-header p {
    margin: var(--space-1) 0 0 0;
    color: var(--text-secondary);
}
.flow-steps-diagram {
    display: flex;
```

```
    flex-direction: column;
    align-items: center;
    gap: 0;
    margin-bottom: var(--space-6);
}
.diagram-step {
    display: flex;
    flex-direction: column;
    align-items: center;
}
.diagram-step .step-box {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-3) var(--space-4);
    background: var(--bg-primary);
    border: 2px solid;
    border-radius: var(--radius-md);
    min-width: 200px;
}
.diagram-step .step-icon {
    width: 40px;
    height: 40px;
    border-radius: var(--radius-md);
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
}
.diagram-step .step-info .step-label {
    display: block;
    font-weight: 500;
    color: var(--text-primary);
}
.diagram-step .step-info .step-module {
    display: block;
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
}
.diagram-step .step-connector {
    display: flex;
    flex-direction: column;
    align-items: center;
    padding: var(--space-2) 0;
}
.diagram-step .connector-line {
    width: 2px;
    height: 16px;
    background: var(--border-color);
}
.diagram-step .connector-arrow {
    color: var(--text-tertiary);
    font-size: var(--font-size-sm);
}
.flow-accounts {
    background: var(--bg-primary);
    border-radius: var(--radius-md);
    padding: var(--space-4);
    border: 1px solid var(--border-color);
}
.flow-accounts h4 {
    margin: 0 0 var(--space-3) 0;
}
.flow-accounts .accounts-list {
    display: flex;
    flex-direction: column;
    gap: var(--space-2);
}
.flow-accounts .account-item {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-2) 0;
    border-bottom: 1px solid var(--border-color);
}
.flow-accounts .account-item:last-child {
    border-bottom: none;
}
.flow-accounts .account-code {
    font-family: var(--font-mono);
    font-size: var(--font-size-sm);
```

```
    color: var(--primary-600);
    min-width: 80px;
}
.flow-accounts .account-name {
    flex: 1;
    color: var(--text-primary);
}
.flow-accounts .account-effect {
    font-size: var(--font-size-sm);
    color: var(--text-secondary);
}
.quick-trace-modal .trace-search {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: var(--space-4);
    margin-bottom: var(--space-6);
}
.quick-trace-modal .recent-docs h4 {
    margin: 0 0 var(--space-3) 0;
    color: var(--text-secondary);
    font-size: var(--font-size-sm);
    text-transform: uppercase;
    letter-spacing: 0.05em;
}
.quick-trace-modal .docs-list {
    display: flex;
    flex-direction: column;
    gap: var(--space-2);
    max-height: 300px;
    overflow-y: auto;
}
.quick-trace-modal .doc-item {
    display: flex;
    align-items: center;
    gap: var(--space-3);
    padding: var(--space-2) var(--space-3);
    border: 1px solid var(--border-color);
    border-radius: var(--radius-md);
    cursor: pointer;
    transition: all 0.2s;
}
.quick-trace-modal .doc-item:hover {
    background: var(--bg-secondary);
    border-color: var(--primary-300);
}
.quick-trace-modal .doc-icon {
    font-size: 1.25rem;
}
.quick-trace-modal .doc-info {
    flex: 1;
    display: flex;
    flex-direction: column;
}
.quick-trace-modal .doc-name {
    font-size: var(--font-size-sm);
    color: var(--text-primary);
}
.quick-trace-modal .doc-number {
    font-size: var(--font-size-xs);
    font-family: var(--font-mono);
    color: var(--text-secondary);
}
.quick-trace-modal .doc-date {
    font-size: var(--font-size-xs);
    color: var(--text-tertiary);
}
.entry-validation-modal .validation-status {
    display: flex;
    align-items: center;
    justify-content: center;
    gap: var(--space-2);
    padding: var(--space-4);
    border-radius: var(--radius-lg);
    font-weight: 600;
    margin-bottom: var(--space-4);
}
.entry-validation-modal .validation-status.success {
    background: var(--success-100);
    color: var(--success-700);
}
.entry-validation-modal .validation-status.error {
```

```
    background: var(--error-100);
    color: var(--error-700);
}
.entry-validation-modal .validation-status i {
    font-size: 1.5rem;
}
.entry-validation-modal .validation-results-list {
    display: flex;
    flex-direction: column;
    gap: var(--space-3);
}
.entry-validation-modal .result-item {
    padding: var(--space-3);
    border-radius: var(--radius-md);
    border: 1px solid var(--border-color);
}
.entry-validation-modal .result-item.error {
    border-left: 3px solid var(--error-500);
    background: var(--error-50);
}
.entry-validation-modal .result-item.warning {
    border-left: 3px solid var(--warning-500);
    background: var(--warning-50);
}
.entry-validation-modal .result-item.success,
.entry-validation-modal .result-item.info {
    border-left: 3px solid var(--success-500);
    background: var(--success-50);
}
.entry-validation-modal .result-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: var(--space-2);
}
.entry-validation-modal .result-code {
    font-family: var(--font-mono);
    font-size: var(--font-size-xs);
    color: var(--text-secondary);
}
.entry-validation-modal .result-message {
    font-size: var(--font-size-sm);
    color: var(--text-primary);
    white-space: pre-wrap;
}
@media (max-width: 768px) {
    .progress-overview {
        flex-direction: column;
        align-items: center;
    }
    .progress-hero {
        flex-direction: column;
    }
    .trace-summary {
        flex-direction: column;
    }
    .flow-visualization {
        flex-wrap: nowrap;
        overflow-x: auto;
        padding-bottom: var(--space-2);
    }
    .quick-trace-modal .trace-search {
        grid-template-columns: 1fr;
    }
}
```

Archivo: js\app.js

```
const App = {
    currentModule: 'dashboard',
    currentView: null,
    isInitialized: false,
    modules: {
        'dashboard': {
            title: 'Dashboard',
```

```
            icon: 'fa-tachometer-alt',
            handler: DashboardModule
        },
        'contabilidad': {
            title: 'Contabilidad General',
            icon: 'fa-book',
            handler: ContabilidadModule,
            views: {
                'plan-cuentas': 'Plan de Cuentas',
                'asientos': 'Asientos Contables',
                'libro-mayor': 'Libro Mayor',
                'libro-diario': 'Libro Diario',
                'libros-auxiliares': 'Libros Auxiliares'
            }
        },
        'compras': {
            title: 'Compras',
            icon: 'fa-shopping-cart',
            handler: ComprasModule,
            views: {
                'proveedores': 'Proveedores',
                'ordenes-compra': 'Órdenes de Compra',
                'facturas-proveedor': 'Facturas Proveedor'
            }
        },
        'ventas': {
            title: 'Ventas',
            icon: 'fa-store',
            handler: VentasModule,
            views: {
                'clientes': 'Clientes',
                'pedidos-venta': 'Pedidos de Venta',
                'facturas-cliente': 'Facturas de Venta'
            }
        },
        'inventario': {
            title: 'Inventario',
            icon: 'fa-boxes',
            handler: InventarioModule,
            views: {
                'productos': 'Productos',
                'movimientos-stock': 'Movimientos',
                'valoracion': 'Valorización'
            }
        },
        'tesoreria': {
            title: 'Tesorería',
            icon: 'fa-coins',
            handler: TesoreriaModule,
            views: {
                'cuentas-pagar': 'Control de Pagos',
                'cuentas-cobrar': 'Control de Cobros',
                'bancos': 'Bancos',
                'conciliacion': 'Conciliación'
            }
        },
        'costos': {
            title: 'Costos',
            icon: 'fa-calculator',
            handler: CostosModule,
            disabled: true,
            stage: 2,
            views: {
                'centros-costo': 'Centros de Costo',
                'ordenes-internas': 'Órdenes Internas',
                'analisis-costos': 'Análisis de Costos'
            }
        },
        'rrhh': {
            title: 'Recursos Humanos',
            icon: 'fa-users',
            handler: RRHHModule,
            disabled: true,
            stage: 2,
            views: {
                'empleados': 'Empleados',
                'nomina': 'Nómina',
                'provisiones': 'Provisiones'
            }
        },
        'reportes': {
```

```
            title: 'Reportes',
            icon: 'fa-chart-bar',
            handler: ReportesModule,
            views: {
                'balance-general': 'Balance General',
                'balance-comprobacion': 'Balance de Comprobación',
                'estado-resultados': 'Estado de Resultados',
                'ratios-financieros': 'Ratios Financieros',
                'flujo-caja': 'Flujo de Caja',
                'analisis-cuentas': 'Análisis de Cuentas'
            }
        },
        'trazabilidad': {
            title: 'Trazabilidad Pedagógica',
            icon: 'fa-route',
            handler: TrazabilidadModule,
            disabled: true,
            stage: 2,
            views: {
                'overview': 'Vista General',
                'trace-document': 'Trazar Documento',
                'flow-diagrams': 'Diagramas de Flujo',
                'validations': 'Validaciones',
                'progress': 'Progreso del Ejercicio'
            }
        },
        'admin': {
            title: 'Administración',
            icon: 'fa-cog',
            handler: AdminModule,
            views: {
                'empresas': 'Empresas',
                'import-export': 'Importar/Exportar',
                'configuracion': 'Configuración'
            }
        }
    },
    async init() {
        console.log('🚀 Iniciando EDU-TRACE ERP...');
        try {
            await DB.init();
            console.log('✅ Base de datos inicializada');
            Toast.init();
            Modal.init();
            console.log('✅ Componentes UI inicializados');
            await CompanyService.loadSavedCompany();
            console.log('✅ Empresa cargada');
            this.setupNavigation();
            this.setupSidebar();
            this.setupHeaderActions();
            this.setupTheme();
            this.hideLoading();
            const hash = window.location.hash.slice(1);
            if (hash) {
                const parts = hash.split('/');
                this.navigate(parts[0] || 'dashboard', parts[1]);
            } else {
                this.navigate('dashboard');
            }
            this.isInitialized = true;
            console.log('✅ EDU-TRACE ERP iniciado correctamente');
        } catch (error) {
            console.error('❌ Error al iniciar:', error);
            this.showError('Error al iniciar la aplicación: ' + error.message);
        }
    },
    hideLoading() {
        const loading = document.getElementById('loading-screen');
        const app = document.getElementById('app');
        if (loading) {
            loading.classList.add('hidden');
            loading.style.opacity = '0';
            setTimeout(() => {
                loading.style.display = 'none';
            }, 300);
        }
        if (app) {
            app.classList.remove('hidden');
        }
```

```
    },
    setupNavigation() {
        window.addEventListener('hashchange', () => {
            const hash = window.location.hash.slice(1);
            const parts = hash.split('/');
            this.navigate(parts[0] || 'dashboard', parts[1], false);
        });
        document.querySelectorAll('.nav-item[data-module]').forEach(item => {
            item.addEventListener('click', (e) => {
                e.preventDefault();
                const module = item.dataset.module;
                const view = item.dataset.view;
                this.navigate(module, view);
            });
        });
        document.querySelectorAll('.submenu-item').forEach(item => {
            item.addEventListener('click', (e) => {
                e.preventDefault();
                e.stopPropagation();
                const module = item.closest('.nav-item').dataset.module;
                const view = item.dataset.view;
                this.navigate(module, view);
            });
        });
    },
    setupSidebar() {
        const toggleBtn = document.getElementById('sidebar-toggle');
        const sidebar = document.getElementById('sidebar');
        toggleBtn?.addEventListener('click', () => {
            sidebar.classList.toggle('collapsed');
            localStorage.setItem('sidebar-collapsed',
sidebar.classList.contains('collapsed'));
        });
        if (localStorage.getItem('sidebar-collapsed') === 'true') {
            sidebar?.classList.add('collapsed');
        }
        document.querySelectorAll('.nav-item.has-submenu > a').forEach(item => {
            item.addEventListener('click', (e) => {
                e.preventDefault();
                const navItem = item.closest('.nav-item');
                const wasExpanded = navItem.classList.contains('expanded');
                document.querySelectorAll('.nav-item.expanded').forEach(other => {
                    if (other !== navItem) other.classList.remove('expanded');
                });
                navItem.classList.toggle('expanded', !wasExpanded);
            });
        });
        const companyBtn = document.getElementById('company-selector');
        companyBtn?.addEventListener('click', () => this.showCompanySelector());
    },
    setupHeaderActions() {
        document.getElementById('theme-toggle')?.addEventListener('click', () => {
            this.toggleTheme();
        });
        document.getElementById('btn-quick-entry')?.addEventListener('click', () => {
            this.showQuickEntry();
        });
        document.getElementById('notifications-btn')?.addEventListener('click', () => {
            this.showNotifications();
        });
        document.getElementById('help-btn')?.addEventListener('click', () => {
            this.showHelp();
        });
        const exportToggle = document.getElementById('export-dropdown-toggle');
        const exportDropdown = document.getElementById('export-dropdown');
        exportToggle?.addEventListener('click', (e) => {
            e.stopPropagation();
            exportDropdown?.classList.toggle('show');
        });
        document.addEventListener('click', () => {
            exportDropdown?.classList.remove('show');
        });
        document.querySelectorAll('[data-export]').forEach(btn => {
            btn.addEventListener('click', async () => {
                const format = btn.dataset.export;
                await this.exportCurrentView(format);
            });
        });
    },
    setupTheme() {
        const savedTheme = localStorage.getItem('theme') || 'light';
```

```
        document.documentElement.setAttribute('data-theme', savedTheme);
        this.updateThemeIcon(savedTheme);
    },
    toggleTheme() {
        const current = document.documentElement.getAttribute('data-theme') || 'light';
        const newTheme = current === 'light' ? 'dark' : 'light';
        document.documentElement.setAttribute('data-theme', newTheme);
        localStorage.setItem('theme', newTheme);
        this.updateThemeIcon(newTheme);
    },
    updateThemeIcon(theme) {
        const icon = document.querySelector('#theme-toggle i');
        if (icon) {
            icon.className = theme === 'light' ? 'fas fa-moon' : 'fas fa-sun';
        }
    },
    async navigate(moduleName, viewName = null, updateHash = true) {
        const module = this.modules[moduleName];
        if (!module) {
            console.warn(`Módulo ${moduleName} no encontrado`);
            return;
        }
        if (module.disabled) {
            Toast.info(`El módulo "${module.title}" estará disponible en la Etapa
${module.stage || 2}. ¡Próximamente!`);
            console.log(`⏳ Módulo ${moduleName} deshabilitado (Etapa ${module.stage})`);
            return;
        }
        if (updateHash) {
            const hash = viewName ? `${moduleName}/${viewName}` : moduleName;
            history.pushState(null, '', `#${hash}`);
        }
        this.currentModule = moduleName;
        this.currentView = viewName;
        this.updateNavigation(moduleName, viewName);
        this.updateBreadcrumbs(module.title, viewName ? module.views?.[viewName] : null);
        const contentWrapper = document.getElementById('content-wrapper');
        if (contentWrapper && module.handler) {
            try {
                contentWrapper.innerHTML = '<div class="loading-content"><div
class="spinner"></div></div>';
                const html = await module.handler.render(viewName);
                contentWrapper.innerHTML = html;
                if (module.handler.init) {
                    await module.handler.init(viewName);
                }
                if (moduleName === 'dashboard' && module.handler.initCharts) {
                    module.handler.initCharts();
                }
                if (module.handler.bindEvents) {
                    module.handler.bindEvents();
                }
            } catch (error) {
                console.error('Error al renderizar:', error);
                contentWrapper.innerHTML = `
                    <div class="empty-state">
                        <i class="fas fa-exclamation-circle" style="color: var(--error-
500);"></i>
                        <h3>Error al cargar</h3>
                        <p>${error.message}</p>
                        <button class="btn btn-primary"
onclick="App.navigate('dashboard')">
                            Volver al Dashboard
                        </button>
                    </div>
                `;
            }
        }
    },
    updateNavigation(moduleName, viewName) {
        document.querySelectorAll('.nav-item.active').forEach(el =>
el.classList.remove('active'));
        document.querySelectorAll('.submenu-item.active').forEach(el =>
el.classList.remove('active'));
        const navItem = document.querySelector(`.nav-item[data-module="${moduleName}"]`);
        if (navItem) {
            navItem.classList.add('active');
            if (viewName && navItem.classList.contains('has-submenu')) {
                navItem.classList.add('expanded');
                const submenuItem = navItem.querySelector(`.submenu-item[data-
view="${viewName}"]`);
```

```
                submenuItem?.classList.add('active');
            }
        }
    },
    updateBreadcrumbs(moduleTitle, viewTitle) {
        const breadcrumbs = document.getElementById('breadcrumbs');
        if (!breadcrumbs) return;
        let html = '<a href="#dashboard" class="breadcrumb-item">Inicio</a>';
        if (moduleTitle && moduleTitle !== 'Dashboard') {
            html += `<span class="breadcrumb-separator">/</span>`;
            html += `<span class="breadcrumb-item">${moduleTitle}</span>`;
        }
        if (viewTitle) {
            html += `<span class="breadcrumb-separator">/</span>`;
            html += `<span class="breadcrumb-item active">${viewTitle}</span>`;
        }
        breadcrumbs.innerHTML = html;
    },
    async showCompanySelector() {
        const companies = await CompanyService.getAll();
        const current = CompanyService.getCurrent();
        if (companies.length === 0) {
            DashboardModule.showCreateCompanyModal();
            return;
        }
        Modal.open({
            title: 'Seleccionar Empresa',
            size: 'medium',
            content: `
                <div class="company-list">
                    ${companies.map(c => `
                        <div class="company-option ${c.id === current?.id ? 'active' :
''}" data-id="${c.id}">
                            <div class="avatar">${c.name.charAt(0).toUpperCase()}</div>
                            <div class="company-info">
                                <div class="company-name">${c.name}</div>
                                <div class="company-rut">${c.rut || 'Sin RUT'}</div>
                            </div>
                            ${c.id === current?.id ? '<i class="fas fa-check-circle"
style="color: var(--success-500);"></i>' : ''}
                        </div>
                    `).join('')}
                </div>
            `,
            footer: `
                <button class="btn btn-outline" onclick="Modal.close();
DashboardModule.showCreateCompanyModal();">
                    <i class="fas fa-plus"></i> Nueva Empresa
                </button>
            `
        });
        document.querySelectorAll('.company-option').forEach(opt => {
            opt.addEventListener('click', async () => {
                await CompanyService.setCurrent(opt.dataset.id);
                Modal.close();
                this.navigate('dashboard');
                Toast.success('Empresa seleccionada');
            });
        });
    },
    async exportCurrentView(format) {
        const company = CompanyService.getCurrent();
        if (!company) {
            Toast.warning('Selecciona una empresa primero');
            return;
        }
        const loading = Toast.loading(`Exportando a ${format.toUpperCase()}...`);
        try {
            switch (format) {
                case 'json':
                    const backupData = await CompanyService.exportData();
                    await ExportService.toJSON(backupData,
`${company.name}_backup_${Helpers.getCurrentDate()}`);
                    loading.success('Backup JSON descargado');
                    break;
                case 'excel':
                    await this.exportToExcel(company);
                    loading.success('Excel descargado');
                    break;
                case 'csv':
                    await this.exportToCSV(company);
```

```
                    loading.success('CSV descargado');
                    break;
                case 'pdf':
                    await this.exportToPDF(company);
                    loading.success('PDF descargado');
                    break;
                default:
                    loading.error('Formato no soportado');
            }
        } catch (err) {
            console.error('Error al exportar:', err);
            loading.error('Error al exportar: ' + err.message);
        }
    },
    async exportToExcel(company) {
        const wb = XLSX.utils.book_new();
        try {
            const balanceData = await
AccountingService.getBalanceSheet(Helpers.getCurrentDate());
            const balanceRows = [];
            balanceRows.push({ 'Concepto': 'BALANCE GENERAL', 'Monto': '' });
            balanceRows.push({ 'Concepto': `Al ${Formatters.date(balanceData.date || new
Date())}`, 'Monto': '' });
            balanceRows.push({ 'Concepto': '', 'Monto': '' });
            balanceRows.push({ 'Concepto': 'ACTIVOS', 'Monto': '' });
            if (balanceData.assets?.items) {
                balanceData.assets.items.forEach(item => {
                    if (item.balance !== 0) {
                        balanceRows.push({ 'Concepto': `  ${item.name}`, 'Monto':
item.balance });
                    }
                });
            }
            balanceRows.push({ 'Concepto': 'Total Activos', 'Monto':
balanceData.assets?.total || 0 });
            balanceRows.push({ 'Concepto': '', 'Monto': '' });
            balanceRows.push({ 'Concepto': 'PASIVOS', 'Monto': '' });
            if (balanceData.liabilities?.items) {
                balanceData.liabilities.items.forEach(item => {
                    if (item.balance !== 0) {
                        balanceRows.push({ 'Concepto': `  ${item.name}`, 'Monto':
item.balance });
                    }
                });
            }
            balanceRows.push({ 'Concepto': 'Total Pasivos', 'Monto':
balanceData.liabilities?.total || 0 });
            balanceRows.push({ 'Concepto': '', 'Monto': '' });
            balanceRows.push({ 'Concepto': 'PATRIMONIO', 'Monto': '' });
            if (balanceData.equity?.items) {
                balanceData.equity.items.forEach(item => {
                    if (item.balance !== 0) {
                        balanceRows.push({ 'Concepto': `  ${item.name}`, 'Monto':
item.balance });
                    }
                });
            }
            if (balanceData.netIncome) {
                balanceRows.push({ 'Concepto': '  Resultado del Ejercicio', 'Monto':
balanceData.netIncome });
            }
            balanceRows.push({ 'Concepto': 'Total Patrimonio', 'Monto':
(balanceData.equity?.total || 0) + (balanceData.netIncome || 0) });
            const wsBalance = XLSX.utils.json_to_sheet(balanceRows);
            wsBalance['!cols'] = [{ wch: 40 }, { wch: 20 }];
            XLSX.utils.book_append_sheet(wb, wsBalance, 'Balance General');
        } catch (e) {
            console.log('No se pudo generar Balance:', e);
        }
        try {
            const incomeData = await AccountingService.getIncomeStatement(
                Helpers.getFirstDayOfYear(),
                Helpers.getCurrentDate()
            );
            const incomeRows = [];
            incomeRows.push({ 'Concepto': 'ESTADO DE RESULTADOS', 'Monto': '' });
            incomeRows.push({ 'Concepto': `Del ${Formatters.date(incomeData.dateFrom)} al
${Formatters.date(incomeData.dateTo)}`, 'Monto': '' });
            incomeRows.push({ 'Concepto': '', 'Monto': '' });
            incomeRows.push({ 'Concepto': 'INGRESOS OPERACIONALES', 'Monto': '' });
            if (incomeData.revenue?.items) {
```

```
                incomeData.revenue.items.forEach(item => {
                    if (item.balance !== 0) {
                        incomeRows.push({ 'Concepto': `  ${item.name}`, 'Monto':
Math.abs(item.balance) });
                    }
                });
            }
            incomeRows.push({ 'Concepto': 'Total Ingresos', 'Monto':
incomeData.revenue?.total || 0 });
            incomeRows.push({ 'Concepto': '', 'Monto': '' });
            incomeRows.push({ 'Concepto': 'COSTOS DE VENTA', 'Monto': '' });
            if (incomeData.costOfSales?.items) {
                incomeData.costOfSales.items.forEach(item => {
                    if (item.balance !== 0) {
                        incomeRows.push({ 'Concepto': `  ${item.name}`, 'Monto':
Math.abs(item.balance) });
                    }
                });
            }
            incomeRows.push({ 'Concepto': 'Total Costos', 'Monto':
incomeData.costOfSales?.total || 0 });
            incomeRows.push({ 'Concepto': '', 'Monto': '' });
            incomeRows.push({ 'Concepto': 'MARGEN BRUTO', 'Monto': incomeData.grossMargin
|| 0 });
            incomeRows.push({ 'Concepto': '', 'Monto': '' });
            incomeRows.push({ 'Concepto': 'GASTOS OPERACIONALES', 'Monto': '' });
            if (incomeData.expenses?.items) {
                incomeData.expenses.items.forEach(item => {
                    if (item.balance !== 0) {
                        incomeRows.push({ 'Concepto': `  ${item.name}`, 'Monto':
Math.abs(item.balance) });
                    }
                });
            }
            incomeRows.push({ 'Concepto': 'Total Gastos', 'Monto':
incomeData.expenses?.total || 0 });
            incomeRows.push({ 'Concepto': '', 'Monto': '' });
            incomeRows.push({ 'Concepto': 'RESULTADO OPERACIONAL', 'Monto':
incomeData.operatingIncome || 0 });
            incomeRows.push({ 'Concepto': 'RESULTADO DEL EJERCICIO', 'Monto':
incomeData.netIncome || 0 });
            const wsIncome = XLSX.utils.json_to_sheet(incomeRows);
            wsIncome['!cols'] = [{ wch: 40 }, { wch: 20 }];
            XLSX.utils.book_append_sheet(wb, wsIncome, 'Estado Resultados');
        } catch (e) {
            console.log('No se pudo generar Estado de Resultados:', e);
        }
        try {
            const entries = await DB.getByIndex('journalEntries', 'companyId',
company.id);
            if (entries.length > 0) {
                const entryRows = [];
                for (const entry of entries) {
                    const lines = await AccountingService.getJournalLines(entry.id);
                    for (const line of lines) {
                        const account = await
AccountingService.getAccount(line.accountId);
                        entryRows.push({
                            'Número': entry.number,
                            'Fecha': entry.date,
                            'Descripción': entry.description,
                            'Cuenta': account ? `${account.code} - ${account.name}` :
line.accountId,
                            'Detalle Línea': line.description || '',
                            'Debe': line.debit || 0,
                            'Haber': line.credit || 0,
                            'Estado': entry.status === 'posted' ? 'Contabilizado' :
entry.status === 'draft' ? 'Borrador' : 'Anulado'
                        });
                    }
                    entryRows.push({});
                }
                const wsEntries = XLSX.utils.json_to_sheet(entryRows);
                wsEntries['!cols'] = [
                    { wch: 12 }, { wch: 12 }, { wch: 30 },
                    { wch: 35 }, { wch: 25 }, { wch: 15 }, { wch: 15 }, { wch: 15 }
                ];
                XLSX.utils.book_append_sheet(wb, wsEntries, 'Asientos Contables');
            }
        } catch (e) {
            console.log('No se pudieron exportar asientos:', e);
```

```
        }
        const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
        if (accounts.length > 0) {
            const wsAccounts = XLSX.utils.json_to_sheet(accounts.map(a => ({
                'Código': a.code,
                'Nombre': a.name,
                'Tipo': a.type === 'asset' ? 'Activo' :
                    a.type === 'liability' ? 'Pasivo' :
                        a.type === 'equity' ? 'Patrimonio' :
                            a.type === 'revenue' ? 'Ingreso' : 'Gasto',
                'Es Grupo': a.isGroup ? 'Sí' : 'No',
                'Saldo': a.balance || 0,
                'Activa': a.isActive ? 'Sí' : 'No'
            })));
            wsAccounts['!cols'] = [{ wch: 12 }, { wch: 40 }, { wch: 12 }, { wch: 10 }, {
wch: 15 }, { wch: 8 }];
            XLSX.utils.book_append_sheet(wb, wsAccounts, 'Plan de Cuentas');
        }
        const suppliers = await DB.getByIndex('suppliers', 'companyId', company.id);
        if (suppliers.length > 0) {
            const wsSuppliers = XLSX.utils.json_to_sheet(suppliers.map(s => ({
                'RUT': s.rut,
                'Razón Social': s.name,
                'Contacto': s.contact || '',
                'Email': s.email || '',
                'Teléfono': s.phone || '',
                'Dirección': s.address || ''
            })));
            XLSX.utils.book_append_sheet(wb, wsSuppliers, 'Proveedores');
        }
        const customers = await DB.getByIndex('customers', 'companyId', company.id);
        if (customers.length > 0) {
            const wsCustomers = XLSX.utils.json_to_sheet(customers.map(c => ({
                'RUT': c.rut,
                'Razón Social': c.name,
                'Contacto': c.contact || '',
                'Email': c.email || '',
                'Teléfono': c.phone || '',
                'Límite Crédito': c.creditLimit || 0
            })));
            XLSX.utils.book_append_sheet(wb, wsCustomers, 'Clientes');
        }
        const products = await DB.getByIndex('products', 'companyId', company.id);
        if (products.length > 0) {
            const wsProducts = XLSX.utils.json_to_sheet(products.map(p => ({
                'Código': p.code,
                'Nombre': p.name,
                'Categoría': p.category || '',
                'Unidad': p.unit || 'UN',
                'Costo': p.cost || 0,
                'Precio Venta': p.price || 0,
                'Stock': p.stock || 0,
                'Valor Inventario': (p.stock || 0) * (p.cost || 0)
            })));
            XLSX.utils.book_append_sheet(wb, wsProducts, 'Productos');
        }
        const employees = await DB.getByIndex('employees', 'companyId', company.id);
        if (employees.length > 0) {
            const wsEmployees = XLSX.utils.json_to_sheet(employees.map(e => ({
                'RUT': e.rut,
                'Nombre': e.name,
                'Cargo': e.position || '',
                'Departamento': e.department || '',
                'Sueldo Base': e.baseSalary || 0,
                'Fecha Ingreso': e.hireDate || ''
            })));
            XLSX.utils.book_append_sheet(wb, wsEmployees, 'Empleados');
        }
        const fullFilename =
`${company.name}_ERP_Completo_${Helpers.getCurrentDate()}.xlsx`;
        XLSX.writeFile(wb, fullFilename);
    },
    async exportToCSV(company) {
        let data = [];
        let filename = `${company.name}_export`;
        let headers = [];
        switch (this.currentModule) {
            case 'contabilidad':
                data = await DB.getByIndex('accounts', 'companyId', company.id);
                headers = [
                    { key: 'code', label: 'Código' },
```

```
                    { key: 'name', label: 'Nombre' },
                    { key: 'type', label: 'Tipo' },
                    { key: 'balance', label: 'Saldo' }
                ];
                filename = `${company.name}_plan_cuentas`;
                break;
            case 'compras':
                data = await DB.getByIndex('suppliers', 'companyId', company.id);
                headers = [
                    { key: 'rut', label: 'RUT' },
                    { key: 'name', label: 'Nombre' },
                    { key: 'email', label: 'Email' },
                    { key: 'phone', label: 'Teléfono' }
                ];
                filename = `${company.name}_proveedores`;
                break;
            case 'ventas':
                data = await DB.getByIndex('customers', 'companyId', company.id);
                headers = [
                    { key: 'rut', label: 'RUT' },
                    { key: 'name', label: 'Nombre' },
                    { key: 'email', label: 'Email' },
                    { key: 'creditLimit', label: 'Límite Crédito' }
                ];
                filename = `${company.name}_clientes`;
                break;
            case 'inventario':
                data = await DB.getByIndex('products', 'companyId', company.id);
                headers = [
                    { key: 'code', label: 'Código' },
                    { key: 'name', label: 'Nombre' },
                    { key: 'stock', label: 'Stock' },
                    { key: 'cost', label: 'Costo' },
                    { key: 'price', label: 'Precio' }
                ];
                filename = `${company.name}_productos`;
                break;
            default:
                data = await DB.getByIndex('accounts', 'companyId', company.id);
                headers = [
                    { key: 'code', label: 'Código' },
                    { key: 'name', label: 'Nombre' },
                    { key: 'type', label: 'Tipo' },
                    { key: 'balance', label: 'Saldo' }
                ];
                filename = `${company.name}_datos`;
        }
        await ExportService.toCSV(data, filename, { headers });
    },
    async exportToPDF(company) {
        switch (this.currentModule) {
            case 'reportes':
                if (this.currentView === 'balance-general') {
                    const balance = await AccountingService.generateBalanceSheet();
                    await ExportService.balanceSheetPDF(balance, company.name);
                } else {
                    await this.generateGenericPDF(company, 'Reporte');
                }
                break;
            default:
                await this.generateGenericPDF(company, 'Datos');
        }
    },
    async generateGenericPDF(company, title) {
        let data = [];
        let headers = [];
        let reportTitle = title;
        switch (this.currentModule) {
            case 'contabilidad':
                data = await DB.getByIndex('accounts', 'companyId', company.id);
                data = data.filter(a => !a.isGroup);
                headers = [
                    { key: 'code', label: 'Código' },
                    { key: 'name', label: 'Nombre' },
                    { key: 'type', label: 'Tipo' },
                    { key: 'balance', label: 'Saldo', format: 'currency' }
                ];
                reportTitle = 'Plan de Cuentas';
                break;
            case 'compras':
                data = await DB.getByIndex('suppliers', 'companyId', company.id);
```

```
                headers = [
                    { key: 'rut', label: 'RUT' },
                    { key: 'name', label: 'Razón Social' },
                    { key: 'email', label: 'Email' },
                    { key: 'phone', label: 'Teléfono' }
                ];
                reportTitle = 'Listado de Proveedores';
                break;
            case 'ventas':
                data = await DB.getByIndex('customers', 'companyId', company.id);
                headers = [
                    { key: 'rut', label: 'RUT' },
                    { key: 'name', label: 'Cliente' },
                    { key: 'email', label: 'Email' },
                    { key: 'creditLimit', label: 'Límite Crédito', format: 'currency' }
                ];
                reportTitle = 'Listado de Clientes';
                break;
            case 'inventario':
                data = await DB.getByIndex('products', 'companyId', company.id);
                headers = [
                    { key: 'code', label: 'Código' },
                    { key: 'name', label: 'Producto' },
                    { key: 'stock', label: 'Stock', format: 'number' },
                    { key: 'cost', label: 'Costo', format: 'currency' },
                    { key: 'price', label: 'Precio', format: 'currency' }
                ];
                reportTitle = 'Listado de Productos';
                break;
            case 'rrhh':
                data = await DB.getByIndex('employees', 'companyId', company.id);
                headers = [
                    { key: 'rut', label: 'RUT' },
                    { key: 'name', label: 'Nombre' },
                    { key: 'position', label: 'Cargo' },
                    { key: 'baseSalary', label: 'Sueldo', format: 'currency' }
                ];
                reportTitle = 'Listado de Empleados';
                break;
            default:
                data = await DB.getByIndex('accounts', 'companyId', company.id);
                headers = [
                    { key: 'code', label: 'Código' },
                    { key: 'name', label: 'Nombre' },
                    { key: 'balance', label: 'Saldo', format: 'currency' }
                ];
                reportTitle = 'Datos de la Empresa';
        }
        await ExportService.toPDF(data, `${company.name}_${this.currentModule}`, {
            title: reportTitle,
            subtitle: company.name,
            headers,
            companyName: company.name
        });
    },
    showHelp() {
        HelpSystem.show();
    },
    showError(message) {
        const loading = document.getElementById('loading-screen');
        if (loading) {
            loading.innerHTML = `
                <div style="text-align: center; color: var(--error-500);">
                    <i class="fas fa-exclamation-circle" style="font-size: 3rem; margin-
bottom: 1rem;"></i>
                    <h2>Error</h2>
                    <p>${message}</p>
                    <button class="btn btn-primary"
onclick="location.reload()">Reintentar</button>
                </div>
            `;
        }
    },
    showNotifications() {
        const company = CompanyService.getCurrent();
        const notifications = [
            {
                type: 'info',
                icon: 'fa-info-circle',
                title: 'Bienvenido a EDU-TRACE ERP',
                message: 'Sistema listo para usar',
```

```
                time: 'Ahora'
            }
        ];
        if (company) {
            notifications.push({
                type: 'success',
                icon: 'fa-building',
                title: 'Empresa activa',
                message: `Trabajando con ${company.name}`,
                time: 'Ahora'
            });
        } else {
            notifications.push({
                type: 'warning',
                icon: 'fa-exclamation-triangle',
                title: 'Sin empresa',
                message: 'Crea o selecciona una empresa para comenzar',
                time: 'Ahora',
                action: 'crear-empresa'
            });
        }
        Modal.open({
            title: '<i class="fas fa-bell"></i> Notificaciones',
            size: 'medium',
            content: `
                <div class="notifications-list">
                    ${notifications.length === 0 ? `
                        <div class="empty-state" style="padding: var(--space-6);">
                            <i class="fas fa-bell-slash" style="font-size: 2rem; color:
var(--text-tertiary);"></i>
                            <p>No hay notificaciones</p>
                        </div>
                    ` : notifications.map(n => `
                        <div class="notification-item ${n.type}" ${n.action ? `data-
action="${n.action}"` : ''}>
                            <div class="notification-icon">
                                <i class="fas ${n.icon}"></i>
                            </div>
                            <div class="notification-content">
                                <div class="notification-title">${n.title}</div>
                                <div class="notification-message">${n.message}</div>
                            </div>
                            <div class="notification-time">${n.time}</div>
                        </div>
                    `).join('')}
                </div>
            `,
            footer: `
                <button class="btn btn-outline" onclick="Modal.close()">Cerrar</button>
            `
        });
        document.querySelectorAll('.notification-item[data-action]').forEach(item => {
            item.style.cursor = 'pointer';
            item.addEventListener('click', () => {
                Modal.close();
                if (item.dataset.action === 'crear-empresa') {
                    DashboardModule.showCreateCompanyModal();
                }
            });
        });
    },
    showQuickEntry() {
        const company = CompanyService.getCurrent();
        if (!company) {
            Toast.warning('Selecciona una empresa primero');
            return;
        }
        Modal.open({
            title: '<i class="fas fa-bolt"></i> Entrada Rápida',
            size: 'medium',
            content: `
                <div class="quick-entry-grid">
                    <div class="quick-entry-item" data-action="asiento">
                        <div class="quick-entry-icon" style="background: linear-
gradient(135deg, #0a6ed1, #064280);">
                            <i class="fas fa-file-invoice"></i>
                        </div>
                        <span>Nuevo Asiento</span>
                    </div>
                    <div class="quick-entry-item" data-action="factura-venta">
                        <div class="quick-entry-icon" style="background: linear-
```

```
gradient(135deg, #107e3e, #0d6630);">
                            <i class="fas fa-file-invoice-dollar"></i>
                        </div>
                        <span>Factura de Venta</span>
                    </div>
                    <div class="quick-entry-item" data-action="factura-compra">
                        <div class="quick-entry-icon" style="background: linear-
gradient(135deg, #df6e0c, #c45f0a);">
                            <i class="fas fa-file-alt"></i>
                        </div>
                        <span>Factura de Compra</span>
                    </div>
                    <div class="quick-entry-item" data-action="pago">
                        <div class="quick-entry-icon" style="background: linear-
gradient(135deg, #6c5ce7, #5f3dc4);">
                            <i class="fas fa-money-bill-wave"></i>
                        </div>
                        <span>Registrar Pago</span>
                    </div>
                    <div class="quick-entry-item" data-action="cliente">
                        <div class="quick-entry-icon" style="background: linear-
gradient(135deg, #00b894, #00a187);">
                            <i class="fas fa-user-plus"></i>
                        </div>
                        <span>Nuevo Cliente</span>
                    </div>
                    <div class="quick-entry-item" data-action="proveedor">
                        <div class="quick-entry-icon" style="background: linear-
gradient(135deg, #e17055, #d35400);">
                            <i class="fas fa-truck"></i>
                        </div>
                        <span>Nuevo Proveedor</span>
                    </div>
                    <div class="quick-entry-item" data-action="producto">
                        <div class="quick-entry-icon" style="background: linear-
gradient(135deg, #00cec9, #00b5ad);">
                            <i class="fas fa-box"></i>
                        </div>
                        <span>Nuevo Producto</span>
                    </div>
                    <div class="quick-entry-item" data-action="empleado">
                        <div class="quick-entry-icon" style="background: linear-
gradient(135deg, #fd79a8, #e84393);">
                            <i class="fas fa-user-tie"></i>
                        </div>
                        <span>Nuevo Empleado</span>
                    </div>
                </div>
            `,
            footer: ''
        });
        document.querySelectorAll('.quick-entry-item').forEach(item => {
            item.addEventListener('click', () => {
                Modal.close();
                const action = item.dataset.action;
                switch (action) {
                    case 'asiento':
                        this.navigate('contabilidad', 'asientos');
                        break;
                    case 'factura-venta':
                        this.navigate('ventas', 'facturas-cliente');
                        break;
                    case 'factura-compra':
                        this.navigate('compras', 'facturas-proveedor');
                        break;
                    case 'pago':
                        this.navigate('tesoreria', 'bancos');
                        break;
                    case 'cliente':
                        this.navigate('ventas', 'clientes');
                        setTimeout(() => {
                            if (typeof VentasModule !== 'undefined' &&
VentasModule.showCustomerModal) {
                                VentasModule.showCustomerModal();
                            }
                        }, 300);
                        break;
                    case 'proveedor':
                        this.navigate('compras', 'proveedores');
                        setTimeout(() => {
                            if (typeof ComprasModule !== 'undefined' &&
```

```
ComprasModule.showSupplierModal) {
                                ComprasModule.showSupplierModal();
                            }
                        }, 300);
                        break;
                    case 'producto':
                        this.navigate('inventario', 'productos');
                        setTimeout(() => {
                            if (typeof InventarioModule !== 'undefined' &&
InventarioModule.showProductModal) {
                                InventarioModule.showProductModal();
                            }
                        }, 300);
                        break;
                    case 'empleado':
                        this.navigate('rrhh', 'empleados');
                        setTimeout(() => {
                            if (typeof RRHHModule !== 'undefined' &&
RRHHModule.showEmployeeModal) {
                                RRHHModule.showEmployeeModal();
                            }
                        }, 300);
                        break;
                }
            });
        });
    }
};
const appStyles = document.createElement('style');
appStyles.textContent = `
    .company-list { display: flex; flex-direction: column; gap: var(--space-2); }
    .company-option { display: flex; align-items: center; gap: var(--space-3); padding:
var(--space-3); border-radius: var(--radius-lg); cursor: pointer; transition: all var(--
transition-fast); border: 2px solid transparent; }
    .company-option:hover { background: var(--neutral-50); }
    .company-option.active { background: var(--primary-50); border-color: var(--primary-
500); }
    .company-info { flex: 1; }
    .company-name { font-weight: 600; }
    .company-rut { font-size: var(--font-size-xs); color: var(--text-tertiary); }
    .loading-content { display: flex; justify-content: center; align-items: center; min-
height: 300px; }
    .notifications-list { display: flex; flex-direction: column; gap: var(--space-2); }
    .notification-item { display: flex; align-items: flex-start; gap: var(--space-3);
padding: var(--space-3); border-radius: var(--radius-md); background: var(--neutral-50);
transition: all var(--transition-fast); }
    .notification-item:hover { background: var(--neutral-100); }
    .notification-icon { width: 36px; height: 36px; border-radius: var(--radius-full);
display: flex; align-items: center; justify-content: center; flex-shrink: 0; }
    .notification-item.info .notification-icon { background: var(--info-100); color: var(-
-info-500); }
    .notification-item.success .notification-icon { background: var(--success-100); color:
var(--success-500); }
    .notification-item.warning .notification-icon { background: var(--warning-100); color:
var(--warning-500); }
    .notification-item.error .notification-icon { background: var(--error-100); color:
var(--error-500); }
    .notification-content { flex: 1; min-width: 0; }
    .notification-title { font-weight: 600; font-size: var(--font-size-sm); margin-bottom:
var(--space-1); }
    .notification-message { font-size: var(--font-size-xs); color: var(--text-secondary);
}
    .notification-time { font-size: var(--font-size-xs); color: var(--text-tertiary);
white-space: nowrap; }
    .quick-entry-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: var(--
space-3); }
    .quick-entry-item { display: flex; flex-direction: column; align-items: center; gap:
var(--space-2); padding: var(--space-4); border-radius: var(--radius-lg); cursor: pointer;
transition: all var(--transition-fast); text-align: center; }
    .quick-entry-item:hover { background: var(--neutral-50); transform: translateY(-2px);
}
    .quick-entry-icon { width: 48px; height: 48px; border-radius: var(--radius-lg);
display: flex; align-items: center; justify-content: center; color: white; font-size:
var(--font-size-xl); }
    .quick-entry-item span { font-size: var(--font-size-xs); color: var(--text-secondary);
font-weight: 500; }
    @media (max-width: 600px) {
        .quick-entry-grid { grid-template-columns: repeat(2, 1fr); }
    }
`;
document.head.appendChild(appStyles);
```

```
document.addEventListener('DOMContentLoaded', () => App.init());
window.App = App;
```

Archivo: js\components\charts.js

```
const Charts = {
    instances: {},
    colors: {
        primary: 'rgba(10, 110, 209, 1)',
        primaryLight: 'rgba(10, 110, 209, 0.2)',
        success: 'rgba(16, 126, 62, 1)',
        successLight: 'rgba(16, 126, 62, 0.2)',
        warning: 'rgba(223, 110, 12, 1)',
        warningLight: 'rgba(223, 110, 12, 0.2)',
        error: 'rgba(187, 0, 0, 1)',
        errorLight: 'rgba(187, 0, 0, 0.2)',
        palette: [
            'rgba(10, 110, 209, 0.8)',
            'rgba(16, 126, 62, 0.8)',
            'rgba(223, 110, 12, 0.8)',
            'rgba(187, 0, 0, 0.8)',
            'rgba(156, 39, 176, 0.8)',
            'rgba(0, 188, 212, 0.8)',
            'rgba(255, 193, 7, 0.8)',
            'rgba(96, 125, 139, 0.8)'
        ]
    },
    getBaseConfig() {
        return {
            responsive: true,
            maintainAspectRatio: false,
            plugins: {
                legend: {
                    display: true,
                    position: 'bottom',
                    labels: {
                        padding: 20,
                        usePointStyle: true,
                        font: { family: "'Inter', sans-serif", size: 12 }
                    }
                },
                tooltip: {
                    backgroundColor: 'rgba(0,0,0,0.8)',
                    titleFont: { family: "'Inter', sans-serif", size: 13, weight: '600' },
                    bodyFont: { family: "'Inter', sans-serif", size: 12 },
                    padding: 12,
                    cornerRadius: 8,
                    displayColors: true
                }
            }
        };
    },
    line(canvasId, data, options = {}) {
        const ctx = document.getElementById(canvasId);
        if (!ctx) return null;
        if (this.instances[canvasId]) {
            this.instances[canvasId].destroy();
        }
        const config = {
            type: 'line',
            data: {
                labels: data.labels,
                datasets: data.datasets.map((ds, i) => ({
                    label: ds.label,
                    data: ds.data,
                    borderColor: ds.color || this.colors.palette[i],
                    backgroundColor: ds.fill ? (ds.backgroundColor ||
this.colors.primaryLight) : 'transparent',
                    fill: ds.fill || false,
                    tension: 0.4,
                    pointRadius: 4,
                    pointHoverRadius: 6,
                    borderWidth: 2
                }))
            },
```

```
            options: {
                ...this.getBaseConfig(),
                scales: {
                    x: {
                        grid: { display: false },
                        ticks: { font: { family: "'Inter', sans-serif", size: 11 } }
                    },
                    y: {
                        beginAtZero: options.beginAtZero !== false,
                        grid: { color: 'rgba(0,0,0,0.05)' },
                        ticks: {
                            font: { family: "'Inter', sans-serif", size: 11 },
                            callback: (value) => options.currency ?
Formatters.currency(value, { showSymbol: false }) : Formatters.number(value)
                        }
                    }
                },
                ...options
            }
        };
        this.instances[canvasId] = new Chart(ctx, config);
        return this.instances[canvasId];
    },
    bar(canvasId, data, options = {}) {
        const ctx = document.getElementById(canvasId);
        if (!ctx) return null;
        if (this.instances[canvasId]) {
            this.instances[canvasId].destroy();
        }
        const config = {
            type: 'bar',
            data: {
                labels: data.labels,
                datasets: data.datasets.map((ds, i) => ({
                    label: ds.label,
                    data: ds.data,
                    backgroundColor: ds.colors || this.colors.palette[i],
                    borderColor: ds.borderColors || this.colors.palette[i],
                    borderWidth: 1,
                    borderRadius: 4
                }))
            },
            options: {
                ...this.getBaseConfig(),
                scales: {
                    x: {
                        grid: { display: false },
                        ticks: { font: { family: "'Inter', sans-serif", size: 11 } }
                    },
                    y: {
                        beginAtZero: true,
                        grid: { color: 'rgba(0,0,0,0.05)' },
                        ticks: {
                            font: { family: "'Inter', sans-serif", size: 11 },
                            callback: (value) => options.currency ?
Formatters.currency(value, { showSymbol: false }) : value
                        }
                    }
                },
                ...options
            }
        };
        this.instances[canvasId] = new Chart(ctx, config);
        return this.instances[canvasId];
    },
    horizontalBar(canvasId, data, options = {}) {
        return this.bar(canvasId, data, { ...options, indexAxis: 'y' });
    },
    doughnut(canvasId, data, options = {}) {
        const ctx = document.getElementById(canvasId);
        if (!ctx) return null;
        if (this.instances[canvasId]) {
            this.instances[canvasId].destroy();
        }
        const config = {
            type: options.pie ? 'pie' : 'doughnut',
            data: {
                labels: data.labels,
                datasets: [{
                    data: data.values,
                    backgroundColor: data.colors || this.colors.palette,
```

```
                    borderWidth: 2,
                    borderColor: '#fff'
                }]
            },
            options: {
                ...this.getBaseConfig(),
                cutout: options.pie ? 0 : '65%',
                plugins: {
                    ...this.getBaseConfig().plugins,
                    tooltip: {
                        ...this.getBaseConfig().plugins.tooltip,
                        callbacks: {
                            label: (context) => {
                                const value = context.raw;
                                const total = context.dataset.data.reduce((a, b) => a + b,
0);
                                const percentage = ((value / total) * 100).toFixed(1);
                                return options.currency
                                    ? `${context.label}: ${Formatters.currency(value)}
(${percentage}%)`
                                    : `${context.label}: ${Formatters.number(value)}
(${percentage}%)`;
                            }
                        }
                    }
                },
                ...options
            }
        };
        this.instances[canvasId] = new Chart(ctx, config);
        return this.instances[canvasId];
    },
    area(canvasId, data, options = {}) {
        return this.line(canvasId, {
            ...data,
            datasets: data.datasets.map(ds => ({ ...ds, fill: true }))
        }, options);
    },
    mixed(canvasId, data, options = {}) {
        const ctx = document.getElementById(canvasId);
        if (!ctx) return null;
        if (this.instances[canvasId]) {
            this.instances[canvasId].destroy();
        }
        const config = {
            type: 'bar',
            data: {
                labels: data.labels,
                datasets: data.datasets.map((ds, i) => ({
                    type: ds.type || 'bar',
                    label: ds.label,
                    data: ds.data,
                    backgroundColor: ds.type === 'line' ? 'transparent' : (ds.color ||
this.colors.palette[i]),
                    borderColor: ds.color || this.colors.palette[i],
                    borderWidth: ds.type === 'line' ? 2 : 1,
                    borderRadius: ds.type === 'line' ? 0 : 4,
                    tension: 0.4,
                    yAxisID: ds.yAxisID || 'y',
                    order: ds.type === 'line' ? 0 : 1
                }))
            },
            options: {
                ...this.getBaseConfig(),
                scales: {
                    x: { grid: { display: false } },
                    y: {
                        type: 'linear',
                        position: 'left',
                        beginAtZero: true,
                        grid: { color: 'rgba(0,0,0,0.05)' }
                    },
                    y1: options.dualAxis ? {
                        type: 'linear',
                        position: 'right',
                        beginAtZero: true,
                        grid: { display: false }
                    } : undefined
                },
                ...options
            }
```

```
        };
        this.instances[canvasId] = new Chart(ctx, config);
        return this.instances[canvasId];
    },
    update(canvasId, newData) {
        const chart = this.instances[canvasId];
        if (!chart) return;
        if (newData.labels) {
            chart.data.labels = newData.labels;
        }
        if (newData.datasets) {
            newData.datasets.forEach((ds, i) => {
                if (chart.data.datasets[i]) {
                    chart.data.datasets[i].data = ds.data;
                    if (ds.label) chart.data.datasets[i].label = ds.label;
                }
            });
        }
        if (newData.values) {
            chart.data.datasets[0].data = newData.values;
        }
        chart.update();
    },
    destroy(canvasId) {
        if (this.instances[canvasId]) {
            this.instances[canvasId].destroy();
            delete this.instances[canvasId];
        }
    },
    destroyAll() {
        Object.keys(this.instances).forEach(id => this.destroy(id));
    },
    createCanvas(containerId, canvasId, height = 300) {
        const container = document.getElementById(containerId);
        if (!container) return null;
        container.innerHTML = `
            <div style="height: ${height}px; position: relative;">
                <canvas id="${canvasId}"></canvas>
            </div>
        `;
        return document.getElementById(canvasId);
    }
};
window.Charts = Charts;
```

Archivo: js\components\help.js

```
const HelpSystem = {
    documentation: {
        general: {
            title: 'Introducción a EDU-TRACE ERP',
            icon: 'fa-home',
            content: `
                <div class="help-section">
                    <h3>¿Qué es EDU-TRACE ERP?</h3>
                    <p>EDU-TRACE ERP es un <strong>sistema ERP educativo</strong> diseñado
específicamente para estudiantes
                    de auditoría y contabilidad. Simula las funcionalidades de sistemas
empresariales,
                    permitiendo aprender la lógica de procesos de negocio sin la
complejidad de un sistema real.</p>
                    <h3>Características Principales</h3>
                    <ul>
                        <li><strong>Empresas Virtuales:</strong> Cada estudiante puede
crear y gestionar múltiples empresas</li>
                        <li><strong>Módulos Integrados:</strong> Contabilidad, Compras,
Ventas, Inventario, Tesorería, Costos y RRHH</li>
                        <li><strong>Datos Offline:</strong> Todo se almacena localmente
usando IndexedDB</li>
                        <li><strong>Importar/Exportar:</strong> Comparte casos de estudio
en JSON o Excel</li>
                        <li><strong>Reportes:</strong> Genera estados financieros
automáticamente</li>
                    </ul>
                    <h3>Primeros Pasos</h3>
```

```
                    <ol>
                        <li>Crea tu primera empresa en <strong>Administración → 
Empresas</strong></li>
                        <li>Revisa el plan de cuentas en <strong>Contabilidad → Plan de 
Cuentas</strong></li>
                        <li>Registra tu primer asiento en <strong>Contabilidad → 
Asientos</strong></li>
                        <li>Genera el Balance General en <strong>Reportes → Balance 
General</strong></li>
                    </ol>
                </div>
            `
        },
        contabilidad: {
            title: 'Módulo de Contabilidad (GL)',
            icon: 'fa-book',
            content: `
                <div class="help-section">
                    <h3>Contabilidad General</h3>
                    <p>El módulo de Contabilidad (General Ledger - GL) es el núcleo del 
sistema ERP.
                    Aquí se registran todas las transacciones financieras de la empresa. 
</p>
                    <h4><i class="fas fa-sitemap"></i> Plan de Cuentas</h4>
                    <p>El plan de cuentas viene precargado con una estructura IFRS/PCGA 
chilena:</p>
                    <table class="help-table">
                        <tr><td><strong>1</strong></td><td>Activos</td><td>Bienes y 
derechos</td></tr>
                        <tr><td><strong>2</strong></td><td>Pasivos</td>
<td>Obligaciones</td></tr>
                        <tr><td><strong>3</strong></td><td>Patrimonio</td><td>Capital y 
resultados</td></tr>
                        <tr><td><strong>4</strong></td><td>Ingresos</td><td>Ventas y otros 
ingresos</td></tr>
                        <tr><td><strong>5</strong></td><td>Costos</td><td>Costo de 
ventas</td></tr>
                        <tr><td><strong>6</strong></td><td>Gastos</td><td>Gastos 
operacionales</td></tr>
                    </table>
                    <h4><i class="fas fa-file-invoice"></i> Asientos Contables</h4>
                    <p>Para crear un asiento:</p>
                    <ol>
                        <li>Selecciona la fecha del documento</li>
                        <li>Ingresa una glosa descriptiva</li>
                        <li>Agrega líneas al Debe y al Haber</li>
                        <li>Verifica que <strong>Debe = Haber</strong></li>
                        <li>Guarda y contabiliza</li>
                    </ol>
                    <div class="help-tip">
                        <i class="fas fa-lightbulb"></i>
                        <span>Los asientos deben estar balanceados. El sistema no 
permitirá guardar
                        un asiento donde el total del Debe no sea igual al total del 
Haber.</span>
                    </div>
                    <h4><i class="fas fa-book-open"></i> Libro Mayor</h4>
                    <p>Muestra todos los movimientos de una cuenta específica,
                    con saldo inicial, movimientos y saldo final.</p>
                    <h4><i class="fas fa-list"></i> Libro Diario</h4>
                    <p>Registro cronológico de todos los asientos contables,
                    ordenados por fecha de operación.</p>
                </div>
            `
        },
        compras: {
            title: 'Módulo de Compras (MM)',
            icon: 'fa-shopping-cart',
            content: `
                <div class="help-section">
                    <h3>Gestión de Compras</h3>
                    <p>El módulo de Compras (Materials Management - MM) gestiona todo el 
proceso
                    de adquisición de bienes y servicios.</p>
                    <h4><i class="fas fa-users"></i> Proveedores</h4>
                    <p>Mantiene el registro de proveedores con:</p>
                    <ul>
                        <li>Datos fiscales (RUT, razón social)</li>
                        <li>Información de contacto</li>
                        <li>Condiciones de pago</li>
                        <li>Historial de transacciones</li>
```

```
                    </ul>
                    <h4><i class="fas fa-file-alt"></i> Órdenes de Compra</h4>
                    <p>Documento que formaliza una solicitud de compra:</p>
                    <ol>
                        <li>Selecciona el proveedor</li>
                        <li>Agrega los productos o servicios</li>
                        <li>Define cantidades y precios</li>
                        <li>Aprueba la orden</li>
                    </ol>
                    <h4><i class="fas fa-file-invoice-dollar"></i> Facturas de
Proveedor</h4>
                    <p>Registro de documentos tributarios recibidos:</p>
                    <ul>
                        <li>Genera automáticamente cuentas por pagar</li>
                        <li>Registra el IVA crédito fiscal</li>
                        <li>Actualiza el inventario (si corresponde)</li>
                        <li>Crea el asiento contable automático</li>
                    </ul>
                    <div class="help-warning">
                        <i class="fas fa-exclamation-triangle"></i>
                        <span>Las facturas de proveedor deben asociarse a una orden de
compra
                        para mantener la trazabilidad del proceso.</span>
                    </div>
                </div>
            `
        },
        ventas: {
            title: 'Módulo de Ventas (SD)',
            icon: 'fa-store',
            content: `
                <div class="help-section">
                    <h3>Gestión de Ventas</h3>
                    <p>El módulo de Ventas (Sales & Distribution - SD) gestiona todo el
proceso
                    comercial con clientes.</p>
                    <h4><i class="fas fa-user-tie"></i> Clientes</h4>
                    <p>Registro maestro de clientes:</p>
                    <ul>
                        <li>Datos fiscales y comerciales</li>
                        <li>Límite de crédito</li>
                        <li>Condiciones de pago</li>
                        <li>Historial de ventas</li>
                    </ul>
                    <h4><i class="fas fa-clipboard-list"></i> Pedidos de Venta</h4>
                    <p>Proceso de venta:</p>
                    <ol>
                        <li>Registra el pedido del cliente</li>
                        <li>Verifica disponibilidad de stock</li>
                        <li>Confirma precios y descuentos</li>
                        <li>Genera la factura</li>
                    </ol>
                    <h4><i class="fas fa-file-invoice"></i> Facturas de Venta</h4>
                    <p>Documentos tributarios emitidos:</p>
                    <ul>
                        <li>Genera automáticamente cuentas por cobrar</li>
                        <li>Registra el IVA débito fiscal</li>
                        <li>Rebaja el inventario</li>
                        <li>Contabiliza el ingreso</li>
                    </ul>
                </div>
            `
        },
        inventario: {
            title: 'Módulo de Inventario (WM)',
            icon: 'fa-boxes',
            content: `
                <div class="help-section">
                    <h3>Gestión de Inventario</h3>
                    <p>El módulo de Inventario (Warehouse Management - WM) controla el
stock
                    de productos y su valorización.</p>
                    <h4><i class="fas fa-box"></i> Productos</h4>
                    <p>Catálogo de productos y servicios:</p>
                    <ul>
                        <li>Código SKU único</li>
                        <li>Descripción y categoría</li>
                        <li>Unidad de medida</li>
                        <li>Costo promedio y precio de venta</li>
                        <li>Stock actual</li>
                    </ul>
```

```
                    <h4><i class="fas fa-exchange-alt"></i> Movimientos de Stock</h4>
                    <p>Tipos de movimientos:</p>
                    <ul>
                        <li><strong>Entrada:</strong> Recepción de compras</li>
                        <li><strong>Salida:</strong> Despacho por ventas</li>
                        <li><strong>Ajuste (+):</strong> Corrección positiva</li>
                        <li><strong>Ajuste (-):</strong> Mermas, pérdidas</li>
                        <li><strong>Transferencia:</strong> Entre bodegas</li>
                    </ul>
                    <h4><i class="fas fa-calculator"></i> Valorización</h4>
                    <p>Métodos de costeo disponibles:</p>
                    <ul>
                        <li><strong>Costo Promedio Ponderado:</strong> (Por defecto)
Recalcula con cada entrada</li>
                        <li><strong>FIFO:</strong> Primero en entrar, primero en
salir</li>
                        <li><strong>Costo Específico:</strong> Para items únicos</li>
                    </ul>
                </div>
            `
        },
        tesoreria: {
            title: 'Módulo de Tesorería (FI)',
            icon: 'fa-coins',
            content: `
                <div class="help-section">
                    <h3>Gestión de Tesorería</h3>
                    <p>El módulo de Tesorería (Financial - FI) controla el flujo de
efectivo
                    y las obligaciones financieras.</p>
                    <h4><i class="fas fa-file-invoice-dollar"></i> Cuentas por Pagar</h4>
                    <p>Control de obligaciones con proveedores:</p>
                    <ul>
                        <li>Facturas pendientes de pago</li>
                        <li>Vencimientos y antigüedad</li>
                        <li>Estado de pagos parciales</li>
                    </ul>
                    <h4><i class="fas fa-hand-holding-usd"></i> Cuentas por Cobrar</h4>
                    <p>Control de créditos con clientes:</p>
                    <ul>
                        <li>Facturas pendientes de cobro</li>
                        <li>Análisis de morosidad</li>
                        <li>Gestión de cobranza</li>
                    </ul>
                    <h4><i class="fas fa-university"></i> Bancos</h4>
                    <p>Gestión de cuentas bancarias:</p>
                    <ul>
                        <li>Registro de cuentas corrientes</li>
                        <li>Movimientos bancarios</li>
                        <li>Saldos disponibles</li>
                    </ul>
                    <h4><i class="fas fa-balance-scale"></i> Conciliación Bancaria</h4>
                    <p>Comparación entre libro banco y cartola:</p>
                    <ul>
                        <li>Identificar cheques pendientes</li>
                        <li>Detectar depósitos en tránsito</li>
                        <li>Cuadrar saldos contables con reales</li>
                    </ul>
                </div>
            `
        },
        costos: {
            title: 'Módulo de Costos (CO)',
            icon: 'fa-calculator',
            content: `
                <div class="help-section">
                    <h3>Contabilidad de Costos</h3>
                    <p>El módulo de Costos (Controlling - CO) permite analizar y controlar
                    los costos de la organización.</p>
                    <h4><i class="fas fa-sitemap"></i> Centros de Costo</h4>
                    <p>Unidades organizacionales para acumular costos:</p>
                    <ul>
                        <li>Departamentos (Ventas, Administración, Producción)</li>
                        <li>Sucursales o unidades de negocio</li>
                        <li>Proyectos específicos</li>
                    </ul>
                    <h4><i class="fas fa-tasks"></i> Órdenes Internas</h4>
                    <p>Para control de costos específicos:</p>
                    <ul>
                        <li>Proyectos con presupuesto definido</li>
                        <li>Campañas de marketing</li>
```

```
                        <li>Mantenimiento de activos</li>
                    </ul>
                    <h4><i class="fas fa-chart-pie"></i> Análisis de Costos</h4>
                    <p>Reportes de gestión:</p>
                    <ul>
                        <li>Presupuesto vs Real</li>
                        <li>Distribución de costos</li>
                        <li>Tendencias y variaciones</li>
                    </ul>
                </div>
            `
        },
        rrhh: {
            title: 'Módulo de RRHH (HR)',
            icon: 'fa-users',
            content: `
                <div class="help-section">
                    <h3>Recursos Humanos</h3>
                    <p>El módulo de RRHH (Human Resources - HR) gestiona el personal
                    y la nómina de la empresa.</p>
                    <h4><i class="fas fa-id-card"></i> Empleados</h4>
                    <p>Ficha del personal:</p>
                    <ul>
                        <li>Datos personales y contrato</li>
                        <li>Cargo y departamento</li>
                        <li>Remuneración pactada</li>
                        <li>Fechas de ingreso y términos</li>
                    </ul>
                    <h4><i class="fas fa-money-check-alt"></i> Nómina</h4>
                    <p>Cálculo de remuneraciones (normativa chilena):</p>
                    <table class="help-table">
                        <tr><td>AFP</td><td>10.00% + comisión</td></tr>
                        <tr><td>Salud</td><td>7.00%</td></tr>
                        <tr><td>Seguro Cesantía</td><td>0.60%</td></tr>
                        <tr><td>Impuesto Único</td><td>Según tabla SII</td></tr>
                    </table>
                    <h4><i class="fas fa-piggy-bank"></i> Provisiones</h4>
                    <p>Obligaciones laborales:</p>
                    <ul>
                        <li><strong>Vacaciones:</strong> 15 días hábiles al año</li>
                        <li><strong>Indemnización:</strong> Un mes por año de
servicio</li>
                        <li><strong>Gratificación:</strong> 25% sobre sueldo con tope</li>
                    </ul>
                </div>
            `
        },
        reportes: {
            title: 'Reportes Financieros',
            icon: 'fa-chart-bar',
            content: `
                <div class="help-section">
                    <h3>Estados Financieros</h3>
                    <p>Genera los reportes contables y financieros de la empresa.</p>
                    <h4><i class="fas fa-balance-scale-right"></i> Balance General</h4>
                    <p>Estado de situación financiera:</p>
                    <p style="text-align: center; padding: var(--space-3); background:
var(--neutral-100);
                        border-radius: var(--radius-md); font-weight: 600;">
                        Activos = Pasivos + Patrimonio
                    </p>
                    <p>Muestra los bienes, derechos y obligaciones a una fecha
determinada.</p>
                    <h4><i class="fas fa-chart-line"></i> Estado de Resultados</h4>
                    <p>Pérdidas y ganancias del período:</p>
                    <ul>
                        <li>(+) Ingresos operacionales</li>
                        <li>(-) Costos de venta</li>
                        <li>(=) Margen bruto</li>
                        <li>(-) Gastos operacionales</li>
                        <li>(=) Resultado operacional</li>
                        <li>(±) Otros ingresos/gastos</li>
                        <li>(=) Resultado antes de impuestos</li>
                    </ul>
                    <h4><i class="fas fa-money-bill-wave"></i> Flujo de Caja</h4>
                    <p>Movimientos de efectivo clasificados:</p>
                    <ul>
                        <li><strong>Operación:</strong> Actividades del giro</li>
                        <li><strong>Inversión:</strong> Compra/venta de activos</li>
                        <li><strong>Financiamiento:</strong> Préstamos, aportes</li>
                    </ul>
```

```
                    <div class="help-tip">
                        <i class="fas fa-download"></i>
                        <span>Todos los reportes pueden exportarse a PDF, Excel o CSV
                        usando los botones en la barra superior.</span>
                    </div>
                </div>
            `
        },
        trazabilidad: {
            title: 'Módulo de Trazabilidad',
            icon: 'fa-search-dollar',
            content: `
                <div class="help-section">
                    <h3>Trazabilidad y Auditoría</h3>
                    <p>El módulo de Trazabilidad permite rastrear el origen y destino de
cada
                    transacción en el sistema, esencial para la auditoría.</p>
                    <h4><i class="fas fa-link"></i> Seguimiento de Documentos</h4>
                    <p>Cada documento está vinculado a su origen:</p>
                    <ul>
                        <li><strong>Orden de Compra → Factura → Asiento</strong></li>
                        <li><strong>Pedido de Venta → Despacho → Factura →
Asiento</strong></li>
                        <li><strong>Movimiento de Inventario → Asiento</strong></li>
                    </ul>
                    <h4><i class="fas fa-check-double"></i> Validación de Datos</h4>
                    <p>El sistema valida automáticamente:</p>
                    <ul>
                        <li>Ecuación contable (Activos = Pasivos + Patrimonio)</li>
                        <li>Balance de asientos (Debe = Haber)</li>
                        <li>Integridad de referencias entre documentos</li>
                        <li>Consistencia de saldos de cuentas</li>
                    </ul>
                    <h4><i class="fas fa-history"></i> Log de Auditoría</h4>
                    <p>Registro de todas las acciones del sistema:</p>
                    <ul>
                        <li>Creación de registros</li>
                        <li>Modificaciones y quién las realizó</li>
                        <li>Eliminaciones y motivos</li>
                        <li>Fecha y hora de cada acción</li>
                    </ul>
                    <div class="help-tip">
                        <i class="fas fa-lightbulb"></i>
                        <span>La trazabilidad es fundamental para demostrar la integridad
de los
                        datos ante una auditoría externa.</span>
                    </div>
                </div>
            `
        },
        admin: {
            title: 'Administración del Sistema',
            icon: 'fa-cog',
            content: `
                <div class="help-section">
                    <h3>Administración</h3>
                    <p>Gestiona las empresas, configuración y datos del sistema.</p>
                    <h4><i class="fas fa-building"></i> Gestión de Empresas</h4>
                    <p>Desde aquí puedes:</p>
                    <ul>
                        <li>Crear nuevas empresas virtuales</li>
                        <li>Editar datos de la empresa actual</li>
                        <li>Cambiar entre empresas</li>
                        <li>Eliminar empresas (con precaución)</li>
                    </ul>
                    <h4><i class="fas fa-file-import"></i> Importar/Exportar</h4>
                    <p>Funciones de respaldo y transferencia:</p>
                    <ul>
                        <li>><strong>Exportar JSON:</strong> Respaldo completo de la
empresa</li>
                        <li><strong>Exportar Excel:</strong> Datos en hojas separadas</li>
                        <li><strong>Importar JSON:</strong> Restaurar respaldo o caso de
estudio</li>
                    </ul>
                    <h4><i class="fas fa-database"></i> Datos de Demostración</h4>
                    <p>Carga casos de estudio predefinidos:</p>
                    <ul>
                        <li><strong>Caso Comercial:</strong> Empresa de comercio con
inventario</li>
                        <li><strong>Caso Servicios:</strong> Consultora de servicios
profesionales</li>
```

```
                        <li><strong>Datos Básicos:</strong> Proveedores, clientes y
productos de ejemplo</li>
                    </ul>
                    <div class="help-warning">
                        <i class="fas fa-exclamation-triangle"></i>
                        <span>Recuerda hacer respaldos periódicos exportando tu empresa a
JSON.
                        Los datos solo se guardan en el navegador.</span>
                    </div>
                </div>
            `
        },
        diagrama: {
            title: 'Diagrama Contable Interactivo',
            icon: 'fa-project-diagram',
            content: `
                <div class="help-section">
                    <h3>Diagrama Contable</h3>
                    <p>Herramienta visual para entender el flujo de las transacciones
contables.</p>
                    <h4><i class="fas fa-exchange-alt"></i> Flujo de Transacción</h4>
                    <p>Visualiza cómo una operación afecta las cuentas:</p>
                    <ul>
                        <li>Documento origen (factura, nota, etc.)</li>
                        <li>Cuentas afectadas al Debe y Haber</li>
                        <li>Impacto en el Balance General</li>
                    </ul>
                    <h4><i class="fas fa-balance-scale"></i> Ecuación Contable</h4>
                    <p>Muestra en tiempo real cómo se cumple:</p>
                    <p style="text-align: center; padding: var(--space-3); background:
var(--success-50);
                        border-radius: var(--radius-md); font-weight: 600; color: var(--
success-600);">
                        ACTIVO = PASIVO + PATRIMONIO
                    </p>
                    <h4><i class="fas fa-shopping-cart"></i> Ejemplos Interactivos</h4>
                    <p>Practica con transacciones comunes:</p>
                    <ul>
                        <li>Compra a crédito:</strong> Aumenta inventario y
proveedores</li>
                        <li><strong>Venta al contado:</strong> Aumenta caja e
ingresos</li>
                        <li><strong>Pago a proveedor:</strong> Disminuye caja y
proveedores</li>
                        <li><strong>Cobro a cliente:</strong> Aumenta caja, disminuye
clientes</li>
                    </ul>
                    <div class="help-tip">
                        <i class="fas fa-lightbulb"></i>
                        <span>Accede al Diagrama Contable desde el botón "Ver Diagrama
Contable"
                        en el Dashboard principal.</span>
                    </div>
                </div>
            `
        }
    },
    faqs: [
        {
            question: '¿Cómo creo una nueva empresa?',
            answer: 'Ve a <strong>Administración → Empresas</strong> y haz clic en "Nueva
Empresa". Completa el formulario con el nombre, RUT y configuración inicial.'
        },
        {
            question: '¿Puedo exportar mis datos?',
            answer: 'Sí, puedes exportar todos los datos de tu empresa en formato JSON
(respaldo completo) o Excel (tablas separadas) desde <strong>Administración →
Importar/Exportar</strong>.'
        },
        {
            question: '¿Los datos se guardan automáticamente?',
            answer: 'Sí, todos los datos se guardan localmente en tu navegador usando
IndexedDB. No requiere conexión a internet.'
        },
        {
            question: '¿Qué pasa si borro los datos del navegador?',
            answer: 'Los datos de EDU-TRACE ERP se perderán. Recomendamos hacer respaldos
periódicos exportando la empresa a JSON.'
        },
        {
            question: '¿Puedo compartir casos con compañeros?',
```

```
            answer: 'Sí, exporta tu empresa a JSON y comparte el archivo. Tu compañero
puede importarlo en su navegador.'
        },
        {
            question: '¿Se generan asientos automáticos?',
            answer: 'Sí, al registrar facturas de compra, venta, pagos y otros documentos,
el sistema genera automáticamente los asientos contables correspondientes.'
        },
        {
            question: '¿Qué normativa contable usa el sistema?',
            answer: 'EDU-TRACE ERP usa un plan de cuentas basado en IFRS adaptado a la
normativa chilena, con IVA al 19%.'
        }
    ],
    show(moduleId = null) {
        const currentModule = moduleId || App.currentModule || 'general';
        const doc = this.documentation[currentModule] || this.documentation.general;
        Modal.open({
            title: `<i class="fas ${doc.icon}"></i> ${doc.title}`,
            size: 'large',
            content: `
                <div class="help-container">
                    <div class="help-sidebar">
                        <h4>Módulos</h4>
                        <ul class="help-nav">
                            ${Object.entries(this.documentation).map(([id, d]) => `
                                <li class="${id === currentModule ? 'active' : ''}" data-
module="${id}">
                                    <i class="fas ${d.icon}"></i>
                                    <span>${d.title.replace('Módulo de ', '').split(' (')
[0]}</span>
                                </li>
                            `).join('')}
                        </ul>
                        <h4>Accesos Rápidos</h4>
                        <ul class="help-quick">
                            <li onclick="Modal.close();
TutorialSystem.showTutorialSelector();">
                                <i class="fas fa-graduation-cap"></i> Tutoriales
                            </li>
                            <li onclick="HelpSystem.showFAQ()">
                                <i class="fas fa-question-circle"></i> FAQ
                            </li>
                            <li onclick="HelpSystem.showShortcuts()">
                                <i class="fas fa-keyboard"></i> Atajos
                            </li>
                        </ul>
                    </div>
                    <div class="help-content" id="help-content">
                        ${doc.content}
                    </div>
                </div>
            `,
            footer: `
                <button class="btn btn-primary" onclick="Modal.close();
TutorialSystem.start();">
                    <i class="fas fa-play"></i> Iniciar Tutorial
                </button>
                <button class="btn btn-outline" onclick="Modal.close()">Cerrar</button>
            `
        });
        document.querySelectorAll('.help-nav li').forEach(li => {
            li.addEventListener('click', () => {
                const moduleId = li.dataset.module;
                const doc = this.documentation[moduleId];
                if (doc) {
                    document.querySelectorAll('.help-nav li').forEach(l =>
l.classList.remove('active'));
                    li.classList.add('active');
                    document.getElementById('help-content').innerHTML = doc.content;
                }
            });
        });
    },
    showFAQ() {
        Modal.open({
            title: '<i class="fas fa-question-circle"></i> Preguntas Frecuentes',
            size: 'large',
            content: `
                <div class="faq-list">
                    ${this.faqs.map((faq, i) => `
```

```
                        <div class="faq-item">
                            <div class="faq-question"
onclick="this.parentElement.classList.toggle('open')">
                                <span>${faq.question}</span>
                                <i class="fas fa-chevron-down"></i>
                            </div>
                            <div class="faq-answer">
                                <p>${faq.answer}</p>
                            </div>
                        </div>
                    `).join('')}
                </div>
            `,
            footer: `
                <button class="btn btn-outline" onclick="HelpSystem.show()">
                    <i class="fas fa-arrow-left"></i> Volver a Ayuda
                </button>
                <button class="btn btn-primary" onclick="Modal.close()">Cerrar</button>
            `
        });
    },
    showShortcuts() {
        Modal.open({
            title: '<i class="fas fa-keyboard"></i> Atajos de Teclado',
            size: 'medium',
            content: `
                <div class="shortcuts-list">
                    <div class="shortcut-group">
                        <h4>Navegación</h4>
                        <div class="shortcut-item">
                            <span class="shortcut-keys"><kbd>Alt</kbd> + <kbd>D</kbd>
</span>
                            <span class="shortcut-desc">Ir al Dashboard</span>
                        </div>
                        <div class="shortcut-item">
                            <span class="shortcut-keys"><kbd>Alt</kbd> + <kbd>C</kbd>
</span>
                            <span class="shortcut-desc">Ir a Contabilidad</span>
                        </div>
                        <div class="shortcut-item">
                            <span class="shortcut-keys"><kbd>Alt</kbd> + <kbd>R</kbd>
</span>
                            <span class="shortcut-desc">Ir a Reportes</span>
                        </div>
                    </div>
                    <div class="shortcut-group">
                        <h4>Acciones</h4>
                        <div class="shortcut-item">
                            <span class="shortcut-keys"><kbd>Ctrl</kbd> + <kbd>S</kbd>
</span>
                            <span class="shortcut-desc">Guardar</span>
                        </div>
                        <div class="shortcut-item">
                            <span class="shortcut-keys"><kbd>Ctrl</kbd> + <kbd>N</kbd>
</span>
                            <span class="shortcut-desc">Nuevo registro</span>
                        </div>
                        <div class="shortcut-item">
                            <span class="shortcut-keys"><kbd>Ctrl</kbd> + <kbd>F</kbd>
</span>
                            <span class="shortcut-desc">Buscar</span>
                        </div>
                    </div>
                    <div class="shortcut-group">
                        <h4>Sistema</h4>
                        <div class="shortcut-item">
                            <span class="shortcut-keys"><kbd>Esc</kbd></span>
                            <span class="shortcut-desc">Cerrar modal/cancelar</span>
                        </div>
                        <div class="shortcut-item">
                            <span class="shortcut-keys"><kbd>F1</kbd></span>
                            <span class="shortcut-desc">Abrir ayuda</span>
                        </div>
                        <div class="shortcut-item">
                            <span class="shortcut-keys"><kbd>?</kbd></span>
                            <span class="shortcut-desc">Mostrar atajos</span>
                        </div>
                    </div>
                </div>
            `,
            footer: `
```

```
                <button class="btn btn-outline" onclick="HelpSystem.show()">
                    <i class="fas fa-arrow-left"></i> Volver
                </button>
                <button class="btn btn-primary" onclick="Modal.close()">Cerrar</button>
            `
        });
    }
};
const helpStyles = document.createElement('style');
helpStyles.textContent = `
    .help-container {
        display: flex;
        gap: var(--space-6);
        min-height: 400px;
    }
    .help-sidebar {
        width: 200px;
        flex-shrink: 0;
        border-right: 1px solid var(--border-light);
        padding-right: var(--space-4);
    }
    .help-sidebar h4 {
        font-size: var(--font-size-xs);
        text-transform: uppercase;
        color: var(--text-tertiary);
        margin: 0 0 var(--space-2);
        padding-top: var(--space-3);
    }
    .help-sidebar h4:first-child { padding-top: 0; }
    .help-nav, .help-quick {
        list-style: none;
        margin: 0 0 var(--space-4);
        padding: 0;
    }
    .help-nav li, .help-quick li {
        display: flex;
        align-items: center;
        gap: var(--space-2);
        padding: var(--space-2) var(--space-3);
        border-radius: var(--radius-md);
        cursor: pointer;
        font-size: var(--font-size-sm);
        color: var(--text-secondary);
        transition: all var(--transition-fast);
    }
    .help-nav li:hover, .help-quick li:hover {
        background: var(--neutral-100);
        color: var(--text-primary);
    }
    .help-nav li.active {
        background: var(--primary-50);
        color: var(--primary-600);
        font-weight: 500;
    }
    .help-nav li i, .help-quick li i {
        width: 16px;
        text-align: center;
    }
    .help-content {
        flex: 1;
        overflow-y: auto;
        max-height: 500px;
    }
    .help-section h3 {
        margin: 0 0 var(--space-3);
        color: var(--text-primary);
        font-size: var(--font-size-lg);
    }
    .help-section h4 {
        margin: var(--space-5) 0 var(--space-2);
        color: var(--primary-600);
        font-size: var(--font-size-base);
        display: flex;
        align-items: center;
        gap: var(--space-2);
    }
    .help-section p {
        margin: 0 0 var(--space-3);
        line-height: 1.6;
        color: var(--text-secondary);
    }
```

```
.help-section ul, .help-section ol {
    margin: var(--space-2) 0;
    padding-left: var(--space-5);
    color: var(--text-secondary);
}
.help-section li {
    margin-bottom: var(--space-1);
    line-height: 1.5;
}
.help-table {
    width: 100%;
    border-collapse: collapse;
    margin: var(--space-3) 0;
    font-size: var(--font-size-sm);
}
.help-table td {
    padding: var(--space-2) var(--space-3);
    border: 1px solid var(--border-light);
}
.help-table tr:nth-child(even) {
    background: var(--neutral-50);
}
.help-tip, .help-warning {
    display: flex;
    gap: var(--space-3);
    padding: var(--space-3);
    border-radius: var(--radius-md);
    margin: var(--space-4) 0;
    font-size: var(--font-size-sm);
}
.help-tip {
    background: var(--info-50);
    color: var(--info-500);
}
.help-warning {
    background: var(--warning-50);
    color: var(--warning-600);
}
.help-tip i, .help-warning i {
    font-size: var(--font-size-lg);
    flex-shrink: 0;
}
.faq-list {
    display: flex;
    flex-direction: column;
    gap: var(--space-2);
}
.faq-item {
    border: 1px solid var(--border-light);
    border-radius: var(--radius-md);
    overflow: hidden;
}
.faq-question {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: var(--space-4);
    cursor: pointer;
    font-weight: 500;
    background: var(--neutral-50);
    transition: all var(--transition-fast);
}
.faq-question:hover {
    background: var(--neutral-100);
}
.faq-question i {
    transition: transform var(--transition-fast);
    color: var(--text-tertiary);
}
.faq-item.open .faq-question i {
    transform: rotate(180deg);
}
.faq-answer {
    max-height: 0;
    overflow: hidden;
    transition: max-height var(--transition-normal);
}
.faq-item.open .faq-answer {
    max-height: 200px;
}
.faq-answer p {
```

```
        padding: var(--space-4);
        margin: 0;
        color: var(--text-secondary);
        border-top: 1px solid var(--border-light);
    }
    .shortcuts-list {
        display: flex;
        flex-direction: column;
        gap: var(--space-5);
    }
    .shortcut-group h4 {
        margin: 0 0 var(--space-3);
        font-size: var(--font-size-sm);
        color: var(--text-tertiary);
        text-transform: uppercase;
    }
    .shortcut-item {
        display: flex;
        justify-content: space-between;
        align-items: center;
        padding: var(--space-2) 0;
        border-bottom: 1px solid var(--border-light);
    }
    .shortcut-item:last-child {
        border-bottom: none;
    }
    .shortcut-keys {
        display: flex;
        gap: var(--space-1);
    }
    .shortcut-keys kbd {
        background: var(--neutral-100);
        border: 1px solid var(--border-medium);
        border-radius: var(--radius-sm);
        padding: var(--space-1) var(--space-2);
        font-family: monospace;
        font-size: var(--font-size-xs);
        box-shadow: 0 1px 0 var(--neutral-300);
    }
    .shortcut-desc {
        color: var(--text-secondary);
        font-size: var(--font-size-sm);
    }
`;
document.head.appendChild(helpStyles);
document.addEventListener('keydown', (e) => {
    if (e.key === 'F1') {
        e.preventDefault();
        HelpSystem.show();
    }
    if (e.key === '?' && !e.ctrlKey && !e.altKey) {
        const activeElement = document.activeElement;
        if (!activeElement || !['INPUT', 'TEXTAREA'].includes(activeElement.tagName)) {
            HelpSystem.showShortcuts();
        }
    }
});
window.HelpSystem = HelpSystem;
```

Archivo: js\components\modal.js

```
const Modal = {
    container: null,
    modal: null,
    isOpen: false,
    onClose: null,
    init() {
        this.container = document.getElementById('modal-container');
        this.modal = document.getElementById('modal');
        document.getElementById('modal-close')?.addEventListener('click', () =>
this.close());
        document.querySelector('.modal-overlay')?.addEventListener('click', () =>
this.close());
        document.addEventListener('keydown', (e) => {
            if (e.key === 'Escape' && this.isOpen) this.close();
```

```
        });
    },
    open(options = {}) {
        const {
            title = 'Modal',
            content = '',
            footer = '',
            size = 'medium',
            onClose = null,
            closeOnOverlay = true
        } = options;
        this.onClose = onClose;
        this.modal.className = `modal modal-${size}`;
        document.getElementById('modal-title').textContent = title;
        document.getElementById('modal-body').innerHTML = content;
        document.getElementById('modal-footer').innerHTML = footer;
        if (!closeOnOverlay) {
            document.querySelector('.modal-overlay').style.pointerEvents = 'none';
        }
        this.container.classList.remove('hidden');
        document.body.style.overflow = 'hidden';
        this.isOpen = true;
        requestAnimationFrame(() => {
            this.modal.classList.add('modal-open');
        });
        return this;
    },
    close() {
        if (!this.isOpen) return;
        this.modal.classList.remove('modal-open');
        setTimeout(() => {
            this.container.classList.add('hidden');
            document.body.style.overflow = '';
            this.isOpen = false;
            if (this.onClose) this.onClose();
        }, 200);
    },
    confirm(options = {}) {
        const {
            title = 'Confirmar',
            message = '¿Está seguro?',
            confirmText = 'Confirmar',
            cancelText = 'Cancelar',
            confirmClass = 'btn-primary',
            onConfirm = () => { },
            onCancel = () => { }
        } = options;
        return new Promise((resolve) => {
            this.open({
                title,
                content: `<p class="confirm-message">${message}</p>`,
                footer: `
                    <button class="btn btn-secondary" id="modal-cancel">${cancelText}
</button>
                    <button class="btn ${confirmClass}" id="modal-confirm">${confirmText}
</button>
                `,
                size: 'small',
                onClose: () => {
                    onCancel();
                    resolve(false);
                }
            });
            document.getElementById('modal-cancel').onclick = () => {
                this.close();
                onCancel();
                resolve(false);
            };
            document.getElementById('modal-confirm').onclick = () => {
                this.close();
                onConfirm();
                resolve(true);
            };
        });
    },
    alert(options = {}) {
        const {
            title = 'Aviso',
            message = '',
            type = 'info',
            buttonText = 'Aceptar'
```

```
        } = options;
        const icons = {
            info: 'fa-info-circle',
            success: 'fa-check-circle',
            warning: 'fa-exclamation-triangle',
            error: 'fa-times-circle'
        };
        return new Promise((resolve) => {
            this.open({
                title,
                content: `
                    <div class="alert-modal alert-${type}">
                        <i class="fas ${icons[type]}"></i>
                        <p>${message}</p>
                    </div>
                `,
                footer: `<button class="btn btn-primary" id="modal-accept">${buttonText}
</button>`,
                size: 'small'
            });
            document.getElementById('modal-accept').onclick = () => {
                this.close();
                resolve(true);
            };
        });
    },
    form(options = {}) {
        const {
            title = 'Formulario',
            fields = [],
            data = {},
            submitText = 'Guardar',
            cancelText = 'Cancelar',
            onSubmit = () => { },
            validate = () => true
        } = options;
        const fieldsHtml = fields.map(field => {
            const value = data[field.name] ?? field.default ?? '';
            const required = field.required ? 'required' : '';
            const requiredMark = field.required ? '<span class="text-error">*</span>' :
'';
            let input = '';
            switch (field.type) {
                case 'select':
                    const optionsHtml = field.options.map(opt =>
                        `<option value="${opt.value}" ${opt.value == value ? 'selected' :
''}>${opt.label}</option>`
                    ).join('');
                    input = `<select class="form-control" name="${field.name}"
id="field-${field.name}" ${required}>${optionsHtml}</select>`;
                    break;
                case 'textarea':
                    input = `<textarea class="form-control" name="${field.name}"
id="field-${field.name}" rows="${field.rows || 3}" ${required}>${value}</textarea>`;
                    break;
                case 'number':
                    input = `<input type="number" class="form-control"
name="${field.name}" id="field-${field.name}" value="${value}"
                        ${field.min !== undefined ? `min="${field.min}"` : ''}
                        ${field.max !== undefined ? `max="${field.max}"` : ''}
                        ${field.step ? `step="${field.step}"` : ''} ${required}>`;
                    break;
                case 'date':
                    input = `<input type="date" class="form-control" name="${field.name}"
id="field-${field.name}" value="${value}" ${required}>`;
                    break;
                case 'checkbox':
                    input = `<label class="checkbox-item"><input type="checkbox"
name="${field.name}" id="field-${field.name}" ${value ? 'checked' : ''}>
${field.checkLabel || ''}</label>`;
                    break;
                default:
                    input = `<input type="${field.type || 'text'}" class="form-control"
name="${field.name}" id="field-${field.name}" value="${value}"
                        ${field.placeholder ? `placeholder="${field.placeholder}"` : ''}
${required}>`;
            }
            return `
                <div class="form-group">
                    <label class="form-label" for="field-${field.name}">${field.label}
${requiredMark}</label>
```

```
                    ${input}
                    ${field.hint ? `<span class="form-hint">${field.hint}</span>` : ''}
                </div>
            `;
        }).join('');
        return new Promise((resolve) => {
            this.open({
                title,
                content: `<form id="modal-form" class="modal-form">${fieldsHtml}</form>`,
                footer: `
                    <button type="button" class="btn btn-secondary" id="modal-
cancel">${cancelText}</button>
                    <button type="submit" form="modal-form" class="btn btn-primary"
id="modal-submit">${submitText}</button>
                `,
                size: 'medium',
                closeOnOverlay: false
            });
            document.getElementById('modal-cancel').onclick = () => {
                this.close();
                resolve(null);
            };
            document.getElementById('modal-form').onsubmit = async (e) => {
                e.preventDefault();
                const formData = new FormData(e.target);
                const result = {};
                fields.forEach(field => {
                    if (field.type === 'checkbox') {
                        result[field.name] = formData.has(field.name);
                    } else if (field.type === 'number') {
                        result[field.name] = parseFloat(formData.get(field.name)) || 0;
                    } else {
                        result[field.name] = formData.get(field.name);
                    }
                });
                const validationResult = await validate(result);
                if (validationResult !== true) {
                    Toast.show(validationResult || 'Error de validación', 'error');
                    return;
                }
                await onSubmit(result);
                this.close();
                resolve(result);
            };
        });
    },
    setContent(content) {
        document.getElementById('modal-body').innerHTML = content;
    },
    setFooter(footer) {
        document.getElementById('modal-footer').innerHTML = footer;
    },
    custom(options = {}) {
        const {
            title = 'Modal',
            content = '',
            size = 'medium',
            buttons = [],
            onClose = null,
            closeOnOverlay = true
        } = options;
        const sizeMap = {
            'sm': 'small',
            'md': 'medium',
            'lg': 'large',
            'xl': 'xlarge'
        };
        const modalSize = sizeMap[size] || size;
        return new Promise((resolve) => {
            const buttonsHtml = buttons.map((btn, idx) => {
                const btnClass = btn.class || 'btn-secondary';
                return `<button class="btn ${btnClass}" id="modal-custom-
btn-${idx}">${btn.text || 'Botón'}</button>`;
            }).join('');
            this.open({
                title,
                content,
                footer: buttonsHtml || '<button class="btn btn-secondary" id="modal-
custom-close">Cerrar</button>',
                size: modalSize,
                closeOnOverlay,
```

```
                onClose: () => {
                    if (onClose) onClose();
                    resolve(null);
                }
            });
            buttons.forEach((btn, idx) => {
                const btnElement = document.getElementById(`modal-custom-btn-${idx}`);
                if (btnElement) {
                    btnElement.onclick = async () => {
                        if (btn.action === 'close') {
                            this.close();
                            resolve(null);
                        } else if (typeof btn.action === 'function') {
                            const result = await btn.action();
                            if (result !== false) {
                            }
                            resolve(result);
                        } else {
                            this.close();
                            resolve(btn.action);
                        }
                    };
                }
            });
            const defaultCloseBtn = document.getElementById('modal-custom-close');
            if (defaultCloseBtn) {
                defaultCloseBtn.onclick = () => {
                    this.close();
                    resolve(null);
                };
            }
        });
    }
};
const modalStyles = document.createElement('style');
modalStyles.textContent = `
    .modal-container { position: fixed; inset: 0; z-index: var(--z-modal); display: flex;
align-items: center; justify-content: center; }
    .modal-overlay { position: absolute; inset: 0; background: rgba(0,0,0,0.5); backdrop-
filter: blur(2px); }
    .modal { position: relative; background: var(--bg-secondary); border-radius: var(--
radius-xl); box-shadow: var(--shadow-xl); max-height: 90vh; display: flex; flex-direction:
column; transform: scale(0.95); opacity: 0; transition: all 0.2s ease; }
    .modal.modal-open { transform: scale(1); opacity: 1; }
    .modal-small { width: 400px; }
    .modal-medium { width: 600px; }
    .modal-large { width: 900px; }
    .modal-xlarge { width: 1200px; max-width: 95vw; }
    .modal-header { padding: var(--space-4) var(--space-5); border-bottom: 1px solid var(-
-border-light); display: flex; align-items: center; justify-content: space-between; }
    .modal-title { font-size: var(--font-size-lg); font-weight: 600; color: var(--text-
primary); }
    .modal-close { background: none; border: none; font-size: var(--font-size-lg); color:
var(--text-tertiary); cursor: pointer; padding: var(--space-2); border-radius: var(--
radius-md); transition: all var(--transition-fast); }
    .modal-close:hover { background: var(--neutral-100); color: var(--text-primary); }
    .modal-body { padding: var(--space-5); overflow-y: auto; flex: 1; }
    .modal-footer { padding: var(--space-4) var(--space-5); border-top: 1px solid var(--
border-light); display: flex; justify-content: flex-end; gap: var(--space-3); background:
var(--neutral-50); border-radius: 0 0 var(--radius-xl) var(--radius-xl); }
    .confirm-message { font-size: var(--font-size-base); color: var(--text-primary); text-
align: center; padding: var(--space-4) 0; }
    .alert-modal { text-align: center; padding: var(--space-4); }
    .alert-modal i { font-size: 3rem; margin-bottom: var(--space-4); }
    .alert-modal.alert-info i { color: var(--info-500); }
    .alert-modal.alert-success i { color: var(--success-500); }
    .alert-modal.alert-warning i { color: var(--warning-500); }
    .alert-modal.alert-error i { color: var(--error-500); }
    .alert-modal p { font-size: var(--font-size-base); color: var(--text-primary); }
`;
document.head.appendChild(modalStyles);
window.Modal = Modal;
```

Archivo: js\components\system-flow.js

```
const SystemFlowDiagram = {
    show() {
        Modal.open({
            title: '<i class="fas fa-project-diagram"></i> Diagrama de Flujo del Sistema',
            size: 'xlarge',
            content: this.render(),
            footer: `
                <button class="btn btn-outline" onclick="Modal.close()">Cerrar</button>
                <button class="btn btn-primary" onclick="SystemFlowDiagram.downloadPDF()">
                    <i class="fas fa-download"></i> Descargar PDF
                </button>
            `
        });
        setTimeout(() => this.initInteractivity(), 100);
    },
    render() {
        return `
            <div class="flow-diagram-container">
                <div class="flow-legend">
                    <h4><i class="fas fa-info-circle"></i> Guía del Diagrama</h4>
                    <p>Haz clic en cada módulo para ver más detalles. Las flechas muestran
cómo fluye la información.</p>
                    <div class="legend-items">
                        <span><i class="fas fa-circle" style="color: var(--primary-500);">
</i> Módulo Principal</span>
                        <span><i class="fas fa-arrow-right" style="color: var(--success-
500);"></i> Genera asiento</span>
                        <span><i class="fas fa-arrow-right" style="color: var(--warning-
500);"></i> Actualiza datos</span>
                    </div>
                </div>
                <div class="flow-diagram" id="flow-diagram">
                    <!-- Nivel 1: Punto de Entrada -->
                    <div class="flow-level">
                        <div class="flow-node entry-point" data-module="dashboard">
                            <div class="node-icon"><i class="fas fa-home"></i></div>
                            <div class="node-content">
                                <h4>Dashboard</h4>
                                <p>Vista general</p>
                            </div>
                        </div>
                    </div>
                    <!-- Nivel 2: Operaciones -->
                    <div class="flow-arrows-row">
                        <div class="arrow-down"><i class="fas fa-chevron-down"></i></div>
                    </div>
                    <div class="flow-level operations">
                        <div class="flow-node module-node" data-module="compras">
                            <div class="node-icon mm"><i class="fas fa-shopping-cart"></i>
</div>
                            <div class="node-content">
                                <h4>MM - Compras</h4>
                                <p>Órdenes de compra<br>Facturas proveedor</p>
                            </div>
                            <div class="node-badge">Proveedores</div>
                        </div>
                        <div class="flow-node module-node" data-module="ventas">
                            <div class="node-icon sd"><i class="fas fa-chart-line"></i>
</div>
                            <div class="node-content">
                                <h4>SD - Ventas</h4>
                                <p>Cotizaciones<br>Facturas cliente</p>
                            </div>
                            <div class="node-badge">Clientes</div>
                        </div>
                        <div class="flow-node module-node" data-module="rrhh">
                            <div class="node-icon hr"><i class="fas fa-users"></i></div>
                            <div class="node-content">
                                <h4>HR - RRHH</h4>
                                <p>Nómina<br>Liquidaciones</p>
                            </div>
                            <div class="node-badge">Empleados</div>
                        </div>
                        <div class="flow-node module-node" data-module="trazabilidad">
                            <div class="node-icon trace"><i class="fas fa-project-
diagram"></i></div>
```

```
                            <div class="node-content">
                                <h4>Trazabilidad</h4>
                                <p>Auditoría<br>Validaciones</p>
                            </div>
                            <div class="node-badge">Control</div>
                        </div>
                    </div>
                    <!-- Flechas hacia Inventario y Tesorería -->
                    <div class="flow-arrows-row complex">
                        <div class="arrow-connector">
                            <svg viewBox="0 0 800 60" class="connector-svg">
                                <path d="M 130 0 L 130 30 L 270 30 L 270 60" stroke="var(-
-warning-500)" fill="none" stroke-width="2" marker-end="url(#arrowhead-warning)"/>
                                <path d="M 400 0 L 400 60" stroke="var(--success-500)"
fill="none" stroke-width="2" marker-end="url(#arrowhead-success)"/>
                                <path d="M 670 0 L 670 30 L 530 30 L 530 60" stroke="var(-
-success-500)" fill="none" stroke-width="2" marker-end="url(#arrowhead-success)"/>
                                <defs>
                                    <marker id="arrowhead-warning" markerWidth="10"
markerHeight="7" refX="10" refY="3.5" orient="auto">
                                        <polygon points="0 0, 10 3.5, 0 7" fill="var(--
warning-500)"/>
                                    </marker>
                                    <marker id="arrowhead-success" markerWidth="10"
markerHeight="7" refX="10" refY="3.5" orient="auto">
                                        <polygon points="0 0, 10 3.5, 0 7" fill="var(--
success-500)"/>
                                    </marker>
                                </defs>
                            </svg>
                        </div>
                    </div>
                    <!-- Nivel 3: Inventario y Tesorería -->
                    <div class="flow-level support">
                        <div class="flow-node module-node" data-module="inventario">
                            <div class="node-icon wm"><i class="fas fa-boxes"></i></div>
                            <div class="node-content">
                                <h4>WM - Inventario</h4>
                                <p>Stock<br>Valorización</p>
                            </div>
                            <div class="node-badge">Productos</div>
                        </div>
                        <div class="flow-node module-node" data-module="tesoreria">
                            <div class="node-icon fi"><i class="fas fa-landmark"></i>
</div>
                            <div class="node-content">
                                <h4>FI - Tesorería</h4>
                                <p>Bancos<br>Pagos y Cobros</p>
                            </div>
                            <div class="node-badge">Caja/Bancos</div>
                        </div>
                    </div>
                    <!-- Flechas hacia Contabilidad -->
                    <div class="flow-arrows-row final">
                        <div class="arrow-connector all-to-one">
                            <svg viewBox="0 0 800 80" class="connector-svg">
                                <path d="M 120 0 L 120 40 L 400 40 L 400 80" stroke="var(-
-success-500)" fill="none" stroke-width="2"/>
                                <path d="M 270 0 L 270 40" stroke="var(--success-500)"
fill="none" stroke-width="2"/>
                                <path d="M 530 0 L 530 40" stroke="var(--success-500)"
fill="none" stroke-width="2"/>
                                <path d="M 680 0 L 680 40 L 400 40" stroke="var(--success-
500)" fill="none" stroke-width="2"/>
                                <circle cx="400" cy="40" r="5" fill="var(--success-500)"/>
                                <path d="M 400 40 L 400 80" stroke="var(--success-500)"
fill="none" stroke-width="3" marker-end="url(#arrowhead-success-lg)"/>
                                <defs>
                                    <marker id="arrowhead-success-lg" markerWidth="12"
markerHeight="9" refX="12" refY="4.5" orient="auto">
                                        <polygon points="0 0, 12 4.5, 0 9" fill="var(--
success-500)"/>
                                    </marker>
                                </defs>
                            </svg>
                        </div>
                    </div>
                    <!-- Nivel 4: Contabilidad Central -->
                    <div class="flow-level central">
                        <div class="flow-node core-node" data-module="contabilidad">
                            <div class="node-icon gl"><i class="fas fa-book"></i></div>
```

```
                            <div class="node-content">
                                <h4>GL - Contabilidad</h4>
                                <p>Plan de Cuentas • Asientos • Libro Mayor • Libro
Diario</p>
                                <span class="core-badge">Módulo Central</span>
                            </div>
                        </div>
                    </div>
                    <!-- Flecha hacia Reportes -->
                    <div class="flow-arrows-row">
                        <div class="arrow-down"><i class="fas fa-chevron-down"></i></div>
                    </div>
                    <!-- Nivel 5: Salidas -->
                    <div class="flow-level outputs">
                        <div class="flow-node output-node" data-module="reportes">
                            <div class="node-icon reports"><i class="fas fa-chart-pie">
</i></div>
                            <div class="node-content">
                                <h4>Reportes Financieros</h4>
                                <p>Balance General • Estado de Resultados • Flujo de
Caja</p>
                            </div>
                        </div>
                        <div class="flow-node output-node" data-module="costos">
                            <div class="node-icon co"><i class="fas fa-calculator"></i>
</div>
                            <div class="node-content">
                                <h4>CO - Costos</h4>
                                <p>Centros de Costo<br>Análisis de Rentabilidad</p>
                            </div>
                        </div>
                    </div>
                </div>
                <!-- Panel de detalles -->
                <div class="flow-details" id="flow-details">
                    <h4><i class="fas fa-mouse-pointer"></i> Selecciona un módulo</h4>
                    <p>Haz clic en cualquier módulo del diagrama para ver cómo se integra
con el sistema.</p>
                </div>
            </div>
        `;
    },
    moduleInfo: {
        dashboard: {
            title: 'Dashboard Principal',
            icon: 'fa-home',
            description: 'Vista centralizada de la situación financiera de la empresa.',
            inputs: [],
            outputs: ['KPIs financieros', 'Gráficos de tendencia', 'Alertas del sistema'],
            accounting: 'No genera asientos directamente, solo visualiza información
consolidada.'
        },
        compras: {
            title: 'Módulo de Compras (MM)',
            icon: 'fa-shopping-cart',
            description: 'Gestiona las adquisiciones de bienes y servicios de la
empresa.',
            inputs: ['Orden de compra', 'Factura de proveedor', 'Nota de crédito'],
            outputs: ['Actualiza inventario', 'Genera asiento contable', 'Crea cuenta por
pagar'],
            accounting: 'Al registrar una compra se generan asientos que afectan
Inventario, IVA Crédito y Proveedores.'
        },
        ventas: {
            title: 'Módulo de Ventas (SD)',
            icon: 'fa-chart-line',
            description: 'Administra el ciclo completo de ventas, desde cotizaciones hasta
facturas.',
            inputs: ['Cotización', 'Pedido de venta', 'Factura a cliente'],
            outputs: ['Actualiza inventario (-)', 'Genera asiento contable', 'Crea cuenta
por cobrar'],
            accounting: 'Al registrar una venta se generan asientos que afectan Clientes,
Ingresos por Ventas, IVA Débito y actualizan Inventario.'
        },
        inventario: {
            title: 'Módulo de Inventario (WM)',
            icon: 'fa-boxes',
            description: 'Controla el stock de productos, ubicaciones y valorización.',
            inputs: ['Entrada de mercadería', 'Salida de mercadería', 'Ajuste de
inventario'],
            outputs: ['Actualiza stock', 'Genera asiento de ajuste', 'Valorización
```

```
FIFO/Promedio'],
            accounting: 'Los ajustes de inventario generan asientos que actualizan
Inventario y registran ajustes o pérdidas.'
        },
        tesoreria: {
            title: 'Módulo de Tesorería (FI)',
            icon: 'fa-landmark',
            description: 'Gestiona la liquidez: cuentas bancarias, pagos, cobros y
conciliaciones.',
            inputs: ['Pago a proveedor', 'Cobro a cliente', 'Transferencia bancaria'],
            outputs: ['Actualiza saldo bancario', 'Genera asiento contable', 'Conciliación
bancaria'],
            accounting: 'Los pagos y cobros generan asientos que afectan Proveedores,
Banco y Clientes según corresponda.'
        },
        rrhh: {
            title: 'Módulo de RRHH (HR)',
            icon: 'fa-users',
            description: 'Administra empleados, contratos, asistencia y remuneraciones.',
            inputs: ['Registro de empleado', 'Control de asistencia', 'Cálculo de
nómina'],
            outputs: ['Liquidación de sueldo', 'Genera asiento de nómina', 'Provisiones
laborales'],
            accounting: 'La nómina genera asientos que registran gastos de remuneraciones
y pasivos por pagar (AFP, salud, impuestos).'
        },
        contabilidad: {
            title: 'Contabilidad General (GL)',
            icon: 'fa-book',
            description: 'Es el corazón del sistema. Todos los módulos generan asientos
aquí.',
            inputs: ['Asientos de todos los módulos', 'Libros auxiliares (Centralizador)',
'Ajustes manuales', 'Cierre de período'],
            outputs: ['Libro Mayor', 'Libro Diario', 'Libros Auxiliares', 'Balances de
comprobación (nexo hacia estados financieros)'],
            accounting: `
                <div class="central-module-info">
                    <p><strong>Sistemas de Contabilización:</strong></p>
                    <ul style="font-size: var(--font-size-sm); margin:var(--space-2) 0;">
                        <li><strong>Jornalizador:</strong> Registro directo en Libro
Diario</li>
                        <li><strong>Centralizador:</strong> Registro en libros auxiliares
+ centralización mensual</li>
                    </ul>
                    <p><strong>Principio Fundamental:</strong></p>
                    <div class="equation">DEBE = HABER</div>
                    <p>Cada transacción afecta al menos dos cuentas, manteniendo siempre
la ecuación contable:</p>
                    <div class="equation">ACTIVO = PASIVO + PATRIMONIO</div>
                </div>
            `
        },
        reportes: {
            title: 'Reportes Financieros',
            icon: 'fa-chart-pie',
            description: 'Genera los estados financieros a partir de los datos
contables.',
            inputs: ['Datos del libro mayor', 'Balance de de comprobación (Nexo)'],
            outputs: ['Balance General', 'Estado de Resultados', 'Flujo de Caja',
'Análisis financiero'],
            accounting: `
                <div class="reports-info">
                    <p><strong>Estados Financieros Principales:</strong></p>
                    <ul>
                        <li><strong>Balance General:</strong> Activo = Pasivo +
Patrimonio</li>
                        <li><strong>Estado de Resultados:</strong> Ingresos - Costos de
Venta - Gastos = Resultado del Ejercicio (Utilidad o Pérdida)</li>
                        <li><strong>Flujo de Caja:</strong> Movimientos de efectivo del
período</li>
                    </ul>
                </div>
            `
        },
        costos: {
            title: 'Módulo de Costos (CO)',
            icon: 'fa-calculator',
            description: 'Analiza costos por centro de costo, proyecto o producto.',
            inputs: ['Asignación de costos', 'Distribución de gastos'],
            outputs: ['Costeo por absorción', 'Costeo variable', 'Rentabilidad por
producto'],
```

```
            accounting: 'El módulo de Costos permite analizar la distribución de costos
por centro sin generar asientos contables.'
        },
        trazabilidad: {
            title: 'Módulo de Trazabilidad',
            icon: 'fa-project-diagram',
            description: 'Sistema de auditoría y seguimiento pedagógico de
transacciones.',
            inputs: ['Todas las transacciones del sistema', 'Asientos contables',
'Documentos fuente'],
            outputs: ['Flujos de trazabilidad', 'Validaciones de integridad', 'Reportes de
auditoría pedagógica'],
            accounting: `
                <div class="traceability-info">
                    <p><strong>Funciones Principales:</strong></p>
                    <ul>
                        <li><strong>Rastreo de documentos:</strong> Desde el documento
origen hasta los estados financieros</li>
                        <li><strong>Validación de asientos:</strong> Verifica que DEBE =
HABER y la correctitud contable</li>
                        <li><strong>Auditoría pedagógica:</strong> Permite a estudiantes
entender el flujo completo de información</li>
                        <li><strong>Detección de inconsistencias:</strong> Identifica
errores en la cadena contable</li>
                    </ul>
                    <p style="margin-top:var(--space-3);"><em>Este módulo es fundamental
para el aprendizaje, ya que permite visualizar cómo cada transacción afecta los diferentes
componentes del sistema contable.</em></p>
                </div>
            `
        }
    },
    initInteractivity() {
        document.querySelectorAll('.flow-node').forEach(node => {
            node.addEventListener('click', () => {
                const moduleId = node.dataset.module;
                this.showModuleDetails(moduleId);
                document.querySelectorAll('.flow-node').forEach(n =>
n.classList.remove('selected'));
                node.classList.add('selected');
            });
        });
    },
    showModuleDetails(moduleId) {
        const info = this.moduleInfo[moduleId];
        if (!info) return;
        const details = document.getElementById('flow-details');
        details.innerHTML = `
                < div class= "module-detail" >
                <div class="detail-header">
                    <div class="detail-icon"><i class="fas ${info.icon}"></i></div>
                    <h4>${info.title}</h4>
                </div>
                <p class="detail-description">${info.description}</p>
                <div class="detail-section">
                    <h5><i class="fas fa-sign-in-alt"></i> Entradas (Inputs)</h5>
                    <ul>${info.inputs.map(i => `<li>${i}</li>`).join('')}</ul>
                </div>
                <div class="detail-section">
                    <h5><i class="fas fa-sign-out-alt"></i> Salidas (Outputs)</h5>
                    <ul>${info.outputs.map(o => `<li>${o}</li>`).join('')}</ul>
                </div>
                <div class="detail-section accounting-section">
                    <h5><i class="fas fa-file-invoice-dollar"></i> Impacto Contable</h5>
                    ${typeof info.accounting === 'string' ? info.accounting : ''}
                </div>
            </div >
    `;
        details.classList.add('active');
    },
    async downloadPDF() {
        Toast.info('Generando PDF del diagrama...');
        try {
            const jsPDF = window.jspdf.jsPDF;
            const doc = new jsPDF('landscape', 'mm', 'letter');
            doc.setFontSize(20);
            doc.setTextColor(10, 110, 209);
            doc.text('Diagrama de Flujo del Sistema ERP', 140, 20, { align: 'center' });
            doc.setFontSize(12);
            doc.setTextColor(100);
            doc.text('Relaciones entre módulos y contabilizaciones', 140, 28, { align:
```

```
'center' });
            let yPos = 40;
            const modules = [
                { name: 'MM - Compras', desc: 'Órdenes de compra y facturas de proveedor -
> Genera asiento de compra' },
                { name: 'SD - Ventas', desc: 'Cotizaciones y facturas a cliente -> Genera
asiento de venta' },
                { name: 'WM - Inventario', desc: 'Control de stock y valorización ->
Ajustes de inventario' },
                { name: 'FI - Tesorería', desc: 'Pagos y cobros -> Movimientos de caja y
banco' },
                { name: 'HR - RRHH', desc: 'Nómina y liquidaciones -> Asiento de
remuneraciones' },
                { name: 'GL - Contabilidad', desc: 'MÓDULO CENTRAL - Recibe todos los
asientos automáticos' },
                { name: 'Reportes', desc: 'Balance General, Estado de Resultados, Flujo de
Caja' },
                { name: 'CO - Costos', desc: 'Análisis de costos por centro' }
            ];
            doc.setFontSize(14);
            doc.setTextColor(30);
            modules.forEach((mod, i) => {
                doc.setFont(undefined, 'bold');
                doc.text(mod.name, 20, yPos);
                doc.setFont(undefined, 'normal');
                doc.text(mod.desc, 60, yPos);
                yPos += 10;
            });
            yPos += 10;
            doc.setFontSize(16);
            doc.setTextColor(10, 110, 209);
            doc.text('Principio Fundamental: DEBE = HABER', 140, yPos, { align: 'center'
});
            yPos += 10;
            doc.setFontSize(14);
            doc.setTextColor(100);
            doc.text('ACTIVO = PASIVO + PATRIMONIO', 140, yPos, { align: 'center' });
            doc.setFontSize(10);
            doc.setTextColor(150);
            doc.text(`EduERP - Generado el ${new Date().toLocaleDateString()} `, 140, 200,
{ align: 'center' });
            doc.save('Diagrama_Flujo_ERP.pdf');
            Toast.success('PDF descargado');
        } catch (err) {
            console.error('Error generando PDF:', err);
            Toast.error('Error al generar PDF');
        }
    }
};
const flowStyles = document.createElement('style');
flowStyles.textContent = `
    .flow - diagram - container {
    display: grid;
    grid - template - columns: 1fr 300px;
    gap: var(--space - 4);
    height: auto;
    max - height: none;
    overflow - y: auto;
}
    .flow - legend {
    grid - column: 1 / -1;
    background: linear - gradient(135deg, var(--primary - 50), var(--info - 50));
    padding: var(--space - 3) var(--space - 4);
    border - radius: var(--radius - lg);
    margin - bottom: var(--space - 2);
}
    .flow - legend h4 {
    margin: 0 0 var(--space - 1);
    font - size: var(--font - size - sm);
    color: var(--primary - 700);
}
    .flow - legend p {
    margin: 0 0 var(--space - 2);
    font - size: var(--font - size - xs);
    color: var(--text - secondary);
}
    .legend - items {
    display: flex;
    gap: var(--space - 4);
    font - size: var(--font - size - xs);
}
```

```
    .legend - items span {
    display: flex;
    align - items: center;
    gap: var(--space - 2);
}
    .flow - diagram {
    display: flex;
    flex - direction: column;
    align - items: center;
    gap: var(--space - 2);
    overflow - y: auto;
    padding: var(--space - 2);
}
    .flow - level {
    display: flex;
    justify - content: center;
    gap: var(--space - 4);
    width: 100 %;
}
    .flow - node {
    background: var(--bg - secondary);
    border: 2px solid var(--border - light);
    border - radius: var(--radius - xl);
    padding: var(--space - 3);
    display: flex;
    align - items: center;
    gap: var(--space - 3);
    cursor: pointer;
    transition: all var(--transition - fast);
    position: relative;
    min - width: 180px;
}
    .flow - node:hover {
    border - color: var(--primary - 500);
    transform: translateY(-2px);
    box - shadow: var(--shadow - lg);
}
    .flow - node.selected {
    border - color: var(--primary - 500);
    background: var(--primary - 50);
    box - shadow: 0 0 0 4px rgba(10, 110, 209, 0.2);
}
    .node - icon {
    width: 40px;
    height: 40px;
    border - radius: var(--radius - lg);
    display: flex;
    align - items: center;
    justify - content: center;
    color: white;
    font - size: var(--font - size - lg);
    flex - shrink: 0;
}
    .node - icon.mm { background: linear - gradient(135deg, #df6e0c, #c45f0a); }
    .node - icon.sd { background: linear - gradient(135deg, #107e3e, #0d6630); }
    .node - icon.wm { background: linear - gradient(135deg, #00b894, #00a187); }
    .node - icon.fi { background: linear - gradient(135deg, #6c5ce7, #5f3dc4); }
    .node - icon.hr { background: linear - gradient(135deg, #fd79a8, #e84393); }
    .node - icon.gl { background: linear - gradient(135deg, #0a6ed1, #064280); }
    .node - icon.reports { background: linear - gradient(135deg, #00cec9, #00b5ad); }
    .node - icon.co { background: linear - gradient(135deg, #fdcb6e, #f39c12); }\r\n
.node - icon.trace { background: linear - gradient(135deg, #a29bfe, #6c5ce7); }
    .node - content h4 {
    margin: 0;
    font - size: var(--font - size - sm);
    font - weight: 600;
}
    .node - content p {
    margin: var(--space - 1) 0 0;
    font - size: var(--font - size - xs);
    color: var(--text - tertiary);
    line - height: 1.3;
}
    .node - badge {
    position: absolute;
    top: -8px;
    right: -8px;
    background: var(--neutral - 700);
    color: white;
    font - size: 10px;
    padding: 2px 8px;
```

```
    border - radius: var(--radius - full);
}
    .core - node {
    background: linear - gradient(135deg, var(--primary - 50), var(--primary - 100));
    border - color: var(--primary - 500);
    border - width: 3px;
    min - width: 350px;
}
    .core - node.node - icon {
    width: 50px;
    height: 50px;
    font - size: var(--font - size - xl);
}
    .core - badge {
    display: inline - block;
    background: var(--primary - 500);
    color: white;
    font - size: 10px;
    padding: 2px 8px;
    border - radius: var(--radius - full);
    margin - top: var(--space - 2);
}
    .output - node {
    border - style: dashed;
}
    .entry - point.node - icon {
    background: linear - gradient(135deg, var(--neutral - 600), var(--neutral - 800));
}
    .flow - arrows - row {
    display: flex;
    justify - content: center;
    padding: var(--space - 2) 0;
}
    .arrow - down {
    color: var(--primary - 500);
    font - size: var(--font - size - xl);
    animation: bounce 1s infinite;
}
@keyframes bounce {
    0 %, 100 % { transform: translateY(0); }
    50 % { transform: translateY(4px); }
}
    .arrow - connector {
    width: 100 %;
    max - width: 800px;
}
    .connector - svg {
    width: 100 %;
    height: 60px;
}
    .arrow - connector.all - to - one.connector - svg {
    height: 80px;
}
    .flow - details {
    background: var(--bg - tertiary);
    border - radius: var(--radius - xl);
    padding: var(--space - 4);
    overflow - y: auto;
}
    .flow - details h4 {
    margin: 0 0 var(--space - 2);
    color: var(--text - tertiary);
    font - size: var(--font - size - sm);
}
    .flow - details.active h4 {
    color: var(--primary - 600);
}
    .module - detail {
    animation: fadeIn 0.3s ease;
}
    .detail - header {
    display: flex;
    align - items: center;
    gap: var(--space - 3);
    margin - bottom: var(--space - 3);
}
    .detail - icon {
    width: 48px;
    height: 48px;
    background: var(--primary - 500);
    border - radius: var(--radius - lg);
```

```
    display: flex;
    align - items: center;
    justify - content: center;
    color: white;
    font - size: var(--font - size - xl);
}
    .detail - header h4 {
    margin: 0;
    font - size: var(--font - size - base);
    color: var(--text - primary);
}
    .detail - description {
    font - size: var(--font - size - sm);
    color: var(--text - secondary);
    margin - bottom: var(--space - 4);
}
    .detail - section {
    margin - bottom: var(--space - 4);
}
    .detail - section h5 {
    margin: 0 0 var(--space - 2);
    font - size: var(--font - size - xs);
    color: var(--text - tertiary);
    text - transform: uppercase;
    letter - spacing: 0.5px;
}
    .detail - section ul {
    margin: 0;
    padding - left: var(--space - 4);
    font - size: var(--font - size - sm);
}
    .detail - section li {
    margin - bottom: var(--space - 1);
}
    .accounting - section {
    background: var(--bg - secondary);
    padding: var(--space - 3);
    border - radius: var(--radius - lg);
    border - left: 4px solid var(--success - 500);
}
    .accounting - example {
    width: 100 %;
    font - size: var(--font - size - xs);
    border - collapse: collapse;
}
    .accounting - example th {
    text - align: left;
    padding: var(--space - 2);
    background: var(--neutral - 100);
    font - weight: 600;
}
    .accounting - example td {
    padding: var(--space - 1) var(--space - 2);
    border - bottom: 1px solid var(--border - light);
}
    .accounting - example.debe {
    color: var(--info - 600);
    font - weight: 600;
}
    .accounting - example.haber {
    color: var(--success - 600);
    font - weight: 600;
}
    .accounting - example.separator {
    height: 10px;
    border: none;
}
    .central - module - info, .reports - info {
    font - size: var(--font - size - sm);
}
    .equation {
    text - align: center;
    font - size: var(--font - size - lg);
    font - weight: 700;
    color: var(--primary - 600);
    padding: var(--space - 3);
    background: var(--primary - 50);
    border - radius: var(--radius - lg);
    margin: var(--space - 3) 0;
}
    .reports - info ul {
```

```
    padding - left: var(--space - 4);
}
@keyframes fadeIn {
        from { opacity: 0; transform: translateY(10px); }
        to { opacity: 1; transform: translateY(0); }
}
@media(max - width: 900px) {
        .flow - diagram - container {
        grid - template - columns: 1fr;
    }
        .flow - level.operations,
        .flow - level.support {
        flex - direction: column;
        align - items: center;
    }
        .connector - svg {
        display: none;
    }
        .flow - arrows - row.complex,
        .flow - arrows - row.final {
        display: none;
    }
}
`;
document.head.appendChild(flowStyles);
window.SystemFlowDiagram = SystemFlowDiagram;
```

Archivo: js\components\table.js

```
const DataTable = {
    instances: {},
    create(containerId, options = {}) {
        const {
            columns = [],
            data = [],
            pageSize = 10,
            searchable = true,
            sortable = true,
            selectable = false,
            actions = [],
            emptyMessage = 'No hay datos para mostrar',
            onRowClick = null,
            onSelect = null
        } = options;
        const instance = {
            id: containerId,
            columns,
            allData: data,
            filteredData: data,
            currentPage: 1,
            pageSize,
            sortColumn: null,
            sortOrder: 'asc',
            searchTerm: '',
            selectedRows: new Set(),
            options: { searchable, sortable, selectable, actions, emptyMessage,
onRowClick, onSelect }
        };
        this.instances[containerId] = instance;
        this.render(containerId);
        return {
            refresh: (newData) => this.setData(containerId, newData),
            getSelected: () => this.getSelected(containerId),
            clearSelection: () => this.clearSelection(containerId),
            destroy: () => delete this.instances[containerId]
        };
    },
    render(containerId) {
        const instance = this.instances[containerId];
        if (!instance) return;
        const container = document.getElementById(containerId);
        if (!container) return;
        const { options, columns } = instance;
        this.applyFilters(containerId);
        this.applySort(containerId);
```

```
        const totalPages = Math.ceil(instance.filteredData.length / instance.pageSize);
        const startIdx = (instance.currentPage - 1) * instance.pageSize;
        const pageData = instance.filteredData.slice(startIdx, startIdx +
instance.pageSize);
        let html = `
            <div class="data-table-wrapper">
                ${options.searchable ? `
                    <div class="table-toolbar">
                        <div class="search-box">
                            <i class="fas fa-search"></i>
                            <input type="text" class="form-control"
placeholder="Buscar..."
                                value="${instance.searchTerm}" data-table-
search="${containerId}">
                        </div>
                        <div class="table-info">
                            ${instance.filteredData.length}
registro${instance.filteredData.length !== 1 ? 's' : ''}
                        </div>
                    </div>
                ` : ''}
                <div class="table-container">
                    <table class="data-table">
                        <thead>
                            <tr>
                                ${options.selectable ? `
                                    <th class="col-checkbox">
                                        <input type="checkbox" data-table-select-
all="${containerId}">
                                    </th>
                                ` : ''}
                                ${columns.map(col => `
                                    <th class="${col.class || ''} ${options.sortable &&
col.sortable !== false ? 'sortable' : ''}"
                                        ${options.sortable && col.sortable !== false ?
`data-table-sort="${containerId}" data-column="${col.key}"` : ''}>
                                        ${col.label}
                                        ${options.sortable && col.sortable !== false ? `
                                            <span class="sort-icon">
                                                ${instance.sortColumn === col.key ?
                    (instance.sortOrder === 'asc' ? '<i class="fas fa-sort-up"></i>' : '<i
class="fas fa-sort-down"></i>') :
                    '<i class="fas fa-sort"></i>'
                }
                                            </span>
                                        ` : ''}
                                    </th>
                                ).join('')}
                                ${options.actions.length ? '<th class="col-
actions">Acciones</th>' : ''}
                            </tr>
                        </thead>
                        <tbody>
        `;
        if (pageData.length === 0) {
            html += `
                <tr>
                    <td colspan="${columns.length + (options.selectable ? 1 : 0) +
(options.actions.length ? 1 : 0)}">
                        <div class="table-empty">
                            <i class="fas fa-inbox"></i>
                            <p>${options.emptyMessage}</p>
                        </div>
                    </td>
                </tr>
            `;
        } else {
            pageData.forEach((row, idx) => {
                const isSelected = instance.selectedRows.has(row.id);
                html += `
                    <tr data-row-id="${row.id}" class="${isSelected ? 'selected' : ''}"
                        ${options.onRowClick ? `data-table-row="${containerId}"` : ''}>
                        ${options.selectable ? `
                            <td class="col-checkbox">
                                <input type="checkbox" data-table-select="${containerId}"
data-row-id="${row.id}"
                                    ${isSelected ? 'checked' : ''}>
                            </td>
                        ` : ''}
                        ${columns.map(col => {
                    let value = Helpers.getNestedValue(row, col.key);
```

```
                    if (col.render) {
                        value = col.render(value, row, idx);
                    } else if (col.format) {
                        switch (col.format) {
                            case 'currency': value = Formatters.currency(value); break;
                            case 'date': value = Formatters.date(value); break;
                            case 'datetime': value = Formatters.datetime(value); break;
                            case 'number': value = Formatters.number(value, col.decimals
|| 0); break;
                            case 'percentage': value = Formatters.percentage(value);
break;
                            case 'status':
                                const status = Formatters.status(value);
                                value = `<span class="badge
${status.class}">${status.label}</span>`;
                                break;
                        }
                    }
                    return `<td class="${col.class || ''}">${value ?? '-'}</td>`;
                }).join('')}
                        ${options.actions.length ? `
                            <td class="col-actions">
                                <div class="actions">
                                    ${options.actions.map(action => `
                                        <button class="btn btn-icon btn-ghost"
title="${action.label}"
                                            data-table-action="${containerId}" data-
action="${action.name}" data-row-id="${row.id}">
                                            <i class="${action.icon}"></i>
                                        </button>
                                    `).join('')}
                                </div>
                            </td>
                        ` : ''}
                    </tr>
                `;
            });
        }
        html += `
                        </tbody>
                    </table>
                </div>
        `;
        if (totalPages > 1) {
            html += `
                <div class="pagination">
                    <button class="page-btn" data-table-page="${containerId}" data-
page="1"
                        ${instance.currentPage === 1 ? 'disabled' : ''}>
                        <i class="fas fa-angle-double-left"></i>
                    </button>
                    <button class="page-btn" data-table-page="${containerId}" data-
page="${instance.currentPage - 1}"
                        ${instance.currentPage === 1 ? 'disabled' : ''}>
                        <i class="fas fa-angle-left"></i>
                    </button>
                    <span class="page-info">Página ${instance.currentPage} de
${totalPages}</span>
                    <button class="page-btn" data-table-page="${containerId}" data-
page="${instance.currentPage + 1}"
                        ${instance.currentPage === totalPages ? 'disabled' : ''}>
                        <i class="fas fa-angle-right"></i>
                    </button>
                    <button class="page-btn" data-table-page="${containerId}" data-
page="${totalPages}"
                        ${instance.currentPage === totalPages ? 'disabled' : ''}>
                        <i class="fas fa-angle-double-right"></i>
                    </button>
                </div>
            `;
        }
        html += '</div>';
        container.innerHTML = html;
        this.bindEvents(containerId);
    },
    bindEvents(containerId) {
        const instance = this.instances[containerId];
        if (!instance) return;
        const searchInput = document.querySelector(`[data-table-
search="${containerId}"]`);
        if (searchInput) {
```

```
            searchInput.oninput = Helpers.debounce((e) => {
                instance.searchTerm = e.target.value;
                instance.currentPage = 1;
                this.render(containerId);
            }, 300);
        }
        document.querySelectorAll(`[data-table-sort="${containerId}"]`).forEach(th => {
            th.onclick = () => {
                const column = th.dataset.column;
                if (instance.sortColumn === column) {
                    instance.sortOrder = instance.sortOrder === 'asc' ? 'desc' : 'asc';
                } else {
                    instance.sortColumn = column;
                    instance.sortOrder = 'asc';
                }
                this.render(containerId);
            };
        });
        document.querySelectorAll(`[data-table-page="${containerId}"]`).forEach(btn => {
            btn.onclick = () => {
                const page = parseInt(btn.dataset.page);
                if (page >= 1 && page <= Math.ceil(instance.filteredData.length /
instance.pageSize)) {
                    instance.currentPage = page;
                    this.render(containerId);
                }
            };
        });
        document.querySelectorAll(`[data-table-select="${containerId}"]`).forEach(checkbox
=> {
            checkbox.onclick = (e) => {
                e.stopPropagation();
                const rowId = checkbox.dataset.rowId;
                if (checkbox.checked) {
                    instance.selectedRows.add(rowId);
                } else {
                    instance.selectedRows.delete(rowId);
                }
                if (instance.options.onSelect) {
                    instance.options.onSelect(Array.from(instance.selectedRows));
                }
                this.render(containerId);
            };
        });
        const selectAll = document.querySelector(`[data-table-select-
all="${containerId}"]`);
        if (selectAll) {
            selectAll.onclick = () => {
                const startIdx = (instance.currentPage - 1) * instance.pageSize;
                const pageData = instance.filteredData.slice(startIdx, startIdx +
instance.pageSize);
                if (selectAll.checked) {
                    pageData.forEach(row => instance.selectedRows.add(row.id));
                } else {
                    pageData.forEach(row => instance.selectedRows.delete(row.id));
                }
                if (instance.options.onSelect) {
                    instance.options.onSelect(Array.from(instance.selectedRows));
                }
                this.render(containerId);
            };
        }
        document.querySelectorAll(`[data-table-action="${containerId}"]`).forEach(btn => {
            btn.onclick = (e) => {
                e.stopPropagation();
                const actionName = btn.dataset.action;
                const rowId = btn.dataset.rowId;
                const row = instance.allData.find(r => r.id === rowId);
                const action = instance.options.actions.find(a => a.name === actionName);
                if (action && action.handler) {
                    action.handler(row, rowId);
                }
            };
        });
        if (instance.options.onRowClick) {
            document.querySelectorAll(`[data-table-row="${containerId}"]`).forEach(tr => {
                tr.onclick = () => {
                    const rowId = tr.dataset.rowId;
                    const row = instance.allData.find(r => r.id === rowId);
                    instance.options.onRowClick(row, rowId);
                };
```

```
                tr.style.cursor = 'pointer';
            });
        }
    },
    applyFilters(containerId) {
        const instance = this.instances[containerId];
        if (!instance.searchTerm) {
            instance.filteredData = [...instance.allData];
            return;
        }
        const searchKeys = instance.columns.filter(c => c.searchable !== false).map(c =>
c.key);
        instance.filteredData = Helpers.filterBySearch(instance.allData,
instance.searchTerm, searchKeys);
    },
    applySort(containerId) {
        const instance = this.instances[containerId];
        if (!instance.sortColumn) return;
        instance.filteredData = Helpers.sortBy(instance.filteredData, instance.sortColumn,
instance.sortOrder);
    },
    setData(containerId, data) {
        const instance = this.instances[containerId];
        if (!instance) return;
        instance.allData = data;
        instance.currentPage = 1;
        instance.selectedRows.clear();
        this.render(containerId);
    },
    getSelected(containerId) {
        const instance = this.instances[containerId];
        if (!instance) return [];
        return Array.from(instance.selectedRows);
    },
    clearSelection(containerId) {
        const instance = this.instances[containerId];
        if (!instance) return;
        instance.selectedRows.clear();
        this.render(containerId);
    }
};
const tableStyles = document.createElement('style');
tableStyles.textContent = `
    .data-table-wrapper { background: var(--bg-secondary); border-radius: var(--radius-
xl); overflow: hidden; }
    .table-toolbar { padding: var(--space-4); border-bottom: 1px solid var(--border-
light); display: flex; align-items: center; justify-content: space-between; gap: var(--
space-4); flex-wrap: wrap; }
    .table-info { font-size: var(--font-size-sm); color: var(--text-secondary); }
    .data-table th.sortable { cursor: pointer; user-select: none; }
    .data-table th.sortable:hover { background: var(--neutral-100); }
    .sort-icon { margin-left: var(--space-2); color: var(--text-tertiary); }
    .col-checkbox { width: 40px; text-align: center; }
    .col-actions { width: 100px; }
    .data-table tbody tr.selected { background: var(--primary-50); }
    .data-table tbody tr:hover { background: var(--neutral-50); }
`;
document.head.appendChild(tableStyles);
window.DataTable = DataTable;
```

Archivo: js\components\toast.js

```
const Toast = {
    container: null,
    queue: [],
    maxVisible: 5,
    init() {
        this.container = document.getElementById('toast-container');
        if (!this.container) {
            this.container = document.createElement('div');
            this.container.id = 'toast-container';
            this.container.className = 'toast-container';
            document.body.appendChild(this.container);
        }
    },
```

```
    show(message, type = 'info', options = {}) {
        const {
            duration = 4000,
            title = null,
            dismissible = true,
            action = null,
            actionText = 'Deshacer'
        } = options;
        const icons = {
            info: 'fa-info-circle',
            success: 'fa-check-circle',
            warning: 'fa-exclamation-triangle',
            error: 'fa-times-circle'
        };
        const toast = document.createElement('div');
        toast.className = `toast toast-${type}`;
        toast.innerHTML = `
            <div class="toast-icon">
                <i class="fas ${icons[type]}"></i>
            </div>
            <div class="toast-content">
                ${title ? `<div class="toast-title">${title}</div>` : ''}
                <div class="toast-message">${message}</div>
                ${action ? `<button class="toast-action">${actionText}</button>` : ''}
            </div>
            ${dismissible ? `<button class="toast-close"><i class="fas fa-times"></i>
</button>` : ''}
            <div class="toast-progress"></div>
        `;
        this.container.appendChild(toast);
        requestAnimationFrame(() => {
            toast.classList.add('toast-show');
        });
        const progress = toast.querySelector('.toast-progress');
        progress.style.animationDuration = `${duration}ms`;
        const close = () => {
            toast.classList.remove('toast-show');
            toast.classList.add('toast-hide');
            setTimeout(() => toast.remove(), 300);
        };
        const timeout = setTimeout(close, duration);
        if (dismissible) {
            toast.querySelector('.toast-close').onclick = () => {
                clearTimeout(timeout);
                close();
            };
        }
        if (action) {
            toast.querySelector('.toast-action').onclick = () => {
                clearTimeout(timeout);
                action();
                close();
            };
        }
        toast.addEventListener('mouseenter', () => {
            progress.style.animationPlayState = 'paused';
        });
        toast.addEventListener('mouseleave', () => {
            progress.style.animationPlayState = 'running';
        });
        return toast;
    },
    success(message, options = {}) {
        return this.show(message, 'success', options);
    },
    error(message, options = {}) {
        return this.show(message, 'error', { duration: 6000, ...options });
    },
    warning(message, options = {}) {
        return this.show(message, 'warning', options);
    },
    info(message, options = {}) {
        return this.show(message, 'info', options);
    },
    loading(message) {
        const toast = document.createElement('div');
        toast.className = 'toast toast-loading';
        toast.innerHTML = `
            <div class="toast-icon">
                <div class="spinner"></div>
            </div>
```

```
            <div class="toast-content">
                <div class="toast-message">${message}</div>
            </div>
        `;
        this.container.appendChild(toast);
        requestAnimationFrame(() => toast.classList.add('toast-show'));
        return {
            update: (newMessage) => {
                toast.querySelector('.toast-message').textContent = newMessage;
            },
            success: (msg) => {
                toast.remove();
                this.success(msg);
            },
            error: (msg) => {
                toast.remove();
                this.error(msg);
            },
            close: () => {
                toast.classList.remove('toast-show');
                toast.classList.add('toast-hide');
                setTimeout(() => toast.remove(), 300);
            }
        };
    },
    clear() {
        this.container.innerHTML = '';
    }
};
const toastStyles = document.createElement('style');
toastStyles.textContent = `
    .toast-container { position: fixed; top: var(--space-6); right: var(--space-6); z-
index: var(--z-toast); display: flex; flex-direction: column; gap: var(--space-3); max-
width: 400px; }
    .toast { display: flex; align-items: flex-start; gap: var(--space-3); padding: var(--
space-4); background: var(--bg-secondary); border-radius: var(--radius-lg); box-shadow:
var(--shadow-lg); border-left: 4px solid; transform: translateX(120%); opacity: 0;
transition: all 0.3s ease; position: relative; overflow: hidden; }
    .toast.toast-show { transform: translateX(0); opacity: 1; }
    .toast.toast-hide { transform: translateX(120%); opacity: 0; }
    .toast-info { border-left-color: var(--info-500); }
    .toast-success { border-left-color: var(--success-500); }
    .toast-warning { border-left-color: var(--warning-500); }
    .toast-error { border-left-color: var(--error-500); }
    .toast-icon { font-size: var(--font-size-xl); flex-shrink: 0; }
    .toast-info .toast-icon { color: var(--info-500); }
    .toast-success .toast-icon { color: var(--success-500); }
    .toast-warning .toast-icon { color: var(--warning-500); }
    .toast-error .toast-icon { color: var(--error-500); }
    .toast-content { flex: 1; min-width: 0; }
    .toast-title { font-weight: 600; color: var(--text-primary); margin-bottom: var(--
space-1); }
    .toast-message { font-size: var(--font-size-sm); color: var(--text-secondary); }
    .toast-action { background: none; border: none; color: var(--primary-500); font-
weight: 500; cursor: pointer; padding: 0; margin-top: var(--space-2); font-size: var(--
font-size-sm); }
    .toast-action:hover { text-decoration: underline; }
    .toast-close { background: none; border: none; color: var(--text-tertiary); cursor:
pointer; padding: var(--space-1); font-size: var(--font-size-sm); flex-shrink: 0; }
    .toast-close:hover { color: var(--text-primary); }
    .toast-progress { position: absolute; bottom: 0; left: 0; height: 3px; background:
currentColor; opacity: 0.3; animation: toast-progress linear forwards; }
    @keyframes toast-progress { from { width: 100%; } to { width: 0%; } }
    .toast-loading .toast-icon { display: flex; }
    .spinner { width: 20px; height: 20px; border: 2px solid var(--neutral-200); border-
top-color: var(--primary-500); border-radius: 50%; animation: spin 0.8s linear infinite; }
    @keyframes spin { to { transform: rotate(360deg); } }
    @media (max-width: 480px) { .toast-container { left: var(--space-4); right: var(--
space-4); top: auto; bottom: var(--space-4); max-width: none; } }
`;
document.head.appendChild(toastStyles);
window.Toast = Toast;
```

Archivo: js\components\tutorial.js

```
const TutorialSystem = {
    isActive: false,
    currentStep: 0,
    steps: [],
    overlay: null,
    tooltip: null,
    tutorials: {
        quickStart: {
            name: 'Tutorial Rápido',
            description: 'Aprende los conceptos básicos de EDU-TRACE ERP en 5 minutos',
            steps: [
                {
                    title: '¡Bienvenido a EDU-TRACE ERP!',
                    content: `
                        <p>EDU-TRACE ERP es un <strong>simulador de sistema ERP</strong>
diseñado para que los estudiantes
                        de auditoría comprendan cómo funcionan los sistemas empresariales.
</p>
                        <p><strong>En este tutorial aprenderás:</strong></p>
                        <ul>
                            <li>Navegar por los módulos del sistema</li>
                            <li>Gestionar empresas virtuales</li>
                            <li>Registrar operaciones contables</li>
                            <li>Generar reportes financieros</li>
                            <li>Validar la integridad de los datos</li>
                        </ul>
                    `,
                    target: null,
                    position: 'center'
                },
                {
                    title: 'Menú de Navegación',
                    content: `
                        <p>El <strong>menú lateral</strong> contiene todos los módulos del
ERP:</p>
                        <ul>
                            <li><strong>Dashboard:</strong> Panel principal con KPIs</li>
                            <li><strong>GL:</strong> Contabilidad General</li>
                            <li><strong>MM:</strong> Compras y Proveedores</li>
                            <li><strong>SD:</strong> Ventas y Clientes</li>
                            <li><strong>WM:</strong> Inventario</li>
                            <li><strong>FI:</strong> Tesorería</li>
                            <li><strong>CO:</strong> Costos</li>
                            <li><strong>HR:</strong> Recursos Humanos</li>
                        </ul>
                        <p>Haz clic en cada módulo para ver sus opciones.</p>
                    `,
                    target: '#sidebar',
                    position: 'right'
                },
                {
                    title: 'Selector de Empresa',
                    content: `
                        <p>Aquí puedes <strong>cambiar la empresa activa</strong>.</p>
                        <p>Cada estudiante puede crear múltiples empresas virtuales para
practicar
                        diferentes escenarios contables.</p>
                        <p><strong>Tip:</strong> Puedes importar casos de estudio
predefinidos
                        desde Administración → Importar/Exportar.</p>
                    `,
                    target: '#company-selector',
                    position: 'right'
                },
                {
                    title: 'Módulo de Contabilidad',
                    content: `
                        <p>El módulo de <strong>Contabilidad (GL)</strong> es el corazón
del sistema:</p>
                        <ul>
                            <li><strong>Plan de Cuentas:</strong> Estructura
IFRS/PCGA</li>
                            <li><strong>Asientos Contables:</strong> Registro de
operaciones</li>
                            <li><strong>Libro Mayor:</strong> Movimientos por cuenta</li>
                            <li><strong>Libro Diario:</strong> Registro cronológico</li>
```

```
                        </ul>
                        <p>Todos los módulos generan asientos automáticos.</p>
                    `,
                    target: '[data-module="contabilidad"]',
                    position: 'right'
                },
                {
                    title: 'Reportes Financieros',
                    content: `
                        <p>En <strong>Reportes</strong> puedes generar:</p>
                        <ul>
                            <li><strong>Balance General:</strong> Situación
patrimonial</li>
                            <li><strong>Estado de Resultados:</strong> Pérdidas y
ganancias</li>
                            <li><strong>Flujo de Caja:</strong> Movimientos de
efectivo</li>
                        </ul>
                        <p>Exporta a <strong>PDF</strong>, <strong>Excel</strong> o
<strong>CSV</strong>.</p>
                    `,
                    target: '[data-module="reportes"]',
                    position: 'right'
                },
                {
                    title: 'Trazabilidad de Datos',
                    content: `
                        <p>La sección de <strong>Trazabilidad</strong> permite:</p>
                        <ul>
                            <li><strong>Auditar operaciones:</strong> Ver el origen de
cada movimiento</li>
                            <li><strong>Validar datos:</strong> Detectar
inconsistencias</li>
                            <li><strong>Seguir el flujo:</strong> Desde documento origen
hasta asiento</li>
                        </ul>
                        <p>Esencial para auditoría y control interno.</p>
                    `,
                    target: '[data-module="trazabilidad"]',
                    position: 'right'
                },
                {
                    title: 'Administración',
                    content: `
                        <p>En <strong>Administración</strong> encontrarás:</p>
                        <ul>
                            <li><strong>Empresas:</strong> Crear y gestionar empresas</li>
                            <li><strong>Importar/Exportar:</strong> Cargar casos de
estudio</li>
                            <li><strong>Datos Demo:</strong> Crear datos de ejemplo</li>
                            <li><strong>Configuración:</strong> Ajustes del sistema</li>
                        </ul>
                        <p>Aquí puedes cargar los <strong>casos de estudio</strong>
predefinidos.</p>
                    `,
                    target: '[data-module="admin"]',
                    position: 'right'
                },
                {
                    title: 'Diagrama Contable',
                    content: `
                        <p>El <strong>Diagrama Contable Interactivo</strong> es una
herramienta visual para:</p>
                        <ul>
                            <li>Ver el flujo de una transacción comercial</li>
                            <li>Entender la ecuación contable</li>
                            <li>Visualizar asientos de compra y venta</li>
                            <li>Comprender el impacto en estados financieros</li>
                        </ul>
                        <p>Accede desde el botón <strong>"Ver Diagrama Contable"</strong>
en el Dashboard.</p>
                    `,
                    target: null,
                    position: 'center'
                },
                {
                    title: '¡Listo para comenzar!',
                    content: `
                        <p>Ahora conoces lo básico de EDU-TRACE ERP. Te recomendamos:</p>
                        <ol>
                            <li>Crear una empresa o cargar un caso de estudio</li>
```

```
                            <li>Explorar el plan de cuentas</li>
                            <li>Registrar un asiento contable</li>
                            <li>Ver el Balance General</li>
                            <li>Usar el Diagrama Contable para visualizar</li>
                        </ol>
                        <p><strong>¿Necesitas ayuda?</strong> Haz clic en el botón <i
class="fas fa-question-circle"></i>
                        en cualquier momento.</p>
                        <p style="margin-top: var(--space-3);">
                            <strong>¡Buena suerte en tu aprendizaje!</strong> 🎓
                        </p>
                    `,
                    target: null,
                    position: 'center'
                }
            ]
        },
        accounting: {
            name: 'Contabilidad Básica',
            description: 'Aprende a registrar asientos contables',
            steps: [
                {
                    title: 'Introducción a Contabilidad',
                    content: `
                        <p>En este tutorial aprenderás los fundamentos de la contabilidad
en EDU-TRACE ERP.</p>
                        <p>Cubriremos:</p>
                        <ul>
                            <li>Estructura del plan de cuentas</li>
                            <li>Cómo crear asientos contables</li>
                            <li>La partida doble (Debe = Haber)</li>
                        </ul>
                    `,
                    target: null,
                    position: 'center'
                },
                {
                    title: 'Plan de Cuentas',
                    content: `
                        <p>El <strong>Plan de Cuentas</strong> es la estructura base de la
contabilidad.</p>
                        <p>Se organiza jerárquicamente:</p>
                        <ul>
                            <li><strong>1:</strong> Activos (lo que tenemos)</li>
                            <li><strong>2:</strong> Pasivos (lo que debemos)</li>
                            <li><strong>3:</strong> Patrimonio (capital de los dueños)
</li>
                            <li><strong>4:</strong> Ingresos (lo que ganamos)</li>
                            <li><strong>5:</strong> Costos y Gastos (lo que gastamos)</li>
                        </ul>
                    `,
                    target: null,
                    position: 'center'
                },
                {
                    title: 'Partida Doble',
                    content: `
                        <p>El principio fundamental de la contabilidad:</p>
                        <p style="text-align: center; font-size: 1.2em; font-weight: bold;
                            padding: var(--space-4); background: var(--primary-50);
border-radius: var(--radius-lg);">
                            DEBE = HABER
                        </p>
                        <p>Por cada operación:</p>
                        <ul>
                            <li><strong>Debe:</strong> Aumenta Activos o disminuye
Pasivos/Patrimonio</li>
                            <li><strong>Haber:</strong> Disminuye Activos o aumenta
Pasivos/Patrimonio</li>
                        </ul>
                    `,
                    target: null,
                    position: 'center'
                }
            ]
        }
    },
    start(tutorialId = 'quickStart') {
        const tutorial = this.tutorials[tutorialId];
        if (!tutorial) {
            Toast.error('Tutorial no encontrado');
```

```
            return;
        }
        this.cleanup();
        this.steps = tutorial.steps;
        this.currentStep = 0;
        this.isActive = true;
        this.createOverlay();
        this.showStep(0);
        localStorage.setItem(`tutorial_${tutorialId}_seen`, 'true');
    },
    cleanup() {
        document.querySelectorAll('.tutorial-highlight').forEach(el => {
            el.classList.remove('tutorial-highlight');
        });
        document.querySelectorAll('.tutorial-overlay').forEach(el => el.remove());
        document.querySelectorAll('.tutorial-tooltip').forEach(el => el.remove());
        this.overlay = null;
        this.tooltip = null;
    },
    createOverlay() {
        this.overlay = document.createElement('div');
        this.overlay.className = 'tutorial-overlay';
        this.overlay.innerHTML = '<div class="tutorial-backdrop"></div>';
        document.body.appendChild(this.overlay);
        this.tooltip = document.createElement('div');
        this.tooltip.className = 'tutorial-tooltip';
        document.body.appendChild(this.tooltip);
    },
    showStep(index) {
        if (!this.isActive) return;
        if (index < 0 || index >= this.steps.length) {
            this.end();
            return;
        }
        this.currentStep = index;
        const step = this.steps[index];
        this.highlightElement(step.target);
        this.tooltip.innerHTML = `
            <div class="tutorial-header">
                <span class="tutorial-step-count">Paso ${index + 1} de
${this.steps.length}</span>
                <button class="tutorial-close" id="tutorial-close-btn" type="button">
                    <i class="fas fa-times"></i>
                </button>
            </div>
            <h3 class="tutorial-title">${step.title}</h3>
            <div class="tutorial-content">${step.content}</div>
            <div class="tutorial-footer">
                <div class="tutorial-progress">
                    ${this.steps.map((_, i) => `
                        <span class="tutorial-dot ${i === index ? 'active' : ''} ${i <
index ? 'completed' : ''}"></span>
                    `).join('')}
                </div>
                <div class="tutorial-actions">
                    ${index > 0 ? `
                        <button class="btn btn-outline btn-sm" id="tutorial-prev-btn"
type="button">
                            <i class="fas fa-arrow-left"></i> Anterior
                        </button>
                    ` : ''}
                    ${index < this.steps.length - 1 ? `
                        <button class="btn btn-primary btn-sm" id="tutorial-next-btn"
type="button">
                            Siguiente <i class="fas fa-arrow-right"></i>
                        </button>
                    ` : `
                        <button class="btn btn-success btn-sm" id="tutorial-finish-btn"
type="button">
                            <i class="fas fa-check"></i> Finalizar
                        </button>
                    `}
                </div>
            </div>
        `;
        this.positionTooltip(step.target, step.position);
        this.bindStepEvents();
    },
    bindStepEvents() {
        const closeBtn = document.getElementById('tutorial-close-btn');
        const prevBtn = document.getElementById('tutorial-prev-btn');
```

```
        const nextBtn = document.getElementById('tutorial-next-btn');
        const finishBtn = document.getElementById('tutorial-finish-btn');
        if (closeBtn) {
            closeBtn.onclick = (e) => {
                e.preventDefault();
                e.stopPropagation();
                this.end();
            };
        }
        if (prevBtn) {
            prevBtn.onclick = (e) => {
                e.preventDefault();
                e.stopPropagation();
                this.prev();
            };
        }
        if (nextBtn) {
            nextBtn.onclick = (e) => {
                e.preventDefault();
                e.stopPropagation();
                this.next();
            };
        }
        if (finishBtn) {
            finishBtn.onclick = (e) => {
                e.preventDefault();
                e.stopPropagation();
                this.end();
            };
        }
    },
    highlightElement(selector) {
        document.querySelectorAll('.tutorial-highlight').forEach(el => {
            el.classList.remove('tutorial-highlight');
        });
        if (!selector) {
            const backdrop = this.overlay?.querySelector('.tutorial-backdrop');
            if (backdrop) backdrop.style.opacity = '0.8';
            return;
        }
        const element = document.querySelector(selector);
        if (element) {
            element.classList.add('tutorial-highlight');
            const backdrop = this.overlay?.querySelector('.tutorial-backdrop');
            if (backdrop) backdrop.style.opacity = '0.7';
            element.scrollIntoView({ behavior: 'smooth', block: 'center' });
        }
    },
    positionTooltip(selector, position) {
        if (!this.tooltip) return;
        const tooltip = this.tooltip;
        tooltip.className = 'tutorial-tooltip';
        tooltip.style.top = '';
        tooltip.style.left = '';
        tooltip.style.right = '';
        tooltip.style.bottom = '';
        tooltip.style.transform = '';
        if (!selector || position === 'center') {
            tooltip.classList.add('center');
            tooltip.style.top = '50%';
            tooltip.style.left = '50%';
            tooltip.style.transform = 'translate(-50%, -50%)';
            return;
        }
        const element = document.querySelector(selector);
        if (!element) {
            tooltip.classList.add('center');
            tooltip.style.top = '50%';
            tooltip.style.left = '50%';
            tooltip.style.transform = 'translate(-50%, -50%)';
            return;
        }
        const rect = element.getBoundingClientRect();
        const tooltipRect = tooltip.getBoundingClientRect();
        const viewportWidth = window.innerWidth;
        const viewportHeight = window.innerHeight;
        const padding = 20;
        const tooltipWidth = Math.min(480, viewportWidth * 0.9);
        const tooltipHeight = Math.min(tooltipRect.height || 400, viewportHeight * 0.85);
        tooltip.classList.add(position);
        let top, left;
```

```
        switch (position) {
            case 'right':
                top = rect.top + rect.height / 2;
                left = rect.right + padding;
                if (left + tooltipWidth > viewportWidth - padding) {
                    left = Math.max(padding, rect.left - tooltipWidth - padding);
                    tooltip.classList.remove('right');
                    tooltip.classList.add('left');
                }
                if (top - tooltipHeight / 2 < padding) {
                    top = padding + tooltipHeight / 2;
                } else if (top + tooltipHeight / 2 > viewportHeight - padding) {
                    top = viewportHeight - padding - tooltipHeight / 2;
                }
                tooltip.style.top = `${top}px`;
                tooltip.style.left = `${left}px`;
                tooltip.style.transform = 'translateY(-50%)';
                break;
            case 'left':
                top = rect.top + rect.height / 2;
                left = rect.left - padding;
                if (left - tooltipWidth < padding) {
                    left = rect.right + padding;
                    tooltip.classList.remove('left');
                    tooltip.classList.add('right');
                    tooltip.style.transform = 'translateY(-50%)';
                } else {
                    tooltip.style.transform = 'translate(-100%, -50%)';
                }
                if (top - tooltipHeight / 2 < padding) {
                    top = padding + tooltipHeight / 2;
                } else if (top + tooltipHeight / 2 > viewportHeight - padding) {
                    top = viewportHeight - padding - tooltipHeight / 2;
                }
                tooltip.style.top = `${top}px`;
                tooltip.style.left = `${left}px`;
                break;
            case 'bottom':
                top = rect.bottom + padding;
                left = rect.left + rect.width / 2;
                if (top + tooltipHeight > viewportHeight - padding) {
                    top = rect.top - tooltipHeight - padding;
                    tooltip.classList.remove('bottom');
                    tooltip.classList.add('top');
                }
                if (left - tooltipWidth / 2 < padding) {
                    left = padding + tooltipWidth / 2;
                } else if (left + tooltipWidth / 2 > viewportWidth - padding) {
                    left = viewportWidth - padding - tooltipWidth / 2;
                }
                tooltip.style.top = `${top}px`;
                tooltip.style.left = `${left}px`;
                tooltip.style.transform = 'translateX(-50%)';
                break;
            case 'top':
                top = rect.top - padding;
                left = rect.left + rect.width / 2;
                if (top - tooltipHeight < padding) {
                    top = rect.bottom + padding;
                    tooltip.classList.remove('top');
                    tooltip.classList.add('bottom');
                    tooltip.style.transform = 'translateX(-50%)';
                } else {
                    tooltip.style.transform = 'translate(-50%, -100%)';
                }
                if (left - tooltipWidth / 2 < padding) {
                    left = padding + tooltipWidth / 2;
                } else if (left + tooltipWidth / 2 > viewportWidth - padding) {
                    left = viewportWidth - padding - tooltipWidth / 2;
                }
                tooltip.style.top = `${top}px`;
                tooltip.style.left = `${left}px`;
                break;
        }
    },
    next() {
        if (this.isActive) {
            this.showStep(this.currentStep + 1);
        }
    },
    prev() {
```

```
        if (this.isActive) {
            this.showStep(this.currentStep - 1);
        }
    },
    end() {
        this.isActive = false;
        this.cleanup();
        Toast.success('¡Tutorial completado!');
    },
    showTutorialSelector() {
        const tutorials = Object.entries(this.tutorials).map(([id, tutorial]) => ({
            id,
            ...tutorial,
            seen: localStorage.getItem(`tutorial_${id}_seen`) === 'true'
        }));
        Modal.open({
            title: '<i class="fas fa-graduation-cap"></i> Tutoriales Disponibles',
            size: 'medium',
            content: `
                <div class="tutorial-list">
                    ${tutorials.map(t => `
                        <div class="tutorial-card" data-tutorial="${t.id}">
                            <div class="tutorial-card-icon">
                                <i class="fas fa-graduation-cap"></i>
                            </div>
                            <div class="tutorial-card-content">
                                <h4>${t.name}</h4>
                                <p>${t.description}</p>
                                <span class="badge ${t.seen ? 'badge-success' : 'badge-
primary'}">
                                    ${t.seen ? 'Completado' : `${t.steps.length} pasos`}
                                </span>
                            </div>
                            <button class="btn btn-primary btn-sm">
                                ${t.seen ? 'Repetir' : 'Iniciar'}
                            </button>
                        </div>
                    `).join('')}
                </div>
            `,
            footer: `
                <button class="btn btn-outline" onclick="Modal.close()">Cerrar</button>
            `
        });
        setTimeout(() => {
            document.querySelectorAll('.tutorial-card').forEach(card => {
                card.onclick = () => {
                    Modal.close();
                    setTimeout(() => {
                        this.start(card.dataset.tutorial);
                    }, 350);
                };
            });
        }, 100);
    }
};
const tutorialStyles = document.createElement('style');
tutorialStyles.textContent = `
    .tutorial-overlay {
        position: fixed;
        inset: 0;
        z-index: 9998;
        pointer-events: none;
    }
    .tutorial-backdrop {
        position: absolute;
        inset: 0;
        background: rgba(0, 0, 0, 0.7);
        transition: opacity var(--transition-normal);
    }
    .tutorial-highlight {
        position: relative;
        z-index: 9999 !important;
        box-shadow: 0 0 0 4px var(--primary-500), 0 0 20px rgba(10, 110, 209, 0.5)
!important;
        border-radius: var(--radius-lg);
    }
    .tutorial-tooltip {
        position: fixed;
        z-index: 10000;
        background: var(--bg-secondary);
```

```
        border-radius: var(--radius-xl);
        box-shadow: var(--shadow-xl), 0 0 40px rgba(0, 0, 0, 0.3);
        max-width: 480px;
        min-width: 320px;
        width: 90%;
        max-height: 85vh;
        overflow-y: auto;
        pointer-events: auto;
        animation: tooltipIn 0.3s ease;
    }
    @keyframes tooltipIn {
        from { opacity: 0; transform: translateY(-50%) scale(0.95); }
        to { opacity: 1; transform: translateY(-50%) scale(1); }
    }
    .tutorial-tooltip.center {
        animation: tooltipCenter 0.3s ease forwards;
    }
    @keyframes tooltipCenter {
        from { opacity: 0; transform: translate(-50%, -50%) scale(0.9); }
        to { opacity: 1; transform: translate(-50%, -50%) scale(1); }
    }
    .tutorial-tooltip::before {
        content: '';
        position: absolute;
        width: 16px;
        height: 16px;
        background: var(--bg-secondary);
        transform: rotate(45deg);
    }
    .tutorial-tooltip.right::before { left: -8px; top: 50%; margin-top: -8px; }
    .tutorial-tooltip.left::before { right: -8px; top: 50%; margin-top: -8px; }
    .tutorial-tooltip.bottom::before { top: -8px; left: 50%; margin-left: -8px; }
    .tutorial-tooltip.top::before { bottom: -8px; left: 50%; margin-left: -8px; }
    .tutorial-tooltip.center::before { display: none; }
    .tutorial-header {
        display: flex;
        justify-content: space-between;
        align-items: center;
        padding: var(--space-3) var(--space-4);
        border-bottom: 1px solid var(--border-light);
    }
    .tutorial-step-count {
        font-size: var(--font-size-xs);
        color: var(--text-tertiary);
        font-weight: 500;
    }
    .tutorial-close {
        background: none;
        border: none;
        color: var(--text-tertiary);
        cursor: pointer;
        padding: var(--space-2);
        font-size: var(--font-size-lg);
        transition: color var(--transition-fast);
        border-radius: var(--radius-md);
    }
    .tutorial-close:hover {
        color: var(--error-500);
        background: var(--error-50);
    }
    .tutorial-title {
        margin: 0;
        padding: var(--space-4) var(--space-4) 0;
        font-size: var(--font-size-lg);
        font-weight: 600;
        color: var(--primary-600);
    }
    .tutorial-content {
        padding: var(--space-3) var(--space-4);
        font-size: var(--font-size-sm);
        color: var(--text-secondary);
        line-height: 1.6;
    }
    .tutorial-content p { margin: 0 0 var(--space-3); }
    .tutorial-content p:last-child { margin-bottom: 0; }
    .tutorial-content ul, .tutorial-content ol {
        margin: var(--space-2) 0;
        padding-left: var(--space-5);
    }
    .tutorial-content li { margin-bottom: var(--space-1); }
    .tutorial-content strong { color: var(--text-primary); }
```

```
    .tutorial-footer {
        padding: var(--space-4);
        border-top: 1px solid var(--border-light);
        display: flex;
        justify-content: space-between;
        align-items: center;
        gap: var(--space-4);
    }
    .tutorial-progress {
        display: flex;
        gap: var(--space-1);
    }
    .tutorial-dot {
        width: 8px;
        height: 8px;
        border-radius: 50%;
        background: var(--neutral-300);
        transition: all var(--transition-fast);
    }
    .tutorial-dot.active {
        background: var(--primary-500);
        transform: scale(1.25);
    }
    .tutorial-dot.completed { background: var(--success-500); }
    .tutorial-actions {
        display: flex;
        gap: var(--space-2);
    }
    .tutorial-list {
        display: flex;
        flex-direction: column;
        gap: var(--space-3);
    }
    .tutorial-card {
        display: flex;
        align-items: center;
        gap: var(--space-4);
        padding: var(--space-4);
        border: 1px solid var(--border-light);
        border-radius: var(--radius-lg);
        cursor: pointer;
        transition: all var(--transition-fast);
    }
    .tutorial-card:hover {
        border-color: var(--primary-500);
        background: var(--primary-50);
    }
    .tutorial-card-icon {
        width: 48px;
        height: 48px;
        background: linear-gradient(135deg, var(--primary-500), var(--primary-700));
        border-radius: var(--radius-lg);
        display: flex;
        align-items: center;
        justify-content: center;
        color: white;
        font-size: var(--font-size-xl);
        flex-shrink: 0;
    }
    .tutorial-card-content {
        flex: 1;
    }
    .tutorial-card-content h4 {
        margin: 0 0 var(--space-1);
        font-size: var(--font-size-base);
        font-weight: 600;
    }
    .tutorial-card-content p {
        margin: 0 0 var(--space-2);
        font-size: var(--font-size-xs);
        color: var(--text-secondary);
    }
`;
document.head.appendChild(tutorialStyles);
window.TutorialSystem = TutorialSystem;
```

Archivo: js\database\db.js

```
const DB = {
    name: 'EDU_TRACE_ERP_DB',
    version: 3,
    db: null,
    stores: {
        companies: { keyPath: 'id', indexes: ['name', 'rut', 'createdAt'] },
        accounts: { keyPath: 'id', indexes: ['companyId', 'code', 'type', 'parentId'] },
        journalEntries: { keyPath: 'id', indexes: ['companyId', 'date', 'number',
'status'] },
        journalLines: { keyPath: 'id', indexes: ['entryId', 'accountId'] },
        auxiliaryJournals: { keyPath: 'id', indexes: ['companyId', 'module', 'period',
'status'] },
        suppliers: { keyPath: 'id', indexes: ['companyId', 'rut', 'name'] },
        customers: { keyPath: 'id', indexes: ['companyId', 'rut', 'name'] },
        products: { keyPath: 'id', indexes: ['companyId', 'code', 'name', 'category'] },
        purchaseOrders: { keyPath: 'id', indexes: ['companyId', 'supplierId', 'date',
'status'] },
        purchaseOrderLines: { keyPath: 'id', indexes: ['orderId', 'productId'] },
        supplierInvoices: { keyPath: 'id', indexes: ['companyId', 'supplierId', 'date',
'status', 'dueDate'] },
        supplierInvoiceLines: { keyPath: 'id', indexes: ['invoiceId', 'productId'] },
        salesOrders: { keyPath: 'id', indexes: ['companyId', 'customerId', 'date',
'status'] },
        salesOrderLines: { keyPath: 'id', indexes: ['orderId', 'productId'] },
        customerInvoices: { keyPath: 'id', indexes: ['companyId', 'customerId', 'date',
'status', 'dueDate'] },
        customerInvoiceLines: { keyPath: 'id', indexes: ['invoiceId', 'productId'] },
        stockMovements: { keyPath: 'id', indexes: ['companyId', 'productId', 'date',
'type'] },
        bankAccounts: { keyPath: 'id', indexes: ['companyId', 'accountNumber'] },
        bankTransactions: { keyPath: 'id', indexes: ['bankAccountId', 'date', 'type'] },
        payments: { keyPath: 'id', indexes: ['companyId', 'date', 'type', 'entityId'] },
        customerPayments: { keyPath: 'id', indexes: ['companyId', 'customerId',
'invoiceId', 'date'] },
        supplierPayments: { keyPath: 'id', indexes: ['companyId', 'supplierId',
'invoiceId', 'date'] },
        deliveries: { keyPath: 'id', indexes: ['companyId', 'salesOrderId', 'date',
'status'] },
        costCenters: { keyPath: 'id', indexes: ['companyId', 'code', 'parentId'] },
        internalOrders: { keyPath: 'id', indexes: ['companyId', 'costCenterId', 'status']
},
        employees: { keyPath: 'id', indexes: ['companyId', 'rut', 'department'] },
        payroll: { keyPath: 'id', indexes: ['companyId', 'employeeId', 'period'] },
        payrollLines: { keyPath: 'id', indexes: ['payrollId', 'type'] },
        sequences: { keyPath: 'id', indexes: ['companyId', 'type'] },
        settings: { keyPath: 'id', indexes: ['companyId'] },
        auditLog: { keyPath: 'id', indexes: ['companyId', 'timestamp', 'action', 'entity']
}
    },
    async init() {
        return new Promise((resolve, reject) => {
            const request = indexedDB.open(this.name, this.version);
            request.onerror = () => {
                console.error('Error opening database:', request.error);
                reject(request.error);
            };
            request.onsuccess = () => {
                this.db = request.result;
                console.log('Database opened successfully');
                resolve(this.db);
            };
            request.onupgradeneeded = (event) => {
                const db = event.target.result;
                for (const [storeName, config] of Object.entries(this.stores)) {
                    if (!db.objectStoreNames.contains(storeName)) {
                        const store = db.createObjectStore(storeName, { keyPath:
config.keyPath });
                        if (config.indexes) {
                            config.indexes.forEach(indexName => {
                                store.createIndex(indexName, indexName, { unique: false
});
                            });
                        }
                        console.log(`Store created: ${storeName}`);
                    }
                }
```

```
            };
        });
    },
    getTransaction(storeNames, mode = 'readonly') {
        if (!this.db) throw new Error('Database not initialized');
        const names = Array.isArray(storeNames) ? storeNames : [storeNames];
        return this.db.transaction(names, mode);
    },
    getStore(storeName, mode = 'readonly') {
        const transaction = this.getTransaction(storeName, mode);
        return transaction.objectStore(storeName);
    },
    async add(storeName, data) {
        return new Promise((resolve, reject) => {
            const store = this.getStore(storeName, 'readwrite');
            const record = {
                ...data,
                id: data.id || Helpers.generateId(),
                createdAt: data.createdAt || new Date().toISOString(),
                updatedAt: new Date().toISOString()
            };
            const request = store.add(record);
            request.onsuccess = () => resolve(record);
            request.onerror = () => reject(request.error);
        });
    },
    async update(storeName, data) {
        return new Promise((resolve, reject) => {
            const store = this.getStore(storeName, 'readwrite');
            const record = {
                ...data,
                updatedAt: new Date().toISOString()
            };
            const request = store.put(record);
            request.onsuccess = () => resolve(record);
            request.onerror = () => reject(request.error);
        });
    },
    async delete(storeName, id) {
        return new Promise((resolve, reject) => {
            const store = this.getStore(storeName, 'readwrite');
            const request = store.delete(id);
            request.onsuccess = () => resolve(true);
            request.onerror = () => reject(request.error);
        });
    },
    async get(storeName, id) {
        return new Promise((resolve, reject) => {
            const store = this.getStore(storeName, 'readonly');
            const request = store.get(id);
            request.onsuccess = () => resolve(request.result);
            request.onerror = () => reject(request.error);
        });
    },
    async getAll(storeName) {
        return new Promise((resolve, reject) => {
            const store = this.getStore(storeName, 'readonly');
            const request = store.getAll();
            request.onsuccess = () => resolve(request.result || []);
            request.onerror = () => reject(request.error);
        });
    },
    async getByIndex(storeName, indexName, value) {
        return new Promise((resolve, reject) => {
            try {
                const store = this.getStore(storeName, 'readonly');
                if (!store.indexNames.contains(indexName)) {
                    const request = store.getAll();
                    request.onsuccess = () => {
                        const results = (request.result || []).filter(r => r[indexName]
=== value);
                        resolve(results);
                    };
                    request.onerror = () => reject(request.error);
                    return;
                }
                const index = store.index(indexName);
                const request = index.getAll(value);
                request.onsuccess = () => resolve(request.result || []);
                request.onerror = () => reject(request.error);
            } catch (err) {
```

```
                reject(err);
            }
        });
    },
    async count(storeName) {
        return new Promise((resolve, reject) => {
            const store = this.getStore(storeName, 'readonly');
            const request = store.count();
            request.onsuccess = () => resolve(request.result);
            request.onerror = () => reject(request.error);
        });
    },
    async clear(storeName) {
        return new Promise((resolve, reject) => {
            const store = this.getStore(storeName, 'readwrite');
            const request = store.clear();
            request.onsuccess = () => resolve(true);
            request.onerror = () => reject(request.error);
        });
    },
    async getNextSequence(companyId, type) {
        const sequences = await this.getByIndex('sequences', 'companyId', companyId);
        let seq = sequences.find(s => s.type === type);
        if (!seq) {
            seq = await this.add('sequences', {
                companyId,
                type,
                current: 1
            });
            return 1;
        }
        seq.current += 1;
        await this.update('sequences', seq);
        return seq.current;
    },
    async exportCompanyData(companyId) {
        const data = { exportDate: new Date().toISOString(), companyId };
        for (const storeName of Object.keys(this.stores)) {
            if (storeName === 'companies') {
                data[storeName] = [await this.get(storeName, companyId)];
            } else {
                const allRecords = await this.getAll(storeName);
                data[storeName] = allRecords.filter(r => r.companyId === companyId);
            }
        }
        return data;
    },
    normalizeAccountType(type) {
        const typeMap = {
            'activo': 'asset',
            'pasivo': 'liability',
            'patrimonio': 'equity',
            'ingreso': 'revenue',
            'gasto': 'expense'
        };
        return typeMap[type] || type;
    },
    async importCompanyData(data) {
        const results = { success: 0, errors: [] };
        if (data.companies && Array.isArray(data.companies)) {
            for (const company of data.companies) {
                try {
                    await this.update('companies', company);
                    results.success++;
                } catch (err) {
                    results.errors.push({ store: 'companies', record: company, error:
err.message });
                }
            }
        }
        if (data.accounts && Array.isArray(data.accounts)) {
            for (const account of data.accounts) {
                try {
                    const normalizedType = this.normalizeAccountType(account.type);
                    let isContra = account.isContra || false;
                    if (!isContra && account.nature) {
                        const isAssetExpense = ['asset',
'expense'].includes(normalizedType);
                        const isLiabilityEquityRevenue = ['liability', 'equity',
'revenue'].includes(normalizedType);
                        if (isAssetExpense && account.nature === 'credit') {
```

```
                            isContra = true;
                        } else if (isLiabilityEquityRevenue && account.nature === 'debit')
{
                            isContra = true;
                        }
                    }
                    const normalizedAccount = {
                        ...account,
                        type: normalizedType,
                        isContra: isContra
                    };
                    await this.update('accounts', normalizedAccount);
                    results.success++;
                } catch (err) {
                    results.errors.push({ store: 'accounts', record: account, error:
err.message });
                }
            }
        }
        if (data.journalEntries && Array.isArray(data.journalEntries)) {
            for (const entry of data.journalEntries) {
                try {
                    const lines = entry.lines || [];
                    const entryWithoutLines = { ...entry };
                    delete entryWithoutLines.lines;
                    if (!entryWithoutLines.totalDebit) {
                        entryWithoutLines.totalDebit = lines.reduce((sum, l) => sum +
(l.debit || 0), 0);
                    }
                    if (!entryWithoutLines.totalCredit) {
                        entryWithoutLines.totalCredit = lines.reduce((sum, l) => sum +
(l.credit || 0), 0);
                    }
                    if (!entryWithoutLines.number) {
                        entryWithoutLines.number = `ASI-${entry.id}`;
                    }
                    await this.update('journalEntries', entryWithoutLines);
                    results.success++;
                    for (let i = 0; i < lines.length; i++) {
                        const line = lines[i];
                        const lineRecord = {
                            id: line.id || `${entry.id}-line-${i}`,
                            entryId: entry.id,
                            companyId: entry.companyId,
                            accountId: line.accountId,
                            description: line.description || '',
                            debit: line.debit || 0,
                            credit: line.credit || 0
                        };
                        await this.update('journalLines', lineRecord);
                        results.success++;
                    }
                } catch (err) {
                    results.errors.push({ store: 'journalEntries', record: entry, error:
err.message });
                }
            }
        }
        if (data.auxiliaryJournals && Array.isArray(data.auxiliaryJournals)) {
            for (const journal of data.auxiliaryJournals) {
                try {
                    await this.update('auxiliaryJournals', journal);
                    results.success++;
                } catch (err) {
                    results.errors.push({ store: 'auxiliaryJournals', record: journal,
error: err.message });
                }
            }
        }
        for (const [storeName, records] of Object.entries(data)) {
            if (storeName === 'exportDate' || storeName === 'companyId' || storeName ===
'auxiliaryJournals') continue;
            if (storeName === 'companies' || storeName === 'journalEntries' || storeName
=== 'accounts') continue;
            if (!this.stores[storeName]) continue;
            if (!Array.isArray(records)) continue;
            for (const record of records) {
                try {
                    await this.update(storeName, record);
                    results.success++;
                } catch (err) {
```

```
                    results.errors.push({ store: storeName, record, error: err.message });
                }
            }
        }
        return results;
    },
    async logAudit(companyId, action, entity, entityId, details = {}) {
        await this.add('auditLog', {
            companyId,
            timestamp: new Date().toISOString(),
            action,
            entity,
            entityId,
            details,
            user: window.currentUser || 'system'
        });
    }
};
window.DB = DB;
```

Archivo: js\database\migrations.js

```
const Migrations = {
    defaultChartOfAccounts: [
        { code: '1', name: 'ACTIVOS', type: 'asset', level: 1, isGroup: true },
        { code: '1.1', name: 'Activos Corrientes', type: 'asset', level: 2, isGroup: true,
parentCode: '1' },
        { code: '1.1.01', name: 'Efectivo y Equivalentes', type: 'asset', level: 3,
isGroup: true, parentCode: '1.1' },
        { code: '1.1.01.001', name: 'Caja', type: 'asset', level: 4, parentCode: '1.1.01'
},
        { code: '1.1.01.002', name: 'Banco Estado Cta Cte', type: 'asset', level: 4,
parentCode: '1.1.01' },
        { code: '1.1.01.003', name: 'Banco Chile Cta Cte', type: 'asset', level: 4,
parentCode: '1.1.01' },
        { code: '1.1.01.004', name: 'Fondo Fijo', type: 'asset', level: 4, parentCode:
'1.1.01' },
        { code: '1.1.02', name: 'Cuentas por Cobrar', type: 'asset', level: 3, isGroup:
true, parentCode: '1.1' },
        { code: '1.1.02.001', name: 'Clientes Nacionales', type: 'asset', level: 4,
parentCode: '1.1.02' },
        { code: '1.1.02.002', name: 'Documentos por Cobrar', type: 'asset', level: 4,
parentCode: '1.1.02' },
        { code: '1.1.02.003', name: 'Deudores Varios', type: 'asset', level: 4,
parentCode: '1.1.02' },
        { code: '1.1.02.004', name: 'Estimación Deudores Incobrables', type: 'asset',
level: 4, parentCode: '1.1.02', isContra: true },
        { code: '1.1.03', name: 'Inventarios', type: 'asset', level: 3, isGroup: true,
parentCode: '1.1' },
        { code: '1.1.03.001', name: 'Mercaderías', type: 'asset', level: 4, parentCode:
'1.1.03' },
        { code: '1.1.03.002', name: 'Materias Primas', type: 'asset', level: 4,
parentCode: '1.1.03' },
        { code: '1.1.03.003', name: 'Productos en Proceso', type: 'asset', level: 4,
parentCode: '1.1.03' },
        { code: '1.1.03.004', name: 'Productos Terminados', type: 'asset', level: 4,
parentCode: '1.1.03' },
        { code: '1.1.04', name: 'Impuestos por Recuperar', type: 'asset', level: 3,
isGroup: true, parentCode: '1.1' },
        { code: '1.1.04.001', name: 'IVA Crédito Fiscal', type: 'asset', level: 4,
parentCode: '1.1.04' },
        { code: '1.1.04.002', name: 'PPM por Recuperar', type: 'asset', level: 4,
parentCode: '1.1.04' },
        { code: '1.1.05', name: 'Gastos Anticipados', type: 'asset', level: 3, isGroup:
true, parentCode: '1.1' },
        { code: '1.1.05.001', name: 'Seguros Pagados por Anticipado', type: 'asset',
level: 4, parentCode: '1.1.05' },
        { code: '1.1.05.002', name: 'Arriendos Pagados por Anticipado', type: 'asset',
level: 4, parentCode: '1.1.05' },
        { code: '1.2', name: 'Activos No Corrientes', type: 'asset', level: 2, isGroup:
true, parentCode: '1' },
        { code: '1.2.01', name: 'Propiedades, Planta y Equipo', type: 'asset', level: 3,
isGroup: true, parentCode: '1.2' },
        { code: '1.2.01.001', name: 'Terrenos', type: 'asset', level: 4, parentCode:
'1.2.01' },
```

```
        { code: '1.2.01.002', name: 'Edificios', type: 'asset', level: 4, parentCode:
'1.2.01' },
        { code: '1.2.01.003', name: 'Maquinaria y Equipos', type: 'asset', level: 4,
parentCode: '1.2.01' },
        { code: '1.2.01.004', name: 'Vehículos', type: 'asset', level: 4, parentCode:
'1.2.01' },
        { code: '1.2.01.005', name: 'Muebles y Enseres', type: 'asset', level: 4,
parentCode: '1.2.01' },
        { code: '1.2.01.006', name: 'Equipos Computacionales', type: 'asset', level: 4,
parentCode: '1.2.01' },
        { code: '1.2.01.099', name: 'Depreciación Acumulada', type: 'asset', level: 4,
parentCode: '1.2.01', isContra: true },
        { code: '1.2.02', name: 'Activos Intangibles', type: 'asset', level: 3, isGroup:
true, parentCode: '1.2' },
        { code: '1.2.02.001', name: 'Software', type: 'asset', level: 4, parentCode:
'1.2.02' },
        { code: '1.2.02.002', name: 'Licencias', type: 'asset', level: 4, parentCode:
'1.2.02' },
        { code: '1.2.02.099', name: 'Amortización Acumulada', type: 'asset', level: 4,
parentCode: '1.2.02', isContra: true },
        { code: '2', name: 'PASIVOS', type: 'liability', level: 1, isGroup: true },
        { code: '2.1', name: 'Pasivos Corrientes', type: 'liability', level: 2, isGroup:
true, parentCode: '2' },
        { code: '2.1.01', name: 'Cuentas por Pagar', type: 'liability', level: 3, isGroup:
true, parentCode: '2.1' },
        { code: '2.1.01.001', name: 'Proveedores Nacionales', type: 'liability', level: 4,
parentCode: '2.1.01' },
        { code: '2.1.01.002', name: 'Documentos por Pagar', type: 'liability', level: 4,
parentCode: '2.1.01' },
        { code: '2.1.01.003', name: 'Acreedores Varios', type: 'liability', level: 4,
parentCode: '2.1.01' },
        { code: '2.1.02', name: 'Obligaciones Laborales', type: 'liability', level: 3,
isGroup: true, parentCode: '2.1' },
        { code: '2.1.02.001', name: 'Remuneraciones por Pagar', type: 'liability', level:
4, parentCode: '2.1.02' },
        { code: '2.1.02.002', name: 'Retenciones por Pagar', type: 'liability', level: 4,
parentCode: '2.1.02' },
        { code: '2.1.02.003', name: 'AFP por Pagar', type: 'liability', level: 4,
parentCode: '2.1.02' },
        { code: '2.1.02.004', name: 'Isapre/Fonasa por Pagar', type: 'liability', level:
4, parentCode: '2.1.02' },
        { code: '2.1.02.005', name: 'Provisión Vacaciones', type: 'liability', level: 4,
parentCode: '2.1.02' },
        { code: '2.1.02.006', name: 'Provisión Indemnización', type: 'liability', level:
4, parentCode: '2.1.02' },
        { code: '2.1.03', name: 'Impuestos por Pagar', type: 'liability', level: 3,
isGroup: true, parentCode: '2.1' },
        { code: '2.1.03.001', name: 'IVA Débito Fiscal', type: 'liability', level: 4,
parentCode: '2.1.03' },
        { code: '2.1.03.002', name: 'IVA por Pagar', type: 'liability', level: 4,
parentCode: '2.1.03' },
        { code: '2.1.03.003', name: 'PPM por Pagar', type: 'liability', level: 4,
parentCode: '2.1.03' },
        { code: '2.1.03.004', name: 'Impuesto Renta por Pagar', type: 'liability', level:
4, parentCode: '2.1.03' },
        { code: '2.1.04', name: 'Obligaciones Financieras CP', type: 'liability', level:
3, isGroup: true, parentCode: '2.1' },
        { code: '2.1.04.001', name: 'Préstamos Bancarios CP', type: 'liability', level: 4,
parentCode: '2.1.04' },
        { code: '2.2', name: 'Pasivos No Corrientes', type: 'liability', level: 2,
isGroup: true, parentCode: '2' },
        { code: '2.2.01', name: 'Obligaciones Financieras LP', type: 'liability', level:
3, isGroup: true, parentCode: '2.2' },
        { code: '2.2.01.001', name: 'Préstamos Bancarios LP', type: 'liability', level: 4,
parentCode: '2.2.01' },
        { code: '2.2.01.002', name: 'Obligaciones por Leasing', type: 'liability', level:
4, parentCode: '2.2.01' },
        { code: '3', name: 'PATRIMONIO', type: 'equity', level: 1, isGroup: true },
        { code: '3.1', name: 'Capital', type: 'equity', level: 2, isGroup: true,
parentCode: '3' },
        { code: '3.1.01', name: 'Capital Pagado', type: 'equity', level: 3, parentCode:
'3.1' },
        { code: '3.1.02', name: 'Capital por Pagar', type: 'equity', level: 3, parentCode:
'3.1' },
        { code: '3.2', name: 'Reservas', type: 'equity', level: 2, isGroup: true,
parentCode: '3' },
        { code: '3.2.01', name: 'Reserva Legal', type: 'equity', level: 3, parentCode:
'3.2' },
        { code: '3.2.02', name: 'Otras Reservas', type: 'equity', level: 3, parentCode:
'3.2' },
        { code: '3.3', name: 'Resultados', type: 'equity', level: 2, isGroup: true,
```

```
parentCode: '3' },
        { code: '3.3.01', name: 'Resultados Acumulados', type: 'equity', level: 3,
parentCode: '3.3' },
        { code: '3.3.02', name: 'Resultado del Ejercicio', type: 'equity', level: 3,
parentCode: '3.3' },
        { code: '4', name: 'INGRESOS', type: 'revenue', level: 1, isGroup: true },
        { code: '4.1', name: 'Ingresos Operacionales', type: 'revenue', level: 2, isGroup:
true, parentCode: '4' },
        { code: '4.1.01', name: 'Ventas', type: 'revenue', level: 3, isGroup: true,
parentCode: '4.1' },
        { code: '4.1.01.001', name: 'Ventas de Mercaderías', type: 'revenue', level: 4,
parentCode: '4.1.01' },
        { code: '4.1.01.002', name: 'Ventas de Servicios', type: 'revenue', level: 4,
parentCode: '4.1.01' },
        { code: '4.1.01.003', name: 'Descuentos sobre Ventas', type: 'revenue', level: 4,
parentCode: '4.1.01', isContra: true },
        { code: '4.1.01.004', name: 'Devoluciones sobre Ventas', type: 'revenue', level:
4, parentCode: '4.1.01', isContra: true },
        { code: '4.2', name: 'Otros Ingresos', type: 'revenue', level: 2, isGroup: true,
parentCode: '4' },
        { code: '4.2.01', name: 'Ingresos Financieros', type: 'revenue', level: 3,
parentCode: '4.2' },
        { code: '4.2.02', name: 'Otros Ingresos No Operacionales', type: 'revenue', level:
3, parentCode: '4.2' },
        { code: '5', name: 'COSTOS', type: 'expense', level: 1, isGroup: true },
        { code: '5.1', name: 'Costo de Ventas', type: 'expense', level: 2, isGroup: true,
parentCode: '5' },
        { code: '5.1.01', name: 'Costo de Mercaderías Vendidas', type: 'expense', level:
3, parentCode: '5.1' },
        { code: '5.1.02', name: 'Costo de Servicios', type: 'expense', level: 3,
parentCode: '5.1' },
        { code: '6', name: 'GASTOS', type: 'expense', level: 1, isGroup: true },
        { code: '6.1', name: 'Gastos de Administración', type: 'expense', level: 2,
isGroup: true, parentCode: '6' },
        { code: '6.1.01', name: 'Remuneraciones Administración', type: 'expense', level:
3, parentCode: '6.1' },
        { code: '6.1.02', name: 'Arriendos', type: 'expense', level: 3, parentCode: '6.1'
},
        { code: '6.1.03', name: 'Servicios Básicos', type: 'expense', level: 3,
parentCode: '6.1' },
        { code: '6.1.04', name: 'Depreciación', type: 'expense', level: 3, parentCode:
'6.1' },
        { code: '6.1.05', name: 'Gastos Generales', type: 'expense', level: 3, parentCode:
'6.1' },
        { code: '6.2', name: 'Gastos de Ventas', type: 'expense', level: 2, isGroup: true,
parentCode: '6' },
        { code: '6.2.01', name: 'Remuneraciones Ventas', type: 'expense', level: 3,
parentCode: '6.2' },
        { code: '6.2.02', name: 'Comisiones', type: 'expense', level: 3, parentCode: '6.2'
},
        { code: '6.2.03', name: 'Publicidad y Marketing', type: 'expense', level: 3,
parentCode: '6.2' },
        { code: '6.3', name: 'Gastos Financieros', type: 'expense', level: 2, isGroup:
true, parentCode: '6' },
        { code: '6.3.01', name: 'Intereses Bancarios', type: 'expense', level: 3,
parentCode: '6.3' },
        { code: '6.3.02', name: 'Comisiones Bancarias', type: 'expense', level: 3,
parentCode: '6.3' },
        { code: '6.4', name: 'Otros Gastos', type: 'expense', level: 2, isGroup: true,
parentCode: '6' },
        { code: '6.4.01', name: 'Pérdidas por Castigo de Activos', type: 'expense', level:
3, parentCode: '6.4' },
        { code: '6.4.02', name: 'Otros Gastos No Operacionales', type: 'expense', level:
3, parentCode: '6.4' }
    ],
    async createCompanyWithDefaults(companyData) {
        const company = await DB.add('companies', {
            ...companyData,
            fiscalYear: companyData.fiscalYear || new Date().getFullYear(),
            currency: 'CLP',
            status: 'active'
        });
        const accountIdMap = {};
        for (const account of this.defaultChartOfAccounts) {
            const parentId = account.parentCode ? accountIdMap[account.parentCode] : null;
            const newAccount = await DB.add('accounts', {
                companyId: company.id,
                code: account.code,
                name: account.name,
                type: account.type,
                level: account.level,
```

```
                parentId,
                isGroup: account.isGroup || false,
                isContra: account.isContra || false,
                balance: 0,
                active: true
            });
            accountIdMap[account.code] = newAccount.id;
        }
        await DB.add('settings', {
            companyId: company.id,
            fiscalYear: company.fiscalYear,
            inventoryMethod: 'promedio',
            taxRate: 19,
            incomeTaxRate: 27,
            decimalPlaces: 0,
            dateFormat: 'dd/mm/yyyy'
        });
        await DB.logAudit(company.id, 'CREATE', 'company', company.id, { name:
company.name });
        return company;
    },
    async createDemoData(companyId) {
        const suppliers = [
            { name: 'Distribuidora ABC Ltda.', rut: '76.123.456-7', contact: 'Juan Pérez',
email: 'contacto@abc.cl', phone: '+56 2 2345 6789' },
            { name: 'Importadora XYZ S.A.', rut: '96.789.012-3', contact: 'María
González', email: 'ventas@xyz.cl', phone: '+56 2 9876 5432' },
            { name: 'Servicios Tecnológicos SpA', rut: '77.456.789-0', contact: 'Pedro
López', email: 'info@techserv.cl', phone: '+56 2 1234 5678' }
        ];
        for (const sup of suppliers) {
            await DB.add('suppliers', { companyId, ...sup, status: 'active', balance: 0
});
        }
        const customers = [
            { name: 'Comercial Norte Ltda.', rut: '76.234.567-8', contact: 'Ana Torres',
email: 'compras@norte.cl', phone: '+56 2 3456 7890', creditLimit: 5000000 },
            { name: 'Retail Sur S.A.', rut: '96.345.678-9', contact: 'Carlos Muñoz',
email: 'pedidos@sur.cl', phone: '+56 2 4567 8901', creditLimit: 10000000 },
            { name: 'Servicios Centro SpA', rut: '77.567.890-1', contact: 'Laura Soto',
email: 'admin@centro.cl', phone: '+56 2 5678 9012', creditLimit: 3000000 }
        ];
        for (const cust of customers) {
            await DB.add('customers', { companyId, ...cust, status: 'active', balance: 0
});
        }
        const products = [
            { code: 'PROD001', name: 'Laptop HP ProBook', category: 'Tecnología', unit:
'UN', cost: 450000, price: 599000, stock: 15, minStock: 5 },
            { code: 'PROD002', name: 'Monitor Dell 24"', category: 'Tecnología', unit:
'UN', cost: 120000, price: 159000, stock: 25, minStock: 10 },
            { code: 'PROD003', name: 'Teclado Logitech', category: 'Accesorios', unit:
'UN', cost: 25000, price: 35000, stock: 50, minStock: 20 },
            { code: 'PROD004', name: 'Mouse Inalámbrico', category: 'Accesorios', unit:
'UN', cost: 12000, price: 18000, stock: 80, minStock: 30 },
            { code: 'SERV001', name: 'Servicio de Instalación', category: 'Servicios',
unit: 'HH', cost: 15000, price: 25000, stock: 0, minStock: 0, isService: true }
        ];
        for (const prod of products) {
            await DB.add('products', { companyId, ...prod, status: 'active' });
        }
        const costCenters = [
            { code: 'CC001', name: 'Administración', type: 'overhead' },
            { code: 'CC002', name: 'Ventas', type: 'operational' },
            { code: 'CC003', name: 'Bodega', type: 'operational' },
            { code: 'CC004', name: 'Gerencia', type: 'overhead' }
        ];
        for (const cc of costCenters) {
            await DB.add('costCenters', { companyId, ...cc, status: 'active', budget: 0,
actual: 0 });
        }
        const employees = [
            { rut: '12.345.678-9', name: 'Roberto Sánchez', position: 'Gerente General',
department: 'Gerencia', salary: 3500000, startDate: '2020-01-15' },
            { rut: '13.456.789-0', name: 'Carolina Vega', position: 'Contador',
department: 'Administración', salary: 1800000, startDate: '2021-03-01' },
            { rut: '14.567.890-1', name: 'Felipe Rojas', position: 'Vendedor', department:
'Ventas', salary: 900000, startDate: '2022-06-15' },
            { rut: '15.678.901-2', name: 'Valentina Díaz', position: 'Bodeguero',
department: 'Bodega', salary: 650000, startDate: '2023-02-01' }
        ];
```

```
        for (const emp of employees) {
            await DB.add('employees', { companyId, ...emp, status: 'active' });
        }
        const bankAccount = await DB.add('bankAccounts', {
            companyId,
            bank: 'Banco Estado',
            accountNumber: '12345678901',
            accountType: 'Cuenta Corriente',
            currency: 'CLP',
            balance: 5000000,
            status: 'active'
        });
        const accounts = await DB.getByIndex('accounts', 'companyId', companyId);
        const findAcc = (code) => accounts.find(a => a.code === code);
        const accCaja = findAcc('1.1.01.001');
        const accBanco = findAcc('1.1.01.002');
        const accMercaderia = findAcc('1.1.03.001');
        const accCapital = findAcc('3.1.01');
        const accProveedores = findAcc('2.1.01.001');
        const accIvaCredito = findAcc('1.1.04.001');
        if (accCaja && accBanco && accCapital) {
            const entry1 = await DB.add('journalEntries', {
                companyId,
                number: 'AST-2024-0001',
                date: Helpers.getCurrentDate(),
                description: 'Asiento de apertura - Aporte de capital inicial',
                status: 'posted',
                type: 'opening',
                totalDebit: 10000000,
                totalCredit: 10000000
            });
            await DB.add('journalLines', { entryId: entry1.id, accountId: accBanco.id,
debit: 10000000, credit: 0, description: 'Aporte socios' });
            await DB.add('journalLines', { entryId: entry1.id, accountId: accCapital.id,
debit: 0, credit: 10000000, description: 'Capital inicial' });
            accBanco.balance += 10000000;
            accCapital.balance += 10000000;
            await DB.update('accounts', accBanco);
            await DB.update('accounts', accCapital);
        }
        if (accMercaderia && accIvaCredito && accProveedores) {
            const neto = 2000000;
            const iva = Math.round(neto * 0.19);
            const total = neto + iva;
            const entry2 = await DB.add('journalEntries', {
                companyId,
                number: 'AST-2024-0002',
                date: Helpers.getCurrentDate(),
                description: 'Compra inicial de mercaderías según Factura 101',
                status: 'posted',
                type: 'manual',
                totalDebit: total,
                totalCredit: total
            });
            await DB.add('journalLines', { entryId: entry2.id, accountId:
accMercaderia.id, debit: neto, credit: 0 });
            await DB.add('journalLines', { entryId: entry2.id, accountId:
accIvaCredito.id, debit: iva, credit: 0 });
            await DB.add('journalLines', { entryId: entry2.id, accountId:
accProveedores.id, debit: 0, credit: total });
            accMercaderia.balance += neto;
            accIvaCredito.balance += iva;
            accProveedores.balance += total;
            await DB.update('accounts', accMercaderia);
            await DB.update('accounts', accIvaCredito);
            await DB.update('accounts', accProveedores);
        }
        return { success: true, message: 'Datos de demostración y asientos contables
creados exitosamente' };
    }
};
window.Migrations = Migrations;
```

Archivo: js\modules\admin.js

```
const AdminModule = {
    currentView: 'empresas',
    async render(view = 'empresas') {
        this.currentView = view;
        switch (view) {
            case 'empresas': return await this.renderCompanies();
            case 'import-export': return this.renderImportExport();
            case 'configuracion': return await this.renderSettings();
            default: return '<p>Vista no encontrada</p>';
        }
    },
    async renderCompanies() {
        const companies = await CompanyService.getAll();
        const currentCompany = CompanyService.getCurrent();
        return `
            <div class="page-header">
                <h1 class="page-title">Gestión de Empresas</h1>
                <p class="page-subtitle">Administre sus empresas virtuales</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-company">
                        <i class="fas fa-plus"></i> Nueva Empresa
                    </button>
                </div>
            </div>
            <div class="dashboard-grid">
                ${companies.map(company => `
                    <div class="card ${currentCompany?.id === company.id ? 'active-
company' : ''}" data-company-id="${company.id}">
                        <div class="card-body">
                            <div style="display: flex; align-items: center; gap: var(--
space-3); margin-bottom: var(--space-3);">
                                <div class="avatar" style="background: linear-
gradient(135deg, var(--primary-500), var(--primary-700)); color: white;">
                                    ${company.name.charAt(0).toUpperCase()}
                                </div>
                                <div>
                                    <h3 style="font-size: var(--font-size-base); font-
weight: 600; margin: 0;">
                                        ${company.name}
                                    </h3>
                                    <p style="font-size: var(--font-size-xs); color: var(-
-text-tertiary); margin: 0;">
                                        ${company.rut || 'Sin RUT'}
                                    </p>
                                </div>
                                ${currentCompany?.id === company.id ? '<span class="badge
badge-success">Activa</span>' : ''}
                            </div>
                            <div style="font-size: var(--font-size-xs); color: var(--text-
secondary); margin-bottom: var(--space-3);">
                                <div><i class="fas fa-calendar" style="width: 16px;"></i>
Año fiscal: ${company.fiscalYear || new Date().getFullYear()}</div>
                                <div><i class="fas fa-clock" style="width: 16px;"></i>
Creada: ${Formatters.date(company.createdAt)}</div>
                            </div>
                            <div class="btn-group" style="width: 100%;">
                                ${currentCompany?.id !== company.id ? `
                                    <button class="btn btn-primary btn-sm" style="flex:
1;" data-action="select" data-id="${company.id}">
                                        <i class="fas fa-check"></i> Seleccionar
                                    </button>
                                ` : ''}
                                <button class="btn btn-outline btn-sm" data-
action="export" data-id="${company.id}">
                                    <i class="fas fa-download"></i>
                                </button>
                                <button class="btn btn-outline btn-sm" data-action="edit"
data-id="${company.id}">
                                    <i class="fas fa-edit"></i>
                                </button>
                                <button class="btn btn-outline btn-sm" style="color: var(-
-error-500);" data-action="delete" data-id="${company.id}">
                                    <i class="fas fa-trash"></i>
                                </button>
```

```
                            </div>
                        </div>
                    </div>
                `).join('') || `
                    <div class="card span-4">
                        <div class="card-body">
                            <div class="empty-state">
                                <i class="fas fa-building"></i>
                                <h3>Sin empresas</h3>
                                <p>Cree su primera empresa virtual para comenzar</p>
                                <button class="btn btn-primary" id="btn-first-company">
                                    <i class="fas fa-plus"></i> Crear Empresa
                                </button>
                            </div>
                        </div>
                    </div>
                `}
            </div>
        `;
    },
    renderImportExport() {
        return `
            <div class="page-header">
                <h1 class="page-title">Importar / Exportar</h1>
                <p class="page-subtitle">Gestione los datos de su empresa</p>
            </div>
            <div class="dashboard-grid">
                <!-- Exportar -->
                <div class="card span-2">
                    <div class="card-header">
                        <div class="card-title"><i class="fas fa-file-export"></i>
Exportar Datos</div>
                    </div>
                    <div class="card-body">
                        <p style="margin-bottom: var(--space-4); color: var(--text-
secondary);">
                            Exporte todos los datos de la empresa actual para backup o
compartir.
                        </p>
                        <div class="btn-group" style="flex-wrap: wrap;">
                            <button class="btn btn-primary" id="btn-export-json">
                                <i class="fas fa-file-code"></i> JSON (Backup Completo)
                            </button>
                            <button class="btn btn-outline" id="btn-export-excel">
                                <i class="fas fa-file-excel"></i> Excel
                            </button>
                        </div>
                    </div>
                </div>
                <!-- Importar -->
                <div class="card span-4">
                    <div class="card-header">
                        <div class="card-title"><i class="fas fa-file-import"></i>
Importar Datos</div>
                    </div>
                    <div class="card-body">
                        <div style="display: grid; grid-template-columns: 1fr 1fr; gap:
var(--space-6);">
                            <!-- Zona de carga -->
                            <div>
                                <h4 style="margin-bottom: var(--space-3); color: var(--
text-primary);">
                                    <i class="fas fa-upload"></i> Cargar Archivo
                                </h4>
                                <div class="upload-zone" id="upload-zone" style="border:
2px dashed var(--border-medium); border-radius: var(--radius-lg); padding: var(--space-6);
text-align: center; cursor: pointer; transition: all var(--transition-fast);">
                                    <i class="fas fa-cloud-upload-alt" style="font-size:
2rem; color: var(--primary-500); margin-bottom: var(--space-3);"></i>
                                    <p style="margin: 0; font-weight: 500;">Arrastre un
archivo aquí o haga clic para seleccionar</p>
                                    <p style="margin: var(--space-2) 0 0; font-size: var(-
-font-size-xs); color: var(--text-tertiary);">
                                        Formatos soportados: JSON, Excel (.xlsx)
                                    </p>
                                    <input type="file" id="import-file"
accept=".json,.xlsx" style="display: none;">
                                </div>
                                <div style="margin-top: var(--space-3); display: flex;
gap: var(--space-2); flex-wrap: wrap;">
                                    <button class="btn btn-outline btn-sm" id="btn-
```

```
download-template">
                                        <i class="fas fa-download"></i> Descargar
Plantilla
                                    </button>
                                    <button class="btn btn-primary btn-sm" id="btn-
generate-ai-prompt">
                                        <i class="fas fa-robot"></i> Generar con IA
                                    </button>
                                </div>
                            </div>
                            <!-- Documentación de estructura -->
                            <div>
                                <h4 style="margin-bottom: var(--space-3); color: var(--
text-primary);">
                                    <i class="fas fa-book"></i> Estructura del Archivo
JSON
                                </h4>
                                <div style="background: var(--neutral-900); color:
#e5e7eb; padding: var(--space-4); border-radius: var(--radius-lg); font-family: monospace;
font-size: var(--font-size-xs); max-height: 300px; overflow-y: auto;">
<pre style="margin: 0; white-space: pre-wrap;">{
  "companies": [{
    "id": "uuid",
    "name": "Nombre Empresa",
    "rut": "76.123.456-7",
    "accountingSystem": "journalizer|centralizer"
  }],
  "accounts": [{
    "id": "uuid",
    "companyId": "uuid",
    "code": "1.1.01.001",
    "name": "Caja",
    "type": "asset|liability|equity|revenue|expense",
    "balance": 0
  }],
  "journalEntries": [{
    "id": "uuid",
    "companyId": "uuid",
    "number": 1,
    "date": "2024-01-15",
    "description": "Descripción",
    "lines": [...]
  }],
  "auxiliaryJournals": [{
    "id": "uuid",
    "module": "sales|purchases|treasury",
    "period": "2024-01",
    "status": "open|centralized",
    "transactions": [...]
  }],
  "suppliers": [...],
  "customers": [...],
  "products": [...]
}
}</pre>
                                </div>
                            </div>
                        </div>
                        <!-- Sección de referencia de campos -->
                        <div style="margin-top: var(--space-6);">
                            <h4 style="margin-bottom: var(--space-3); color: var(--text-
primary);">
                                <i class="fas fa-list-alt"></i> Referencia de Entidades
                            </h4>
                            <div style="display: grid; grid-template-columns: repeat(auto-
fit, minmax(280px, 1fr)); gap: var(--space-4);">
                                <!-- Empresas -->
                                <div style="background: var(--primary-50); padding: var(--
space-3); border-radius: var(--radius-md); border-left: 4px solid var(--primary-500);">
                                    <h5 style="margin: 0 0 var(--space-2); color: var(--
primary-700);"><i class="fas fa-building"></i> companies (Empresas)</h5>
                                    <ul style="margin: 0; padding-left: var(--space-4);
font-size: var(--font-size-xs); color: var(--text-secondary);">
                                        <li><code>name</code>: Nombre de la empresa</li>
                                        <li><code>rut</code>: RUT único</li>
                                        <li><code>accountingSystem</code>:
<b>journalizer</b> o <b>centralizer</b></li>
                                        <li><code>fiscalYear</code>: Año contable</li>
                                    </ul>
                                </div>
                                <!-- Cuentas -->
```

```
                                <div style="background: var(--info-50); padding: var(--
space-3); border-radius: var(--radius-md); border-left: 4px solid var(--info-500);">
                                    <h5 style="margin: 0 0 var(--space-2); color: var(--
info-700);"><i class="fas fa-book"></i> accounts (Cuentas)</h5>
                                    <ul style="margin: 0; padding-left: var(--space-4);
font-size: var(--font-size-xs); color: var(--text-secondary);">
                                        <li><code>code</code>: Código único (ej:
1.1.01.001)</li>
                                        <li><code>name</code>: Nombre de la cuenta</li>
                                        <li><code>type</code>: asset, liability, equity,
revenue, expense</li>
                                        <li><code>balance</code>: Saldo inicial</li>
                                        <li><code>isGroup</code>: true si es cuenta de
grupo</li>
                                    </ul>
                                </div>
                                <!-- Asientos -->
                                <div style="background: var(--success-50); padding: var(--
space-3); border-radius: var(--radius-md); border-left: 4px solid var(--success-500);">
                                    <h5 style="margin: 0 0 var(--space-2); color: var(--
success-700);"><i class="fas fa-edit"></i> journalEntries (Asientos)</h5>
                                    <ul style="margin: 0; padding-left: var(--space-4);
font-size: var(--font-size-xs); color: var(--text-secondary);">
                                        <li><code>number</code>: Correlativo</li>
                                        <li><code>date</code>: Fecha (YYYY-MM-DD)</li>
                                        <li><code>lines</code>: Array de líneas
contables</li>
                                        <li><code>status</code>: draft, posted,
cancelled</li>
                                    </ul>
                                </div>
                                <!-- Auxiliares (SISTEMA DUAL) -->
                                <div style="background: var(--purple); background-color:
#f5f3ff; padding: var(--space-3); border-radius: var(--radius-md); border-left: 4px solid
var(--purple);">
                                    <h5 style="margin: 0 0 var(--space-2); color:
#6d28d9;"><i class="fas fa-exchange-alt"></i> auxiliaryJournals (Auxiliares)</h5>
                                    <ul style="margin: 0; padding-left: var(--space-4);
font-size: var(--font-size-xs); color: var(--text-secondary);">
                                        <li><code>module</code>: sales, purchases,
treasury</li>
                                        <li><code>period</code>: Formato YYYY-MM</li>
                                        <li><code>status</code>: open, centralized</li>
                                        <li><code>transactions</code>: Detalle de
transacciones</li>
                                    </ul>
                                </div>
                                <!-- Proveedores -->
                                <div style="background: var(--warning-50); padding: var(--
space-3); border-radius: var(--radius-md); border-left: 4px solid var(--warning-500);">
                                    <h5 style="margin: 0 0 var(--space-2); color: var(--
warning-700);"><i class="fas fa-truck"></i> suppliers (Proveedores)</h5>
                                    <ul style="margin: 0; padding-left: var(--space-4);
font-size: var(--font-size-xs); color: var(--text-secondary);">
                                        <li><code>name</code>: Razón social</li>
                                        <li><code>rut</code>: RUT del proveedor</li>
                                        <li><code>contact</code>: Nombre de contacto</li>
                                        <li><code>email</code>: Email de contacto</li>
                                    </ul>
                                </div>
                                <!-- Clientes -->
                                <div style="background: #f0fdfa; padding: var(--space-3);
border-radius: var(--radius-md); border-left: 4px solid #14b8a6;">
                                    <h5 style="margin: 0 0 var(--space-2); color:
#0f766e;"><i class="fas fa-users"></i> customers (Clientes)</h5>
                                    <ul style="margin: 0; padding-left: var(--space-4);
font-size: var(--font-size-xs); color: var(--text-secondary);">
                                        <li><code>name</code>: Razón social</li>
                                        <li><code>rut</code>: RUT del cliente</li>
                                        <li><code>creditLimit</code>: Límite de
crédito</li>
                                        <li><code>email</code>, <code>phone</code>:
Contacto</li>
                                    </ul>
                                </div>
                                <!-- Productos -->
                                <div style="background: var(--error-50); padding: var(--
space-3); border-radius: var(--radius-md); border-left: 4px solid var(--error-500);">
                                    <h5 style="margin: 0 0 var(--space-2); color: var(--
error-700);"><i class="fas fa-box"></i> products (Productos)</h5>
                                    <ul style="margin: 0; padding-left: var(--space-4);
```

```
font-size: var(--font-size-xs); color: var(--text-secondary);">
                                        <li><code>code</code>: SKU único</li>
                                        <li><code>name</code>: Nombre del producto</li>
                                        <li><code>cost</code>: Costo unitario</li>
                                        <li><code>price</code>: Precio de venta</li>
                                        <li><code>stock</code>: Stock actual</li>
                                    </ul>
                                </div>
                                <!-- Empleados -->
                                <div style="background: var(--neutral-100); padding: var(-
-space-3); border-radius: var(--radius-md); border-left: 4px solid var(--neutral-500);">
                                    <h5 style="margin: 0 0 var(--space-2); color: var(--
neutral-700);"><i class="fas fa-id-card"></i> employees (Empleados)</h5>
                                    <ul style="margin: 0; padding-left: var(--space-4);
font-size: var(--font-size-xs); color: var(--text-secondary);">
                                        <li><code>rut</code>: RUT del empleado</li>
                                        <li><code>name</code>: Nombre completo</li>
                                        <li><code>position</code>: Cargo</li>
                                        <li><code>salary</code>: Sueldo bruto</li>
                                    </ul>
                                </div>
                            </div>
                        </div>
                        <!-- Notas importantes -->
                        <div class="alert alert-info" style="margin-top: var(--space-4);">
                            <i class="fas fa-lightbulb"></i>
                            <div class="alert-content">
                                <p style="margin: 0;"><strong>Consejos:</strong></p>
                                <ul style="margin: var(--space-2) 0 0; padding-left: var(-
-space-4); font-size: var(--font-size-sm);">
                                    <li>Todos los IDs deben ser UUIDs únicos (ej:
"a1b2c3d4-e5f6-...")</li>
                                    <li>El campo <code>companyId</code> debe coincidir con
el ID de la empresa</li>
                                    <li>Los asientos deben estar cuadrados: Σ Débitos = Σ
Créditos</li>
                                    <li>Puede ver ejemplos en los archivos de caso de
estudio incluidos</li>
                                </ul>
                            </div>
                        </div>
                    </div>
                </div>
                <!-- Datos de demostración -->
                <div class="card span-2">
                    <div class="card-header">
                        <div class="card-title"><i class="fas fa-flask"></i> Datos de
Demostración</div>
                    </div>
                    <div class="card-body">
                        <p style="margin-bottom: var(--space-4); color: var(--text-
secondary);">
                            Genere datos de ejemplo para probar el sistema. Requiere tener
una empresa seleccionada.
                        </p>
                        <div class="btn-group" style="flex-wrap: wrap; gap: var(--space-
2);">
                            <button class="btn btn-primary" id="btn-create-demo">
                                <i class="fas fa-magic"></i> Crear Datos Básicos
                            </button>
                            <button class="btn btn-success" id="btn-load-case-study">
                                <i class="fas fa-briefcase"></i> Caso Comercial
(Jornalizador)
                            </button>
                            <button class="btn btn-purple" id="btn-load-dual-case"
style="background: var(--purple, #7c3aed); color: white;">
                                <i class="fas fa-exchange-alt"></i> Caso Sistema Dual
(Centralizador)
                            </button>
                            <button class="btn btn-outline" id="btn-load-service-case">
                                <i class="fas fa-cogs"></i> Caso Servicios
                            </button>
                        </div>
                        <div style="margin-top: var(--space-4); padding: var(--space-3);
background: var(--info-50); border-radius: var(--radius-md);">
                            <p style="font-size: var(--font-size-sm); margin: 0;"><i
class="fas fa-info-circle" style="color: var(--info-500);"></i>
                            <strong>Datos Básicos:</strong> Entidades iniciales y 2
asientos contables de apertura.<br>
                            <strong>Caso Comercial:</strong> Empresa completa con Libro
Diario Único (Jornalizador).<br>
```

```
                            <strong>Caso Sistema Dual:</strong> Demostración de Libros
Auxiliares y Centralización.<br>
                            <strong>Caso Servicios:</strong> Empresa de servicios con
facturación y gastos.</p>
                        </div>
                    </div>
                </div>
                <!-- Limpiar datos -->
                <div class="card span-2">
                    <div class="card-header">
                        <div class="card-title"><i class="fas fa-eraser"></i> Limpiar
Datos</div>
                    </div>
                    <div class="card-body">
                        <div class="alert alert-warning" style="margin-bottom: var(--
space-4);">
                            <i class="fas fa-exclamation-triangle"></i>
                            <div class="alert-content">
                                <p><strong>Precaución:</strong> Esta acción eliminará
todos los datos de la empresa actual.</p>
                            </div>
                        </div>
                        <button class="btn btn-danger" id="btn-clear-data">
                            <i class="fas fa-trash-alt"></i> Limpiar Todos los Datos
                        </button>
                    </div>
                </div>
            </div>
        `;
    },
    async renderSettings() {
        const company = CompanyService.getCurrent();
        const settings = company ? await CompanyService.getSettings() : null;
        return `
            <div class="page-header">
                <h1 class="page-title">Configuración</h1>
                <p class="page-subtitle">Ajustes de la empresa y parámetros del
sistema</p>
            </div>
            ${!company ? `
                <div class="alert alert-warning">
                    <i class="fas fa-exclamation-triangle"></i>
                    <div class="alert-content">Seleccione una empresa para ver la
configuración.</div>
                </div>
            ` : `
                <div class="card">
                    <div class="card-header">
                        <div class="card-title"><i class="fas fa-building"></i> Datos de
la Empresa</div>
                    </div>
                    <div class="card-body">
                        <form id="company-form">
                            <div class="form-row">
                                <div class="form-group">
                                    <label class="form-label">Nombre</label>
                                    <input type="text" class="form-control" name="name"
value="${company.name}">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">RUT</label>
                                    <input type="text" class="form-control" name="rut"
value="${company.rut || ''}">
                                </div>
                            </div>
                            <div class="form-row">
                                <div class="form-group">
                                    <label class="form-label">Dirección</label>
                                    <input type="text" class="form-control" name="address"
value="${company.address || ''}">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Teléfono</label>
                                    <input type="text" class="form-control" name="phone"
value="${company.phone || ''}">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Email</label>
                                    <input type="email" class="form-control" name="email"
value="${company.email || ''}">
                                </div>
```

```
                            </div>
                            <button type="submit" class="btn btn-primary">
                                <i class="fas fa-save"></i> Guardar Cambios
                            </button>
                        </form>
                    </div>
                </div>
                <div class="card" style="margin-top: var(--space-4);">
                    <div class="card-header">
                        <div class="card-title"><i class="fas fa-cog"></i> Parámetros
Contables</div>
                    </div>
                    <div class="card-body">
                        <form id="settings-form">
                            <div class="form-row">
                                <div class="form-group">
                                    <label class="form-label">Año Fiscal</label>
                                    <input type="number" class="form-control"
name="fiscalYear"
                                        value="${settings?.fiscalYear || new
Date().getFullYear()}">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">IVA (%)</label>
                                    <input type="number" class="form-control"
name="taxRate"
                                        value="${settings?.taxRate || 19}" step="0.1">
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Impuesto Renta (%)</label>
                                    <input type="number" class="form-control"
name="incomeTaxRate"
                                        value="${settings?.incomeTaxRate || 27}"
step="0.1">
                                </div>
                            </div>
                            <div class="form-row">
                                <div class="form-group">
                                    <label class="form-label">Método de Inventario</label>
                                    <select class="form-control" name="inventoryMethod">
                                        <option value="promedio"
${settings?.inventoryMethod === 'promedio' ? 'selected' : ''}>Costo Promedio</option>
                                        <option value="fifo" ${settings?.inventoryMethod
=== 'fifo' ? 'selected' : ''}>FIFO (PEPS)</option>
                                        <option value="lifo" ${settings?.inventoryMethod
=== 'lifo' ? 'selected' : ''}>LIFO (UEPS)</option>
                                    </select>
                                </div>
                                <div class="form-group">
                                    <label class="form-label">
                                        Sistema de Contabilización
                                        <i class="fas fa-info-circle" style="color: var(--
info-500); cursor: help;"
                                           title="Jornalizador: Registro directo en Libro
Diario. Centralizador: Registro en libros auxiliares + centralización mensual."></i>
                                    </label>
                                    <select class="form-control" name="accountingSystem">
                                        <option value="journalizer"
${company.accountingSystem === 'journalizer' || !company.accountingSystem ? 'selected' :
''}>
                                            🔵 Jornalizador (Diario Único)
                                        </option>
                                        <option value="centralizer"
${company.accountingSystem === 'centralizer' ? 'selected' : ''}>
                                            🟢 Centralizador (Libros Auxiliares)
                                        </option>
                                    </select>
                                    <small class="form-text" style="color: var(--text-
tertiary);">
                                        ${!company.accountingSystem ||
company.accountingSystem === 'journalizer'
                ? 'Modo actual: Registro directo (ideal para principiantes)'
                : 'Modo actual: Libros auxiliares (empresas avanzadas)'
            }
                                    </small>
                                </div>
                                <div class="form-group">
                                    <label class="form-label">Decimales</label>
                                    <select class="form-control" name="decimalPlaces">
                                        <option value="0" ${settings?.decimalPlaces === 0
```

```
? 'selected' : ''}>Sin decimales</option>
                                    <option value="2" ${settings?.decimalPlaces === 2
? 'selected' : ''}>2 decimales</option>
                                </select>
                            </div>
                        </div>
                        <button type="submit" class="btn btn-primary">
                            <i class="fas fa-save"></i> Guardar Configuración
                        </button>
                    </form>
                </div>
            </div>
            <div class="card" style="margin-top: var(--space-4); border-color: var(--
error-300);">
                <div class="card-header" style="background: var(--error-50);">
                    <div class="card-title" style="color: var(--error-600);"><i
class="fas fa-exclamation-triangle"></i> Zona de Peligro</div>
                </div>
                <div class="card-body">
                    <div style="display: flex; justify-content: space-between; align-
items: center; margin-bottom: var(--space-4); padding-bottom: var(--space-4); border-
bottom: 1px solid var(--border-light);">
                        <div>
                            <h4 style="margin: 0; font-size: var(--font-size-
base);">Limpiar Datos</h4>
                            <p style="margin: var(--space-1) 0 0; font-size: var(--
font-size-sm); color: var(--text-tertiary);">
                                Elimina todos los datos de la empresa (cuentas,
asientos, etc.) pero mantiene la empresa.
                            </p>
                        </div>
                        <button class="btn btn-outline" style="color: var(--error-
500); border-color: var(--error-300);" id="btn-clear-company-data">
                            <i class="fas fa-eraser"></i> Limpiar Datos
                        </button>
                    </div>
                    <div style="display: flex; justify-content: space-between; align-
items: center;">
                        <div>
                            <h4 style="margin: 0; font-size: var(--font-size-
base);">Eliminar Empresa</h4>
                            <p style="margin: var(--space-1) 0 0; font-size: var(--
font-size-sm); color: var(--text-tertiary);">
                                Elimina permanentemente la empresa y todos sus datos.
Esta acción no se puede deshacer.
                            </p>
                        </div>
                        <button class="btn btn-danger" id="btn-delete-company">
                            <i class="fas fa-trash-alt"></i> Eliminar Empresa
                        </button>
                    </div>
                </div>
            </div>
        `}
    `;
},
async init(view) {
    switch (view) {
        case 'empresas': this.initCompanies(); break;
        case 'import-export': this.initImportExport(); break;
        case 'configuracion': this.initSettings(); break;
    }
},
initCompanies() {
    document.getElementById('btn-new-company')?.addEventListener('click', () => {
        DashboardModule.showCreateCompanyModal();
    });
    document.getElementById('btn-first-company')?.addEventListener('click', () => {
        DashboardModule.showCreateCompanyModal();
    });
    document.querySelectorAll('[data-action]').forEach(btn => {
        btn.addEventListener('click', async () => {
            const action = btn.dataset.action;
            const id = btn.dataset.id;
            switch (action) {
                case 'select':
                    await CompanyService.setCurrent(id);
                    Toast.success('Empresa seleccionada');
                    App.navigate('dashboard');
                    break;
                case 'edit':
```

```
                        Toast.info('Editar empresa');
                        break;
                    case 'export':
                        const loading = Toast.loading('Exportando...');
                        await CompanyService.setCurrent(id);
                        await CompanyService.exportData('json');
                        loading.success('Datos exportados');
                        break;
                    case 'delete':
                        const confirm = await Modal.confirm({
                            title: 'Eliminar Empresa',
                            message: '¿Está seguro de eliminar esta empresa? Se perderán
todos los datos asociados.',
                            confirmText: 'Eliminar',
                            confirmClass: 'btn-danger'
                        });
                        if (confirm) {
                            await CompanyService.delete(id);
                            Toast.success('Empresa eliminada');
                            App.navigate('admin', 'empresas');
                        }
                        break;
                }
            });
        });
    },
    initImportExport() {
        document.getElementById('btn-export-json')?.addEventListener('click', async () =>
{
            const company = CompanyService.getCurrent();
            if (!company) {
                Toast.warning('Seleccione una empresa primero');
                return;
            }
            const loading = Toast.loading('Exportando...');
            await CompanyService.exportData('json');
            loading.success('Backup exportado correctamente');
        });
        document.getElementById('btn-export-excel')?.addEventListener('click', async () =>
{
            const company = CompanyService.getCurrent();
            if (!company) {
                Toast.warning('Seleccione una empresa primero');
                return;
            }
            const loading = Toast.loading('Exportando a Excel...');
            try {
                await App.exportToExcel(company);
                loading.success('Excel exportado correctamente');
            } catch (err) {
                loading.error('Error: ' + err.message);
            }
        });
        document.getElementById('btn-download-template')?.addEventListener('click', () =>
{
            const template = {
                "_info": "Plantilla de importación EDU-TRACE ERP - Modifique los datos
según su necesidad",
                "_instructions": [
                    "1. Todos los IDs deben ser UUIDs únicos",
                    "2. El companyId debe coincidir en todos los registros",
                    "3. Los asientos deben estar cuadrados (débitos = créditos)",
                    "4. Las fechas deben estar en formato YYYY-MM-DD",
                    "5. Los tipos de cuenta válidos: asset, liability, equity, revenue,
expense"
                ],
                "companies": [{
                    "id": "empresa-001",
                    "name": "Mi Empresa SpA",
                    "rut": "76.000.000-0",
                    "address": "Av. Principal 123, Santiago",
                    "phone": "+56 2 1234 5678",
                    "email": "contacto@miempresa.cl",
                    "fiscalYear": 2024,
                    "accountingSystem": "journalizer"
                }],
                "accounts": [
                    { "id": "acc-001", "companyId": "empresa-001", "code": "1.1.01.001",
"name": "Caja", "type": "asset", "balance": 1000000, "isGroup": false },
                    { "id": "acc-002", "companyId": "empresa-001", "code": "1.1.01.002",
"name": "Banco Estado Cta Cte", "type": "asset", "balance": 5000000, "isGroup": false },
```

```
                    { "id": "acc-003", "companyId": "empresa-001", "code": "2.1.01.001",
"name": "Proveedores", "type": "liability", "balance": 2000000, "isGroup": false },
                    { "id": "acc-004", "companyId": "empresa-001", "code": "3.1.01",
"name": "Capital", "type": "equity", "balance": 4000000, "isGroup": false },
                    { "id": "acc-005", "companyId": "empresa-001", "code": "4.1.01.001",
"name": "Ventas", "type": "revenue", "balance": 0, "isGroup": false },
                    { "id": "acc-006", "companyId": "empresa-001", "code": "5.1.01",
"name": "Costo de Ventas", "type": "expense", "balance": 0, "isGroup": false }
                ],
                "suppliers": [
                    { "id": "sup-001", "companyId": "empresa-001", "name": "Proveedor
Ejemplo Ltda.", "rut": "76.111.111-1", "contact": "Juan Pérez", "email":
"contacto@proveedor.cl", "phone": "+56 2 1111 1111", "status": "active" }
                ],
                "customers": [
                    { "id": "cust-001", "companyId": "empresa-001", "name": "Cliente
Ejemplo S.A.", "rut": "96.222.222-2", "contact": "María González", "email":
"compras@cliente.cl", "phone": "+56 2 2222 2222", "creditLimit": 5000000, "status":
"active" }
                ],
                "products": [
                    { "id": "prod-001", "companyId": "empresa-001", "code": "PROD001",
"name": "Producto Ejemplo", "category": "General", "unit": "UN", "cost": 10000, "price":
15000, "stock": 100, "minStock": 10, "status": "active" }
                ],
                "employees": [
                    { "id": "emp-001", "companyId": "empresa-001", "rut": "12.345.678-9",
"name": "Empleado Ejemplo", "position": "Contador", "department": "Administración",
"salary": 1500000, "startDate": "2024-01-01", "status": "active" }
                ],
                "journalEntries": [
                    {
                        "id": "entry-001",
                        "companyId": "empresa-001",
                        "number": 1,
                        "date": "2024-01-15",
                        "description": "Asiento de ejemplo - Venta al contado",
                        "reference": "FAC-001",
                        "status": "posted",
                        "lines": [
                            { "id": "line-001", "entryId": "entry-001", "accountId": "acc-
001", "accountCode": "1.1.01.001", "accountName": "Caja", "description": "Ingreso por
venta", "debit": 119000, "credit": 0 },
                            { "id": "line-002", "entryId": "entry-001", "accountId": "acc-
005", "accountCode": "4.1.01.001", "accountName": "Ventas", "description": "Venta de
productos", "debit": 0, "credit": 100000 },
                            { "id": "line-003", "entryId": "entry-001", "accountId": "acc-
iva", "accountCode": "2.1.03.001", "accountName": "IVA Débito", "description": "IVA 19%",
"debit": 0, "credit": 19000 }
                        ]
                    }
                ]
            };
            const json = JSON.stringify(template, null, 2);
            Helpers.downloadFile(json, 'plantilla_importacion_edu_trace_erp.json',
'application/json');
            Toast.success('Plantilla descargada');
        });
        document.getElementById('btn-generate-ai-prompt')?.addEventListener('click', () =>
{
            const prompt = `Eres un experto en contabilidad y sistemas ERP. Necesito que
generes un caso de estudio contable completo en formato JSON para importar a un sistema
ERP educativo que soporta SISTEMA DUAL (Jornalizador y Centralizador).
## REQUISITOS CRÍTICOS:
1. **ECUACIÓN CONTABLE**: Los datos DEBEN cumplir: Activos = Pasivos + Patrimonio
2. **ASIENTOS CUADRADOS**: Cada asiento o transacción debe cumplir: Σ Débitos = Σ Créditos
3. **IDs ÚNICOS**: Usa UUIDs o identificadores únicos consistentes
4. **FORMATO DE FECHAS**: YYYY-MM-DD (ej: 2024-01-15)
5. **SISTEMA CONTABLE**: Define "accountingSystem" como "journalizer" (clásico) o
"centralizer" (avanzado).
## INSTRUCCIONES DEL SISTEMA DUAL:
- Si "accountingSystem" es **journalizer**: Todas las transacciones van en
"journalEntries" (Libro Diario).
- Si "accountingSystem" es **centralizer**:
    - Las transacciones de Ventas, Compras y Tesorería deben ir en **"auxiliaryJournals"**
(Libros Auxiliares).
    - Los asientos de apertura, ajustes y cierres siempre van en "journalEntries".
## CONTEXTO DEL CASO:
[DESCRIBE AQUÍ EL TIPO DE EMPRESA Y SITUACIÓN QUE DESEAS]
## ESTRUCTURA JSON REQUERIDA (Ejemplo]):
\`\`\`json
```

```
{
  "companies": [{
    "id": "uuid",
    "name": "Nombre Empresa SpA",
    "accountingSystem": "centralizer",
    "fiscalYear": 2024
  }],
  "accounts": [
    {"id": "acc-1", "companyId": "uuid", "code": "1.1.01.001", "name": "Caja", "type":
"asset", "balance": 0}
  ],
  "journalEntries": [
    {
      "id": "entry-1",
      "number": "ASI-001",
      "date": "2024-01-01",
      "description": "Asiento de Apertura",
      "lines": [...]
    }
  ],
  "auxiliaryJournals": [
    {
      "id": "aux-1",
      "companyId": "uuid",
      "module": "sales",
      "period": "2024-01",
      "status": "open",
      "transactions": [
        {
          "transactionId": "t-1",
          "date": "2024-01-05",
          "description": "Venta Factura N° 101",
          "lines": [...]
        }
      ]
    }
  ],
  "suppliers": [...],
  "customers": [...],
  "products": [...]
}
\`\`\`
## TRANSACCIONES A INCLUIR (mínimo 10):
1. Aporte de capital inicial.
2. Compra de mercadería y activos fijos.
3. Ventas con IVA (19%).
4. Pagos a proveedores y cobros a clientes.
5. Gastos operacionales y remuneraciones.
## VERIFICACIÓN FINAL:
- [ ] ¿Los asientos de "journalEntries" están cuadrados?
- [ ] ¿Las transacciones en "auxiliaryJournals" están cuadradas?
- [ ] ¿El catálogo de cuentas es coherente con los movimientos?
- [ ] ¿Los periodos de los auxiliares coinciden con las fechas de sus transacciones?
Genera el JSON completo y válido, listo para copiar e importar.`;
            const modalHTML = `
                <div id="ai-prompt-modal" style="position: fixed; top: 0; left: 0; right:
0; bottom: 0; background: rgba(0,0,0,0.7); display: flex; align-items: center; justify-
content: center; z-index: 9999; padding: var(--space-4);">
                    <div style="background: var(--bg-primary); border-radius: var(--
radius-xl); max-width: 900px; width: 100%; max-height: 90vh; display: flex; flex-
direction: column; box-shadow: var(--shadow-xl);">
                        <div style="padding: var(--space-4); border-bottom: 1px solid
var(--border-light); display: flex; justify-content: space-between; align-items: center;">
                            <h3 style="margin: 0;"><i class="fas fa-robot" style="color:
var(--primary-500);"></i> Prompt para Generar Datos con IA</h3>
                            <button id="close-ai-modal" style="background: none; border:
none; font-size: 1.5rem; cursor: pointer; color: var(--text-secondary);">&times;</button>
                        </div>
                        <div style="padding: var(--space-4); overflow-y: auto; flex: 1;">
                            <div class="alert alert-info" style="margin-bottom: var(--
space-4);">
                                <i class="fas fa-lightbulb"></i>
                                <div class="alert-content">
                                    <p style="margin: 0;"><strong>Instrucciones:</strong>
</p>
                                    <ol style="margin: var(--space-2) 0 0; padding-left:
var(--space-4); font-size: var(--font-size-sm);">
                                        <li>Copia el prompt de abajo</li>
                                        <li>Pégalo en ChatGPT, Claude, Gemini u otro
chatbot</li>
                                        <li>Personaliza la sección "[DESCRIBE AQUÍ...]"
```

```
con tu caso</li>
                                        <li>La IA generará un JSON válido</li>
                                        <li>Descarga o copia el JSON y súbelo aquí</li>
                                    </ol>
                                </div>
                            </div>
                            <div style="position: relative;">
                                <textarea id="ai-prompt-text" readonly style="width: 100%;
height: 400px; font-family: monospace; font-size: var(--font-size-sm); padding: var(--
space-4); border: 1px solid var(--border-medium); border-radius: var(--radius-lg); resize:
vertical; background: var(--neutral-900); color: #e5e7eb;">${prompt.replace(/</g,
'&lt;').replace(/>/g, '&gt;')}</textarea>
                            </div>
                        </div>
                        <div style="padding: var(--space-4); border-top: 1px solid var(--
border-light); display: flex; gap: var(--space-3); justify-content: flex-end;">
                            <button id="copy-ai-prompt" class="btn btn-primary">
                                <i class="fas fa-copy"></i> Copiar Prompt
                            </button>
                            <button id="close-ai-modal-btn" class="btn btn-outline">
                                Cerrar
                            </button>
                        </div>
                    </div>
                </div>
            `;
            document.body.insertAdjacentHTML('beforeend', modalHTML);
            const modal = document.getElementById('ai-prompt-modal');
            const closeBtn = document.getElementById('close-ai-modal');
            const closeBtnAlt = document.getElementById('close-ai-modal-btn');
            const copyBtn = document.getElementById('copy-ai-prompt');
            const textarea = document.getElementById('ai-prompt-text');
            const closeModal = () => modal.remove();
            closeBtn.addEventListener('click', closeModal);
            closeBtnAlt.addEventListener('click', closeModal);
            modal.addEventListener('click', (e) => {
                if (e.target === modal) closeModal();
            });
            copyBtn.addEventListener('click', () => {
                textarea.select();
                navigator.clipboard.writeText(prompt).then(() => {
                    Toast.success('Prompt copiado al portapapeles');
                    copyBtn.innerHTML = '<i class="fas fa-check"></i> ¡Copiado!';
                    setTimeout(() => {
                        copyBtn.innerHTML = '<i class="fas fa-copy"></i> Copiar Prompt';
                    }, 2000);
                }).catch(() => {
                    document.execCommand('copy');
                    Toast.success('Prompt copiado');
                });
            });
        });
        const uploadZone = document.getElementById('upload-zone');
        const fileInput = document.getElementById('import-file');
        uploadZone?.addEventListener('click', () => fileInput?.click());
        uploadZone?.addEventListener('dragover', (e) => {
            e.preventDefault();
            uploadZone.style.borderColor = 'var(--primary-500)';
            uploadZone.style.background = 'var(--primary-50)';
        });
        uploadZone?.addEventListener('dragleave', () => {
            uploadZone.style.borderColor = 'var(--border-medium)';
            uploadZone.style.background = 'transparent';
        });
        uploadZone?.addEventListener('drop', async (e) => {
            e.preventDefault();
            uploadZone.style.borderColor = 'var(--border-medium)';
            uploadZone.style.background = 'transparent';
            const file = e.dataTransfer.files[0];
            if (file) await this.handleImport(file);
        });
        fileInput?.addEventListener('change', async (e) => {
            if (e.target.files[0]) await this.handleImport(e.target.files[0]);
        });
        document.getElementById('btn-create-demo')?.addEventListener('click', async () =>
{
            const company = CompanyService.getCurrent();
            if (!company) {
                Toast.warning('Seleccione una empresa primero');
                return;
            }
```

```
            const loading = Toast.loading('Creando datos básicos...');
            try {
                await CompanyService.createDemoData();
                loading.success('Datos de demostración creados');
                App.navigate('admin', 'import-export');
            } catch (err) {
                loading.error(err.message);
            }
        });
        document.getElementById('btn-load-case-study')?.addEventListener('click', async ()
=> {
            const company = CompanyService.getCurrent();
            if (!company) {
                Toast.warning('Seleccione una empresa primero');
                return;
            }
            const loading = Toast.loading('Cargando caso de estudio comercial...');
            try {
                const response = await fetch('data/caso_estudio_comercial.json');
                if (!response.ok) throw new Error('No se pudo cargar el archivo');
                const data = await response.json();
                await CompanyService.importData(data);
                loading.success('Caso de estudio comercial cargado exitosamente');
                App.navigate('dashboard');
            } catch (err) {
                if (window.location.protocol === 'file:') {
                    loading.error('Error CORS: Use un servidor web local');
                    Modal.alert({
                        title: 'Restricción de Seguridad del Navegador',
                        message: `<p>Para cargar archivos JSON, necesita ejecutar el
sistema desde un servidor web local.</p>
                        <p style="margin-top: var(--space-3);"><strong>Opciones:</strong>
</p>
                        <ul style="margin: var(--space-2) 0; padding-left: var(--space-
4);">
                            <li>Instale la extensión <strong>Live Server</strong> en VS
Code</li>
                            <li>O ejecute: <code>npx http-server -o</code></li>
                            <li>O use la importación manual arrastrando el archivo
JSON</li>
                        </ul>
                        <p style="margin-top: var(--space-3); color: var(--text-tertiary);
font-size: var(--font-size-sm);">
                            El archivo está en:
<code>data/caso_estudio_comercial.json</code>
                        </p>`
                    });
                } else {
                    loading.error('Error: ' + err.message);
                }
            }
        });
        document.getElementById('btn-load-dual-case')?.addEventListener('click', async ()
=> {
            const company = CompanyService.getCurrent();
            if (!company) {
                Toast.warning('Seleccione una empresa primero');
                return;
            }
            const loading = Toast.loading('Cargando caso de sistema dual...');
            try {
                const response = await fetch('data/caso_prueba_sistema_dual.json');
                if (!response.ok) throw new Error('No se pudo cargar el archivo');
                const data = await response.json();
                const result = await CompanyService.importData(data);
                loading.success(`Caso dual cargado: ${result.success} registros
importados.`);
                App.navigate('dashboard');
            } catch (err) {
                loading.error('Error: ' + err.message);
            }
        });
        document.getElementById('btn-load-service-case')?.addEventListener('click', async
() => {
            const company = CompanyService.getCurrent();
            if (!company) {
                Toast.warning('Seleccione una empresa primero');
                return;
            }
            const loading = Toast.loading('Cargando caso de estudio servicios...');
            try {
```

```
                const response = await fetch('data/caso_estudio_servicios.json');
                if (!response.ok) throw new Error('No se pudo cargar el archivo');
                const data = await response.json();
                await CompanyService.importData(data);
                loading.success('Caso de estudio servicios cargado exitosamente');
                App.navigate('dashboard');
            } catch (err) {
                if (window.location.protocol === 'file:') {
                    loading.error('Error CORS: Use un servidor web local');
                    Modal.alert({
                        title: 'Restricción de Seguridad del Navegador',
                        message: `<p>Para cargar archivos JSON, necesita ejecutar el
sistema desde un servidor web local.</p>
                        <p style="margin-top: var(--space-3);"><strong>Opciones:</strong>
</p>
                        <ul style="margin: var(--space-2) 0; padding-left: var(--space-
4);">
                            <li>Instale la extensión <strong>Live Server</strong> en VS
Code</li>
                            <li>O ejecute: <code>npx http-server -o</code></li>
                            <li>O use la importación manual arrastrando el archivo
JSON</li>
                        </ul>
                        <p style="margin-top: var(--space-3); color: var(--text-tertiary);
font-size: var(--font-size-sm);">
                            El archivo está en:
<code>data/caso_estudio_servicios.json</code>
                        </p>`
                    });
                } else {
                    loading.error('Error: ' + err.message);
                }
            }
        });
        document.getElementById('btn-clear-data')?.addEventListener('click', async () => {
            const company = CompanyService.getCurrent();
            if (!company) {
                Toast.warning('Seleccione una empresa primero');
                return;
            }
            const confirm = await Modal.confirm({
                title: '¿Está seguro?',
                message: `Esta acción eliminará TODOS los datos de "${company.name}"
(cuentas, asientos, proveedores, clientes, productos, empleados, etc.). La empresa en sí
se mantendrá pero vacía. Esta acción NO se puede deshacer.`,
                confirmText: 'Sí, eliminar todo',
                confirmClass: 'btn-danger'
            });
            if (confirm) {
                const loading = Toast.loading('Limpiando datos...');
                try {
                    const storesWithCompanyId = [
                        'accounts', 'journalEntries', 'suppliers', 'customers',
'products',
                        'purchaseOrders', 'supplierInvoices', 'salesOrders',
'customerInvoices',
                        'stockMovements', 'bankAccounts', 'payments', 'costCenters',
                        'internalOrders', 'employees', 'payroll', 'sequences', 'settings',
'auditLog'
                    ];
                    let deleted = 0;
                    for (const storeName of storesWithCompanyId) {
                        try {
                            const records = await DB.getByIndex(storeName, 'companyId',
company.id);
                            for (const record of records) {
                                await DB.delete(storeName, record.id);
                                deleted++;
                                if (storeName === 'journalEntries') {
                                    const lines = await DB.getByIndex('journalLines',
'entryId', record.id);
                                    for (const line of lines) {
                                        await DB.delete('journalLines', line.id);
                                        deleted++;
                                    }
                                }
                            }
                        } catch (storeErr) {
                            console.warn(`Error limpiando ${storeName}:`,
storeErr.message);
                        }
```

```
                    }
                    loading.success(`Limpieza completada: ${deleted} registros
eliminados`);
                    App.navigate('admin', 'import-export');
                } catch (err) {
                    console.error('Error limpiando datos:', err);
                    loading.error('Error: ' + err.message);
                }
            }
        });
    },
    async handleImport(file) {
        const loading = Toast.loading('Importando datos...');
        try {
            let data;
            if (file.name.endsWith('.json')) {
                data = await ExportService.fromJSON(file);
            } else if (file.name.endsWith('.xlsx')) {
                data = await ExportService.fromExcel(file);
            } else {
                throw new Error('Formato de archivo no soportado');
            }
            const result = await CompanyService.importData(data);
            loading.success(`Importación completada: ${result.success} registros`);
            if (result.errors.length > 0) {
                Toast.warning(`${result.errors.length} errores durante la importación`);
            }
        } catch (err) {
            loading.error('Error: ' + err.message);
        }
    },
    initSettings() {
        document.getElementById('company-form')?.addEventListener('submit', async (e) => {
            e.preventDefault();
            const formData = new FormData(e.target);
            const data = Object.fromEntries(formData);
            try {
                await CompanyService.update(CompanyService.getCurrent().id, data);
                Toast.success('Empresa actualizada');
            } catch (err) {
                Toast.error(err.message);
            }
        });
        document.getElementById('settings-form')?.addEventListener('submit', async (e) =>
{
            e.preventDefault();
            const formData = new FormData(e.target);
            const settingsData = {
                fiscalYear: parseInt(formData.get('fiscalYear')),
                taxRate: parseFloat(formData.get('taxRate')),
                incomeTaxRate: parseFloat(formData.get('incomeTaxRate')),
                inventoryMethod: formData.get('inventoryMethod'),
                decimalPlaces: parseInt(formData.get('decimalPlaces'))
            };
            const accountingSystem = formData.get('accountingSystem');
            try {
                await CompanyService.updateSettings(settingsData);
                const company = CompanyService.getCurrent();
                if (company && accountingSystem) {
                    company.accountingSystem = accountingSystem;
                    await CompanyService.update(company.id, { accountingSystem });
                    CompanyService.currentCompany = company;
                    console.log(`✅ Sistema de contabilización cambiado a:
${accountingSystem}`);
                }
                Toast.success('Configuración guardada correctamente');
                if (accountingSystem === 'centralizer') {
                    Toast.info('📗 Sistema Centralizador activo. Las transacciones irán a
Libros Auxiliares.');
                } else {
                    Toast.info('📘 Sistema Jornalizador activo. Las transacciones irán
directo al Libro Diario.');
                }
            } catch (err) {
                Toast.error(err.message);
            }
        });
        document.getElementById('btn-clear-company-data')?.addEventListener('click', async
() => {
            const company = CompanyService.getCurrent();
```

```
            if (!company) return;
            const confirm = await Modal.confirm({
                title: 'Limpiar Datos',
                message: `¿Está seguro de eliminar TODOS los datos de "${company.name}"?
La empresa se mantendrá pero vacía. Esta acción NO se puede deshacer.`,
                confirmText: 'Sí, limpiar todo',
                confirmClass: 'btn-danger'
            });
            if (confirm) {
                const loading = Toast.loading('Limpiando datos...');
                try {
                    const stores = ['accounts', 'journalEntries', 'journalLines',
'suppliers', 'customers',
                        'products', 'purchaseOrders', 'supplierInvoices', 'salesOrders',
'customerInvoices',
                        'stockMovements', 'bankAccounts', 'payments', 'costCenters',
'employees',
                        'payroll', 'sequences', 'settings', 'auditLog'];
                    let deleted = 0;
                    for (const storeName of stores) {
                        try {
                            const allRecords = await DB.getAll(storeName);
                            const companyRecords = allRecords.filter(r => r.companyId ===
company.id);
                            for (const record of companyRecords) {
                                await DB.delete(storeName, record.id);
                                deleted++;
                            }
                        } catch (e) {
                            console.warn(`Error en ${storeName}:`, e.message);
                        }
                    }
                    loading.success(`Limpieza completada: ${deleted} registros
eliminados`);
                    App.navigate('admin', 'configuracion');
                } catch (err) {
                    loading.error('Error: ' + err.message);
                }
            }
        });
        document.getElementById('btn-delete-company')?.addEventListener('click', async ()
=> {
            const company = CompanyService.getCurrent();
            if (!company) return;
            const confirm = await Modal.confirm({
                title: 'Eliminar Empresa',
                message: `¿Está ABSOLUTAMENTE seguro de eliminar "${company.name}" y TODOS
sus datos? Esta acción es PERMANENTE y NO se puede deshacer.`,
                confirmText: 'Sí, eliminar permanentemente',
                confirmClass: 'btn-danger'
            });
            if (confirm) {
                const loading = Toast.loading('Eliminando empresa...');
                try {
                    await CompanyService.delete(company.id);
                    loading.success('Empresa eliminada');
                    App.navigate('admin', 'empresas');
                } catch (err) {
                    loading.error('Error: ' + err.message);
                }
            }
        });
    }
};
const adminStyles = document.createElement('style');
adminStyles.textContent = `
    .active-company { border: 2px solid var(--primary-500); background: var(--primary-50);
}
    .btn-purple { background: #7c3aed; color: white; border: none; }
    .btn-purple:hover { background: #6d28d9; }
`;
document.head.appendChild(adminStyles);
window.AdminModule = AdminModule;
```

Archivo: js\modules\compras.js

```
const ComprasModule = {
    currentView: 'proveedores',
    async render(view = 'proveedores') {
        this.currentView = view;
        switch (view) {
            case 'proveedores': return await this.renderSuppliers();
            case 'ordenes-compra': return await this.renderPurchaseOrders();
            case 'facturas-proveedor': return await this.renderSupplierInvoices();
            default: return '<p>Vista no encontrada</p>';
        }
    },
    async renderSuppliers() {
        return `
            <div class="page-header">
                <h1 class="page-title">Proveedores</h1>
                <p class="page-subtitle">Gestión de proveedores y condiciones de
compra</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <div class="search-box">
                        <i class="fas fa-search"></i>
                        <input type="text" class="form-control" placeholder="Buscar
proveedor..." id="search-suppliers">
                    </div>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-supplier">
                        <i class="fas fa-plus"></i> Nuevo Proveedor
                    </button>
                </div>
            </div>
            <div id="suppliers-table"></div>
        `;
    },
    async renderPurchaseOrders() {
        return `
            <div class="page-header">
                <h1 class="page-title">Órdenes de Compra</h1>
                <p class="page-subtitle">Gestión de pedidos a proveedores</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="filter-date-from">
                    <input type="date" class="form-control" id="filter-date-to">
                    <select class="form-control" id="filter-status">
                        <option value="">Todos</option>
                        <option value="draft">Borrador</option>
                        <option value="approved">Aprobada</option>
                        <option value="received">Recibida</option>
                        <option value="cancelled">Anulada</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-order">
                        <i class="fas fa-plus"></i> Nueva Orden
                    </button>
                </div>
            </div>
            <div id="orders-table"></div>
        `;
    },
    async renderSupplierInvoices() {
        return `
            <div class="page-header">
                <h1 class="page-title">Facturas de Proveedor</h1>
                <p class="page-subtitle">Registro y control de facturas recibidas</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="filter-date-from">
                    <input type="date" class="form-control" id="filter-date-to">
                    <select class="form-control" id="filter-status">
                        <option value="">Todos</option>
                        <option value="pending">Pendiente</option>
                        <option value="posted">Contabilizada</option>
```

```
                        <option value="paid">Pagada</option>
                        <option value="cancelled">Anulada</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-invoice">
                        <i class="fas fa-plus"></i> Nueva Factura
                    </button>
                </div>
            </div>
            <div id="invoices-table"></div>
        `;
    },
    async init(view) {
        const company = CompanyService.getCurrent();
        if (!company) return;
        switch (view) {
            case 'proveedores':
                await this.initSuppliers();
                break;
            case 'ordenes-compra':
                await this.initPurchaseOrders();
                break;
            case 'facturas-proveedor':
                await this.initSupplierInvoices();
                break;
        }
    },
    async initSuppliers() {
        const company = CompanyService.getCurrent();
        const suppliers = await DB.getByIndex('suppliers', 'companyId', company.id);
        DataTable.create('suppliers-table', {
            columns: [
                { key: 'rut', label: 'RUT' },
                { key: 'name', label: 'Nombre' },
                { key: 'contact', label: 'Contacto' },
                { key: 'email', label: 'Email' },
                { key: 'phone', label: 'Teléfono' },
                { key: 'balance', label: 'Saldo', format: 'currency', class: 'text-right'
}
            ],
            data: suppliers,
            actions: [
                { name: 'edit', label: 'Editar', icon: 'fas fa-edit', handler: (row) =>
this.showSupplierModal(row.id) },
                { name: 'delete', label: 'Eliminar', icon: 'fas fa-trash', handler: (row)
=> this.deleteSupplier(row.id) }
            ],
            emptyMessage: 'No hay proveedores registrados'
        });
        document.getElementById('btn-new-supplier')?.addEventListener('click', () =>
this.showSupplierModal());
    },
    async showSupplierModal(supplierId = null) {
        const isEdit = !!supplierId;
        let data = {};
        if (isEdit) {
            data = await DB.get('suppliers', supplierId);
        }
        await Modal.form({
            title: isEdit ? 'Editar Proveedor' : 'Nuevo Proveedor',
            fields: [
                { name: 'rut', label: 'RUT', required: true, placeholder: '76.123.456-7',
default: data.rut },
                { name: 'name', label: 'Razón Social', required: true, placeholder:
'Nombre empresa', default: data.name },
                { name: 'contact', label: 'Contacto', placeholder: 'Nombre contacto',
default: data.contact },
                { name: 'email', label: 'Email', type: 'email', placeholder:
'email@empresa.cl', default: data.email },
                { name: 'phone', label: 'Teléfono', placeholder: '+56 9 1234 5678',
default: data.phone },
                { name: 'address', label: 'Dirección', placeholder: 'Dirección comercial',
default: data.address }
            ],
            validate: (formData) => {
                if (!formData.name) return 'El nombre es requerido';
                if (!Helpers.validateRUT(formData.rut)) return 'RUT inválido';
                return true;
            },
            onSubmit: async (formData) => {
```

```
                const company = CompanyService.getCurrent();
                formData.rut = Helpers.formatRUT(formData.rut);
                if (isEdit) {
                    await DB.update('suppliers', { ...data, ...formData });
                    Toast.success('Proveedor actualizado');
                } else {
                    await DB.add('suppliers', { companyId: company.id, ...formData,
status: 'active', balance: 0 });
                    Toast.success('Proveedor creado');
                }
                App.navigate('compras', 'proveedores');
            }
        });
    },
    async deleteSupplier(id) {
        const confirm = await Modal.confirm({
            title: 'Eliminar Proveedor',
            message: '¿Está seguro de eliminar este proveedor?',
            confirmText: 'Eliminar',
            confirmClass: 'btn-danger'
        });
        if (confirm) {
            await DB.delete('suppliers', id);
            Toast.success('Proveedor eliminado');
            App.navigate('compras', 'proveedores');
        }
    },
    async initPurchaseOrders() {
        const company = CompanyService.getCurrent();
        const allOrders = await DB.getByIndex('purchaseOrders', 'companyId', company.id);
        for (let order of allOrders) {
            const supplier = await DB.get('suppliers', order.supplierId);
            order.supplierName = supplier?.name || 'Proveedor desconocido';
        }
        const applyFilters = () => {
            const dateFrom = document.getElementById('filter-date-from')?.value;
            const dateTo = document.getElementById('filter-date-to')?.value;
            const status = document.getElementById('filter-status')?.value;
            let filteredOrders = allOrders;
            if (dateFrom) {
                filteredOrders = filteredOrders.filter(o => o.date >= dateFrom);
            }
            if (dateTo) {
                filteredOrders = filteredOrders.filter(o => o.date <= dateTo);
            }
            if (status) {
                filteredOrders = filteredOrders.filter(o => o.status === status);
            }
            DataTable.create('orders-table', {
                columns: [
                    { key: 'number', label: 'Número' },
                    { key: 'date', label: 'Fecha', format: 'date' },
                    { key: 'supplierName', label: 'Proveedor' },
                    { key: 'total', label: 'Total', format: 'currency', class: 'text-
right' },
                    { key: 'status', label: 'Estado', format: 'status' }
                ],
                data: filteredOrders,
                actions: [
                    { name: 'view', label: 'Ver', icon: 'fas fa-eye', handler: (row) =>
this.showOrderDetail(row) },
                    { name: 'edit', label: 'Editar', icon: 'fas fa-edit', handler: (row)
=> this.editPurchaseOrder(row) },
                    { name: 'approve', label: 'Aprobar', icon: 'fas fa-check', handler:
(row) => this.approveOrder(row) },
                    { name: 'receive', label: 'Recibir', icon: 'fas fa-truck', handler:
(row) => this.receiveOrder(row) },
                    { name: 'cancel', label: 'Anular', icon: 'fas fa-ban', handler: (row)
=> this.cancelPurchaseOrder(row) }
                ],
                emptyMessage: 'No hay órdenes de compra'
            });
        };
        applyFilters();
        document.getElementById('filter-date-from')?.addEventListener('change',
applyFilters);
        document.getElementById('filter-date-to')?.addEventListener('change',
applyFilters);
        document.getElementById('filter-status')?.addEventListener('change',
applyFilters);
        document.getElementById('btn-new-order')?.addEventListener('click', () => {
```

```
            this.showPurchaseOrderModal();
        });
    },
    async showPurchaseOrderModal() {
        const company = CompanyService.getCurrent();
        const suppliers = await DB.getByIndex('suppliers', 'companyId', company.id);
        const products = await DB.getByIndex('products', 'companyId', company.id);
        if (suppliers.length === 0) {
            Toast.warning('Primero debe crear al menos un proveedor');
            return;
        }
        Modal.open({
            title: 'Nueva Orden de Compra',
            size: 'large',
            content: `
                <form id="purchase-order-form">
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Proveedor <span class="text-
danger">*</span></label>
                            <select class="form-control" name="supplierId" required>
                                <option value="">Seleccione proveedor...</option>
                                ${suppliers.map(s => `<option value="${s.id}">${s.name}
</option>`).join('')}
                            </select>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Fecha</label>
                            <input type="date" class="form-control" name="date"
value="${Helpers.getCurrentDate()}" required>
                        </div>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Observaciones</label>
                        <textarea class="form-control" name="notes" rows="2"
placeholder="Observaciones opcionales"></textarea>
                    </div>
                    <h4 style="margin: var(--space-4) 0 var(--space-2);">Productos</h4>
                    <div class="table-container">
                        <table class="data-table" id="order-lines-table">
                            <thead>
                                <tr>
                                    <th style="width: 40%;">Producto</th>
                                    <th style="width: 15%;">Cantidad</th>
                                    <th style="width: 15%;">Precio Unit.</th>
                                    <th style="width: 20%;">Subtotal</th>
                                    <th style="width: 10%;"></th>
                                </tr>
                            </thead>
                            <tbody id="order-lines">
                                <tr class="order-line" data-idx="0">
                                    <td>
                                        <select class="form-control product-select"
name="productId" required>
                                            <option value="">Seleccione...</option>
                                            ${products.map(p => `<option value="${p.id}"
data-cost="${p.cost}">${p.code} - ${p.name}</option>`).join('')}
                                        </select>
                                    </td>
                                    <td><input type="number" class="form-control quantity-
input" name="quantity" value="1" min="1" required></td>
                                    <td><input type="number" class="form-control price-
input" name="price" value="0" min="0" required></td>
                                    <td class="text-right subtotal">$0</td>
                                    <td><button type="button" class="btn btn-icon btn-
ghost remove-line"><i class="fas fa-times"></i></button></td>
                                </tr>
                            </tbody>
                            <tfoot>
                                <tr>
                                    <td colspan="3" class="text-right"><strong>TOTAL:
</strong></td>
                                    <td class="text-right"><strong id="order-
total">$0</strong></td>
                                    <td></td>
                                </tr>
                            </tfoot>
                        </table>
                    </div>
                    <button type="button" class="btn btn-outline" id="btn-add-line"
style="margin-top: var(--space-2);">
```

```
                        <i class="fas fa-plus"></i> Agregar Producto
                    </button>
                </form>
            `,
            footer: `
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-primary" id="btn-save-order"><i class="fas fa-
save"></i> Guardar Orden</button>
            `
        });
        this.initOrderModalEvents(products);
    },
    initOrderModalEvents(products) {
        let lineIdx = 1;
        document.getElementById('btn-add-line')?.addEventListener('click', () => {
            const tbody = document.getElementById('order-lines');
            const newRow = document.createElement('tr');
            newRow.className = 'order-line';
            newRow.dataset.idx = lineIdx++;
            newRow.innerHTML = `
                <td>
                    <select class="form-control product-select" name="productId" required>
                        <option value="">Seleccione...</option>
                        ${products.map(p => `<option value="${p.id}" data-
cost="${p.cost}">${p.code} - ${p.name}</option>`).join('')}
                    </select>
                </td>
                <td><input type="number" class="form-control quantity-input"
name="quantity" value="1" min="1" required></td>
                <td><input type="number" class="form-control price-input" name="price"
value="0" min="0" required></td>
                <td class="text-right subtotal">$0</td>
                <td><button type="button" class="btn btn-icon btn-ghost remove-line"><i
class="fas fa-times"></i></button></td>
            `;
            tbody.appendChild(newRow);
            this.bindOrderLineEvents();
        });
        this.bindOrderLineEvents();
        document.getElementById('btn-save-order')?.addEventListener('click', async () => {
            await this.savePurchaseOrder();
        });
    },
    bindOrderLineEvents() {
        const self = this;
        document.querySelectorAll('.product-select').forEach(select => {
            select.removeEventListener('change', select._changeHandler);
            select._changeHandler = function () {
                const option = select.selectedOptions[0];
                const cost = parseFloat(option?.dataset.cost || 0);
                const row = select.closest('tr');
                row.querySelector('.price-input').value = cost;
                self.updateOrderTotals();
            };
            select.addEventListener('change', select._changeHandler);
        });
        document.querySelectorAll('.quantity-input, .price-input').forEach(input => {
            input.removeEventListener('input', input._inputHandler);
            input._inputHandler = function () {
                self.updateOrderTotals();
            };
            input.addEventListener('input', input._inputHandler);
        });
        document.querySelectorAll('.remove-line').forEach(btn => {
            btn.removeEventListener('click', btn._clickHandler);
            btn._clickHandler = function () {
                const lines = document.querySelectorAll('.order-line');
                if (lines.length > 1) {
                    btn.closest('tr').remove();
                    self.updateOrderTotals();
                } else {
                    Toast.warning('Debe haber al menos una línea');
                }
            };
            btn.addEventListener('click', btn._clickHandler);
        });
        this.updateOrderTotals();
    },
    updateOrderTotals() {
        let total = 0;
```

```
        document.querySelectorAll('.order-line').forEach(row => {
            const qty = parseFloat(row.querySelector('.quantity-input')?.value) || 0;
            const price = parseFloat(row.querySelector('.price-input')?.value) || 0;
            const subtotal = qty * price;
            const subtotalEl = row.querySelector('.subtotal');
            if (subtotalEl) subtotalEl.textContent = Formatters.currency(subtotal);
            total += subtotal;
        });
        const totalEl = document.getElementById('order-total');
        if (totalEl) totalEl.textContent = Formatters.currency(total);
    },
    async savePurchaseOrder() {
        const form = document.getElementById('purchase-order-form');
        const formData = new FormData(form);
        const supplierId = formData.get('supplierId');
        if (!supplierId) {
            Toast.error('Seleccione un proveedor');
            return;
        }
        const lines = [];
        let total = 0;
        document.querySelectorAll('.order-line').forEach(row => {
            const productId = row.querySelector('.product-select').value;
            const quantity = parseFloat(row.querySelector('.quantity-input').value) || 0;
            const price = parseFloat(row.querySelector('.price-input').value) || 0;
            if (productId && quantity > 0) {
                const subtotal = quantity * price;
                lines.push({ productId, quantity, price, subtotal });
                total += subtotal;
            }
        });
        if (lines.length === 0) {
            Toast.error('Agregue al menos un producto');
            return;
        }
        const company = CompanyService.getCurrent();
        const orderNumber = await this.getNextOrderNumber();
        const order = {
            id: Helpers.generateId(),
            companyId: company.id,
            number: orderNumber,
            supplierId,
            date: formData.get('date'),
            notes: formData.get('notes'),
            status: 'draft',
            total,
            createdAt: new Date().toISOString()
        };
        try {
            await DB.add('purchaseOrders', order);
            for (const line of lines) {
                await DB.add('purchaseOrderLines', {
                    id: Helpers.generateId(),
                    orderId: order.id,
                    ...line
                });
            }
            Toast.success('Orden de compra creada');
            Modal.close();
            App.navigate('compras', 'ordenes-compra');
        } catch (err) {
            Toast.error('Error: ' + err.message);
        }
    },
    async getNextOrderNumber() {
        const company = CompanyService.getCurrent();
        const orders = await DB.getByIndex('purchaseOrders', 'companyId', company.id);
        const year = new Date().getFullYear();
        const prefix = `OC-${year}-`;
        const existingNumbers = orders
            .filter(o => {
                const numStr = String(o.number || '');
                return numStr.startsWith(prefix);
            })
            .map(o => parseInt(String(o.number || '').replace(prefix, '')) || 0);
        const nextNum = Math.max(0, ...existingNumbers) + 1;
        return `${prefix}${String(nextNum).padStart(4, '0')}`;
    },
    async showOrderDetail(order) {
        const supplier = await DB.get('suppliers', order.supplierId);
        const lines = await DB.getByIndex('purchaseOrderLines', 'orderId', order.id);
```

```
        for (let line of lines) {
            const product = await DB.get('products', line.productId);
            line.productName = product?.name || 'Producto desconocido';
            line.productCode = product?.code || '';
        }
        const statusLabels = {
            draft: '<span class="badge badge-warning">Borrador</span>',
            approved: '<span class="badge badge-info">Aprobada</span>',
            received: '<span class="badge badge-success">Recibida</span>',
            cancelled: '<span class="badge badge-error">Cancelada</span>'
        };
        Modal.open({
            title: `Orden de Compra ${order.number}`,
            size: 'large',
            content: `
                <div style="margin-bottom: var(--space-4);">
                    <div style="display: grid; grid-template-columns: repeat(3, 1fr); gap:
var(--space-4);">
                        <div>
                            <label style="font-size: var(--font-size-xs); color: var(--
text-tertiary);">Proveedor</label>
                            <strong>${supplier?.name || 'N/A'}</strong>
                        </div>
                        <div>
                            <label style="font-size: var(--font-size-xs); color: var(--
text-tertiary);">Fecha</label>
                            <strong>${Formatters.date(order.date)}</strong>
                        </div>
                        <div>
                            <label style="font-size: var(--font-size-xs); color: var(--
text-tertiary);">Estado</label>
                            ${statusLabels[order.status] || order.status}
                        </div>
                    </div>
                </div>
                <div class="table-container">
                    <table class="data-table">
                        <thead>
                            <tr>
                                <th>Código</th>
                                <th>Producto</th>
                                <th class="text-right">Cantidad</th>
                                <th class="text-right">Precio</th>
                                <th class="text-right">Subtotal</th>
                            </tr>
                        </thead>
                        <tbody>
                            ${lines.map(l => `
                                <tr>
                                    <td><code>${l.productCode}</code></td>
                                    <td>${l.productName}</td>
                                    <td class="text-right">${l.quantity}</td>
                                    <td class="text-right">${Formatters.currency(l.price)}
</td>
                                    <td class="text-
right">${Formatters.currency(l.subtotal)}</td>
                                </tr>
                            `).join('')}
                        </tbody>
                        <tfoot>
                            <tr style="font-weight: 600;">
                                <td colspan="4" class="text-right">TOTAL</td>
                                <td class="text-right">${Formatters.currency(order.total)}
</td>
                            </tr>
                        </tfoot>
                    </table>
                </div>
            `,
            footer: `<button class="btn btn-secondary"
onclick="Modal.close()">Cerrar</button>`
        });
    },
    async editPurchaseOrder(order) {
        if (order.status !== 'draft') {
            const statusName = {
                approved: 'aprobada',
                received: 'recibida',
                cancelled: 'anulada'
            }[order.status] || order.status;
            Toast.warning(`No se puede editar una orden ${statusName}. Solo las órdenes en
```

```
borrador son editables.`);
            return;
        }
        const company = CompanyService.getCurrent();
        const suppliers = await DB.getByIndex('suppliers', 'companyId', company.id);
        const products = await DB.getByIndex('products', 'companyId', company.id);
        const existingLines = await DB.getByIndex('purchaseOrderLines', 'orderId',
order.id);
        let linesHtml = '';
        for (let i = 0; i < existingLines.length; i++) {
            const line = existingLines[i];
            const product = products.find(p => p.id === line.productId);
            const linePrice = line.cost || line.price || 0;
            linesHtml += `
                <tr class="order-line" data-idx="${i}">
                    <td>
                        <select class="form-control product-select" name="productId"
required>
                            <option value="">Seleccione...</option>
                            ${products.map(p => `<option value="${p.id}" data-
cost="${p.cost}" ${p.id === line.productId ? 'selected' : ''}>${p.code} - ${p.name}
</option>`).join('')}
                        </select>
                    </td>
                    <td><input type="number" class="form-control quantity-input"
name="quantity" value="${line.quantity}" min="1" required></td>
                    <td><input type="number" class="form-control price-input" name="price"
value="${linePrice}" min="0" required></td>
                    <td class="text-right subtotal">${Formatters.currency(line.subtotal)}
</td>
                    <td><button type="button" class="btn btn-icon btn-ghost remove-line">
<i class="fas fa-times"></i></button></td>
                </tr>
            `;
        }
        Modal.open({
            title: `Editar Orden ${order.number}`,
            size: 'large',
            content: `
                <form id="edit-order-form">
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Proveedor <span class="text-error">*
</span></label>
                            <select class="form-control" name="supplierId" required>
                                <option value="">Seleccione...</option>
                                ${suppliers.map(s => `<option value="${s.id}" ${s.id ===
order.supplierId ? 'selected' : ''}>${s.rut} - ${s.name}</option>`).join('')}
                            </select>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Fecha</label>
                            <input type="date" class="form-control" name="date"
value="${order.date}" required>
                        </div>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Observaciones</label>
                        <textarea class="form-control" name="notes" rows="2">${order.notes
|| ''}</textarea>
                    </div>
                    <h4 style="margin: var(--space-4) 0 var(--space-2);">Productos</h4>
                    <div class="table-container">
                        <table class="data-table" id="order-lines-table">
                            <thead>
                                <tr>
                                    <th style="width: 40%;">Producto</th>
                                    <th style="width: 15%;">Cantidad</th>
                                    <th style="width: 20%;">Costo Unit.</th>
                                    <th style="width: 15%;">Subtotal</th>
                                    <th style="width: 10%;"></th>
                                </tr>
                            </thead>
                            <tbody id="order-lines">
                                ${linesHtml}
                            </tbody>
                        </table>
                    </div>
                    <button type="button" class="btn btn-outline btn-sm" id="btn-add-line"
style="margin-top: var(--space-2);">
                        <i class="fas fa-plus"></i> Agregar línea
```

```
                    </button>
                    <div class="order-totals" style="margin-top: var(--space-4); text-
align: right;">
                        <div style="font-size: 1.2em;">Total: <strong id="order-
total">$0</strong></div>
                    </div>
                </form>
            `,
            footer: `
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-primary" id="btn-save-edit-order">Guardar
Cambios</button>
            `
        });
        this.bindOrderLineEvents();
        this.updateOrderTotals();
        document.getElementById('btn-add-line')?.addEventListener('click', () => {
            const tbody = document.getElementById('order-lines');
            const idx = tbody.querySelectorAll('tr').length;
            const newRow = document.createElement('tr');
            newRow.className = 'order-line';
            newRow.dataset.idx = idx;
            newRow.innerHTML = `
                <td>
                    <select class="form-control product-select" name="productId" required>
                        <option value="">Seleccione...</option>
                        ${products.map(p => `<option value="${p.id}" data-
cost="${p.cost}">${p.code} - ${p.name}</option>`).join('')}
                    </select>
                </td>
                <td><input type="number" class="form-control quantity-input"
name="quantity" value="1" min="1" required></td>
                <td><input type="number" class="form-control price-input" name="price"
value="0" min="0" required></td>
                <td class="text-right subtotal">$0</td>
                <td><button type="button" class="btn btn-icon btn-ghost remove-line"><i
class="fas fa-times"></i></button></td>
            `;
            tbody.appendChild(newRow);
            this.bindOrderLineEvents();
        });
        document.getElementById('btn-save-edit-order')?.addEventListener('click', async ()
=> {
            await this.saveEditedPurchaseOrder(order);
        });
    },
    async saveEditedPurchaseOrder(originalOrder) {
        const form = document.getElementById('edit-order-form');
        const formData = new FormData(form);
        const supplierId = formData.get('supplierId');
        const date = formData.get('date');
        const notes = formData.get('notes');
        if (!supplierId) {
            Toast.error('Debe seleccionar un proveedor');
            return;
        }
        const lines = [];
        document.querySelectorAll('#order-lines .order-line').forEach(row => {
            const productId = row.querySelector('.product-select')?.value;
            const quantity = parseFloat(row.querySelector('.quantity-input')?.value) || 0;
            const price = parseFloat(row.querySelector('.price-input')?.value) || 0;
            if (productId && quantity > 0) {
                lines.push({
                    productId,
                    quantity,
                    price,
                    cost: price,
                    subtotal: quantity * price
                });
            }
        });
        if (lines.length === 0) {
            Toast.error('Debe agregar al menos un producto');
            return;
        }
        try {
            const company = CompanyService.getCurrent();
            const subtotal = lines.reduce((sum, l) => sum + l.subtotal, 0);
            const iva = Math.round(subtotal * 0.19);
            const total = subtotal + iva;
```

```
            await DB.update('purchaseOrders', {
                ...originalOrder,
                supplierId,
                date,
                notes,
                subtotal,
                iva,
                total,
                updatedAt: new Date().toISOString()
            });
            const oldLines = await DB.getByIndex('purchaseOrderLines', 'orderId',
originalOrder.id);
            for (const oldLine of oldLines) {
                await DB.delete('purchaseOrderLines', oldLine.id);
            }
            for (const line of lines) {
                await DB.add('purchaseOrderLines', {
                    id: Helpers.generateId(),
                    orderId: originalOrder.id,
                    companyId: company.id,
                    ...line
                });
            }
            Toast.success('Orden actualizada');
            Modal.close();
            App.navigate('compras', 'ordenes');
        } catch (err) {
            Toast.error('Error: ' + err.message);
        }
    },
    async approveOrder(order) {
        if (order.status !== 'draft') {
            Toast.warning('Solo se pueden aprobar órdenes en borrador');
            return;
        }
        const confirm = await Modal.confirm({
            title: 'Aprobar Orden',
            message: `¿Aprobar la orden ${order.number}?`,
            confirmText: 'Aprobar'
        });
        if (confirm) {
            await DB.update('purchaseOrders', { ...order, status: 'approved' });
            Toast.success('Orden aprobada');
            App.navigate('compras', 'ordenes-compra');
        }
    },
    async receiveOrder(order) {
        if (order.status !== 'approved' && order.status !== 'partial') {
            Toast.warning('Solo se pueden recibir órdenes aprobadas o parcialmente
recibidas');
            return;
        }
        const company = CompanyService.getCurrent();
        const lines = await DB.getByIndex('purchaseOrderLines', 'orderId', order.id);
        const supplier = await DB.get('suppliers', order.supplierId);
        let pendingLinesCount = 0;
        let htmlLines = '';
        for (const line of lines) {
            const pending = line.quantity - (line.received || 0);
            if (pending <= 0) continue;
            pendingLinesCount++;
            const product = await DB.get('products', line.productId);
            const prodName = product ? `${product.code} - ${product.name}` : `Producto
${line.productId}`;
            htmlLines += `
                <div class="line-item" data-line-id="${line.id}" data-product-
id="${line.productId}" data-pending="${pending}" data-price="${line.price}"
style="padding: 1rem; border: 1px solid var(--border-color); border-radius: var(--radius-
md); margin-bottom: 1rem;">
                    <div style="font-weight: 500; margin-bottom: 0.5rem;">${prodName}
<span class="badge badge-neutral" style="float: right;">Pendiente: ${pending}</span></div>
                    <div class="form-row">
                        <div class="form-group" style="flex: 1;">
                            <label class="form-label">Recibir</label>
                            <input type="number" class="form-control to-receive"
value="${pending}" max="${pending}" min="0" step="any">
                        </div>
                        <div class="form-group" style="flex: 2;">
                            <label class="form-label">Comentario (Diferencias)</label>
                            <input type="text" class="form-control comment"
placeholder="Justificación si recibe menos de ${pending}">
```

```
                        </div>
                    </div>
                </div>
            `;
        }
        if (pendingLinesCount === 0) {
            Toast.info('Esta orden ya fue recibida en su totalidad');
            return;
        }
        Modal.open({
            title: `Recepción Guía de Despacho (OC ${order.number})`,
            size: 'large',
            content: `
                <div class="alert alert-info" style="margin-bottom: 1rem;">
                    Indique la cantidad real recibida. Las recepciones parciales
mantendrán la orden abierta y permitirán su recepción futura.
                </div>
                <div id="receipt-lines-container">
                    ${htmlLines}
                </div>
                <div class="modal-footer" style="margin-top: 1.5rem; display: flex;
justify-content: flex-end; gap: 0.5rem;">
                    <button class="btn btn-outline"
onclick="Modal.close()">Cancelar</button>
                    <button class="btn btn-primary" id="btn-confirm-receipt"><i class="fas
fa-check"></i> Confirmar Recepción</button>
                </div>
            `
        });
        document.getElementById('btn-confirm-receipt').addEventListener('click', async ()
=> {
            const lineItems = Array.from(document.querySelectorAll('.line-item'));
            const receiptData = [];
            let hasErrors = false;
            let totalReceiptValueNet = 0;
            for (const item of lineItems) {
                const pending = parseFloat(item.dataset.pending);
                const toReceive = parseFloat(item.querySelector('.to-receive').value) ||
0;
                const comment = item.querySelector('.comment').value.trim();
                const price = parseFloat(item.dataset.price) || 0;
                if (toReceive < 0 || toReceive > pending) {
                    Toast.error('La cantidad a recibir debe estar entre 0 y la cantidad
pendiente');
                    hasErrors = true;
                    break;
                }
                if (toReceive > 0 && toReceive < pending && !comment) {
                    Toast.error('Debe ingresar un comentario para justificar la diferencia
recibida');
                    hasErrors = true;
                    break;
                }
                if (toReceive > 0) {
                    receiptData.push({
                        lineId: item.dataset.lineId,
                        productId: item.dataset.productId,
                        pending,
                        toReceive,
                        comment,
                        price
                    });
                    totalReceiptValueNet += toReceive * price;
                }
            }
            if (hasErrors) return;
            if (receiptData.length === 0) {
                Toast.error('Debe recibir al menos una unidad para confirmar');
                return;
            }
            const loading = Toast.loading('Procesando recepción...');
            try {
                const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
                let inventoryAccount = accounts.find(a =>
                    a.type === 'asset' && (a.name?.toLowerCase().includes('mercader') ||
a.name?.toLowerCase().includes('inventario'))
                ) || accounts.find(a => a.type === 'asset' && a.code === '1.1.07');
                let bridgeAccount = accounts.find(a =>
                    a.code === '2.1.05' || a.name?.toLowerCase().includes('facturas por
recibir')
                );
```

```
                if (!inventoryAccount) {
                    inventoryAccount = { id: Helpers.generateId(), companyId: company.id,
code: '1.1.07', name: 'Mercaderías', type: 'asset', nature: 'debit', level: 2, isActive:
true };
                    await DB.add('accounts', inventoryAccount);
                }
                if (!bridgeAccount) {
                    bridgeAccount = { id: Helpers.generateId(), companyId: company.id,
code: '2.1.05', name: 'Proveedores - Facturas por Recibir', type: 'liability', nature:
'credit', level: 2, isActive: true };
                    await DB.add('accounts', bridgeAccount);
                }
                for (const data of receiptData) {
                    const product = await DB.get('products', data.productId);
                    if (product && !product.isService) {
                        const newStock = (product.stock || 0) + data.toReceive;
                        await DB.update('products', { ...product, stock: newStock });
                        await DB.add('stockMovements', {
                            id: Helpers.generateId(),
                            companyId: company.id,
                            productId: data.productId,
                            type: 'in',
                            quantity: data.toReceive,
                            unitCost: data.price,
                            date: Helpers.getCurrentDate(),
                            reference: order.number,
                            description: `Recepción ${data.toReceive < data.pending ?
'Parcial ' : ''}OC ${order.number}${data.comment ? ' - ' + data.comment : ''}`,
                            createdAt: new Date().toISOString()
                        });
                    }
                    const line = lines.find(l => l.id === data.lineId);
                    if (line) {
                        await DB.update('purchaseOrderLines', {
                            ...line,
                            received: (line.received || 0) + data.toReceive,
                            comment: data.comment || line.comment
                        });
                    }
                }
                if (totalReceiptValueNet > 0) {
                    await AccountingService.registerTransaction('purchases', {
                        date: Helpers.getCurrentDate(),
                        description: `Recepción Guía s/OC ${order.number} -
${supplier?.name || 'Proveedor'}`,
                        reference: `GR-${order.number}`,
                        sourceDocument: 'purchaseOrder',
                        sourceDocumentId: order.id,
                        autoPost: true,
                        lines: [
                            { accountId: inventoryAccount.id, description: `Ingreso
mercadería OC ${order.number}`, debit: totalReceiptValueNet, credit: 0 },
                            { accountId: bridgeAccount.id, description: `Factura por
recibir s/OC ${order.number}`, debit: 0, credit: totalReceiptValueNet }
                        ]
                    });
                } else {
                    console.warn('[receiveOrder] totalReceiptValueNet es 0 o negativo, no
se genera asiento:', totalReceiptValueNet);
                }
                const updatedLines = await DB.getByIndex('purchaseOrderLines', 'orderId',
order.id);
                const allReceived = updatedLines.every(l => (l.received || 0) >=
l.quantity);
                await DB.update('purchaseOrders', {
                    ...order,
                    status: allReceived ? 'received' : 'partial',
                    receivedAt: new Date().toISOString()
                });
                loading.success('Recepción registrada y contabilizada exitosamente');
                Modal.close();
                App.navigate('compras', 'ordenes-compra');
            } catch (err) {
                loading.error('Error: ' + err.message);
                console.error(err);
            }
        });
    },
    async initSupplierInvoices() {
        const company = CompanyService.getCurrent();
        const invoices = await DB.getByIndex('supplierInvoices', 'companyId', company.id);
```

```
        for (let inv of invoices) {
            const supplier = await DB.get('suppliers', inv.supplierId);
            inv.supplierName = supplier?.name || 'Proveedor desconocido';
        }
        DataTable.create('invoices-table', {
            columns: [
                { key: 'number', label: 'Número' },
                { key: 'date', label: 'Fecha', format: 'date' },
                { key: 'supplierName', label: 'Proveedor' },
                { key: 'dueDate', label: 'Vencimiento', format: 'date' },
                { key: 'total', label: 'Total', format: 'currency', class: 'text-right' },
                { key: 'status', label: 'Estado', format: 'status' }
            ],
            data: invoices,
            actions: [
                { name: 'view', label: 'Ver', icon: 'fas fa-eye', handler: (row) =>
this.showInvoiceDetail(row) },
                { name: 'post', label: 'Contabilizar', icon: 'fas fa-book', handler: (row)
=> this.postSupplierInvoice(row) },
                { name: 'pay', label: 'Pagar', icon: 'fas fa-money-bill', handler: (row)
=> this.paySupplierInvoice(row) }
            ],
            emptyMessage: 'No hay facturas de proveedor'
        });
        document.getElementById('btn-new-invoice')?.addEventListener('click', () => {
            this.showInvoiceModal();
        });
    },
    async showInvoiceModal(preloadOrderId = null) {
        const company = CompanyService.getCurrent();
        const suppliers = await DB.getByIndex('suppliers', 'companyId', company.id);
        const products = await DB.getByIndex('products', 'companyId', company.id);
        if (suppliers.length === 0) {
            Toast.warning('Primero debe crear al menos un proveedor');
            return;
        }
        const allOrders = await DB.getByIndex('purchaseOrders', 'companyId', company.id);
        const receivedOrders = allOrders.filter(o => o.status === 'received' || o.status
=== 'partial');
        let preloadOrder = null;
        if (preloadOrderId) {
            preloadOrder = await DB.get('purchaseOrders', preloadOrderId);
        }
        const ocOptions = receivedOrders.map(o => {
            const sup = suppliers.find(s => s.id === o.supplierId);
            return `<option value="${o.id}" ${o.id === preloadOrderId ? 'selected' :
''}>${o.number} - ${sup?.name || 'Proveedor'} (${o.status === 'partial' ? 'Parcial' :
'Recibida'})</option>`;
        }).join('');
        Modal.open({
            title: 'Registrar Factura de Proveedor',
            size: 'large',
            content: `
                <form id="supplier-invoice-form">
                    ${receivedOrders.length > 0 ? `
                    <div class="alert alert-info" style="margin-bottom: 1rem;">
                        <i class="fas fa-info-circle"></i>
                        <strong>Secuencia documental:</strong> Seleccione la Orden de
Compra recibida para pre-completar los datos.
                        El asiento de esta factura será: <em>Proveedores-Facturas por
Recibir (D) + IVA CF (D) / Proveedores (C)</em>
                    </div>
                    <div class="form-group" style="margin-bottom: 1rem;">
                        <label class="form-label">Vincular con Orden de Compra recibida
(opcional)</label>
                        <select class="form-control" id="oc-link-select">
                            <option value="">-- Sin vincular (ingreso manual) --</option>
                            ${ocOptions}
                        </select>
                    </div>
                    ` : ''}
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Proveedor <span class="text-
danger">*</span></label>
                            <select class="form-control" name="supplierId" id="invoice-
supplier" required>
                                <option value="">Seleccione proveedor...</option>
                                ${suppliers.map(s => `<option value="${s.id}" ${s.id ===
preloadOrder?.supplierId ? 'selected' : ''}>${s.rut} - ${s.name}</option>`).join('')}
                            </select>
```

```
                        </div>
                        <div class="form-group">
                            <label class="form-label">N° Factura <span class="text-
danger">*</span></label>
                            <input type="text" class="form-control" name="number"
id="invoice-number" placeholder="Ej: 12345" required>
                        </div>
                    </div>
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Fecha Emisión</label>
                            <input type="date" class="form-control" name="date"
value="${Helpers.getCurrentDate()}" required>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Fecha Vencimiento</label>
                            <input type="date" class="form-control" name="dueDate"
value="${Helpers.getDatePlusDays(30)}" required>
                        </div>
                    </div>
                    <h4 style="margin: var(--space-4) 0 var(--space-2);">Detalle</h4>
                    <div class="table-container">
                        <table class="data-table" id="invoice-lines-table">
                            <thead>
                                <tr>
                                    <th style="width: 35%;">Producto/Servicio</th>
                                    <th style="width: 12%;">Cantidad</th>
                                    <th style="width: 15%;">Precio Neto</th>
                                    <th style="width: 15%;">IVA</th>
                                    <th style="width: 15%;">Total</th>
                                    <th style="width: 8%;"></th>
                                </tr>
                            </thead>
                            <tbody id="invoice-lines">
                                <tr class="invoice-line" data-idx="0">
                                    <td>
                                        <select class="form-control product-select"
name="productId">
                                            <option value="">Seleccione...</option>
                                            ${products.map(p => `<option value="${p.id}"
data-cost="${p.cost}">${p.code} - ${p.name}</option>`).join('')}
                                        </select>
                                    </td>
                                    <td><input type="number" class="form-control quantity-
input" name="quantity" value="1" min="1"></td>
                                    <td><input type="number" class="form-control price-
input" name="price" value="0" min="0"></td>
                                    <td class="text-right iva-amount">$0</td>
                                    <td class="text-right line-total">$0</td>
                                    <td><button type="button" class="btn btn-icon btn-
ghost remove-line"><i class="fas fa-times"></i></button></td>
                                </tr>
                            </tbody>
                            <tfoot>
                                <tr>
                                    <td colspan="4" class="text-right">Subtotal Neto:</td>
                                    <td class="text-right" id="invoice-subtotal">$0</td>
                                    <td></td>
                                </tr>
                                <tr>
                                    <td colspan="4" class="text-right">IVA (19%):</td>
                                    <td class="text-right" id="invoice-iva">$0</td>
                                    <td></td>
                                </tr>
                                <tr style="font-weight: 600; font-size: 1.1em;">
                                    <td colspan="4" class="text-right">TOTAL:</td>
                                    <td class="text-right" id="invoice-total">$0</td>
                                    <td></td>
                                </tr>
                            </tfoot>
                        </table>
                    </div>
                    <button type="button" class="btn btn-outline" id="btn-add-invoice-
line" style="margin-top: var(--space-2);">
                        <i class="fas fa-plus"></i> Agregar Línea
                    </button>
                    <input type="hidden" id="linked-order-id" value="${preloadOrderId ||
''}">
                </form>
            `,
            footer: `
```

```
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-outline" id="btn-save-invoice-draft"><i class="fas
fa-save"></i> Guardar Borrador</button>
                <button class="btn btn-primary" id="btn-save-invoice-post"><i class="fas
fa-book"></i> Guardar y Contabilizar</button>
            `
        });
        if (preloadOrder) {
            await this.prefillInvoiceFromOrder(preloadOrder, products);
        }
        document.getElementById('oc-link-select')?.addEventListener('change', async (e) =>
{
            const orderId = e.target.value;
            document.getElementById('linked-order-id').value = orderId;
            if (orderId) {
                const order = await DB.get('purchaseOrders', orderId);
                if (order) {
                    await this.prefillInvoiceFromOrder(order, products);
                }
            }
        });
        this.initInvoiceModalEvents(products);
    },
    async prefillInvoiceFromOrder(order, products) {
        const supplierSelect = document.getElementById('invoice-supplier');
        if (supplierSelect) supplierSelect.value = order.supplierId;
        const orderLines = await DB.getByIndex('purchaseOrderLines', 'orderId', order.id);
        const tbody = document.getElementById('invoice-lines');
        if (!tbody) return;
        tbody.innerHTML = '';
        const IVA_RATE = 0.19;
        orderLines.forEach((line, idx) => {
            const qtyToInvoice = (line.received || line.quantity);
            const tr = document.createElement('tr');
            tr.className = 'invoice-line';
            tr.dataset.idx = idx;
            tr.innerHTML = `
                <td>
                    <select class="form-control product-select" name="productId">
                        <option value="">Seleccione...</option>
                        ${products.map(p => `<option value="${p.id}" data-cost="${p.cost}"
${p.id === line.productId ? 'selected' : ''}>${p.code} - ${p.name}</option>`).join('')}
                    </select>
                </td>
                <td><input type="number" class="form-control quantity-input"
name="quantity" value="${qtyToInvoice}" min="1"></td>
                <td><input type="number" class="form-control price-input" name="price"
value="${line.price || 0}" min="0"></td>
                <td class="text-right iva-amount">$0</td>
                <td class="text-right line-total">$0</td>
                <td><button type="button" class="btn btn-icon btn-ghost remove-line"><i
class="fas fa-times"></i></button></td>
            `;
            tbody.appendChild(tr);
        });
        this.bindInvoiceLineEvents(IVA_RATE);
        this.updateInvoiceTotals(IVA_RATE);
    },
    initInvoiceModalEvents(products) {
        let lineIdx = 1;
        const IVA_RATE = 0.19;
        document.getElementById('btn-add-invoice-line')?.addEventListener('click', () => {
            const tbody = document.getElementById('invoice-lines');
            const newRow = document.createElement('tr');
            newRow.className = 'invoice-line';
            newRow.dataset.idx = lineIdx++;
            newRow.innerHTML = `
    < td >
    <select class="form-control product-select" name="productId">
        <option value="">Seleccione...</option>
        ${products.map(p => `<option value="${p.id}" data-cost="${p.cost}">${p.code} -
${p.name}</option>`).join('')}
    </select>
                </td >
                <td><input type="number" class="form-control quantity-input"
name="quantity" value="1" min="1"></td>
                <td><input type="number" class="form-control price-input" name="price"
value="0" min="0"></td>
                <td class="text-right iva-amount">$0</td>
                <td class="text-right line-total">$0</td>
```

```
                <td><button type="button" class="btn btn-icon btn-ghost remove-line"><i
class="fas fa-times"></i></button></td>
`;
            tbody.appendChild(newRow);
            this.bindInvoiceLineEvents(IVA_RATE);
        });
        this.bindInvoiceLineEvents(IVA_RATE);
        document.getElementById('btn-save-invoice-draft')?.addEventListener('click', async
() => {
            await this.saveSupplierInvoice(false);
        });
        document.getElementById('btn-save-invoice-post')?.addEventListener('click', async
() => {
            await this.saveSupplierInvoice(true);
        });
    },
    bindInvoiceLineEvents(IVA_RATE) {
        document.querySelectorAll('#invoice-lines .product-select').forEach(select => {
            select.onchange = () => {
                const option = select.selectedOptions[0];
                const cost = parseFloat(option?.dataset.cost || 0);
                const row = select.closest('tr');
                row.querySelector('.price-input').value = cost;
                this.updateInvoiceTotals(IVA_RATE);
            };
        });
        document.querySelectorAll('#invoice-lines .quantity-input, #invoice-lines .price-
input').forEach(input => {
            input.oninput = () => this.updateInvoiceTotals(IVA_RATE);
        });
        document.querySelectorAll('#invoice-lines .remove-line').forEach(btn => {
            btn.onclick = () => {
                const lines = document.querySelectorAll('.invoice-line');
                if (lines.length > 1) {
                    btn.closest('tr').remove();
                    this.updateInvoiceTotals(IVA_RATE);
                } else {
                    Toast.warning('Debe haber al menos una línea');
                }
            };
        });
    },
    updateInvoiceTotals(IVA_RATE) {
        let subtotal = 0;
        let totalIva = 0;
        document.querySelectorAll('.invoice-line').forEach(row => {
            const qty = parseFloat(row.querySelector('.quantity-input').value) || 0;
            const price = parseFloat(row.querySelector('.price-input').value) || 0;
            const neto = qty * price;
            const iva = neto * IVA_RATE;
            const total = neto + iva;
            row.querySelector('.iva-amount').textContent = Formatters.currency(iva);
            row.querySelector('.line-total').textContent = Formatters.currency(total);
            subtotal += neto;
            totalIva += iva;
        });
        document.getElementById('invoice-subtotal').textContent =
Formatters.currency(subtotal);
        document.getElementById('invoice-iva').textContent =
Formatters.currency(totalIva);
        document.getElementById('invoice-total').textContent =
Formatters.currency(subtotal + totalIva);
    },
    async saveSupplierInvoice(autoPost) {
        const form = document.getElementById('supplier-invoice-form');
        const formData = new FormData(form);
        const supplierId = formData.get('supplierId');
        const invoiceNumber = formData.get('number');
        if (!supplierId) {
            Toast.error('Seleccione un proveedor');
            return;
        }
        if (!invoiceNumber) {
            Toast.error('Ingrese el número de factura');
            return;
        }
        const IVA_RATE = 0.19;
        const lines = [];
        let subtotal = 0;
        let totalIva = 0;
        document.querySelectorAll('.invoice-line').forEach(row => {
```

```
            const productId = row.querySelector('.product-select').value;
            const quantity = parseFloat(row.querySelector('.quantity-input').value) || 0;
            const price = parseFloat(row.querySelector('.price-input').value) || 0;
            if (quantity > 0 && price > 0) {
                const neto = quantity * price;
                const iva = neto * IVA_RATE;
                lines.push({ productId, quantity, price, neto, iva, total: neto + iva });
                subtotal += neto;
                totalIva += iva;
            }
        });
        if (lines.length === 0) {
            Toast.error('Agregue al menos una línea con cantidad y precio');
            return;
        }
        const company = CompanyService.getCurrent();
        const supplier = await DB.get('suppliers', supplierId);
        const total = subtotal + totalIva;
        const invoice = {
            id: Helpers.generateId(),
            companyId: company.id,
            supplierId,
            number: invoiceNumber,
            date: formData.get('date'),
            dueDate: formData.get('dueDate'),
            subtotal,
            iva: totalIva,
            total,
            status: autoPost ? 'posted' : 'pending',
            createdAt: new Date().toISOString()
        };
        const loading = Toast.loading(autoPost ? 'Guardando y contabilizando...' :
'Guardando factura...');
        try {
            await DB.add('supplierInvoices', invoice);
            for (const line of lines) {
                await DB.add('supplierInvoiceLines', {
                    id: Helpers.generateId(),
                    invoiceId: invoice.id,
                    ...line
                });
            }
            if (autoPost) {
                const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
                let purchasesAccount = accounts.find(a =>
                    a.type === 'expense' && (a.code?.includes('5.1') ||
a.name?.toLowerCase().includes('compra'))
                );
                let ivaCredAccount = accounts.find(a =>
                    a.type === 'asset' && (a.code?.includes('1.1.07') ||
a.name?.toLowerCase().includes('iva') && a.name?.toLowerCase().includes('crédito'))
                );
                let suppliersAccount = accounts.find(a =>
                    a.type === 'liability' && (a.code?.includes('2.1.01') ||
a.name?.toLowerCase().includes('proveedor'))
                );
                if (!purchasesAccount) {
                    purchasesAccount = {
                        id: Helpers.generateId(),
                        companyId: company.id,
                        code: '5.1.01',
                        name: 'Compras',
                        type: 'expense',
                        nature: 'debit',
                        level: 2,
                        isActive: true
                    };
                    await DB.add('accounts', purchasesAccount);
                }
                if (!ivaCredAccount) {
                    ivaCredAccount = {
                        id: Helpers.generateId(),
                        companyId: company.id,
                        code: '1.1.07',
                        name: 'IVA Crédito Fiscal',
                        type: 'asset',
                        nature: 'debit',
                        level: 2,
                        isActive: true
                    };
                    await DB.add('accounts', ivaCredAccount);
```

```
                }
                if (!suppliersAccount) {
                    suppliersAccount = {
                        id: Helpers.generateId(),
                        companyId: company.id,
                        code: '2.1.01',
                        name: 'Proveedores',
                        type: 'liability',
                        nature: 'credit',
                        level: 2,
                        isActive: true
                    };
                    await DB.add('accounts', suppliersAccount);
                }
                const transactionData = {
                    date: invoice.date,
                    description: `Factura ${invoiceNumber} - ${supplier?.name ||
'Proveedor'} `,
                    reference: invoiceNumber,
                    sourceDocument: 'supplierInvoice',
                    sourceDocumentId: invoice.id,
                    autoPost: true,
                    lines: [
                        { accountId: purchasesAccount.id, description: `Compras
Fact.${invoiceNumber} `, debit: subtotal, credit: 0 },
                        { accountId: ivaCredAccount.id, description: `IVA CF
Fact.${invoiceNumber} `, debit: totalIva, credit: 0 },
                        { accountId: suppliersAccount.id, description: `${supplier?.name
|| 'Proveedor'} - Fact.${invoiceNumber} `, debit: 0, credit: total }
                    ]
                };
                await AccountingService.registerTransaction('purchases', transactionData);
                if (supplier) {
                    const newBalance = (supplier.balance || 0) + total;
                    await DB.update('suppliers', { ...supplier, balance: newBalance });
                }
                loading.success('Factura guardada y contabilizada');
            } else {
                loading.success('Factura guardada como borrador');
            }
            Modal.close();
            App.navigate('compras', 'facturas-proveedor');
        } catch (err) {
            loading.error('Error: ' + err.message);
            console.error(err);
        }
    },
    async showInvoiceDetail(invoice) {
        const supplier = await DB.get('suppliers', invoice.supplierId);
        const lines = await DB.getByIndex('supplierInvoiceLines', 'invoiceId',
invoice.id);
        for (let line of lines) {
            if (line.productId) {
                const product = await DB.get('products', line.productId);
                line.productName = product?.name || 'Producto';
                line.productCode = product?.code || '';
            } else {
                line.productName = 'Servicio/Gasto';
                line.productCode = '-';
            }
        }
        const statusLabels = {
            pending: '<span class="badge badge-warning">Pendiente</span>',
            posted: '<span class="badge badge-info">Contabilizada</span>',
            paid: '<span class="badge badge-success">Pagada</span>'
        };
        Modal.open({
            title: `Factura ${invoice.number} `,
            size: 'large',
            content: `
    < div style = "margin-bottom: var(--space-4);" >
        <div style="display: grid; grid-template-columns: repeat(4, 1fr); gap: var(--
space-4);">
            <div>
                <label style="font-size: var(--font-size-xs); color: var(--text-
tertiary);">Proveedor</label>
                <strong>${supplier?.name || 'N/A'}</strong>
            </div>
            <div>
                <label style="font-size: var(--font-size-xs); color: var(--text-
tertiary);">Fecha</label>
```

```
                <strong>${Formatters.date(invoice.date)}</strong>
            </div>
            <div>
                <label style="font-size: var(--font-size-xs); color: var(--text-
tertiary);">Vencimiento</label>
                <strong>${Formatters.date(invoice.dueDate)}</strong>
            </div>
            <div>
                <label style="font-size: var(--font-size-xs); color: var(--text-
tertiary);">Estado</label>
                ${statusLabels[invoice.status] || invoice.status}
            </div>
        </div>
                </div >
    <div class="table-container">
        <table class="data-table">
            <thead>
                <tr>
                    <th>Código</th>
                    <th>Descripción</th>
                    <th class="text-right">Cantidad</th>
                    <th class="text-right">Precio Neto</th>
                    <th class="text-right">IVA</th>
                    <th class="text-right">Total</th>
                </tr>
            </thead>
            <tbody>
                ${lines.map(l => `
                                <tr>
                                    <td><code>${l.productCode}</code></td>
                                    <td>${l.productName}</td>
                                    <td class="text-right">${l.quantity}</td>
                                    <td class="text-right">${Formatters.currency(l.neto ||
l.price * l.quantity)}</td>
                                    <td class="text-right">${Formatters.currency(l.iva ||
0)}</td>
                                    <td class="text-right">${Formatters.currency(l.total
|| l.price * l.quantity)}</td>
                                </tr>
                            `).join('')}
            </tbody>
            <tfoot>
                <tr>
                    <td colspan="5" class="text-right">Subtotal Neto:</td>
                    <td class="text-right">${Formatters.currency(invoice.subtotal || 0)}
</td>
                </tr>
                <tr>
                    <td colspan="5" class="text-right">IVA (19%):</td>
                    <td class="text-right">${Formatters.currency(invoice.iva || 0)}</td>
                </tr>
                <tr style="font-weight: 600; font-size: 1.1em;">
                    <td colspan="5" class="text-right">TOTAL:</td>
                    <td class="text-right">${Formatters.currency(invoice.total)}</td>
                </tr>
            </tfoot>
        </table>
    </div>
`,
            footer: `< button class="btn btn-secondary" onclick = "Modal.close()" >
Cerrar</button > `
        });
    },
    async postSupplierInvoice(invoice) {
        if (invoice.status !== 'pending') {
            Toast.warning('Esta factura ya está contabilizada o pagada');
            return;
        }
        const confirm = await Modal.confirm({
            title: 'Contabilizar Factura',
            message: `¿Contabilizar la factura ${invoice.number}?`,
            confirmText: 'Contabilizar'
        });
        if (!confirm) return;
        const loading = Toast.loading('Contabilizando...');
        try {
            const company = CompanyService.getCurrent();
            const supplier = await DB.get('suppliers', invoice.supplierId);
            const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
            let bridgeAccount = accounts.find(a =>
                a.type === 'liability' && a.name?.toLowerCase().includes('facturas por
```

```
recibir')
            ) || accounts.find(a => a.code === '2.1.05');
            let ivaCredAccount = accounts.find(a =>
                a.type === 'asset' && (a.code?.includes('1.1.07') ||
a.name?.toLowerCase().includes('iva'))
            );
            let suppliersAccount = accounts.find(a =>
                a.type === 'liability' && (a.code?.includes('2.1.01') ||
a.name?.toLowerCase().includes('proveedor'))
            );
            if (!bridgeAccount) {
                bridgeAccount = { id: Helpers.generateId(), companyId: company.id, code:
'2.1.05', name: 'Proveedores - Facturas por Recibir', type: 'liability', nature: 'credit',
level: 2, isActive: true };
                await DB.add('accounts', bridgeAccount);
            }
            if (!ivaCredAccount) {
                ivaCredAccount = { id: Helpers.generateId(), companyId: company.id, code:
'1.1.07', name: 'IVA Crédito Fiscal', type: 'asset', nature: 'debit', level: 2, isActive:
true };
                await DB.add('accounts', ivaCredAccount);
            }
            if (!suppliersAccount) {
                suppliersAccount = { id: Helpers.generateId(), companyId: company.id,
code: '2.1.01', name: 'Proveedores', type: 'liability', nature: 'credit', level: 2,
isActive: true };
                await DB.add('accounts', suppliersAccount);
            }
            const subtotal = invoice.subtotal || (invoice.total / 1.19);
            const iva = invoice.iva || (invoice.total - subtotal);
            const transactionData = {
                date: invoice.date,
                description: `Factura ${invoice.number} - ${supplier?.name || 'Proveedor'}
`,
                reference: invoice.number,
                sourceDocument: 'supplierInvoice',
                sourceDocumentId: invoice.id,
                autoPost: true,
                lines: [
                    { accountId: bridgeAccount.id, description: `Facturas por Recibir
Fact.${invoice.number} `, debit: subtotal, credit: 0 },
                    { accountId: ivaCredAccount.id, description: `IVA CF
Fact.${invoice.number} `, debit: iva, credit: 0 },
                    { accountId: suppliersAccount.id, description: `${supplier?.name} -
Fact.${invoice.number} `, debit: 0, credit: invoice.total }
                ]
            };
            await AccountingService.registerTransaction('purchases', transactionData);
            await DB.update('supplierInvoices', { ...invoice, status: 'posted' });
            if (supplier) {
                await DB.update('suppliers', { ...supplier, balance: (supplier.balance ||
0) + invoice.total });
            }
            loading.success('Factura contabilizada');
            App.navigate('compras', 'facturas-proveedor');
        } catch (err) {
            loading.error('Error: ' + err.message);
        }
    },
    async paySupplierInvoice(invoice) {
        if (invoice.status === 'pending') {
            Toast.warning('Primero debe contabilizar la factura');
            return;
        }
        if (invoice.status === 'paid') {
            Toast.warning('Esta factura ya está pagada');
            return;
        }
        const company = CompanyService.getCurrent();
        const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
        const bankAccounts = accounts.filter(a =>
            a.type === 'asset' && (a.code?.startsWith('1.1.01') ||
a.code?.startsWith('1.1.02') ||
                a.name?.toLowerCase().includes('caja') ||
a.name?.toLowerCase().includes('banco'))
        );
        if (bankAccounts.length === 0) {
            Toast.error('No hay cuentas de Banco/Caja configuradas');
            return;
        }
        const supplier = await DB.get('suppliers', invoice.supplierId);
```

```
        Modal.open({
            title: 'Registrar Pago',
            content: `
    < form id = "payment-form" >
                    <div class="form-group">
                        <label class="form-label">Factura</label>
                        <input type="text" class="form-control" value="${invoice.number} -
${supplier?.name}" disabled>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Monto a Pagar</label>
                        <input type="text" class="form-control"
value="${Formatters.currency(invoice.total)}" disabled>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Cuenta de Pago <span class="text-
danger">*</span></label>
                        <select class="form-control" name="accountId" required>
                            ${bankAccounts.map(a => `<option value="${a.id}">${a.code} -
${a.name}</option>`).join('')}
                        </select>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Fecha de Pago</label>
                        <input type="date" class="form-control" name="paymentDate"
value="${Helpers.getCurrentDate()}" required>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Referencia (N° Cheque/Transferencia)
</label>
                        <input type="text" class="form-control" name="reference"
placeholder="Opcional">
                    </div>
                </form >
    `,
            footer: `
    < button class="btn btn-secondary" onclick = "Modal.close()" > Cancelar</button >
        <button class="btn btn-success" id="btn-confirm-payment"><i class="fas fa-check">
</i> Confirmar Pago</button>
`
        });
        document.getElementById('btn-confirm-payment')?.addEventListener('click', async ()
=> {
            const form = document.getElementById('payment-form');
            const formData = new FormData(form);
            const paymentAccountId = formData.get('accountId');
            const paymentDate = formData.get('paymentDate');
            const reference = formData.get('reference');
            const loading = Toast.loading('Registrando pago...');
            try {
                let suppliersAccount = accounts.find(a =>
                    a.type === 'liability' && (a.code?.includes('2.1.01') ||
a.name?.toLowerCase().includes('proveedor'))
                );
                if (!suppliersAccount) {
                    suppliersAccount = { id: Helpers.generateId(), companyId: company.id,
code: '2.1.01', name: 'Proveedores', type: 'liability', nature: 'credit', level: 2,
isActive: true };
                    await DB.add('accounts', suppliersAccount);
                }
                const transactionData = {
                    date: paymentDate,
                    description: `Pago Factura ${invoice.number} - ${supplier?.name ||
'Proveedor'} `,
                    reference: reference || `PAG - ${invoice.number} `,
                    sourceDocument: 'supplierPayment',
                    sourceDocumentId: invoice.id,
                    autoPost: true,
                    lines: [
                        { accountId: suppliersAccount.id, description: `Pago
Fact.${invoice.number} `, debit: invoice.total, credit: 0 },
                        { accountId: paymentAccountId, description: `Pago a
${supplier?.name || 'Proveedor'} `, debit: 0, credit: invoice.total }
                    ]
                };
                await AccountingService.registerTransaction('treasury', transactionData);
                await DB.update('supplierInvoices', {
                    ...invoice,
                    status: 'paid',
                    paidAt: new Date().toISOString(),
                    paymentReference: reference
```

```
                });
                if (supplier) {
                    const newBalance = (supplier.balance || 0) - invoice.total;
                    await DB.update('suppliers', { ...supplier, balance: newBalance });
                }
                Modal.close();
                loading.success(`Pago registrado y contabilizado`);
                App.navigate('compras', 'facturas-proveedor');
            } catch (err) {
                loading.error('Error: ' + err.message);
                console.error(err);
            }
        });
    },
    async cancelPurchaseOrder(order) {
        if (order.status === 'cancelled') {
            Toast.warning('Esta orden ya está anulada');
            return;
        }
        const wasReceived = order.status === 'received';
        let message = `¿Está seguro de anular la orden ${order.number}?`;
        if (wasReceived) {
            message += `\n\n⚠️ Esta orden ya fue recibida.Al anularla: \n• Se reducirá el
stock(devolución) \n• Se anularán los asientos contables\n• Se anulará la factura de
proveedor asociada`;
        }
        const confirm = await Modal.confirm({
            title: 'Anular Orden de Compra',
            message: message,
            confirmText: 'Anular Orden',
            confirmClass: 'btn-danger'
        });
        if (!confirm) return;
        const loading = Toast.loading('Anulando orden...');
        try {
            const company = CompanyService.getCurrent();
            if (wasReceived) {
                const lines = await DB.getByIndex('purchaseOrderLines', 'orderId',
order.id);
                for (const line of lines) {
                    const product = await DB.get('products', line.productId);
                    if (product && !product.isService) {
                        const newStock = Math.max(0, (product.stock || 0) -
line.quantity);
                        await DB.update('products', {
                            ...product,
                            stock: newStock
                        });
                        await DB.add('stockMovements', {
                            id: Helpers.generateId(),
                            companyId: company.id,
                            productId: line.productId,
                            type: 'out',
                            quantity: line.quantity,
                            reference: `ANULACIÓN ${order.number} `,
                            date: Helpers.getCurrentDate(),
                            notes: `Reversión por anulación de orden de compra
${order.number} `,
                            createdAt: new Date().toISOString()
                        });
                    }
                }
                const entries = await DB.getByIndex('journalEntries', 'companyId',
company.id);
                const relatedEntries = entries.filter(e => e.reference === order.number);
                for (const entry of relatedEntries) {
                    const originalLines = await DB.getByIndex('journalLines', 'entryId',
entry.id);
                    const reversalNumber = await AccountingService.getNextEntryNumber();
                    const reversalEntry = {
                        id: Helpers.generateId(),
                        companyId: company.id,
                        number: reversalNumber,
                        date: Helpers.getCurrentDate(),
                        description: `ANULACIÓN: ${entry.description} `,
                        reference: `ANUL - ${entry.reference} `,
                        status: 'posted',
                        totalDebit: entry.totalCredit,
                        totalCredit: entry.totalDebit,
                        createdAt: new Date().toISOString()
                    };
```

```
                    await DB.add('journalEntries', reversalEntry);
                    for (const line of originalLines) {
                        await DB.add('journalLines', {
                            id: Helpers.generateId(),
                            entryId: reversalEntry.id,
                            accountId: line.accountId,
                            description: `Reversión: ${line.description} `,
                            debit: line.credit,
                            credit: line.debit
                        });
                    }
                    await DB.update('journalEntries', { ...entry, status: 'cancelled' });
                }
                const invoices = await DB.getByIndex('supplierInvoices', 'companyId',
company.id);
                const relatedInvoice = invoices.find(i => i.purchaseOrderId === order.id);
                if (relatedInvoice) {
                    await DB.update('supplierInvoices', { ...relatedInvoice, status:
'cancelled' });
                    const supplier = await DB.get('suppliers', order.supplierId);
                    if (supplier) {
                        await DB.update('suppliers', {
                            ...supplier,
                            balance: Math.max(0, (supplier.balance || 0) -
relatedInvoice.total)
                        });
                    }
                }
            }
            await DB.update('purchaseOrders', {
                ...order,
                status: 'cancelled',
                cancelledAt: new Date().toISOString()
            });
            loading.success(`Orden ${order.number} anulada`);
            App.navigate('compras', 'ordenes');
        } catch (err) {
            loading.error('Error al anular: ' + err.message);
            console.error(err);
        }
    }
};
window.ComprasModule = ComprasModule;
```

Archivo: js\modules\contabilidad.js

```
const ContabilidadModule = {
    currentView: 'plan-cuentas',
    table: null,
    async render(view = 'plan-cuentas') {
        this.currentView = view;
        switch (view) {
            case 'plan-cuentas':
                return await this.renderChartOfAccounts();
            case 'asientos':
                return await this.renderJournalEntries();
            case 'libro-mayor':
                return await this.renderLedger();
            case 'libro-diario':
                return await this.renderJournal();
            case 'libros-auxiliares':
                return await this.renderAuxiliaryJournals();
            default:
                return '<p>Vista no encontrada</p>';
        }
    },
    async renderChartOfAccounts() {
        const accounts = await AccountingService.getChartOfAccounts();
        return `
            <div class="page-header">
                <h1 class="page-title">Plan de Cuentas</h1>
                <p class="page-subtitle">Estructura contable según normativa IFRS
chilena</p>
            </div>
            <div class="toolbar">
```

```
                <div class="toolbar-left">
                    <div class="search-box">
                        <i class="fas fa-search"></i>
                        <input type="text" class="form-control" placeholder="Buscar
cuenta..." id="search-accounts">
                    </div>
                    <select class="form-control" id="filter-type" style="width: 150px;">
                        <option value="">Todos los tipos</option>
                        <option value="asset">Activos</option>
                        <option value="liability">Pasivos</option>
                        <option value="equity">Patrimonio</option>
                        <option value="revenue">Ingresos</option>
                        <option value="expense">Gastos</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-recalculate-balances"
title="Recalcula los saldos de todas las cuentas basándose en los asientos contables">
                        <i class="fas fa-sync-alt"></i> Recalcular Saldos
                    </button>
                    <button class="btn btn-outline" id="btn-expand-all">
                        <i class="fas fa-expand-alt"></i> Expandir
                    </button>
                    <button class="btn btn-primary" id="btn-new-account">
                        <i class="fas fa-plus"></i> Nueva Cuenta
                    </button>
                </div>
            </div>
            <div class="card">
                <div class="card-body" style="padding: 0;">
                    <div id="accounts-tree" class="accounts-tree">
                        ${this.renderAccountsTree(accounts)}
                    </div>
                </div>
            </div>
        `;
    },
    renderAccountsTree(accounts, parentId = null, level = 0) {
        const children = accounts.filter(a => a.parentId === parentId);
        if (children.length === 0) return '';
        const typeLabels = {
            asset: 'Activo', liability: 'Pasivo', equity: 'Patrimonio',
            revenue: 'Ingreso', expense: 'Gasto'
        };
        const typeColors = {
            asset: 'primary', liability: 'error', equity: 'success',
            revenue: 'success', expense: 'warning'
        };
        return children.map(account => {
            const hasChildren = accounts.some(a => a.parentId === account.id);
            const indent = level * 24;
            return `
                <div class="account-row level-${level}" data-account-id="${account.id}"
style="padding-left: ${indent + 16}px;">
                    <div class="account-info">
                        ${hasChildren ? `
                            <button class="tree-toggle" data-toggle="${account.id}">
                                <i class="fas fa-chevron-right"></i>
                            </button>
                        ` : '<span style="width: 24px;"></span>'}
                        <span class="account-code">${account.code}</span>
                        <span class="account-name ${account.isGroup ? 'is-group' :
''}">${account.name}</span>
                        ${account.isContra ? '<span class="badge badge-
neutral">Contra</span>' : ''}
                    </div>
                    <div class="account-meta">
                        <span class="badge
badge-${typeColors[account.type]}">${typeLabels[account.type]}</span>
                        ${!account.isGroup ? `
                            <span class="account-balance ${account.balance >= 0 ?
'positive' : 'negative'}">
                                ${Formatters.currency(Math.abs(account.balance))}
                            </span>
                        ` : ''}
                        <div class="account-actions">
                            <button class="btn btn-icon btn-ghost" title="Ver movimientos"
data-action="ledger" data-id="${account.id}">
                                <i class="fas fa-list"></i>
                            </button>
                            <button class="btn btn-icon btn-ghost" title="Editar" data-
```

```
action="edit" data-id="${account.id}">
                                <i class="fas fa-edit"></i>
                            </button>
                        </div>
                    </div>
                </div>
                <div class="account-children" data-parent="${account.id}" style="display:
none;">
                    ${this.renderAccountsTree(accounts, account.id, level + 1)}
                </div>
            `;
        }).join('');
    },
    async renderJournalEntries() {
        return `
            <div class="page-header">
                <h1 class="page-title">Asientos Contables</h1>
                <p class="page-subtitle">Registro de operaciones contables</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="filter-date-from"
style="width: 150px;" placeholder="Desde">
                    <input type="date" class="form-control" id="filter-date-to"
style="width: 150px;" placeholder="Hasta">
                    <select class="form-control" id="filter-status" style="width: 150px;">
                        <option value="">Todos los estados</option>
                        <option value="draft">Borrador</option>
                        <option value="posted">Contabilizado</option>
                        <option value="cancelled">Anulado</option>
                    </select>
                    <button class="btn btn-ghost" id="btn-clear-filters" title="Limpiar
filtros">
                        <i class="fas fa-times"></i> Limpiar
                    </button>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-export-entries">
                        <i class="fas fa-download"></i> Exportar
                    </button>
                    <button class="btn btn-primary" id="btn-new-entry">
                        <i class="fas fa-plus"></i> Nuevo Asiento
                    </button>
                </div>
            </div>
            <div id="entries-table"></div>
        `;
    },
    async renderLedger() {
        const accounts = await AccountingService.getMovementAccounts();
        return `
            <div class="page-header">
                <h1 class="page-title">Libro Mayor</h1>
                <p class="page-subtitle">Movimientos por cuenta contable</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <select class="form-control" id="ledger-account" style="width:
300px;">
                        <option value="">Seleccione una cuenta...</option>
                        ${accounts.map(a => `<option value="${a.id}">${a.code} - ${a.name}
</option>`).join('')}
                    </select>
                    <input type="date" class="form-control" id="ledger-from" style="width:
150px;">
                    <input type="date" class="form-control" id="ledger-to" style="width:
150px;">
                    <button class="btn btn-primary" id="btn-view-ledger">
                        <i class="fas fa-search"></i> Consultar
                    </button>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-export-ledger">
                        <i class="fas fa-download"></i> Exportar
                    </button>
                </div>
            </div>
            <div id="ledger-content">
                <div class="empty-state">
                    <i class="fas fa-book"></i>
                    <h3>Seleccione una cuenta</h3>
```

```
                    <p>Elija una cuenta contable para ver sus movimientos</p>
                </div>
            </div>
        `;
    },
    async renderJournal() {
        return `
            <div class="page-header">
                <h1 class="page-title">Libro Diario</h1>
                <p class="page-subtitle">Registro cronológico de transacciones</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="journal-from"
style="width: 150px;">
                    <input type="date" class="form-control" id="journal-to" style="width:
150px;">
                    <button class="btn btn-primary" id="btn-view-journal">
                        <i class="fas fa-search"></i> Consultar
                    </button>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-export-journal-excel">
                        <i class="fas fa-file-excel"></i> Excel
                    </button>
                    <button class="btn btn-outline" id="btn-export-journal-pdf">
                        <i class="fas fa-file-pdf"></i> PDF
                    </button>
                    <button class="btn btn-outline" id="btn-print-journal">
                        <i class="fas fa-print"></i> Imprimir
                    </button>
                </div>
            </div>
            <div id="journal-content"></div>
        `;
    },
    async init(view) {
        switch (view) {
            case 'plan-cuentas':
                this.initChartOfAccounts();
                break;
            case 'asientos':
                await this.initJournalEntries();
                break;
            case 'libro-mayor':
                this.initLedger();
                break;
            case 'libro-diario':
                this.initJournal();
                break;
            case 'libros-auxiliares':
                await this.initAuxiliaryJournals();
                break;
        }
    },
    initChartOfAccounts() {
        document.querySelectorAll('.tree-toggle').forEach(btn => {
            btn.onclick = () => {
                const id = btn.dataset.toggle;
                const children = document.querySelector(`[data-parent="${id}"]`);
                const icon = btn.querySelector('i');
                if (children.style.display === 'none') {
                    children.style.display = 'block';
                    icon.classList.replace('fa-chevron-right', 'fa-chevron-down');
                } else {
                    children.style.display = 'none';
                    icon.classList.replace('fa-chevron-down', 'fa-chevron-right');
                }
            };
        });
        document.getElementById('btn-expand-all')?.addEventListener('click', () => {
            document.querySelectorAll('.account-children').forEach(el => el.style.display
= 'block');
            document.querySelectorAll('.tree-toggle i').forEach(i =>
i.classList.replace('fa-chevron-right', 'fa-chevron-down'));
        });
        document.getElementById('search-accounts')?.addEventListener('input',
Helpers.debounce((e) => {
            const term = e.target.value.toLowerCase();
            document.querySelectorAll('.account-row').forEach(row => {
                const text = row.textContent.toLowerCase();
```

```
                row.style.display = text.includes(term) ? 'flex' : 'none';
                if (text.includes(term)) {
                    let parent = row.parentElement;
                    while (parent && parent.classList.contains('account-children')) {
                        parent.style.display = 'block';
                        parent = parent.parentElement;
                    }
                }
            });
        }, 300));
        document.getElementById('filter-type')?.addEventListener('change', async (e) => {
            const type = e.target.value;
            if (!type) {
                document.querySelectorAll('.account-row').forEach(row => row.style.display
= 'flex');
                document.querySelectorAll('.account-children').forEach(el =>
el.style.display = 'none');
            } else {
                const typeMap = {
                    'asset': 'Activo', 'liability': 'Pasivo', 'equity': 'Patrimonio',
                    'revenue': 'Ingreso', 'expense': 'Gasto'
                };
                const typeLabel = typeMap[type];
                document.querySelectorAll('.account-row').forEach(row => {
                    const badge = row.querySelector('.badge');
                    const matchesType = badge && badge.textContent === typeLabel;
                    row.style.display = matchesType ? 'flex' : 'none';
                    if (matchesType) {
                        let parent = row.parentElement;
                        while (parent && parent.classList.contains('account-children')) {
                            parent.style.display = 'block';
                            parent = parent.parentElement;
                        }
                    }
                });
            }
        });
        document.getElementById('btn-new-account')?.addEventListener('click', () => {
            this.showAccountModal();
        });
        document.getElementById('btn-recalculate-balances')?.addEventListener('click',
async () => {
            const confirm = await Modal.confirm({
                title: 'Recalcular Saldos de Cuentas',
                message: `¿Desea recalcular todos los saldos de las cuentas contables?
Esto procesará todos los asientos contabilizados y actualizará los saldos de las cuentas
según los movimientos registrados.
<strong>Útil cuando:</strong>
• Se importaron datos históricos
• Se detectan inconsistencias en los saldos
• Se realizaron correcciones directas en la base de datos`,
                confirmText: 'Recalcular',
                confirmClass: 'btn-primary'
            });
            if (!confirm) return;
            const loading = Toast.loading('Recalculando saldos...');
            try {
                const result = await AccountingService.recalculateAccountBalances();
                if (result.linesProcessed === 0) {
                    loading.error(`No se procesaron líneas de asientos. Asientos
encontrados: ${result.entriesProcessed}. Ver consola (F12) para más detalles.`);
                } else {
                    loading.success(`Saldos recalculados: ${result.accountsUpdated}
cuentas, ${result.entriesProcessed} asientos, ${result.linesProcessed} líneas`);
                }
                App.navigate('contabilidad', 'plan-cuentas');
            } catch (err) {
                loading.error('Error: ' + err.message);
            }
        });
        document.querySelectorAll('[data-action]').forEach(btn => {
            btn.onclick = () => {
                const action = btn.dataset.action;
                const id = btn.dataset.id;
                if (action === 'edit') this.showAccountModal(id);
                if (action === 'ledger') App.navigate('contabilidad', 'libro-mayor');
            };
        });
    },
    async initJournalEntries() {
        const entries = await AccountingService.getJournalEntries();
```

```
        for (let entry of entries) {
            entry.lines = await AccountingService.getJournalLines(entry.id);
        }
        this.allEntries = entries;
        this.table = DataTable.create('entries-table', {
            columns: [
                { key: 'number', label: 'Número', sortable: true },
                { key: 'date', label: 'Fecha', format: 'date', sortable: true },
                { key: 'description', label: 'Descripción' },
                { key: 'totalDebit', label: 'Debe', format: 'currency', class: 'text-
right' },
                { key: 'totalCredit', label: 'Haber', format: 'currency', class: 'text-
right' },
                { key: 'status', label: 'Estado', format: 'status' }
            ],
            data: entries,
            actions: [
                { name: 'view', label: 'Ver detalle', icon: 'fas fa-eye', handler: (row)
=> this.showEntryDetail(row) },
                { name: 'post', label: 'Contabilizar', icon: 'fas fa-check', handler:
(row) => this.postEntry(row) },
                { name: 'cancel', label: 'Anular', icon: 'fas fa-times', handler: (row) =>
this.cancelEntry(row) }
            ],
            emptyMessage: 'No hay asientos contables',
            onRowClick: (row) => this.showEntryDetail(row)
        });
        const applyFilters = () => {
            const dateFrom = document.getElementById('filter-date-from')?.value;
            const dateTo = document.getElementById('filter-date-to')?.value;
            const status = document.getElementById('filter-status')?.value;
            let filtered = [...this.allEntries];
            if (dateFrom) {
                filtered = filtered.filter(e => e.date >= dateFrom);
            }
            if (dateTo) {
                filtered = filtered.filter(e => e.date <= dateTo);
            }
            if (status) {
                filtered = filtered.filter(e => e.status === status);
            }
            if (this.table && this.table.refresh) {
                this.table.refresh(filtered);
            } else {
                DataTable.setData('entries-table', filtered);
            }
        };
        document.getElementById('filter-date-from')?.addEventListener('change',
applyFilters);
        document.getElementById('filter-date-to')?.addEventListener('change',
applyFilters);
        document.getElementById('filter-status')?.addEventListener('change',
applyFilters);
        document.getElementById('btn-clear-filters')?.addEventListener('click', () => {
            document.getElementById('filter-date-from').value = '';
            document.getElementById('filter-date-to').value = '';
            document.getElementById('filter-status').value = '';
            applyFilters();
        });
        document.getElementById('btn-new-entry')?.addEventListener('click', () => {
            this.showEntryModal();
        });
        document.getElementById('btn-export-entries')?.addEventListener('click', async ()
=> {
            const loading = Toast.loading('Generando Excel con detalle...');
            try {
                const exportRows = [];
                for (const entry of entries) {
                    let lines = entry.lines;
                    if (!lines || lines.length === 0) {
                        lines = await AccountingService.getJournalLines(entry.id);
                    }
                    exportRows.push({
                        'Asiento': entry.number,
                        'Fecha': entry.date,
                        'Referencia': entry.reference || '',
                        'Estado': entry.status === 'posted' ? 'Contabilizado' :
                            entry.status === 'draft' ? 'Borrador' : 'Anulado',
                        'Código': '',
                        'Cuenta': '',
                        'Descripción': entry.description,
```

```
                        'Debe': '',
                        'Haber': ''
                    });
                    for (const line of lines) {
                        const account = await
AccountingService.getAccount(line.accountId);
                        exportRows.push({
                            'Asiento': '',
                            'Fecha': '',
                            'Referencia': '',
                            'Estado': '',
                            'Código': account?.code || '',
                            'Cuenta': account?.name || '',
                            'Descripción': line.description || '',
                            'Debe': line.debit > 0 ? line.debit : '',
                            'Haber': line.credit > 0 ? line.credit : ''
                        });
                    }
                    const totalDebit = lines.reduce((sum, l) => sum + (l.debit || 0), 0);
                    const totalCredit = lines.reduce((sum, l) => sum + (l.credit || 0),
0);
                    exportRows.push({
                        'Asiento': '',
                        'Fecha': '',
                        'Referencia': '',
                        'Estado': '',
                        'Código': '',
                        'Cuenta': '',
                        'Descripción': 'TOTALES',
                        'Debe': totalDebit,
                        'Haber': totalCredit
                    });
                    exportRows.push({
                        'Asiento': '', 'Fecha': '', 'Referencia': '', 'Estado': '',
                        'Código': '', 'Cuenta': '', 'Descripción': '', 'Debe': '',
'Haber': ''
                    });
                }
                const wb = XLSX.utils.book_new();
                const ws = XLSX.utils.json_to_sheet(exportRows);
                ws['!cols'] = [
                    { wch: 15 },
                    { wch: 12 },
                    { wch: 15 },
                    { wch: 12 },
                    { wch: 10 },
                    { wch: 25 },
                    { wch: 40 },
                    { wch: 15 },
                    { wch: 15 }
                ];
                XLSX.utils.book_append_sheet(wb, ws, 'Libro Diario');
                XLSX.writeFile(wb, `libro_diario_${Helpers.getCurrentDate()}.xlsx`);
                loading.success('Excel exportado correctamente');
            } catch (err) {
                loading.error('Error: ' + err.message);
            }
        });
    },
    initLedger() {
        this.currentLedgerData = null;
        document.getElementById('btn-view-ledger')?.addEventListener('click', async () =>
{
            const accountId = document.getElementById('ledger-account').value;
            if (!accountId) {
                Toast.warning('Seleccione una cuenta');
                return;
            }
            const dateFrom = document.getElementById('ledger-from').value;
            const dateTo = document.getElementById('ledger-to').value;
            const movements = await AccountingService.getLedger(accountId, dateFrom,
dateTo);
            const account = await AccountingService.getAccount(accountId);
            this.currentLedgerData = { account, movements, dateFrom, dateTo };
            this.renderLedgerContent(account, movements);
        });
        document.getElementById('btn-export-ledger')?.addEventListener('click', async ()
=> {
            if (!this.currentLedgerData) {
                Toast.warning('Primero consulte una cuenta');
                return;
```

```
            }
            const { account, movements, dateFrom, dateTo } = this.currentLedgerData;
            const loading = Toast.loading('Generando Excel...');
            try {
                let balance = 0;
                const isDebitNature = ['asset', 'expense'].includes(account.type);
                const exportRows = [];
                exportRows.push({
                    'Fecha': 'LIBRO MAYOR',
                    'Asiento': '',
                    'Descripción': account.code + ' - ' + account.name,
                    'Debe': '',
                    'Haber': '',
                    'Saldo': ''
                });
                if (dateFrom || dateTo) {
                    exportRows.push({
                        'Fecha': 'Período:',
                        'Asiento': '',
                        'Descripción': `${dateFrom || 'Inicio'} al ${dateTo || 'Hoy'}`,
                        'Debe': '',
                        'Haber': '',
                        'Saldo': ''
                    });
                }
                exportRows.push({ 'Fecha': '', 'Asiento': '', 'Descripción': '', 'Debe':
'', 'Haber': '', 'Saldo': '' });
                exportRows.push({
                    'Fecha': 'Fecha',
                    'Asiento': 'Asiento',
                    'Descripción': 'Descripción',
                    'Debe': 'Debe',
                    'Haber': 'Haber',
                    'Saldo': 'Saldo'
                });
                for (const mov of movements) {
                    if (isDebitNature) {
                        balance += (mov.debit || 0) - (mov.credit || 0);
                    } else {
                        balance += (mov.credit || 0) - (mov.debit || 0);
                    }
                    exportRows.push({
                        'Fecha': mov.date,
                        'Asiento': mov.entryNumber || '',
                        'Descripción': mov.description || '',
                        'Debe': mov.debit > 0 ? mov.debit : '',
                        'Haber': mov.credit > 0 ? mov.credit : '',
                        'Saldo': balance
                    });
                }
                const totalDebit = movements.reduce((sum, m) => sum + (m.debit || 0), 0);
                const totalCredit = movements.reduce((sum, m) => sum + (m.credit || 0),
0);
                exportRows.push({ 'Fecha': '', 'Asiento': '', 'Descripción': '', 'Debe':
'', 'Haber': '', 'Saldo': '' });
                exportRows.push({
                    'Fecha': '',
                    'Asiento': '',
                    'Descripción': 'TOTALES',
                    'Debe': totalDebit,
                    'Haber': totalCredit,
                    'Saldo': balance
                });
                const wb = XLSX.utils.book_new();
                const ws = XLSX.utils.json_to_sheet(exportRows, { skipHeader: true });
                ws['!cols'] = [
                    { wch: 12 },
                    { wch: 15 },
                    { wch: 40 },
                    { wch: 15 },
                    { wch: 15 },
                    { wch: 15 }
                ];
                XLSX.utils.book_append_sheet(wb, ws, 'Libro Mayor');
                XLSX.writeFile(wb,
`libro_mayor_${account.code}_${Helpers.getCurrentDate()}.xlsx`);
                loading.success('Excel exportado correctamente');
            } catch (err) {
                loading.error('Error: ' + err.message);
            }
        });
```

```
    },
    renderLedgerContent(account, movements) {
        let balance = 0;
        const isDebitNature = ['asset', 'expense'].includes(account.type);
        const content = document.getElementById('ledger-content');
        content.innerHTML = `
            <div class="card">
                <div class="card-header">
                    <div class="card-title">
                        <i class="fas fa-list"></i>
                        ${account.code} - ${account.name}
                    </div>
                    <span class="badge badge-primary">${movements.length}
movimientos</span>
                </div>
                <div class="card-body" style="padding: 0;">
                    <div class="table-container">
                        <table class="data-table">
                            <thead>
                                <tr>
                                    <th>Fecha</th>
                                    <th>Asiento</th>
                                    <th>Descripción</th>
                                    <th class="text-right">Debe</th>
                                    <th class="text-right">Haber</th>
                                    <th class="text-right">Saldo</th>
                                </tr>
                            </thead>
                            <tbody>
                                ${movements.length === 0 ? `
                                    <tr><td colspan="6" class="table-empty">Sin
movimientos en el período</td></tr>
                                ` : movements.map(m => {
            balance += isDebitNature ? (m.debit - m.credit) : (m.credit - m.debit);
            return `
                                        <tr>
                                            <td>${Formatters.date(m.date)}</td>
                                            <td>${m.entryNumber}</td>
                                            <td>${m.description}</td>
                                            <td class="text-right">${m.debit > 0 ?
Formatters.currency(m.debit) : '-'}</td>
                                            <td class="text-right">${m.credit > 0 ?
Formatters.currency(m.credit) : '-'}</td>
                                            <td class="text-right ${balance >= 0 ?
'positive' : 'negative'}">${Formatters.currency(balance)}</td>
                                        </tr>
                                    `;
        }).join('')}
                            </tbody>
                            <tfoot>
                                <tr style="font-weight: 600; background: var(--neutral-
100);">
                                    <td colspan="3">TOTALES</td>
                                    <td class="text-
right">${Formatters.currency(Helpers.sumBy(movements, 'debit'))}</td>
                                    <td class="text-
right">${Formatters.currency(Helpers.sumBy(movements, 'credit'))}</td>
                                    <td class="text-right">${Formatters.currency(balance)}
</td>
                                </tr>
                            </tfoot>
                        </table>
                    </div>
                </div>
            </div>
        `;
    },
    initJournal() {
        this.currentJournalData = null;
        document.getElementById('btn-view-journal')?.addEventListener('click', async () =>
{
            const dateFrom = document.getElementById('journal-from').value;
            const dateTo = document.getElementById('journal-to').value;
            const entries = await AccountingService.getJournalEntries({
                status: 'posted',
                dateFrom,
                dateTo
            });
            for (let entry of entries) {
                entry.lines = await AccountingService.getJournalLines(entry.id);
                for (let line of entry.lines) {
```

```
                    const account = await AccountingService.getAccount(line.accountId);
                    line.accountCode = account?.code || '';
                    line.accountName = account?.name || '';
                }
            }
            this.currentJournalData = { entries, dateFrom, dateTo };
            this.renderJournalContent(entries);
        });
        document.getElementById('btn-export-journal-excel')?.addEventListener('click',
async () => {
            if (!this.currentJournalData) {
                Toast.warning('Primero consulte el libro diario');
                return;
            }
            const { entries, dateFrom, dateTo } = this.currentJournalData;
            const loading = Toast.loading('Generando Excel...');
            try {
                const exportRows = [];
                for (const entry of entries) {
                    exportRows.push({
                        'Fecha': entry.date,
                        'Asiento': entry.number,
                        'Referencia': entry.reference || '',
                        'Código': '',
                        'Cuenta': '',
                        'Descripción': entry.description,
                        'Debe': '',
                        'Haber': ''
                    });
                    for (const line of entry.lines || []) {
                        exportRows.push({
                            'Fecha': '',
                            'Asiento': '',
                            'Referencia': '',
                            'Código': line.accountCode,
                            'Cuenta': line.accountName,
                            'Descripción': line.description || '',
                            'Debe': line.debit > 0 ? line.debit : '',
                            'Haber': line.credit > 0 ? line.credit : ''
                        });
                    }
                    const totalDebit = (entry.lines || []).reduce((s, l) => s + (l.debit
|| 0), 0);
                    const totalCredit = (entry.lines || []).reduce((s, l) => s + (l.credit
|| 0), 0);
                    exportRows.push({
                        'Fecha': '', 'Asiento': '', 'Referencia': '', 'Código': '',
'Cuenta': '',
                        'Descripción': 'TOTALES',
                        'Debe': totalDebit,
                        'Haber': totalCredit
                    });
                    exportRows.push({ 'Fecha': '', 'Asiento': '', 'Referencia': '',
'Código': '', 'Cuenta': '', 'Descripción': '', 'Debe': '', 'Haber': '' });
                }
                const wb = XLSX.utils.book_new();
                const ws = XLSX.utils.json_to_sheet(exportRows);
                ws['!cols'] = [
                    { wch: 12 }, { wch: 15 }, { wch: 15 }, { wch: 10 },
                    { wch: 25 }, { wch: 40 }, { wch: 15 }, { wch: 15 }
                ];
                XLSX.utils.book_append_sheet(wb, ws, 'Libro Diario');
                XLSX.writeFile(wb, `libro_diario_${Helpers.getCurrentDate()}.xlsx`);
                loading.success('Excel exportado correctamente');
            } catch (err) {
                loading.error('Error: ' + err.message);
            }
        });
        document.getElementById('btn-export-journal-pdf')?.addEventListener('click', async
() => {
            if (!this.currentJournalData) {
                Toast.warning('Primero consulte el libro diario');
                return;
            }
            const { entries, dateFrom, dateTo } = this.currentJournalData;
            const loading = Toast.loading('Generando PDF...');
            try {
                const { jsPDF } = window.jspdf;
                const doc = new jsPDF('landscape', 'mm', 'letter');
                doc.setFontSize(16);
                doc.text('LIBRO DIARIO', 140, 15, { align: 'center' });
```

```
                doc.setFontSize(10);
                const company = CompanyService.getCurrent();
                doc.text(company?.name || '', 140, 22, { align: 'center' });
                if (dateFrom || dateTo) {
                    doc.text(`Período: ${dateFrom || 'Inicio'} al ${dateTo || 'Hoy'}`,
140, 28, { align: 'center' });
                }
                const tableData = [];
                for (const entry of entries) {
                    tableData.push([
                        entry.date, entry.number, entry.reference || '', '', '',
entry.description, '', ''
                    ]);
                    for (const line of entry.lines || []) {
                        tableData.push([
                            '', '', '', line.accountCode, line.accountName,
line.description || '',
                            line.debit > 0 ? Formatters.number(line.debit, 0) : '',
                            line.credit > 0 ? Formatters.number(line.credit, 0) : ''
                        ]);
                    }
                    const totalDebit = (entry.lines || []).reduce((s, l) => s + (l.debit
|| 0), 0);
                    const totalCredit = (entry.lines || []).reduce((s, l) => s + (l.credit
|| 0), 0);
                    tableData.push(['', '', '', '', '', 'TOTALES',
Formatters.number(totalDebit, 0), Formatters.number(totalCredit, 0)]);
                    tableData.push(['', '', '', '', '', '', '', '']);
                }
                doc.autoTable({
                    head: [['Fecha', 'Asiento', 'Ref.', 'Código', 'Cuenta', 'Descripción',
'Debe', 'Haber']],
                    body: tableData,
                    startY: 35,
                    styles: { fontSize: 8 },
                    headStyles: { fillColor: [66, 139, 202] }
                });
                doc.save(`libro_diario_${Helpers.getCurrentDate()}.pdf`);
                loading.success('PDF generado correctamente');
            } catch (err) {
                loading.error('Error: ' + err.message);
            }
        });
        document.getElementById('btn-print-journal')?.addEventListener('click', () => {
            if (!this.currentJournalData) {
                Toast.warning('Primero consulte el libro diario');
                return;
            }
            const content = document.getElementById('journal-content');
            if (!content) return;
            const printWindow = window.open('', '_blank');
            printWindow.document.write(`
                <html>
                <head>
                    <title>Libro Diario</title>
                    <style>
                        body { font-family: Arial, sans-serif; padding: 20px; }
                        table { width: 100%; border-collapse: collapse; margin-top: 20px;
}
                        th, td { border: 1px solid #ccc; padding: 8px; text-align: left; }
                        th { background: #f0f0f0; }
                        .text-right { text-align: right; }
                        h1, h2 { text-align: center; }
                        @media print {
                            button { display: none; }
                        }
                    </style>
                </head>
                <body>
                    <h1>LIBRO DIARIO</h1>
                    <h2>${CompanyService.getCurrent()?.name || ''}</h2>
                    ${content.innerHTML}
                    <script>setTimeout(() => window.print(), 500);<\/script>
                </body>
                </html>
            `);
            printWindow.document.close();
        });
    },
    async renderJournalContent(entries) {
        const content = document.getElementById('journal-content');
```

```
        if (entries.length === 0) {
            content.innerHTML = `
                <div class="empty-state">
                    <i class="fas fa-book-open"></i>
                    <h3>Sin asientos en el período</h3>
                    <p>No hay asientos contabilizados en el rango de fechas
seleccionado</p>
                </div>
            `;
            return;
        }
        let html = '<div class="journal-entries">';
        for (const entry of entries) {
            const lines = await AccountingService.getJournalLines(entry.id);
            html += `
                <div class="card" style="margin-bottom: var(--space-4);">
                    <div class="card-header">
                        <div class="card-title">${entry.number}</div>
                        <span>${Formatters.date(entry.date, 'long')}</span>
                    </div>
                    <div class="card-body" style="padding: 0;">
                        <table class="data-table">
                            <thead>
                                <tr>
                                    <th>Cuenta</th>
                                    <th>Descripción</th>
                                    <th class="text-right">Debe</th>
                                    <th class="text-right">Haber</th>
                                </tr>
                            </thead>
                            <tbody>
                                ${await Promise.all(lines.map(async line => {
                const account = await AccountingService.getAccount(line.accountId);
                return `
                                        <tr>
                                            <td>${account?.code} - ${account?.name}</td>
                                            <td>${line.description}</td>
                                            <td class="text-right">${line.debit > 0 ?
Formatters.currency(line.debit) : ''}</td>
                                            <td class="text-right">${line.credit > 0 ?
Formatters.currency(line.credit) : ''}</td>
                                        </tr>
                                    `;
            })).then(rows => rows.join(''))}
                            </tbody>
                            <tfoot>
                                <tr style="font-weight: 600;">
                                    <td colspan="2">${entry.description}</td>
                                    <td class="text-
right">${Formatters.currency(entry.totalDebit)}</td>
                                    <td class="text-
right">${Formatters.currency(entry.totalCredit)}</td>
                                </tr>
                            </tfoot>
                        </table>
                    </div>
                </div>
            `;
        }
        html += '</div>';
        content.innerHTML = html;
    },
    async showEntryModal(entryId = null) {
        const accounts = await AccountingService.getMovementAccounts();
        const isEdit = !!entryId;
        let lines = [{ accountId: '', description: '', debit: 0, credit: 0 }];
        let entryData = { date: Helpers.getCurrentDate(), description: '' };
        if (isEdit) {
            entryData = await DB.get('journalEntries', entryId);
            lines = await AccountingService.getJournalLines(entryId);
        }
        const accountOptions = accounts.map(a =>
            `<option value="${a.id}">${a.code} - ${a.name}</option>`
        ).join('');
        Modal.open({
            title: isEdit ? 'Editar Asiento' : 'Nuevo Asiento Contable',
            size: 'xlarge',
            content: `
                <form id="entry-form">
                    <div class="form-row">
                        <div class="form-group">
```

```
                            <label class="form-label required">Fecha</label>
                            <input type="date" class="form-control" name="date"
value="${entryData.date}" required>
                        </div>
                        <div class="form-group" style="flex: 2;">
                            <label class="form-label required">Descripción</label>
                            <input type="text" class="form-control" name="description"
value="${entryData.description}"
                                   placeholder="Glosa del asiento" required>
                        </div>
                    </div>
                    <div class="section-title">Detalle del Asiento</div>
                    <div class="table-container">
                        <table class="data-table" id="entry-lines-table">
                            <thead>
                                <tr>
                                    <th style="width: 40%;">Cuenta</th>
                                    <th>Descripción</th>
                                    <th style="width: 120px;">Debe</th>
                                    <th style="width: 120px;">Haber</th>
                                    <th style="width: 50px;"></th>
                                </tr>
                            </thead>
                            <tbody id="entry-lines">
                                ${lines.map((line, idx) => this.renderEntryLine(line, idx,
accountOptions)).join('')}
                            </tbody>
                            <tfoot>
                                <tr>
                                    <td colspan="2">
                                        <button type="button" class="btn btn-ghost btn-sm"
id="btn-add-line">
                                            <i class="fas fa-plus"></i> Agregar línea
                                        </button>
                                    </td>
                                    <td class="text-right" id="total-debit" style="font-
weight: 600;">$ 0</td>
                                    <td class="text-right" id="total-credit" style="font-
weight: 600;">$ 0</td>
                                    <td></td>
                                </tr>
                                <tr>
                                    <td colspan="2"></td>
                                    <td colspan="2" class="text-center" id="balance-
status">
                                        <span class="badge badge-
warning">Descuadrado</span>
                                    </td>
                                    <td></td>
                                </tr>
                            </tfoot>
                        </table>
                    </div>
                </form>
            `,
            footer: `
                <button type="button" class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button type="button" class="btn btn-outline" id="btn-save-draft">Guardar
Borrador</button>
                <button type="button" class="btn btn-primary" id="btn-save-post">Guardar y
Contabilizar</button>
            `
        });
        this.initEntryModalEvents(accountOptions);
        this.updateEntryTotals();
    },
    renderEntryLine(line, idx, accountOptions) {
        return `
            <tr class="entry-line" data-idx="${idx}">
                <td>
                    <select class="form-control line-account" name="lines[${idx}]
[accountId]" required>
                        <option value="">Seleccione...</option>
                        ${accountOptions.replace(`value="${line.accountId}"`,
`value="${line.accountId}" selected`)}
                    </select>
                </td>
                <td>
                    <input type="text" class="form-control" name="lines[${idx}]
[description]"
```

```
                        value="${line.description || ''}" placeholder="Descripción">
                </td>
                <td>
                    <input type="number" class="form-control line-debit text-right"
name="lines[${idx}][debit]"
                        value="${line.debit || 0}" min="0" step="1">
                </td>
                <td>
                    <input type="number" class="form-control line-credit text-right"
name="lines[${idx}][credit]"
                        value="${line.credit || 0}" min="0" step="1">
                </td>
                <td>
                    <button type="button" class="btn btn-icon btn-ghost btn-remove-line"
title="Eliminar">
                        <i class="fas fa-trash"></i>
                    </button>
                </td>
            </tr>
        `;
    },
    initEntryModalEvents(accountOptions) {
        let lineCount = document.querySelectorAll('.entry-line').length;
        document.getElementById('btn-add-line').onclick = () => {
            const tbody = document.getElementById('entry-lines');
            const newRow = document.createElement('tr');
            newRow.className = 'entry-line';
            newRow.dataset.idx = lineCount;
            newRow.innerHTML = this.renderEntryLine({ accountId: '', description: '',
debit: 0, credit: 0 }, lineCount, accountOptions)
                .replace(/<tr[^>]*>|<\/tr>/g, '');
            tbody.appendChild(newRow);
            lineCount++;
            this.bindLineEvents();
        };
        this.bindLineEvents();
        document.getElementById('btn-save-draft').onclick = () => this.saveEntry(false);
        document.getElementById('btn-save-post').onclick = () => this.saveEntry(true);
    },
    bindLineEvents() {
        document.querySelectorAll('.btn-remove-line').forEach(btn => {
            btn.onclick = () => {
                if (document.querySelectorAll('.entry-line').length > 1) {
                    btn.closest('tr').remove();
                    this.updateEntryTotals();
                }
            };
        });
        document.querySelectorAll('.line-debit, .line-credit').forEach(input => {
            input.oninput = () => this.updateEntryTotals();
        });
    },
    updateEntryTotals() {
        let totalDebit = 0, totalCredit = 0;
        document.querySelectorAll('.line-debit').forEach(input => {
            totalDebit += parseFloat(input.value) || 0;
        });
        document.querySelectorAll('.line-credit').forEach(input => {
            totalCredit += parseFloat(input.value) || 0;
        });
        document.getElementById('total-debit').textContent =
Formatters.currency(totalDebit);
        document.getElementById('total-credit').textContent =
Formatters.currency(totalCredit);
        const isBalanced = Math.abs(totalDebit - totalCredit) < 0.01;
        document.getElementById('balance-status').innerHTML = isBalanced
            ? '<span class="badge badge-success"><i class="fas fa-check"></i>
Cuadrado</span>'
            : '<span class="badge badge-warning"><i class="fas fa-exclamation-triangle">
</i> Descuadrado</span>';
    },
    async saveEntry(autoPost) {
        const form = document.getElementById('entry-form');
        const formData = new FormData(form);
        const lines = [];
        document.querySelectorAll('.entry-line').forEach(row => {
            const accountId = row.querySelector('.line-account').value;
            const description = row.querySelector('input[name*="description"]').value;
            const debit = parseFloat(row.querySelector('.line-debit').value) || 0;
            const credit = parseFloat(row.querySelector('.line-credit').value) || 0;
            if (accountId && (debit > 0 || credit > 0)) {
```

```
                lines.push({ accountId, description, debit, credit });
            }
        });
        if (lines.length < 2) {
            Toast.warning('El asiento debe tener al menos 2 líneas');
            return;
        }
        const loading = Toast.loading('Guardando asiento...');
        try {
            await AccountingService.createJournalEntry({
                date: formData.get('date'),
                description: formData.get('description'),
                lines,
                autoPost
            });
            loading.success(autoPost ? 'Asiento contabilizado' : 'Asiento guardado como
borrador');
            Modal.close();
            App.navigate('contabilidad', 'asientos');
        } catch (err) {
            loading.error(err.message);
        }
    },
    async postEntry(entry) {
        if (entry.status !== 'draft') {
            Toast.warning('Solo se pueden contabilizar asientos en borrador');
            return;
        }
        const confirm = await Modal.confirm({
            title: 'Contabilizar Asiento',
            message: `¿Desea contabilizar el asiento ${entry.number}? Esta acción
actualizará los saldos de las cuentas.`,
            confirmText: 'Contabilizar',
            confirmClass: 'btn-success'
        });
        if (confirm) {
            try {
                await AccountingService.postJournalEntry(entry.id);
                Toast.success('Asiento contabilizado');
                App.navigate('contabilidad', 'asientos');
            } catch (err) {
                Toast.error(err.message);
            }
        }
    },
    async cancelEntry(entry) {
        if (entry.status === 'cancelled') {
            Toast.warning('El asiento ya está anulado');
            return;
        }
        const confirm = await Modal.confirm({
            title: 'Anular Asiento',
            message: `¿Desea anular el asiento ${entry.number}? ${entry.status ===
'posted' ? 'Se revertirán los saldos de las cuentas.' : ''}`,
            confirmText: 'Anular',
            confirmClass: 'btn-danger'
        });
        if (confirm) {
            try {
                await AccountingService.cancelJournalEntry(entry.id, 'Anulación manual');
                Toast.success('Asiento anulado');
                App.navigate('contabilidad', 'asientos');
            } catch (err) {
                Toast.error(err.message);
            }
        }
    },
    async showEntryDetail(entry) {
        if (!entry.lines || entry.lines.length === 0) {
            entry.lines = await AccountingService.getJournalLines(entry.id);
        }
        const linesWithAccounts = await Promise.all(entry.lines.map(async line => {
            const account = await AccountingService.getAccount(line.accountId);
            return {
                ...line,
                accountCode: account?.code || '',
                accountName: account?.name || 'Cuenta desconocida'
            };
        }));
        const statusLabels = {
            draft: '<span class="badge badge-warning">Borrador</span>',
```

```
            posted: '<span class="badge badge-success">Contabilizado</span>',
            cancelled: '<span class="badge badge-error">Anulado</span>'
        };
        Modal.open({
            title: `Asiento ${entry.number}`,
            size: 'large',
            content: `
                <div style="margin-bottom: var(--space-4);">
                    <div style="display: grid; grid-template-columns: repeat(3, 1fr); gap:
var(--space-4); margin-bottom: var(--space-4);">
                        <div>
                            <label style="font-size: var(--font-size-xs); color: var(--
text-tertiary); display: block;">Fecha</label>
                            <strong>${Formatters.date(entry.date, 'long')}</strong>
                        </div>
                        <div>
                            <label style="font-size: var(--font-size-xs); color: var(--
text-tertiary); display: block;">Estado</label>
                            ${statusLabels[entry.status] || entry.status}
                        </div>
                        <div>
                            <label style="font-size: var(--font-size-xs); color: var(--
text-tertiary); display: block;">Referencia</label>
                            <strong>${entry.reference || '-'}</strong>
                        </div>
                    </div>
                    <div>
                        <label style="font-size: var(--font-size-xs); color: var(--text-
tertiary); display: block;">Descripción</label>
                        <p style="margin: var(--space-1) 0 0; font-weight:
500;">${entry.description}</p>
                    </div>
                </div>
                <div class="table-container">
                    <table class="data-table">
                        <thead>
                            <tr>
                                <th>Código</th>
                                <th>Cuenta</th>
                                <th>Descripción</th>
                                <th class="text-right">Debe</th>
                                <th class="text-right">Haber</th>
                            </tr>
                        </thead>
                        <tbody>
                            ${linesWithAccounts.map(line => `
                                <tr>
                                    <td><code>${line.accountCode}</code></td>
                                    <td>${line.accountName}</td>
                                    <td>${line.description || '-'}</td>
                                    <td class="text-right">${line.debit > 0 ?
Formatters.currency(line.debit) : ''}</td>
                                    <td class="text-right">${line.credit > 0 ?
Formatters.currency(line.credit) : ''}</td>
                                </tr>
                            `).join('')}
                        </tbody>
                        <tfoot>
                            <tr style="font-weight: 600; background: var(--neutral-100);">
                                <td colspan="3">TOTALES</td>
                                <td class="text-
right">${Formatters.currency(entry.totalDebit || linesWithAccounts.reduce((s, l) => s +
(l.debit || 0), 0))}</td>
                                <td class="text-
right">${Formatters.currency(entry.totalCredit || linesWithAccounts.reduce((s, l) => s +
(l.credit || 0), 0))}</td>
                            </tr>
                        </tfoot>
                    </table>
                </div>
                ${entry.createdAt ? `
                    <div style="margin-top: var(--space-4); font-size: var(--font-size-
xs); color: var(--text-tertiary);">
                        Creado: ${Formatters.date(entry.createdAt, 'long')}
                        ${entry.updatedAt ? ` | Actualizado:
${Formatters.date(entry.updatedAt, 'long')}` : ''}
                    </div>
                ` : ''}
            `,
            footer: `
                ${entry.status === 'draft' ? `
```

```
                    <button class="btn btn-success" id="btn-post-entry-modal">
                        <i class="fas fa-check"></i> Contabilizar
                    </button>
                ` : ''}
                <button class="btn btn-secondary" onclick="Modal.close()">Cerrar</button>
            `
        });
        document.getElementById('btn-post-entry-modal')?.addEventListener('click', async
() => {
            Modal.close();
            await this.postEntry(entry);
        });
    },
    async showAccountModal(accountId = null) {
        Toast.info('Modal de cuenta ' + (accountId ? 'editar' : 'nueva'));
    }
};
const treeStyles = document.createElement('style');
treeStyles.textContent = `
    .accounts-tree { font-size: var(--font-size-sm); }
    .account-row { display: flex; align-items: center; justify-content: space-between;
padding: var(--space-2) var(--space-4); border-bottom: 1px solid var(--border-light);
transition: background var(--transition-fast); }
    .account-row:hover { background: var(--neutral-50); }
    .account-info { display: flex; align-items: center; gap: var(--space-2); }
    .tree-toggle { background: none; border: none; cursor: pointer; padding: var(--space-
1); color: var(--text-tertiary); }
    .account-code { font-family: monospace; color: var(--primary-500); font-weight: 500;
min-width: 80px; }
    .account-name { color: var(--text-primary); }
    .account-name.is-group { font-weight: 600; }
    .account-meta { display: flex; align-items: center; gap: var(--space-3); }
    .account-balance { font-weight: 500; min-width: 100px; text-align: right; }
    .account-balance.positive { color: var(--text-primary); }
    .account-balance.negative { color: var(--error-500); }
    .account-actions { display: flex; gap: var(--space-1); opacity: 0; transition: opacity
var(--transition-fast); }
    .account-row:hover .account-actions { opacity: 1; }
    .level-0 { font-weight: 700; background: var(--neutral-50); }
    .level-1 { font-weight: 600; }
    .auxiliary-card { background: white; border-radius: var(--radius-lg); border: 1px
solid var(--neutral-200); margin-bottom: var(--space-4); }
    .auxiliary-header { display: flex; justify-content: space-between; align-items:
center; padding: var(--space-4); border-bottom: 1px solid var(--neutral-200); }
    .auxiliary-title { display: flex; align-items: center; gap: var(--space-2); font-
weight: 600; color: var(--text-primary); }
    .auxiliary-transactions { padding: var(--space-4); }
    .aux-transaction { background: var(--neutral-50); padding: var(--space-3); border-
radius: var(--radius-md); margin-bottom: var(--space-2); border-left: 3px solid var(--
primary); }
    .aux-transaction-header { display: flex; justify-content: space-between; margin-
bottom: var(--space-2); }
    .aux-transaction-date { color: var(--text-secondary); font-size: 0.875rem; }
    .aux-transaction-lines { font-size: 0.875rem; color: var(--text-tertiary); }
`;
document.head.appendChild(treeStyles);
ContabilidadModule.renderAuxiliaryJournals = async function () {
    const company = CompanyService.getCurrent();
    const accountingSystem = company?.accountingSystem || 'journalizer';
    const currentDate = new Date();
    const currentPeriod = `${currentDate.getFullYear()}-${String(currentDate.getMonth() +
1).padStart(2, '0')}`;
    return `
        <div class="page-header">
            <h1 class="page-title"><i class="fas fa-books"></i> Libros Auxiliares</h1>
            <p class="page-subtitle">Sistema Centralizador - Registro por módulos</p>
        </div>
        ${accountingSystem !== 'centralizer' ? `
            <div class="alert alert-warning" style="margin-bottom: var(--space-4);">
                <i class="fas fa-exclamation-triangle"></i>
                <div>
                    <strong>Sistema Jornalizador Activo</strong>
                    <p>Actualmente el sistema está configurado como
<strong>Jornalizador</strong>.
                    Los libros auxiliares solo se usan en el sistema
<strong>Centralizador</strong>.</p>
                    <p style="margin-top: 0.5rem;">
                        <a href="#admin/configuracion" class="link-primary">Cambiar a
Centralizador en Configuración →</a>
                    </p>
                </div>
```

```
            </div>
        ` : ''}
        <div class="toolbar">
            <div class="toolbar-left">
                <select class="form-control" id="aux-period" style="width: 180px;">
                    <option value="${currentPeriod}">Período Actual: ${currentPeriod}
</option>
                    ${this.generatePeriodOptions(12)}
                </select>
                <button class="btn btn-primary" id="btn-load-aux">
                    <i class="fas fa-search"></i> Consultar
                </button>
            </div>
            <div class="toolbar-right">
                <span id="system-indicator" class="badge ${accountingSystem ===
'centralizer' ? 'badge-success' : 'badge-warning'}">
                    ${accountingSystem === 'centralizer' ? '📗 Centralizador' : '📘
Jornalizador'}
                </span>
            </div>
        </div>
        <div id="auxiliary-content">
            <div class="empty-state">
                <i class="fas fa-layer-group"></i>
                <h3>Seleccione un período</h3>
                <p>Elija un período para ver los libros auxiliares</p>
            </div>
        </div>
    `;
};
ContabilidadModule.generatePeriodOptions = function (months) {
    const options = [];
    const now = new Date();
    for (let i = 1; i <= months; i++) {
        const date = new Date(now.getFullYear(), now.getMonth() - i, 1);
        const period = `${date.getFullYear()}-${String(date.getMonth() + 1).padStart(2,
'0')}`;
        const label = date.toLocaleDateString('es-CL', { month: 'long', year: 'numeric'
});
        options.push(`<option value="${period}">${label}</option>`);
    }
    return options.join('');
};
ContabilidadModule.initAuxiliaryJournals = async function () {
    document.getElementById('btn-load-aux')?.addEventListener('click', async () => {
        const period = document.getElementById('aux-period').value;
        await this.loadAuxiliaryJournals(period);
    });
    const period = document.getElementById('aux-period')?.value;
    if (period) {
        await this.loadAuxiliaryJournals(period);
    }
};
ContabilidadModule.loadAuxiliaryJournals = async function (period) {
    const container = document.getElementById('auxiliary-content');
    if (!container) return;
    container.innerHTML = '<div class="loading">Cargando libros auxiliares...</div>';
    try {
        const journals = await AccountingService.getAuxiliaryJournals(period);
        const moduleNames = {
            sales: { name: 'Ventas', icon: 'fa-shopping-cart', color: 'success' },
            purchases: { name: 'Compras', icon: 'fa-truck', color: 'primary' },
            treasury: { name: 'Tesorería', icon: 'fa-wallet', color: 'warning' }
        };
        const modules = ['sales', 'purchases', 'treasury'];
        let html = `<div class="grid grid-cols-1 gap-4">`;
        for (const module of modules) {
            const journal = journals.find(j => j.module === module);
            const info = moduleNames[module];
            const transactionCount = journal?.transactions?.length || 0;
            let isCentralized = journal?.status === 'centralized';
            if (isCentralized && journal.centralizedEntryId) {
                const entry = await DB.get('journalEntries', journal.centralizedEntryId);
                if (entry && entry.status === 'cancelled') {
                    isCentralized = false;
                }
            }
            html += `
                <div class="auxiliary-card">
                    <div class="auxiliary-header">
                        <div class="auxiliary-title">
```

```
                                <i class="fas ${info.icon}" style="color: var(-
-${info.color})"></i>
                                <span>Libro Auxiliar de ${info.name}</span>
                                <span class="badge badge-neutral">${period}</span>
                            </div>
                            <div style="display: flex; align-items: center; gap: var(--space-
2);">
                                ${isCentralized ? `
                                    <span class="badge badge-success">
                                        <i class="fas fa-check"></i> Centralizado
                                    </span>
                                ` : `
                                    <span class="badge badge-warning">
                                        ${transactionCount} transacciones
                                    </span>
                                    ${transactionCount > 0 ? `
                                        <button class="btn btn-primary btn-sm" data-
action="centralize" data-module="${module}" data-period="${period}">
                                            <i class="fas fa-compress-arrows-alt"></i>
Centralizar
                                        </button>
                                    ` : ''}
                                `}
                            </div>
                        </div>
                        <div class="auxiliary-transactions">
                            ${!journal || transactionCount === 0 ? `
                                <div style="text-align: center; color: var(--text-tertiary);
padding: var(--space-4);">
                                    <i class="fas fa-inbox" style="font-size: 2rem; opacity:
0.5;"></i>
                                    <p>Sin transacciones en este período</p>
                                </div>
                            ` : journal.transactions.map(t => `
                                <div class="aux-transaction">
                                    <div class="aux-transaction-header">
                                        <strong>${t.description}</strong>
                                        <span class="aux-transaction-date">${t.date}</span>
                                    </div>
                                    <div class="aux-transaction-lines">
                                        ${t.lines.map(l => `
                                            <span style="margin-right: 1rem;">
                                                ${l.debit > 0 ? `D:
$${l.debit.toLocaleString()}` : `C: $${l.credit.toLocaleString()}`}
                                            </span>
                                        `).join('')}
                                    </div>
                                </div>
                            `).join('')}
                            ${isCentralized && journal.centralizedEntryId ? `
                                <div style="margin-top: var(--space-3); padding: var(--space-
3); background: var(--success-50); border-radius: var(--radius-md); border-left: 3px solid
var(--success);">
                                    <strong><i class="fas fa-check-circle" style="color: var(-
-success);"></i> Centralizado</strong>
                                    <p style="margin: 0.5rem 0 0; font-size: 0.875rem; color:
var(--text-secondary);">
                                        Asiento resumen generado el ${new
Date(journal.centralizedAt).toLocaleDateString('es-CL')}
                                    </p>
                                    <a href="#contabilidad/asientos" class="link-primary"
style="font-size: 0.875rem;">
                                        Ver asiento en Libro Diario →
                                    </a>
                                </div>
                            ` : ''}
                        </div>
                    </div>
                `;
            }
            html += `</div>`;
            container.innerHTML = html;
            container.querySelectorAll('[data-action="centralize"]').forEach(btn => {
                btn.addEventListener('click', async () => {
                    const module = btn.dataset.module;
                    const period = btn.dataset.period;
                    await this.centralizeJournal(module, period);
                });
            });
        } catch (err) {
            container.innerHTML = `<div class="alert alert-error">Error: ${err.message}
```

```
</div>`;
    }
};
ContabilidadModule.centralizeJournal = async function (module, period) {
    const moduleNames = { sales: 'Ventas', purchases: 'Compras', treasury: 'Tesorería' };
    const confirm = await Modal.confirm({
        title: `Centralizar ${moduleNames[module]}`,
        message: `¿Desea centralizar el libro auxiliar de <strong>${moduleNames[module]}
</strong> para el período <strong>${period}</strong>?
        <div style="margin-top: 1rem; padding: 1rem; background: var(--warning-50);
border-radius: 0.5rem; border-left: 3px solid var(--warning);">
            <strong>⚠️ Esta acción:</strong>
            <ul style="margin: 0.5rem 0 0; padding-left: 1.5rem;">
                <li>Generará un asiento resumen en el Libro Diario</li>
                <li>Consolidará todas las transacciones del período</li>
                <li>No se puede revertir automáticamente</li>
            </ul>
        </div>`,
        confirmText: 'Centralizar',
        confirmClass: 'btn-primary'
    });
    if (!confirm) return;
    const loading = Toast.loading(`Centralizando ${moduleNames[module]}...`);
    try {
        const result = await AccountingService.centralizeAuxiliaryJournal(module, period);
        loading.success(`¡Centralizado! Se generó asiento resumen con
${result.linesConsolidated} líneas.`);
        await this.loadAuxiliaryJournals(period);
    } catch (err) {
        loading.error('Error: ' + err.message);
    }
};
window.ContabilidadModule = ContabilidadModule;
```

Archivo: js\modules\costos.js

```
const CostosModule = {
    currentView: 'centros-costo',
    async render(view = 'centros-costo') {
        this.currentView = view;
        switch (view) {
            case 'centros-costo': return await this.renderCostCenters();
            case 'ordenes-internas': return this.renderInternalOrders();
            case 'analisis-costos': return await this.renderCostAnalysis();
            default: return '<p>Vista no encontrada</p>';
        }
    },
    async renderCostCenters() {
        return `
            <div class="page-header">
                <h1 class="page-title">Centros de Costo</h1>
                <p class="page-subtitle">Estructura de centros de costo para distribución
de gastos</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <div class="search-box">
                        <i class="fas fa-search"></i>
                        <input type="text" class="form-control" placeholder="Buscar..."
id="search-cc">
                    </div>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-cc">
                        <i class="fas fa-plus"></i> Nuevo Centro de Costo
                    </button>
                </div>
            </div>
            <div id="cc-table"></div>
        `;
    },
    renderInternalOrders() {
        return `
            <div class="page-header">
                <h1 class="page-title">Órdenes Internas</h1>
```

```
                <p class="page-subtitle">Seguimiento de proyectos y actividades
especiales</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <select class="form-control" id="filter-status" style="width: 150px;">
                        <option value="">Todos</option>
                        <option value="open">Abiertas</option>
                        <option value="closed">Cerradas</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-order">
                        <i class="fas fa-plus"></i> Nueva Orden
                    </button>
                </div>
            </div>
            <div id="orders-table"></div>
        `;
    },
    async renderCostAnalysis() {
        const company = CompanyService.getCurrent();
        let costCenters = [];
        if (company) {
            costCenters = await DB.getByIndex('costCenters', 'companyId', company.id);
        }
        return `
            <div class="page-header">
                <h1 class="page-title">Análisis de Costos</h1>
                <p class="page-subtitle">Distribución y análisis de gastos por centro de
costo</p>
            </div>
            <div class="dashboard-grid">
                ${costCenters.map(cc => `
                    <div class="kpi-card">
                        <div class="kpi-header">
                            <div class="kpi-icon ${cc.type === 'overhead' ? 'warning' :
'primary'}">
                                <i class="fas fa-${cc.type === 'overhead' ? 'building' :
'cogs'}"></i>
                            </div>
                        </div>
                        <div class="kpi-value">${Formatters.currency(cc.actual || 0)}
</div>
                        <div class="kpi-label">${cc.code} - ${cc.name}</div>
                        <div class="kpi-footer">
                            <span>Presupuesto: ${Formatters.currency(cc.budget || 0)}
</span>
                            ${cc.budget > 0 ? `
                                <span class="kpi-trend ${cc.actual <= cc.budget ? 'up' :
'down'}">
                                    ${((cc.actual / cc.budget) * 100).toFixed(0)}%
                                </span>
                            ` : ''}
                        </div>
                    </div>
                `).join('') || `
                    <div class="empty-state span-4">
                        <i class="fas fa-sitemap"></i>
                        <h3>Sin centros de costo</h3>
                        <p>Cree centros de costo para distribuir y analizar gastos</p>
                    </div>
                `}
            </div>
            <div class="card" style="margin-top: var(--space-4);">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-chart-pie"></i> Distribución
de Costos</div>
                </div>
                <div class="card-body">
                    <div style="display: flex; gap: var(--space-4);">
                        <div style="flex: 1; height: 300px;">
                            <canvas id="costs-chart"></canvas>
                        </div>
                        <div style="flex: 1; height: 300px;">
                            <canvas id="budget-chart"></canvas>
                        </div>
                    </div>
                </div>
            </div>
        `;
```

```
    },
    async init(view) {
        const company = CompanyService.getCurrent();
        if (!company) return;
        switch (view) {
            case 'centros-costo': await this.initCostCenters(); break;
            case 'ordenes-internas': await this.initInternalOrders(); break;
            case 'analisis-costos': await this.initCostAnalysis(); break;
        }
    },
    async initCostCenters() {
        const company = CompanyService.getCurrent();
        const costCenters = await DB.getByIndex('costCenters', 'companyId', company.id);
        DataTable.create('cc-table', {
            columns: [
                { key: 'code', label: 'Código' },
                { key: 'name', label: 'Nombre' },
                {
                    key: 'type', label: 'Tipo', render: (v) => {
                        return v === 'overhead' ?
                            '<span class="badge badge-warning">Overhead</span>' :
                            '<span class="badge badge-primary">Operacional</span>';
                    }
                },
                { key: 'budget', label: 'Presupuesto', format: 'currency', class: 'text-
right' },
                { key: 'actual', label: 'Real', format: 'currency', class: 'text-right' },
                {
                    key: 'variance', label: 'Variación', class: 'text-right', render: (v,
row) => {
                        const variance = (row.budget || 0) - (row.actual || 0);
                        return `<span class="${variance >= 0 ? 'positive' :
'negative'}">${Formatters.currency(variance)}</span>`;
                    }
                }
            ],
            data: costCenters,
            actions: [
                { name: 'edit', label: 'Editar', icon: 'fas fa-edit', handler: (row) =>
this.showCostCenterModal(row.id) },
                { name: 'delete', label: 'Eliminar', icon: 'fas fa-trash', handler: (row)
=> this.deleteCostCenter(row.id) }
            ],
            emptyMessage: 'No hay centros de costo'
        });
        document.getElementById('btn-new-cc')?.addEventListener('click', () =>
this.showCostCenterModal());
    },
    async showCostCenterModal(ccId = null) {
        const isEdit = !!ccId;
        let data = {};
        if (isEdit) {
            data = await DB.get('costCenters', ccId);
        }
        await Modal.form({
            title: isEdit ? 'Editar Centro de Costo' : 'Nuevo Centro de Costo',
            fields: [
                { name: 'code', label: 'Código', required: true, placeholder: 'CC001',
default: data.code },
                { name: 'name', label: 'Nombre', required: true, placeholder: 'Nombre del
centro', default: data.name },
                {
                    name: 'type', label: 'Tipo', type: 'select', options: [
                        { value: 'operational', label: 'Operacional' },
                        { value: 'overhead', label: 'Overhead (Gastos Generales)' }
                    ], default: data.type || 'operational'
                },
                { name: 'budget', label: 'Presupuesto Mensual', type: 'number', default:
data.budget || 0 }
            ],
            onSubmit: async (formData) => {
                const company = CompanyService.getCurrent();
                if (isEdit) {
                    await DB.update('costCenters', { ...data, ...formData });
                    Toast.success('Centro de costo actualizado');
                } else {
                    await DB.add('costCenters', { companyId: company.id, ...formData,
actual: 0, status: 'active' });
                    Toast.success('Centro de costo creado');
                }
                App.navigate('costos', 'centros-costo');
```

```
            }
        });
    },
    async deleteCostCenter(id) {
        const confirm = await Modal.confirm({
            title: 'Eliminar Centro de Costo',
            message: '¿Está seguro de eliminar este centro de costo?',
            confirmText: 'Eliminar',
            confirmClass: 'btn-danger'
        });
        if (confirm) {
            await DB.delete('costCenters', id);
            Toast.success('Centro de costo eliminado');
            App.navigate('costos', 'centros-costo');
        }
    },
    async initInternalOrders() {
        const company = CompanyService.getCurrent();
        const orders = await DB.getByIndex('internalOrders', 'companyId', company.id);
        DataTable.create('orders-table', {
            columns: [
                { key: 'number', label: 'Número' },
                { key: 'description', label: 'Descripción' },
                { key: 'costCenterName', label: 'Centro de Costo' },
                { key: 'budget', label: 'Presupuesto', format: 'currency', class: 'text-
right' },
                { key: 'actual', label: 'Ejecutado', format: 'currency', class: 'text-
right' },
                { key: 'status', label: 'Estado', format: 'status' }
            ],
            data: orders,
            emptyMessage: 'No hay órdenes internas'
        });
        document.getElementById('btn-new-order')?.addEventListener('click', () => {
            Toast.info('Nueva orden interna');
        });
    },
    async initCostAnalysis() {
        const company = CompanyService.getCurrent();
        const costCenters = await DB.getByIndex('costCenters', 'companyId', company.id);
        if (costCenters.length > 0) {
            Charts.doughnut('costs-chart', {
                labels: costCenters.map(cc => cc.name),
                values: costCenters.map(cc => cc.actual || 0)
            }, { currency: true });
            Charts.bar('budget-chart', {
                labels: costCenters.map(cc => cc.code),
                datasets: [
                    { label: 'Presupuesto', data: costCenters.map(cc => cc.budget || 0),
color: Charts.colors.primary },
                    { label: 'Real', data: costCenters.map(cc => cc.actual || 0), color:
Charts.colors.warning }
                ]
            }, { currency: true });
        }
    }
};
window.CostosModule = CostosModule;
```

Archivo: js\modules\dashboard.js

```
const DashboardModule = {
    async render() {
        const company = CompanyService.getCurrent();
        if (!company) {
            return this.renderNoCompany();
        }
        const stats = await this.getStats();
        return `
            <div class="dashboard">
                <!-- Banner de bienvenida -->
                ${this.renderWelcomeBanner(company)}
                <!-- KPIs principales -->
                <div class="dashboard-grid">
                    ${this.renderKPICards(stats)}
```

```
                </div>
                <!-- Gráficos y actividad -->
                <div class="dashboard-grid" style="margin-top: var(--space-4);">
                    <!-- Gráfico de ingresos vs gastos -->
                    <div class="chart-card span-2">
                        <div class="chart-header">
                            <div>
                                <div class="chart-title">Ingresos vs Gastos</div>
                                <div class="chart-subtitle">Últimos 6 meses</div>
                            </div>
                            <div class="period-selector">
                                <button class="period-btn active" data-
period="6m">6M</button>
                                <button class="period-btn" data-period="1y">1A</button>
                            </div>
                        </div>
                        <div class="chart-body">
                            <canvas id="revenue-expenses-chart"></canvas>
                        </div>
                    </div>
                    <!-- Distribución de gastos -->
                    <div class="chart-card span-2">
                        <div class="chart-header">
                            <div>
                                <div class="chart-title">Distribución de Gastos</div>
                                <div class="chart-subtitle">Por categoría</div>
                            </div>
                        </div>
                        <div class="chart-body" style="display: flex; align-items: center;
justify-content: center;">
                            <div style="width: 250px; height: 250px;">
                                <canvas id="expenses-distribution-chart"></canvas>
                            </div>
                        </div>
                    </div>
                </div>
                <div class="dashboard-grid" style="margin-top: var(--space-4);">
                    <!-- Acciones rápidas -->
                    <div class="card span-2">
                        <div class="card-header">
                            <div class="card-title">
                                <i class="fas fa-bolt"></i>
                                Acciones Rápidas
                            </div>
                        </div>
                        <div class="card-body">
                            <div class="quick-actions">
                                <a href="#" class="quick-action-btn" data-action="new-
entry">
                                    <i class="fas fa-file-invoice"></i>
                                    <span>Nuevo Asiento</span>
                                </a>
                                <a href="#" class="quick-action-btn" data-action="new-
sale">
                                    <i class="fas fa-shopping-cart"></i>
                                    <span>Nueva Venta</span>
                                </a>
                                <a href="#" class="quick-action-btn" data-action="new-
purchase">
                                    <i class="fas fa-truck"></i>
                                    <span>Nueva Compra</span>
                                </a>
                                <a href="#" class="quick-action-btn" data-action="new-
payment">
                                    <i class="fas fa-money-bill-wave"></i>
                                    <span>Registrar Pago</span>
                                </a>
                                <a href="#" class="quick-action-btn" data-
action="reports">
                                    <i class="fas fa-chart-pie"></i>
                                    <span>Reportes</span>
                                </a>
                            </div>
                        </div>
                    </div>
                    <!-- Actividad reciente -->
                    <div class="card span-2">
                        <div class="card-header">
                            <div class="card-title">
                                <i class="fas fa-history"></i>
                                Actividad Reciente
```

```
                            </div>
                            <a href="#" class="btn btn-ghost btn-sm">Ver todo</a>
                        </div>
                        <div class="card-body">
                            ${await this.renderRecentActivity()}
                        </div>
                    </div>
                </div>
                <!-- Resumen financiero -->
                <div class="dashboard-grid" style="margin-top: var(--space-4);">
                    <div class="card span-4">
                        <div class="card-header">
                            <div class="card-title">
                                <i class="fas fa-balance-scale"></i>
                                Resumen Financiero
                            </div>
                            <span class="badge badge-info">Al día de hoy</span>
                        </div>
                        <div class="card-body">
                            ${await this.renderFinancialSummary(stats)}
                        </div>
                    </div>
                </div>
                <!-- Banner de Autoría -->
                <div class="authorship-banner" style="margin-top: var(--space-6);">
                    <div class="authorship-content">
                        <div class="authorship-logo">
                            <i class="fas fa-graduation-cap"></i>
                        </div>
                        <div class="authorship-info">
                            <div class="authorship-title">
                                <span class="system-name">EDU-TRACE ERP</span>
                                <span class="version-badge">v1.0 - Etapa 1</span>
                            </div>
                            <div class="authorship-subtitle">
                                Sistema de Gestión Empresarial (ERP) Educativo con
Trazabilidad Pedagógica Integrada
                            </div>
                            <div class="authorship-credits">
                                <div class="credit-line">
                                    <i class="fas fa-user-edit"></i>
                                    <span><strong>Autora:</strong> Sandy Iturra
Mena</span>
                                </div>
                            </div>
                        </div>
                        <div class="authorship-badge">
                            <div class="institution-logo-img">
                                <img src="https://www.uv.cl/images/descargas-
corporativas/uv_logo_alta_rgba_blanco-2023.png" alt="Logo Universidad de Valparaíso"
style="height: 50px; width: auto;">
                            </div>
                            <div class="institution-text">
                                <div class="institution-name">Universidad de
Valparaíso</div>
                                <div class="institution-dept">Escuela de Auditoría</div>
                            </div>
                        </div>
                    </div>
                    <div class="authorship-footer">
                        <span>© 2025 Sandy Iturra Mena. Todos los derechos reservados.
</span>
                        <span class="separator">|</span>
                        <span>Desarrollado para la enseñanza de Contabilidad</span>
                    </div>
                </div>
            </div>
        `;
    },
    renderNoCompany() {
        return `
            <div class="empty-state" style="padding: var(--space-16) var(--space-8);">
                <i class="fas fa-building"></i>
                <h3>Bienvenido a EDU-TRACE ERP</h3>
                <p>Para comenzar, crea tu primera empresa virtual o importa un caso de
estudio existente.</p>
                <div class="btn-group" style="justify-content: center; margin-top: var(--
space-4);">
                    <button class="btn btn-primary" id="btn-create-company">
                        <i class="fas fa-plus"></i> Crear Empresa
                    </button>
```

```
                    <button class="btn btn-outline" id="btn-import-company">
                        <i class="fas fa-file-import"></i> Importar Datos
                    </button>
                </div>
            </div>
        `;
    },
    renderWelcomeBanner(company) {
        const hour = new Date().getHours();
        let greeting = 'Buenos días';
        if (hour >= 12 && hour < 19) greeting = 'Buenas tardes';
        else if (hour >= 19) greeting = 'Buenas noches';
        return `
            <div class="welcome-banner">
                <div class="welcome-content">
                    <h1 class="welcome-title">${greeting}!</h1>
                    <p class="welcome-subtitle">
                        Estás trabajando con <strong>${company.name}</strong>.
                        Aquí tienes un resumen de la situación actual.
                    </p>
                    <div class="welcome-actions">
                        <button class="welcome-btn primary" data-action="tutorial">
                            <i class="fas fa-play-circle"></i> Tutorial Rápido
                        </button>
                        <button class="welcome-btn" data-action="system-flow">
                            <i class="fas fa-project-diagram"></i> Diagrama del Sistema
                        </button>
                        <button class="welcome-btn" data-action="accounting-course">
                            <i class="fas fa-graduation-cap"></i> Curso de Contabilidad
                        </button>
                        <button class="welcome-btn" data-action="help">
                            <i class="fas fa-question-circle"></i> Ayuda
                        </button>
                    </div>
                </div>
            </div>
        `;
    },
    renderKPICards(stats) {
        const kpis = [
            {
                label: 'Ingresos del Mes',
                value: Formatters.currency(stats.monthlyRevenue),
                icon: 'fa-arrow-trend-up',
                type: 'success',
                trend: stats.revenueTrend,
                trendLabel: 'vs mes anterior'
            },
            {
                label: 'Gastos del Mes',
                value: Formatters.currency(stats.monthlyExpenses),
                icon: 'fa-arrow-trend-down',
                type: 'error',
                trend: stats.expensesTrend,
                trendLabel: 'vs mes anterior'
            },
            {
                label: 'Cuentas por Cobrar',
                value: Formatters.currency(stats.accountsReceivable),
                icon: 'fa-file-invoice-dollar',
                type: 'warning',
                count: `${stats.pendingInvoices} facturas`
            },
            {
                label: 'Cuentas por Pagar',
                value: Formatters.currency(stats.accountsPayable),
                icon: 'fa-file-invoice',
                type: 'primary',
                count: `${stats.pendingBills} facturas`
            }
        ];
        return kpis.map(kpi => `
            <div class="kpi-card ${kpi.type}">
                <div class="kpi-header">
                    <div class="kpi-icon ${kpi.type}">
                        <i class="fas ${kpi.icon}"></i>
                    </div>
                    <button class="kpi-menu btn btn-ghost btn-icon">
                        <i class="fas fa-ellipsis-v"></i>
                    </button>
                </div>
```

```
                <div class="kpi-value">${kpi.value}</div>
                <div class="kpi-label">${kpi.label}</div>
                <div class="kpi-footer">
                    ${kpi.trend !== undefined ? `
                        <span class="kpi-trend ${kpi.trend >= 0 ? 'up' : 'down'}">
                            <i class="fas fa-arrow-${kpi.trend >= 0 ? 'up' : 'down'}"></i>
                            ${Math.abs(kpi.trend).toFixed(1)}%
                        </span>
                        <span>${kpi.trendLabel}</span>
                    ` : ''}
                    ${kpi.count ? `<span>${kpi.count}</span>` : ''}
                </div>
            </div>
        `).join('');
    },
    async renderRecentActivity() {
        const company = CompanyService.getCurrent();
        if (!company) return '<p class="text-muted">No hay actividad</p>';
        const logs = await DB.getByIndex('auditLog', 'companyId', company.id);
        const recentLogs = logs.slice(-5).reverse();
        if (recentLogs.length === 0) {
            return `
                <div class="empty-state" style="padding: var(--space-4);">
                    <i class="fas fa-clock" style="font-size: 2rem;"></i>
                    <p>No hay actividad reciente</p>
                </div>
            `;
        }
        const activityTypes = {
            'CREATE': { icon: 'fa-plus-circle', class: 'success', label: 'Creación' },
            'UPDATE': { icon: 'fa-edit', class: 'info', label: 'Modificación' },
            'DELETE': { icon: 'fa-trash', class: 'error', label: 'Eliminación' },
            'POST': { icon: 'fa-check-circle', class: 'success', label: 'Contabilización'
},
            'CANCEL': { icon: 'fa-times-circle', class: 'warning', label: 'Anulación' }
        };
        return `
            <div class="activity-list">
                ${recentLogs.map(log => {
            const type = activityTypes[log.action] || { icon: 'fa-circle', class:
'neutral', label: log.action };
            return `
                        <div class="activity-item">
                            <div class="activity-icon ${type.class}">
                                <i class="fas ${type.icon}"></i>
                            </div>
                            <div class="activity-content">
                                <div class="activity-title">${type.label} de ${log.entity}
</div>
                                <div class="activity-
desc">${JSON.stringify(log.details).substring(0, 50)}...</div>
                            </div>
                            <div class="activity-
time">${Formatters.relativeTime(log.timestamp)}</div>
                        </div>
                    `;
        }).join('')}
            </div>
        `;
    },
    async renderFinancialSummary(stats) {
        return `
            <div class="financial-summary">
                <div class="summary-item">
                    <div class="summary-value positive">
                        ${Formatters.currency(stats.totalAssets)}
                    </div>
                    <div class="summary-label">Total Activos</div>
                </div>
                <div class="summary-item">
                    <div class="summary-value negative">
                        ${Formatters.currency(stats.totalLiabilities)}
                    </div>
                    <div class="summary-label">Total Pasivos</div>
                </div>
                <div class="summary-item">
                    <div class="summary-value ${stats.netEquity >= 0 ? 'positive' :
'negative'}">
                        ${Formatters.currency(stats.netEquity)}
                    </div>
                    <div class="summary-label">Patrimonio Neto</div>
```

```
                </div>
            </div>
            <div class="divider"></div>
            <div style="display: flex; justify-content: space-between; align-items:
center; padding-top: var(--space-2);">
                <div>
                    <span style="color: var(--text-secondary); font-size: var(--font-size-
sm);">
                        Resultado del ejercicio:
                    </span>
                    <strong style="font-size: var(--font-size-lg); margin-left: var(--
space-2); color: ${stats.netIncome >= 0 ? 'var(--success-500)' : 'var(--error-500)'}">
                        ${Formatters.currency(stats.netIncome)}
                    </strong>
                </div>
                <a href="#" class="btn btn-outline btn-sm" data-view="balance-general">
                    Ver Balance Completo <i class="fas fa-arrow-right"></i>
                </a>
            </div>
        `;
    },
    async getStats() {
        const company = CompanyService.getCurrent();
        if (!company) return this.getEmptyStats();
        try {
            const accounts = await AccountingService.getChartOfAccounts();
            const assets = accounts.filter(a => a.type === 'asset' && !a.isGroup);
            const liabilities = accounts.filter(a => a.type === 'liability' &&
!a.isGroup);
            const equity = accounts.filter(a => a.type === 'equity' && !a.isGroup);
            const revenues = accounts.filter(a => a.type === 'revenue' && !a.isGroup);
            const expenses = accounts.filter(a => a.type === 'expense' && !a.isGroup);
            const totalAssets = Helpers.sumBy(assets, 'balance');
            const totalLiabilities = Helpers.sumBy(liabilities, 'balance');
            const totalEquity = Helpers.sumBy(equity, 'balance');
            const totalRevenue = Helpers.sumBy(revenues, 'balance');
            const totalExpenses = Helpers.sumBy(expenses, 'balance');
            const customerInvoices = await DB.getByIndex('customerInvoices', 'companyId',
company.id);
            const supplierInvoices = await DB.getByIndex('supplierInvoices', 'companyId',
company.id);
            const pendingInvoices = customerInvoices.filter(i => i.status === 'pending' ||
i.status === 'partial');
            const pendingBills = supplierInvoices.filter(i => i.status === 'pending' ||
i.status === 'partial');
            return {
                monthlyRevenue: totalRevenue,
                monthlyExpenses: totalExpenses,
                revenueTrend: 5.2,
                expensesTrend: -2.1,
                accountsReceivable: Helpers.sumBy(pendingInvoices, 'balance'),
                accountsPayable: Helpers.sumBy(pendingBills, 'balance'),
                pendingInvoices: pendingInvoices.length,
                pendingBills: pendingBills.length,
                totalAssets,
                totalLiabilities,
                netEquity: totalEquity,
                netIncome: totalRevenue - totalExpenses
            };
        } catch (err) {
            console.error('Error getting stats:', err);
            return this.getEmptyStats();
        }
    },
    getEmptyStats() {
        return {
            monthlyRevenue: 0, monthlyExpenses: 0, revenueTrend: 0, expensesTrend: 0,
            accountsReceivable: 0, accountsPayable: 0, pendingInvoices: 0, pendingBills:
0,
            totalAssets: 0, totalLiabilities: 0, netEquity: 0, netIncome: 0
        };
    },
    initCharts() {
        const company = CompanyService.getCurrent();
        if (!company) return;
        Charts.bar('revenue-expenses-chart', {
            labels: ['Ene', 'Feb', 'Mar', 'Abr', 'May', 'Jun'],
            datasets: [
                { label: 'Ingresos', data: [1200000, 1500000, 1300000, 1800000, 1600000,
2000000], color: Charts.colors.success },
                { label: 'Gastos', data: [800000, 900000, 850000, 950000, 880000, 920000],
```

```
color: Charts.colors.error }
            ]
        }, { currency: true });
        Charts.doughnut('expenses-distribution-chart', {
            labels: ['Remuneraciones', 'Arriendos', 'Servicios', 'Marketing', 'Otros'],
            values: [45, 20, 15, 10, 10]
        }, { currency: false });
    },
    init() {
        this.bindEvents();
        this.initCharts();
    },
    bindEvents() {
        document.getElementById('btn-create-company')?.addEventListener('click', () => {
            this.showCreateCompanyModal();
        });
        document.getElementById('btn-import-company')?.addEventListener('click', () => {
            App.navigate('admin', 'import-export');
        });
        document.querySelectorAll('[data-action]').forEach(btn => {
            btn.addEventListener('click', (e) => {
                e.preventDefault();
                const action = btn.dataset.action;
                this.handleQuickAction(action);
            });
        });
    },
    async showCreateCompanyModal() {
        const result = await Modal.form({
            title: 'Crear Nueva Empresa',
            fields: [
                { name: 'name', label: 'Nombre de la Empresa', required: true,
placeholder: 'Ej: Mi Empresa S.A.' },
                { name: 'rut', label: 'RUT', required: true, placeholder: 'Ej: 76.123.456-
7' },
                { name: 'address', label: 'Dirección', placeholder: 'Dirección comercial'
},
                { name: 'phone', label: 'Teléfono', placeholder: '+56 9 1234 5678' },
                { name: 'email', label: 'Email', type: 'email', placeholder:
'contacto@empresa.cl' },
                { name: 'fiscalYear', label: 'Año Fiscal', type: 'number', default: new
Date().getFullYear() },
                {
                    name: 'createDemo', label: 'Datos de demostración', type: 'checkbox',
default: true,
                    checkLabel: 'Crear datos de ejemplo (clientes, proveedores,
productos)'
                }
            ],
            submitText: 'Crear Empresa',
            validate: (data) => {
                if (!data.name) return 'El nombre es requerido';
                if (!data.rut) return 'El RUT es requerido';
                if (!Helpers.validateRUT(data.rut)) return 'El RUT no es válido';
                return true;
            },
            onSubmit: async (data) => {
                const loading = Toast.loading('Creando empresa...');
                try {
                    const company = await CompanyService.create({
                        name: data.name,
                        rut: Helpers.formatRUT(data.rut),
                        address: data.address,
                        phone: data.phone,
                        email: data.email,
                        fiscalYear: data.fiscalYear
                    });
                    await CompanyService.setCurrent(company.id);
                    if (data.createDemo) {
                        loading.update('Creando datos de demostración...');
                        await CompanyService.createDemoData();
                    }
                    loading.success('Empresa creada exitosamente');
                    App.navigate('dashboard');
                } catch (err) {
                    loading.error('Error al crear empresa: ' + err.message);
                }
            }
        });
    },
    handleQuickAction(action) {
```

```
        switch (action) {
            case 'new-entry':
                App.navigate('contabilidad', 'asientos');
                break;
            case 'new-sale':
                App.navigate('ventas', 'facturas-cliente');
                break;
            case 'new-purchase':
                App.navigate('compras', 'facturas-proveedor');
                break;
            case 'new-payment':
                App.navigate('tesoreria', 'bancos');
                break;
            case 'reports':
                App.navigate('reportes', 'balance-general');
                break;
            case 'tutorial':
                TutorialSystem.start('quickStart');
                break;
            case 'help':
                HelpSystem.show();
                break;
            case 'system-flow':
                SystemFlowDiagram.show();
                break;
            case 'accounting-course':
                window.open('diagrama-contable.html', 'CursoContabilidad',
'width=1400,height=900,scrollbars=yes,resizable=yes');
                break;
        }
    }
};
window.DashboardModule = DashboardModule;
```

Archivo: js\modules\inventario.js

```
const InventarioModule = {
    currentView: 'productos',
    async render(view = 'productos') {
        this.currentView = view;
        switch (view) {
            case 'productos': return await this.renderProducts();
            case 'movimientos-stock': return await this.renderStockMovements();
            case 'valoracion': return await this.renderValuation();
            default: return '<p>Vista no encontrada</p>';
        }
    },
    async renderProducts() {
        return `
            <div class="page-header">
                <h1 class="page-title">Productos</h1>
                <p class="page-subtitle">Catálogo de productos y servicios</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <div class="search-box">
                        <i class="fas fa-search"></i>
                        <input type="text" class="form-control" placeholder="Buscar
producto..." id="search-products">
                    </div>
                    <select class="form-control" id="filter-category" style="width:
150px;">
                        <option value="">Todas las categorías</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-import-products">
                        <i class="fas fa-upload"></i> Importar
                    </button>
                    <button class="btn btn-primary" id="btn-new-product">
                        <i class="fas fa-plus"></i> Nuevo Producto
                    </button>
                </div>
            </div>
            <div id="products-table"></div>
```

```
        `;
    },
    async renderStockMovements() {
        return `
            <div class="page-header">
                <h1 class="page-title">Movimientos de Stock</h1>
                <p class="page-subtitle">Registro de entradas y salidas de inventario</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="filter-date-from">
                    <input type="date" class="form-control" id="filter-date-to">
                    <select class="form-control" id="filter-type" style="width: 150px;">
                        <option value="">Todos</option>
                        <option value="in">Entradas</option>
                        <option value="out">Salidas</option>
                        <option value="adjustment">Ajustes</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-adjustment">
                        <i class="fas fa-balance-scale"></i> Ajuste de Inventario
                    </button>
                </div>
            </div>
            <div id="movements-table"></div>
        `;
    },
    async renderValuation() {
        return `
            <div class="page-header">
                <h1 class="page-title">Valorización de Inventario</h1>
                <p class="page-subtitle">Valoración del stock según método contable</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="valuation-date"
value="${Helpers.getCurrentDate()}">
                    <select class="form-control" id="valuation-method" style="width:
180px;">
                        <option value="promedio">Costo Promedio</option>
                        <option value="fifo">FIFO (PEPS)</option>
                        <option value="lifo">LIFO (UEPS)</option>
                    </select>
                    <button class="btn btn-primary" id="btn-calculate">
                        <i class="fas fa-calculator"></i> Calcular
                    </button>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-export-valuation">
                        <i class="fas fa-file-excel"></i> Exportar
                    </button>
                </div>
            </div>
            <div id="valuation-content">
                ${await this.renderValuationTable()}
            </div>
        `;
    },
    async renderValuationTable(method = 'promedio') {
        const company = CompanyService.getCurrent();
        if (!company) return '';
        const products = await DB.getByIndex('products', 'companyId', company.id);
        const physicalProducts = products.filter(p => !p.isService && (p.stock || 0) > 0);
        const movements = await DB.getByIndex('stockMovements', 'companyId', company.id);
        const valuatedProducts = await Promise.all(physicalProducts.map(async (p) => {
            const productMovements = movements
                .filter(m => m.productId === p.id)
                .sort((a, b) => new Date(a.date) - new Date(b.date));
            const baseCost = p.cost || 0;
            let unitCost = baseCost;
            const entries = productMovements.filter(m => m.type === 'in');
            if (method === 'promedio') {
                if (entries.length > 0) {
                    const totalCost = entries.reduce((sum, m) => sum + (m.quantity *
(m.unitCost || baseCost)), 0);
                    const totalQty = entries.reduce((sum, m) => sum + m.quantity, 0);
                    unitCost = totalQty > 0 ? totalCost / totalQty : baseCost;
                }
            } else if (method === 'fifo') {
                if (entries.length > 0) {
```

```
                    const reversedEntries = [...entries].reverse();
                    let remainingQty = p.stock || 0;
                    let totalCost = 0;
                    for (const entry of reversedEntries) {
                        if (remainingQty <= 0) break;
                        const entryQty = entry.quantity || 0;
                        const entryCost = entry.unitCost || baseCost;
                        const qtyToUse = Math.min(remainingQty, entryQty);
                        totalCost += qtyToUse * entryCost;
                        remainingQty -= qtyToUse;
                    }
                    if (remainingQty > 0) {
                        totalCost += remainingQty * baseCost;
                    }
                    unitCost = (p.stock || 0) > 0 ? totalCost / p.stock : baseCost;
                }
            } else if (method === 'lifo') {
                if (entries.length > 0) {
                    let remainingQty = p.stock || 0;
                    let totalCost = 0;
                    for (const entry of entries) {
                        if (remainingQty <= 0) break;
                        const entryQty = entry.quantity || 0;
                        const entryCost = entry.unitCost || baseCost;
                        const qtyToUse = Math.min(remainingQty, entryQty);
                        totalCost += qtyToUse * entryCost;
                        remainingQty -= qtyToUse;
                    }
                    if (remainingQty > 0) {
                        totalCost += remainingQty * baseCost;
                    }
                    unitCost = (p.stock || 0) > 0 ? totalCost / p.stock : baseCost;
                }
            }
            return {
                ...p,
                valuatedCost: unitCost,
                totalValue: (p.stock || 0) * unitCost
            };
        }));
        const totalValue = valuatedProducts.reduce((sum, p) => sum + p.totalValue, 0);
        const totalUnits = valuatedProducts.reduce((sum, p) => sum + (p.stock || 0), 0);
        const methodNames = {
            promedio: 'Costo Promedio Ponderado',
            fifo: 'FIFO (Primera Entrada, Primera Salida)',
            lifo: 'LIFO (Última Entrada, Primera Salida)'
        };
        return `
            <div class="alert alert-info" style="margin-bottom: var(--space-4);">
                <i class="fas fa-info-circle"></i>
                <strong>Método:</strong> ${methodNames[method] || method}
            </div>
            <div class="dashboard-grid" style="margin-bottom: var(--space-4);">
                <div class="kpi-card">
                    <div class="kpi-header">
                        <div class="kpi-icon primary"><i class="fas fa-boxes"></i></div>
                    </div>
                    <div class="kpi-value">${valuatedProducts.length}</div>
                    <div class="kpi-label">Productos en Stock</div>
                </div>
                <div class="kpi-card">
                    <div class="kpi-header">
                        <div class="kpi-icon success"><i class="fas fa-cubes"></i></div>
                    </div>
                    <div class="kpi-value">${Formatters.number(totalUnits)}</div>
                    <div class="kpi-label">Unidades Totales</div>
                </div>
                <div class="kpi-card span-2">
                    <div class="kpi-header">
                        <div class="kpi-icon warning"><i class="fas fa-dollar-sign"></i>
</div>
                    </div>
                    <div class="kpi-value">${Formatters.currency(totalValue)}</div>
                    <div class="kpi-label">Valor Total del Inventario</div>
                </div>
            </div>
            <div class="card">
                <div class="card-body" style="padding: 0;">
                    <div class="table-container">
                        <table class="data-table">
                            <thead>
```

```
                                <tr>
                                    <th>Código</th>
                                    <th>Producto</th>
                                    <th>Categoría</th>
                                    <th class="text-right">Stock</th>
                                    <th class="text-right">Costo Unit.
(${method.toUpperCase()})</th>
                                    <th class="text-right">Valor Total</th>
                                </tr>
                            </thead>
                            <tbody>
                                ${valuatedProducts.length === 0 ? `
                                    <tr><td colspan="6" class="table-empty">No hay
productos con stock</td></tr>
                                ` : valuatedProducts.map(p => `
                                    <tr>
                                        <td><code>${p.code}</code></td>
                                        <td>${p.name}</td>
                                        <td><span class="badge badge-neutral">${p.category
|| '-'}</span></td>
                                        <td class="text-right">${Formatters.number(p.stock
|| 0)} ${p.unit || 'UN'}</td>
                                        <td class="text-
right">${Formatters.currency(p.valuatedCost)}</td>
                                        <td class="text-right" style="font-weight:
600;">${Formatters.currency(p.totalValue)}</td>
                                    </tr>
                                `).join('')}
                            </tbody>
                            <tfoot>
                                <tr style="font-weight: 700; background: var(--neutral-
100);">
                                    <td colspan="5">TOTAL VALORIZACIÓN
(${methodNames[method]})</td>
                                    <td class="text-
right">${Formatters.currency(totalValue)}</td>
                                </tr>
                            </tfoot>
                        </table>
                    </div>
                </div>
            </div>
        `;
    },
    async init(view) {
        const company = CompanyService.getCurrent();
        if (!company) return;
        switch (view) {
            case 'productos': await this.initProducts(); break;
            case 'movimientos-stock': await this.initStockMovements(); break;
            case 'valoracion': this.initValuation(); break;
        }
    },
    async initProducts() {
        const company = CompanyService.getCurrent();
        const products = await DB.getByIndex('products', 'companyId', company.id);
        const categories = [...new Set(products.map(p => p.category).filter(c =>
c))].sort();
        const categorySelect = document.getElementById('filter-category');
        if (categorySelect) {
            categorySelect.innerHTML = '<option value="">Todas las categorías</option>' +
                categories.map(c => `<option value="${c}">${c}</option>`).join('');
        }
        this.allProducts = products;
        this.renderProductsTable(products);
        document.getElementById('btn-new-product')?.addEventListener('click', () =>
this.showProductModal());
        document.getElementById('btn-import-products')?.addEventListener('click', () =>
this.importProducts());
        document.getElementById('search-products')?.addEventListener('input', (e) => {
            this.filterProducts();
        });
        document.getElementById('filter-category')?.addEventListener('change', (e) => {
            this.filterProducts();
        });
    },
    filterProducts() {
        const search = (document.getElementById('search-products')?.value ||
'').toLowerCase();
        const category = document.getElementById('filter-category')?.value || '';
        let filtered = this.allProducts || [];
```

```
        if (search) {
            filtered = filtered.filter(p =>
                p.code?.toLowerCase().includes(search) ||
                p.name?.toLowerCase().includes(search) ||
                p.category?.toLowerCase().includes(search)
            );
        }
        if (category) {
            filtered = filtered.filter(p => p.category === category);
        }
        this.renderProductsTable(filtered);
    },
    renderProductsTable(products) {
        DataTable.create('products-table', {
            columns: [
                { key: 'code', label: 'Código' },
                { key: 'name', label: 'Nombre' },
                { key: 'category', label: 'Categoría' },
                { key: 'stock', label: 'Stock', class: 'text-right', render: (v, row) =>
row.isService ? '-' : Formatters.number(v || 0) },
                { key: 'cost', label: 'Costo', format: 'currency', class: 'text-right' },
                { key: 'price', label: 'Precio', format: 'currency', class: 'text-right'
},
                {
                    key: 'margin', label: 'Margen', class: 'text-right', render: (v, row)
=> {
                        if (!row.price || row.price === 0) return '-';
                        const margin = ((row.price - row.cost) / row.price * 100);
                        return `<span class="${margin > 0 ? 'positive' :
'negative'}">${margin.toFixed(1)}%</span>`;
                    }
                }
            ],
            data: products,
            actions: [
                { name: 'edit', label: 'Editar', icon: 'fas fa-edit', handler: (row) =>
this.showProductModal(row.id) },
                { name: 'delete', label: 'Eliminar', icon: 'fas fa-trash', handler: (row)
=> this.deleteProduct(row.id) }
            ],
            emptyMessage: 'No hay productos registrados'
        });
    },
    async showProductModal(productId = null) {
        const isEdit = !!productId;
        let data = {};
        const company = CompanyService.getCurrent();
        if (isEdit) {
            data = await DB.get('products', productId);
        }
        const allProducts = await DB.getByIndex('products', 'companyId', company.id);
        const categories = [...new Set(allProducts.map(p => p.category).filter(c =>
c))].sort();
        const defaultCategories = ['Tecnología', 'Oficina', 'Materiales', 'Servicios',
'Otros'];
        defaultCategories.forEach(cat => {
            if (!categories.includes(cat)) categories.push(cat);
        });
        categories.sort();
        Modal.open({
            title: isEdit ? 'Editar Producto' : 'Nuevo Producto',
            content: `
                <form id="product-form">
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Código <span class="text-danger">*
</span></label>
                            <input type="text" class="form-control" name="code"
placeholder="PROD001" value="${data.code || ''}" required>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Nombre <span class="text-danger">*
</span></label>
                            <input type="text" class="form-control" name="name"
placeholder="Nombre del producto" value="${data.name || ''}" required>
                        </div>
                    </div>
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Categoría</label>
                            <div style="display: flex; gap: var(--space-2);">
```

```
                                <select class="form-control" name="category" id="category-
select" style="flex: 1;">
                                    <option value="">Sin categoría</option>
                                    ${categories.map(c => `<option value="${c}"
${data.category === c ? 'selected' : ''}>${c}</option>`).join('')}
                                    <option value="__new__">+ Agregar nueva...</option>
                                </select>
                            </div>
                            <input type="text" class="form-control" id="new-category-
input" placeholder="Nueva categoría..." style="margin-top: var(--space-2); display:
none;">
                        </div>
                        <div class="form-group">
                            <label class="form-label">Unidad</label>
                            <select class="form-control" name="unit">
                                <option value="UN" ${(data.unit || 'UN') === 'UN' ?
'selected' : ''}>UN - Unidad</option>
                                <option value="KG" ${data.unit === 'KG' ? 'selected' :
''}>KG - Kilogramo</option>
                                <option value="LT" ${data.unit === 'LT' ? 'selected' :
''}>LT - Litro</option>
                                <option value="MT" ${data.unit === 'MT' ? 'selected' :
''}>MT - Metro</option>
                                <option value="CJ" ${data.unit === 'CJ' ? 'selected' :
''}>CJ - Caja</option>
                                <option value="PQ" ${data.unit === 'PQ' ? 'selected' :
''}>PQ - Paquete</option>
                            </select>
                        </div>
                    </div>
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Costo <span class="text-danger">*
</span></label>
                            <input type="number" class="form-control" name="cost"
value="${data.cost || 0}" min="0" step="0.01" required>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Precio de Venta <span class="text-
danger">*</span></label>
                            <input type="number" class="form-control" name="price"
value="${data.price || 0}" min="0" step="0.01" required>
                        </div>
                    </div>
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Stock Inicial</label>
                            <input type="number" class="form-control" name="stock"
value="${data.stock || 0}" min="0">
                        </div>
                        <div class="form-group">
                            <label class="form-label">Stock Mínimo</label>
                            <input type="number" class="form-control" name="minStock"
value="${data.minStock || 0}" min="0">
                        </div>
                    </div>
                    <div class="form-group">
                        <label class="form-check">
                            <input type="checkbox" name="isService" ${data.isService ?
'checked' : ''}>
                            <span>Es un servicio (sin control de stock)</span>
                        </label>
                    </div>
                </form>
            `,
            footer: `
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-primary" id="btn-save-product"><i class="fas fa-
save"></i> Guardar</button>
            `
        });
        document.getElementById('category-select')?.addEventListener('change', (e) => {
            const newCatInput = document.getElementById('new-category-input');
            if (e.target.value === '__new__') {
                newCatInput.style.display = 'block';
                newCatInput.focus();
            } else {
                newCatInput.style.display = 'none';
            }
        });
```

```
        document.getElementById('btn-save-product')?.addEventListener('click', async () =>
{
            const form = document.getElementById('product-form');
            const formData = new FormData(form);
            const code = formData.get('code');
            const name = formData.get('name');
            if (!code) { Toast.error('El código es requerido'); return; }
            if (!name) { Toast.error('El nombre es requerido'); return; }
            let category = formData.get('category');
            if (category === '__new__') {
                category = document.getElementById('new-category-input').value;
            }
            const productData = {
                code,
                name,
                category,
                unit: formData.get('unit'),
                cost: parseFloat(formData.get('cost')) || 0,
                price: parseFloat(formData.get('price')) || 0,
                stock: parseInt(formData.get('stock')) || 0,
                minStock: parseInt(formData.get('minStock')) || 0,
                isService: formData.get('isService') === 'on'
            };
            try {
                if (isEdit) {
                    await DB.update('products', { ...data, ...productData });
                    Toast.success('Producto actualizado');
                } else {
                    await DB.add('products', { companyId: company.id, ...productData,
status: 'active' });
                    Toast.success('Producto creado');
                }
                Modal.close();
                App.navigate('inventario', 'productos');
            } catch (err) {
                Toast.error('Error: ' + err.message);
            }
        });
    },
    async deleteProduct(id) {
        const confirm = await Modal.confirm({
            title: 'Eliminar Producto',
            message: '¿Está seguro de eliminar este producto?',
            confirmText: 'Eliminar',
            confirmClass: 'btn-danger'
        });
        if (confirm) {
            await DB.delete('products', id);
            Toast.success('Producto eliminado');
            App.navigate('inventario', 'productos');
        }
    },
    async initStockMovements() {
        const company = CompanyService.getCurrent();
        const allMovements = await DB.getByIndex('stockMovements', 'companyId',
company.id);
        for (let mov of allMovements) {
            const product = await DB.get('products', mov.productId);
            mov.productName = product?.name || 'Producto eliminado';
            mov.productCode = product?.code || '-';
        }
        allMovements.sort((a, b) => new Date(b.date) - new Date(a.date));
        this.stockMovements = allMovements;
        const self = this;
        const applyFilters = function () {
            const dateFrom = document.getElementById('filter-date-from')?.value || '';
            const dateTo = document.getElementById('filter-date-to')?.value || '';
            const typeFilter = document.getElementById('filter-type')?.value || '';
            let filteredMovements = [...self.stockMovements];
            if (dateFrom) {
                filteredMovements = filteredMovements.filter(m => m.date >= dateFrom);
            }
            if (dateTo) {
                filteredMovements = filteredMovements.filter(m => m.date <= dateTo);
            }
            if (typeFilter) {
                filteredMovements = filteredMovements.filter(m => m.type === typeFilter);
            }
            DataTable.create('movements-table', {
                columns: [
                    { key: 'date', label: 'Fecha', format: 'date' },
```

```
                    { key: 'productCode', label: 'Código' },
                    { key: 'productName', label: 'Producto' },
                    {
                        key: 'type', label: 'Tipo', render: (v) => {
                            const types = { in: 'Entrada', out: 'Salida', adjustment:
'Ajuste' };
                            const classes = { in: 'success', out: 'error', adjustment:
'warning' };
                            return `<span class="badge badge-${classes[v] ||
'neutral'}">${types[v] || v}</span>`;
                        }
                    },
                    {
                        key: 'quantity', label: 'Cantidad', class: 'text-right',
                        render: (v, row) => {
                            const sign = row.type === 'in' ? '+' : (row.type === 'out' ?
'-' : '');
                            const color = row.type === 'in' ? 'color: var(--success-500)'
: (row.type === 'out' ? 'color: var(--error-500)' : '');
                            return `<span style="${color}; font-weight: 600;">${sign}${v}
</span>`;
                        }
                    },
                    { key: 'reference', label: 'Referencia' },
                    { key: 'description', label: 'Descripción' }
                ],
                data: filteredMovements,
                emptyMessage: 'No hay movimientos de stock'
            });
        };
        applyFilters();
        const filterDateFrom = document.getElementById('filter-date-from');
        const filterDateTo = document.getElementById('filter-date-to');
        const filterType = document.getElementById('filter-type');
        if (filterDateFrom) {
            filterDateFrom.removeEventListener('change', filterDateFrom._handler);
            filterDateFrom._handler = applyFilters;
            filterDateFrom.addEventListener('change', applyFilters);
        }
        if (filterDateTo) {
            filterDateTo.removeEventListener('change', filterDateTo._handler);
            filterDateTo._handler = applyFilters;
            filterDateTo.addEventListener('change', applyFilters);
        }
        if (filterType) {
            filterType.removeEventListener('change', filterType._handler);
            filterType._handler = applyFilters;
            filterType.addEventListener('change', applyFilters);
        }
        document.getElementById('btn-adjustment')?.addEventListener('click', () => {
            this.showAdjustmentModal();
        });
    },
    async showAdjustmentModal() {
        const company = CompanyService.getCurrent();
        const products = await DB.getByIndex('products', 'companyId', company.id);
        const physicalProducts = products.filter(p => !p.isService);
        if (physicalProducts.length === 0) {
            Toast.warning('No hay productos físicos para ajustar');
            return;
        }
        Modal.open({
            title: 'Ajuste de Inventario',
            content: `
                <form id="adjustment-form">
                    <div class="form-group">
                        <label class="form-label">Producto <span class="text-danger">*
</span></label>
                        <select class="form-control" name="productId" id="adjustment-
product" required>
                            <option value="">Seleccione producto...</option>
                            ${physicalProducts.map(p => `<option value="${p.id}" data-
stock="${p.stock || 0}">${p.code} - ${p.name} (Stock actual: ${p.stock || 0})
</option>`).join('')}
                        </select>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Stock Actual</label>
                        <input type="number" class="form-control" id="current-stock"
disabled value="0">
                    </div>
```

```
                    <div class="form-group">
                        <label class="form-label">Stock Real (Conteo Físico) <span
class="text-danger">*</span></label>
                        <input type="number" class="form-control" name="newStock" id="new-
stock" min="0" required>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Diferencia</label>
                        <input type="text" class="form-control" id="difference" disabled
value="0">
                    </div>
                    <div class="form-group">
                        <label class="form-label">Motivo del Ajuste <span class="text-
danger">*</span></label>
                        <select class="form-control" name="reason" required>
                            <option value="">Seleccione...</option>
                            <option value="Conteo físico">Conteo físico</option>
                            <option value="Merma">Merma / Deterioro</option>
                            <option value="Robo/Pérdida">Robo / Pérdida</option>
                            <option value="Error de registro">Error de registro
previo</option>
                            <option value="Devolución">Devolución</option>
                            <option value="Otro">Otro</option>
                        </select>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Descripción</label>
                        <textarea class="form-control" name="description" rows="2"
placeholder="Detalles adicionales..."></textarea>
                    </div>
                </form>
            `,
            footer: `
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-primary" id="btn-save-adjustment"><i class="fas fa-
save"></i> Guardar Ajuste</button>
            `
        });
        const productSelect = document.getElementById('adjustment-product');
        const currentStockInput = document.getElementById('current-stock');
        const newStockInput = document.getElementById('new-stock');
        const differenceInput = document.getElementById('difference');
        productSelect.addEventListener('change', () => {
            const stock = parseInt(productSelect.selectedOptions[0]?.dataset.stock || 0);
            currentStockInput.value = stock;
            newStockInput.value = stock;
            differenceInput.value = '0';
        });
        newStockInput.addEventListener('input', () => {
            const current = parseInt(currentStockInput.value) || 0;
            const newVal = parseInt(newStockInput.value) || 0;
            const diff = newVal - current;
            differenceInput.value = diff > 0 ? `+${diff}` : diff;
            differenceInput.style.color = diff > 0 ? 'var(--success-500)' : (diff < 0 ?
'var(--error-500)' : '');
        });
        document.getElementById('btn-save-adjustment')?.addEventListener('click', async ()
=> {
            await this.saveAdjustment();
        });
    },
    async saveAdjustment() {
        const form = document.getElementById('adjustment-form');
        const formData = new FormData(form);
        const productId = formData.get('productId');
        const newStock = parseInt(formData.get('newStock'));
        const reason = formData.get('reason');
        const description = formData.get('description');
        if (!productId) { Toast.error('Seleccione un producto'); return; }
        if (isNaN(newStock) || newStock < 0) { Toast.error('Ingrese un stock válido');
return; }
        if (!reason) { Toast.error('Seleccione un motivo'); return; }
        const company = CompanyService.getCurrent();
        const product = await DB.get('products', productId);
        const currentStock = product.stock || 0;
        const difference = newStock - currentStock;
        if (difference === 0) {
            Toast.warning('No hay diferencia de stock');
            return;
        }
```

```
        const loading = Toast.loading('Registrando ajuste...');
        try {
            await DB.update('products', { ...product, stock: newStock });
            const adjustmentRef = `AJ-${Date.now()}`;
            await DB.add('stockMovements', {
                id: Helpers.generateId(),
                companyId: company.id,
                productId,
                type: 'adjustment',
                quantity: Math.abs(difference),
                date: Helpers.getCurrentDate(),
                reference: adjustmentRef,
                description: `${reason}: ${description || 'Ajuste de inventario'}. Stock
anterior: ${currentStock}, Stock nuevo: ${newStock}`,
                createdAt: new Date().toISOString()
            });
            const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
            const inventoryAccount = accounts.find(a =>
                a.type === 'asset' &&
                (a.name.toLowerCase().includes('mercadería') ||
                    a.name.toLowerCase().includes('inventario') ||
                    a.code === '1.1.07')
            );
            let adjustmentAccount;
            if (difference < 0) {
                adjustmentAccount = accounts.find(a =>
                    a.type === 'expense' &&
                    (a.name.toLowerCase().includes('pérdida') ||
                        a.name.toLowerCase().includes('merma') ||
                        a.name.toLowerCase().includes('ajuste'))
                );
                if (!adjustmentAccount) {
                    adjustmentAccount = accounts.find(a => a.type === 'expense');
                }
            } else {
                adjustmentAccount = accounts.find(a =>
                    a.type === 'income' &&
                    (a.name.toLowerCase().includes('ganancia') ||
                        a.name.toLowerCase().includes('ajuste') ||
                        a.name.toLowerCase().includes('sobrante'))
                );
                if (!adjustmentAccount) {
                    adjustmentAccount = accounts.find(a => a.type === 'income');
                }
            }
            if (inventoryAccount && adjustmentAccount) {
                const adjustmentValue = Math.abs(difference) * (product.cost || 0);
                if (adjustmentValue > 0) {
                    const entryNumber = await AccountingService.getNextEntryNumber();
                    const journalEntry = {
                        id: Helpers.generateId(),
                        companyId: company.id,
                        number: entryNumber,
                        date: Helpers.getCurrentDate(),
                        description: `Ajuste de inventario: ${product.name} - ${reason}`,
                        reference: adjustmentRef,
                        status: 'posted',
                        totalDebit: adjustmentValue,
                        totalCredit: adjustmentValue,
                        createdAt: new Date().toISOString()
                    };
                    await DB.add('journalEntries', journalEntry);
                    if (difference < 0) {
                        await DB.add('journalLines', {
                            id: Helpers.generateId(),
                            entryId: journalEntry.id,
                            accountId: adjustmentAccount.id,
                            description: `Pérdida inventario: ${product.name}`,
                            debit: adjustmentValue,
                            credit: 0
                        });
                        await DB.add('journalLines', {
                            id: Helpers.generateId(),
                            entryId: journalEntry.id,
                            accountId: inventoryAccount.id,
                            description: `Baja inventario: ${product.name}`,
                            debit: 0,
                            credit: adjustmentValue
                        });
                    } else {
                        await DB.add('journalLines', {
```

```
                            id: Helpers.generateId(),
                            entryId: journalEntry.id,
                            accountId: inventoryAccount.id,
                            description: `Alta inventario: ${product.name}`,
                            debit: adjustmentValue,
                            credit: 0
                        });
                        await DB.add('journalLines', {
                            id: Helpers.generateId(),
                            entryId: journalEntry.id,
                            accountId: adjustmentAccount.id,
                            description: `Ganancia inventario: ${product.name}`,
                            debit: 0,
                            credit: adjustmentValue
                        });
                    }
                }
            }
            Modal.close();
            loading.success(`Ajuste registrado: ${difference > 0 ? '+' : ''}${difference}
unidades`);
            App.navigate('inventario', 'movimientos-stock');
        } catch (err) {
            loading.error('Error: ' + err.message);
        }
    },
    initValuation() {
        document.getElementById('btn-calculate')?.addEventListener('click', async () => {
            const method = document.getElementById('valuation-method')?.value ||
'promedio';
            const content = document.getElementById('valuation-content');
            content.innerHTML = '<div style="text-align: center; padding: var(--space-
8);"><i class="fas fa-spinner fa-spin fa-2x"></i><p>Calculando valorización...</p></div>';
            content.innerHTML = await this.renderValuationTable(method);
            Toast.success(`Valorización calculada (${method.toUpperCase()})`);
        });
        document.getElementById('btn-export-valuation')?.addEventListener('click', async
() => {
            const method = document.getElementById('valuation-method')?.value ||
'promedio';
            const company = CompanyService.getCurrent();
            const products = await DB.getByIndex('products', 'companyId', company.id);
            const movements = await DB.getByIndex('stockMovements', 'companyId',
company.id);
            const methodNames = {
                promedio: 'Costo Promedio',
                fifo: 'FIFO',
                lifo: 'LIFO'
            };
            const data = products.filter(p => !p.isService && (p.stock || 0) > 0).map(p =>
{
                const productMovements = movements.filter(m => m.productId === p.id &&
m.type === 'in');
                let unitCost = p.cost || 0;
                if (method === 'promedio' && productMovements.length > 0) {
                    const totalCost = productMovements.reduce((sum, m) => sum +
(m.quantity * (m.unitCost || p.cost)), 0);
                    const totalQty = productMovements.reduce((sum, m) => sum + m.quantity,
0);
                    unitCost = totalQty > 0 ? totalCost / totalQty : p.cost;
                }
                return {
                    codigo: p.code,
                    nombre: p.name,
                    categoria: p.category,
                    stock: p.stock || 0,
                    metodo: methodNames[method],
                    costo_unitario: unitCost,
                    valor_total: (p.stock || 0) * unitCost
                };
            });
            await ExportService.toExcel(data,
`valorizacion_${method}_${Helpers.getCurrentDate()}`);
            Toast.success('Exportado correctamente');
        });
    },
    async importProducts() {
        Modal.open({
            title: 'Importar Productos desde Excel',
            size: 'large',
            content: `
```

```
                <div style="margin-bottom: var(--space-4);">
                    <div class="alert alert-info">
                        <i class="fas fa-info-circle"></i>
                        <strong>Formato requerido:</strong> El archivo Excel debe tener
las siguientes columnas:
                        <ul style="margin-top: var(--space-2); margin-left: var(--space-
4);">
                            <li><strong>codigo</strong> - Código único del producto
(requerido)</li>
                            <li><strong>nombre</strong> - Nombre del producto (requerido)
</li>
                            <li><strong>categoria</strong> - Categoría del producto</li>
                            <li><strong>unidad</strong> - Unidad de medida (UN, KG, LT,
etc.)</li>
                            <li><strong>costo</strong> - Costo unitario</li>
                            <li><strong>precio</strong> - Precio de venta</li>
                            <li><strong>stock</strong> - Stock inicial</li>
                            <li><strong>stock_minimo</strong> - Stock mínimo</li>
                        </ul>
                    </div>
                </div>
                <div class="form-group">
                    <label class="form-label">Seleccionar archivo Excel (.xlsx, .xls,
.csv)</label>
                    <input type="file" class="form-control" id="import-file"
accept=".xlsx,.xls,.csv">
                </div>
                <div id="import-preview" style="display: none;">
                    <h4 style="margin: var(--space-4) 0 var(--space-2);">Vista previa</h4>
                    <div class="table-container" style="max-height: 300px; overflow-y:
auto;">
                        <table class="data-table" id="preview-table">
                            <thead></thead>
                            <tbody></tbody>
                        </table>
                    </div>
                    <div id="import-summary" style="margin-top: var(--space-3);"></div>
                </div>
            `,
            footer: `
                <button class="btn btn-outline" id="btn-download-template"><i class="fas
fa-download"></i> Descargar Plantilla</button>
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-primary" id="btn-import" disabled><i class="fas fa-
upload"></i> Importar</button>
            `
        });
        let importData = [];
        document.getElementById('btn-download-template')?.addEventListener('click', async
() => {
            const template = [
                { codigo: 'PROD001', nombre: 'Producto Ejemplo 1', categoria:
'Tecnología', unidad: 'UN', costo: 10000, precio: 15000, stock: 100, stock_minimo: 10 },
                { codigo: 'PROD002', nombre: 'Producto Ejemplo 2', categoria: 'Oficina',
unidad: 'CJ', costo: 5000, precio: 8000, stock: 50, stock_minimo: 5 },
                { codigo: 'SERV001', nombre: 'Servicio Ejemplo', categoria: 'Servicios',
unidad: 'UN', costo: 0, precio: 25000, stock: 0, stock_minimo: 0 }
            ];
            await ExportService.toExcel(template, 'plantilla_productos');
            Toast.success('Plantilla descargada');
        });
        document.getElementById('import-file')?.addEventListener('change', async (e) => {
            const file = e.target.files[0];
            if (!file) return;
            const loading = Toast.loading('Procesando archivo...');
            try {
                const data = await this.readExcelFile(file);
                importData = this.validateImportData(data);
                const previewDiv = document.getElementById('import-preview');
                const thead = document.querySelector('#preview-table thead');
                const tbody = document.querySelector('#preview-table tbody');
                const summary = document.getElementById('import-summary');
                if (importData.valid.length > 0) {
                    thead.innerHTML = `<tr>
                        <th>Código</th>
                        <th>Nombre</th>
                        <th>Categoría</th>
                        <th>Unidad</th>
                        <th>Costo</th>
                        <th>Precio</th>
```

```
                        <th>Stock</th>
                        <th>Estado</th>
                    </tr>`;
                    tbody.innerHTML = importData.valid.slice(0, 20).map(p => `
                        <tr>
                            <td><code>${p.codigo}</code></td>
                            <td>${p.nombre}</td>
                            <td>${p.categoria || '-'}</td>
                            <td>${p.unidad || 'UN'}</td>
                            <td class="text-right">${Formatters.currency(p.costo || 0)}
</td>
                            <td class="text-right">${Formatters.currency(p.precio || 0)}
</td>
                            <td class="text-right">${p.stock || 0}</td>
                            <td><span class="badge badge-success">OK</span></td>
                        </tr>
                    `).join('');
                    if (importData.valid.length > 20) {
                        tbody.innerHTML += `<tr><td colspan="8" class="text-center">... y
${importData.valid.length - 20} productos más</td></tr>`;
                    }
                }
                summary.innerHTML = `
                    <div style="display: flex; gap: var(--space-4);">
                        <div class="badge badge-success" style="padding: var(--space-2)
var(--space-3);">
                            <i class="fas fa-check"></i> ${importData.valid.length}
productos válidos
                        </div>
                        ${importData.errors.length > 0 ? `
                            <div class="badge badge-error" style="padding: var(--space-2)
var(--space-3);">
                                <i class="fas fa-times"></i> ${importData.errors.length}
con errores
                            </div>
                        ` : ''}
                    </div>
                    ${importData.errors.length > 0 ? `
                        <div class="alert alert-warning" style="margin-top: var(--space-
3);">
                            <strong>Errores encontrados:</strong>
                            <ul style="margin-top: var(--space-2); margin-left: var(--
space-4);">
                                ${importData.errors.slice(0, 5).map(e => `<li>${e}
</li>`).join('')}
                                ${importData.errors.length > 5 ? `<li>... y
${importData.errors.length - 5} errores más</li>` : ''}
                            </ul>
                        </div>
                    ` : ''}
                `;
                previewDiv.style.display = 'block';
                document.getElementById('btn-import').disabled = importData.valid.length
=== 0;
                loading.success(`${importData.valid.length} productos listos para
importar`);
            } catch (err) {
                loading.error('Error al procesar archivo: ' + err.message);
            }
        });
        document.getElementById('btn-import')?.addEventListener('click', async () => {
            if (importData.valid.length === 0) return;
            const confirm = await Modal.confirm({
                title: 'Confirmar Importación',
                message: `¿Importar ${importData.valid.length} productos?`,
                confirmText: 'Importar'
            });
            if (!confirm) return;
            const loading = Toast.loading('Importando productos...');
            try {
                const company = CompanyService.getCurrent();
                let imported = 0;
                let updated = 0;
                for (const p of importData.valid) {
                    const existing = (await DB.getByIndex('products', 'companyId',
company.id))
                        .find(prod => prod.code === p.codigo);
                    const productData = {
                        code: p.codigo,
                        name: p.nombre,
                        category: p.categoria || '',
```

```
                        unit: p.unidad || 'UN',
                        cost: parseFloat(p.costo) || 0,
                        price: parseFloat(p.precio) || 0,
                        stock: parseInt(p.stock) || 0,
                        minStock: parseInt(p.stock_minimo) || 0,
                        isService: (p.unidad === 'SERV' || p.stock === 0 && p.costo ===
0),
                        status: 'active'
                    };
                    if (existing) {
                        await DB.update('products', { ...existing, ...productData });
                        updated++;
                    } else {
                        await DB.add('products', { id: Helpers.generateId(), companyId:
company.id, ...productData });
                        imported++;
                    }
                }
                Modal.close();
                loading.success(`Importación completada: ${imported} nuevos, ${updated}
actualizados`);
                App.navigate('inventario', 'productos');
            } catch (err) {
                loading.error('Error: ' + err.message);
            }
        });
    },
    async readExcelFile(file) {
        return new Promise((resolve, reject) => {
            const reader = new FileReader();
            reader.onload = async (e) => {
                try {
                    const data = new Uint8Array(e.target.result);
                    const workbook = XLSX.read(data, { type: 'array' });
                    const firstSheet = workbook.Sheets[workbook.SheetNames[0]];
                    const jsonData = XLSX.utils.sheet_to_json(firstSheet);
                    resolve(jsonData);
                } catch (err) {
                    reject(err);
                }
            };
            reader.onerror = reject;
            reader.readAsArrayBuffer(file);
        });
    },
    validateImportData(data) {
        const valid = [];
        const errors = [];
        data.forEach((row, index) => {
            const rowNum = index + 2;
            const normalized = {
                codigo: row.codigo || row.code || row.Codigo || row.Code || row.CODIGO,
                nombre: row.nombre || row.name || row.Nombre || row.Name || row.NOMBRE,
                categoria: row.categoria || row.category || row.Categoria || row.Category
|| row.CATEGORIA,
                unidad: row.unidad || row.unit || row.Unidad || row.Unit || row.UNIDAD,
                costo: row.costo || row.cost || row.Costo || row.Cost || row.COSTO,
                precio: row.precio || row.price || row.Precio || row.Price || row.PRECIO,
                stock: row.stock || row.Stock || row.STOCK,
                stock_minimo: row.stock_minimo || row.stockMinimo || row.min_stock ||
row['Stock Mínimo'] || row.STOCK_MINIMO
            };
            if (!normalized.codigo) {
                errors.push(`Fila ${rowNum}: Código es requerido`);
                return;
            }
            if (!normalized.nombre) {
                errors.push(`Fila ${rowNum}: Nombre es requerido`);
                return;
            }
            valid.push(normalized);
        });
        return { valid, errors };
    },
    async registerStockMovement(productId, type, quantity, reference, description) {
        const company = CompanyService.getCurrent();
        await DB.add('stockMovements', {
            id: Helpers.generateId(),
            companyId: company.id,
            productId,
            type,
```

```
            quantity,
            date: Helpers.getCurrentDate(),
            reference,
            description,
            createdAt: new Date().toISOString()
        });
    }
};
window.InventarioModule = InventarioModule;
```

Archivo: js\modules\reportes.js

```
const ReportesModule = {
    currentView: 'balance-general',
    async render(view = 'balance-general') {
        this.currentView = view;
        switch (view) {
            case 'balance-general': return await this.renderBalanceSheet();
            case 'balance-comprobacion': return await this.renderTrialBalance();
            case 'estado-resultados': return await this.renderIncomeStatement();
            case 'ratios-financieros': return await this.renderFinancialRatios();
            case 'flujo-caja': return this.renderCashFlow();
            case 'analisis-cuentas': return await this.renderAccountAnalysis();
            default: return '<p>Vista no encontrada</p>';
        }
    },
    async renderBalanceSheet() {
        const company = CompanyService.getCurrent();
        if (!company) return '<p>Seleccione una empresa</p>';
        const balanceData = await
AccountingService.getBalanceSheet(Helpers.getCurrentDate());
        return `
            <div class="page-header">
                <h1 class="page-title">Balance General</h1>
                <p class="page-subtitle">Estado de Situación Financiera al
${Formatters.date(new Date(), 'long')}</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="balance-date"
value="${Helpers.getCurrentDate()}">
                    <button class="btn btn-primary" id="btn-generate-balance">
                        <i class="fas fa-sync"></i> Actualizar
                    </button>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-print-balance">
                        <i class="fas fa-print"></i> Imprimir
                    </button>
                    <button class="btn btn-outline" id="btn-export-balance-pdf">
                        <i class="fas fa-file-pdf"></i> PDF
                    </button>
                    <button class="btn btn-outline" id="btn-export-balance-excel">
                        <i class="fas fa-file-excel"></i> Excel
                    </button>
                </div>
            </div>
            <div class="card">
                <div class="card-header">
                    <div class="card-title">
                        <i class="fas fa-balance-scale"></i>
                        ${company.name}
                    </div>
                    <span class="badge ${balanceData.balanced ? 'badge-success' : 'badge-
error'}">
                        ${balanceData.balanced
                ? 'Cuadrado'
                : `Descuadrado (${Formatters.currency(Math.abs(balanceData.difference ||
0))})`}
                    </span>
                </div>
                <div class="card-body">
                    <div style="display: grid; grid-template-columns: 1fr 1fr; gap: var(--
space-6);">
                        <!-- ACTIVOS -->
```

```
                        <div>
                            <h3 style="color: var(--primary-500); margin-bottom: var(--
space-4); padding-bottom: var(--space-2); border-bottom: 2px solid var(--primary-500);">
                                ACTIVOS
                            </h3>
                            <h4 style="color: var(--primary-600); margin-bottom: var(--
space-2); font-size: var(--font-size-md);">Activos Corrientes</h4>
                            ${this.renderAccountGroup(balanceData.assets.current.items,
'asset')}
                            <div style="display: flex; justify-content: space-between;
font-weight: 600; margin: var(--space-3) 0 var(--space-4); padding-bottom: var(--space-2);
border-bottom: 1px dashed var(--border-medium);">
                                <span>Total Activos Corrientes</span>

<span>${Formatters.currency(balanceData.assets.current.total)}</span>
                            </div>
                            <h4 style="color: var(--primary-600); margin-bottom: var(--
space-2); font-size: var(--font-size-md);">Activos No Corrientes</h4>
                            ${this.renderAccountGroup(balanceData.assets.nonCurrent.items,
'asset')}
                            <div style="display: flex; justify-content: space-between;
font-weight: 600; margin: var(--space-3) 0 var(--space-4); padding-bottom: var(--space-2);
border-bottom: 1px dashed var(--border-medium);">
                                <span>Total Activos No Corrientes</span>

<span>${Formatters.currency(balanceData.assets.nonCurrent.total)}</span>
                            </div>
                            <div style="display: flex; justify-content: space-between;
font-weight: 700; font-size: var(--font-size-lg); margin-top: var(--space-4); padding-top:
var(--space-2); border-top: 2px solid var(--border-medium);">
                                <span>TOTAL ACTIVOS</span>
                                <span>${Formatters.currency(balanceData.assets.total)}
</span>
                            </div>
                        </div>
                        <!-- PASIVOS Y PATRIMONIO -->
                        <div>
                            <h3 style="color: var(--error-500); margin-bottom: var(--
space-4); padding-bottom: var(--space-2); border-bottom: 2px solid var(--error-500);">
                                PASIVOS
                            </h3>
                            <h4 style="color: var(--error-600); margin-bottom: var(--
space-2); font-size: var(--font-size-md);">Pasivos Corrientes</h4>

${this.renderAccountGroup(balanceData.liabilities.current.items, 'liability')}
                            <div style="display: flex; justify-content: space-between;
font-weight: 600; margin: var(--space-3) 0 var(--space-4); padding-bottom: var(--space-2);
border-bottom: 1px dashed var(--border-medium);">
                                <span>Total Pasivos Corrientes</span>

<span>${Formatters.currency(balanceData.liabilities.current.total)}</span>
                            </div>
                            <h4 style="color: var(--error-600); margin-bottom: var(--
space-2); font-size: var(--font-size-md);">Pasivos No Corrientes</h4>

${this.renderAccountGroup(balanceData.liabilities.nonCurrent.items, 'liability')}
                            <div style="display: flex; justify-content: space-between;
font-weight: 600; margin: var(--space-3) 0 var(--space-4); padding-bottom: var(--space-2);
border-bottom: 1px dashed var(--border-medium);">
                                <span>Total Pasivos No Corrientes</span>

<span>${Formatters.currency(balanceData.liabilities.nonCurrent.total)}</span>
                            </div>
                            <div style="display: flex; justify-content: space-between;
font-weight: 600; margin-top: var(--space-3);">
                                <span>Total Pasivos</span>

<span>${Formatters.currency(balanceData.liabilities.total)}</span>
                            </div>
                            <h3 style="color: var(--success-500); margin: var(--space-6) 0
var(--space-4); padding-bottom: var(--space-2); border-bottom: 2px solid var(--success-
500);">
                                PATRIMONIO
                            </h3>
                            ${this.renderAccountGroup(balanceData.equity.items, 'equity')}
                            ${balanceData.netIncome !== 0 ? `
                                <div style="display: flex; justify-content: space-between;
padding: var(--space-2) 1rem var(--space-2) var(--space-4); color: ${balanceData.netIncome
>= 0 ? 'var(--success-600)' : 'var(--error-600)'}; font-weight: 500; border-bottom: 1px
dotted var(--border-light);">
                                    <span>Resultado del Ejercicio</span>
```

```
                                    <span>${Formatters.currency(balanceData.netIncome)}
</span>
                                </div>
                            ` : ''}
                            <div style="display: flex; justify-content: space-between;
font-weight: 600; margin-top: var(--space-3); padding-top: var(--space-2); border-top: 1px
dashed var(--border-medium);">
                                <span>Total Patrimonio</span>
                                <span>${Formatters.currency(balanceData.equity.total)}
</span>
                            </div>
                            <div style="display: flex; justify-content: space-between;
font-weight: 700; font-size: var(--font-size-lg); margin-top: var(--space-4); padding-top:
var(--space-2); border-top: 2px solid var(--border-medium);">
                                <span>TOTAL PASIVO + PATRIMONIO</span>
                                <span>${Formatters.currency(balanceData.liabilities.total
+ balanceData.equity.total)}</span>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        `;
    },
    renderAccountGroup(accounts, type) {
        const filtered = accounts.filter(a => a.balance !== 0);
        if (filtered.length === 0) {
            return '<p style="color: var(--text-tertiary); font-style: italic;">Sin
movimientos</p>';
        }
        return filtered.map(acc => `
            <div style="display: flex; justify-content: space-between; padding: var(--
space-2) 0 var(--space-2) var(--space-4); border-bottom: 1px dotted var(--border-light);">
                <span>${acc.code} ${acc.name}</span>
                <span style="font-weight:
500;">${Formatters.currency(Math.abs(acc.balance))}</span>
            </div>
        `).join('');
    },
    async renderTrialBalance() {
        const company = CompanyService.getCurrent();
        if (!company) return '<p>Seleccione una empresa</p>';
        const accounts = await AccountingService.getChartOfAccounts();
        const detailAccounts = accounts.filter(a => !a.isGroup);
        let totalDebit = 0;
        let totalCredit = 0;
        detailAccounts.forEach(acc => {
            if (acc.balance > 0) {
                if (['asset', 'expense'].includes(acc.type)) {
                    totalDebit += acc.balance;
                } else {
                    totalCredit += acc.balance;
                }
            } else if (acc.balance < 0) {
                if (['asset', 'expense'].includes(acc.type)) {
                    totalCredit += Math.abs(acc.balance);
                } else {
                    totalDebit += Math.abs(acc.balance);
                }
            }
        });
        const isBalanced = Math.abs(totalDebit - totalCredit) < 0.01;
        return `
            <div class="page-header">
                <h1 class="page-title">Balance de Comprobación</h1>
                <p class="page-subtitle">Al ${Formatters.date(new Date(), 'long')}</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="trial-balance-date"
value="${Helpers.getCurrentDate()}">
                    <button class="btn btn-primary" id="btn-generate-trial">
                        <i class="fas fa-sync"></i> Actualizar
                    </button>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-print-trial">
                        <i class="fas fa-print"></i> Imprimir
                    </button>
                    <button class="btn btn-outline" id="btn-export-trial-pdf">
                        <i class="fas fa-file-pdf"></i> PDF
```

```
                    </button>
                    <button class="btn btn-outline" id="btn-export-trial-excel">
                        <i class="fas fa-file-excel"></i> Excel
                    </button>
                </div>
            </div>
            <div class="dashboard-grid" style="margin-bottom: var(--space-4);">
                <div class="kpi-card">
                    <div class="kpi-header">
                        <div class="kpi-icon error"><i class="fas fa-arrow-left"></i>
</div>
                    </div>
                    <div class="kpi-value">${Formatters.currency(totalDebit)}</div>
                    <div class="kpi-label">Total Débitos</div>
                </div>
                <div class="kpi-card">
                    <div class="kpi-header">
                        <div class="kpi-icon success"><i class="fas fa-arrow-right"></i>
</div>
                    </div>
                    <div class="kpi-value">${Formatters.currency(totalCredit)}</div>
                    <div class="kpi-label">Total Créditos</div>
                </div>
                <div class="kpi-card span-2 ${isBalanced ? 'success' : 'error'}">
                    <div class="kpi-header">
                        <div class="kpi-icon ${isBalanced ? 'success' : 'error'}">
                            <i class="fas fa-${isBalanced ? 'check-circle' : 'exclamation-
triangle'}"></i>
                        </div>
                    </div>
                    <div class="kpi-value">${Formatters.currency(Math.abs(totalDebit -
totalCredit))}</div>
                    <div class="kpi-label">${isBalanced ? 'Balance Cuadrado ✓' :
'Diferencia (Descuadre)'}</div>
                </div>
            </div>
            <div class="card">
                <div class="card-header">
                    <div class="card-title">
                        <i class="fas fa-balance-scale-right"></i>
                        ${company.name} - Balance de Comprobación
                    </div>
                    <span class="badge ${isBalanced ? 'badge-success' : 'badge-error'}">
                        ${isBalanced ? 'Cuadrado' : 'Descuadrado'}
                    </span>
                </div>
                <div class="card-body">
                    <table class="data-table" style="width: 100%;">
                        <thead>
                            <tr>
                                <th style="width: 100px;">Código</th>
                                <th>Cuenta</th>
                                <th style="width: 100px; text-align: center;">Tipo</th>
                                <th style="width: 150px; text-align: right;">Debe</th>
                                <th style="width: 150px; text-align: right;">Haber</th>
                            </tr>
                        </thead>
                        <tbody>
                            ${detailAccounts.filter(a => a.balance !== 0).map(acc => {
            let debit = 0, credit = 0;
            if (acc.balance > 0) {
                if (['asset', 'expense'].includes(acc.type)) {
                    debit = acc.balance;
                } else {
                    credit = acc.balance;
                }
            } else if (acc.balance < 0) {
                if (['asset', 'expense'].includes(acc.type)) {
                    credit = Math.abs(acc.balance);
                } else {
                    debit = Math.abs(acc.balance);
                }
            }
            const typeLabels = {
                asset: 'Activo',
                liability: 'Pasivo',
                equity: 'Patrimonio',
                revenue: 'Ingreso',
                expense: 'Gasto'
            };
            const typeColors = {
```

```
                asset: 'var(--primary-500)',
                liability: 'var(--error-500)',
                equity: 'var(--success-500)',
                revenue: 'var(--info-500)',
                expense: 'var(--warning-500)'
            };
            return `
                                    <tr>
                                        <td><code>${acc.code}</code></td>
                                        <td>${acc.name}</td>
                                        <td style="text-align: center;">
                                            <span class="badge" style="background:
${typeColors[acc.type]}20; color: ${typeColors[acc.type]};">
                                                ${typeLabels[acc.type] || acc.type}
                                            </span>
                                        </td>
                                        <td style="text-align: right; color: var(--error-
500); font-weight: ${debit > 0 ? '600' : '400'};">
                                            ${debit > 0 ? Formatters.currency(debit) : '-
'}
                                        </td>
                                        <td style="text-align: right; color: var(--
success-500); font-weight: ${credit > 0 ? '600' : '400'};">
                                            ${credit > 0 ? Formatters.currency(credit) :
'-'}
                                        </td>
                                    </tr>
                                `;
        }).join('')}
                        </tbody>
                        <tfoot>
                            <tr style="background: var(--neutral-100); font-weight: 700;
font-size: var(--font-size-lg);">
                                <td colspan="3" style="text-align: right; padding: var(--
space-3);">TOTALES</td>
                                <td style="text-align: right; color: var(--error-600);
padding: var(--space-3);">
                                    ${Formatters.currency(totalDebit)}
                                </td>
                                <td style="text-align: right; color: var(--success-600);
padding: var(--space-3);">
                                    ${Formatters.currency(totalCredit)}
                                </td>
                            </tr>
                        </tfoot>
                    </table>
                </div>
            </div>
            <div class="card" style="margin-top: var(--space-4);">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-info-circle"></i> ¿Qué es el
Balance de Comprobación?</div>
                </div>
                <div class="card-body">
                    <p style="margin-bottom: var(--space-3);">
                        El <strong>Balance de Comprobación</strong> (o Balance de Sumas y
Saldos) es un informe contable
                        que lista todas las cuentas del libro mayor con sus saldos
deudores y acreedores.
                    </p>
                    <ul style="list-style: disc; padding-left: var(--space-6);">
                        <li><strong>Propósito:</strong> Verificar que la suma de débitos
sea igual a la suma de créditos (partida doble)</li>
                        <li><strong>Saldo Deudor:</strong> Activos y Gastos tienen saldo
normal deudor</li>
                        <li><strong>Saldo Acreedor:</strong> Pasivos, Patrimonio e
Ingresos tienen saldo normal acreedor</li>
                        <li><strong>Cuadre:</strong> Si Total Débitos = Total Créditos, el
balance está "cuadrado"</li>
                    </ul>
                </div>
            </div>
        `;
    },
    async renderFinancialRatios() {
        const company = CompanyService.getCurrent();
        if (!company) return '<p>Seleccione una empresa</p>';
        const balanceData = await
AccountingService.getBalanceSheet(Helpers.getCurrentDate());
        const range = Helpers.getYearRange();
        const incomeData = await AccountingService.getIncomeStatement(range.start,
```

```
range.end);
        const totalAssets = balanceData.assets.total;
        const totalLiabilities = balanceData.liabilities.total;
        const totalEquity = balanceData.equity.total + balanceData.netIncome;
        const netIncome = incomeData.netIncome;
        const totalRevenue = incomeData.revenues.total;
        const totalExpenses = incomeData.expenses.total;
        const currentAssets = balanceData.assets.items
            .filter(a => ['1.1.1', '1.1.2', '1.1.3', '1.1.4', '1.1.5'].some(code =>
a.code.startsWith(code)))
            .reduce((sum, a) => sum + Math.abs(a.balance), 0) || totalAssets * 0.6;
        const currentLiabilities = balanceData.liabilities.items
            .filter(a => ['2.1'].some(code => a.code.startsWith(code)))
            .reduce((sum, a) => sum + Math.abs(a.balance), 0) || totalLiabilities * 0.5;
        const inventory = balanceData.assets.items
            .filter(a => a.code.startsWith('1.1.4'))
            .reduce((sum, a) => sum + Math.abs(a.balance), 0) || currentAssets * 0.3;
        const ratios = {
            currentRatio: currentLiabilities > 0 ? currentAssets / currentLiabilities : 0,
            quickRatio: currentLiabilities > 0 ? (currentAssets - inventory) /
currentLiabilities : 0,
            debtRatio: totalAssets > 0 ? (totalLiabilities / totalAssets) * 100 : 0,
            debtToEquity: totalEquity > 0 ? totalLiabilities / totalEquity : 0,
            equityRatio: totalAssets > 0 ? (totalEquity / totalAssets) * 100 : 0,
            roa: totalAssets > 0 ? (netIncome / totalAssets) * 100 : 0,
            roe: totalEquity > 0 ? (netIncome / totalEquity) * 100 : 0,
            netProfitMargin: totalRevenue > 0 ? (netIncome / totalRevenue) * 100 : 0,
            grossProfitMargin: totalRevenue > 0 ? ((totalRevenue - totalExpenses) /
totalRevenue) * 100 : 0
        };
        const getRatioStatus = (value, type) => {
            switch (type) {
                case 'currentRatio':
                    if (value >= 2) return { status: 'success', text: 'Excelente', icon:
'check-circle' };
                    if (value >= 1.5) return { status: 'warning', text: 'Bueno', icon:
'exclamation-circle' };
                    if (value >= 1) return { status: 'warning', text: 'Aceptable', icon:
'exclamation-circle' };
                    return { status: 'error', text: 'Bajo', icon: 'times-circle' };
                case 'quickRatio':
                    if (value >= 1) return { status: 'success', text: 'Excelente', icon:
'check-circle' };
                    if (value >= 0.7) return { status: 'warning', text: 'Aceptable', icon:
'exclamation-circle' };
                    return { status: 'error', text: 'Bajo', icon: 'times-circle' };
                case 'debtRatio':
                    if (value <= 40) return { status: 'success', text: 'Bajo riesgo',
icon: 'check-circle' };
                    if (value <= 60) return { status: 'warning', text: 'Moderado', icon:
'exclamation-circle' };
                    return { status: 'error', text: 'Alto riesgo', icon: 'times-circle' };
                case 'roe':
                case 'roa':
                    if (value >= 15) return { status: 'success', text: 'Excelente', icon:
'check-circle' };
                    if (value >= 8) return { status: 'warning', text: 'Bueno', icon:
'exclamation-circle' };
                    if (value > 0) return { status: 'warning', text: 'Bajo', icon:
'exclamation-circle' };
                    return { status: 'error', text: 'Negativo', icon: 'times-circle' };
                default:
                    return { status: 'info', text: '-', icon: 'info-circle' };
            }
        };
        const currentStatus = getRatioStatus(ratios.currentRatio, 'currentRatio');
        const quickStatus = getRatioStatus(ratios.quickRatio, 'quickRatio');
        const debtStatus = getRatioStatus(ratios.debtRatio, 'debtRatio');
        const roeStatus = getRatioStatus(ratios.roe, 'roe');
        const roaStatus = getRatioStatus(ratios.roa, 'roa');
        return `
            <div class="page-header">
                <h1 class="page-title">Ratios Financieros</h1>
                <p class="page-subtitle">Análisis de Indicadores Financieros -
${company.name}</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="ratios-date"
value="${Helpers.getCurrentDate()}">
                    <button class="btn btn-primary" id="btn-generate-ratios">
```

```
                        <i class="fas fa-sync"></i> Actualizar
                    </button>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-print-ratios">
                        <i class="fas fa-print"></i> Imprimir
                    </button>
                    <button class="btn btn-outline" id="btn-export-ratios-pdf">
                        <i class="fas fa-file-pdf"></i> PDF
                    </button>
                </div>
            </div>
            <!-- Ratios de Liquidez -->
            <div class="card" style="margin-bottom: var(--space-4);">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-tint"></i> Ratios de
Liquidez</div>
                    <span class="badge badge-info">Capacidad de pago a corto plazo</span>
                </div>
                <div class="card-body">
                    <div class="dashboard-grid">
                        <div class="kpi-card ${currentStatus.status}">
                            <div class="kpi-header">
                                <div class="kpi-icon ${currentStatus.status}"><i
class="fas fa-${currentStatus.icon}"></i></div>
                            </div>
                            <div class="kpi-value">${ratios.currentRatio.toFixed(2)}</div>
                            <div class="kpi-label">Razón Corriente</div>
                            <div class="kpi-change
${currentStatus.status}">${currentStatus.text}</div>
                        </div>
                        <div class="kpi-card ${quickStatus.status}">
                            <div class="kpi-header">
                                <div class="kpi-icon ${quickStatus.status}"><i class="fas
fa-${quickStatus.icon}"></i></div>
                            </div>
                            <div class="kpi-value">${ratios.quickRatio.toFixed(2)}</div>
                            <div class="kpi-label">Prueba Ácida</div>
                            <div class="kpi-change
${quickStatus.status}">${quickStatus.text}</div>
                        </div>
                        <div class="kpi-card span-2">
                            <div class="kpi-header">
                                <div class="kpi-icon info"><i class="fas fa-info-circle">
</i></div>
                            </div>
                            <div style="padding: var(--space-2); font-size: var(--font-
size-sm);">
                                <p><strong>Razón Corriente:</strong> Activo Corr. / Pasivo
Corr. (Ideal: ≥ 1.5)</p>
                                <p><strong>Prueba Ácida:</strong> (Activo Corr. -
Inventario) / Pasivo Corr. (Ideal: ≥ 1)</p>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
            <!-- Ratios de Solvencia -->
            <div class="card" style="margin-bottom: var(--space-4);">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-shield-alt"></i> Ratios de
Solvencia / Endeudamiento</div>
                    <span class="badge badge-warning">Estructura de capital</span>
                </div>
                <div class="card-body">
                    <div class="dashboard-grid">
                        <div class="kpi-card ${debtStatus.status}">
                            <div class="kpi-header">
                                <div class="kpi-icon ${debtStatus.status}"><i class="fas
fa-${debtStatus.icon}"></i></div>
                            </div>
                            <div class="kpi-value">${ratios.debtRatio.toFixed(1)}%</div>
                            <div class="kpi-label">Nivel de Endeudamiento</div>
                            <div class="kpi-change
${debtStatus.status}">${debtStatus.text}</div>
                        </div>
                        <div class="kpi-card">
                            <div class="kpi-header">
                                <div class="kpi-icon info"><i class="fas fa-balance-
scale">></i></div>
                            </div>
```

```
                            <div class="kpi-value">${ratios.debtToEquity.toFixed(2)}</div>
                            <div class="kpi-label">Deuda / Patrimonio</div>
                        </div>
                        <div class="kpi-card">
                            <div class="kpi-header">
                                <div class="kpi-icon success"><i class="fas fa-
percentage"></i></div>
                            </div>
                            <div class="kpi-value">${ratios.equityRatio.toFixed(1)}%</div>
                            <div class="kpi-label">Autonomía Financiera</div>
                        </div>
                        <div class="chart-card">
                            <div class="chart-header">
                                <div class="chart-title">Estructura de Capital</div>
                            </div>
                            <div class="chart-body" style="height: 200px;">
                                <canvas id="capital-structure-chart"></canvas>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
            <!-- Ratios de Rentabilidad -->
            <div class="card" style="margin-bottom: var(--space-4);">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-chart-line"></i> Ratios de
Rentabilidad</div>
                    <span class="badge badge-success">Generación de resultados</span>
                </div>
                <div class="card-body">
                    <div class="dashboard-grid">
                        <div class="kpi-card ${roaStatus.status}">
                            <div class="kpi-header">
                                <div class="kpi-icon ${roaStatus.status}"><i class="fas
fa-${roaStatus.icon}"></i></div>
                            </div>
                            <div class="kpi-value">${ratios.roa.toFixed(2)}%</div>
                            <div class="kpi-label">ROA (Retorno sobre Activos)</div>
                            <div class="kpi-change ${roaStatus.status}">${roaStatus.text}
</div>
                        </div>
                        <div class="kpi-card ${roeStatus.status}">
                            <div class="kpi-header">
                                <div class="kpi-icon ${roeStatus.status}"><i class="fas
fa-${roeStatus.icon}"></i></div>
                            </div>
                            <div class="kpi-value">${ratios.roe.toFixed(2)}%</div>
                            <div class="kpi-label">ROE (Retorno sobre Patrimonio)</div>
                            <div class="kpi-change ${roeStatus.status}">${roeStatus.text}
</div>
                        </div>
                        <div class="kpi-card">
                            <div class="kpi-header">
                                <div class="kpi-icon info"><i class="fas fa-percentage">
</i></div>
                            </div>
                            <div class="kpi-value">${ratios.netProfitMargin.toFixed(2)}%
</div>
                            <div class="kpi-label">Margen de Resultado Neto</div>
                        </div>
                        <div class="chart-card">
                            <div class="chart-header">
                                <div class="chart-title">Indicadores de Rentabilidad</div>
                            </div>
                            <div class="chart-body" style="height: 200px;">
                                <canvas id="profitability-chart"></canvas>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
            <!-- Gráfico Resumen -->
            <div class="dashboard-grid">
                <div class="chart-card span-2">
                    <div class="chart-header">
                        <div class="chart-title"><i class="fas fa-chart-radar"></i>
Resumen de Ratios Financieros</div>
                    </div>
                    <div class="chart-body" style="height: 300px;">
                        <canvas id="ratios-summary-chart"></canvas>
                    </div>
```

```
                </div>
                <div class="chart-card span-2">
                    <div class="chart-header">
                        <div class="chart-title"><i class="fas fa-chart-bar"></i>
Comparación de Indicadores</div>
                    </div>
                    <div class="chart-body" style="height: 300px;">
                        <canvas id="ratios-bar-chart"></canvas>
                    </div>
                </div>
            </div>
            <!-- Tabla Resumen -->
            <div class="card" style="margin-top: var(--space-4);">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-table"></i> Resumen de
Indicadores</div>
                </div>
                <div class="card-body">
                    <table class="data-table" style="width: 100%;">
                        <thead>
                            <tr>
                                <th>Categoría</th>
                                <th>Indicador</th>
                                <th>Fórmula</th>
                                <th style="text-align: right;">Valor</th>
                                <th style="text-align: center;">Interpretación</th>
                            </tr>
                        </thead>
                        <tbody>
                            <tr>
                                <td rowspan="2" style="font-weight: 600; background: var(-
-info-50);">Liquidez</td>
                                <td>Razón Corriente</td>
                                <td><code>Act. Corr. / Pas. Corr.</code></td>
                                <td style="text-align: right; font-weight:
600;">${ratios.currentRatio.toFixed(2)}</td>
                                <td style="text-align: center;"><span class="badge
badge-${currentStatus.status}">${currentStatus.text}</span></td>
                            </tr>
                            <tr>
                                <td>Prueba Ácida</td>
                                <td><code>(Act. Corr. - Inv.) / Pas. Corr.</code></td>
                                <td style="text-align: right; font-weight:
600;">${ratios.quickRatio.toFixed(2)}</td>
                                <td style="text-align: center;"><span class="badge
badge-${quickStatus.status}">${quickStatus.text}</span></td>
                            </tr>
                            <tr>
                                <td rowspan="3" style="font-weight: 600; background: var(-
-warning-50);">Solvencia</td>
                                <td>Endeudamiento</td>
                                <td><code>Pasivo / Activo × 100</code></td>
                                <td style="text-align: right; font-weight:
600;">${ratios.debtRatio.toFixed(1)}%</td>
                                <td style="text-align: center;"><span class="badge
badge-${debtStatus.status}">${debtStatus.text}</span></td>
                            </tr>
                            <tr>
                                <td>Deuda/Patrimonio</td>
                                <td><code>Pasivo / Patrimonio</code></td>
                                <td style="text-align: right; font-weight:
600;">${ratios.debtToEquity.toFixed(2)}</td>
                                <td style="text-align: center;">-</td>
                            </tr>
                            <tr>
                                <td>Autonomía</td>
                                <td><code>Patrimonio / Activo × 100</code></td>
                                <td style="text-align: right; font-weight:
600;">${ratios.equityRatio.toFixed(1)}%</td>
                                <td style="text-align: center;">-</td>
                            </tr>
                            <tr>
                                <td rowspan="3" style="font-weight: 600; background: var(-
-success-50);">Rentabilidad</td>
                                <td>ROA</td>
                                <td><code>Resultado / Activo × 100</code></td>
                                <td style="text-align: right; font-weight:
600;">${ratios.roa.toFixed(2)}%</td>
                                <td style="text-align: center;"><span class="badge
badge-${roaStatus.status}">${roaStatus.text}</span></td>
                            </tr>
```

```
                            <tr>
                                <td>ROE</td>
                                <td><code>Resultado / Patrimonio × 100</code></td>
                                <td style="text-align: right; font-weight:
600;">${ratios.roe.toFixed(2)}%</td>
                                <td style="text-align: center;"><span class="badge
badge-${roeStatus.status}">${roeStatus.text}</span></td>
                            </tr>
                            <tr>
                                <td>Margen Neto</td>
                                <td><code>Utilidad / Ventas × 100</code></td>
                                <td style="text-align: right; font-weight:
600;">${ratios.netProfitMargin.toFixed(2)}%</td>
                                <td style="text-align: center;">-</td>
                            </tr>
                        </tbody>
                    </table>
                </div>
            </div>
            <!-- Información Educativa -->
            <div class="card" style="margin-top: var(--space-4);">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-graduation-cap"></i> ¿Cómo
interpretar los Ratios Financieros?</div>
                </div>
                <div class="card-body">
                    <div style="display: grid; grid-template-columns: repeat(auto-fit,
minmax(300px, 1fr)); gap: var(--space-4);">
                        <div style="background: var(--info-50); padding: var(--space-4);
border-radius: var(--radius-lg);">
                            <h4 style="color: var(--info-600); margin-bottom: var(--space-
2);"><i class="fas fa-tint"></i> Liquidez</h4>
                            <p style="font-size: var(--font-size-sm);">Miden la capacidad
de la empresa para cumplir con sus obligaciones a corto plazo. Una razón corriente > 1.5
indica buena liquidez.</p>
                        </div>
                        <div style="background: var(--warning-50); padding: var(--space-
4); border-radius: var(--radius-lg);">
                            <h4 style="color: var(--warning-600); margin-bottom: var(--
space-2);"><i class="fas fa-shield-alt"></i> Solvencia</h4>
                            <p style="font-size: var(--font-size-sm);">Miden el nivel de
endeudamiento y la capacidad de pago a largo plazo. Un endeudamiento < 60% generalmente es
aceptable.</p>
                        </div>
                        <div style="background: var(--success-50); padding: var(--space-
4); border-radius: var(--radius-lg);">
                            <h4 style="color: var(--success-600); margin-bottom: var(--
space-2);"><i class="fas fa-chart-line"></i> Rentabilidad</h4>
                            <p style="font-size: var(--font-size-sm);">Miden la eficiencia
en la generación de resultados. Un ROE > 15% se considera excelente para la mayoría de
industrias.</p>
                        </div>
                    </div>
                </div>
            </div>
        `;
    },
    async renderIncomeStatement() {
        const company = CompanyService.getCurrent();
        if (!company) return '<p>Seleccione una empresa</p>';
        const range = Helpers.getYearRange();
        const statement = await AccountingService.getIncomeStatement(range.start,
range.end);
        return `
            <div class="page-header">
                <h1 class="page-title">Estado de Resultados (Utilidad o Pérdida del
Ejercicio)</h1>
                <p class="page-subtitle">Del ${Formatters.date(range.start)} al
${Formatters.date(range.end)} - Formato por Función</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="date-from"
value="${range.start}">
                    <input type="date" class="form-control" id="date-to"
value="${range.end}">
                    <button class="btn btn-primary" id="btn-generate-statement">
                        <i class="fas fa-sync"></i> Generar
                    </button>
                </div>
                <div class="toolbar-right">
```

```
                    <button class="btn btn-outline" id="btn-export-statement">
                        <i class="fas fa-file-pdf"></i> PDF
                    </button>
                </div>
            </div>
            <div class="dashboard-grid" style="margin-bottom: var(--space-4);">
                <div class="kpi-card success">
                    <div class="kpi-header">
                        <div class="kpi-icon success"><i class="fas fa-arrow-up"></i>
</div>
                    </div>
                    <div class="kpi-
value">${Formatters.currency(statement.revenues.total)}</div>
                    <div class="kpi-label">Ingresos Totales</div>
                </div>
                <div class="kpi-card error">
                    <div class="kpi-header">
                        <div class="kpi-icon error"><i class="fas fa-arrow-down"></i>
</div>
                    </div>
                    <div class="kpi-
value">${Formatters.currency(statement.expenses.total)}</div>
                    <div class="kpi-label">Gastos Totales</div>
                </div>
                <div class="kpi-card span-2 ${statement.netIncome >= 0 ? 'success' :
'error'}">
                    <div class="kpi-header">
                        <div class="kpi-icon ${statement.netIncome >= 0 ? 'success' :
'error'}">
                            <i class="fas fa-${statement.netIncome >= 0 ? 'trophy' :
'exclamation-triangle'}"></i>
                        </div>
                        ${statement.isProvisional ? `
                            <div class="kpi-badge info" title="Incluye movimientos de
libros auxiliares no centralizados">
                                <i class="fas fa-clock"></i> Provisional (Centralizador)
                            </div>
                        ` : ''}
                    </div>
                    <div class="kpi-value">${Formatters.currency(statement.netIncome)}
</div>
                    <div class="kpi-label">${statement.netIncome >= 0 ? 'Utilidad Neta' :
'Pérdida Neta'}</div>
                </div>
            </div>
            <div class="card">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-chart-line"></i>
Detalle</div>
                </div>
                <div class="card-body">
                    <h4 style="color: var(--success-500); margin-bottom: var(--space-
3);">INGRESOS OPERACIONALES</h4>
                    ${this.renderAccountGroup(statement.revenues.items, 'revenue')}
                    <div style="display: flex; justify-content: space-between; font-
weight: 600; margin: var(--space-3) 0; padding: var(--space-2) 0; border-top: 1px solid
var(--border-medium);">
                        <span>Total Ventas e Ingresos</span>
                        <span>${Formatters.currency(statement.revenues.total)}</span>
                    </div>
                    <h4 style="color: var(--error-500); margin: var(--space-4) 0 var(--
space-3);">COSTOS DE VENTAS</h4>
                    ${this.renderAccountGroup(statement.costOfSales.items, 'expense')}
                    <div style="display: flex; justify-content: space-between; font-
weight: 600; margin: var(--space-3) 0; padding: var(--space-2) 0; border-top: 1px solid
var(--border-medium); color: var(--error-600);">
                        <span>Menos: Total Costos de Ventas</span>
                        <span>(${Formatters.currency(statement.costOfSales.total)})</span>
                    </div>
                    <div style="display: flex; justify-content: space-between; font-
weight: 700; font-size: var(--font-size-md); margin-bottom: var(--space-4); padding: var(-
-space-3); background: var(--primary-50); border-radius: var(--radius-md);">
                        <span>MARGEN BRUTO (Resultado Operacional Libre)</span>
                        <span style="color: var(--primary-600);">
                            ${Formatters.currency(statement.grossProfit)}
                        </span>
                    </div>
                    <h4 style="color: var(--warning-600); margin: var(--space-4) 0 var(--
space-3);">OTROS GASTOS (Administración, Ventas y Financieros)</h4>
                    ${this.renderAccountGroup(statement.otherExpenses.items, 'expense')}
                    <div style="display: flex; justify-content: space-between; font-
```

```
weight: 600; margin: var(--space-3) 0; padding: var(--space-2) 0; border-top: 1px solid
var(--border-medium); color: var(--warning-600);">
                        <span>Total Otros Gastos</span>
                        <span>(${Formatters.currency(statement.otherExpenses.total)})
</span>
                    </div>
                    <div style="display: flex; justify-content: space-between; font-
weight: 700; font-size: var(--font-size-lg); margin-top: var(--space-4); padding: var(--
space-3); background: ${statement.netIncome >= 0 ? 'var(--success-50)' : 'var(--error-
50)'}; border-radius: var(--radius-lg);">
                        <span>${statement.netIncome >= 0 ? 'UTILIDAD DEL EJERCICIO' :
'PÉRDIDA DEL EJERCICIO'}</span>
                        <span style="color: ${statement.netIncome >= 0 ? 'var(--success-
500)' : 'var(--error-500)'}">
                            ${statement.netIncome >= 0 ?
Formatters.currency(statement.netIncome) :
`(${Formatters.currency(Math.abs(statement.netIncome))})`}
                        </span>
                    </div>
                </div>
            </div>
        `;
    },
    renderCashFlow() {
        return `
            <div class="page-header">
                <h1 class="page-title">Flujo de Caja</h1>
                <p class="page-subtitle">Estado de Flujos de Efectivo</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="date-from">
                    <input type="date" class="form-control" id="date-to">
                    <button class="btn btn-primary" id="btn-generate-cashflow">
                        <i class="fas fa-sync"></i> Generar
                    </button>
                </div>
            </div>
            <div class="card">
                <div class="card-body">
                    <div class="empty-state">
                        <i class="fas fa-money-bill-wave"></i>
                        <h3>Flujo de Caja</h3>
                        <p>Seleccione un período para generar el estado de flujo de
efectivo</p>
                    </div>
                </div>
            </div>
        `;
    },
    async renderAccountAnalysis() {
        const accounts = await AccountingService.getMovementAccounts();
        return `
            <div class="page-header">
                <h1 class="page-title">Análisis de Cuentas</h1>
                <p class="page-subtitle">Análisis detallado de saldos y movimientos</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <select class="form-control" id="analysis-type" style="width: 200px;">
                        <option value="summary">Resumen de Saldos</option>
                        <option value="movements">Movimientos del Período</option>
                        <option value="comparison">Comparativo</option>
                    </select>
                    <input type="date" class="form-control" id="date-from">
                    <input type="date" class="form-control" id="date-to">
                    <button class="btn btn-primary" id="btn-analyze">
                        <i class="fas fa-chart-bar"></i> Analizar
                    </button>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-export-analysis">
                        <i class="fas fa-file-excel"></i> Exportar
                    </button>
                </div>
            </div>
            <div class="dashboard-grid">
                <div class="chart-card span-2">
                    <div class="chart-header">
                        <div class="chart-title">Composición del Activo</div>
                    </div>
```

```
                    <div class="chart-body">
                        <canvas id="assets-chart"></canvas>
                    </div>
                </div>
                <div class="chart-card span-2">
                    <div class="chart-header">
                        <div class="chart-title">Estructura Financiera</div>
                    </div>
                    <div class="chart-body">
                        <canvas id="structure-chart"></canvas>
                    </div>
                </div>
            </div>
            <div class="card" style="margin-top: var(--space-4);">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-list"></i> Detalle de
Cuentas</div>
                </div>
                <div id="analysis-table"></div>
            </div>
        `;
    },
    async init(view) {
        switch (view) {
            case 'balance-general': this.initBalanceSheet(); break;
            case 'balance-comprobacion': this.initTrialBalance(); break;
            case 'estado-resultados': this.initIncomeStatement(); break;
            case 'ratios-financieros': await this.initFinancialRatios(); break;
            case 'flujo-caja': this.initCashFlow(); break;
            case 'analisis-cuentas': await this.initAccountAnalysis(); break;
        }
    },
    initBalanceSheet() {
        document.getElementById('btn-generate-balance')?.addEventListener('click', async
() => {
            App.navigate('reportes', 'balance-general');
        });
        document.getElementById('btn-export-balance-pdf')?.addEventListener('click', async
() => {
            const company = CompanyService.getCurrent();
            const balanceData = await
AccountingService.getBalanceSheet(document.getElementById('balance-date').value);
            await ExportService.balanceSheetPDF(balanceData, company.name);
            Toast.success('PDF generado');
        });
        document.getElementById('btn-export-balance-excel')?.addEventListener('click', ()
=> {
            Toast.info('Exportar a Excel');
        });
    },
    initTrialBalance() {
        document.getElementById('btn-generate-trial')?.addEventListener('click', async ()
=> {
            App.navigate('reportes', 'balance-comprobacion');
        });
        document.getElementById('btn-print-trial')?.addEventListener('click', () => {
            window.print();
        });
        document.getElementById('btn-export-trial-pdf')?.addEventListener('click', async
() => {
            Toast.info('Exportando a PDF...');
        });
        document.getElementById('btn-export-trial-excel')?.addEventListener('click', () =>
{
            Toast.info('Exportando a Excel...');
        });
    },
    async initFinancialRatios() {
        document.getElementById('btn-generate-ratios')?.addEventListener('click', async ()
=> {
            App.navigate('reportes', 'ratios-financieros');
        });
        document.getElementById('btn-print-ratios')?.addEventListener('click', () => {
            window.print();
        });
        document.getElementById('btn-export-ratios-pdf')?.addEventListener('click', async
() => {
            Toast.info('Exportando PDF...');
        });
        const balanceData = await
AccountingService.getBalanceSheet(Helpers.getCurrentDate());
```

```
        const range = Helpers.getYearRange();
        const incomeData = await AccountingService.getIncomeStatement(range.start,
range.end);
        const totalAssets = balanceData.assets.total;
        const totalLiabilities = balanceData.liabilities.total;
        const totalEquity = balanceData.equity.total + balanceData.netIncome;
        const netIncome = incomeData.netIncome;
        const totalRevenue = incomeData.revenues.total;
        Charts.doughnut('capital-structure-chart', {
            labels: ['Pasivos', 'Patrimonio'],
            values: [totalLiabilities, totalEquity]
        }, {
            colors: ['#ef4444', '#22c55e'],
            currency: true
        });
        const roa = totalAssets > 0 ? (netIncome / totalAssets) * 100 : 0;
        const roe = totalEquity > 0 ? (netIncome / totalEquity) * 100 : 0;
        const npm = totalRevenue > 0 ? (netIncome / totalRevenue) * 100 : 0;
        Charts.bar('profitability-chart', {
            labels: ['ROA', 'ROE', 'Margen Neto'],
            datasets: [{
                label: '%',
                data: [roa, roe, npm]
            }]
        }, {
            percentage: true,
            colors: ['#3b82f6', '#22c55e', '#f59e0b']
        });
        const currentAssets = balanceData.assets.items
            .filter(a => ['1.1.1', '1.1.2', '1.1.3', '1.1.4', '1.1.5'].some(code =>
a.code.startsWith(code)))
            .reduce((sum, a) => sum + Math.abs(a.balance), 0) || totalAssets * 0.6;
        const currentLiabilities = balanceData.liabilities.items
            .filter(a => ['2.1'].some(code => a.code.startsWith(code)))
            .reduce((sum, a) => sum + Math.abs(a.balance), 0) || totalLiabilities * 0.5;
        const inventory = currentAssets * 0.3;
        const currentRatio = currentLiabilities > 0 ? currentAssets / currentLiabilities :
0;
        const quickRatio = currentLiabilities > 0 ? (currentAssets - inventory) /
currentLiabilities : 0;
        const debtRatio = totalAssets > 0 ? (totalLiabilities / totalAssets) * 100 : 0;
        Charts.doughnut('ratios-summary-chart', {
            labels: ['Liquidez', 'Solvencia', 'Rentabilidad'],
            values: [
                Math.min(currentRatio * 50, 100),
                Math.max(100 - debtRatio, 0),
                Math.min(Math.max(roe, 0) * 5, 100)
            ]
        }, {
            colors: ['#3b82f6', '#f59e0b', '#22c55e']
        });
        Charts.bar('ratios-bar-chart', {
            labels: ['Razón Corriente', 'Prueba Ácida', 'Endeudamiento %', 'ROA %', 'ROE
%'],
            datasets: [{
                label: 'Valor',
                data: [currentRatio, quickRatio, debtRatio, roa, roe]
            }]
        }, {
            horizontal: true
        });
    },
    initIncomeStatement() {
        document.getElementById('btn-generate-statement')?.addEventListener('click', async
() => {
            App.navigate('reportes', 'estado-resultados');
        });
    },
    initCashFlow() {
        document.getElementById('btn-generate-cashflow')?.addEventListener('click', () =>
{
            Toast.info('Generar flujo de caja');
        });
    },
    async initAccountAnalysis() {
        const accounts = await AccountingService.getChartOfAccounts();
        const assets = accounts.filter(a => a.type === 'asset' && !a.isGroup && a.balance
> 0);
        if (assets.length > 0) {
            Charts.doughnut('assets-chart', {
                labels: assets.slice(0, 8).map(a => a.name),
```

```
                values: assets.slice(0, 8).map(a => a.balance)
            }, { currency: true });
        }
        const structure = [
            { type: 'Activos', value: Helpers.sumBy(accounts.filter(a => a.type ===
'asset' && !a.isGroup), 'balance') },
            { type: 'Pasivos', value: Helpers.sumBy(accounts.filter(a => a.type ===
'liability' && !a.isGroup), 'balance') },
            { type: 'Patrimonio', value: Helpers.sumBy(accounts.filter(a => a.type ===
'equity' && !a.isGroup), 'balance') }
        ];
        Charts.bar('structure-chart', {
            labels: structure.map(s => s.type),
            datasets: [{ label: 'Saldo', data: structure.map(s => s.value) }]
        }, { currency: true });
        DataTable.create('analysis-table', {
            columns: [
                { key: 'code', label: 'Código' },
                { key: 'name', label: 'Cuenta' },
                {
                    key: 'type', label: 'Tipo', render: (v) => {
                        const labels = { asset: 'Activo', liability: 'Pasivo', equity:
'Patrimonio', revenue: 'Ingreso', expense: 'Gasto' };
                        return labels[v] || v;
                    }
                },
                { key: 'balance', label: 'Saldo', format: 'currency', class: 'text-right'
}
            ],
            data: accounts.filter(a => !a.isGroup && a.balance !== 0),
            emptyMessage: 'No hay cuentas con saldo'
        });
    }
};
window.ReportesModule = ReportesModule;
```

Archivo: js\modules\rrhh.js

```
const RRHHModule = {
    currentView: 'empleados',
    async render(view = 'empleados') {
        this.currentView = view;
        switch (view) {
            case 'empleados': return await this.renderEmployees();
            case 'nomina': return await this.renderPayroll();
            case 'provisiones': return await this.renderProvisions();
            default: return '<p>Vista no encontrada</p>';
        }
    },
    async renderEmployees() {
        return `
            <div class="page-header">
                <h1 class="page-title">Empleados</h1>
                <p class="page-subtitle">Gestión de personal y datos laborales</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <div class="search-box">
                        <i class="fas fa-search"></i>
                        <input type="text" class="form-control" placeholder="Buscar
empleado..." id="search-employees">
                    </div>
                    <select class="form-control" id="filter-department" style="width:
150px;">
                        <option value="">Todos los dptos.</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-employee">
                        <i class="fas fa-user-plus"></i> Nuevo Empleado
                    </button>
                </div>
            </div>
            <div id="employees-table"></div>
        `;
```

```
    },
    async renderPayroll() {
        return `
            <div class="page-header">
                <h1 class="page-title">Nómina</h1>
                <p class="page-subtitle">Liquidación de remuneraciones</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="month" class="form-control" id="payroll-period"
value="${new Date().toISOString().slice(0, 7)}">
                    <button class="btn btn-primary" id="btn-calculate-payroll">
                        <i class="fas fa-calculator"></i> Calcular Nómina
                    </button>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-export-payroll">
                        <i class="fas fa-file-excel"></i> Exportar
                    </button>
                    <button class="btn btn-success" id="btn-post-payroll">
                        <i class="fas fa-check"></i> Contabilizar
                    </button>
                </div>
            </div>
            <div id="payroll-content">
                <div class="empty-state">
                    <i class="fas fa-file-invoice-dollar"></i>
                    <h3>Seleccione un período</h3>
                    <p>Elija el mes y haga clic en "Calcular Nómina"</p>
                </div>
            </div>
        `;
    },
    async renderProvisions() {
        return `
            <div class="page-header">
                <h1 class="page-title">Provisiones</h1>
                <p class="page-subtitle">Cálculo de provisiones laborales</p>
            </div>
            <div class="dashboard-grid">
                <div class="kpi-card">
                    <div class="kpi-header">
                        <div class="kpi-icon warning"><i class="fas fa-umbrella-beach">
</i></div>
                    </div>
                    <div class="kpi-value" id="vacation-provision">$ 0</div>
                    <div class="kpi-label">Provisión Vacaciones</div>
                </div>
                <div class="kpi-card">
                    <div class="kpi-header">
                        <div class="kpi-icon error"><i class="fas fa-hand-holding-usd">
</i></div>
                    </div>
                    <div class="kpi-value" id="severance-provision">$ 0</div>
                    <div class="kpi-label">Provisión Indemnización</div>
                </div>
                <div class="kpi-card span-2">
                    <div class="kpi-header">
                        <div class="kpi-icon primary"><i class="fas fa-coins"></i></div>
                    </div>
                    <div class="kpi-value" id="total-provision">$ 0</div>
                    <div class="kpi-label">Total Provisiones</div>
                </div>
            </div>
            <div class="card" style="margin-top: var(--space-4);">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-table"></i> Detalle por
Empleado</div>
                    <button class="btn btn-outline btn-sm" id="btn-calculate-provisions">
                        <i class="fas fa-sync"></i> Recalcular
                    </button>
                </div>
                <div class="card-body" style="padding: 0;">
                    <div id="provisions-table"></div>
                </div>
            </div>
        `;
    },
    async init(view) {
        const company = CompanyService.getCurrent();
        if (!company) return;
```

```
        switch (view) {
            case 'empleados': await this.initEmployees(); break;
            case 'nomina': this.initPayroll(); break;
            case 'provisiones': await this.initProvisions(); break;
        }
    },
    async initEmployees() {
        const company = CompanyService.getCurrent();
        const employees = await DB.getByIndex('employees', 'companyId', company.id);
        const departments = [...new Set(employees.map(e =>
e.department).filter(Boolean))];
        const deptSelect = document.getElementById('filter-department');
        departments.forEach(dept => {
            deptSelect.innerHTML += `<option value="${dept}">${dept}</option>`;
        });
        DataTable.create('employees-table', {
            columns: [
                { key: 'rut', label: 'RUT' },
                { key: 'name', label: 'Nombre' },
                { key: 'position', label: 'Cargo' },
                { key: 'department', label: 'Departamento' },
                { key: 'startDate', label: 'Ingreso', format: 'date' },
                { key: 'salary', label: 'Sueldo Base', format: 'currency', class: 'text-
right' },
                { key: 'status', label: 'Estado', format: 'status' }
            ],
            data: employees,
            actions: [
                { name: 'edit', label: 'Editar', icon: 'fas fa-edit', handler: (row) =>
this.showEmployeeModal(row.id) },
                { name: 'payslip', label: 'Liquidación', icon: 'fas fa-file-invoice',
handler: (row) => this.showPayslip(row.id) }
            ],
            emptyMessage: 'No hay empleados registrados'
        });
        document.getElementById('btn-new-employee')?.addEventListener('click', () =>
this.showEmployeeModal());
    },
    async showEmployeeModal(employeeId = null) {
        const isEdit = !!employeeId;
        let data = {};
        if (isEdit) {
            data = await DB.get('employees', employeeId);
        }
        await Modal.form({
            title: isEdit ? 'Editar Empleado' : 'Nuevo Empleado',
            fields: [
                { name: 'rut', label: 'RUT', required: true, placeholder: '12.345.678-9',
default: data.rut },
                { name: 'name', label: 'Nombre Completo', required: true, placeholder:
'Nombre y apellidos', default: data.name },
                { name: 'position', label: 'Cargo', required: true, placeholder: 'Cargo',
default: data.position },
                { name: 'department', label: 'Departamento', placeholder: 'Área de
trabajo', default: data.department },
                { name: 'startDate', label: 'Fecha de Ingreso', type: 'date', required:
true, default: data.startDate },
                { name: 'salary', label: 'Sueldo Base', type: 'number', required: true,
default: data.salary || 0 },
                { name: 'email', label: 'Email', type: 'email', placeholder:
'email@empresa.cl', default: data.email },
                { name: 'phone', label: 'Teléfono', placeholder: '+56 9 1234 5678',
default: data.phone }
            ],
            validate: (formData) => {
                if (!Helpers.validateRUT(formData.rut)) return 'RUT inválido';
                return true;
            },
            onSubmit: async (formData) => {
                const company = CompanyService.getCurrent();
                formData.rut = Helpers.formatRUT(formData.rut);
                if (isEdit) {
                    await DB.update('employees', { ...data, ...formData });
                    Toast.success('Empleado actualizado');
                } else {
                    await DB.add('employees', { companyId: company.id, ...formData,
status: 'active' });
                    Toast.success('Empleado creado');
                }
                App.navigate('rrhh', 'empleados');
            }
```

```
        });
    },
    showPayslip(employeeId) {
        Toast.info('Ver liquidación de sueldo');
    },
    initPayroll() {
        document.getElementById('btn-calculate-payroll')?.addEventListener('click', async
() => {
            await this.calculatePayroll();
        });
        document.getElementById('btn-post-payroll')?.addEventListener('click', () => {
            Toast.info('Contabilizar nómina');
        });
        document.getElementById('btn-export-payroll')?.addEventListener('click', () => {
            Toast.info('Exportar nómina');
        });
    },
    async calculatePayroll() {
        const period = document.getElementById('payroll-period').value;
        const company = CompanyService.getCurrent();
        const employees = await DB.getByIndex('employees', 'companyId', company.id);
        const activeEmployees = employees.filter(e => e.status === 'active');
        const UFM = 37000;
        const salarioMinimo = 460000;
        const topeSeguros = 84.1 * UFM;
        const afpRate = 0.1144;
        const saludRate = 0.07;
        const seguroCesantia = 0.006;
        const payrollData = activeEmployees.map(emp => {
            const bruto = emp.salary;
            const imponible = Math.min(bruto, topeSeguros);
            const afp = Math.round(imponible * afpRate);
            const salud = Math.round(imponible * saludRate);
            const cesantia = Math.round(imponible * seguroCesantia);
            const totalDescuentos = afp + salud + cesantia;
            const liquido = bruto - totalDescuentos;
            return {
                employeeId: emp.id,
                name: emp.name,
                rut: emp.rut,
                department: emp.department,
                bruto,
                afp,
                salud,
                cesantia,
                totalDescuentos,
                liquido
            };
        });
        const totals = {
            bruto: Helpers.sumBy(payrollData, 'bruto'),
            afp: Helpers.sumBy(payrollData, 'afp'),
            salud: Helpers.sumBy(payrollData, 'salud'),
            cesantia: Helpers.sumBy(payrollData, 'cesantia'),
            descuentos: Helpers.sumBy(payrollData, 'totalDescuentos'),
            liquido: Helpers.sumBy(payrollData, 'liquido')
        };
        document.getElementById('payroll-content').innerHTML = `
            <div class="dashboard-grid" style="margin-bottom: var(--space-4);">
                <div class="kpi-card">
                    <div class="kpi-value">${Formatters.currency(totals.bruto)}</div>
                    <div class="kpi-label">Total Bruto</div>
                </div>
                <div class="kpi-card">
                    <div class="kpi-value">${Formatters.currency(totals.descuentos)}</div>
                    <div class="kpi-label">Total Descuentos</div>
                </div>
                <div class="kpi-card span-2">
                    <div class="kpi-value">${Formatters.currency(totals.liquido)}</div>
                    <div class="kpi-label">Total Líquido a Pagar</div>
                </div>
            </div>
            <div class="card">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-users"></i> Nómina ${period}
</div>
                    <span class="badge badge-info">${payrollData.length} empleados</span>
                </div>
                <div class="card-body" style="padding: 0;">
                    <div class="table-container">
                        <table class="data-table">
```

```
                            <thead>
                                <tr>
                                    <th>Empleado</th>
                                    <th>Departamento</th>
                                    <th class="text-right">Bruto</th>
                                    <th class="text-right">AFP</th>
                                    <th class="text-right">Salud</th>
                                    <th class="text-right">Cesantía</th>
                                    <th class="text-right">Líquido</th>
                                </tr>
                            </thead>
                            <tbody>
                                ${payrollData.map(row => `
                                    <tr>
                                        <td>
                                            <div style="font-weight: 500;">${row.name}
</div>
                                            <div style="font-size: var(--font-size-xs);
color: var(--text-tertiary);">${row.rut}</div>
                                        </td>
                                        <td>${row.department || '-'}</td>
                                        <td class="text-
right">${Formatters.currency(row.bruto)}</td>
                                        <td class="text-
right">${Formatters.currency(row.afp)}</td>
                                        <td class="text-
right">${Formatters.currency(row.salud)}</td>
                                        <td class="text-
right">${Formatters.currency(row.cesantia)}</td>
                                        <td class="text-right" style="font-weight:
600;">${Formatters.currency(row.liquido)}</td>
                                    </tr>
                                `).join('')}
                            </tbody>
                            <tfoot>
                                <tr style="font-weight: 700; background: var(--neutral-
100);">
                                    <td colspan="2">TOTALES</td>
                                    <td class="text-
right">${Formatters.currency(totals.bruto)}</td>
                                    <td class="text-
right">${Formatters.currency(totals.afp)}</td>
                                    <td class="text-
right">${Formatters.currency(totals.salud)}</td>
                                    <td class="text-
right">${Formatters.currency(totals.cesantia)}</td>
                                    <td class="text-
right">${Formatters.currency(totals.liquido)}</td>
                                </tr>
                            </tfoot>
                        </table>
                    </div>
                </div>
            </div>
        `;
        Toast.success('Nómina calculada');
    },
    async initProvisions() {
        await this.calculateProvisions();
        document.getElementById('btn-calculate-provisions')?.addEventListener('click',
async () => {
            await this.calculateProvisions();
            Toast.success('Provisiones recalculadas');
        });
    },
    async calculateProvisions() {
        const company = CompanyService.getCurrent();
        const employees = await DB.getByIndex('employees', 'companyId', company.id);
        const activeEmployees = employees.filter(e => e.status === 'active');
        const today = new Date();
        const provisionsData = activeEmployees.map(emp => {
            const startDate = new Date(emp.startDate);
            const monthsWorked = Math.floor((today - startDate) / (1000 * 60 * 60 * 24 *
30));
            const yearsWorked = monthsWorked / 12;
            const dailySalary = emp.salary / 30;
            const vacationDays = (15 / 12) * monthsWorked;
            const vacationProvision = Math.round(vacationDays * dailySalary);
            const severanceYears = Math.min(yearsWorked, 11);
            const severanceProvision = Math.round(severanceYears * emp.salary);
            return {
```

```
                name: emp.name,
                department: emp.department,
                monthsWorked,
                salary: emp.salary,
                vacationProvision,
                severanceProvision,
                total: vacationProvision + severanceProvision
            };
        });
        const totals = {
            vacation: Helpers.sumBy(provisionsData, 'vacationProvision'),
            severance: Helpers.sumBy(provisionsData, 'severanceProvision'),
            total: Helpers.sumBy(provisionsData, 'total')
        };
        document.getElementById('vacation-provision').textContent =
Formatters.currency(totals.vacation);
        document.getElementById('severance-provision').textContent =
Formatters.currency(totals.severance);
        document.getElementById('total-provision').textContent =
Formatters.currency(totals.total);
        DataTable.create('provisions-table', {
            columns: [
                { key: 'name', label: 'Empleado' },
                { key: 'department', label: 'Departamento' },
                { key: 'monthsWorked', label: 'Meses', class: 'text-right' },
                { key: 'salary', label: 'Sueldo', format: 'currency', class: 'text-right'
},
                { key: 'vacationProvision', label: 'Prov. Vacaciones', format: 'currency',
class: 'text-right' },
                { key: 'severanceProvision', label: 'Prov. Indemnización', format:
'currency', class: 'text-right' },
                { key: 'total', label: 'Total', format: 'currency', class: 'text-right' }
            ],
            data: provisionsData,
            emptyMessage: 'No hay empleados para calcular provisiones'
        });
    }
};
window.RRHHModule = RRHHModule;
```

Archivo: js\modules\tesoreria.js

```
const TesoreriaModule = {
    currentView: 'cuentas-pagar',
    async render(view = null) {
        const currentView = view || 'cuentas-pagar';
        this.currentView = currentView;
        switch (currentView) {
            case 'cuentas-pagar': return this.renderAccountsPayable();
            case 'cuentas-cobrar': return this.renderAccountsReceivable();
            case 'bancos': return await this.renderBanks();
            case 'conciliacion': return this.renderReconciliation();
            default: return '<p>Vista no encontrada</p>';
        }
    },
    renderAccountsPayable() {
        return `
            <div class="page-header">
                <h1 class="page-title">Control de Pagos</h1>
                <p class="page-subtitle">Control de obligaciones con proveedores</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <div class="search-box">
                        <i class="fas fa-search"></i>
                        <input type="text" class="form-control" placeholder="Buscar..."
id="search-payables">
                    </div>
                    <select class="form-control" id="filter-status" style="width: 150px;">
                        <option value="">Todos</option>
                        <option value="pending">Pendientes</option>
                        <option value="overdue">Vencidas</option>
                        <option value="paid">Pagadas</option>
                    </select>
                </div>
```

```
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-register-payment">
                        <i class="fas fa-money-bill-wave"></i> Registrar Pago
                    </button>
                </div>
            </div>
            <div id="payables-table"></div>
        `;
    },
    renderAccountsReceivable() {
        return `
            <div class="page-header">
                <h1 class="page-title">Control de Cobros</h1>
                <p class="page-subtitle">Control de créditos con clientes</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <div class="search-box">
                        <i class="fas fa-search"></i>
                        <input type="text" class="form-control" placeholder="Buscar..."
id="search-receivables">
                    </div>
                    <select class="form-control" id="filter-status" style="width: 150px;">
                        <option value="">Todos</option>
                        <option value="pending">Pendientes</option>
                        <option value="overdue">Vencidas</option>
                        <option value="paid">Cobradas</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-register-collection">
                        <i class="fas fa-hand-holding-usd"></i> Registrar Cobro
                    </button>
                </div>
            </div>
            <div id="receivables-table"></div>
        `;
    },
    async renderBanks() {
        const company = CompanyService.getCurrent();
        let accounts = [];
        if (company) {
            accounts = await DB.getByIndex('bankAccounts', 'companyId', company.id);
        }
        return `
            <div class="page-header">
                <h1 class="page-title">Bancos</h1>
                <p class="page-subtitle">Gestión de cuentas bancarias y movimientos</p>
            </div>
            <div class="dashboard-grid" style="margin-bottom: var(--space-4);">
                ${accounts.map(acc => `
                    <div class="kpi-card">
                        <div class="kpi-header">
                            <div class="kpi-icon primary"><i class="fas fa-university">
</i></div>
                            <button class="btn btn-ghost btn-icon"><i class="fas fa-
ellipsis-v"></i></button>
                        </div>
                        <div class="kpi-value">${Formatters.currency(acc.balance)}</div>
                        <div class="kpi-label">${acc.bank} - ${acc.accountType}</div>
                        <div class="kpi-footer">
                            <span>N° ${acc.accountNumber}</span>
                        </div>
                    </div>
                `).join('') || `
                    <div class="kpi-card span-4">
                        <div class="kpi-header">
                            <div class="kpi-icon primary"><i class="fas fa-university">
</i></div>
                        </div>
                        <div class="kpi-value">Sin cuentas</div>
                        <div class="kpi-label">Agregue una cuenta bancaria para
comenzar</div>
                    </div>
                `}
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <select class="form-control" id="select-account" style="width:
250px;">
                        <option value="">Seleccione cuenta...</option>
```

```
                        ${accounts.map(acc => `<option value="${acc.id}">${acc.bank} -
${acc.accountNumber}</option>`).join('')}
                    </select>
                    <input type="date" class="form-control" id="filter-date-from">
                    <input type="date" class="form-control" id="filter-date-to">
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-outline" id="btn-new-account">
                        <i class="fas fa-plus"></i> Nueva Cuenta
                    </button>
                    <button class="btn btn-primary" id="btn-new-transaction">
                        <i class="fas fa-exchange-alt"></i> Nuevo Movimiento
                    </button>
                </div>
            </div>
            <div id="transactions-table"></div>
        `;
    },
    renderReconciliation() {
        return `
            <div class="page-header">
                <h1 class="page-title">Conciliación Bancaria</h1>
                <p class="page-subtitle">Concilie los movimientos bancarios con la
contabilidad</p>
            </div>
            <div class="card">
                <div class="card-header">
                    <div class="card-title"><i class="fas fa-balance-scale"></i>
Seleccionar Período</div>
                </div>
                <div class="card-body">
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Cuenta Bancaria</label>
                            <select class="form-control" id="reconcile-account">
                                <option value="">Seleccione...</option>
                            </select>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Mes</label>
                            <input type="month" class="form-control" id="reconcile-month">
                        </div>
                        <div class="form-group">
                            <label class="form-label">Saldo según Cartola</label>
                            <input type="number" class="form-control" id="bank-balance"
placeholder="0">
                        </div>
                        <div class="form-group" style="display: flex; align-items: flex-
end;">
                            <button class="btn btn-primary" id="btn-start-reconciliation">
                                <i class="fas fa-play"></i> Iniciar Conciliación
                            </button>
                        </div>
                    </div>
                </div>
            </div>
            <div id="reconciliation-content" style="margin-top: var(--space-4);"></div>
        `;
    },
    async init(view) {
        const company = CompanyService.getCurrent();
        if (!company) return;
        switch (view) {
            case 'cuentas-pagar': await this.initAccountsPayable(); break;
            case 'cuentas-cobrar': await this.initAccountsReceivable(); break;
            case 'bancos': await this.initBanks(); break;
            case 'conciliacion': this.initReconciliation(); break;
        }
    },
    async initAccountsPayable() {
        const company = CompanyService.getCurrent();
        const invoices = await DB.getByIndex('supplierInvoices', 'companyId', company.id);
        const pending = invoices.filter(i => i.status !== 'paid' && i.status !==
'cancelled');
        for (const inv of pending) {
            if (!inv.supplierName) {
                const supplier = await DB.get('suppliers', inv.supplierId);
                inv.supplierName = supplier?.name || 'Proveedor desconocido';
            }
        }
        DataTable.create('payables-table', {
```

```
            columns: [
                { key: 'number', label: 'Factura' },
                { key: 'supplierName', label: 'Proveedor' },
                { key: 'date', label: 'Fecha', format: 'date' },
                { key: 'dueDate', label: 'Vencimiento', format: 'date' },
                { key: 'total', label: 'Total', format: 'currency', class: 'text-right' },
                { key: 'balance', label: 'Saldo', format: 'currency', class: 'text-right'
},
                { key: 'status', label: 'Estado', format: 'status' },
                {
                    key: 'actions', label: '', render: (_, row) => `
                        <button class="btn btn-ghost btn-sm"
onclick="TesoreriaModule.showPaymentModal('${row.id}')" title="Registrar Pago">
                            <i class="fas fa-money-bill-wave"></i>
                        </button>
                    `
                }
            ],
            data: pending,
            emptyMessage: 'No hay pagos pendientes'
        });
        document.getElementById('btn-register-payment')?.addEventListener('click', () => {
            this.showPaymentModal();
        });
    },
    async showPaymentModal(invoiceId = null) {
        const company = CompanyService.getCurrent();
        if (!company) {
            Toast.error('Seleccione una empresa primero');
            return;
        }
        const invoices = await DB.getByIndex('supplierInvoices', 'companyId', company.id);
        const pendingInvoices = invoices.filter(i => i.status !== 'paid' && i.status !==
'cancelled');
        const bankAccounts = await DB.getByIndex('bankAccounts', 'companyId', company.id);
        if (pendingInvoices.length === 0) {
            Toast.info('No hay facturas pendientes de pago');
            return;
        }
        for (const inv of pendingInvoices) {
            if (!inv.supplierName) {
                const supplier = await DB.get('suppliers', inv.supplierId);
                inv.supplierName = supplier?.name || 'Proveedor desconocido';
            }
        }
        const bankOptions = `<option value="">Efectivo (Caja)</option>` +
            bankAccounts.map(a => `<option value="${a.id}">${a.bank} - ${a.accountNumber}
</option>`).join('');
        Modal.open({
            title: 'Registrar Pago Múltiple a Proveedor',
            size: 'large',
            content: `
                <div class="form-row">
                    <div class="form-group" style="flex: 2;">
                        <label class="form-label">Factura a Pagar <span class="text-
danger">*</span></label>
                        <select class="form-control" id="pay-invoice-select" required>
                            <option value="">Seleccione factura...</option>
                            ${pendingInvoices.map(i => `<option value="${i.id}" data-
pending="${i.total - (i.paid || 0)}" ${invoiceId === i.id ? 'selected' : ''}>${i.number} -
${i.supplierName} - PEND: ${Formatters.currency(i.total - (i.paid || 0))}
</option>`).join('')}
                        </select>
                    </div>
                    <div class="form-group" style="flex: 1;">
                        <label class="form-label">Fecha de Pago</label>
                        <input type="date" class="form-control" id="pay-date"
value="${Helpers.getCurrentDate()}" required>
                    </div>
                </div>
                <div class="alert alert-info" style="margin: 1rem 0;">
                    Puede distribuir el pago combinando múltiples métodos y definiendo
porcentajes.
                </div>
                <div class="table-container" style="margin-bottom: 1rem;">
                    <table class="data-table">
                        <thead>
                            <tr>
                                <th style="width: 25%">Método de Pago</th>
                                <th style="width: 25%">Cuenta Origen</th>
                                <th style="width: 15%">Porcentaje %</th>
```

```
                                <th style="width: 25%">Monto</th>
                                <th style="width: 10%"></th>
                            </tr>
                        </thead>
                        <tbody id="pay-payment-lines">
                        </tbody>
                        <tfoot>
                            <tr>
                                <td colspan="3" class="text-right"><strong>Distribuido /
Pendiente:</strong></td>
                                <td colspan="2">
                                    <span id="pay-amount-paid" style="font-weight:bold;
color: var(--primary-600)">$0</span> /
                                    <span id="pay-amount-required" style="font-
weight:bold;">$0</span>
                                </td>
                            </tr>
                        </tfoot>
                    </table>
                </div>
                <button class="btn btn-outline btn-sm" id="btn-add-pay-line"><i class="fas
fa-plus"></i> Añadir Línea de Pago</button>
                <div class="form-group" style="margin-top: 1rem;">
                    <label class="form-label">Notas o Referencia General</label>
                    <textarea class="form-control" id="pay-notes" rows="2"
placeholder="Observaciones adicionales, n° comprobantes"></textarea>
                </div>
            `,
            footer: `
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-primary" id="btn-confirm-payment"><i class="fas fa-
check"></i> Registrar Pago</button>
            `
        });
        let targetAmount = 0;
        const updateRowTotals = () => {
            let total = 0;
            const rows = document.querySelectorAll('.pay-payment-line');
            rows.forEach(row => {
                const amount = parseFloat(row.querySelector('.pay-amount').value) || 0;
                total += amount;
                const pctField = row.querySelector('.pay-percent');
                if (targetAmount > 0) {
                    if (document.activeElement === row.querySelector('.pay-amount')) {
                        pctField.value = ((amount / targetAmount) * 100).toFixed(2);
                    }
                }
            });
            document.getElementById('pay-amount-paid').textContent =
Formatters.currency(total);
            const remaining = targetAmount - total;
            document.getElementById('pay-amount-required').innerHTML = `<span
style="color: ${Math.abs(remaining) < 0.01 ? 'green' :
'red'}">${Formatters.currency(targetAmount)}</span>`;
        };
        const addLine = (monto = 0) => {
            const tbody = document.getElementById('pay-payment-lines');
            const tr = document.createElement('tr');
            tr.className = 'pay-payment-line';
            tr.innerHTML = `
                <td>
                    <select class="form-control pay-method">
                        <option value="transfer">Transferencia</option>
                        <option value="cash">Efectivo</option>
                        <option value="check">Cheque</option>
                        <option value="card">Tarjeta</option>
                    </select>
                </td>
                <td>
                    <select class="form-control pay-bank">${bankOptions}</select>
                </td>
                <td>
                    <input type="number" class="form-control pay-percent" min="0"
max="100" step="any" placeholder="%">
                </td>
                <td>
                    <input type="number" class="form-control pay-amount" min="0"
step="any" value="${monto || ''}">
                </td>
                <td>
```

```
                    <button class="btn btn-icon btn-ghost btn-remove-row"><i class="fas
fa-trash text-danger"></i></button>
                </td>
            `;
            tbody.appendChild(tr);
            tr.querySelector('.btn-remove-row').onclick = () => {
                tr.remove();
                updateRowTotals();
            };
            const amtField = tr.querySelector('.pay-amount');
            const pctField = tr.querySelector('.pay-percent');
            amtField.oninput = updateRowTotals;
            pctField.oninput = () => {
                if (targetAmount > 0) {
                    const pct = parseFloat(pctField.value) || 0;
                    amtField.value = (targetAmount * (pct / 100)).toFixed(2);
                    updateRowTotals();
                }
            };
            if (monto > 0 && targetAmount > 0) {
                pctField.value = ((monto / targetAmount) * 100).toFixed(2);
            }
            const methodSelect = tr.querySelector('.pay-method');
            const bankSelect = tr.querySelector('.pay-bank');
            const updateAccountLabel = () => {
                const method = methodSelect.value;
                const firstOption = bankSelect.options[0];
                if (method === 'check' || method === 'card') {
                    firstOption.textContent = 'Cuentas por Pagar (Pendiente)';
                } else {
                    firstOption.textContent = 'Efectivo (Caja)';
                }
            };
            methodSelect.addEventListener('change', updateAccountLabel);
            updateAccountLabel();
        };
        const onInvoiceChange = () => {
            const select = document.getElementById('pay-invoice-select');
            const option = select.selectedOptions[0];
            targetAmount = parseFloat(option?.dataset.pending) || 0;
            document.getElementById('pay-payment-lines').innerHTML = '';
            if (targetAmount > 0) {
                addLine(targetAmount);
            }
            updateRowTotals();
        };
        document.getElementById('pay-invoice-select').addEventListener('change',
onInvoiceChange);
        document.getElementById('btn-add-pay-line').addEventListener('click', () =>
addLine(0));
        if (invoiceId) setTimeout(onInvoiceChange, 50);
        document.getElementById('btn-confirm-payment').addEventListener('click', async ()
=> {
            const invoiceId = document.getElementById('pay-invoice-select').value;
            if (!invoiceId) {
                Toast.error('Debe seleccionar una factura');
                return;
            }
            const invoice = pendingInvoices.find(i => i.id === invoiceId);
            if (!invoice) return;
            const lines = Array.from(document.querySelectorAll('.pay-payment-
line')).map(row => ({
                method: row.querySelector('.pay-method').value,
                bankId: row.querySelector('.pay-bank').value,
                amount: parseFloat(row.querySelector('.pay-amount').value) || 0
            })).filter(l => l.amount > 0);
            const totalPaid = lines.reduce((acc, l) => acc + l.amount, 0);
            if (totalPaid <= 0) {
                Toast.error('Debe ingresar un monto válido > 0');
                return;
            }
            if (totalPaid > parseFloat((invoice.total - (invoice.paid || 0)).toFixed(2)) +
0.1) {
                Toast.error('El monto a pagar supera el saldo de la factura');
                return;
            }
            const loading = Toast.loading('Registrando pagos múltiples...');
            try {
                const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
                let suppliersAccount = accounts.find(a => a.code === '2.1.01' ||
a.name?.toLowerCase().includes('proveedor'));
```

```
                if (!suppliersAccount) {
                    suppliersAccount = { id: Helpers.generateId(), companyId: company.id,
code: '2.1.01', name: 'Proveedores', type: 'liability', nature: 'credit', level: 2,
isActive: true };
                    await DB.add('accounts', suppliersAccount);
                }
                const transactionData = {
                    date: document.getElementById('pay-date').value,
                    description: `Pago Fact ${invoice.number} (${lines.length} vías)`,
                    reference: `PAG-${invoice.number}`,
                    sourceDocument: 'supplierPayment',
                    sourceDocumentId: invoice.id,
                    autoPost: true,
                    lines: []
                };
                const supplier = await DB.get('suppliers', invoice.supplierId);
                transactionData.lines.push({
                    accountId: suppliersAccount.id,
                    description: `Pago a ${supplier?.name || 'Proveedor'} Fact.
${invoice.number}`,
                    debit: totalPaid,
                    credit: 0
                });
                const currentDateIso = new Date().toISOString();
                for (const pl of lines) {
                    await DB.add('supplierPayments', {
                        companyId: company.id,
                        invoiceId: invoice.id,
                        supplierId: invoice.supplierId,
                        date: transactionData.date,
                        amount: pl.amount,
                        paymentMethod: pl.method,
                        bankAccountId: pl.bankId || null,
                        reference: document.getElementById('pay-notes').value,
                        status: 'completed',
                        createdAt: currentDateIso
                    });
                    let srcAccount;
                    if (pl.method === 'check') {
                        srcAccount = accounts.find(a => a.code === '2.1.03' ||
a.name?.toLowerCase().includes('cuentas por pagar')) ||
                            { id: Helpers.generateId(), companyId: company.id, code:
'2.1.03', name: 'Cuentas por Pagar', type: 'liability', nature: 'credit', level: 2,
isActive: true };
                    } else if (pl.bankId) {
                        const bankAccount = await DB.get('bankAccounts', pl.bankId);
                        if (bankAccount) {
                            await DB.update('bankAccounts', { ...bankAccount, balance:
bankAccount.balance - pl.amount });
                        }
                        srcAccount = accounts.find(a => a.code === '1.1.01' ||
a.name?.toLowerCase() === 'banco') ||
                            { id: Helpers.generateId(), companyId: company.id, code:
'1.1.01', name: 'Banco', type: 'asset', nature: 'debit', level: 2, isActive: true };
                    } else {
                        srcAccount = accounts.find(a => a.code === '1.1.02' ||
a.name?.toLowerCase() === 'caja') ||
                            { id: Helpers.generateId(), companyId: company.id, code:
'1.1.02', name: 'Caja', type: 'asset', nature: 'debit', level: 2, isActive: true };
                    }
                    if (!srcAccount.id) await DB.add('accounts', srcAccount);
                    transactionData.lines.push({
                        accountId: srcAccount.id,
                        description: `Pago Fact. ${invoice.number} vía
${pl.method}${pl.method === 'check' ? ' (Cheque Pendiente)' : ' (Efectivo/Banco)'}`,
                        debit: 0,
                        credit: pl.amount
                    });
                }
                await AccountingService.registerTransaction('treasury', transactionData);
                const newPaid = (invoice.paid || 0) + totalPaid;
                const newStatus = parseFloat(newPaid.toFixed(2)) >=
parseFloat(invoice.total.toFixed(2)) ? 'paid' : 'partial';
                await DB.update('supplierInvoices', {
                    ...invoice,
                    paid: newPaid,
                    balance: invoice.total - newPaid,
                    status: newStatus
                });
                if (supplier) {
                    await DB.update('suppliers', {
```

```
                        ...supplier,
                        balance: Math.max(0, (supplier.balance || 0) - totalPaid)
                    });
                }
                loading.success('Pagos registrados exitosamente');
                Modal.close();
                App.navigate('tesoreria', 'cuentas-pagar');
            } catch (err) {
                loading.error('Error: ' + err.message);
                console.error(err);
            }
        });
    },
    async initAccountsReceivable() {
        const company = CompanyService.getCurrent();
        const invoices = await DB.getByIndex('customerInvoices', 'companyId', company.id);
        const pending = invoices.filter(i => i.status !== 'paid' && i.status !==
'cancelled');
        for (const inv of pending) {
            if (!inv.customerName) {
                const customer = await DB.get('customers', inv.customerId);
                inv.customerName = customer?.name || 'Cliente desconocido';
            }
        }
        DataTable.create('receivables-table', {
            columns: [
                { key: 'number', label: 'Factura' },
                { key: 'customerName', label: 'Cliente' },
                { key: 'date', label: 'Fecha', format: 'date' },
                { key: 'dueDate', label: 'Vencimiento', format: 'date' },
                { key: 'total', label: 'Total', format: 'currency', class: 'text-right' },
                { key: 'balance', label: 'Saldo', format: 'currency', class: 'text-right'
},
                { key: 'status', label: 'Estado', format: 'status' },
                {
                    key: 'actions', label: '', render: (_, row) => `
                        <button class="btn btn-ghost btn-sm"
onclick="TesoreriaModule.showCollectionModal('${row.id}')" title="Registrar Cobro">
                            <i class="fas fa-hand-holding-usd"></i>
                        </button>
                    `
                }
            ],
            data: pending,
            emptyMessage: 'No hay cobros pendientes'
        });
        document.getElementById('btn-register-collection')?.addEventListener('click', ()
=> {
            this.showCollectionModal();
        });
    },
    async showCollectionModal(invoiceId = null) {
        const company = CompanyService.getCurrent();
        if (!company) {
            Toast.error('Seleccione una empresa primero');
            return;
        }
        const invoices = await DB.getByIndex('customerInvoices', 'companyId', company.id);
        const pendingInvoices = invoices.filter(i => i.status !== 'paid' && i.status !==
'cancelled');
        const bankAccounts = await DB.getByIndex('bankAccounts', 'companyId', company.id);
        if (pendingInvoices.length === 0) {
            Toast.info('No hay facturas pendientes de cobro');
            return;
        }
        for (const inv of pendingInvoices) {
            if (!inv.customerName) {
                const customer = await DB.get('customers', inv.customerId);
                inv.customerName = customer?.name || 'Cliente desconocido';
            }
        }
        const bankOptions = `<option value="">Efectivo (Caja)</option>` +
            bankAccounts.map(a => `<option value="${a.id}">${a.bank} - ${a.accountNumber}
</option>`).join('');
        Modal.open({
            title: 'Registrar Cobro Multiple',
            size: 'large',
            content: `
                <div class="form-row">
                    <div class="form-group" style="flex: 2;">
                        <label class="form-label">Factura a Cobrar <span class="text-
```

```
danger">*</span></label>
                        <select class="form-control" id="col-invoice-select" required>
                            <option value="">Seleccione factura...</option>
                            ${pendingInvoices.map(i => `<option value="${i.id}" data-
pending="${i.total - (i.paid || 0)}" ${invoiceId === i.id ? 'selected' : ''}>${i.number} -
${i.customerName} - PEND: ${Formatters.currency(i.total - (i.paid || 0))}
</option>`).join('')}
                        </select>
                    </div>
                    <div class="form-group" style="flex: 1;">
                        <label class="form-label">Fecha de Cobro</label>
                        <input type="date" class="form-control" id="col-date"
value="${Helpers.getCurrentDate()}" required>
                    </div>
                </div>
                <div class="alert alert-info" style="margin: 1rem 0;">
                    Puede distribuir el pago combinando múltiples métodos y definiendo
porcentajes.
                </div>
                <div class="table-container" style="margin-bottom: 1rem;">
                    <table class="data-table">
                        <thead>
                            <tr>
                                <th style="width: 25%">Método de Pago</th>
                                <th style="width: 25%">Cuenta Destino</th>
                                <th style="width: 15%">Porcentaje %</th>
                                <th style="width: 25%">Monto</th>
                                <th style="width: 10%"></th>
                            </tr>
                        </thead>
                        <tbody id="col-payment-lines">
                        </tbody>
                        <tfoot>
                            <tr>
                                <td colspan="3" class="text-right"><strong>Distribuido /
Pendiente:</strong></td>
                                <td colspan="2">
                                    <span id="col-amount-paid" style="font-weight:bold;
color: var(--primary-600)">$0</span> /
                                    <span id="col-amount-required" style="font-
weight:bold;">$0</span>
                                </td>
                            </tr>
                        </tfoot>
                    </table>
                </div>
                <button class="btn btn-outline btn-sm" id="btn-add-col-line"><i class="fas
fa-plus"></i> Añadir Línea de Pago</button>
                <div class="form-group" style="margin-top: 1rem;">
                    <label class="form-label">Notas o Referencia General</label>
                    <textarea class="form-control" id="col-notes" rows="2"
placeholder="Observaciones adicionales, n° comprobantes"></textarea>
                </div>
            `,
            footer: `
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-primary" id="btn-confirm-collection"><i class="fas
fa-check"></i> Registrar Cobro</button>
            `
        });
        let targetAmount = 0;
        const updateRowTotals = () => {
            let total = 0;
            const rows = document.querySelectorAll('.col-payment-line');
            rows.forEach(row => {
                const amount = parseFloat(row.querySelector('.col-amount').value) || 0;
                total += amount;
                const pctField = row.querySelector('.col-percent');
                if (targetAmount > 0) {
                    if (document.activeElement === row.querySelector('.col-amount')) {
                        pctField.value = ((amount / targetAmount) * 100).toFixed(2);
                    }
                }
            });
            document.getElementById('col-amount-paid').textContent =
Formatters.currency(total);
            const remaining = targetAmount - total;
            document.getElementById('col-amount-required').innerHTML = `<span
style="color: ${Math.abs(remaining) < 0.01 ? 'green' :
'red'}">${Formatters.currency(targetAmount)}</span>`;
```

```
        };
        const addLine = (monto = 0) => {
            const tbody = document.getElementById('col-payment-lines');
            const tr = document.createElement('tr');
            tr.className = 'col-payment-line';
            tr.innerHTML = `
                <td>
                    <select class="form-control col-method">
                        <option value="transfer">Transferencia</option>
                        <option value="cash">Efectivo</option>
                        <option value="check">Cheque</option>
                        <option value="card">Tarjeta</option>
                    </select>
                </td>
                <td>
                    <select class="form-control col-bank">${bankOptions}</select>
                </td>
                <td>
                    <input type="number" class="form-control col-percent" min="0"
max="100" step="any" placeholder="%">
                </td>
                <td>
                    <input type="number" class="form-control col-amount" min="0"
step="any" value="${monto || ''}">
                </td>
                <td>
                    <button class="btn btn-icon btn-ghost btn-remove-row"><i class="fas
fa-trash text-danger"></i></button>
                </td>
            `;
            tbody.appendChild(tr);
            tr.querySelector('.btn-remove-row').onclick = () => {
                tr.remove();
                updateRowTotals();
            };
            const amtField = tr.querySelector('.col-amount');
            const pctField = tr.querySelector('.col-percent');
            amtField.oninput = updateRowTotals;
            pctField.oninput = () => {
                if (targetAmount > 0) {
                    const pct = parseFloat(pctField.value) || 0;
                    amtField.value = (targetAmount * (pct / 100)).toFixed(2);
                    updateRowTotals();
                }
            };
            if (monto > 0 && targetAmount > 0) {
                pctField.value = ((monto / targetAmount) * 100).toFixed(2);
            }
            const methodSelect = tr.querySelector('.col-method');
            const bankSelect = tr.querySelector('.col-bank');
            const updateAccountLabel = () => {
                const method = methodSelect.value;
                const firstOption = bankSelect.options[0];
                if (method === 'check' || method === 'card') {
                    firstOption.textContent = 'Cuentas por Cobrar (Pendiente)';
                } else {
                    firstOption.textContent = 'Efectivo (Caja)';
                }
            };
            methodSelect.addEventListener('change', updateAccountLabel);
            updateAccountLabel();
        };
        const onInvoiceChange = () => {
            const select = document.getElementById('col-invoice-select');
            const option = select.selectedOptions[0];
            targetAmount = parseFloat(option?.dataset.pending) || 0;
            document.getElementById('col-payment-lines').innerHTML = '';
            if (targetAmount > 0) {
                addLine(targetAmount);
            }
            updateRowTotals();
        };
        document.getElementById('col-invoice-select').addEventListener('change',
onInvoiceChange);
        document.getElementById('btn-add-col-line').addEventListener('click', () =>
addLine(0));
        if (invoiceId) setTimeout(onInvoiceChange, 50);
        document.getElementById('btn-confirm-collection').addEventListener('click', async
() => {
            const invoiceId = document.getElementById('col-invoice-select').value;
            if (!invoiceId) {
```

```
                Toast.error('Debe seleccionar una factura');
                return;
            }
            const invoice = pendingInvoices.find(i => i.id === invoiceId);
            if (!invoice) return;
            const lines = Array.from(document.querySelectorAll('.col-payment-
line')).map(row => ({
                method: row.querySelector('.col-method').value,
                bankId: row.querySelector('.col-bank').value,
                amount: parseFloat(row.querySelector('.col-amount').value) || 0
            })).filter(l => l.amount > 0);
            const totalCollected = lines.reduce((acc, l) => acc + l.amount, 0);
            if (totalCollected <= 0) {
                Toast.error('Debe ingresar un monto válido > 0');
                return;
            }
            if (totalCollected > parseFloat((invoice.total - (invoice.paid ||
0)).toFixed(2)) + 0.1) {
                Toast.error('El monto a cobrar supera el saldo de la factura');
                return;
            }
            const loading = Toast.loading('Registrando cobros múltiples...');
            try {
                const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
                let customersAccount = accounts.find(a => a.code === '1.1.05' ||
a.name?.toLowerCase().includes('clientes por cobrar'));
                if (!customersAccount) {
                    customersAccount = { id: Helpers.generateId(), companyId: company.id,
code: '1.1.05', name: 'Clientes por Cobrar', type: 'asset', nature: 'debit', level: 2,
isActive: true };
                    await DB.add('accounts', customersAccount);
                }
                const transactionData = {
                    date: document.getElementById('col-date').value,
                    description: `Cobro Fact ${invoice.number} (${lines.length} vías)`,
                    reference: `MUL-${invoice.number}`,
                    sourceDocument: 'customerInvoice',
                    sourceDocumentId: invoice.id,
                    autoPost: true,
                    lines: []
                };
                const customer = await DB.get('customers', invoice.customerId);
                transactionData.lines.push({
                    accountId: customersAccount.id,
                    description: `Cobro de ${customer?.name || 'Cliente'} Fact.
${invoice.number}`,
                    debit: 0,
                    credit: totalCollected
                });
                const currentDateIso = new Date().toISOString();
                for (const pl of lines) {
                    await DB.add('customerPayments', {
                        companyId: company.id,
                        invoiceId: invoice.id,
                        customerId: invoice.customerId,
                        date: transactionData.date,
                        amount: pl.amount,
                        paymentMethod: pl.method,
                        bankAccountId: pl.bankId || null,
                        reference: document.getElementById('col-notes').value,
                        status: 'completed',
                        createdAt: currentDateIso
                    });
                    let destAccount;
                    if (pl.method === 'check') {
                        destAccount = accounts.find(a => a.code === '1.1.07' ||
a.name?.toLowerCase().includes('cuentas por cobrar')) ||
                            { id: Helpers.generateId(), companyId: company.id, code:
'1.1.07', name: 'Cuentas por Cobrar', type: 'asset', nature: 'debit', level: 2, isActive:
true };
                    } else if (pl.bankId) {
                        const bankAccount = await DB.get('bankAccounts', pl.bankId);
                        if (bankAccount) {
                            await DB.update('bankAccounts', { ...bankAccount, balance:
bankAccount.balance + pl.amount });
                        }
                        destAccount = accounts.find(a => a.code === '1.1.01' ||
a.name?.toLowerCase() === 'banco') ||
                            { id: Helpers.generateId(), companyId: company.id, code:
'1.1.01', name: 'Banco', type: 'asset', nature: 'debit', level: 2, isActive: true };
                    } else {
```

```
                        destAccount = accounts.find(a => a.code === '1.1.02' ||
a.name?.toLowerCase() === 'caja') ||
                            { id: Helpers.generateId(), companyId: company.id, code:
'1.1.02', name: 'Caja', type: 'asset', nature: 'debit', level: 2, isActive: true };
                    }
                    if (!destAccount.id) await DB.add('accounts', destAccount);
                    transactionData.lines.push({
                        accountId: destAccount.id,
                        description: `Cobro Fact. ${invoice.number} vía
${pl.method}${pl.method === 'check' ? ' (Cheque Pendiente)' : ' (Efectivo/Banco)'}`,
                        debit: pl.amount,
                        credit: 0
                    });
                }
                await AccountingService.registerTransaction('treasury', transactionData);
                const newPaid = (invoice.paid || 0) + totalCollected;
                const newStatus = parseFloat(newPaid.toFixed(2)) >=
parseFloat(invoice.total.toFixed(2)) ? 'paid' : 'partial';
                await DB.update('customerInvoices', {
                    ...invoice,
                    paid: newPaid,
                    balance: invoice.total - newPaid,
                    status: newStatus
                });
                if (customer) {
                    await DB.update('customers', {
                        ...customer,
                        balance: Math.max(0, (customer.balance || 0) - totalCollected)
                    });
                }
                loading.success('Cobros registrados exitosamente');
                Modal.close();
                App.navigate('tesoreria', 'cuentas-cobrar');
            } catch (err) {
                loading.error('Error: ' + err.message);
                console.error(err);
            }
        });
    },
    async initBanks() {
        const company = CompanyService.getCurrent();
        document.getElementById('btn-new-account')?.addEventListener('click', () =>
this.showBankAccountModal());
        document.getElementById('btn-new-transaction')?.addEventListener('click', () =>
this.showTransactionModal());
        document.getElementById('select-account')?.addEventListener('change', async (e) =>
{
            if (e.target.value) {
                await this.loadTransactions(e.target.value);
            }
        });
    },
    async showBankAccountModal(accountId = null) {
        const isEdit = !!accountId;
        let data = {};
        if (isEdit) {
            data = await DB.get('bankAccounts', accountId);
        }
        await Modal.form({
            title: isEdit ? 'Editar Cuenta Bancaria' : 'Nueva Cuenta Bancaria',
            fields: [
                { name: 'bank', label: 'Banco', required: true, placeholder: 'Ej: Banco
Estado', default: data.bank },
                { name: 'accountNumber', label: 'Número de Cuenta', required: true,
placeholder: '12345678901', default: data.accountNumber },
                {
                    name: 'accountType', label: 'Tipo de Cuenta', type: 'select', options:
[
                        { value: 'Cuenta Corriente', label: 'Cuenta Corriente' },
                        { value: 'Cuenta Vista', label: 'Cuenta Vista' },
                        { value: 'Cuenta de Ahorro', label: 'Cuenta de Ahorro' }
                    ], default: data.accountType
                },
                { name: 'balance', label: 'Saldo Inicial', type: 'number', default:
data.balance || 0 }
            ],
            onSubmit: async (formData) => {
                const company = CompanyService.getCurrent();
                if (isEdit) {
                    await DB.update('bankAccounts', { ...data, ...formData });
                    Toast.success('Cuenta actualizada');
```

```
                } else {
                    await DB.add('bankAccounts', { companyId: company.id, ...formData,
currency: 'CLP', status: 'active' });
                    Toast.success('Cuenta creada');
                }
                App.navigate('tesoreria', 'bancos');
            }
        });
    },
    async showTransactionModal() {
        Toast.info('Nuevo movimiento bancario');
    },
    async loadTransactions(accountId) {
        const transactions = await DB.getByIndex('bankTransactions', 'bankAccountId',
accountId);
        DataTable.create('transactions-table', {
            columns: [
                { key: 'date', label: 'Fecha', format: 'date' },
                { key: 'description', label: 'Descripción' },
                { key: 'reference', label: 'Referencia' },
                {
                    key: 'type', label: 'Tipo', render: (v) => {
                        return v === 'deposit' ? '<span class="badge badge-
success">Depósito</span>' :
                            v === 'withdrawal' ? '<span class="badge badge-
error">Retiro</span>' :
                                '<span class="badge badge-neutral">Otro</span>';
                    }
                },
                { key: 'amount', label: 'Monto', format: 'currency', class: 'text-right' }
            ],
            data: transactions,
            emptyMessage: 'No hay movimientos en esta cuenta'
        });
    },
    initReconciliation() {
        document.getElementById('btn-start-reconciliation')?.addEventListener('click', ()
=> {
            Toast.info('Iniciar conciliación bancaria');
        });
    }
};
window.TesoreriaModule = TesoreriaModule;
```

Archivo: js\modules\trazabilidad.js

```
const TrazabilidadModule = {
    currentView: 'overview',
    async render(view = 'overview') {
        this.currentView = view;
        const company = CompanyService.getCurrent();
        if (!company) {
            return this.renderNoCompany();
        }
        switch (view) {
            case 'overview':
                return await this.renderOverview();
            case 'trace-document':
                return await this.renderTraceDocument();
            case 'flow-diagrams':
                return await this.renderFlowDiagrams();
            case 'validations':
                return await this.renderValidations();
            case 'progress':
                return await this.renderProgress();
            default:
                return await this.renderOverview();
        }
    },
    renderNoCompany() {
        return `
            <div class="empty-state">
                <i class="fas fa-route"></i>
                <h3>Trazabilidad Pedagógica</h3>
                <p>Selecciona una empresa para acceder al módulo de trazabilidad.</p>
```

```
            </div>
        `;
    },
    async renderOverview() {
        const progress = await ValidationService.getTransactionProgress();
        const integrity = await TraceabilityService.verifyDataIntegrity();
        const todayTx = await TraceabilityService.getTodayTransactions();
        return `
            <div class="trazabilidad-module">
                <!-- Header del módulo -->
                <div class="module-header glass-card">
                    <div class="header-content">
                        <div class="header-icon">
                            <i class="fas fa-route"></i>
                        </div>
                        <div class="header-text">
                            <h1>Trazabilidad Pedagógica</h1>
                            <p>Sigue cada transacción desde su origen hasta los estados
financieros</p>
                        </div>
                    </div>
                    <div class="header-actions">
                        <button class="btn btn-primary"
onclick="TrazabilidadModule.showQuickTrace()">
                            <i class="fas fa-search"></i> Trazar Documento
                        </button>
                    </div>
                </div>
                <!-- Grid principal -->
                <div class="trazabilidad-grid">
                    <!-- Progreso del ejercicio -->
                    <div class="card span-2 progress-card">
                        <div class="card-header">
                            <div class="card-title">
                                <i class="fas fa-tasks"></i>
                                Progreso del Ejercicio
                            </div>
                            <span class="badge ${progress.percentage === 100 ? 'badge-
success' : 'badge-warning'}">
                                ${progress.percentage}% Completado
                            </span>
                        </div>
                        <div class="card-body">
                            <div class="progress-overview">
                                <div class="progress-ring" style="--progress:
${progress.percentage}">
                                    <div class="progress-ring-inner">
                                        <span class="progress-
value">${progress.completed}/${progress.total}</span>
                                        <span class="progress-label">Pasos</span>
                                    </div>
                                </div>
                                <div class="progress-steps">
                                    ${progress.transactions.map((tx, idx) => `
                                        <div class="progress-step ${tx.completed ?
'completed' : ''}"
                                             onclick="App.navigate('${tx.module}',
'${tx.view}')">
                                            <div class="step-indicator">
                                                ${tx.completed ? '<i class="fas fa-check">
</i>' : idx + 1}
                                            </div>
                                            <div class="step-content">
                                                <div class="step-name">${tx.name}</div>
                                                <div class="step-desc">${tx.description}
</div>
                                            </div>
                                            <i class="fas fa-chevron-right step-arrow">
</i>
                                        </div>
                                    `).join('')}
                                </div>
                            </div>
                        </div>
                    </div>
                    <!-- Integridad de datos -->
                    <div class="card span-2 integrity-card">
                        <div class="card-header">
                            <div class="card-title">
                                <i class="fas fa-shield-alt"></i>
                                Integridad de Datos
```

```
                            </div>
                            <span class="badge ${integrity.isHealthy ? 'badge-success' :
'badge-error'}">
                                ${integrity.isHealthy ? 'Saludable' :
`${integrity.summary.errors} Errores`}
                            </span>
                        </div>
                        <div class="card-body">
                            ${integrity.issues.length === 0 ? `
                                <div class="integrity-healthy">
                                    <i class="fas fa-check-circle"></i>
                                    <p>Todos los datos están sincronizados correctamente
entre módulos</p>
                                </div>
                            ` : `
                                <div class="integrity-issues">
                                    ${integrity.issues.slice(0, 5).map(issue => `
                                        <div class="issue-item ${issue.severity}">
                                            <i class="fas fa-${issue.severity === 'error'
? 'times-circle' : 'exclamation-triangle'}"></i>
                                            <span>${issue.message}</span>
                                        </div>
                                    ).join('')}
                                    ${integrity.issues.length > 5 ? `
                                        <button class="btn btn-link"
onclick="TrazabilidadModule.showAllIssues()">
                                            Ver todos (${integrity.issues.length})
                                        </button>
                                    ` : ''}
                                </div>
                            `}
                        </div>
                    </div>
                    <!-- Transacciones recientes -->
                    <div class="card span-4">
                        <div class="card-header">
                            <div class="card-title">
                                <i class="fas fa-history"></i>
                                Transacciones de Hoy
                            </div>
                            <button class="btn btn-ghost btn-sm"
onclick="TrazabilidadModule.refreshTransactions()">
                                <i class="fas fa-sync-alt"></i>
                            </button>
                        </div>
                        <div class="card-body">
                            ${todayTx.length === 0 ? `
                                <div class="empty-transactions">
                                    <i class="fas fa-inbox"></i>
                                    <p>No hay transacciones registradas hoy</p>
                                    <button class="btn btn-outline btn-sm"
onclick="App.showQuickEntry()">
                                        <i class="fas fa-plus"></i> Crear Primera
Transacción
                                    </button>
                                </div>
                            ` : `
                                <div class="transactions-list">
                                    ${todayTx.slice(0, 10).map(tx => `
                                        <div class="transaction-item"
onclick="TrazabilidadModule.traceDocument('${tx.type}', '${tx.document.id}')">
                                            <div class="tx-icon" style="background:
${tx.config.color}20; color: ${tx.config.color}">
                                                <i class="fas ${tx.config.icon}"></i>
                                            </div>
                                            <div class="tx-content">
                                                <div class="tx-title">${tx.config.name}
</div>
                                                <div class="tx-
number">${tx.document.number || '#' + tx.document.id}</div>
                                            </div>
                                            <div class="tx-meta">
                                                <span class="tx-
module">${tx.config.module}</span>
                                                <button class="btn btn-ghost btn-icon btn-
xs" title="Trazar">
                                                    <i class="fas fa-route"></i>
                                                </button>
                                            </div>
                                        </div>
                                    ).join('')}
```

```
                                </div>
                            `}
                        </div>
                    </div>
                    <!-- Flujos de negocio estándar -->
                    <div class="card span-4">
                        <div class="card-header">
                            <div class="card-title">
                                <i class="fas fa-project-diagram"></i>
                                Flujos de Negocio Estándar
                            </div>
                        </div>
                        <div class="card-body">
                            <div class="business-flows">

${Object.entries(TraceabilityService.standardFlows).map(([key, flow]) => `
                                    <div class="flow-card"
onclick="TrazabilidadModule.showFlowDiagram('${key}')">
                                        <div class="flow-header">
                                            <h4>${flow.name}</h4>
                                            <span class="badge">${flow.steps.length}
pasos</span>
                                        </div>
                                        <p class="flow-description">${flow.description}
</p>
                                        <div class="flow-preview">
                                            ${flow.steps.slice(0, 4).map((step, idx) => `
                                                <div class="flow-step-mini">
                                                    <div class="step-dot"
style="background: ${TraceabilityService.documentTypes[step.type]?.color || '#666'}">
</div>
                                                    ${idx < 3 ? '<div class="step-line">
</div>' : ''}
                                                </div>
                                            `).join('')}
                                            ${flow.steps.length > 4 ? `<span class="more-
steps">+${flow.steps.length - 4}</span>` : ''}
                                        </div>
                                        <div class="flow-footer">
                                            <button class="btn btn-outline btn-sm">
                                                <i class="fas fa-eye"></i> Ver Diagrama
                                            </button>
                                        </div>
                                    </div>
                                `).join('')}
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        `;
    },
    async renderTraceDocument() {
        return `
            <div class="trace-document-view">
                <div class="card">
                    <div class="card-header">
                        <div class="card-title">
                            <i class="fas fa-search"></i>
                            Trazar Documento
                        </div>
                    </div>
                    <div class="card-body">
                        <form id="trace-form" class="trace-form">
                            <div class="form-row">
                                <div class="form-group">
                                    <label>Tipo de Documento</label>
                                    <select id="trace-doc-type" class="form-control"
required>
                                        <option value="">Seleccionar...</option>

${Object.entries(TraceabilityService.documentTypes).map(([key, dt]) => `
                                            <option value="${key}">${dt.name}
(${dt.module})</option>
                                        `).join('')}
                                    </select>
                                </div>
                                <div class="form-group">
                                    <label>Número o ID del Documento</label>
                                    <input type="text" id="trace-doc-id" class="form-
control"
```

```
                                            placeholder="Ej: FAC-2024-0001" required>
                                </div>
                                <div class="form-group form-actions">
                                    <button type="submit" class="btn btn-primary">
                                        <i class="fas fa-search"></i> Trazar
                                    </button>
                                </div>
                            </div>
                        </form>
                    </div>
                </div>
                <!-- Área de resultados -->
                <div id="trace-results" class="trace-results"></div>
            </div>
        `;
    },
    async renderFlowDiagrams() {
        return `
            <div class="flow-diagrams-view">
                <div class="card">
                    <div class="card-header">
                        <div class="card-title">
                            <i class="fas fa-project-diagram"></i>
                            Diagramas de Flujo ERP
                        </div>
                    </div>
                    <div class="card-body">
                        <div class="flow-selector">
                            ${Object.entries(TraceabilityService.standardFlows).map(([key,
flow]) => `
                                <button class="flow-tab" data-flow="${key}">
                                    ${flow.name}
                                </button>
                            `).join('')}
                        </div>
                        <div id="flow-diagram-container" class="flow-diagram-container">
                            <!-- El diagrama se cargará aquí -->
                            <div class="flow-placeholder">
                                <i class="fas fa-hand-pointer"></i>
                                <p>Selecciona un flujo para ver su diagrama</p>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        `;
    },
    async renderValidations() {
        const company = CompanyService.getCurrent();
        const entries = await AccountingService.getJournalEntries({ status: 'draft' });
        return `
            <div class="validations-view">
                <div class="card">
                    <div class="card-header">
                        <div class="card-title">
                            <i class="fas fa-check-double"></i>
                            Centro de Validaciones
                        </div>
                        <button class="btn btn-primary btn-sm"
onclick="TrazabilidadModule.runAllValidations()">
                            <i class="fas fa-play"></i> Ejecutar Validaciones
                        </button>
                    </div>
                    <div class="card-body">
                        <!-- Panel de reglas -->
                        <div class="validation-rules">
                            <h4>Reglas de Validación Activas</h4>
                            <div class="rules-grid">
                                <div class="rule-card">
                                    <div class="rule-icon success">
                                        <i class="fas fa-balance-scale"></i>
                                    </div>
                                    <div class="rule-content">
                                        <h5>Debe = Haber</h5>
                                        <p>Verifica que todos los asientos estén
cuadrados</p>
                                    </div>
                                    <span class="rule-status active">Activa</span>
                                </div>
                                <div class="rule-card">
                                    <div class="rule-icon success">
```

```
                                        <i class="fas fa-list-alt"></i>
                                    </div>
                                    <div class="rule-content">
                                        <h5>Cuenta en Plan</h5>
                                        <p>Verifica que las cuentas existan en el plan de
cuentas</p>
                                    </div>
                                    <span class="rule-status active">Activa</span>
                                </div>
                                <div class="rule-card">
                                    <div class="rule-icon success">
                                        <i class="fas fa-arrows-alt-h"></i>
                                    </div>
                                    <div class="rule-content">
                                        <h5>Naturaleza Correcta</h5>
                                        <p>Verifica movimientos según naturaleza
deudora/acreedora</p>
                                    </div>
                                    <span class="rule-status active">Activa</span>
                                </div>
                                <div class="rule-card">
                                    <div class="rule-icon success">
                                        <i class="fas fa-id-card"></i>
                                    </div>
                                    <div class="rule-content">
                                        <h5>RUT Válido</h5>
                                        <p>Valida formato y dígito verificador de RUT</p>
                                    </div>
                                    <span class="rule-status active">Activa</span>
                                </div>
                                <div class="rule-card">
                                    <div class="rule-icon success">
                                        <i class="fas fa-calendar-check"></i>
                                    </div>
                                    <div class="rule-content">
                                        <h5>Fechas Coherentes</h5>
                                        <p>Verifica fechas no futuras y en período
abierto</p>
                                    </div>
                                    <span class="rule-status active">Activa</span>
                                </div>
                                <div class="rule-card">
                                    <div class="rule-icon success">
                                        <i class="fas fa-sort-numeric-down"></i>
                                    </div>
                                    <div class="rule-content">
                                        <h5>Secuencia Lógica</h5>
                                        <p>Verifica orden de operaciones (venta antes de
cobro)</p>
                                    </div>
                                    <span class="rule-status active">Activa</span>
                                </div>
                            </div>
                        </div>
                        <!-- Resultados de validación -->
                        <div id="validation-results" class="validation-results">
                            <div class="results-placeholder">
                                <i class="fas fa-clipboard-check"></i>
                                <p>Ejecuta las validaciones para ver los resultados</p>
                            </div>
                        </div>
                        <!-- Asientos pendientes de validar -->
                        ${entries.length > 0 ? `
                            <div class="pending-entries">
                                <h4>Asientos en Borrador (${entries.length})</h4>
                                <div class="entries-list">
                                    ${entries.slice(0, 5).map(entry => `
                                        <div class="entry-item"
onclick="TrazabilidadModule.validateEntry('${entry.id}')">
                                            <span class="entry-number">${entry.number}
</span>
                                            <span class="entry-date">${entry.date}</span>
                                            <span class="entry-desc">${entry.description}
</span>
                                            <button class="btn btn-ghost btn-xs">
                                                <i class="fas fa-check"></i> Validar
                                            </button>
                                        </div>
                                    `).join('')}
                                </div>
                            </div>
```

```
                        ` : ''}
                    </div>
                </div>
            </div>
        `;
    },
    async renderProgress() {
        const progress = await ValidationService.getTransactionProgress();
        return `
            <div class="progress-view">
                <div class="card">
                    <div class="card-header">
                        <div class="card-title">
                            <i class="fas fa-graduation-cap"></i>
                            Progreso del Ejercicio Pedagógico
                        </div>
                    </div>
                    <div class="card-body">
                        <!-- Progreso visual -->
                        <div class="progress-hero">
                            <div class="progress-circle-large" style="--progress:
${progress.percentage}">
                                <div class="circle-content">
                                    <span class="percentage">${progress.percentage}%
</span>
                                    <span class="label">Completado</span>
                                </div>
                            </div>
                            <div class="progress-stats">
                                <div class="stat">
                                    <span class="stat-value">${progress.completed}</span>
                                    <span class="stat-label">Completados</span>
                                </div>
                                <div class="stat">
                                    <span class="stat-value">${progress.total -
progress.completed}</span>
                                    <span class="stat-label">Pendientes</span>
                                </div>
                                <div class="stat">
                                    <span class="stat-value">${progress.total}</span>
                                    <span class="stat-label">Total</span>
                                </div>
                            </div>
                        </div>
                        <!-- Timeline de pasos -->
                        <div class="progress-timeline">
                            ${progress.transactions.map((tx, idx) => `
                                <div class="timeline-item ${tx.completed ? 'completed' :
(idx === progress.completed ? 'current' : 'pending')}">
                                    <div class="timeline-marker">
                                        ${tx.completed
                ? '<i class="fas fa-check"></i>'
                : (idx === progress.completed ? '<i class="fas fa-play"></i>' : idx + 1)}
                                    </div>
                                    <div class="timeline-connector"></div>
                                    <div class="timeline-content">
                                        <h4>${tx.name}</h4>
                                        <p>${tx.description}</p>
                                        ${!tx.completed ? `
                                            <button class="btn btn-sm ${idx ===
progress.completed ? 'btn-primary' : 'btn-outline'}"
                                                    onclick="App.navigate('${tx.module}',
'${tx.view}')">
                                                ${idx === progress.completed ? 'Comenzar'
: 'Ir al módulo'}
                                            </button>
                                        ` : `
                                            <span class="completed-badge">
                                                <i class="fas fa-check-circle"></i>
Completado
                                            </span>
                                        `}
                                    </div>
                                </div>
                            `).join('')}
                        </div>
                        ${progress.completed === progress.total ? `
                            <div class="completion-message">
                                <div class="completion-icon">
                                    <i class="fas fa-trophy"></i>
                                </div>
```

```
                                <h3>¡Felicitaciones!</h3>
                                <p>Has completado todos los pasos del ejercicio
pedagógico.
                                   Ahora puedes revisar el Balance General para ver el
resultado de todas las operaciones.</p>
                                <button class="btn btn-primary"
onclick="App.navigate('reportes', 'balance-general')">
                                    <i class="fas fa-chart-bar"></i> Ver Estados
Financieros
                                </button>
                            </div>
                        ` : ''}
                    </div>
                </div>
            </div>
        `;
    },
    init(view) {
        this.bindEvents();
    },
    bindEvents() {
        document.getElementById('trace-form')?.addEventListener('submit', async (e) => {
            e.preventDefault();
            const type = document.getElementById('trace-doc-type').value;
            const id = document.getElementById('trace-doc-id').value;
            await this.executeTrace(type, id);
        });
        document.querySelectorAll('.flow-tab').forEach(tab => {
            tab.addEventListener('click', () => {
                document.querySelectorAll('.flow-tab').forEach(t =>
t.classList.remove('active'));
                tab.classList.add('active');
                this.showFlowDiagram(tab.dataset.flow);
            });
        });
    },
    async showQuickTrace() {
        const company = CompanyService.getCurrent();
        if (!company) {
            Toast.warning('Selecciona una empresa primero');
            return;
        }
        const invoices = await DB.getByIndex('customerInvoices', 'companyId', company.id);
        const orders = await DB.getByIndex('salesOrders', 'companyId', company.id);
        const entries = await DB.getByIndex('journalEntries', 'companyId', company.id);
        const recentDocs = [
            ...invoices.slice(-5).map(d => ({ type: 'customerInvoice', doc: d })),
            ...orders.slice(-5).map(d => ({ type: 'salesOrder', doc: d })),
            ...entries.slice(-5).map(d => ({ type: 'journalEntry', doc: d }))
        ].sort((a, b) => new Date(b.doc.date) - new Date(a.doc.date)).slice(0, 10);
        await Modal.custom({
            title: 'Trazar Documento',
            size: 'lg',
            content: `
                <div class="quick-trace-modal">
                    <div class="trace-search">
                        <div class="form-group">
                            <label>Tipo de Documento</label>
                            <select id="quick-trace-type" class="form-control">

${Object.entries(TraceabilityService.documentTypes).map(([key, dt]) => `
                                    <option value="${key}">${dt.name}</option>
                                ).join('')}
                            </select>
                        </div>
                        <div class="form-group">
                            <label>ID del Documento</label>
                            <input type="text" id="quick-trace-id" class="form-control"
                                   placeholder="ID numérico del documento">
                        </div>
                    </div>
                    ${recentDocs.length > 0 ? `
                        <div class="recent-docs">
                            <h4>Documentos Recientes</h4>
                            <div class="docs-list">
                                ${recentDocs.map(item => {
                const config = TraceabilityService.documentTypes[item.type];
                return `
                                        <div class="doc-item"
onclick="TrazabilidadModule.traceFromModal('${item.type}', '${item.doc.id}')">
                                            <div class="doc-icon" style="color:
```

```
${config.color}">
                                                <i class="fas ${config.icon}"></i>
                                            </div>
                                            <div class="doc-info">
                                                <span class="doc-name">${config.name}
</span>
                                                <span class="doc-number">${item.doc.number
|| '#' + item.doc.id}</span>
                                            </div>
                                            <span class="doc-date">${item.doc.date || ''}
</span>
                                        </div>
                                    `;
            }).join('')}
                        </div>
                    </div>
                ` : ''}
            </div>
            `,
            buttons: [
                { text: 'Cancelar', class: 'btn-outline', action: 'close' },
                {
                    text: 'Trazar',
                    class: 'btn-primary',
                    action: async () => {
                        const type = document.getElementById('quick-trace-type').value;
                        const id = document.getElementById('quick-trace-id').value;
                        if (id) {
                            Modal.close();
                            await new Promise(resolve => setTimeout(resolve, 300));
                            await this.traceDocument(type, id);
                        }
                    }
                }
            ]
        });
    },
    async traceFromModal(type, id) {
        Modal.close();
        await new Promise(resolve => setTimeout(resolve, 300));
        await this.traceDocument(type, id);
    },
    async traceDocument(documentType, documentId) {
        const loading = Toast.loading('Generando trazabilidad...');
        try {
            const report = await TraceabilityService.generateTraceReport(documentType,
documentId);
            if (!report || report.trace.error) {
                throw new Error(report?.trace?.error || 'Error al trazar documento');
            }
            loading.close();
            this.showTraceResults(report);
        } catch (error) {
            loading.error('Error: ' + error.message);
        }
    },
    async showTraceResults(report) {
        const { trace, visualization, summary, pedagogicalNotes } = report;
        await Modal.custom({
            title: 'Trazabilidad del Documento',
            size: 'xl',
            content: `
                <div class="trace-results-modal">
                    <!-- Summary -->
                    <div class="trace-summary">
                        <div class="summary-stat">
                            <span class="stat-label">Estado</span>
                            <span class="stat-value badge ${summary.status === 'Ciclo
Completo' ? 'badge-success' : 'badge-warning'}">
                                ${summary.status}
                            </span>
                        </div>
                        <div class="summary-stat">
                            <span class="stat-label">Documentos</span>
                            <span class="stat-value">${summary.documentsCount}</span>
                        </div>
                        <div class="summary-stat">
                            <span class="stat-label">Módulos</span>
                            <span class="stat-value">${summary.modulesInvolved.join(', ')}
</span>
                        </div>
```

```
                        ${summary.financials.totalDebit > 0 ? `
                            <div class="summary-stat">
                                <span class="stat-label">Total Débitos</span>
                                <span class="stat-
value">${Formatters.currency(summary.financials.totalDebit)}</span>
                            </div>
                            <div class="summary-stat">
                                <span class="stat-label">Total Créditos</span>
                                <span class="stat-
value">${Formatters.currency(summary.financials.totalCredit)}</span>
                            </div>
                        ` : ''}
                    </div>
                    <!-- Flow visualization -->
                    <div class="trace-flow">
                        <h4>Flujo de la Transacción</h4>
                        <div class="flow-visualization">
                            ${visualization.nodes.map((node, idx) => `
                                <div class="flow-node ${node.isOrigin ? 'origin' : ''}">
                                    <div class="node-icon" style="background:
${node.color}20; color: ${node.color}">
                                        <i class="fas ${node.icon}"></i>
                                    </div>
                                    <div class="node-content">
                                        <span class="node-label">${node.label}</span>
                                        <span class="node-number">${node.number}</span>
                                        <span class="node-module">${node.module}</span>
                                    </div>
                                </div>
                                ${idx < visualization.nodes.length - 1 ? `
                                    <div class="flow-arrow">
                                        <i class="fas fa-arrow-right"></i>
                                    </div>
                                ` : ''}
                            `).join('')}
                        </div>
                    </div>
                    <!-- Financial impact -->
                    ${trace.financialImpact.length > 0 ? `
                        <div class="financial-impact">
                            <h4>Impacto Financiero</h4>
                            <table class="table">
                                <thead>
                                    <tr>
                                        <th>Cuenta</th>
                                        <th class="text-right">Debe</th>
                                        <th class="text-right">Haber</th>
                                        <th>Descripción</th>
                                    </tr>
                                </thead>
                                <tbody>
                                    ${trace.financialImpact
                        .filter(i => i.debit !== undefined)
                        .map(impact => {
                            const accountDisplay = impact.account
                                ? (typeof impact.account === 'object'
                                    ? `${impact.account.code || ''} ${impact.account.name
|| ''}`.trim()
                                    : impact.account)
                                : 'N/A';
                            return `
                                            <tr>
                                                <td>${accountDisplay}</td>
                                                <td class="text-right">${impact.debit ?
Formatters.currency(impact.debit) : '-'}</td>
                                                <td class="text-right">${impact.credit ?
Formatters.currency(impact.credit) : '-'}</td>
                                                <td>${impact.description || ''}</td>
                                            </tr>
                                        `;
                        }).join('')}
                                </tbody>
                                <tfoot>
                                    <tr>
                                        <th>TOTALES</th>
                                        <th class="text-
right">${Formatters.currency(summary.financials.totalDebit)}</th>
                                        <th class="text-
right">${Formatters.currency(summary.financials.totalCredit)}</th>
                                        <th>
                                            ${summary.financials.isBalanced
```

```
                        ? '<span class="badge badge-success">Cuadrado</span>'
                        : '<span class="badge badge-error">Descuadrado</span>'}
                                        </th>
                                    </tr>
                                </tfoot>
                            </table>
                        </div>
                    ` : ''}
                    <!-- Pedagogical notes -->
                    <div class="pedagogical-notes">
                        <h4><i class="fas fa-graduation-cap"></i> Notas Pedagógicas</h4>
                        <div class="notes-grid">
                            ${pedagogicalNotes.map(note => `
                                <div class="note-card">
                                    <h5>${note.title}</h5>
                                    <p>${note.content}</p>
                                </div>
                            `).join('')}
                        </div>
                    </div>
                    <!-- Timeline -->
                    <div class="trace-timeline">
                        <h4>Línea de Tiempo</h4>
                        <div class="timeline">
                            ${trace.timeline.map((item, idx) => {
                            const config = TraceabilityService.documentTypes[item.type];
                            return `
                                    <div class="timeline-item">
                                        <div class="timeline-dot" style="background:
${config?.color || '#666'}"></div>
                                        <div class="timeline-content">
                                            <span class="timeline-date">${item.date || ''}
</span>
                                            <span class="timeline-
desc">${item.description}</span>
                                        </div>
                                    </div>
                                `;
                        }).join('')}
                        </div>
                    </div>
                </div>
            `,
            buttons: [
                { text: 'Cerrar', class: 'btn-outline', action: 'close' },
                {
                    text: 'Exportar PDF',
                    class: 'btn-primary',
                    action: () => this.exportTraceToPDF(report)
                }
            ]
        });
    },
    async showFlowDiagram(flowKey) {
        const flow = TraceabilityService.standardFlows[flowKey];
        if (!flow) return;
        const container = document.getElementById('flow-diagram-container');
        if (!container) {
            await this.showFlowDiagramModal(flowKey, flow);
            return;
        }
        container.innerHTML = `
            <div class="flow-diagram">
                <div class="flow-header">
                    <h3>${flow.name}</h3>
                    <p>${flow.description}</p>
                </div>
                <div class="flow-steps-diagram">
                    ${flow.steps.map((step, idx) => {
            const docType = TraceabilityService.documentTypes[step.type];
            return `
                            <div class="diagram-step">
                                <div class="step-box" style="border-color:
${docType?.color || '#666'}">
                                    <div class="step-icon" style="background:
${docType?.color || '#666'}">
                                        <i class="fas ${docType?.icon || 'fa-file'}"></i>
                                    </div>
                                    <div class="step-info">
                                        <span class="step-label">${step.label}</span>
                                        <span class="step-module">${docType?.module || ''}
```

```
</span>
                                    </div>
                                </div>
                                ${idx < flow.steps.length - 1 ? `
                                    <div class="step-connector">
                                        <div class="connector-line"></div>
                                        <div class="connector-arrow">
                                            <i class="fas fa-chevron-down"></i>
                                        </div>
                                    </div>
                                ` : ''}
                            </div>
                        `;
        }).join('')}
                </div>
                <div class="flow-accounts">
                    <h4>Cuentas Afectadas</h4>
                    <div class="accounts-list">
                        ${flow.affectedAccounts.map(acc => `
                            <div class="account-item">
                                <span class="account-code">${acc.code}</span>
                                <span class="account-name">${acc.name}</span>
                                <span class="account-effect">${acc.effect}</span>
                            </div>
                        `).join('')}
                    </div>
                </div>
            </div>
        `;
    },
    async showFlowDiagramModal(flowKey, flow) {
        await Modal.custom({
            title: flow.name,
            size: 'xl',
            content: `
                <div class="flow-diagram-modal">
                    <p class="flow-description">${flow.description}</p>
                    <div class="flow-visual">
                        ${flow.steps.map((step, idx) => {
                const docType = TraceabilityService.documentTypes[step.type];
                return `
                                <div class="flow-step-item">
                                    <div class="step-number">${idx + 1}</div>
                                    <div class="step-card" style="border-left-color:
${docType?.color || '#666'}">
                                        <div class="step-icon-wrapper" style="background:
${docType?.color || '#666'}20">
                                            <i class="fas ${docType?.icon || 'fa-file'}"
style="color: ${docType?.color || '#666'}"></i>
                                        </div>
                                        <div class="step-details">
                                            <strong>${step.label}</strong>
                                            <span>${docType?.module || 'Módulo'}</span>
                                        </div>
                                    </div>
                                    ${idx < flow.steps.length - 1 ? '<div class="step-
arrow"><i class="fas fa-arrow-down"></i></div>' : ''}
                                </div>
                            `;
            }).join('')}
                    </div>
                    <div class="affected-accounts">
                        <h4><i class="fas fa-book"></i> Impacto en Plan de Cuentas</h4>
                        <table class="table table-sm">
                            <thead>
                                <tr>
                                    <th>Código</th>
                                    <th>Cuenta</th>
                                    <th>Efecto</th>
                                </tr>
                            </thead>
                            <tbody>
                                ${flow.affectedAccounts.map(acc => `
                                    <tr>
                                        <td><code>${acc.code}</code></td>
                                        <td>${acc.name}</td>
                                        <td>${acc.effect}</td>
                                    </tr>
                                `).join('')}
                            </tbody>
                        </table>
```

```
                    </div>
                </div>
            `,
            buttons: [
                { text: 'Cerrar', class: 'btn-outline', action: 'close' }
            ]
        });
    },
    async runAllValidations() {
        const resultsContainer = document.getElementById('validation-results');
        if (!resultsContainer) return;
        resultsContainer.innerHTML = '<div class="loading"><i class="fas fa-spinner fa-
spin"></i> Ejecutando validaciones...</div>';
        try {
            const company = CompanyService.getCurrent();
            const results = [];
            const entries = await AccountingService.getJournalEntries();
            for (const entry of entries) {
                const lines = await AccountingService.getJournalLines(entry.id);
                const validation = await ValidationService.validateJournalEntry({
                    ...entry,
                    lines
                });
                if (!validation.isValid || validation.warnings.length > 0) {
                    results.push({
                        document: entry,
                        type: 'journalEntry',
                        validation
                    });
                }
            }
            const invoices = await DB.getByIndex('customerInvoices', 'companyId',
company.id);
            for (const invoice of invoices) {
                const flowValidation = await
ValidationService.validateBusinessFlow('customerInvoice', invoice.id);
                if (!flowValidation.allValid) {
                    results.push({
                        document: invoice,
                        type: 'customerInvoice',
                        validation: flowValidation
                    });
                }
            }
            const balanceValidation = await ValidationService.validateBalanceSheet();
            const balanceIssue = !balanceValidation.isValid ? {
                document: { id: 'balance', number: 'Balance General' },
                type: 'balanceSheet',
                validation: {
                    errors: [balanceValidation],
                    warnings: [],
                    results: [balanceValidation]
                }
            } : null;
            const allResults = balanceIssue ? [balanceIssue, ...results] : results;
            const totalIssues = allResults.length;
            resultsContainer.innerHTML = `
                <div class="validation-summary">
                    <div class="summary-icon ${totalIssues === 0 ? 'success' :
'warning'}">
                        <i class="fas fa-${totalIssues === 0 ? 'check-circle' :
'exclamation-triangle'}"></i>
                    </div>
                    <div class="summary-text">
                        ${totalIssues === 0
                    ? '<h4>¡Todas las validaciones pasaron!</h4><p>No se encontraron
errores ni advertencias.</p>'
                    : `<h4>Se encontraron ${totalIssues} observación(es)</h4><p>Revisa los
detalles a continuación.</p>`}
                    </div>
                </div>
                ${totalIssues > 0 ? `
                    <div class="validation-details">
                        ${allResults.map(r => `
                            <div class="validation-item">
                                <div class="item-header">
                                    <span class="item-
type">${TraceabilityService.documentTypes[r.type]?.name || r.type}</span>
                                    <span class="item-number">${r.document.number || '#' +
r.document.id}</span>
                                </div>
```

```
                                <div class="item-issues">
                                    ${r.validation.errors?.map(e => `
                                        <div class="issue error"><i class="fas fa-times-
circle"></i> ${e.message}</div>
                                    `).join('') || ''}
                                    ${r.validation.warnings?.map(w => `
                                        <div class="issue warning"><i class="fas fa-
exclamation-triangle"></i> ${w.message}</div>
                                    `).join('') || ''}
                                    ${r.validation.results?.filter(res => !res.isValid &&
res.severity === 'error').map(e => `
                                        <div class="issue error"><i class="fas fa-times-
circle"></i> ${e.message}</div>
                                    `).join('') || ''}
                                </div>
                            </div>
                        `).join('')}
                    </div>
                ` : ''}
            `;
        } catch (error) {
            resultsContainer.innerHTML = `
                <div class="validation-error">
                    <i class="fas fa-exclamation-triangle"></i>
                    <p>Error al ejecutar validaciones: ${error.message}</p>
                </div>
            `;
        }
    },
    async validateEntry(entryId) {
        const loading = Toast.loading('Validando asiento...');
        try {
            const entry = await DB.get('journalEntries', entryId);
            const lines = await AccountingService.getJournalLines(entryId);
            const validation = await ValidationService.validateJournalEntry({
                ...entry,
                lines
            });
            loading.close();
            await Modal.custom({
                title: `Validación: ${entry.number}`,
                size: 'lg',
                content: `
                    <div class="entry-validation-modal">
                        <div class="validation-status ${validation.isValid ? 'success' :
'error'}">
                            <i class="fas fa-${validation.isValid ? 'check-circle' :
'times-circle'}"></i>
                            <span>${validation.isValid ? 'Asiento válido' : 'Asiento con
errores'}</span>
                        </div>
                        <div class="validation-results-list">
                            ${validation.results.map(r => `
                                <div class="result-item ${r.severity}">
                                    <div class="result-header">
                                        <span class="result-code">${r.code}</span>
                                        <span class="result-severity badge
badge-${r.severity}">${r.severity}</span>
                                    </div>
                                    <div class="result-message">${r.message}</div>
                                </div>
                            `).join('')}
                        </div>
                    </div>
                `,
                buttons: [
                    { text: 'Cerrar', class: 'btn-outline', action: 'close' },
                    ...(validation.isValid ? [{
                        text: 'Contabilizar',
                        class: 'btn-primary',
                        action: async () => {
                            try {
                                await AccountingService.postJournalEntry(entryId);
                                Toast.success('Asiento contabilizado exitosamente');
                                Modal.close();
                                App.navigate('trazabilidad', 'validations');
                            } catch (e) {
                                Toast.error('Error: ' + e.message);
                            }
                        }
                    }] : [])
```

```
                ]
            });
        } catch (error) {
            loading.error('Error: ' + error.message);
        }
    },
    async exportTraceToPDF(report) {
        Toast.info('Funcionalidad de exportación PDF próximamente disponible');
    },
    async refreshTransactions() {
        App.navigate('trazabilidad', 'overview');
    },
    async showAllIssues() {
        const integrity = await TraceabilityService.verifyDataIntegrity();
        await Modal.custom({
            title: 'Problemas de Integridad de Datos',
            size: 'lg',
            content: `
                <div class="integrity-issues-modal">
                    ${integrity.issues.map(issue => `
                        <div class="issue-card ${issue.severity}">
                            <div class="issue-icon">
                                <i class="fas fa-${issue.severity === 'error' ? 'times-
circle' : 'exclamation-triangle'}"></i>
                            </div>
                            <div class="issue-content">
                                <span class="issue-type">${issue.type}</span>
                                <p>${issue.message}</p>
                            </div>
                        </div>
                    `).join('')}
                </div>
            `,
            buttons: [
                { text: 'Cerrar', class: 'btn-outline', action: 'close' }
            ]
        });
    }
};
window.TrazabilidadModule = TrazabilidadModule;
```

Archivo: js\modules\ventas.js

```
const VentasModule = {
    currentView: 'clientes',
    async render(view = 'clientes') {
        this.currentView = view;
        switch (view) {
            case 'clientes': return await this.renderCustomers();
            case 'pedidos-venta': return await this.renderSalesOrders();
            case 'facturas-cliente': return await this.renderCustomerInvoices();
            default: return '<p>Vista no encontrada</p>';
        }
    },
    async renderCustomers() {
        return `
            <div class="page-header">
                <h1 class="page-title">Clientes</h1>
                <p class="page-subtitle">Gestión de clientes y condiciones de venta</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <div class="search-box">
                        <i class="fas fa-search"></i>
                        <input type="text" class="form-control" placeholder="Buscar
cliente..." id="search-customers">
                    </div>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-customer">
                        <i class="fas fa-plus"></i> Nuevo Cliente
                    </button>
                </div>
            </div>
            <div id="customers-table"></div>
```

```
        `;
    },
    async renderSalesOrders() {
        return `
            <div class="page-header">
                <h1 class="page-title">Pedidos de Venta</h1>
                <p class="page-subtitle">Gestión de pedidos de clientes</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="filter-date-from">
                    <input type="date" class="form-control" id="filter-date-to">
                    <select class="form-control" id="filter-status">
                        <option value="">Todos</option>
                        <option value="draft">Borrador</option>
                        <option value="confirmed">Confirmado</option>
                        <option value="delivered">Entregado</option>
                        <option value="cancelled">Anulado</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-order">
                        <i class="fas fa-plus"></i> Nuevo Pedido
                    </button>
                </div>
            </div>
            <div id="orders-table"></div>
        `;
    },
    async renderCustomerInvoices() {
        return `
            <div class="page-header">
                <h1 class="page-title">Facturas de Venta</h1>
                <p class="page-subtitle">Facturación y documentos tributarios</p>
            </div>
            <div class="toolbar">
                <div class="toolbar-left">
                    <input type="date" class="form-control" id="filter-date-from">
                    <input type="date" class="form-control" id="filter-date-to">
                    <select class="form-control" id="filter-status">
                        <option value="">Todos</option>
                        <option value="pending">Pendiente</option>
                        <option value="posted">Contabilizada</option>
                        <option value="paid">Pagada</option>
                        <option value="cancelled">Anulada</option>
                    </select>
                </div>
                <div class="toolbar-right">
                    <button class="btn btn-primary" id="btn-new-invoice">
                        <i class="fas fa-plus"></i> Nueva Factura
                    </button>
                </div>
            </div>
            <div id="invoices-table"></div>
        `;
    },
    async init(view) {
        const company = CompanyService.getCurrent();
        if (!company) return;
        switch (view) {
            case 'clientes': await this.initCustomers(); break;
            case 'pedidos-venta': await this.initSalesOrders(); break;
            case 'facturas-cliente': await this.initCustomerInvoices(); break;
        }
    },
    async initCustomers() {
        const company = CompanyService.getCurrent();
        const customers = await DB.getByIndex('customers', 'companyId', company.id);
        DataTable.create('customers-table', {
            columns: [
                { key: 'rut', label: 'RUT' },
                { key: 'name', label: 'Nombre' },
                { key: 'contact', label: 'Contacto' },
                { key: 'email', label: 'Email' },
                { key: 'phone', label: 'Teléfono' },
                { key: 'creditLimit', label: 'Límite Crédito', format: 'currency', class:
'text-right' },
                { key: 'balance', label: 'Saldo', format: 'currency', class: 'text-right'
}
            ],
            data: customers,
```

```
            actions: [
                { name: 'edit', label: 'Editar', icon: 'fas fa-edit', handler: (row) =>
this.showCustomerModal(row.id) },
                { name: 'delete', label: 'Eliminar', icon: 'fas fa-trash', handler: (row)
=> this.deleteCustomer(row.id) }
            ],
            emptyMessage: 'No hay clientes registrados'
        });
        document.getElementById('btn-new-customer')?.addEventListener('click', () =>
this.showCustomerModal());
    },
    async showCustomerModal(customerId = null) {
        const isEdit = !!customerId;
        let data = {};
        if (isEdit) {
            data = await DB.get('customers', customerId);
        }
        await Modal.form({
            title: isEdit ? 'Editar Cliente' : 'Nuevo Cliente',
            fields: [
                { name: 'rut', label: 'RUT', required: true, placeholder: '76.123.456-7',
default: data.rut },
                { name: 'name', label: 'Razón Social', required: true, placeholder:
'Nombre empresa', default: data.name },
                { name: 'contact', label: 'Contacto', placeholder: 'Nombre contacto',
default: data.contact },
                { name: 'email', label: 'Email', type: 'email', placeholder:
'email@empresa.cl', default: data.email },
                { name: 'phone', label: 'Teléfono', placeholder: '+56 9 1234 5678',
default: data.phone },
                { name: 'address', label: 'Dirección', placeholder: 'Dirección comercial',
default: data.address },
                { name: 'creditLimit', label: 'Límite de Crédito', type: 'number',
default: data.creditLimit || 0 }
            ],
            validate: (formData) => {
                if (!formData.name) return 'El nombre es requerido';
                if (!Helpers.validateRUT(formData.rut)) return 'RUT inválido';
                return true;
            },
            onSubmit: async (formData) => {
                const company = CompanyService.getCurrent();
                formData.rut = Helpers.formatRUT(formData.rut);
                if (isEdit) {
                    await DB.update('customers', { ...data, ...formData });
                    Toast.success('Cliente actualizado');
                } else {
                    await DB.add('customers', { companyId: company.id, ...formData,
status: 'active', balance: 0 });
                    Toast.success('Cliente creado');
                }
                App.navigate('ventas', 'clientes');
            }
        });
    },
    async deleteCustomer(id) {
        const confirm = await Modal.confirm({
            title: 'Eliminar Cliente',
            message: '¿Está seguro de eliminar este cliente?',
            confirmText: 'Eliminar',
            confirmClass: 'btn-danger'
        });
        if (confirm) {
            await DB.delete('customers', id);
            Toast.success('Cliente eliminado');
            App.navigate('ventas', 'clientes');
        }
    },
    async initSalesOrders() {
        const company = CompanyService.getCurrent();
        const allOrders = await DB.getByIndex('salesOrders', 'companyId', company.id);
        for (let order of allOrders) {
            const customer = await DB.get('customers', order.customerId);
            order.customerName = customer?.name || 'Cliente desconocido';
        }
        const applyFilters = () => {
            const dateFrom = document.getElementById('filter-date-from')?.value;
            const dateTo = document.getElementById('filter-date-to')?.value;
            const status = document.getElementById('filter-status')?.value;
            let filteredOrders = allOrders;
            if (dateFrom) {
```

```
                filteredOrders = filteredOrders.filter(o => o.date >= dateFrom);
            }
            if (dateTo) {
                filteredOrders = filteredOrders.filter(o => o.date <= dateTo);
            }
            if (status) {
                filteredOrders = filteredOrders.filter(o => o.status === status);
            }
            DataTable.create('orders-table', {
                columns: [
                    { key: 'number', label: 'Número' },
                    { key: 'date', label: 'Fecha', format: 'date' },
                    { key: 'customerName', label: 'Cliente' },
                    { key: 'total', label: 'Total', format: 'currency', class: 'text-
right' },
                    { key: 'status', label: 'Estado', format: 'status' }
                ],
                data: filteredOrders,
                actions: [
                    { name: 'view', label: 'Ver', icon: 'fas fa-eye', handler: (row) =>
this.showOrderDetail(row) },
                    { name: 'edit', label: 'Editar', icon: 'fas fa-edit', handler: (row)
=> this.editSalesOrder(row) },
                    { name: 'confirm', label: 'Confirmar', icon: 'fas fa-check', handler:
(row) => this.confirmOrder(row) },
                    { name: 'deliver', label: 'Entregar', icon: 'fas fa-truck', handler:
(row) => this.deliverOrder(row) },
                    { name: 'invoice', label: 'Facturar', icon: 'fas fa-file-invoice',
handler: (row) => this.invoiceOrder(row) },
                    { name: 'cancel', label: 'Anular', icon: 'fas fa-ban', handler: (row)
=> this.cancelSalesOrder(row) }
                ],
                emptyMessage: 'No hay pedidos de venta'
            });
        };
        applyFilters();
        document.getElementById('filter-date-from')?.addEventListener('change',
applyFilters);
        document.getElementById('filter-date-to')?.addEventListener('change',
applyFilters);
        document.getElementById('filter-status')?.addEventListener('change',
applyFilters);
        document.getElementById('btn-new-order')?.addEventListener('click', () => {
            this.showSalesOrderModal();
        });
    },
    async showSalesOrderModal() {
        const company = CompanyService.getCurrent();
        const customers = await DB.getByIndex('customers', 'companyId', company.id);
        const products = await DB.getByIndex('products', 'companyId', company.id);
        if (customers.length === 0) {
            Toast.warning('Primero debe crear al menos un cliente');
            return;
        }
        Modal.open({
            title: 'Nuevo Pedido de Venta',
            size: 'large',
            content: `
                <form id="sales-order-form">
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Cliente <span class="text-danger">*
</span></label>
                            <select class="form-control" name="customerId" required>
                                <option value="">Seleccione cliente...</option>
                                ${customers.map(c => `<option value="${c.id}">${c.rut} -
${c.name}</option>`).join('')}
                            </select>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Fecha</label>
                            <input type="date" class="form-control" name="date"
value="${Helpers.getCurrentDate()}" required>
                        </div>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Observaciones</label>
                        <textarea class="form-control" name="notes" rows="2"
placeholder="Observaciones opcionales"></textarea>
                    </div>
                    <h4 style="margin: var(--space-4) 0 var(--space-2);">Productos</h4>
```

```
                    <div class="table-container">
                        <table class="data-table" id="order-lines-table">
                            <thead>
                                <tr>
                                    <th style="width: 35%;">Producto</th>
                                    <th style="width: 10%;">Stock</th>
                                    <th style="width: 12%;">Cantidad</th>
                                    <th style="width: 15%;">Precio</th>
                                    <th style="width: 18%;">Subtotal</th>
                                    <th style="width: 10%;"></th>
                                </tr>
                            </thead>
                            <tbody id="order-lines">
                                <tr class="order-line" data-idx="0">
                                    <td>
                                        <select class="form-control product-select"
name="productId" required>
                                            <option value="">Seleccione...</option>
                                            ${products.map(p => `<option value="${p.id}"
data-price="${p.price}" data-stock="${p.stock || 0}">${p.code} - ${p.name}
</option>`).join('')}
                                        </select>
                                    </td>
                                    <td class="text-right stock-display">-</td>
                                    <td><input type="number" class="form-control quantity-
input" name="quantity" value="1" min="1" required></td>
                                    <td><input type="number" class="form-control price-
input" name="price" value="0" min="0" required></td>
                                    <td class="text-right subtotal">$0</td>
                                    <td><button type="button" class="btn btn-icon btn-
ghost remove-line"><i class="fas fa-times"></i></button></td>
                                </tr>
                            </tbody>
                            <tfoot>
                                <tr>
                                    <td colspan="4" class="text-right">Subtotal Neto:</td>
                                    <td class="text-right" id="order-subtotal">$0</td>
                                    <td></td>
                                </tr>
                                <tr>
                                    <td colspan="4" class="text-right">IVA (19%):</td>
                                    <td class="text-right" id="order-iva">$0</td>
                                    <td></td>
                                </tr>
                                <tr style="font-weight: 600;">
                                    <td colspan="4" class="text-right">TOTAL:</td>
                                    <td class="text-right" id="order-total">$0</td>
                                    <td></td>
                                </tr>
                            </tfoot>
                        </table>
                    </div>
                    <button type="button" class="btn btn-outline" id="btn-add-line"
style="margin-top: var(--space-2);">
                        <i class="fas fa-plus"></i> Agregar Producto
                    </button>
                </form>
            `,
            footer: `
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-primary" id="btn-save-order"><i class="fas fa-
save"></i> Guardar Pedido</button>
            `
        });
        this.initSalesOrderModalEvents(products);
    },
    initSalesOrderModalEvents(products) {
        let lineIdx = 1;
        const IVA_RATE = 0.19;
        document.getElementById('btn-add-line')?.addEventListener('click', () => {
            const tbody = document.getElementById('order-lines');
            const newRow = document.createElement('tr');
            newRow.className = 'order-line';
            newRow.dataset.idx = lineIdx++;
            newRow.innerHTML = `
                <td>
                    <select class="form-control product-select" name="productId" required>
                        <option value="">Seleccione...</option>
                        ${products.map(p => `<option value="${p.id}" data-
price="${p.price}" data-stock="${p.stock || 0}">${p.code} - ${p.name}</option>`).join('')}
```

```
                    </select>
                </td>
                <td class="text-right stock-display">-</td>
                <td><input type="number" class="form-control quantity-input"
name="quantity" value="1" min="1" required></td>
                <td><input type="number" class="form-control price-input" name="price"
value="0" min="0" required></td>
                <td class="text-right subtotal">$0</td>
                <td><button type="button" class="btn btn-icon btn-ghost remove-line"><i
class="fas fa-times"></i></button></td>
            `;
            tbody.appendChild(newRow);
            this.bindSalesOrderLineEvents(IVA_RATE);
        });
        this.bindSalesOrderLineEvents(IVA_RATE);
        document.getElementById('btn-save-order')?.addEventListener('click', async () => {
            await this.saveSalesOrder();
        });
    },
    bindSalesOrderLineEvents(IVA_RATE) {
        const self = this;
        document.querySelectorAll('#order-lines .product-select').forEach(select => {
            select.removeEventListener('change', select._changeHandler);
            select._changeHandler = function () {
                const option = select.selectedOptions[0];
                const price = parseFloat(option?.dataset.price || 0);
                const stock = parseInt(option?.dataset.stock || 0);
                const row = select.closest('tr');
                row.querySelector('.price-input').value = price;
                row.querySelector('.stock-display').textContent = stock;
                self.updateSalesOrderTotals(IVA_RATE);
                self.validateStockInRow(row, stock);
            };
            select.addEventListener('change', select._changeHandler);
        });
        document.querySelectorAll('#order-lines .quantity-input, #order-lines .price-
input').forEach(input => {
            input.removeEventListener('input', input._inputHandler);
            input._inputHandler = function () {
                self.updateSalesOrderTotals(IVA_RATE);
                if (input.classList.contains('quantity-input')) {
                    const row = input.closest('tr');
                    const stockText = row.querySelector('.stock-display')?.textContent;
                    const stock = parseInt(stockText) || 0;
                    self.validateStockInRow(row, stock);
                }
            };
            input.addEventListener('input', input._inputHandler);
        });
        document.querySelectorAll('#order-lines .remove-line').forEach(btn => {
            btn.removeEventListener('click', btn._clickHandler);
            btn._clickHandler = function () {
                const lines = document.querySelectorAll('.order-line');
                if (lines.length > 1) {
                    btn.closest('tr').remove();
                    self.updateSalesOrderTotals(IVA_RATE);
                } else {
                    Toast.warning('Debe haber al menos una línea');
                }
            };
            btn.addEventListener('click', btn._clickHandler);
        });
        this.updateSalesOrderTotals(IVA_RATE);
    },
    updateSalesOrderTotals(IVA_RATE) {
        let subtotal = 0;
        document.querySelectorAll('.order-line').forEach(row => {
            const qty = parseFloat(row.querySelector('.quantity-input')?.value) || 0;
            const price = parseFloat(row.querySelector('.price-input')?.value) || 0;
            const lineSubtotal = qty * price;
            const subtotalEl = row.querySelector('.subtotal');
            if (subtotalEl) subtotalEl.textContent = Formatters.currency(lineSubtotal);
            subtotal += lineSubtotal;
        });
        const iva = subtotal * IVA_RATE;
        const subtotalEl = document.getElementById('order-subtotal');
        const ivaEl = document.getElementById('order-iva');
        const totalEl = document.getElementById('order-total');
        if (subtotalEl) subtotalEl.textContent = Formatters.currency(subtotal);
        if (ivaEl) ivaEl.textContent = Formatters.currency(iva);
        if (totalEl) totalEl.textContent = Formatters.currency(subtotal + iva);
```

```
    },
    validateStockInRow(row, stock) {
        const quantityInput = row.querySelector('.quantity-input');
        const stockDisplay = row.querySelector('.stock-display');
        const quantity = parseFloat(quantityInput?.value) || 0;
        if (quantity > stock && stock >= 0) {
            quantityInput?.classList.add('error');
            stockDisplay?.classList.add('text-danger');
            stockDisplay.style.fontWeight = 'bold';
        } else {
            quantityInput?.classList.remove('error');
            stockDisplay?.classList.remove('text-danger');
            stockDisplay.style.fontWeight = '';
        }
    },
    async saveSalesOrder() {
        const form = document.getElementById('sales-order-form');
        const formData = new FormData(form);
        const customerId = formData.get('customerId');
        if (!customerId) {
            Toast.error('Seleccione un cliente');
            return;
        }
        const IVA_RATE = 0.19;
        const lines = [];
        let subtotal = 0;
        document.querySelectorAll('.order-line').forEach(row => {
            const productId = row.querySelector('.product-select').value;
            const quantity = parseFloat(row.querySelector('.quantity-input').value) || 0;
            const price = parseFloat(row.querySelector('.price-input').value) || 0;
            if (productId && quantity > 0) {
                const lineSubtotal = quantity * price;
                lines.push({ productId, quantity, price, subtotal: lineSubtotal });
                subtotal += lineSubtotal;
            }
        });
        if (lines.length === 0) {
            Toast.error('Agregue al menos un producto');
            return;
        }
        const stockErrors = [];
        for (const line of lines) {
            const product = await DB.get('products', line.productId);
            if (product && !product.isService) {
                const availableStock = product.stock || 0;
                if (line.quantity > availableStock) {
                    stockErrors.push({
                        name: product.name || product.code,
                        requested: line.quantity,
                        available: availableStock
                    });
                }
            }
        }
        if (stockErrors.length > 0) {
            const errorMessages = stockErrors.map(e =>
                `• ${e.name}: solicita ${e.requested} pero solo hay ${e.available}
disponibles`
            ).join('\n');
            Toast.error(`Stock insuficiente:\n${errorMessages}`);
            return;
        }
        const company = CompanyService.getCurrent();
        const iva = subtotal * IVA_RATE;
        const total = subtotal + iva;
        const orderNumber = await this.getNextSalesOrderNumber();
        const order = {
            id: Helpers.generateId(),
            companyId: company.id,
            number: orderNumber,
            customerId,
            date: formData.get('date'),
            notes: formData.get('notes'),
            status: 'draft',
            subtotal,
            iva,
            total,
            createdAt: new Date().toISOString()
        };
        try {
            await DB.add('salesOrders', order);
```

```
            for (const line of lines) {
                await DB.add('salesOrderLines', {
                    id: Helpers.generateId(),
                    orderId: order.id,
                    ...line
                });
            }
            Toast.success('Pedido de venta creado');
            Modal.close();
            App.navigate('ventas', 'pedidos-venta');
        } catch (err) {
            Toast.error('Error: ' + err.message);
        }
    },
    async getNextSalesOrderNumber() {
        const company = CompanyService.getCurrent();
        const orders = await DB.getByIndex('salesOrders', 'companyId', company.id);
        const year = new Date().getFullYear();
        const prefix = `PV-${year}-`;
        const existingNumbers = orders
            .filter(o => {
                const numStr = String(o.number || '');
                return numStr.startsWith(prefix);
            })
            .map(o => parseInt(String(o.number || '').replace(prefix, '')) || 0);
        const nextNum = Math.max(0, ...existingNumbers) + 1;
        return `${prefix}${String(nextNum).padStart(4, '0')}`;
    },
    async showOrderDetail(order) {
        const customer = await DB.get('customers', order.customerId);
        const lines = await DB.getByIndex('salesOrderLines', 'orderId', order.id);
        for (let line of lines) {
            const product = await DB.get('products', line.productId);
            line.productName = product?.name || 'Producto';
            line.productCode = product?.code || '';
        }
        const statusLabels = {
            draft: '<span class="badge badge-warning">Borrador</span>',
            confirmed: '<span class="badge badge-info">Confirmado</span>',
            delivered: '<span class="badge badge-success">Entregado</span>',
            cancelled: '<span class="badge badge-error">Cancelado</span>'
        };
        Modal.open({
            title: `Pedido ${order.number}`,
            size: 'large',
            content: `
                <div style="margin-bottom: var(--space-4);">
                    <div style="display: grid; grid-template-columns: repeat(3, 1fr); gap:
var(--space-4);">
                        <div>
                            <label style="font-size: var(--font-size-xs); color: var(--
text-tertiary);">Cliente</label>
                            <strong>${customer?.name || 'N/A'}</strong>
                        </div>
                        <div>
                            <label style="font-size: var(--font-size-xs); color: var(--
text-tertiary);">Fecha</label>
                            <strong>${Formatters.date(order.date)}</strong>
                        </div>
                        <div>
                            <label style="font-size: var(--font-size-xs); color: var(--
text-tertiary);">Estado</label>
                            ${statusLabels[order.status] || order.status}
                        </div>
                    </div>
                </div>
                <div class="table-container">
                    <table class="data-table">
                        <thead>
                            <tr>
                                <th>Código</th>
                                <th>Producto</th>
                                <th class="text-right">Cantidad</th>
                                <th class="text-right">Precio</th>
                                <th class="text-right">Subtotal</th>
                            </tr>
                        </thead>
                        <tbody>
                            ${lines.map(l => `
                                <tr>
                                    <td><code>${l.productCode}</code></td>
```

```
                                    <td>${l.productName}</td>
                                    <td class="text-right">${l.quantity}</td>
                                    <td class="text-right">${Formatters.currency(l.price)}
</td>
                                    <td class="text-
right">${Formatters.currency(l.subtotal)}</td>
                                </tr>
                            `).join('')}
                        </tbody>
                        <tfoot>
                            <tr>
                                <td colspan="4" class="text-right">Subtotal:</td>
                                <td class="text-
right">${Formatters.currency(order.subtotal || 0)}</td>
                            </tr>
                            <tr>
                                <td colspan="4" class="text-right">IVA (19%):</td>
                                <td class="text-right">${Formatters.currency(order.iva ||
0)}</td>
                            </tr>
                            <tr style="font-weight: 600;">
                                <td colspan="4" class="text-right">TOTAL:</td>
                                <td class="text-right">${Formatters.currency(order.total)}
</td>
                            </tr>
                        </tfoot>
                    </table>
                </div>
            `,
            footer: `<button class="btn btn-secondary"
onclick="Modal.close()">Cerrar</button>`
        });
    },
    async editSalesOrder(order) {
        if (order.status !== 'draft') {
            const statusName = {
                confirmed: 'confirmado',
                delivered: 'entregado',
                cancelled: 'anulado'
            }[order.status] || order.status;
            Toast.warning(`No se puede editar un pedido ${statusName}. Solo los pedidos en
borrador son editables.`);
            return;
        }
        const company = CompanyService.getCurrent();
        const customers = await DB.getByIndex('customers', 'companyId', company.id);
        const products = await DB.getByIndex('products', 'companyId', company.id);
        const existingLines = await DB.getByIndex('salesOrderLines', 'orderId', order.id);
        let linesHtml = '';
        for (let i = 0; i < existingLines.length; i++) {
            const line = existingLines[i];
            const product = products.find(p => p.id === line.productId);
            linesHtml += `
                <tr class="order-line" data-idx="${i}">
                    <td>
                        <select class="form-control product-select" name="productId"
required>
                            <option value="">Seleccione...</option>
                            ${products.map(p => `<option value="${p.id}" data-
price="${p.price}" data-stock="${p.stock || 0}" ${p.id === line.productId ? 'selected' :
''}>${p.code} - ${p.name}</option>`).join('')}
                        </select>
                    </td>
                    <td class="text-right stock-display">${product?.stock || 0}</td>
                    <td><input type="number" class="form-control quantity-input"
name="quantity" value="${line.quantity}" min="1" required></td>
                    <td><input type="number" class="form-control price-input" name="price"
value="${line.price}" min="0" required></td>
                    <td class="text-right subtotal">${Formatters.currency(line.subtotal)}
</td>
                    <td><button type="button" class="btn btn-icon btn-ghost remove-line">
<i class="fas fa-times"></i></button></td>
                </tr>
            `;
        }
        Modal.open({
            title: `Editar Pedido ${order.number}`,
            size: 'large',
            content: `
                <form id="edit-order-form">
                    <div class="form-row">
```

```
                        <div class="form-group">
                            <label class="form-label">Cliente <span class="text-error">*
</span></label>
                            <select class="form-control" name="customerId" required>
                                <option value="">Seleccione...</option>
                                ${customers.map(c => `<option value="${c.id}" ${c.id ===
order.customerId ? 'selected' : ''}>${c.rut} - ${c.name}</option>`).join('')}
                            </select>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Fecha</label>
                            <input type="date" class="form-control" name="date"
value="${order.date}" required>
                        </div>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Observaciones</label>
                        <textarea class="form-control" name="notes" rows="2">${order.notes
|| ''}</textarea>
                    </div>
                    <h4 style="margin: var(--space-4) 0 var(--space-2);">Productos</h4>
                    <div class="table-container">
                        <table class="data-table" id="order-lines-table">
                            <thead>
                                <tr>
                                    <th style="width: 35%;">Producto</th>
                                    <th style="width: 10%;">Stock</th>
                                    <th style="width: 12%;">Cantidad</th>
                                    <th style="width: 15%;">Precio</th>
                                    <th style="width: 18%;">Subtotal</th>
                                    <th style="width: 10%;"></th>
                                </tr>
                            </thead>
                            <tbody id="order-lines">
                                ${linesHtml}
                            </tbody>
                        </table>
                    </div>
                    <button type="button" class="btn btn-outline btn-sm" id="btn-add-line"
style="margin-top: var(--space-2);">
                        <i class="fas fa-plus"></i> Agregar línea
                    </button>
                    <div class="order-totals" style="margin-top: var(--space-4); text-
align: right;">
                        <div>Subtotal: <strong id="order-subtotal">$0</strong></div>
                        <div>IVA (19%): <strong id="order-iva">$0</strong></div>
                        <div style="font-size: 1.2em;">Total: <strong id="order-
total">$0</strong></div>
                    </div>
                </form>
            `,
            footer: `
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-primary" id="btn-save-edit-order">Guardar
Cambios</button>
            `
        });
        const IVA_RATE = 0.19;
        this.bindSalesOrderLineEvents(IVA_RATE);
        this.updateSalesOrderTotals(IVA_RATE);
        document.getElementById('btn-add-line')?.addEventListener('click', () => {
            const tbody = document.getElementById('order-lines');
            const idx = tbody.querySelectorAll('tr').length;
            const newRow = document.createElement('tr');
            newRow.className = 'order-line';
            newRow.dataset.idx = idx;
            newRow.innerHTML = `
                <td>
                    <select class="form-control product-select" name="productId" required>
                        <option value="">Seleccione...</option>
                        ${products.map(p => `<option value="${p.id}" data-
price="${p.price}" data-stock="${p.stock || 0}">${p.code} - ${p.name}</option>`).join('')}
                    </select>
                </td>
                <td class="text-right stock-display">-</td>
                <td><input type="number" class="form-control quantity-input"
name="quantity" value="1" min="1" required></td>
                <td><input type="number" class="form-control price-input" name="price"
value="0" min="0" required></td>
                <td class="text-right subtotal">$0</td>
```

```
                <td><button type="button" class="btn btn-icon btn-ghost remove-line"><i
class="fas fa-times"></i></button></td>
            `;
            tbody.appendChild(newRow);
            this.bindSalesOrderLineEvents(IVA_RATE);
        });
        document.getElementById('btn-save-edit-order')?.addEventListener('click', async ()
=> {
            await this.saveEditedOrder(order);
        });
    },
    async saveEditedOrder(originalOrder) {
        const form = document.getElementById('edit-order-form');
        const formData = new FormData(form);
        const customerId = formData.get('customerId');
        const date = formData.get('date');
        const notes = formData.get('notes');
        if (!customerId) {
            Toast.error('Debe seleccionar un cliente');
            return;
        }
        const lines = [];
        document.querySelectorAll('#order-lines .order-line').forEach(row => {
            const productId = row.querySelector('.product-select')?.value;
            const quantity = parseFloat(row.querySelector('.quantity-input')?.value) || 0;
            const price = parseFloat(row.querySelector('.price-input')?.value) || 0;
            if (productId && quantity > 0) {
                lines.push({
                    productId,
                    quantity,
                    price,
                    subtotal: quantity * price
                });
            }
        });
        if (lines.length === 0) {
            Toast.error('Debe agregar al menos un producto');
            return;
        }
        const stockErrors = [];
        for (const line of lines) {
            const product = await DB.get('products', line.productId);
            if (product && !product.isService) {
                const availableStock = product.stock || 0;
                if (line.quantity > availableStock) {
                    stockErrors.push({
                        name: product.name || product.code,
                        requested: line.quantity,
                        available: availableStock
                    });
                }
            }
        }
        if (stockErrors.length > 0) {
            const errorMessages = stockErrors.map(e =>
                `• ${e.name}: solicita ${e.requested} pero solo hay ${e.available}
disponibles`
            ).join('\n');
            Toast.error(`Stock insuficiente:\n${errorMessages}`);
            return;
        }
        try {
            const company = CompanyService.getCurrent();
            const subtotal = lines.reduce((sum, l) => sum + l.subtotal, 0);
            const iva = Math.round(subtotal * 0.19);
            const total = subtotal + iva;
            await DB.update('salesOrders', {
                ...originalOrder,
                customerId,
                date,
                notes,
                subtotal,
                iva,
                total,
                updatedAt: new Date().toISOString()
            });
            const oldLines = await DB.getByIndex('salesOrderLines', 'orderId',
originalOrder.id);
            for (const oldLine of oldLines) {
                await DB.delete('salesOrderLines', oldLine.id);
            }
```

```
            for (const line of lines) {
                await DB.add('salesOrderLines', {
                    id: Helpers.generateId(),
                    orderId: originalOrder.id,
                    companyId: company.id,
                    ...line
                });
            }
            Toast.success('Pedido actualizado');
            Modal.close();
            App.navigate('ventas', 'pedidos-venta');
        } catch (err) {
            Toast.error('Error: ' + err.message);
        }
    },
    async confirmOrder(order) {
        if (order.status !== 'draft') {
            Toast.warning('Solo se pueden confirmar pedidos en borrador');
            return;
        }
        const confirm = await Modal.confirm({
            title: 'Confirmar Pedido',
            message: `¿Confirmar el pedido ${order.number}?`,
            confirmText: 'Confirmar'
        });
        if (confirm) {
            await DB.update('salesOrders', { ...order, status: 'confirmed' });
            Toast.success('Pedido confirmado');
            App.navigate('ventas', 'pedidos-venta');
        }
    },
    async deliverOrder(order) {
        if (order.status !== 'confirmed' && order.status !== 'partial') {
            Toast.warning('Solo se pueden entregar pedidos confirmados o entregados
parcialmente');
            return;
        }
        const company = CompanyService.getCurrent();
        const customer = await DB.get('customers', order.customerId);
        const lines = await DB.getByIndex('salesOrderLines', 'orderId', order.id);
        let pendingLinesCount = 0;
        let htmlLines = '';
        for (const line of lines) {
            const pending = line.quantity - (line.delivered || 0);
            if (pending <= 0) continue;
            pendingLinesCount++;
            const product = await DB.get('products', line.productId);
            const prodName = product ? `${product.code} - ${product.name}` : `Producto
${line.productId}`;
            htmlLines += `
                <div class="line-item" data-line-id="${line.id}" data-product-
id="${line.productId}" data-pending="${pending}" data-price="${line.price}" data-
cost="${product?.cost || 0}" style="padding: 1rem; border: 1px solid var(--border-color);
border-radius: var(--radius-md); margin-bottom: 1rem;">
                    <div style="font-weight: 500; margin-bottom: 0.5rem;">${prodName}
<span class="badge badge-neutral" style="float: right;">Pendiente: ${pending}</span></div>
                    <div class="form-row">
                        <div class="form-group" style="flex: 1;">
                            <label class="form-label">Entregar</label>
                            <input type="number" class="form-control to-deliver"
value="${pending}" min="0" step="any">
                        </div>
                    </div>
                </div>
            `;
        }
        if (pendingLinesCount === 0) {
            Toast.info('Este pedido ya fue entregado en su totalidad');
            return;
        }
        Modal.open({
            title: `Generar Guía de Despacho (Pedido ${order.number})`,
            size: 'large',
            content: `
                <div class="alert alert-info" style="margin-bottom: 1rem;">
                    Indique la cantidad a entregar. Si necesita entregar más de lo
solicitado, debe generar una nueva orden de venta por el excedente.
                </div>
                <div id="delivery-lines-container">
                    ${htmlLines}
                </div>
```

```
                <div class="modal-footer" style="margin-top: 1.5rem; display: flex;
justify-content: flex-end; gap: 0.5rem;">
                    <button class="btn btn-outline"
onclick="Modal.close()">Cancelar</button>
                    <button class="btn btn-primary" id="btn-confirm-delivery"><i
class="fas fa-truck"></i> Confirmar Entrega</button>
                </div>
            `
        });
        document.getElementById('btn-confirm-delivery').addEventListener('click', async ()
=> {
            const lineItems = Array.from(document.querySelectorAll('.line-item'));
            const deliveryData = [];
            let hasErrors = false;
            let totalCost = 0;
            let totalRevenueNet = 0;
            for (const item of lineItems) {
                const pending = parseFloat(item.dataset.pending);
                const toDeliver = parseFloat(item.querySelector('.to-deliver').value) ||
0;
                const price = parseFloat(item.dataset.price) || 0;
                const cost = parseFloat(item.dataset.cost) || 0;
                if (toDeliver < 0) {
                    Toast.error('La cantidad a entregar no puede ser negativa');
                    hasErrors = true;
                    break;
                }
                if (toDeliver > pending) {
                    Toast.warning(`No puede entregar más de lo solicitado. Para excedentes
(${toDeliver - pending} adicionales), debe generar una nueva Orden de Venta.`);
                    hasErrors = true;
                    break;
                }
                if (toDeliver > 0) {
                    deliveryData.push({
                        lineId: item.dataset.lineId,
                        productId: item.dataset.productId,
                        pending,
                        toDeliver,
                        price,
                        cost
                    });
                    totalCost += toDeliver * cost;
                    totalRevenueNet += toDeliver * price;
                }
            }
            if (hasErrors) return;
            if (deliveryData.length === 0) {
                Toast.error('Debe entregar al menos una unidad para confirmar');
                return;
            }
            const loading = Toast.loading('Procesando entrega y generando guía...');
            try {
                for (const data of deliveryData) {
                    const product = await DB.get('products', data.productId);
                    if (product && !product.isService) {
                        const newStock = Math.max(0, (product.stock || 0) -
data.toDeliver);
                        await DB.update('products', { ...product, stock: newStock });
                        await DB.add('stockMovements', {
                            id: Helpers.generateId(),
                            companyId: company.id,
                            productId: data.productId,
                            type: 'out',
                            quantity: data.toDeliver,
                            unitCost: data.cost,
                            date: Helpers.getCurrentDate(),
                            reference: order.number,
                            description: `Entrega ${data.toDeliver < data.pending ?
'Parcial ' : ''}Pedido ${order.number}`,
                            createdAt: new Date().toISOString()
                        });
                    }
                    const line = lines.find(l => l.id === data.lineId);
                    if (line) {
                        await DB.update('salesOrderLines', {
                            ...line,
                            delivered: (line.delivered || 0) + data.toDeliver
                        });
                    }
                }
```

```
                const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
                let inventoryAccount = accounts.find(a =>
                    a.type === 'asset' && (a.name?.toLowerCase().includes('mercader') ||
a.name?.toLowerCase().includes('inventario'))
                ) || accounts.find(a => a.type === 'asset' && a.code === '1.1.07');
                let bridgeAccount = accounts.find(a =>
                    a.type === 'asset' && a.name?.toLowerCase().includes('facturas por
emitir')
                ) || accounts.find(a => a.code === '1.1.06');
                let costOfSalesAccount = accounts.find(a =>
                    a.type === 'expense' && (a.code?.includes('5.1.02') ||
a.name?.toLowerCase().includes('costo de venta'))
                );
                let salesAccount = accounts.find(a =>
                    a.type === 'revenue' && a.name?.toLowerCase().includes('venta')
                ) || accounts.find(a => a.code?.startsWith('4.1'));
                if (!inventoryAccount) {
                    inventoryAccount = { id: Helpers.generateId(), companyId: company.id,
code: '1.1.07', name: 'Mercaderías', type: 'asset', nature: 'debit', level: 2, isActive:
true };
                    await DB.add('accounts', inventoryAccount);
                }
                if (!bridgeAccount) {
                    bridgeAccount = { id: Helpers.generateId(), companyId: company.id,
code: '1.1.06', name: 'Clientes - Facturas por Emitir', type: 'asset', nature: 'debit',
level: 2, isActive: true };
                    await DB.add('accounts', bridgeAccount);
                }
                if (!costOfSalesAccount) {
                    costOfSalesAccount = { id: Helpers.generateId(), companyId:
company.id, code: '5.1.02', name: 'Costo de Ventas', type: 'expense', nature: 'debit',
level: 2, isActive: true };
                    await DB.add('accounts', costOfSalesAccount);
                }
                if (!salesAccount) {
                    salesAccount = { id: Helpers.generateId(), companyId: company.id,
code: '4.1.01', name: 'Ventas', type: 'revenue', nature: 'credit', level: 2, isActive:
true };
                    await DB.add('accounts', salesAccount);
                }
                const transactionData = {
                    date: Helpers.getCurrentDate(),
                    description: `Guía de Despacho s/Pedido ${order.number} -
${customer?.name || 'Cliente'}`,
                    reference: `GD-${order.number}`,
                    sourceDocument: 'salesOrder',
                    sourceDocumentId: order.id,
                    autoPost: true,
                    lines: []
                };
                if (totalRevenueNet > 0) {
                    transactionData.lines.push({ accountId: bridgeAccount.id, description:
`Provisión Facturas por Emitir s/Pedido ${order.number}`, debit: totalRevenueNet, credit:
0 });
                    transactionData.lines.push({ accountId: salesAccount.id, description:
`Reconocimiento Venta (Neto) s/Pedido ${order.number}`, debit: 0, credit: totalRevenueNet
});
                }
                if (totalCost > 0) {
                    transactionData.lines.push({ accountId: costOfSalesAccount.id,
description: `Costo mercadería vendida s/Pedido ${order.number}`, debit: totalCost,
credit: 0 });
                    transactionData.lines.push({ accountId: inventoryAccount.id,
description: `Salida mercadería s/Pedido ${order.number}`, debit: 0, credit: totalCost });
                }
                if (transactionData.lines.length > 0) {
                    await AccountingService.registerTransaction('sales', transactionData);
                }
                const updatedLines = await DB.getByIndex('salesOrderLines', 'orderId',
order.id);
                const allDelivered = updatedLines.every(l => (l.delivered || 0) >=
l.quantity);
                await DB.update('salesOrders', {
                    ...order,
                    status: allDelivered ? 'delivered' : 'partial',
                    deliveredAt: new Date().toISOString()
                });
                loading.success('Guía de Despacho generada exitosamente. Recuerde emitir
la factura.');
                Modal.close();
                App.navigate('ventas', 'pedidos-venta');
```

```
            } catch (err) {
                loading.error('Error: ' + err.message);
                console.error(err);
            }
        });
    },
    async invoiceOrder(order) {
        if (order.status !== 'delivered' && order.status !== 'partial') {
            Toast.warning('Solo se pueden facturar pedidos con entregas registradas');
            return;
        }
        const lines = await DB.getByIndex('salesOrderLines', 'orderId', order.id);
        const invoiceableLines = lines.filter(l => (l.delivered || 0) > (l.invoiced ||
0));
        if (invoiceableLines.length === 0) {
            Toast.info('Todas las entregas de este pedido ya han sido facturadas');
            return;
        }
        const confirm = await Modal.confirm({
            title: 'Emitir Factura',
            message: `¿Generar factura por las cantidades entregadas y no facturadas del
pedido ${order.number}?`,
            confirmText: 'Generar Factura',
            confirmClass: 'btn-primary'
        });
        if (!confirm) return;
        const loading = Toast.loading('Generando factura...');
        try {
            const company = CompanyService.getCurrent();
            const customer = await DB.get('customers', order.customerId);
            const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
            const invoiceNumber = await this.getNextCustomerInvoiceNumber();
            let subtotal = 0;
            const invoice = {
                id: Helpers.generateId(),
                companyId: company.id,
                customerId: order.customerId,
                salesOrderId: order.id,
                number: invoiceNumber,
                date: Helpers.getCurrentDate(),
                dueDate: Helpers.getDatePlusDays(30),
                status: 'posted',
                createdAt: new Date().toISOString()
            };
            await DB.add('customerInvoices', invoice);
            for (const line of invoiceableLines) {
                const toInvoice = (line.delivered || 0) - (line.invoiced || 0);
                const lineNeto = toInvoice * line.price;
                subtotal += lineNeto;
                await DB.add('customerInvoiceLines', {
                    id: Helpers.generateId(),
                    invoiceId: invoice.id,
                    productId: line.productId,
                    quantity: toInvoice,
                    price: line.price,
                    neto: lineNeto,
                    iva: lineNeto * 0.19,
                    total: lineNeto * 1.19
                });
                await DB.update('salesOrderLines', {
                    ...line,
                    invoiced: (line.invoiced || 0) + toInvoice
                });
            }
            const iva = subtotal * 0.19;
            const total = subtotal + iva;
            await DB.update('customerInvoices', { ...invoice, subtotal, iva, total });
            let customersAccount = accounts.find(a =>
                a.name?.toLowerCase().includes('cliente') &&
!a.name?.toLowerCase().includes('banco')
            ) || accounts.find(a => a.code === '1.1.03' &&
!a.name?.toLowerCase().includes('banco'));
            let bridgeAccount = accounts.find(a =>
                a.type === 'asset' && a.name?.toLowerCase().includes('facturas por
emitir')
            ) || accounts.find(a => a.code === '1.1.06');
            let ivaDebAccount = accounts.find(a =>
                a.name?.toLowerCase().includes('iva') &&
                (a.name?.toLowerCase().includes('débito') ||
a.name?.toLowerCase().includes('debito'))
            ) || accounts.find(a => a.code === '2.1.02');
```

```
            if (!customersAccount) {
                customersAccount = { id: Helpers.generateId(), companyId: company.id,
code: '1.1.05', name: 'Clientes por Cobrar', type: 'asset', nature: 'debit', level: 2,
isActive: true };
                await DB.add('accounts', customersAccount);
            }
            if (!bridgeAccount) {
                bridgeAccount = { id: Helpers.generateId(), companyId: company.id, code:
'1.1.06', name: 'Clientes - Facturas por Emitir', type: 'asset', nature: 'debit', level:
2, isActive: true };
                await DB.add('accounts', bridgeAccount);
            }
            if (!ivaDebAccount) {
                ivaDebAccount = { id: Helpers.generateId(), companyId: company.id, code:
'2.1.02', name: 'IVA Débito Fiscal', type: 'liability', nature: 'credit', level: 2,
isActive: true };
                await DB.add('accounts', ivaDebAccount);
            }
            const salesTransactionData = {
                date: Helpers.getCurrentDate(),
                description: `Emisión Factura ${invoiceNumber} s/Pedido ${order.number} -
${customer?.name || 'Cliente'}`,
                reference: invoiceNumber,
                sourceDocument: 'customerInvoice',
                sourceDocumentId: invoice.id,
                autoPost: true,
                lines: [
                    { accountId: customersAccount.id, description: `${customer?.name} -
Fact. ${invoiceNumber}`, debit: total, credit: 0 },
                    { accountId: bridgeAccount.id, description: `Facturación de entregas
s/Fact. ${invoiceNumber}`, debit: 0, credit: subtotal },
                    { accountId: ivaDebAccount.id, description: `IVA DF Fact.
${invoiceNumber}`, debit: 0, credit: iva }
                ]
            };
            await AccountingService.registerTransaction('sales', salesTransactionData);
            if (customer) {
                await DB.update('customers', { ...customer, balance: (customer.balance ||
0) + total });
            }
            loading.success('Factura generada y contabilizada exitosamente');
            Modal.info({
                title: 'Facturación Completada',
                message: `La Factura N° ${invoiceNumber} fue generada por el total de las
entregas pendientes ($${total.toLocaleString('es-CL')}).`,
            });
            App.navigate('ventas', 'facturas-venta');
        } catch (err) {
            loading.error('Error: ' + err.message);
        }
    },
    async initCustomerInvoices() {
        const company = CompanyService.getCurrent();
        const invoices = await DB.getByIndex('customerInvoices', 'companyId', company.id);
        for (let inv of invoices) {
            const customer = await DB.get('customers', inv.customerId);
            inv.customerName = customer?.name || 'Cliente desconocido';
        }
        DataTable.create('invoices-table', {
            columns: [
                { key: 'number', label: 'Número' },
                { key: 'date', label: 'Fecha', format: 'date' },
                { key: 'customerName', label: 'Cliente' },
                { key: 'dueDate', label: 'Vencimiento', format: 'date' },
                { key: 'total', label: 'Total', format: 'currency', class: 'text-right' },
                { key: 'status', label: 'Estado', format: 'status' }
            ],
            data: invoices,
            actions: [
                { name: 'view', label: 'Ver', icon: 'fas fa-eye', handler: (row) =>
this.showCustomerInvoiceDetail(row) },
                { name: 'post', label: 'Contabilizar', icon: 'fas fa-book', handler: (row)
=> this.postCustomerInvoice(row) }
            ],
            emptyMessage: 'No hay facturas de venta'
        });
        document.getElementById('btn-new-invoice')?.addEventListener('click', () => {
            Modal.open({
                title: 'Crear Factura de Venta',
                content: `
                    <div style="text-align: center; padding: var(--space-4);">
```

```
                        <i class="fas fa-file-invoice" style="font-size: 48px; color:
var(--primary-500); margin-bottom: var(--space-4);"></i>
                        <h3>Emisión desde Pedidos</h3>
                        <p style="margin-top: var(--space-3); color: var(--neutral-600);">
                            Para crear una factura de venta, siga la secuencia documental
correcta:
                        </p>
                        <ol style="text-align: left; margin-top: var(--space-4); margin-
left: var(--space-6);">
                            <li>Ir a <strong>Ventas → Pedidos de Venta</strong></li>
                            <li>Registrar la entrega del pedido (Guía de Despacho)</li>
                            <li>Hacer clic en la acción <strong>"Facturar"</strong> sobre
el pedido entregado.</li>
                        </ol>
                        <p style="margin-top: var(--space-4); color: var(--neutral-600);">
                            Esto permite facturar entregas parciales y mantiene la
trazabilidad.
                        </p>
                    </div>
                `,
                footer: `
                    <button class="btn btn-secondary"
onclick="Modal.close()">Cerrar</button>
                    <button class="btn btn-primary" onclick="Modal.close();
App.navigate('ventas', 'pedidos-venta');"><i class="fas fa-arrow-right"></i> Ir a
Pedidos</button>
                `
            });
        });
    },
    async showCustomerInvoiceModal() {
        const company = CompanyService.getCurrent();
        const customers = await DB.getByIndex('customers', 'companyId', company.id);
        const products = await DB.getByIndex('products', 'companyId', company.id);
        if (customers.length === 0) {
            Toast.warning('Primero debe crear al menos un cliente');
            return;
        }
        Modal.open({
            title: 'Nueva Factura de Venta',
            size: 'large',
            content: `
                <form id="customer-invoice-form">
                    <div class="form-row">
                        <div class="form-group">
                            <label class="form-label">Cliente <span class="text-danger">*
</span></label>
                            <select class="form-control" name="customerId" required>
                                <option value="">Seleccione cliente...</option>
                                ${customers.map(c => `<option value="${c.id}">${c.rut} -
${c.name}</option>`).join('')}
                            </select>
                        </div>
                        <div class="form-group">
                            <label class="form-label">Fecha Emisión</label>
                            <input type="date" class="form-control" name="date"
value="${Helpers.getCurrentDate()}" required>
                        </div>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Fecha Vencimiento</label>
                        <input type="date" class="form-control" name="dueDate"
value="${Helpers.getDatePlusDays(30)}" required>
                    </div>
                    <h4 style="margin: var(--space-4) 0 var(--space-2);">Detalle</h4>
                    <div class="table-container">
                        <table class="data-table">
                            <thead>
                                <tr>
                                    <th style="width: 35%;">Producto/Servicio</th>
                                    <th style="width: 12%;">Cantidad</th>
                                    <th style="width: 15%;">Precio Neto</th>
                                    <th style="width: 15%;">IVA</th>
                                    <th style="width: 15%;">Total</th>
                                    <th style="width: 8%;"></th>
                                </tr>
                            </thead>
                            <tbody id="invoice-lines">
                                <tr class="invoice-line" data-idx="0">
                                    <td>
                                        <select class="form-control product-select-inv"
```

```
name="productId">
                                                <option value="">Seleccione...</option>
                                                ${products.map(p => `<option value="${p.id}"
data-price="${p.price}">${p.code} - ${p.name}</option>`).join('')}
                                            </select>
                                        </td>
                                        <td><input type="number" class="form-control quantity-
input-inv" value="1" min="1"></td>
                                        <td><input type="number" class="form-control price-
input-inv" value="0" min="0"></td>
                                        <td class="text-right iva-amount-inv">$0</td>
                                        <td class="text-right line-total-inv">$0</td>
                                        <td><button type="button" class="btn btn-icon btn-
ghost remove-line-inv"><i class="fas fa-times"></i></button></td>
                                    </tr>
                                </tbody>
                                <tfoot>
                                    <tr><td colspan="4" class="text-right">Subtotal Neto:</td>
<td class="text-right" id="inv-subtotal">$0</td><td></td></tr>
                                    <tr><td colspan="4" class="text-right">IVA (19%):</td><td
class="text-right" id="inv-iva">$0</td><td></td></tr>
                                    <tr style="font-weight: 600;"><td colspan="4" class="text-
right">TOTAL:</td><td class="text-right" id="inv-total">$0</td><td></td></tr>
                                </tfoot>
                            </table>
                        </div>
                        <button type="button" class="btn btn-outline" id="btn-add-inv-line"
style="margin-top: var(--space-2);"><i class="fas fa-plus"></i> Agregar Línea</button>
                    </form>
                `,
                footer: `
                    <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                    <button class="btn btn-outline" id="btn-save-inv-draft"><i class="fas fa-
save"></i> Guardar Borrador</button>
                    <button class="btn btn-primary" id="btn-save-inv-post"><i class="fas fa-
book"></i> Guardar y Contabilizar</button>
                `
            });
            this.initCustomerInvoiceModalEvents(products);
        },
        initCustomerInvoiceModalEvents(products) {
            let lineIdx = 1;
            const IVA_RATE = 0.19;
            document.getElementById('btn-add-inv-line')?.addEventListener('click', () => {
                const tbody = document.getElementById('invoice-lines');
                const newRow = document.createElement('tr');
                newRow.className = 'invoice-line';
                newRow.dataset.idx = lineIdx++;
                newRow.innerHTML = `
                    <td><select class="form-control product-select-inv"><option
value="">Seleccione...</option>${products.map(p => `<option value="${p.id}" data-
price="${p.price}">${p.code} - ${p.name}</option>`).join('')}</select></td>
                    <td><input type="number" class="form-control quantity-input-inv" value="1"
min="1"></td>
                    <td><input type="number" class="form-control price-input-inv" value="0"
min="0"></td>
                    <td class="text-right iva-amount-inv">$0</td>
                    <td class="text-right line-total-inv">$0</td>
                    <td><button type="button" class="btn btn-icon btn-ghost remove-line-inv">
<i class="fas fa-times"></i></button></td>
                `;
                tbody.appendChild(newRow);
                this.bindCustomerInvoiceLineEvents(IVA_RATE);
            });
            this.bindCustomerInvoiceLineEvents(IVA_RATE);
            document.getElementById('btn-save-inv-draft')?.addEventListener('click', () =>
this.saveCustomerInvoice(false));
            document.getElementById('btn-save-inv-post')?.addEventListener('click', () =>
this.saveCustomerInvoice(true));
        },
        bindCustomerInvoiceLineEvents(IVA_RATE) {
            document.querySelectorAll('.product-select-inv').forEach(select => {
                select.onchange = () => {
                    const price = parseFloat(select.selectedOptions[0]?.dataset.price || 0);
                    select.closest('tr').querySelector('.price-input-inv').value = price;
                    this.updateCustomerInvoiceTotals(IVA_RATE);
                };
            });
            document.querySelectorAll('.quantity-input-inv, .price-input-inv').forEach(input
=> {
```

```
            input.oninput = () => this.updateCustomerInvoiceTotals(IVA_RATE);
        });
        document.querySelectorAll('.remove-line-inv').forEach(btn => {
            btn.onclick = () => {
                if (document.querySelectorAll('.invoice-line').length > 1) {
                    btn.closest('tr').remove();
                    this.updateCustomerInvoiceTotals(IVA_RATE);
                }
            };
        });
    },
    updateCustomerInvoiceTotals(IVA_RATE) {
        let subtotal = 0, totalIva = 0;
        document.querySelectorAll('.invoice-line').forEach(row => {
            const qty = parseFloat(row.querySelector('.quantity-input-inv').value) || 0;
            const price = parseFloat(row.querySelector('.price-input-inv').value) || 0;
            const neto = qty * price;
            const iva = neto * IVA_RATE;
            row.querySelector('.iva-amount-inv').textContent = Formatters.currency(iva);
            row.querySelector('.line-total-inv').textContent = Formatters.currency(neto +
iva);
            subtotal += neto;
            totalIva += iva;
        });
        document.getElementById('inv-subtotal').textContent =
Formatters.currency(subtotal);
        document.getElementById('inv-iva').textContent = Formatters.currency(totalIva);
        document.getElementById('inv-total').textContent = Formatters.currency(subtotal +
totalIva);
    },
    async saveCustomerInvoice(autoPost) {
        const form = document.getElementById('customer-invoice-form');
        const formData = new FormData(form);
        const customerId = formData.get('customerId');
        if (!customerId) { Toast.error('Seleccione un cliente'); return; }
        const IVA_RATE = 0.19;
        const lines = [];
        let subtotal = 0, totalIva = 0;
        document.querySelectorAll('.invoice-line').forEach(row => {
            const productId = row.querySelector('.product-select-inv').value;
            const quantity = parseFloat(row.querySelector('.quantity-input-inv').value) ||
0;
            const price = parseFloat(row.querySelector('.price-input-inv').value) || 0;
            if (quantity > 0 && price > 0) {
                const neto = quantity * price;
                const iva = neto * IVA_RATE;
                lines.push({ productId, quantity, price, neto, iva, total: neto + iva });
                subtotal += neto;
                totalIva += iva;
            }
        });
        if (lines.length === 0) { Toast.error('Agregue al menos una línea'); return; }
        const company = CompanyService.getCurrent();
        const customer = await DB.get('customers', customerId);
        const total = subtotal + totalIva;
        const number = await this.getNextCustomerInvoiceNumber();
        const invoice = {
            id: Helpers.generateId(),
            companyId: company.id,
            customerId,
            number,
            date: formData.get('date'),
            dueDate: formData.get('dueDate'),
            subtotal, iva: totalIva, total,
            status: autoPost ? 'posted' : 'pending',
            createdAt: new Date().toISOString()
        };
        const loading = Toast.loading(autoPost ? 'Guardando y contabilizando...' :
'Guardando...');
        try {
            await DB.add('customerInvoices', invoice);
            for (const line of lines) {
                await DB.add('customerInvoiceLines', { id: Helpers.generateId(),
invoiceId: invoice.id, ...line });
            }
            if (autoPost) {
                await this.createSalesJournalEntry(invoice, customer, subtotal, totalIva,
total);
            }
            loading.success(autoPost ? 'Factura contabilizada' : 'Factura guardada');
            Modal.close();
```

```
            App.navigate('ventas', 'facturas-cliente');
        } catch (err) {
            loading.error('Error: ' + err.message);
        }
    },
    async getNextCustomerInvoiceNumber() {
        const company = CompanyService.getCurrent();
        const invoices = await DB.getByIndex('customerInvoices', 'companyId', company.id);
        const year = new Date().getFullYear();
        const prefix = `FV-${year}-`;
        const nums = invoices
            .filter(i => {
                const numStr = String(i.number || '');
                return numStr.startsWith(prefix);
            })
            .map(i => parseInt(String(i.number || '').replace(prefix, '')) || 0);
        return `${prefix}${String(Math.max(0, ...nums) + 1).padStart(4, '0')}`;
    },
    async createSalesJournalEntry(invoice, customer, subtotal, iva, total) {
        const company = CompanyService.getCurrent();
        const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
        let customersAccount = accounts.find(a =>
            a.name?.toLowerCase().includes('cliente') &&
            !a.name?.toLowerCase().includes('banco')
        ) || accounts.find(a => a.code === '1.1.05');
        let salesAccount = accounts.find(a =>
            a.type === 'revenue' && a.name?.toLowerCase().includes('venta')
        ) || accounts.find(a => a.code?.startsWith('4.1'));
        let ivaDebAccount = accounts.find(a =>
            a.name?.toLowerCase().includes('iva') &&
            (a.name?.toLowerCase().includes('débito') ||
a.name?.toLowerCase().includes('debito'))
        ) || accounts.find(a => a.code === '2.1.02');
        if (!customersAccount) { customersAccount = { id: Helpers.generateId(), companyId:
company.id, code: '1.1.05', name: 'Clientes por Cobrar', type: 'asset', nature: 'debit',
level: 2, isActive: true }; await DB.add('accounts', customersAccount); }
        if (!salesAccount) { salesAccount = { id: Helpers.generateId(), companyId:
company.id, code: '4.1.01', name: 'Ventas', type: 'revenue', nature: 'credit', level: 2,
isActive: true }; await DB.add('accounts', salesAccount); }
        if (!ivaDebAccount) { ivaDebAccount = { id: Helpers.generateId(), companyId:
company.id, code: '2.1.02', name: 'IVA Débito Fiscal', type: 'liability', nature:
'credit', level: 2, isActive: true }; await DB.add('accounts', ivaDebAccount); }
        const transactionData = {
            date: invoice.date,
            description: `Factura ${invoice.number} - ${customer?.name}`,
            reference: invoice.number,
            sourceDocument: 'customerInvoice',
            sourceDocumentId: invoice.id,
            autoPost: true,
            lines: [
                { accountId: customersAccount.id, description: `${customer?.name} - Fact.
${invoice.number}`, debit: total, credit: 0 },
                { accountId: salesAccount.id, description: `Ventas Fact.
${invoice.number}`, debit: 0, credit: subtotal },
                { accountId: ivaDebAccount.id, description: `IVA DF Fact.
${invoice.number}`, debit: 0, credit: iva }
            ]
        };
        await AccountingService.registerTransaction('sales', transactionData);
        if (customer) {
            await DB.update('customers', { ...customer, balance: (customer.balance || 0) +
total });
        }
    },
    async showCustomerInvoiceDetail(invoice) {
        const customer = await DB.get('customers', invoice.customerId);
        const lines = await DB.getByIndex('customerInvoiceLines', 'invoiceId',
invoice.id);
        for (let line of lines) {
            const product = await DB.get('products', line.productId);
            line.productName = product?.name || 'Producto';
            line.productCode = product?.code || '';
        }
        const statusLabels = { pending: 'Pendiente', posted: 'Contabilizada', paid:
'Pagada' };
        const statusClass = { pending: 'warning', posted: 'info', paid: 'success' };
        Modal.open({
            title: `Factura ${invoice.number}`,
            size: 'large',
            content: `
                <div class="invoice-detail">
```

```
                    <div style="display: flex; justify-content: space-between; margin-
bottom: var(--space-4);">
                        <div>
                            <p><strong>Cliente:</strong> ${customer?.name || 'Cliente'}
</p>
                            <p><strong>RUT:</strong> ${customer?.rut || '-'}</p>
                        </div>
                        <div style="text-align: right;">
                            <p><strong>Fecha:</strong> ${Formatters.date(invoice.date)}
</p>
                            <p><strong>Vencimiento:</strong>
${Formatters.date(invoice.dueDate)}</p>
                            <p><span class="badge
badge-${statusClass[invoice.status]}">${statusLabels[invoice.status]}</span></p>
                        </div>
                    </div>
                    <table class="data-table">
                        <thead>
                            <tr>
                                <th>Código</th>
                                <th>Producto</th>
                                <th class="text-right">Cantidad</th>
                                <th class="text-right">Precio</th>
                                <th class="text-right">Neto</th>
                            </tr>
                        </thead>
                        <tbody>
                            ${lines.map(l => `
                                <tr>
                                    <td><code>${l.productCode}</code></td>
                                    <td>${l.productName}</td>
                                    <td class="text-right">${l.quantity}</td>
                                    <td class="text-right">${Formatters.currency(l.price)}
</td>
                                    <td class="text-right">${Formatters.currency(l.neto ||
l.quantity * l.price)}</td>
                                </tr>
                            `).join('')}
                        </tbody>
                        <tfoot>
                            <tr>
                                <td colspan="4" class="text-right">Neto:</td>
                                <td class="text-
right">${Formatters.currency(invoice.subtotal || 0)}</td>
                            </tr>
                            <tr>
                                <td colspan="4" class="text-right">IVA (19%):</td>
                                <td class="text-right">${Formatters.currency(invoice.iva
|| 0)}</td>
                            </tr>
                            <tr style="font-weight: 700; font-size: 1.1em;">
                                <td colspan="4" class="text-right">TOTAL:</td>
                                <td class="text-
right">${Formatters.currency(invoice.total)}</td>
                            </tr>
                        </tfoot>
                    </table>
                    ${invoice.salesOrderId ? `<p style="margin-top: var(--space-3);">
<small>Generada desde pedido de venta</small></p>` : ''}
                </div>
            `,
            footer: `
                <button class="btn btn-secondary" onclick="Modal.close()">Cerrar</button>
                ${invoice.status === 'posted' ? `<button class="btn btn-success"
onclick="Modal.close();
VentasModule.collectCustomerInvoice(VentasModule._currentInvoice);"><i class="fas fa-
money-bill"></i> Cobrar</button>` : ''}
            `
        });
        this._currentInvoice = invoice;
    },
    async postCustomerInvoice(invoice) {
        if (invoice.status !== 'pending') {
            Toast.warning('Esta factura ya está contabilizada');
            return;
        }
        const customer = await DB.get('customers', invoice.customerId);
        const subtotal = invoice.subtotal || invoice.total / 1.19;
        const iva = invoice.iva || invoice.total - subtotal;
        await this.createSalesJournalEntry(invoice, customer, subtotal, iva,
invoice.total);
```

```
        await DB.update('customerInvoices', { ...invoice, status: 'posted' });
        Toast.success('Factura contabilizada');
        App.navigate('ventas', 'facturas-cliente');
    },
    async collectCustomerInvoice(invoice) {
        if (invoice.status === 'pending') {
            Toast.warning('Primero debe contabilizar la factura');
            return;
        }
        if (invoice.status === 'paid') {
            Toast.warning('Esta factura ya está pagada');
            return;
        }
        const customer = await DB.get('customers', invoice.customerId);
        const company = CompanyService.getCurrent();
        const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
        const bankAccounts = accounts.filter(a =>
            a.type === 'asset' && (a.code?.startsWith('1.1.01') ||
a.code?.startsWith('1.1.02') ||
                a.name?.toLowerCase().includes('banco') ||
a.name?.toLowerCase().includes('caja'))
        );
        Modal.open({
            title: `Cobrar Factura ${invoice.number}`,
            content: `
                <form id="collect-form">
                    <div class="alert alert-info" style="margin-bottom: var(--space-4);">
                        <strong>Cliente:</strong> ${customer?.name || 'Cliente'}<br>
                        <strong>Total a cobrar:</strong>
${Formatters.currency(invoice.total)}
                    </div>
                    <div class="form-group">
                        <label class="form-label">Fecha de Cobro</label>
                        <input type="date" class="form-control" name="paymentDate"
value="${Helpers.getCurrentDate()}" required>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Cuenta de Ingreso</label>
                        <select class="form-control" name="paymentAccountId" required>
                            ${bankAccounts.length > 0 ?
                    bankAccounts.map(a => `<option value="${a.id}">${a.code} - ${a.name}
</option>`).join('') :
                    `<option value="__create_bank__">Banco (crear automáticamente)
</option>
                                <option value="__create_cash__">Caja (crear
automáticamente)</option>`
                }
                        </select>
                    </div>
                    <div class="form-group">
                        <label class="form-label">Referencia / N° Operación</label>
                        <input type="text" class="form-control" name="reference"
placeholder="N° transferencia, cheque, etc.">
                    </div>
                </form>
            `,
            footer: `
                <button class="btn btn-secondary"
onclick="Modal.close()">Cancelar</button>
                <button class="btn btn-success" id="btn-confirm-collect"><i class="fas fa-
check"></i> Confirmar Cobro</button>
            `
        });
        document.getElementById('btn-confirm-collect')?.addEventListener('click', async ()
=> {
            const form = document.getElementById('collect-form');
            const formData = new FormData(form);
            const paymentDate = formData.get('paymentDate');
            let paymentAccountId = formData.get('paymentAccountId');
            const reference = formData.get('reference');
            const loading = Toast.loading('Procesando cobro...');
            try {
                if (paymentAccountId === '__create_bank__') {
                    const newAccount = { id: Helpers.generateId(), companyId: company.id,
code: '1.1.01', name: 'Banco', type: 'asset', nature: 'debit', level: 2, isActive: true };
                    await DB.add('accounts', newAccount);
                    paymentAccountId = newAccount.id;
                } else if (paymentAccountId === '__create_cash__') {
                    const newAccount = { id: Helpers.generateId(), companyId: company.id,
code: '1.1.02', name: 'Caja', type: 'asset', nature: 'debit', level: 2, isActive: true };
                    await DB.add('accounts', newAccount);
```

```
                    paymentAccountId = newAccount.id;
                }
                let customersAccount = accounts.find(a =>
                    a.name?.toLowerCase().includes('cliente') &&
                    !a.name?.toLowerCase().includes('banco')
                ) || accounts.find(a => a.code === '1.1.05');
                if (!customersAccount) {
                    customersAccount = { id: Helpers.generateId(), companyId: company.id,
code: '1.1.05', name: 'Clientes por Cobrar', type: 'asset', nature: 'debit', level: 2,
isActive: true };
                    await DB.add('accounts', customersAccount);
                }
                const entryNumber = await AccountingService.getNextEntryNumber();
                const entry = {
                    id: Helpers.generateId(),
                    companyId: company.id,
                    number: entryNumber,
                    date: paymentDate,
                    description: `Cobro Factura ${invoice.number} - ${customer?.name ||
'Cliente'}`,
                    reference: reference || `COB-${invoice.number}`,
                    status: 'posted',
                    totalDebit: invoice.total,
                    totalCredit: invoice.total,
                    createdAt: new Date().toISOString()
                };
                await DB.add('journalEntries', entry);
                await DB.add('journalLines', {
                    id: Helpers.generateId(),
                    entryId: entry.id,
                    accountId: paymentAccountId,
                    description: `Cobro Fact. ${invoice.number}`,
                    debit: invoice.total,
                    credit: 0
                });
                await DB.add('journalLines', {
                    id: Helpers.generateId(),
                    entryId: entry.id,
                    accountId: customersAccount.id,
                    description: `Cobro de ${customer?.name || 'Cliente'}`,
                    debit: 0,
                    credit: invoice.total
                });
                await DB.update('customerInvoices', {
                    ...invoice,
                    status: 'paid',
                    paidAt: new Date().toISOString(),
                    paymentReference: reference
                });
                if (customer) {
                    await DB.update('customers', {
                        ...customer,
                        balance: Math.max(0, (customer.balance || 0) - invoice.total)
                    });
                }
                Modal.close();
                loading.success(`Cobro registrado:
${Formatters.currency(invoice.total)}`);
                App.navigate('ventas', 'facturas-cliente');
            } catch (err) {
                loading.error('Error: ' + err.message);
            }
        });
    },
    async cancelSalesOrder(order) {
        if (order.status === 'cancelled') {
            Toast.warning('Este pedido ya está anulado');
            return;
        }
        const wasDelivered = order.status === 'delivered';
        let message = `¿Está seguro de anular el pedido ${order.number}?`;
        if (wasDelivered) {
            message += `\n\n⚠️ Este pedido ya fue entregado. Al anularlo:\n• Se restaurará
el stock\n• Se anularán los asientos contables\n• Se anulará la factura asociada`;
        }
        const confirm = await Modal.confirm({
            title: 'Anular Pedido de Venta',
            message: message,
            confirmText: 'Anular Pedido',
            confirmClass: 'btn-danger'
        });
```

```
        if (!confirm) return;
        const loading = Toast.loading('Anulando pedido...');
        try {
            const company = CompanyService.getCurrent();
            if (wasDelivered) {
                const lines = await DB.getByIndex('salesOrderLines', 'orderId', order.id);
                for (const line of lines) {
                    const product = await DB.get('products', line.productId);
                    if (product && !product.isService) {
                        await DB.update('products', {
                            ...product,
                            stock: (product.stock || 0) + line.quantity
                        });
                        await DB.add('stockMovements', {
                            id: Helpers.generateId(),
                            companyId: company.id,
                            productId: line.productId,
                            type: 'in',
                            quantity: line.quantity,
                            reference: `ANULACIÓN ${order.number}`,
                            date: Helpers.getCurrentDate(),
                            notes: `Reversión por anulación de pedido ${order.number}`,
                            createdAt: new Date().toISOString()
                        });
                    }
                }
                const entries = await DB.getByIndex('journalEntries', 'companyId',
company.id);
                const relatedEntries = entries.filter(e => e.reference === order.number ||
e.reference === order.invoiceNumber);
                for (const entry of relatedEntries) {
                    const originalLines = await DB.getByIndex('journalLines', 'entryId',
entry.id);
                    const reversalNumber = await AccountingService.getNextEntryNumber();
                    const reversalEntry = {
                        id: Helpers.generateId(),
                        companyId: company.id,
                        number: reversalNumber,
                        date: Helpers.getCurrentDate(),
                        description: `ANULACIÓN: ${entry.description}`,
                        reference: `ANUL-${entry.reference}`,
                        status: 'posted',
                        totalDebit: entry.totalCredit,
                        totalCredit: entry.totalDebit,
                        createdAt: new Date().toISOString()
                    };
                    await DB.add('journalEntries', reversalEntry);
                    for (const line of originalLines) {
                        await DB.add('journalLines', {
                            id: Helpers.generateId(),
                            entryId: reversalEntry.id,
                            accountId: line.accountId,
                            description: `Reversión: ${line.description}`,
                            debit: line.credit,
                            credit: line.debit
                        });
                    }
                    await DB.update('journalEntries', { ...entry, status: 'cancelled' });
                }
                if (order.invoiceId) {
                    const invoice = await DB.get('customerInvoices', order.invoiceId);
                    if (invoice) {
                        await DB.update('customerInvoices', { ...invoice, status:
'cancelled' });
                        if (invoice.status === 'posted' || invoice.status === 'paid') {
                            const customer = await DB.get('customers', order.customerId);
                            if (customer) {
                                await DB.update('customers', {
                                    ...customer,
                                    balance: Math.max(0, (customer.balance || 0) -
invoice.total)
                                });
                            }
                        }
                    }
                }
            }
            await DB.update('salesOrders', {
                ...order,
                status: 'cancelled',
                cancelledAt: new Date().toISOString()
```

```
            });
            loading.success(`Pedido ${order.number} anulado`);
            App.navigate('ventas', 'pedidos-venta');
        } catch (err) {
            loading.error('Error al anular: ' + err.message);
            console.error(err);
        }
    }
};
window.VentasModule = VentasModule;
```

Archivo: js\services\accounting-service.js

```
const AccountingService = {
    async getChartOfAccounts() {
        const company = CompanyService.getCurrent();
        if (!company) return [];
        const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
        return Helpers.sortBy(accounts, 'code');
    },
    async getAccount(id) {
        return await DB.get('accounts', id);
    },
    async getAccountByCode(code) {
        const accounts = await this.getChartOfAccounts();
        return accounts.find(a => a.code === code);
    },
    async getMovementAccounts() {
        const accounts = await this.getChartOfAccounts();
        return accounts.filter(a => !a.isGroup);
    },
    async createJournalEntry(data) {
        const company = CompanyService.getCurrent();
        if (!company) throw new Error('No hay empresa seleccionada');
        const totalDebit = Helpers.sumBy(data.lines, 'debit');
        const totalCredit = Helpers.sumBy(data.lines, 'credit');
        if (Math.abs(totalDebit - totalCredit) > 0.01) {
            throw new Error(`El asiento no está cuadrado. Débitos: ${totalDebit},
Créditos: ${totalCredit}`);
        }
        const number = await DB.getNextSequence(company.id, 'journalEntry');
        const entry = await DB.add('journalEntries', {
            companyId: company.id,
            number: Helpers.generateDocNumber('AST', number),
            date: data.date || Helpers.getCurrentDate(),
            description: data.description,
            reference: data.reference || null,
            type: data.type || 'manual',
            status: data.autoPost ? 'posted' : 'draft',
            totalDebit,
            totalCredit,
            sourceDocument: data.sourceDocument || null,
            sourceDocumentId: data.sourceDocumentId || null
        });
        for (const line of data.lines) {
            await DB.add('journalLines', {
                entryId: entry.id,
                accountId: line.accountId,
                description: line.description || data.description,
                debit: line.debit || 0,
                credit: line.credit || 0,
                costCenterId: line.costCenterId || null
            });
            if (entry.status === 'posted') {
                await this.updateAccountBalance(line.accountId, line.debit, line.credit);
            }
        }
        await DB.logAudit(company.id, 'CREATE', 'journalEntry', entry.id, { number:
entry.number });
        return entry;
    },
    async postJournalEntry(entryId) {
        const entry = await DB.get('journalEntries', entryId);
        if (!entry) throw new Error('Asiento no encontrado');
        if (entry.status === 'posted') throw new Error('El asiento ya está
```

```
contabilizado');
        if (entry.status === 'cancelled') throw new Error('No se puede contabilizar un
asiento anulado');
        const lines = await DB.getByIndex('journalLines', 'entryId', entryId);
        for (const line of lines) {
            await this.updateAccountBalance(line.accountId, line.debit, line.credit);
        }
        entry.status = 'posted';
        entry.postedAt = new Date().toISOString();
        await DB.update('journalEntries', entry);
        await DB.logAudit(entry.companyId, 'POST', 'journalEntry', entry.id, { number:
entry.number });
        return entry;
    },
    async cancelJournalEntry(entryId, reason) {
        const entry = await DB.get('journalEntries', entryId);
        if (!entry) throw new Error('Asiento no encontrado');
        if (entry.status === 'cancelled') throw new Error('El asiento ya está anulado');
        if (entry.status === 'posted') {
            const lines = await DB.getByIndex('journalLines', 'entryId', entryId);
            for (const line of lines) {
                await this.updateAccountBalance(line.accountId, -line.debit, -
line.credit);
            }
        }
        entry.status = 'cancelled';
        entry.cancelledAt = new Date().toISOString();
        entry.cancelReason = reason;
        await DB.update('journalEntries', entry);
        await DB.logAudit(entry.companyId, 'CANCEL', 'journalEntry', entry.id, { reason
});
        return entry;
    },
    async updateAccountBalance(accountId, debit, credit) {
        const account = await DB.get('accounts', accountId);
        if (!account) return;
        const debitNature = ['asset', 'expense'].includes(account.type);
        if (account.isContra) {
            account.balance += debitNature ? (credit - debit) : (debit - credit);
        } else {
            account.balance += debitNature ? (debit - credit) : (credit - debit);
        }
        await DB.update('accounts', account);
    },
    async recalculateAccountBalances() {
        const company = CompanyService.getCurrent();
        if (!company) throw new Error('No hay empresa seleccionada');
        const accounts = await DB.getByIndex('accounts', 'companyId', company.id);
        console.log(`[Recálculo] Cuentas encontradas: ${accounts.length}`);
        const entries = await DB.getByIndex('journalEntries', 'companyId', company.id);
        const postedEntries = entries.filter(e => e.status === 'posted');
        console.log(`[Recálculo] Asientos totales: ${entries.length}, Contabilizados:
${postedEntries.length}`);
        const allLines = await DB.getAll('journalLines');
        console.log(`[Recálculo] Total de líneas en BD: ${allLines.length}`);
        const accountsMap = new Map();
        for (const account of accounts) {
            account.balance = 0;
            accountsMap.set(account.id, account);
        }
        let processedLines = 0;
        let linesNotFound = 0;
        let accountsNotFound = 0;
        for (const entry of postedEntries) {
            const lines = allLines.filter(l => l.entryId === entry.id);
            if (lines.length === 0) {
                linesNotFound++;
                console.log(`[Recálculo] Asiento ${entry.number || entry.id} no tiene
líneas`);
            }
            for (const line of lines) {
                const account = accountsMap.get(line.accountId);
                if (!account) {
                    accountsNotFound++;
                    console.log(`[Recálculo] Cuenta no encontrada para línea:`,
line.accountId);
                    continue;
                }
                const debitNature = ['asset', 'expense'].includes(account.type);
                const debit = line.debit || 0;
                const credit = line.credit || 0;
```

```
                if (account.isContra) {
                    account.balance += debitNature ? (credit - debit) : (debit - credit);
                } else {
                    account.balance += debitNature ? (debit - credit) : (credit - debit);
                }
                processedLines++;
            }
        }
        console.log(`[Recálculo] Líneas procesadas: ${processedLines}, Asientos sin
líneas: ${linesNotFound}, Cuentas no encontradas: ${accountsNotFound}`);
        if (processedLines === 0 && postedEntries.length > 0) {
            console.warn('[Recálculo] ADVERTENCIA: Se encontraron asientos pero no se
procesaron líneas. Verificar vinculación de journalLines.');
        }
        for (const account of accounts) {
            await DB.update('accounts', account);
        }
        return {
            accountsUpdated: accounts.length,
            entriesProcessed: postedEntries.length,
            linesProcessed: processedLines,
            linesNotFound,
            accountsNotFound
        };
    },
    async getLedger(accountId, dateFrom, dateTo) {
        const entries = await this.getJournalEntries({ status: 'posted', dateFrom, dateTo
});
        const movements = [];
        for (const entry of entries) {
            const lines = await DB.getByIndex('journalLines', 'entryId', entry.id);
            const accountLines = lines.filter(l => l.accountId === accountId);
            for (const line of accountLines) {
                movements.push({
                    date: entry.date,
                    entryNumber: entry.number,
                    description: line.description || entry.description,
                    debit: line.debit,
                    credit: line.credit,
                    reference: entry.reference
                });
            }
        }
        return Helpers.sortBy(movements, 'date');
    },
    async getNextEntryNumber() {
        const company = CompanyService.getCurrent();
        if (!company) return 'AST-0001';
        const entries = await DB.getByIndex('journalEntries', 'companyId', company.id);
        const year = new Date().getFullYear();
        const prefix = `AST-${year}-`;
        const existingNumbers = entries
            .filter(e => {
                const numStr = String(e.number || '');
                return numStr.startsWith(prefix) || numStr.startsWith('AST-');
            })
            .map(e => {
                const numStr = String(e.number || '');
                const match = numStr.match(/(\d+)$/);
                return match ? parseInt(match[1]) : 0;
            });
        const nextNum = Math.max(0, ...existingNumbers) + 1;
        return `${prefix}${String(nextNum).padStart(4, '0')}`;
    },
    async getJournalEntries(filters = {}) {
        const company = CompanyService.getCurrent();
        if (!company) return [];
        let entries = await DB.getByIndex('journalEntries', 'companyId', company.id);
        if (filters.status) {
            entries = entries.filter(e => e.status === filters.status);
        }
        if (filters.dateFrom) {
            entries = entries.filter(e => e.date >= filters.dateFrom);
        }
        if (filters.dateTo) {
            entries = entries.filter(e => e.date <= filters.dateTo);
        }
        if (filters.type) {
            entries = entries.filter(e => e.type === filters.type);
        }
        return Helpers.sortBy(entries, 'date', 'desc');
```

```
    },
    async getJournalLines(entryId) {
        return await DB.getByIndex('journalLines', 'entryId', entryId);
    },
    async getBalanceSheet(date) {
        const accounts = await this.getChartOfAccounts();
        const assets = accounts.filter(a => a.type === 'asset' && !a.isGroup);
        const liabilities = accounts.filter(a => a.type === 'liability' && !a.isGroup);
        const isCurrentPeriodResultAccount = (acc) => {
            const nameLower = acc.name.toLowerCase();
            const isResultDelEjercicio = nameLower.includes('resultado del ejercicio') &&
!nameLower.includes('acumulad');
            const isSpecificCode = acc.code === '3.3.02' || acc.code === '3.2.02';
            return isResultDelEjercicio || isSpecificCode;
        };
        const equityExcludingCurrentResult = accounts.filter(a =>
            a.type === 'equity' &&
            !a.isGroup &&
            !isCurrentPeriodResultAccount(a)
        );
        const currentResultAccounts = accounts.filter(a =>
            a.type === 'equity' &&
            !a.isGroup &&
            isCurrentPeriodResultAccount(a)
        );
        const totalAssets = Helpers.sumBy(assets, 'balance');
        const totalLiabilities = Helpers.sumBy(liabilities, 'balance');
        const totalEquityExcludingCurrentResult =
Helpers.sumBy(equityExcludingCurrentResult, 'balance');
        const totalCurrentResultAccounts = Helpers.sumBy(currentResultAccounts,
'balance');
        const result = await this.getNetIncome(date);
        const displayNetIncome = totalCurrentResultAccounts !== 0 ?
totalCurrentResultAccounts : result.netIncome;
        const totalEquity = totalEquityExcludingCurrentResult + displayNetIncome;
        const difference = totalAssets - (totalLiabilities + totalEquity);
        const balanced = Math.abs(difference) < 0.01;
        const currentAssets = assets.filter(a => a.code.startsWith('1.1'));
        const nonCurrentAssets = assets.filter(a => a.code.startsWith('1.2') ||
(!a.code.startsWith('1.1') && !a.code.startsWith('1.2')));
        const currentLiabilities = liabilities.filter(l => l.code.startsWith('2.1'));
        const nonCurrentLiabilities = liabilities.filter(l => l.code.startsWith('2.2') ||
(!l.code.startsWith('2.1') && !l.code.startsWith('2.2')));
        const totalCurrentAssets = Helpers.sumBy(currentAssets, 'balance');
        const totalNonCurrentAssets = Helpers.sumBy(nonCurrentAssets, 'balance');
        const totalCurrentLiabilities = Helpers.sumBy(currentLiabilities, 'balance');
        const totalNonCurrentLiabilities = Helpers.sumBy(nonCurrentLiabilities,
'balance');
        return {
            date,
            assets: {
                items: assets,
                current: { items: currentAssets, total: totalCurrentAssets },
                nonCurrent: { items: nonCurrentAssets, total: totalNonCurrentAssets },
                total: totalAssets
            },
            liabilities: {
                items: liabilities,
                current: { items: currentLiabilities, total: totalCurrentLiabilities },
                nonCurrent: { items: nonCurrentLiabilities, total:
totalNonCurrentLiabilities },
                total: totalLiabilities
            },
            equity: {
                items: equityExcludingCurrentResult.concat(currentResultAccounts),
                total: totalEquity
            },
            netIncome: displayNetIncome,
            difference: difference,
            balanced: balanced
        };
    },
    async getIncomeStatement(dateFrom, dateTo) {
        const accounts = await this.getChartOfAccounts();
        const revenues = accounts.filter(a => a.type === 'revenue' && !a.isGroup);
        const expenses = accounts.filter(a => a.type === 'expense' && !a.isGroup);
        let totalRevenue = Helpers.sumBy(revenues, 'balance');
        let totalExpenses = Helpers.sumBy(expenses, 'balance');
        const company = CompanyService.getCurrent();
        if (company && company.accountingSystem === 'centralizer') {
            const auxiliaryJournals = await DB.getByIndex('auxiliaryJournals',
```

```
'companyId', company.id);
            const openJournals = auxiliaryJournals.filter(j => j.status === 'open');
            for (const journal of openJournals) {
                for (const trans of journal.transactions) {
                    if (dateFrom && trans.date < dateFrom) continue;
                    if (dateTo && trans.date > dateTo) continue;
                    for (const line of trans.lines) {
                        const account = revenues.find(a => a.id === line.accountId) ||
                            expenses.find(a => a.id === line.accountId);
                        if (!account) continue;
                        const amount = (['revenue', 'liability',
'equity'].includes(account.type))
                            ? (line.credit - line.debit)
                            : (line.debit - line.credit);
                        if (account.type === 'revenue') {
                            totalRevenue += amount;
                        } else {
                            totalExpenses += amount;
                        }
                        account.balance = (account.balance || 0) + amount;
                    }
                }
            }
        }
        const netIncome = totalRevenue - totalExpenses;
        const isProvisional = company && company.accountingSystem === 'centralizer';
        const costOfSalesItems = expenses.filter(e => e.code.startsWith('5') && e.balance
!== 0);
        const totalCostOfSales = Helpers.sumBy(costOfSalesItems, 'balance');
        const otherExpenseItems = expenses.filter(e => !e.code.startsWith('5') &&
e.balance !== 0);
        const totalOtherExpenses = Helpers.sumBy(otherExpenseItems, 'balance');
        return {
            period: { from: dateFrom, to: dateTo },
            isProvisional,
            revenues: { items: revenues.filter(r => r.balance !== 0), total: totalRevenue
},
            expenses: { items: expenses.filter(e => e.balance !== 0), total: totalExpenses
},
            costOfSales: { items: costOfSalesItems, total: totalCostOfSales },
            otherExpenses: { items: otherExpenseItems, total: totalOtherExpenses },
            grossProfit: totalRevenue - totalCostOfSales,
            operatingIncome: totalRevenue - Helpers.sumBy(expenses.filter(e =>
e.code.startsWith('5') || e.code.startsWith('6.1') || e.code.startsWith('6.2')),
'balance'),
            netIncome
        };
    },
    async getNetIncome(date) {
        const statement = await this.getIncomeStatement(null, date);
        return { netIncome: statement.netIncome };
    },
    async generateClosingEntry(fiscalYear) {
        const statement = await this.getIncomeStatement(`${fiscalYear}-01-01`,
`${fiscalYear}-12-31`);
        const lines = [];
        for (const revenue of statement.revenues.items) {
            if (revenue.balance > 0) {
                lines.push({ accountId: revenue.id, debit: revenue.balance, credit: 0 });
            }
        }
        for (const expense of statement.expenses.items) {
            if (expense.balance > 0) {
                lines.push({ accountId: expense.id, debit: 0, credit: expense.balance });
            }
        }
        const resultAccount = await this.getAccountByCode('3.3.02');
        if (resultAccount) {
            if (statement.netIncome > 0) {
                lines.push({ accountId: resultAccount.id, debit: 0, credit:
statement.netIncome });
            } else {
                lines.push({ accountId: resultAccount.id, debit:
Math.abs(statement.netIncome), credit: 0 });
            }
        }
        return await this.createJournalEntry({
            date: `${fiscalYear}-12-31`,
            description: `Asiento de cierre ejercicio ${fiscalYear}`,
            type: 'closing',
            lines,
```

```
        });
    },
    async registerTransaction(moduleSource, transactionData) {
        const company = CompanyService.getCurrent();
        if (!company) throw new Error('No hay empresa seleccionada');
        const accountingSystem = company.accountingSystem || 'journalizer';
        if (accountingSystem === 'centralizer') {
            return await this.addToAuxiliaryJournal(moduleSource, transactionData);
        } else {
            return await this.createJournalEntry(transactionData);
        }
    },
    async addToAuxiliaryJournal(module, transactionData) {
        const company = CompanyService.getCurrent();
        if (!company) throw new Error('No hay empresa seleccionada');
        const period = transactionData.date.substring(0, 7);
        let auxiliaryJournal = await this.getAuxiliaryJournal(module, period);
        if (!auxiliaryJournal) {
            auxiliaryJournal = await DB.add('auxiliaryJournals', {
                companyId: company.id,
                module,
                period,
                status: 'open',
                transactions: [],
                centralizedEntryId: null
            });
        }
        const transaction = {
            transactionId: Helpers.generateId(),
            date: transactionData.date,
            description: transactionData.description,
            reference: transactionData.reference || null,
            sourceDocument: transactionData.sourceDocument || null,
            sourceDocumentId: transactionData.sourceDocumentId || null,
            lines: transactionData.lines
        };
        auxiliaryJournal.transactions.push(transaction);
        await DB.update('auxiliaryJournals', auxiliaryJournal);
        await DB.logAudit(company.id, 'ADD_TO_AUXILIARY', module,
transaction.transactionId, {
            module, period
        });
        return { auxiliaryJournal, transaction };
    },
    async getAuxiliaryJournal(module, period) {
        const company = CompanyService.getCurrent();
        if (!company) return null;
        const journals = await DB.getByIndex('auxiliaryJournals', 'companyId',
company.id);
        return journals.find(j => j.module === module && j.period === period);
    },
    async getAuxiliaryJournals(period = null) {
        const company = CompanyService.getCurrent();
        if (!company) return [];
        let journals = await DB.getByIndex('auxiliaryJournals', 'companyId', company.id);
        if (period) {
            journals = journals.filter(j => j.period === period);
        }
        return journals;
    },
    async centralizeAuxiliaryJournal(module, period) {
        const company = CompanyService.getCurrent();
        if (!company) throw new Error('No hay empresa seleccionada');
        const auxiliaryJournal = await this.getAuxiliaryJournal(module, period);
        if (!auxiliaryJournal) {
            throw new Error(`No se encontró libro auxiliar para ${module} en período
${period}`);
        }
        if (auxiliaryJournal.status === 'centralized') {
            if (auxiliaryJournal.centralizedEntryId) {
                const prevEntry = await DB.get('journalEntries',
auxiliaryJournal.centralizedEntryId);
                if (prevEntry && prevEntry.status === 'cancelled') {
                    auxiliaryJournal.status = 'open';
                    auxiliaryJournal.centralizedEntryId = null;
                    auxiliaryJournal.centralizedAt = null;
                    await DB.update('auxiliaryJournals', auxiliaryJournal);
                } else {
                    throw new Error('Este libro auxiliar ya fue centralizado. Anule
primero el asiento de centralización para volver a centralizar.');
                }
```

```
            } else {
                throw new Error('Este libro auxiliar ya fue centralizado.');
            }
        }
        const consolidatedLines = {};
        for (const transaction of auxiliaryJournal.transactions) {
            for (const line of transaction.lines) {
                const key = line.accountId;
                if (!consolidatedLines[key]) {
                    consolidatedLines[key] = {
                        accountId: line.accountId,
                        debit: 0,
                        credit: 0
                    };
                }
                consolidatedLines[key].debit += (line.debit || 0);
                consolidatedLines[key].credit += (line.credit || 0);
            }
        }
        const lines = Object.values(consolidatedLines).filter(l =>
            l.debit !== 0 || l.credit !== 0
        );
        const moduleNames = {
            sales: 'Ventas',
            purchases: 'Compras',
            treasury: 'Tesorería'
        };
        const [year, month] = period.split('-').map(Number);
        const lastDay = new Date(year, month, 0).getDate();
        const centralizedEntry = await this.createJournalEntry({
            date: `${period}-${String(lastDay).padStart(2, '0')}`,
            description: `Centralización ${moduleNames[module]} - ${period}`,
            reference: `CENTRAL-${module.toUpperCase()}-${period}`,
            type: 'centralization',
            sourceDocument: 'auxiliaryJournal',
            sourceDocumentId: auxiliaryJournal.id,
            lines,
            autoPost: false
        });
        auxiliaryJournal.status = 'centralized';
        auxiliaryJournal.centralizedEntryId = centralizedEntry.id;
        auxiliaryJournal.centralizedAt = new Date().toISOString();
        await DB.update('auxiliaryJournals', auxiliaryJournal);
        await DB.logAudit(company.id, 'CENTRALIZE', module, auxiliaryJournal.id, {
            period, entryId: centralizedEntry.id, transactionCount:
auxiliaryJournal.transactions.length
        });
        return {
            auxiliaryJournal,
            centralizedEntry,
            transactionCount: auxiliaryJournal.transactions.length,
            linesConsolidated: lines.length
        };
    }
};
window.AccountingService = AccountingService;
```

Archivo: js\services\company-service.js

```
const CompanyService = {
    currentCompany: null,
    async getAll() {
        return await DB.getAll('companies');
    },
    async getById(id) {
        return await DB.get('companies', id);
    },
    async create(data) {
        return await Migrations.createCompanyWithDefaults(data);
    },
    async update(id, data) {
        const company = await this.getById(id);
        if (!company) throw new Error('Empresa no encontrada');
        const updated = { ...company, ...data };
        await DB.update('companies', updated);
```

```
        await DB.logAudit(id, 'UPDATE', 'company', id, data);
        if (this.currentCompany?.id === id) {
            this.currentCompany = updated;
            this.updateUI();
        }
        return updated;
    },
    async delete(id) {
        const stores = Object.keys(DB.stores).filter(s => s !== 'companies');
        for (const store of stores) {
            const records = await DB.getByIndex(store, 'companyId', id);
            for (const record of records) {
                await DB.delete(store, record.id);
            }
        }
        await DB.delete('companies', id);
        if (this.currentCompany?.id === id) {
            this.currentCompany = null;
            this.updateUI();
        }
        return true;
    },
    async setCurrent(id) {
        const company = await this.getById(id);
        if (!company) throw new Error('Empresa no encontrada');
        this.currentCompany = company;
        localStorage.setItem('currentCompanyId', id);
        this.updateUI();
        window.dispatchEvent(new CustomEvent('companyChanged', { detail: company }));
        return company;
    },
    getCurrent() {
        return this.currentCompany;
    },
    hasCompany() {
        return this.currentCompany !== null;
    },
    async loadSavedCompany() {
        const savedId = localStorage.getItem('currentCompanyId');
        if (savedId) {
            try {
                await this.setCurrent(savedId);
                return true;
            } catch (e) {
                localStorage.removeItem('currentCompanyId');
            }
        }
        return false;
    },
    updateUI() {
        const nameEl = document.getElementById('company-name');
        if (nameEl) {
            nameEl.textContent = this.currentCompany?.name || 'Sin empresa';
        }
    },
    async exportData(format = 'json') {
        if (!this.currentCompany) throw new Error('No hay empresa seleccionada');
        const data = await DB.exportCompanyData(this.currentCompany.id);
        if (format === 'json') {
            const json = JSON.stringify(data, null, 2);
            Helpers.downloadFile(json, `${this.currentCompany.name}_backup.json`,
'application/json');
        }
        return data;
    },
    async importData(jsonData) {
        const data = typeof jsonData === 'string' ? JSON.parse(jsonData) : jsonData;
        const result = await DB.importCompanyData(data);
        if (data.companies && data.companies.length > 0) {
            const importedCompany = data.companies[0];
            await this.setCurrent(importedCompany.id);
            if (typeof App !== 'undefined' && App.updateCompanySelector) {
                await App.updateCompanySelector();
            }
        } else if (data.companyId) {
            await this.setCurrent(data.companyId);
        }
        return result;
    },
    async createDemoData() {
        if (!this.currentCompany) throw new Error('No hay empresa seleccionada');
```

```
        return await Migrations.createDemoData(this.currentCompany.id);
    },
    async getSettings() {
        if (!this.currentCompany) return null;
        const settings = await DB.getByIndex('settings', 'companyId',
this.currentCompany.id);
        return settings[0] || null;
    },
    async updateSettings(data) {
        if (!this.currentCompany) throw new Error('No hay empresa seleccionada');
        let settings = await this.getSettings();
        if (settings) {
            settings = { ...settings, ...data };
            await DB.update('settings', settings);
        } else {
            settings = await DB.add('settings', { companyId: this.currentCompany.id,
...data });
        }
        return settings;
    },
    async getStats() {
        if (!this.currentCompany) return null;
        const companyId = this.currentCompany.id;
        const [suppliers, customers, products, employees, entries] = await Promise.all([
            DB.getByIndex('suppliers', 'companyId', companyId),
            DB.getByIndex('customers', 'companyId', companyId),
            DB.getByIndex('products', 'companyId', companyId),
            DB.getByIndex('employees', 'companyId', companyId),
            DB.getByIndex('journalEntries', 'companyId', companyId)
        ]);
        return {
            suppliersCount: suppliers.length,
            customersCount: customers.length,
            productsCount: products.length,
            employeesCount: employees.length,
            entriesCount: entries.length,
            lastActivity: entries.length > 0 ? entries[entries.length - 1].createdAt :
null
        };
    }
};
window.CompanyService = CompanyService;
```

Archivo: js\services\export-service.js

```
const ExportService = {
    async toExcel(data, filename, options = {}) {
        const { sheetName = 'Datos', headers = null, columnWidths = [] } = options;
        let exportData = data;
        if (headers) {
            exportData = data.map(row => {
                const newRow = {};
                headers.forEach(h => {
                    newRow[h.label] = row[h.key] ?? '';
                });
                return newRow;
            });
        }
        const wb = XLSX.utils.book_new();
        const ws = XLSX.utils.json_to_sheet(exportData);
        if (columnWidths.length > 0) {
            ws['!cols'] = columnWidths.map(w => ({ wch: w }));
        }
        XLSX.utils.book_append_sheet(wb, ws, sheetName);
        const fullFilename = filename.endsWith('.xlsx') ? filename : `${filename}.xlsx`;
        XLSX.writeFile(wb, fullFilename);
        return true;
    },
    async toCSV(data, filename, options = {}) {
        const { separator = ';', headers = null } = options;
        if (data.length === 0) {
            Toast.show('No hay datos para exportar', 'warning');
            return false;
        }
        const headerRow = headers
```

```
            ? headers.map(h => h.label)
            : Object.keys(data[0]);
        const rows = [headerRow.join(separator)];
        data.forEach(row => {
            const values = headers
                ? headers.map(h => this.formatCSVValue(row[h.key] ?? ''))
                : Object.values(row).map(v => this.formatCSVValue(v));
            rows.push(values.join(separator));
        });
        const csv = rows.join('\n');
        const bom = '\uFEFF';
        Helpers.downloadFile(bom + csv, `${filename}.csv`, 'text/csv;charset=utf-8');
        return true;
    },
    formatCSVValue(value) {
        if (value === null || value === undefined) return '';
        const str = String(value);
        if (str.includes(';') || str.includes('"') || str.includes('\n')) {
            return `"${str.replace(/"/g, '""')}"`;
        }
        return str;
    },
    async toPDF(data, filename, options = {}) {
        const {
            title = 'Reporte',
            subtitle = '',
            headers = null,
            orientation = 'portrait',
            pageSize = 'letter',
            companyName = '',
            date = new Date()
        } = options;
        const jsPDF = window.jspdf.jsPDF;
        const doc = new jsPDF(orientation, 'mm', pageSize);
        const pageWidth = doc.internal.pageSize.getWidth();
        const margin = 15;
        let yPos = margin;
        doc.setFontSize(18);
        doc.setFont('helvetica', 'bold');
        doc.text(companyName || 'EDU-TRACE ERP', margin, yPos);
        yPos += 5;
        doc.setDrawColor(10, 110, 209);
        doc.setLineWidth(0.5);
        doc.line(margin, yPos, pageWidth - margin, yPos);
        yPos += 10;
        doc.setFontSize(14);
        doc.text(title, margin, yPos);
        if (subtitle) {
            yPos += 6;
            doc.setFontSize(10);
            doc.setFont('helvetica', 'normal');
            doc.text(subtitle, margin, yPos);
        }
        yPos += 6;
        doc.setFontSize(9);
        doc.setTextColor(100);
        doc.text(`Generado: ${Formatters.datetime(date)}`, margin, yPos);
        doc.setTextColor(0);
        yPos += 10;
        if (data.length > 0) {
            const tableHeaders = headers
                ? headers.map(h => h.label)
                : Object.keys(data[0]);
            const tableData = data.map(row => {
                return headers
                    ? headers.map(h => {
                        const val = row[h.key];
                        if (h.format === 'currency') return Formatters.currency(val);
                        if (h.format === 'date') return Formatters.date(val);
                        if (h.format === 'number') return Formatters.number(val);
                        return val ?? '';
                    })
                    : Object.values(row);
            });
            doc.autoTable({
                startY: yPos,
                head: [tableHeaders],
                body: tableData,
                theme: 'striped',
                headStyles: {
                    fillColor: [10, 110, 209],
```

```
                    textColor: 255,
                    fontSize: 9,
                    fontStyle: 'bold'
                },
                bodyStyles: {
                    fontSize: 8
                },
                alternateRowStyles: {
                    fillColor: [245, 245, 245]
                },
                margin: { left: margin, right: margin },
                styles: {
                    overflow: 'linebreak',
                    cellPadding: 2
                }
            });
        }
        const pageCount = doc.internal.getNumberOfPages();
        for (let i = 1; i <= pageCount; i++) {
            doc.setPage(i);
            doc.setFontSize(8);
            doc.setTextColor(150);
            doc.text(
                `Página ${i} de ${pageCount}`,
                pageWidth / 2,
                doc.internal.pageSize.getHeight() - 10,
                { align: 'center' }
            );
        }
        const fullFilename = filename.endsWith('.pdf') ? filename : `${filename}.pdf`;
        doc.save(fullFilename);
        return true;
    },
    async toJSON(data, filename) {
        const json = JSON.stringify(data, null, 2);
        Helpers.downloadFile(json, `${filename}.json`, 'application/json');
        return true;
    },
    async fromExcel(file) {
        return new Promise((resolve, reject) => {
            const reader = new FileReader();
            reader.onload = (e) => {
                try {
                    const data = new Uint8Array(e.target.result);
                    const workbook = XLSX.read(data, { type: 'array' });
                    const result = {};
                    workbook.SheetNames.forEach(sheetName => {
                        const sheet = workbook.Sheets[sheetName];
                        result[sheetName] = XLSX.utils.sheet_to_json(sheet);
                    });
                    resolve(result);
                } catch (err) {
                    reject(err);
                }
            };
            reader.onerror = () => reject(reader.error);
            reader.readAsArrayBuffer(file);
        });
    },
    async fromCSV(file, separator = ';') {
        return new Promise((resolve, reject) => {
            const reader = new FileReader();
            reader.onload = (e) => {
                try {
                    const text = e.target.result;
                    const lines = text.split('\n').filter(l => l.trim());
                    if (lines.length < 2) {
                        resolve([]);
                        return;
                    }
                    const headers = lines[0].split(separator).map(h =>
h.trim().replace(/^"|"$/g, ''));
                    const data = [];
                    for (let i = 1; i < lines.length; i++) {
                        const values = lines[i].split(separator).map(v =>
v.trim().replace(/^"|"$/g, ''));
                        const row = {};
                        headers.forEach((h, idx) => {
                            row[h] = values[idx] ?? '';
                        });
                        data.push(row);
```

```
                    }
                    resolve(data);
                } catch (err) {
                    reject(err);
                }
            };
            reader.onerror = () => reject(reader.error);
            reader.readAsText(file, 'UTF-8');
        });
    },
    async fromJSON(file) {
        return new Promise((resolve, reject) => {
            const reader = new FileReader();
            reader.onload = (e) => {
                try {
                    const data = JSON.parse(e.target.result);
                    resolve(data);
                } catch (err) {
                    reject(err);
                }
            };
            reader.onerror = () => reject(reader.error);
            reader.readAsText(file, 'UTF-8');
        });
    },
    async balanceSheetPDF(balanceData, companyName) {
        const jsPDF = window.jspdf.jsPDF;
        const doc = new jsPDF('portrait', 'mm', 'letter');
        const pageWidth = doc.internal.pageSize.getWidth();
        const margin = 20;
        let y = margin;
        doc.setFontSize(16);
        doc.setFont('helvetica', 'bold');
        doc.text(companyName, pageWidth / 2, y, { align: 'center' });
        y += 8;
        doc.setFontSize(14);
        doc.text('BALANCE GENERAL', pageWidth / 2, y, { align: 'center' });
        y += 6;
        doc.setFontSize(10);
        doc.setFont('helvetica', 'normal');
        doc.text(`Al ${Formatters.date(balanceData.date, 'long')}`, pageWidth / 2, y, {
align: 'center' });
        y += 10;
        doc.setDrawColor(10, 110, 209);
        doc.setLineWidth(0.5);
        doc.line(margin, y, pageWidth - margin, y);
        y += 10;
        const addSection = (title, items, total) => {
            doc.setFontSize(12);
            doc.setFont('helvetica', 'bold');
            doc.text(title, margin, y);
            y += 6;
            doc.setFontSize(9);
            doc.setFont('helvetica', 'normal');
            items.filter(i => i.balance !== 0).forEach(item => {
                doc.text(`    ${item.name}`, margin, y);
                doc.text(Formatters.currency(item.balance), pageWidth - margin - 40, y, {
align: 'right' });
                y += 5;
            });
            y += 2;
            doc.setFont('helvetica', 'bold');
            doc.text(`Total ${title}`, margin + 10, y);
            doc.text(Formatters.currency(total), pageWidth - margin - 40, y, { align:
'right' });
            y += 8;
        };
        addSection('ACTIVOS', balanceData.assets.items, balanceData.assets.total);
        addSection('PASIVOS', balanceData.liabilities.items,
balanceData.liabilities.total);
        addSection('PATRIMONIO', balanceData.equity.items, balanceData.equity.total);
        if (balanceData.netIncome !== 0) {
            doc.text('    Resultado del Ejercicio', margin, y);
            doc.text(Formatters.currency(balanceData.netIncome), pageWidth - margin - 40,
y, { align: 'right' });
            y += 8;
        }
        doc.setDrawColor(0);
        doc.line(margin, y, pageWidth - margin, y);
        y += 5;
        const totalPassiveEquity = balanceData.liabilities.total +
```

```
balanceData.equity.total + balanceData.netIncome;
        doc.setFontSize(10);
        doc.text('TOTAL ACTIVOS:', margin, y);
        doc.text(Formatters.currency(balanceData.assets.total), pageWidth / 2 - 10, y, {
align: 'right' });
        y += 5;
        doc.text('TOTAL PASIVO + PATRIMONIO:', margin, y);
        doc.text(Formatters.currency(totalPassiveEquity), pageWidth / 2 - 10, y, { align:
'right' });
        const fullFilename =
`Balance_General_${Formatters.date(balanceData.date).replace(/\
        doc.save(fullFilename);
        return true;
    }
};
window.ExportService = ExportService;
```

Archivo: js\services\traceability-service.js

```
const TraceabilityService = {
    documentTypes: {
        salesOrder: {
            name: 'Pedido de Venta',
            icon: 'fa-file-alt',
            color: '#3B82F6',
            module: 'SD (Ventas)',
            store: 'salesOrders'
        },
        delivery: {
            name: 'Entrega',
            icon: 'fa-truck',
            color: '#10B981',
            module: 'WM (Inventario)',
            store: 'deliveries'
        },
        customerInvoice: {
            name: 'Factura de Venta',
            icon: 'fa-file-invoice-dollar',
            color: '#F59E0B',
            module: 'SD (Ventas)',
            store: 'customerInvoices'
        },
        purchaseOrder: {
            name: 'Orden de Compra',
            icon: 'fa-shopping-cart',
            color: '#8B5CF6',
            module: 'MM (Compras)',
            store: 'purchaseOrders'
        },
        supplierInvoice: {
            name: 'Factura de Proveedor',
            icon: 'fa-file-invoice',
            color: '#EF4444',
            module: 'MM (Compras)',
            store: 'supplierInvoices'
        },
        journalEntry: {
            name: 'Asiento Contable',
            icon: 'fa-book',
            color: '#6366F1',
            module: 'GL (Contabilidad)',
            store: 'journalEntries'
        },
        customerPayment: {
            name: 'Cobro',
            icon: 'fa-hand-holding-usd',
            color: '#22C55E',
            module: 'FI (Tesorería)',
            store: 'customerPayments'
        },
        supplierPayment: {
            name: 'Pago a Proveedor',
            icon: 'fa-money-bill-wave',
            color: '#EC4899',
            module: 'FI (Tesorería)',
```

```
            store: 'supplierPayments'
        },
        stockMovement: {
            name: 'Movimiento de Stock',
            icon: 'fa-boxes',
            color: '#14B8A6',
            module: 'WM (Inventario)',
            store: 'stockMovements'
        },
        balanceSheet: {
            name: 'Balance General',
            icon: 'fa-balance-scale',
            color: '#DC2626',
            module: 'GL (Contabilidad)',
            store: null
        }
    },
    standardFlows: {
        'order-to-cash': {
            name: 'Order-to-Cash (O2C)',
            description: 'Ciclo completo de venta: desde el pedido hasta el cobro',
            steps: [
                { type: 'salesOrder', label: 'Pedido de Venta' },
                { type: 'delivery', label: 'Entrega' },
                { type: 'stockMovement', label: 'Salida de Inventario' },
                { type: 'customerInvoice', label: 'Facturación' },
                { type: 'journalEntry', label: 'Asiento Contable' },
                { type: 'customerPayment', label: 'Cobro' },
                { type: 'journalEntry', label: 'Asiento de Cobro' }
            ],
            affectedAccounts: [
                { code: '1.1.02.001', name: 'Clientes Nacionales', effect: 'aumento →
disminución' },
                { code: '4.1.01.001', name: 'Ventas de Mercaderías', effect: 'aumento' },
                { code: '1.1.03.001', name: 'Mercaderías', effect: 'disminución' },
                { code: '5.1.01', name: 'Costo de Mercaderías Vendidas', effect: 'aumento'
},
                { code: '1.1.01.001', name: 'Caja/Banco', effect: 'aumento' },
                { code: '2.1.03.001', name: 'IVA Débito Fiscal', effect: 'aumento' }
            ]
        },
        'procure-to-pay': {
            name: 'Procure-to-Pay (P2P)',
            description: 'Ciclo completo de compra: desde la orden hasta el pago',
            steps: [
                { type: 'purchaseOrder', label: 'Orden de Compra' },
                { type: 'stockMovement', label: 'Entrada de Inventario' },
                { type: 'supplierInvoice', label: 'Factura de Proveedor' },
                { type: 'journalEntry', label: 'Asiento Contable' },
                { type: 'supplierPayment', label: 'Pago' },
                { type: 'journalEntry', label: 'Asiento de Pago' }
            ],
            affectedAccounts: [
                { code: '1.1.03.001', name: 'Mercaderías', effect: 'aumento' },
                { code: '2.1.01.001', name: 'Proveedores Nacionales', effect: 'aumento →
disminución' },
                { code: '1.1.04.001', name: 'IVA Crédito Fiscal', effect: 'aumento' },
                { code: '1.1.01.001', name: 'Caja/Banco', effect: 'disminución' }
            ]
        }
    },
    async getDocumentTrace(documentType, documentId) {
        const company = CompanyService.getCurrent();
        if (!company) return null;
        const trace = {
            origin: { type: documentType, id: documentId, data: null },
            chain: [],
            financialImpact: [],
            timeline: [],
            status: 'incomplete'
        };
        try {
            const typeInfo = this.documentTypes[documentType];
            if (!typeInfo) {
                throw new Error(`Tipo de documento desconocido: ${documentType}`);
            }
            const originDoc = await DB.get(typeInfo.store, documentId);
            if (!originDoc) {
                throw new Error(`Documento no encontrado: ${documentType}
#${documentId}`);
            }
```

```
            trace.origin.data = originDoc;
            trace.timeline.push({
                type: documentType,
                document: originDoc,
                date: originDoc.date || originDoc.createdAt,
                description: `${typeInfo.name} ${originDoc.number || '#' + documentId}`
            });
            await this.traceFromDocument(documentType, originDoc, trace, company);
            trace.status = this.calculateTraceStatus(trace);
        } catch (error) {
            trace.error = error.message;
        }
        return trace;
    },
    async traceFromDocument(documentType, document, trace, company) {
        switch (documentType) {
            case 'salesOrder':
                await this.traceSalesOrderChain(document, trace, company);
                break;
            case 'customerInvoice':
                await this.traceCustomerInvoiceChain(document, trace, company);
                break;
            case 'purchaseOrder':
                await this.tracePurchaseOrderChain(document, trace, company);
                break;
            case 'journalEntry':
                await this.traceJournalEntryImpact(document, trace, company);
                break;
        }
    },
    async traceSalesOrderChain(order, trace, company) {
        const deliveries = await DB.getByIndex('deliveries', 'companyId', company.id);
        const relatedDeliveries = deliveries.filter(d => d.salesOrderId === order.id);
        for (const delivery of relatedDeliveries) {
            trace.chain.push({
                type: 'delivery',
                document: delivery,
                linkedBy: 'salesOrderId'
            });
            trace.timeline.push({
                type: 'delivery',
                document: delivery,
                date: delivery.date,
                description: `Entrega ${delivery.number}`
            });
        }
        const movements = await DB.getByIndex('stockMovements', 'companyId', company.id);
        const relatedMovements = movements.filter(m =>
            m.reference === String(order.number) || m.salesOrderId === order.id ||
            (m.description && m.description.includes(String(order.number)))
        );
        for (const movement of relatedMovements) {
            movement.displayNumber = movement.reference || `Mov-${movement.type}`;
            trace.chain.push({
                type: 'stockMovement',
                document: movement,
                linkedBy: 'reference'
            });
        }
        const allEntries = await DB.getByIndex('journalEntries', 'companyId', company.id);
        const deliveryEntries = allEntries.filter(e =>
            (e.sourceDocument === 'salesOrder' && e.sourceDocumentId === order.id) ||
            (e.description && e.description.includes(String(order.number))) ||
            (e.reference && e.reference.includes(String(order.number)))
        );
        for (const entry of deliveryEntries) {
            if (!trace.chain.some(c => c.type === 'journalEntry' && c.document.id ===
entry.id)) {
                trace.chain.push({
                    type: 'journalEntry',
                    document: entry,
                    linkedBy: 'sourceDocumentId'
                });
                trace.timeline.push({
                    type: 'journalEntry',
                    document: entry,
                    date: entry.date,
                    description: `Asiento ${entry.number || '#' + entry.id.slice(0, 8)}`
                });
                const lines = await DB.getByIndex('journalLines', 'entryId', entry.id);
                for (const line of lines) {
```

```
                    const account = await DB.get('accounts', line.accountId);
                    trace.financialImpact.push({
                        type: 'accounting',
                        account: account ? `${account.code} - ${account.name}` : 'Cuenta
desconocida',
                        debit: line.debit,
                        credit: line.credit,
                        description: line.description || entry.description
                    });
                }
            }
        }
        const invoices = await DB.getByIndex('customerInvoices', 'companyId', company.id);
        const relatedInvoices = invoices.filter(i => i.salesOrderId === order.id);
        for (const invoice of relatedInvoices) {
            trace.chain.push({
                type: 'customerInvoice',
                document: invoice,
                linkedBy: 'salesOrderId'
            });
            trace.timeline.push({
                type: 'customerInvoice',
                document: invoice,
                date: invoice.date,
                description: `Factura ${invoice.number} -
${Formatters.currency(invoice.total)}`
            });
            await this.traceCustomerInvoiceChain(invoice, trace, company);
        }
    },
    async traceCustomerInvoiceChain(invoice, trace, company) {
        const entries = await DB.getByIndex('journalEntries', 'companyId', company.id);
        const relatedEntry = entries.find(e =>
            e.sourceDocument === 'customerInvoice' && e.sourceDocumentId === invoice.id
        );
        if (relatedEntry) {
            trace.chain.push({
                type: 'journalEntry',
                document: relatedEntry,
                linkedBy: 'sourceDocumentId'
            });
            trace.timeline.push({
                type: 'journalEntry',
                document: relatedEntry,
                date: relatedEntry.date,
                description: `Asiento ${relatedEntry.number}`
            });
            const lines = await DB.getByIndex('journalLines', 'entryId', relatedEntry.id);
            for (const line of lines) {
                const account = await DB.get('accounts', line.accountId);
                trace.financialImpact.push({
                    type: 'accounting',
                    account: account ? `${account.code} - ${account.name}` : 'Cuenta
desconocida',
                    debit: line.debit,
                    credit: line.credit,
                    description: line.description
                });
            }
        }
        const payments = await DB.getByIndex('customerPayments', 'companyId', company.id);
        const relatedPayments = payments ? payments.filter(p => p.invoiceId ===
invoice.id) : [];
        for (const payment of relatedPayments) {
            trace.chain.push({
                type: 'customerPayment',
                document: payment,
                linkedBy: 'invoiceId'
            });
            trace.timeline.push({
                type: 'customerPayment',
                document: payment,
                date: payment.date,
                description: `Cobro ${Formatters.currency(payment.amount)}`
            });
            trace.financialImpact.push({
                type: 'cash',
                description: `Ingreso de efectivo por cobro`,
                amount: payment.amount,
                direction: 'increase'
            });
```

```
        }
    },
    async tracePurchaseOrderChain(order, trace, company) {
        const movements = await DB.getByIndex('stockMovements', 'companyId', company.id);
        const relatedMovements = movements.filter(m =>
            m.reference === order.number || m.purchaseOrderId === order.id
        );
        for (const movement of relatedMovements) {
            trace.chain.push({
                type: 'stockMovement',
                document: movement,
                linkedBy: 'reference'
            });
            trace.financialImpact.push({
                type: 'inventory',
                description: `Entrada de inventario: ${movement.quantity} unidades`,
                amount: movement.cost || 0,
                direction: 'increase'
            });
        }
        const invoices = await DB.getByIndex('supplierInvoices', 'companyId', company.id);
        const relatedInvoices = invoices.filter(i => i.purchaseOrderId === order.id);
        for (const invoice of relatedInvoices) {
            trace.chain.push({
                type: 'supplierInvoice',
                document: invoice,
                linkedBy: 'purchaseOrderId'
            });
            trace.timeline.push({
                type: 'supplierInvoice',
                document: invoice,
                date: invoice.date,
                description: `Factura Proveedor ${invoice.number} -
${Formatters.currency(invoice.total)}`
            });
        }
    },
    async traceJournalEntryImpact(entry, trace, company) {
        const lines = await DB.getByIndex('journalLines', 'entryId', entry.id);
        for (const line of lines) {
            const account = await DB.get('accounts', line.accountId);
            if (!account) continue;
            const impact = {
                account,
                debit: line.debit,
                credit: line.credit,
                balanceEffect: 0,
                statements: []
            };
            const isDebitNature = ['asset', 'expense'].includes(account.type);
            if (account.isContra) {
                impact.balanceEffect = isDebitNature ? (line.credit - line.debit) :
(line.debit - line.credit);
            } else {
                impact.balanceEffect = isDebitNature ? (line.debit - line.credit) :
(line.credit - line.debit);
            }
            if (['asset', 'liability', 'equity'].includes(account.type)) {
                impact.statements.push('Balance General');
            }
            if (['revenue', 'expense'].includes(account.type)) {
                impact.statements.push('Estado de Resultados');
            }
            if (account.code.startsWith('1.1.01')) {
                impact.statements.push('Flujo de Caja');
            }
            trace.financialImpact.push(impact);
        }
    },
    calculateTraceStatus(trace) {
        if (trace.error) return 'error';
        if (trace.chain.length === 0) return 'origin_only';
        const hasJournalEntry = trace.chain.some(c => c.type === 'journalEntry');
        const hasPayment = trace.chain.some(c =>
            c.type === 'customerPayment' || c.type === 'supplierPayment'
        );
        if (hasJournalEntry && hasPayment) return 'complete';
        if (hasJournalEntry) return 'accounted';
        return 'in_progress';
    },
    async generateTraceReport(documentType, documentId) {
```

```
        const trace = await this.getDocumentTrace(documentType, documentId);
        if (!trace) return null;
        return {
            trace,
            visualization: this.generateVisualization(trace),
            summary: this.generateSummary(trace),
            pedagogicalNotes: this.generatePedagogicalNotes(trace)
        };
    },
    generateVisualization(trace) {
        const nodes = [];
        const edges = [];
        const originType = this.documentTypes[trace.origin.type];
        nodes.push({
            id: `${trace.origin.type}_${trace.origin.id}`,
            label: originType.name,
            number: trace.origin.data?.number || `#${trace.origin.id}`,
            color: originType.color,
            icon: originType.icon,
            module: originType.module,
            isOrigin: true
        });
        let prevNodeId = `${trace.origin.type}_${trace.origin.id}`;
        for (const item of trace.chain) {
            const typeInfo = this.documentTypes[item.type];
            const nodeId = `${item.type}_${item.document.id}`;
            let displayNumber = item.document.number;
            if (!displayNumber || displayNumber === item.document.id) {
                if (item.type === 'stockMovement') {
                    displayNumber = item.document.reference || item.document.displayNumber
|| `Mov-${item.document.type || 'stock'}`;
                } else {
                    displayNumber = `#${String(item.document.id).slice(0, 8)}`;
                }
            }
            nodes.push({
                id: nodeId,
                label: typeInfo.name,
                number: displayNumber,
                color: typeInfo.color,
                icon: typeInfo.icon,
                module: typeInfo.module
            });
            edges.push({
                from: prevNodeId,
                to: nodeId,
                label: item.linkedBy
            });
            prevNodeId = nodeId;
        }
        return { nodes, edges };
    },
    generateSummary(trace) {
        const statusLabels = {
            'complete': 'Ciclo Completo',
            'accounted': 'Contabilizado',
            'in_progress': 'En Proceso',
            'origin_only': 'Solo Origen',
            'error': 'Error'
        };
        let totalDebit = 0;
        let totalCredit = 0;
        let inventoryChange = 0;
        let cashChange = 0;
        for (const impact of trace.financialImpact) {
            if (impact.debit !== undefined) {
                totalDebit += impact.debit;
                totalCredit += impact.credit;
            }
            if (impact.type === 'inventory') {
                inventoryChange += impact.direction === 'increase' ? impact.amount : -
impact.amount;
            }
            if (impact.type === 'cash') {
                cashChange += impact.direction === 'increase' ? impact.amount : -
impact.amount;
            }
        }
        return {
            status: statusLabels[trace.status] || trace.status,
            documentsCount: trace.chain.length + 1,
```

```
            modulesInvolved: [...new Set([
                this.documentTypes[trace.origin.type]?.module,
                ...trace.chain.map(c => this.documentTypes[c.type]?.module)
            ].filter(Boolean))],
            financials: {
                totalDebit,
                totalCredit,
                isBalanced: Math.abs(totalDebit - totalCredit) < 0.01,
                inventoryChange,
                cashChange
            }
        };
    },
    generatePedagogicalNotes(trace) {
        const notes = [];
        notes.push({
            title: '📋 Documento Origen',
            content: `La transacción inicia en el módulo
${this.documentTypes[trace.origin.type]?.module}. ` +
                `Este es el punto de entrada del ciclo de negocio.`
        });
        if (trace.chain.length > 0) {
            const modules = [...new Set(trace.chain.map(c =>
this.documentTypes[c.type]?.module))];
            notes.push({
                title: '🔗 Integración de Módulos',
                content: `Esta transacción atraviesa ${modules.length} módulo(s) del ERP:
${modules.join(', ')}. ` +
                    `Esto demuestra la integración característica de un sistema ERP.`
            });
        }
        const hasAccounting = trace.chain.some(c => c.type === 'journalEntry');
        if (hasAccounting) {
            notes.push({
                title: '📚 Impacto Contable',
                content: `La transacción generó asiento(s) contable(s) automáticamente, ` +
                    `afectando el Libro Mayor (GL) y los estados financieros.`
            });
            const summary = this.generateSummary(trace);
            if (summary.financials.isBalanced) {
                notes.push({
                    title: '✅ Partida Doble',
                    content: `El principio de partida doble se cumple: ` +
                        `Débitos (${Formatters.currency(summary.financials.totalDebit)}) = ` +
                        `Créditos
(${Formatters.currency(summary.financials.totalCredit)})`
                });
            }
        } else {
            notes.push({
                title: '⚠️ Pendiente de Contabilización',
                content: `Esta transacción aún no ha generado asientos contables. ` +
                    `Para completar el ciclo, debe facturarse y contabilizarse.`
            });
        }
        if (trace.status === 'complete') {
            notes.push({
                title: '🎯 Ciclo Completo',
                content: `¡Excelente! Esta transacción completó todo el ciclo de negocio, ` +
                    `desde el origen hasta el cobro/pago y su reflejo en estados
financieros.`
            });
        }
        return notes;
    },
    async getTodayTransactions() {
        const company = CompanyService.getCurrent();
        if (!company) return [];
        const today = new Date().toISOString().split('T')[0];
        const transactions = [];
        for (const [type, config] of Object.entries(this.documentTypes)) {
            try {
                const docs = await DB.getByIndex(config.store, 'companyId', company.id);
                const todayDocs = docs.filter(d => d.date === today ||
d.createdAt?.startsWith(today));
                for (const doc of todayDocs) {
```

```
                    transactions.push({
                        type,
                        document: doc,
                        config
                    });
                }
            } catch (e) {
            }
        }
        return transactions.sort((a, b) =>
            new Date(b.document.createdAt || b.document.date) -
            new Date(a.document.createdAt || a.document.date)
        );
    },
    async verifyDataIntegrity() {
        const company = CompanyService.getCurrent();
        if (!company) return { issues: [], isHealthy: false };
        const issues = [];
        const invoices = await DB.getByIndex('customerInvoices', 'companyId', company.id);
        const entries = await DB.getByIndex('journalEntries', 'companyId', company.id);
        for (const invoice of invoices) {
            if (invoice.status === 'posted') {
                const hasEntry = entries.some(e =>
                    e.sourceDocument === 'customerInvoice' && e.sourceDocumentId ===
invoice.id
                );
                if (!hasEntry) {
                    issues.push({
                        type: 'missing_entry',
                        severity: 'warning',
                        document: invoice,
                        message: `Factura ${invoice.number} está contabilizada pero no
tiene asiento`
                    });
                }
            }
        }
        for (const entry of entries) {
            if (Math.abs(entry.totalDebit - entry.totalCredit) > 0.01) {
                issues.push({
                    type: 'unbalanced_entry',
                    severity: 'error',
                    document: entry,
                    message: `Asiento ${entry.number} está descuadrado`
                });
            }
        }
        const products = await DB.getByIndex('products', 'companyId', company.id);
        for (const product of products) {
            if (product.stock < 0) {
                issues.push({
                    type: 'negative_stock',
                    severity: 'warning',
                    document: product,
                    message: `Producto ${product.code} tiene stock negativo
(${product.stock})`
                });
            }
        }
        return {
            issues,
            isHealthy: issues.filter(i => i.severity === 'error').length === 0,
            summary: {
                total: issues.length,
                errors: issues.filter(i => i.severity === 'error').length,
                warnings: issues.filter(i => i.severity === 'warning').length
            }
        };
    }
};
window.TraceabilityService = TraceabilityService;
```

Archivo: js\services\validation-service.js

```
const ValidationService = {
    createResult(isValid, code, message, details = {}, severity = 'error') {
        return {
            isValid,
            code,
            message,
            details,
            severity,
            timestamp: new Date().toISOString()
        };
    },
    validateDebitCreditBalance(lines) {
        const totalDebit = lines.reduce((sum, line) => sum + (parseFloat(line.debit) ||
0), 0);
        const totalCredit = lines.reduce((sum, line) => sum + (parseFloat(line.credit) ||
0), 0);
        const difference = Math.abs(totalDebit - totalCredit);
        const isBalanced = difference < 0.01;
        return this.createResult(
            isBalanced,
            'DEBIT_CREDIT_BALANCE',
            isBalanced
                ? `✅ Asiento cuadrado: Debe ($${this.formatNumber(totalDebit)}) = Haber
($${this.formatNumber(totalCredit)})`
                : `❌ Asiento descuadrado: Debe ($${this.formatNumber(totalDebit)}) ≠
Haber ($${this.formatNumber(totalCredit)}). Diferencia:
$${this.formatNumber(difference)}`,
            { totalDebit, totalCredit, difference },
            isBalanced ? 'success' : 'error'
        );
    },
    async validateAccountExists(accountId) {
        if (!accountId) {
            return this.createResult(false, 'ACCOUNT_EXISTS', '❌ No se especificó
cuenta', {});
        }
        const account = await DB.get('accounts', accountId);
        return this.createResult(
            !!account,
            'ACCOUNT_EXISTS',
            account
                ? `✅ Cuenta válida: ${account.code} - ${account.name}`
                : '❌ Cuenta no encontrada en el plan de cuentas',
            { accountId, account },
            account ? 'success' : 'error'
        );
    },
    async validateAccountExistsByCode(code) {
        if (!code) {
            return this.createResult(false, 'ACCOUNT_EXISTS_BY_CODE', '❌ No se especificó
código de cuenta', {});
        }
        const accounts = await AccountingService.getChartOfAccounts();
        const account = accounts.find(a => a.code === code);
        return this.createResult(
            !!account,
            'ACCOUNT_EXISTS_BY_CODE',
            account
                ? `✅ Cuenta válida: ${account.code} - ${account.name}`
                : `❌ Código de cuenta "${code}" no encontrado en el plan de cuentas`,
            { code, account },
            account ? 'success' : 'error'
        );
    },
    async validateAccountNature(accountId, debit, credit) {
        const account = await DB.get('accounts', accountId);
        if (!account) {
            return this.createResult(false, 'ACCOUNT_NATURE', '❌ Cuenta no encontrada', {
accountId });
        }
        const debitNature = ['asset', 'expense'].includes(account.type);
        const creditNature = ['liability', 'equity', 'revenue'].includes(account.type);
        const hasDebit = parseFloat(debit) > 0;
        const hasCredit = parseFloat(credit) > 0;
```

```
        let isNaturalMovement = true;
        let explanation = '';
        if (account.isContra) {
            explanation = `Cuenta CONTRA (${account.type}): aumenta con ${debitNature ?
'crédito' : 'débito'}`;
        } else if (debitNature) {
            explanation = `Cuenta de naturaleza DEUDORA (${account.type}): aumenta con
débito, disminuye con crédito`;
        } else {
            explanation = `Cuenta de naturaleza ACREEDORA (${account.type}): aumenta con
crédito, disminuye con débito`;
        }
        const natureLabel = debitNature
            ? (account.isContra ? 'Contra-Deudora' : 'Deudora')
            : (account.isContra ? 'Contra-Acreedora' : 'Acreedora');
        return this.createResult(
            true,
            'ACCOUNT_NATURE',
            `📚 ${account.code} - ${account.name}\n   Naturaleza: ${natureLabel}\n
${explanation}`,
            {
                account: account.name,
                type: account.type,
                isDebitNature: debitNature,
                isContra: account.isContra,
                debit,
                credit,
                explanation
            },
            'info'
        );
    },
    validateRUT(rut) {
        if (!rut) {
            return this.createResult(false, 'RUT_FORMAT', '❌ RUT no proporcionado', {});
        }
        const cleanRut = rut.replace(/[.-]/g, '').toUpperCase();
        if (cleanRut.length < 2) {
            return this.createResult(false, 'RUT_FORMAT', '❌ RUT muy corto', { rut });
        }
        const body = cleanRut.slice(0, -1);
        const verifier = cleanRut.slice(-1);
        let sum = 0;
        let multiplier = 2;
        for (let i = body.length - 1; i >= 0; i--) {
            sum += parseInt(body[i]) * multiplier;
            multiplier = multiplier === 7 ? 2 : multiplier + 1;
        }
        const remainder = 11 - (sum % 11);
        let expectedVerifier = '';
        if (remainder === 11) expectedVerifier = '0';
        else if (remainder === 10) expectedVerifier = 'K';
        else expectedVerifier = remainder.toString();
        const isValid = verifier === expectedVerifier;
        return this.createResult(
            isValid,
            'RUT_FORMAT',
            isValid
                ? `✅ RUT válido: ${this.formatRUT(rut)}`
                : `❌ RUT inválido. Dígito verificador esperado: ${expectedVerifier},
recibido: ${verifier}`,
            { rut, formattedRut: this.formatRUT(rut), expectedVerifier, receivedVerifier:
verifier },
            isValid ? 'success' : 'error'
        );
    },
    validateDate(date, options = {}) {
        const {
            allowFuture = false,
            allowPast = true,
            checkPeriodOpen = true,
            periodStart = null,
            periodEnd = null
        } = options;
        if (!date) {
            return this.createResult(false, 'DATE_COHERENT', '❌ Fecha no proporcionada',
{});
        }
        const inputDate = new Date(date);
```

```
        const today = new Date();
        today.setHours(23, 59, 59, 999);
        const errors = [];
        const warnings = [];
        if (!allowFuture && inputDate > today) {
            errors.push('La fecha es futura y no está permitida');
        }
        const twoYearsAgo = new Date();
        twoYearsAgo.setFullYear(twoYearsAgo.getFullYear() - 2);
        if (inputDate < twoYearsAgo) {
            warnings.push('La fecha es de hace más de 2 años');
        }
        if (checkPeriodOpen) {
            const fiscalYear = inputDate.getFullYear();
            const currentYear = today.getFullYear();
            if (fiscalYear < currentYear - 1) {
                errors.push(`El período fiscal ${fiscalYear} podría estar cerrado`);
            }
        }
        if (periodStart && inputDate < new Date(periodStart)) {
            errors.push(`La fecha es anterior al período permitido (${periodStart})`);
        }
        if (periodEnd && inputDate > new Date(periodEnd)) {
            errors.push(`La fecha es posterior al período permitido (${periodEnd})`);
        }
        const isValid = errors.length === 0;
        const hasWarnings = warnings.length > 0;
        return this.createResult(
            isValid,
            'DATE_COHERENT',
            isValid
                ? (hasWarnings
                    ? `⚠️ Fecha aceptada con advertencias: ${warnings.join(', ')}`
                    : `✅ Fecha válida: ${this.formatDate(inputDate)}`)
                : `❌ Fecha inválida: ${errors.join(', ')}`,
            { date, inputDate: inputDate.toISOString(), errors, warnings },
            isValid ? (hasWarnings ? 'warning' : 'success') : 'error'
        );
    },
    async validateBalanceSheet() {
        const company = CompanyService.getCurrent();
        if (!company) {
            return this.createResult(false, 'BALANCE_SHEET', '❌ No hay empresa
seleccionada', {});
        }
        const accounts = await AccountingService.getChartOfAccounts();
        const assets = accounts.filter(a => a.type === 'asset' && !a.isGroup);
        const liabilities = accounts.filter(a => a.type === 'liability' && !a.isGroup);
        const equity = accounts.filter(a => a.type === 'equity' && !a.isGroup);
        const revenues = accounts.filter(a => a.type === 'revenue' && !a.isGroup);
        const expenses = accounts.filter(a => a.type === 'expense' && !a.isGroup);
        const totalAssets = assets.reduce((sum, a) => sum + (a.balance || 0), 0);
        const totalLiabilities = liabilities.reduce((sum, a) => sum + (a.balance || 0),
0);
        const totalEquity = equity.reduce((sum, a) => sum + (a.balance || 0), 0);
        const totalRevenues = revenues.reduce((sum, a) => sum + (a.balance || 0), 0);
        const totalExpenses = expenses.reduce((sum, a) => sum + (a.balance || 0), 0);
        const netIncome = totalRevenues - totalExpenses;
        const rightSide = totalLiabilities + totalEquity + netIncome;
        const difference = Math.abs(totalAssets - rightSide);
        const isBalanced = difference < 0.01;
        return this.createResult(
            isBalanced,
            'BALANCE_SHEET',
            isBalanced
                ? `✅ Balance General cuadrado:\n   Activos:
$${this.formatNumber(totalAssets)}\n   Pasivos + Patrimonio + Resultado:
$${this.formatNumber(rightSide)}`
                : `❌ Balance General DESCUADRADO:\n   Activos:
$${this.formatNumber(totalAssets)}\n   Pasivo + Patrimonio + Resultado:
$${this.formatNumber(rightSide)}\n   Diferencia: $${this.formatNumber(difference)}\n\n
💡 Esto puede deberse a datos de demostración pre-cargados con saldos iniciales que no
cuadran. En un sistema real, todos los saldos deben provenir de asientos contables
balanceados.`,
            {
                totalAssets,
                totalLiabilities,
                totalEquity,
                netIncome,
```

```
                rightSide,
                difference,
                breakdown: {
                    assets: { count: assets.length, total: totalAssets },
                    liabilities: { count: liabilities.length, total: totalLiabilities },
                    equity: { count: equity.length, total: totalEquity },
                    revenues: { count: revenues.length, total: totalRevenues },
                    expenses: { count: expenses.length, total: totalExpenses }
                }
            },
            isBalanced ? 'success' : 'error'
        );
    },
    async validateSaleGeneratesEntry(invoiceId) {
        const invoice = await DB.get('customerInvoices', invoiceId);
        if (!invoice) {
            return this.createResult(false, 'SALE_GENERATES_ENTRY', '❌ Factura no
encontrada', { invoiceId });
        }
        const company = CompanyService.getCurrent();
        const entries = await DB.getByIndex('journalEntries', 'companyId', company.id);
        const relatedEntry = entries.find(e =>
            e.sourceDocument === 'customerInvoice' && e.sourceDocumentId === invoiceId
        );
        const hasEntry = !!relatedEntry;
        return this.createResult(
            hasEntry,
            'SALE_GENERATES_ENTRY',
            hasEntry
                ? `✅ Venta ${invoice.number} generó asiento contable
${relatedEntry.number}`
                : `❌ Venta ${invoice.number} NO ha generado asiento contable`,
            { invoice, relatedEntry },
            hasEntry ? 'success' : 'warning'
        );
    },
    async validateSaleReducesInventory(invoiceId) {
        const invoice = await DB.get('customerInvoices', invoiceId);
        if (!invoice) {
            return this.createResult(false, 'SALE_REDUCES_INVENTORY', '❌ Factura no
encontrada', { invoiceId });
        }
        const company = CompanyService.getCurrent();
        const movements = await DB.getByIndex('stockMovements', 'companyId', company.id);
        const relatedMovements = movements.filter(m =>
            m.reference === invoice.number && m.type === 'sale'
        );
        const hasMovements = relatedMovements.length > 0;
        return this.createResult(
            hasMovements,
            'SALE_REDUCES_INVENTORY',
            hasMovements
                ? `✅ Venta ${invoice.number} generó ${relatedMovements.length}
movimiento(s) de inventario (salida)`
                : `❌ Venta ${invoice.number} NO ha generado movimientos de inventario`,
            { invoice, relatedMovements },
            hasMovements ? 'success' : 'warning'
        );
    },
    async validatePaymentAffectsTreasury(paymentId) {
        const payment = await DB.get('customerPayments', paymentId);
        if (!payment) {
            return this.createResult(false, 'PAYMENT_AFFECTS_TREASURY', '❌ Pago no
encontrado', { paymentId });
        }
        const bankTransactions = await DB.getAll('bankTransactions');
        const relatedTransaction = bankTransactions.find(t =>
            t.reference === payment.reference || t.sourceDocumentId === paymentId
        );
        const hasTreasuryEffect = !!relatedTransaction;
        return this.createResult(
            hasTreasuryEffect,
            'PAYMENT_AFFECTS_TREASURY',
            hasTreasuryEffect
                ? `✅ Cobro afecta tesorería: ${relatedTransaction.description ||
'Transacción bancaria registrada'}`
                : `⚠️ Cobro registrado pero sin movimiento bancario explícito`,
            { payment, relatedTransaction },
            hasTreasuryEffect ? 'success' : 'info'
```

```
        );
    },
    async validateSequenceLogic(documentType, documentId) {
        const validations = [];
        switch (documentType) {
            case 'customerPayment':
                const payment = await DB.get('customerPayments', documentId);
                if (payment && payment.invoiceId) {
                    const invoice = await DB.get('customerInvoices', payment.invoiceId);
                    if (invoice) {
                        const paymentDate = new Date(payment.date);
                        const invoiceDate = new Date(invoice.date);
                        if (paymentDate < invoiceDate) {
                            validations.push(this.createResult(
                                false,
                                'SEQUENCE_PAYMENT_AFTER_INVOICE',
                                `❌ Error de secuencia: Cobro (${payment.date}) es
anterior a la factura (${invoice.date})`,
                                { payment, invoice },
                                'error'
                            ));
                        } else {
                            validations.push(this.createResult(
                                true,
                                'SEQUENCE_PAYMENT_AFTER_INVOICE',
                                `✅ Secuencia correcta: Factura (${invoice.date}) → Cobro
(${payment.date})`,
                                { payment, invoice },
                                'success'
                            ));
                        }
                    }
                }
                break;
            case 'customerInvoice':
                const invoice = await DB.get('customerInvoices', documentId);
                if (invoice && invoice.salesOrderId) {
                    const order = await DB.get('salesOrders', invoice.salesOrderId);
                    if (order) {
                        const invoiceDate = new Date(invoice.date);
                        const orderDate = new Date(order.date);
                        if (invoiceDate < orderDate) {
                            validations.push(this.createResult(
                                false,
                                'SEQUENCE_INVOICE_AFTER_ORDER',
                                `❌ Error de secuencia: Factura (${invoice.date}) es
anterior al pedido (${order.date})`,
                                { invoice, order },
                                'error'
                            ));
                        } else {
                            validations.push(this.createResult(
                                true,
                                'SEQUENCE_INVOICE_AFTER_ORDER',
                                `✅ Secuencia correcta: Pedido (${order.date}) → Factura
(${invoice.date})`,
                                { invoice, order },
                                'success'
                            ));
                        }
                    }
                }
                break;
        }
        return validations.length > 0 ? validations : [this.createResult(true,
'SEQUENCE_OK', '✅ Secuencia lógica correcta', {}, 'success')];
    },
    async getTransactionProgress() {
        const company = CompanyService.getCurrent();
        if (!company) return { completed: 0, total: 6, transactions: [] };
        const transactions = [
            {
                id: 'create_customer',
                name: 'Crear Cliente',
                description: 'Registrar un cliente en el sistema',
                module: 'ventas',
                view: 'clientes',
                check: async () => {
                    const customers = await DB.getByIndex('customers', 'companyId',
company.id);
```

```
                    return customers.length > 0;
                }
            },
            {
                id: 'create_sales_order',
                name: 'Crear Pedido de Venta',
                description: 'Registrar un pedido de venta (SD)',
                module: 'ventas',
                view: 'pedidos-venta',
                check: async () => {
                    const orders = await DB.getByIndex('salesOrders', 'companyId',
company.id);
                    return orders.length > 0;
                }
            },
            {
                id: 'create_invoice',
                name: 'Emitir Factura',
                description: 'Generar factura de venta y asiento automático',
                module: 'ventas',
                view: 'facturas-cliente',
                check: async () => {
                    const invoices = await DB.getByIndex('customerInvoices', 'companyId',
company.id);
                    return invoices.length > 0;
                }
            },
            {
                id: 'create_journal_entry',
                name: 'Crear Asiento Contable',
                description: 'Registrar transacción en el Libro Mayor (GL)',
                module: 'contabilidad',
                view: 'asientos',
                check: async () => {
                    const entries = await DB.getByIndex('journalEntries', 'companyId',
company.id);
                    return entries.length > 0;
                }
            },
            {
                id: 'register_payment',
                name: 'Registrar Cobro',
                description: 'Registrar pago recibido del cliente (FI)',
                module: 'tesoreria',
                view: 'cuentas-cobrar',
                check: async () => {
                    const payments = await DB.getByIndex('customerPayments', 'companyId',
company.id);
                    return payments && payments.length > 0;
                }
            },
            {
                id: 'view_balance',
                name: 'Consultar Balance',
                description: 'Visualizar el Balance General',
                module: 'reportes',
                view: 'balance-general',
                check: async () => {
                    const entries = await DB.getByIndex('journalEntries', 'companyId',
company.id);
                    return entries.some(e => e.status === 'posted');
                }
            }
        ];
        const results = [];
        for (const tx of transactions) {
            const completed = await tx.check();
            results.push({
                ...tx,
                completed,
                check: undefined
            });
        }
        return {
            completed: results.filter(r => r.completed).length,
            total: results.length,
            percentage: Math.round((results.filter(r => r.completed).length /
results.length) * 100),
            transactions: results
        };
    },
```

```
    async validateCompleteTransaction(startDocType, startDocId) {
        const tracechain = [];
        const company = CompanyService.getCurrent();
        let currentDoc = { type: startDocType, id: startDocId };
        const addToChain = (step, status, details) => {
            tracechain.push({ step, status, details, timestamp: new Date().toISOString()
});
        };
        try {
            switch (startDocType) {
                case 'salesOrder':
                    const order = await DB.get('salesOrders', startDocId);
                    if (!order) {
                        addToChain('Pedido de Venta', 'not_found', { id: startDocId });
                        break;
                    }
                    addToChain('Pedido de Venta', 'found', { number: order.number, status:
order.status, total: order.total });
                    const deliveries = await DB.getByIndex('deliveries', 'companyId',
company.id);
                    const relatedDelivery = deliveries.find(d => d.salesOrderId ===
startDocId);
                    if (relatedDelivery) {
                        addToChain('Entrega', 'found', { number: relatedDelivery.number,
status: relatedDelivery.status });
                    } else {
                        addToChain('Entrega', 'pending', { message: 'Aún no se ha
procesado la entrega' });
                    }
                    const invoices = await DB.getByIndex('customerInvoices', 'companyId',
company.id);
                    const relatedInvoice = invoices.find(i => i.salesOrderId ===
startDocId);
                    if (relatedInvoice) {
                        addToChain('Factura de Venta', 'found', {
                            number: relatedInvoice.number,
                            status: relatedInvoice.status,
                            total: relatedInvoice.total
                        });
                        const entries = await DB.getByIndex('journalEntries', 'companyId',
company.id);
                        const relatedEntry = entries.find(e =>
                            e.sourceDocument === 'customerInvoice' && e.sourceDocumentId
=== relatedInvoice.id
                        );
                        if (relatedEntry) {
                            addToChain('Asiento Contable', 'found', {
                                number: relatedEntry.number,
                                status: relatedEntry.status,
                                totalDebit: relatedEntry.totalDebit
                            });
                        } else {
                            addToChain('Asiento Contable', 'pending', { message: 'Factura
no contabilizada' });
                        }
                        const payments = await DB.getByIndex('customerPayments',
'companyId', company.id);
                        const relatedPayments = payments ? payments.filter(p =>
p.invoiceId === relatedInvoice.id) : [];
                        if (relatedPayments.length > 0) {
                            const totalPaid = relatedPayments.reduce((sum, p) => sum +
p.amount, 0);
                            addToChain('Cobros', 'found', {
                                count: relatedPayments.length,
                                totalPaid,
                                balance: relatedInvoice.total - totalPaid
                            });
                        } else {
                            addToChain('Cobros', 'pending', { message: 'No hay cobros
registrados' });
                        }
                    } else {
                        addToChain('Factura de Venta', 'pending', { message: 'Pedido aún
no facturado' });
                    }
                    break;
                case 'customerInvoice':
                    const inv = await DB.get('customerInvoices', startDocId);
                    if (!inv) {
                        addToChain('Factura de Venta', 'not_found', { id: startDocId });
                        break;
```

```
                    }
                    addToChain('Factura de Venta', 'found', { number: inv.number, status:
inv.status, total: inv.total });
                    break;
                }
            } catch (error) {
                addToChain('Error', 'error', { message: error.message });
            }
            return {
                startDocument: { type: startDocType, id: startDocId },
                tracechain,
                isComplete: tracechain.every(t => t.status === 'found'),
                summary: this.generateTraceSummary(tracechain)
            };
        },
        generateTraceSummary(tracechain) {
            const found = tracechain.filter(t => t.status === 'found').length;
            const pending = tracechain.filter(t => t.status === 'pending').length;
            const errors = tracechain.filter(t => t.status === 'error' || t.status ===
'not_found').length;
            return {
                found,
                pending,
                errors,
                total: tracechain.length,
                completeness: Math.round((found / tracechain.length) * 100)
            };
        },
        formatNumber(num) {
            return new Intl.NumberFormat('es-CL').format(num);
        },
        formatRUT(rut) {
            const clean = rut.replace(/[.-]/g, '');
            const body = clean.slice(0, -1);
            const dv = clean.slice(-1).toUpperCase();
            return body.replace(/\B(?=(\d{3})+(?!\d))/g, '.') + '-' + dv;
        },
        formatDate(date) {
            return new Intl.DateTimeFormat('es-CL', {
                day: '2-digit',
                month: '2-digit',
                year: 'numeric'
            }).format(date);
        },
        async validateJournalEntry(entryData) {
            const results = [];
            results.push(this.validateDebitCreditBalance(entryData.lines));
            results.push(this.validateDate(entryData.date));
            for (const line of entryData.lines) {
                results.push(await this.validateAccountExists(line.accountId));
                results.push(await this.validateAccountNature(line.accountId, line.debit,
line.credit));
            }
            return {
                isValid: results.every(r => r.isValid || r.severity !== 'error'),
                results,
                errors: results.filter(r => r.severity === 'error'),
                warnings: results.filter(r => r.severity === 'warning'),
                info: results.filter(r => r.severity === 'info' || r.severity === 'success')
            };
        },
        async validateBusinessFlow(documentType, documentId) {
            const results = [];
            switch (documentType) {
                case 'customerInvoice':
                    results.push(await this.validateSaleGeneratesEntry(documentId));
                    results.push(await this.validateSaleReducesInventory(documentId));
                    results.push(...await this.validateSequenceLogic('customerInvoice',
documentId));
                    break;
                case 'customerPayment':
                    results.push(await this.validatePaymentAffectsTreasury(documentId));
                    results.push(...await this.validateSequenceLogic('customerPayment',
documentId));
                    break;
            }
            return {
                documentType,
                documentId,
                results,
                allValid: results.every(r => r.isValid || r.severity !== 'error')
```

```
        };
    }
};
window.ValidationService = ValidationService;
```

Archivo: js\utils\formatters.js

```
const Formatters = {
    currency(value, options = {}) {
        const { currency = 'CLP', decimals = 0, showSymbol = true } = options;
        const num = parseFloat(value) || 0;
        if (currency === 'CLP') {
            const formatted = num.toLocaleString('es-CL', {
                minimumFractionDigits: decimals,
                maximumFractionDigits: decimals
            });
            return showSymbol ? `$ ${formatted}` : formatted;
        }
        return num.toLocaleString('es-CL', {
            style: 'currency',
            currency: currency,
            minimumFractionDigits: decimals,
            maximumFractionDigits: decimals
        });
    },
    number(value, decimals = 0) {
        const num = parseFloat(value) || 0;
        return num.toLocaleString('es-CL', {
            minimumFractionDigits: decimals,
            maximumFractionDigits: decimals
        });
    },
    percentage(value, decimals = 1) {
        const num = parseFloat(value) || 0;
        return `${num.toFixed(decimals)}%`;
    },
    date(dateString, format = 'short') {
        if (!dateString) return '-';
        const date = new Date(dateString);
        if (isNaN(date.getTime())) return '-';
        const options = {
            short: { day: '2-digit', month: '2-digit', year: 'numeric' },
            medium: { day: '2-digit', month: 'short', year: 'numeric' },
            long: { day: '2-digit', month: 'long', year: 'numeric' },
            full: { weekday: 'long', day: '2-digit', month: 'long', year: 'numeric' }
        };
        return date.toLocaleDateString('es-CL', options[format] || options.short);
    },
    datetime(dateString, showSeconds = false) {
        if (!dateString) return '-';
        const date = new Date(dateString);
        if (isNaN(date.getTime())) return '-';
        const dateFormat = { day: '2-digit', month: '2-digit', year: 'numeric' };
        const timeFormat = { hour: '2-digit', minute: '2-digit' };
        if (showSeconds) timeFormat.second = '2-digit';
        return `${date.toLocaleDateString('es-CL', dateFormat)}
${date.toLocaleTimeString('es-CL', timeFormat)}`;
    },
    relativeTime(dateString) {
        if (!dateString) return '-';
        const date = new Date(dateString);
        if (isNaN(date.getTime())) return '-';
        const now = new Date();
        const diffMs = now - date;
        const diffSecs = Math.floor(diffMs / 1000);
        const diffMins = Math.floor(diffSecs / 60);
        const diffHours = Math.floor(diffMins / 60);
        const diffDays = Math.floor(diffHours / 24);
        if (diffSecs < 60) return 'Hace un momento';
        if (diffMins < 60) return `Hace ${diffMins} min`;
        if (diffHours < 24) return `Hace ${diffHours}h`;
        if (diffDays < 7) return `Hace ${diffDays}d`;
        return this.date(dateString);
    },
    accountCode(code) {
```

```
        if (!code) return '';
        const str = String(code).replace(/\D/g, '');
        if (str.length <= 1) return str;
        let result = str[0];
        if (str.length > 1) result += '.' + str[1];
        if (str.length > 2) result += '.' + str.substring(2, 4);
        if (str.length > 4) result += '.' + str.substring(4);
        return result;
    },
    truncate(text, maxLength = 50) {
        if (!text) return '';
        if (text.length <= maxLength) return text;
        return text.substring(0, maxLength - 3) + '...';
    },
    fileSize(bytes) {
        if (bytes === 0) return '0 Bytes';
        const k = 1024;
        const sizes = ['Bytes', 'KB', 'MB', 'GB'];
        const i = Math.floor(Math.log(bytes) / Math.log(k));
        return parseFloat((bytes / Math.pow(k, i)).toFixed(2)) + ' ' + sizes[i];
    },
    documentNumber(type, number) {
        const prefixes = {
            'invoice': 'FAC',
            'order': 'OC',
            'receipt': 'REC',
            'entry': 'AST',
            'payment': 'PAG'
        };
        const prefix = prefixes[type] || 'DOC';
        return `${prefix}-${String(number).padStart(6, '0')}`;
    },
    numberToWords(num) {
        const units = ['', 'uno', 'dos', 'tres', 'cuatro', 'cinco', 'seis', 'siete',
'ocho', 'nueve'];
        const teens = ['diez', 'once', 'doce', 'trece', 'catorce', 'quince', 'dieciséis',
'diecisiete', 'dieciocho', 'diecinueve'];
        const tens = ['', '', 'veinte', 'treinta', 'cuarenta', 'cincuenta', 'sesenta',
'setenta', 'ochenta', 'noventa'];
        const hundreds = ['', 'ciento', 'doscientos', 'trescientos', 'cuatrocientos',
'quinientos', 'seiscientos', 'setecientos', 'ochocientos', 'novecientos'];
        if (num === 0) return 'cero';
        if (num === 100) return 'cien';
        let result = '';
        if (num >= 1000000) {
            const millions = Math.floor(num / 1000000);
            result += (millions === 1 ? 'un millón' : this.numberToWords(millions) + '
millones') + ' ';
            num %= 1000000;
        }
        if (num >= 1000) {
            const thousands = Math.floor(num / 1000);
            result += (thousands === 1 ? 'mil' : this.numberToWords(thousands) + ' mil') +
' ';
            num %= 1000;
        }
        if (num >= 100) {
            result += hundreds[Math.floor(num / 100)] + ' ';
            num %= 100;
        }
        if (num >= 20) {
            result += tens[Math.floor(num / 10)];
            if (num % 10 !== 0) result += ' y ' + units[num % 10];
        } else if (num >= 10) {
            result += teens[num - 10];
        } else if (num > 0) {
            result += units[num];
        }
        return result.trim();
    },
    status(status) {
        const statusMap = {
            'draft': { label: 'Borrador', class: 'badge-neutral' },
            'pending': { label: 'Pendiente', class: 'badge-warning' },
            'approved': { label: 'Aprobado', class: 'badge-info' },
            'confirmed': { label: 'Confirmado', class: 'badge-info' },
            'delivered': { label: 'Entregado', class: 'badge-success' },
            'received': { label: 'Recibido', class: 'badge-success' },
            'posted': { label: 'Contabilizado', class: 'badge-success' },
            'paid': { label: 'Pagado', class: 'badge-success' },
            'partial': { label: 'Parcial', class: 'badge-warning' },
```

```
            'cancelled': { label: 'Anulado', class: 'badge-error' },
            'overdue': { label: 'Vencido', class: 'badge-error' }
        };
        return statusMap[status] || { label: status, class: 'badge-neutral' };
    }
};
window.Formatters = Formatters;
```

Archivo: js\utils\helpers.js

```
const Helpers = {
    generateId() {
        return Date.now().toString(36) + Math.random().toString(36).substr(2);
    },
    generateDocNumber(prefix, sequence) {
        const year = new Date().getFullYear();
        const seq = String(sequence).padStart(6, '0');
        return `${prefix}-${year}-${seq}`;
    },
    debounce(func, wait) {
        let timeout;
        return function executedFunction(...args) {
            const later = () => {
                clearTimeout(timeout);
                func(...args);
            };
            clearTimeout(timeout);
            timeout = setTimeout(later, wait);
        };
    },
    deepClone(obj) {
        return JSON.parse(JSON.stringify(obj));
    },
    isEmpty(obj) {
        if (obj === null || obj === undefined) return true;
        if (Array.isArray(obj)) return obj.length === 0;
        if (typeof obj === 'object') return Object.keys(obj).length === 0;
        if (typeof obj === 'string') return obj.trim() === '';
        return false;
    },
    capitalize(str) {
        if (!str) return '';
        return str.charAt(0).toUpperCase() + str.slice(1).toLowerCase();
    },
    slugify(str) {
        return str
            .toLowerCase()
            .trim()
            .replace(/[^\w\s-]/g, '')
            .replace(/[\s_-]+/g, '-')
            .replace(/^-+|-+$/g, '');
    },
    getNestedValue(obj, path) {
        return path.split('.').reduce((acc, part) => acc && acc[part], obj);
    },
    sortBy(array, key, order = 'asc') {
        return [...array].sort((a, b) => {
            const valA = this.getNestedValue(a, key);
            const valB = this.getNestedValue(b, key);
            if (valA < valB) return order === 'asc' ? -1 : 1;
            if (valA > valB) return order === 'asc' ? 1 : -1;
            return 0;
        });
    },
    filterBySearch(array, searchTerm, keys) {
        if (!searchTerm) return array;
        const term = searchTerm.toLowerCase();
        return array.filter(item =>
            keys.some(key => {
                const value = this.getNestedValue(item, key);
                return value && String(value).toLowerCase().includes(term);
            })
        );
    },
    groupBy(array, key) {
```

```
        return array.reduce((acc, item) => {
            const groupKey = this.getNestedValue(item, key);
            if (!acc[groupKey]) acc[groupKey] = [];
            acc[groupKey].push(item);
            return acc;
        }, {});
    },
    sumBy(array, key) {
        return array.reduce((sum, item) => {
            const value = this.getNestedValue(item, key);
            return sum + (parseFloat(value) || 0);
        }, 0);
    },
    percentage(value, total) {
        if (total === 0) return 0;
        return (value / total) * 100;
    },
    getCurrentDate() {
        return new Date().toISOString().split('T')[0];
    },
    getDatePlusDays(days) {
        const date = new Date();
        date.setDate(date.getDate() + days);
        return date.toISOString().split('T')[0];
    },
    getMonthRange(date = new Date()) {
        const firstDay = new Date(date.getFullYear(), date.getMonth(), 1);
        const lastDay = new Date(date.getFullYear(), date.getMonth() + 1, 0);
        return {
            start: firstDay.toISOString().split('T')[0],
            end: lastDay.toISOString().split('T')[0]
        };
    },
    getYearRange(year = new Date().getFullYear()) {
        return {
            start: `${year}-01-01`,
            end: `${year}-12-31`
        };
    },
    getFirstDayOfYear() {
        const year = new Date().getFullYear();
        return `${year}-01-01`;
    },
    validateRUT(rut) {
        if (!rut || typeof rut !== 'string') return false;
        const cleanRut = rut.replace(/[\.\-\s]/g, '').toUpperCase().trim();
        if (cleanRut.length < 2) return false;
        const body = cleanRut.slice(0, -1);
        const dv = cleanRut.slice(-1);
        if (!/^\d+$/.test(body)) return false;
        if (!/^[\dK]$/.test(dv)) return false;
        let sum = 0;
        let multiplier = 2;
        for (let i = body.length - 1; i >= 0; i--) {
            sum += parseInt(body[i], 10) * multiplier;
            multiplier = multiplier === 7 ? 2 : multiplier + 1;
        }
        const remainder = sum % 11;
        const expectedDV = 11 - remainder;
        let calculatedDV;
        if (expectedDV === 11) {
            calculatedDV = '0';
        } else if (expectedDV === 10) {
            calculatedDV = 'K';
        } else {
            calculatedDV = String(expectedDV);
        }
        return dv === calculatedDV;
    },
    formatRUT(rut) {
        if (!rut) return '';
        const cleanRut = rut.replace(/[.-]/g, '');
        const body = cleanRut.slice(0, -1);
        const dv = cleanRut.slice(-1);
        let formatted = '';
        for (let i = body.length - 1, j = 0; i >= 0; i--, j++) {
            if (j > 0 && j % 3 === 0) formatted = '.' + formatted;
            formatted = body[i] + formatted;
        }
        return `${formatted}-${dv}`;
    },
```

```
    validateEmail(email) {
        const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        return re.test(email);
    },
    sleep(ms) {
        return new Promise(resolve => setTimeout(resolve, ms));
    },
    async copyToClipboard(text) {
        try {
            await navigator.clipboard.writeText(text);
            return true;
        } catch (err) {
            console.error('Error copying to clipboard:', err);
            return false;
        }
    },
    downloadFile(content, filename, mimeType = 'application/octet-stream') {
        const blob = new Blob([content], { type: mimeType });
        if (typeof saveAs !== 'undefined') {
            saveAs(blob, filename);
        } else {
            const url = URL.createObjectURL(blob);
            const link = document.createElement('a');
            link.href = url;
            link.setAttribute('download', filename);
            link.style.display = 'none';
            document.body.appendChild(link);
            link.click();
            setTimeout(() => {
                document.body.removeChild(link);
                URL.revokeObjectURL(url);
            }, 100);
        }
    }
};
window.Helpers = Helpers;
```